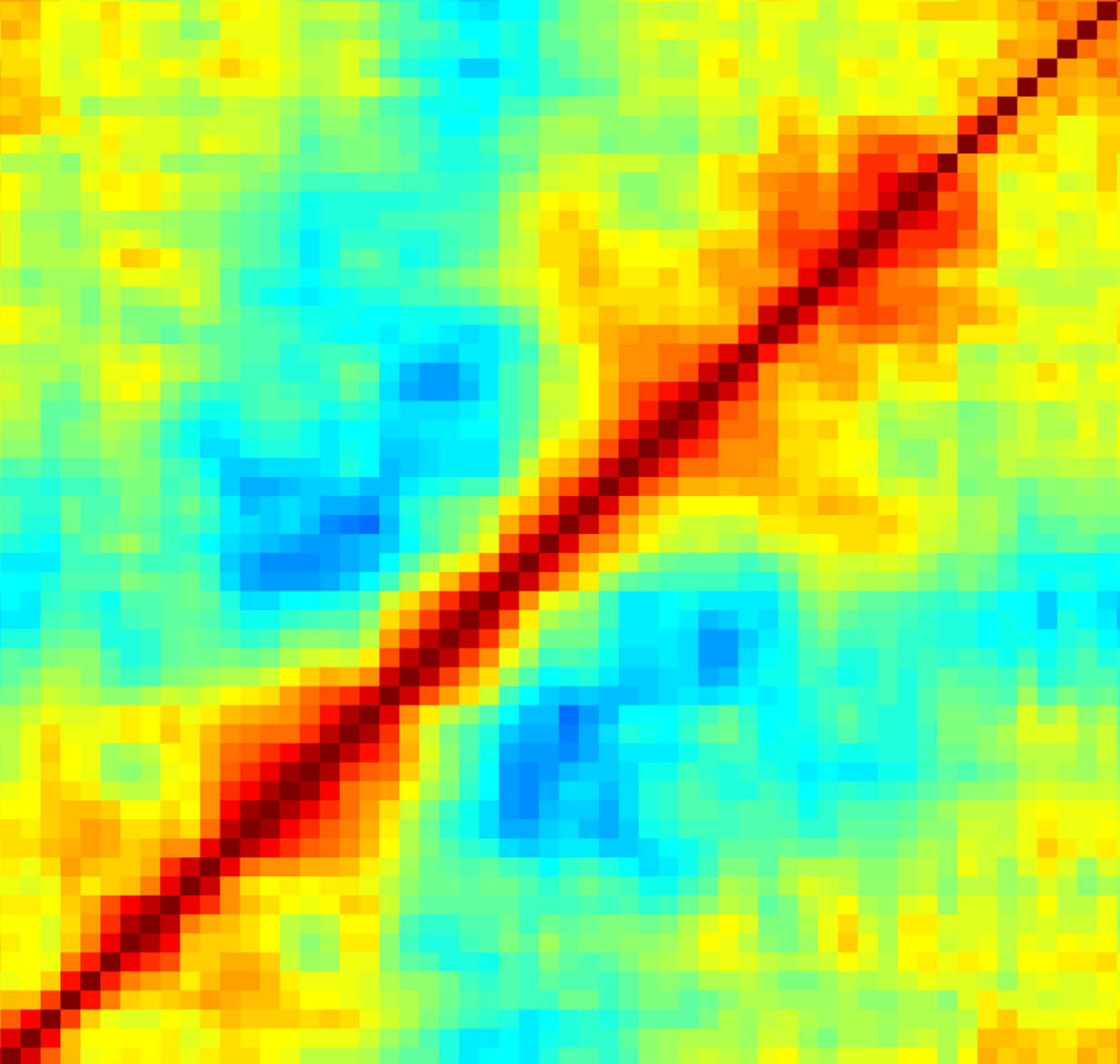




INTELIGÊNCIA COMPUTACIONAL

Fundamentos e aplicações

Alexandre G. Evsukoff

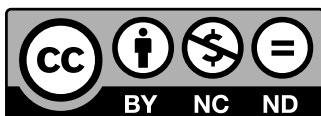
INTELIGÊNCIA COMPUTACIONAL

Fundamentos e aplicações

Alexandre G. Evsukoff

Rio de Janeiro, 2020

 **e-papers**



Este trabalho está licenciado sob a Licença Atribuição-NãoComercial-SemDerivações 4.0 Internacional Creative Commons. Para visualizar uma cópia desta licença, visite <http://creativecommons.org/licenses/by-nc-nd/4.0/> ou mande uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

ISBN 978-65-87065-02-1

Ilustração de capa

Matriz de correlação do conjunto de dados Sonar, elaborada pelo autor

Diagramação

Ana Claudia Ribeiro

Revisão

Rodrigo Reis

Disponível no site da Editora E-papers

<http://www.e-papers.com.br>

Avenida das Américas, 3.200 bloco 1 sala 138

Barra da Tijuca – Rio de Janeiro – Brasil

CEP 22.640-102

CIP-Brasil. Catalogação na fonte
Sindicato Nacional dos Editores de Livros, RJ

E94i

Evsukoff, Alexandre

Inteligência computacional : Fundamentos e aplicações [recurso eletrônico] / Alexandre Evsukoff. - 1. ed. - Rio de Janeiro : E-papers, 2020.
recurso digital : il. ; 20 MB

Formato: eletrônico

Requisitos do sistema:

Modo de acesso: world wide web

Inclui bibliografia

ISBN 978-65-87065-02-1 (recurso eletrônico)

1. Inteligência computacional. 2. Redes neurais (Computação). 3. Livros eletrônicos. I. Título.

20-64566

CDD: 006.3

CDU: 004.8

Sumário

APRESENTAÇÃO	8
1 INTRODUÇÃO	9
1.1 CIÊNCIA DE DADOS	12
1.2 TIPOS DE MODELOS	15
1.2.1 Regressão.....	16
1.2.2 Classificação.....	18
1.2.3 Análise de agrupamentos	20
1.3 RECURSOS COMPLEMENTARES	21
1.3.1 Bibliografia de referência	21
1.3.2 Recursos na internet.....	23
1.3.3 Softwares	26
1.4 COMENTÁRIOS FINAIS	29
2 PRÉ-PROCESSAMENTO	30
2.1 CONJUNTOS DE DADOS	30
2.2 CARACTERIZAÇÃO ESTATÍSTICA	33
2.2.1 Análise monovariável	34
2.2.2 Análise multivariada	40
2.2.3 Matriz de distâncias e similaridades.....	44
2.3 LIMPEZA DE DADOS	47
2.3.1 Padronização de variáveis.....	48
2.3.2 Detecção de outliers.....	49
2.3.3 Valores ausentes.....	53
2.4 TRANSFORMAÇÕES LINEARES.....	56
2.4.1 Análise de componentes principais	57
2.4.2 Decomposição em valores singulares	62
2.4.3 Análise discriminante linear	63
2.4.4 Análise de correlação canônica	66
2.5 APLICAÇÕES	69
2.5.1 Padrões de mobilidade	69
2.5.2 Equações de paridade	72
2.6 COMENTÁRIOS FINAIS	76

3	REGRESSÃO LINEAR	78
3.1	O PROBLEMA DE REGRESSÃO.....	79
3.1.1	O Modelo linear	81
3.1.2	Método dos mínimos quadrados	82
3.1.3	Interpretação geométrica	84
3.1.4	Exemplo sintético	86
3.2	REGULARIZAÇÃO	89
3.2.1	Bias e variância.....	90
3.2.2	Regularização de Tikhonov.....	93
3.2.3	Regularização por componentes principais	97
3.3	VALIDAÇÃO DE MODELOS	98
3.3.1	Estatísticas de validação	99
3.3.2	Validação cruzada	104
3.4	APLICAÇÕES	106
3.4.1	Conjunto de dados Pollution	106
3.4.2	Previsão da elevação adiabática de temperatura do concreto.....	110
3.5	COMENTÁRIOS FINAIS	114
4	MODELOS DINÂMICOS	116
4.1	AUTOVARIÂNCIA E AUTOCORRELAÇÃO.....	117
4.2	MODELOS DE SÉRIES TEMPORAIS	121
4.2.1	Modelo AR.....	122
4.2.2	Modelo MA.....	126
4.2.3	Estruturas mistas	129
4.3	SISTEMAS DINÂMICOS LINEARES	131
4.3.1	Modelo ARX.....	133
4.3.2	Modelo ARMAX.....	135
4.3.3	Modelo OE.....	136
4.3.4	Modelo Box & Jenkins.....	138
4.3.5	Equações de estado	139
4.4	APLICAÇÕES	140
4.4.1	Wolf's sunspot numbers.....	140
4.4.2	Modelo chuva-vazão	142
4.5	COMENTÁRIOS FINAIS	146
5	CLASSIFICAÇÃO	148
5.1	CLASSIFICAÇÃO BAYESIANA	149
5.1.1	Decisão bayesiana	149
5.1.2	Classificador bayesiano simples	155
5.1.3	Classificador bayesiano quadrático.....	160

5.2	MODELOS LINEARES DE CLASSIFICAÇÃO.....	164
5.2.1	Mínimos quadrados.....	164
5.2.2	Mínimos quadrados ponderados.....	166
5.2.3	Regressão logística	168
5.3	AVALIAÇÃO DE CLASSIFICADORES.....	171
5.3.1	Estatísticas de validação	172
5.3.2	Espaço ROC e AUC.....	175
5.4	APLICAÇÕES	179
5.4.1	Testes benchmark.....	179
5.4.2	Análise de cancelamento em telefonia.....	183
5.5	COMENTÁRIOS FINAIS	187
6	ANÁLISE DE AGRUPAMENTOS	189
6.1	O PROBLEMA DE ANÁLISE DE AGRUPAMENTOS.....	190
6.2	MÉTODOS DE PARTIÇÃO	194
6.2.1	Algoritmo k-médias.....	194
6.2.2	Exemplos.....	198
6.3	MÉTODOS HIERÁRQUICOS	201
6.3.1	Algoritmos aglomerativos	202
6.3.2	Exemplos.....	203
6.4	PARTICIONAMENTO DE GRAFOS.....	204
6.4.1	Algoritmos de agrupamento espectral	210
6.4.2	Exemplos.....	216
6.5	VALIDAÇÃO DE GRUPOS	218
6.5.1	Medidas externas.....	218
6.5.2	Medidas internas	220
6.5.3	Medidas relativas	222
6.6	APLICAÇÕES	227
6.6.1	Segmentação de imagens de satélite	227
6.6.2	Agrupamento de documentos.....	230
6.7	COMENTÁRIOS FINAIS	234
7	MODELOS LINEARES LOCAIS.....	236
7.1	APRENDIZADO DE MÁQUINAS.....	237
7.2	MODELOS DE BASE RADIAL (RBF)	241
7.2.1	Aproximação de funções	242
7.2.2	Modelos RBF.....	246
7.2.3	Ajuste de parâmetros	250
7.3	MÁQUINAS DE VETOR DE SUPORTE	253
7.3.1	Funções de núcleo.....	253
7.3.2	Minimização do risco estrutural.....	258

7.3.3	Classificadores de máxima margem.....	262
7.4	APLICAÇÕES	273
7.4.1	Testes benchmark.....	273
7.4.2	Algoritmo da decomposição espectral.....	274
7.5	COMENTÁRIOS FINAIS	278
8	SISTEMAS FUZZY	280
8.1	CONJUNTOS FUZZY	282
8.1.1	Funções de pertinência.....	285
8.1.2	Operadores fuzzy.....	289
8.1.3	Relações fuzzy.....	293
8.2	MODELOS DE REGRAS FUZZY	294
8.2.1	Variáveis linguísticas.....	295
8.2.2	Cálculo de regras fuzzy	298
8.2.3	Sistemas fuzzy simbólicos	304
8.3	ALGORITMOS PARA CIÊNCIA DE DADOS	313
8.3.1	Fuzzy c-médias	313
8.3.2	Fuzzy pattern matching.....	317
8.3.3	Algoritmo de Wang & Mendel.....	322
8.3.4	Modelo fuzzy-RBF	326
8.4	APLICAÇÕES	329
8.4.1	Testes benchmark.....	329
8.4.2	Diagnóstico de falhas.....	331
8.4.3	Classificação espacial.....	336
8.4.4	Modelo chuva-vazão	339
8.5	COMENTÁRIOS FINAIS	344
9	REDES NEURAIS	346
9.1	REDES NEURAIS LINEARES.....	350
9.1.1	Perceptron e ADALINE.....	352
9.1.2	Método do gradiente.....	356
9.1.3	Gradiente estocástico	360
9.2	REDES NEURAIS NÃO LINEARES	362
9.2.1	Perceptron de Múltiplas Camadas (MLP).....	362
9.2.2	Avaliação pelo máximo da verossimilhança.....	368
9.2.3	Algoritmo da retropropagação.....	374
9.3	REDES NEURAIS PROFUNDAS.....	388
9.3.1	Redes neurais convolucionais	391
9.3.2	Redes neurais recorrentes	411
9.3.3	Aprendizado da representação	439
9.4	APLICAÇÕES	452

9.4.1	Previsão de inadimplência	452
9.4.2	Previsão de emissões de combustível	455
9.4.3	Previsão de demanda de energia elétrica.....	458
9.4.4	Previsão de mercado financeiro	465
9.5	COMENTÁRIOS FINAIS	470
REFERÊNCIAS BIBLIOGRÁFICAS.....		471
REFERÊNCIAS NA INTERNET.....		521

Apresentação

A **inteligência computacional** é um conjunto de paradigmas computacionais para o desenvolvimento e aplicação de sistemas inteligentes em problemas complexos do mundo real. A **ciência de dados** reúne o conhecimento de matemática, estatística, computação e engenharia com o conhecimento do domínio de cada aplicação.

A ubiquidade dos dados na era digital faz a ciência de dados ter aplicações em todas as áreas de engenharia e, cada vez mais, em ciências naturais, biológicas e humanas. Este livro apresenta uma introdução às principais técnicas e modelos de inteligência computacional em aplicações de ciência de dados. O livro aborda os fundamentos para a preparação de dados, ajustes de modelos preditivos e descritivos e apresenta aplicações em engenharia, resultados dos projetos realizados na Coppe/UFRJ. O material pode ser utilizado como livro-texto em disciplinas de graduação e pós-graduação.

O livro é dividido em nove capítulos. Os cinco primeiros contêm uma introdução às técnicas de estatística através da apresentação dos principais métodos de regressão linear, sistemas dinâmicos, classificação e análise de agrupamentos. O Capítulo 7 discute teoricamente o problema de aproximação de funções e apresenta os modelos de função de base radial (RBF) e máquinas de vetor de suporte (SVM). O Capítulo 8 apresenta os sistemas *fuzzy* e o Capítulo 9, os modelos de redes neurais com uma introdução aos modelos recentes de redes neurais profundas.

1 INTRODUÇÃO

A capacidade de uma máquina exibir um comportamento inteligente foi objeto de reflexão filosófica durante muito tempo. Na década de 1920, Wittgenstein abordou o tema a partir da questão “pode uma máquina pensar?” A sua conclusão foi que, assim como a consciência, pensar é uma capacidade inerentemente humana [483]. Em 1950, tendo em vista a dificuldade de definir “pensar” e “inteligência”, Alan Turing propôs um teste para verificar se um computador pode exibir um comportamento inteligente [529]. O teste de Turing¹ é proposto como um jogo em que um avaliador isolado numa sala é colocado em contato com um interlocutor em outra sala, que pode ser humano ou computador. Se o avaliador não puder identificar se o interlocutor é um computador, então pode-se concluir que o computador tem um comportamento inteligente.

A inteligência artificial como área de pesquisa surgiu alguns anos mais tarde, como uma proposta de integração de lógica, ciência da computação e neurofisiologia para o desenvolvimento de máquinas inteligentes. No início da década de 1960, as duas principais correntes para o desenvolvimento de inteligência artificial eram chamadas de simbolistas e conexionistas. A corrente simbolista era formada por lógicos matemáticos que desenvolveram os primeiros provadores de teoremas. Na corrente conexionista, o objetivo era simular o processamento cerebral com neurônios artificiais. Os resultados iniciais do algoritmo de aprendizado do Perceptron [457] geraram grande expectativa com a possibilidade de construção de cérebros artificiais, mas sua limitação como classificador linear não permitia o ajuste de modelos mais complexos.

Ao longo da década de 1970, os computadores *mainframe* se difundiram como produtos para a automação do processamento de dados em organizações. Os provadores de teorema evoluíram para os sistemas especialistas, que continham um motor de raciocínio que processava o conhecimento representado por um conjunto de regras e fatos. Logo ficou claro que a aquisição do conhecimento dos especialistas não era uma tarefa trivial, o que fomentou o interesse por técnicas de **aprendizado de máquina** (*machine learning*) para sínteses de modelos diretamente a partir de dados. O algoritmo ID3 de indução de árvores de decisão foi apresentado em 1986 por Quinlan. O ID3 tinha a capacidade de abstrair um conjunto de regras lógicas a partir de dados de variáveis nominais. O artigo inaugurou a revista *Machine Learning* [445].

Os modelos conexionistas retornariam com grande impacto em 1986 a partir da publicação de artigo na revista *Nature* [464] que apresentava um algoritmo de treinamento de redes neurais de múltiplas camadas baseado na retropropagação do erro através das camadas. Os resultados mostravam a capacidade do método na solução de problemas complexos de reconhecimento de padrões, como a identificação de dígitos manuscritos. As redes neurais se desenvolveram rapidamente devido a sua capacidade de ajuste de modelos não lineares computacionalmente robustos, com bons resultados em diversas aplicações. Na década de 1980, a introdução do IBM PC marca o início da difusão de computadores pessoais para o grande público e empresas de pequeno porte.

Nas décadas de 1980 e 1990, houve grande avanço na área de inteligência artificial, com o desenvolvimento de novos paradigmas computacionais como os sistemas *fuzzy*, redes neurais e máquinas de vetor de suporte. A integração destes modelos em abordagens híbridas, incluindo a utilização de algoritmos evolucionários para otimização, foi chamada de computação macia (*soft computing*) ou **inteligência computacional** (*computational intelligence*). Atualmente, a inteligência computacional inclui também métodos estatísticos e de aprendizado de máquina.²

Na década de 1990, os computadores pessoais já haviam se tornado um produto de uso comum e, na virada para o ano 2000, aproximadamente 30% da população dos EUA e da Europa estava conectada à internet.³ A expectativa com o desenvolvimento da internet foi enorme no final da década de 1990, o que resultou na chamada bolha da internet,⁴ que estourou em 2000 e provocou a queda das cotações das ações negociadas na bolsa de tecnologia NASDAQ. A partir da bolha “.com”, a competição ficou mais intensa e a internet se tornou um mercado mais maduro, dominado por grandes *players*.

Ao longo da década de 1990, o termo **mineração de dados** (*data mining*) começou a se difundir como um conjunto de técnicas que incluem estatística, aprendizado de máquina, inteligência computacional, bancos de dados, entre outras, para o desenvolvimento de modelos a partir de dados. Em 1997, surgiu a primeira plataforma de código livre de mineração de dados, o Weka,⁵ desenvolvido na Universidade de Waikato, Nova Zelândia, tornando-se uma referência no mundo acadêmico. A partir de 2000, cada vez mais *softwares* comerciais e bibliotecas oferecem novos algoritmos.⁶ Ao longo da década de 2000, as plataformas começaram a se apresentar no mercado como soluções de inteligência analítica [329], com foco em análise de dados e alinhamento dos resultados com os objetivos do negócio.

A redução de custos de armazenamento, o desenvolvimento dos processadores e a internet alavancaram o processo de coleta e armazenamento de bases de dados. O começo do século XXI marca o início da predominância do armazenamento de dados em formato digital sobre as tecnologias analógicas. O tráfego de dados na internet aumentou exponencialmente com a disseminação de dispositivos móveis a partir de 2007. Em 2010, o termo *big data* aparece como uma *buzzword* para definir o conjunto de tecnologias para coletar, armazenar, processar, analisar e visualizar grandes bases de dados.

A internet permite a oferta de produtos e serviços a um número virtualmente ilimitado de usuários numa rede livre de escala [29]. Nos últimos anos, novos modelos de negócios ou novas versões de negócios tradicionais escaláveis globalmente mudaram a vida de milhares de pessoas. Empresas como IBM, Apple, Microsoft, Google, Amazon e Facebook utilizam extensivamente algoritmos de inteligência computacional em seus negócios. Novos negócios prosperam rapidamente em todas as áreas com a evolução da tecnologia de sensores e o desenvolvimento da internet das coisas.

A partir de 2012 modelos de redes neurais profundas* [212][333] têm obtido excelentes resultados em problemas complexos de reconhecimento de padrões. Os modelos são processados em hardware massivamente paralelo com unidades de processamento gráfico (GPUs), que têm alta capacidade de processamento e baixo consumo de energia em relação aos processadores convencionais (CPUs). Modelos de redes neurais profundas em problemas de visão computacional e processamento de linguagem natu-

* *Deep neural networks.*

ral têm obtido resultados comparáveis, e até superiores, aos de humanos. Estes resultados têm gerado grande expectativa em relação ao potencial de inteligência computacional para ciência de dados. O assunto é vasto e tem se expandido rapidamente. Este livro corresponde a uma formação básica no tema incluindo aplicações desenvolvidas em diferentes áreas.

Este capítulo apresenta uma introdução dos principais tipos de problemas em aplicações reais que serão abordados no livro. A seguir, um resumo da bibliografia e das principais ferramentas disponíveis, tanto comerciais como de código aberto.

1.1 Ciência de Dados

No método científico clássico, hipóteses teóricas levam ao desenvolvimento de modelos, que devem ser verificados pela observação do fenômeno. Nas ciências exatas, o modelo é representado por fórmulas matemáticas que codificam o conhecimento sobre o fenômeno [451]. O notável desenvolvimento da computação no século XX permitiu a modelagem e simulação de fenômenos altamente complexos, gerando enormes avanços na representação da realidade e na sua utilização para a tomada de decisões.

Os parâmetros dos modelos devem ser ajustados por dados observados para que o modelo seja efetivamente a representação da realidade. No entanto, esses dados, mesmo quando obtidos em condições experimentais controladas, contêm erros e incertezas devidos a imperfeições nos instrumentos e, principalmente, à dificuldade ou impossibilidade da observação do fenômeno modelado [419].

A realidade, muitas vezes, é bem mais complexa que qualquer teoria e, em alguns casos, não há um modelo físico capaz de descrever adequadamente um determinado fenômeno. A ciência de dados é um conjunto de métodos para coleta, armazenamento, processamento, análise e visualização de dados. O objetivo é o desenvolvimento de modelos diretamente a partir de dados, sem hipóteses teóricas. Em geral, os dados são obtidos de forma não controlada, sem o planejamento de experimentos ou o conhecimento da representatividade da amostra.

Os dados observados correspondem sempre a uma amostra do processo, obtida em condições específicas que determinam a sua validade. Um modelo obtido a partir de

uma amostra é uma representação dos dados da amostra e, portanto, uma representação indireta do processo real que produziu os dados. A validade do modelo é sempre limitada pela qualidade da amostra.

A **ciência de dados** (*data science*) se situa na intercessão do conhecimento de matemática, estatística, computação e engenharia com o conhecimento do domínio e dos dados de cada aplicação. A ubiquidade dos dados na era digital faz a ciência de dados ter aplicações em todos os setores, além de ser utilizada cada vez mais em problemas de ciências humanas, naturais e biológicas [295].

A integração de diversas disciplinas necessárias para lidar com grandes bases de dados era chamada de mineração de dados durante a década de 1990, mas passou a ser conhecida como ciência de dados principalmente a partir de 2010. Os dois termos são utilizados frequentemente com o significado similar. Podemos considerar que ciência de dados se refere ao conhecimento necessário, enquanto mineração de dados se refere ao processo de desenvolvimento de modelos a partir de dados.

A descoberta de conhecimento de bases de dados (KDD)*, ou simplesmente, mineração de dados, é um processo cíclico, de forma que, se os objetivos não forem atingidos, uma nova iteração deve ser realizada, refinando os resultados ou utilizando novos dados/ferramentas. A Figura 1.1 ilustra o processo de mineração de dados com seus possíveis ciclos e iterações.

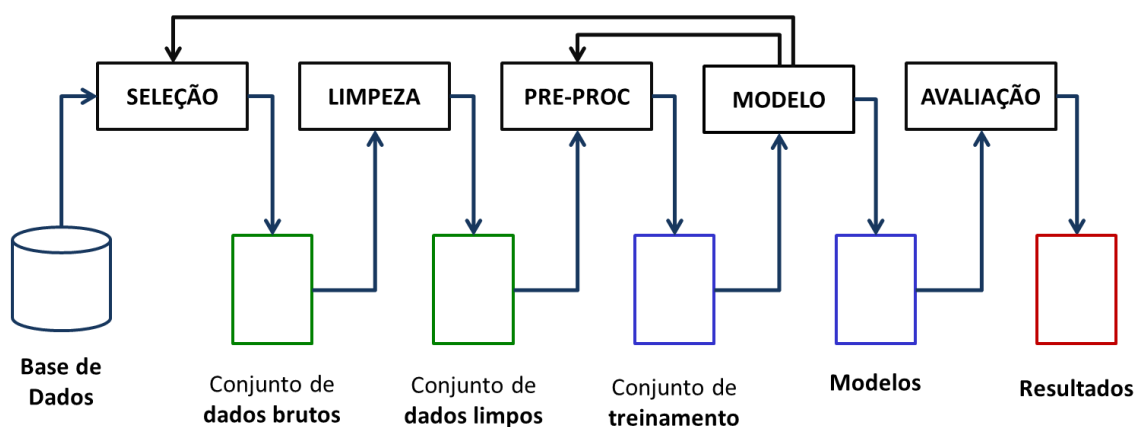


Figura 1.1: O processo de mineração de dados.

* *Knowledge Discovering in Databases*

A coleta e o armazenamento de dados, que aparece condensada na Figura 1.1, é um elemento extremamente importante do processo, principalmente para grandes corporações, que têm uma infraestrutura complexa de tecnologia da informação (TI). O processo começa com a seleção de um subconjunto de dados para ser utilizado para o ajuste de modelos. Esse conjunto de dados, chamado de conjunto de treinamento, é uma amostra considerada válida, podendo ser, dependendo do caso, a própria base de dados. Os dados do conjunto de treinamento são utilizados para o desenvolvimento do modelo. Durante o processo de construção do modelo, o conjunto de dados é particionado para o ajuste de parâmetros e a validação dos resultados.

A etapa de seleção de dados inclui o importante processo de escolha/seleção de variáveis, que pode ser feita a partir do conhecimento de especialistas ou de forma automática. Uma base de dados frequentemente possui valores ausentes e valores extremos ou outliers, que devem ser removidos para não prejudicar o ajuste do modelo.

O entendimento da aplicação e a clareza nos objetivos finais a serem atingidos são fundamentais para o sucesso de uma aplicação e a participação de especialistas do domínio é muito importante. O cientista de dados deve identificar os requisitos da aplicação, desenvolver e avaliar os resultados dos modelos, escolher o(s) modelo(s) mais adequado e coordenar a sua implantação.

O entendimento dos dados é realizado a partir da caracterização estatística e de técnicas de visualização. Esta etapa é importante para o desenvolvimento do modelo e será discutida no Capítulo 2. O pré-processamento dos dados com as principais técnicas de transformação linear também são apresentadas no Capítulo 2.

A principal etapa da mineração de dados é a geração dos modelos. Grande parte do conteúdo deste livro é orientada para o desenvolvimento e a análise dos resultados de modelos. Nos Capítulos 3 ao 6 são abordados os fundamentos estatísticos dos problemas de regressão, classificação e análise de agrupamentos. Os Capítulos 7 a 9 apresentam os principais métodos não lineares de inteligência computacional.

É importante avaliar corretamente o desempenho do modelo antes que a solução seja implantada. A qualidade dos modelos é avaliada através de estatísticas de validação, cujo objetivo é verificar se os requisitos da modelagem foram atingidos. Diversas estatísticas evidenciam diferentes características dos modelos e serão discutidas na parte de fundamentos.

A próxima seção apresenta os principais tipos de modelos que serão tratados neste livro. Na prática os problemas nem sempre se apresentam de forma clara. É importante entender corretamente as características dos problemas típicos para poder desenvolver uma solução específica num caso mais complexo.

1.2 Tipos de Modelos

A palavra “modelo” é utilizada em diversas áreas com conceitos diferentes e é importante definir claramente o que se entende por um modelo. O dicionário *Aurélio*⁷ define modelo como algo “que serve como objeto de imitação” ou, no contexto matemático, como uma “representação matemática de um fenômeno”. Neste livro, um modelo é uma representação simplificada de um sistema ou processo real obtida a partir dos dados da amostra. Os modelos de ciência de dados representam apenas a porção do processo que foi capturada pelos dados. Essa constatação, apesar de óbvia, é muito importante pois o modelo pode não ser capaz de prever corretamente o comportamento que não está representado nos dados de treinamento.

Um modelo, entendido como representação matemática de um processo real, pode ser classificado segundo a natureza do conhecimento empregado para a sua construção como **modelo de conhecimento** ou **modelo de comportamento**. Nos modelos de conhecimento (ou analíticos), as relações que descrevem o comportamento do sistema são escritas a partir das leis fundamentais da física ou do conhecimento humano. Nos modelos de comportamento (ou de entrada/saída), as relações que descrevem o comportamento do sistema são obtidas diretamente através da observação dos dados de entrada e saída. Os modelos de ciência de dados são essencialmente modelos de comportamento, embora o conhecimento do especialista do domínio e o conhecimento do analista sejam essenciais para o desenvolvimento de bons modelos.

As aplicações em ciência de dados são orientadas a problemas preditivos ou descritivos. Os problemas preditivos utilizam dados observados em variáveis de saída (variáveis dependentes), cujos valores são preditos pelo modelo a partir das variáveis de entrada (variáveis independentes). Nesse caso, o modelo realiza uma aproximação do mapeamento de entrada e saída observado. Nos problemas descritivos não há variáveis de saída e o objetivo do modelo é uma representação dos dados de entrada. Os modelos preditivos são ajustados por algoritmos de **aprendizado supervisionado**, enquanto

modelos descritivos são ajustados por algoritmos de **aprendizado não supervisionado**. Há também algoritmos de **aprendizado por reforço**, em que o modelo é ajustado diretamente através de sua interação com o ambiente. Esta abordagem é muito utilizada em problemas de controle e para o treinamento de agentes inteligentes.

Os problemas preditivos usuais são a classificação e a regressão. Na classificação, também chamada de classificação supervisionada ou reconhecimento de padrões, a variável de saída é uma classe, em geral uma variável discreta e não ordenada. Na regressão, a variável de saída é numérica, em geral um número real. Os dois problemas são, na verdade, similares e os modelos de inteligência computacional podem ser utilizados para a solução de ambos.

Em problemas descritivos, o objetivo do modelo é realizar uma descrição do conjunto de dados. Os principais problemas descritivos são análise de agrupamentos e regras de associação. A análise de agrupamentos, também chamada de classificação não supervisionada ou segmentação, utiliza diversos algoritmos para encontrar grupos de registros similares em grandes bases de dados. Algoritmos de regras de associação são muito utilizados em problemas descritivos cujo objetivo é encontrar itens que se relacionam na base de dados, como na análise de cestas de mercado e sistemas de recomendação. Recentemente, novos algoritmos de aprendizado não supervisionado têm surgido como **aprendizado da representação** em que o objetivo do modelo é produzir uma representação da entrada.

As técnicas de combinação de modelos, chamadas de *ensemble*, permitem o aumento do desempenho de modelos [150][475][604]. Em problemas preditivos, os melhores resultados do Prêmio Netflix⁸ e outras competições de ciência de dados⁹ são obtidos com *ensembles*.

A seguir, apresentamos uma descrição sucinta dos problemas que serão tratados neste livro, mostrando alguns exemplos de conjuntos de dados bastante conhecidos.

1.2.1 Regressão

O objetivo da regressão ou aproximação de funções é um modelo de sistema ou processo capaz de realizar uma estimativa do valor da variável de saída numérica a partir das variáveis de entrada. A regressão é uma tarefa preditiva e a avaliação da precisão

é feita em função do erro de predição, ou seja, a diferença entre o valor predito e o valor correto. O erro de predição é utilizado para o ajuste do modelo.

O problema de regressão linear é clássico na estatística, com diversas aplicações em engenharia e ciências. Na regressão linear, a variável de saída é calculada como uma combinação linear das variáveis de entrada ou de funções das variáveis de entrada chamadas de “**regressores**” ou de **características** (*features*). Além de eficiente, o modelo linear permite a análise de diversas características do problema. A regressão linear é explorada no Capítulo 3, sua extensão para a modelagem de séries temporais e sistemas dinâmicos é apresentada no Capítulo 4.

A Figura 1.2 mostra um exemplo de problema de regressão linear monovariável. O modelo (em vermelho) é ajustado a partir dos dados da amostra (em azul) e deve ser o mais próximo possível da relação alvo (em preto). Em geral, não se conhece precisamente o comportamento estatístico do processo além dos dados da amostra, de modo que não se pode afirmar se o modelo ajustado representa o processo (relação alvo) ou é apenas um modelo daquela amostra.

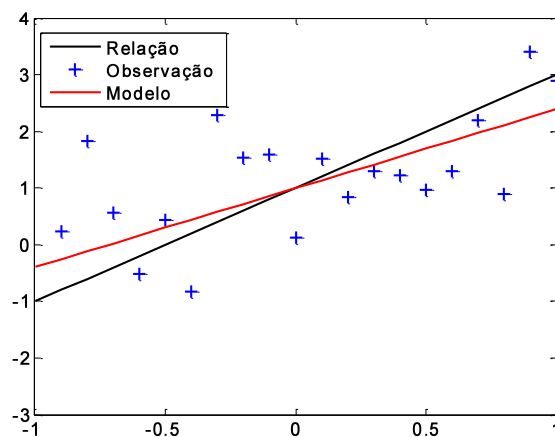


Figura 1.2: Problema de regressão linear.

Em muitos processos reais, o relacionamento entre as variáveis de entrada e saída é não linear. Nesse caso, o modelo é válido para as condições em que a amostra foi realizada. Se o processo tem um comportamento muito instável, como, por exemplo, o mercado financeiro, a validade de um modelo expira rapidamente e um modelo ajustado com dados históricos não consegue prever adequadamente o futuro. No caso de modelos não lineares, a predição de um valor muito diferente dos dados de entrada-saída

utilizados no conjunto de treinamento pode resultar em valores incorretos, pois o modelo foi ajustado para a amostra, que representa um determinado domínio.

Os problemas de regressão e de classificação têm muitas características em comum, uma vez que um modelo de classificação pode ser visto como um modelo de aproximação de funções discriminantes, como descrito a seguir.

1.2.2 Classificação

O objetivo da classificação é um modelo, chamado de classificador, capaz de classificar uma observação em uma das diferentes classes predefinidas a partir de valores das variáveis de entrada, que representam as características ou atributos da observação. O modelo representa o relacionamento entre os atributos e a classe, que é a variável de saída. A classificação é uma tarefa preditiva, pois o modelo deve ser capaz de prever a classe para um novo registro previamente desconhecido, isto é, um registro não utilizado no conjunto de treinamento. A avaliação de um modelo de classificação é realizada principalmente em função da sua precisão na predição da classe correta.

O conjunto de dados das flores Iris é certamente o exemplo mais conhecido e estudado em ciência de dados. O problema consiste em prever a categoria de flores Iris, mostradas na Figura 1.3, a partir dos valores de comprimento e largura das pétalas e sépalas. O conjunto de dados tem, portanto, quatro variáveis numéricas de entrada e uma variável de saída, cujos valores são as três classes possíveis. O conjunto de dados apresenta 50 instâncias para cada classe e foi utilizado pela primeira vez por Fisher em 1936 [183] como exemplo de funções discriminantes lineares.

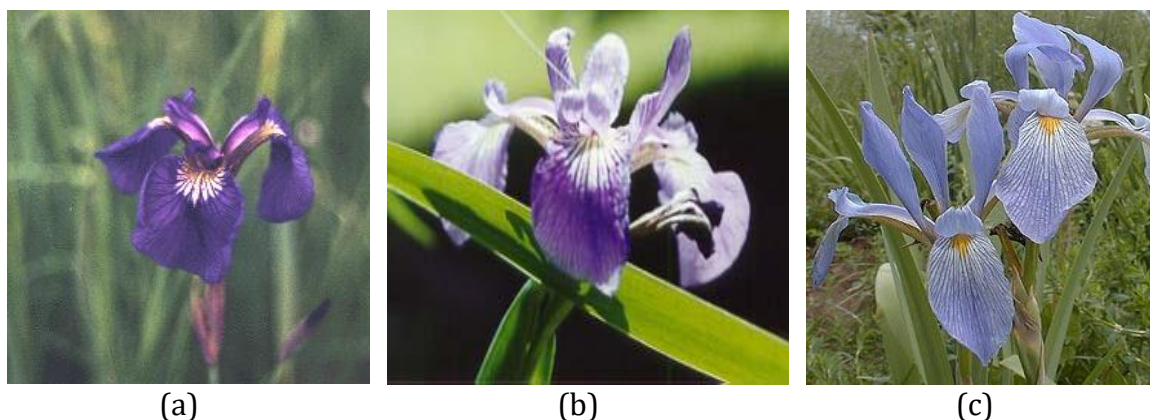


Figura 1.3: Flores Iris: (a) *Iris setosa*; (b) *Iris versicolor*; (c) *Iris virginica*.

O problema de classificação pode ser visto como uma busca por funções discriminantes (lineares ou não lineares) capazes de dividir o espaço das variáveis de entrada em regiões e associar cada região a uma classe (cf. Figura 1.4).

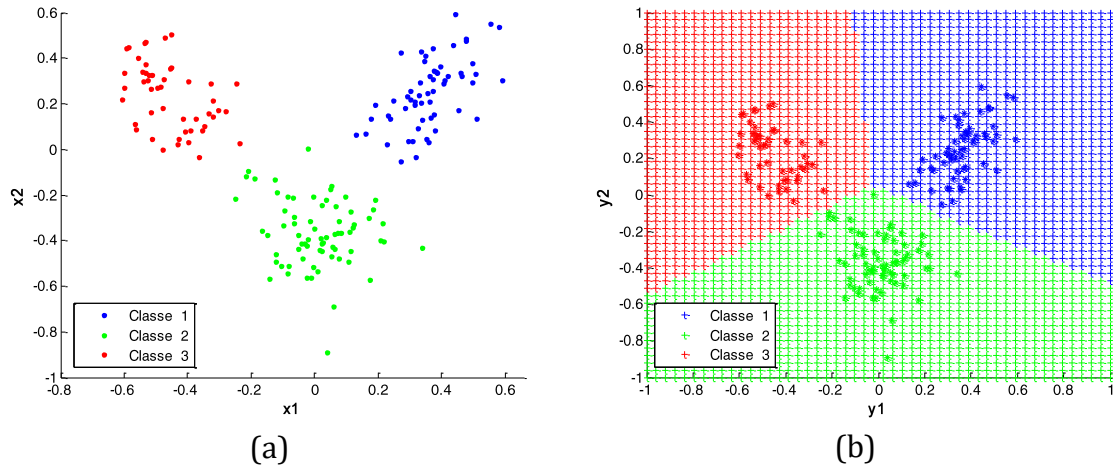


Figura 1.4: Problema de classificação: (a) conjunto de treinamento; (b) conjunto de testes.

Em situações em que é possível separar todos os registros de uma classe por uma função linear, diz-se que a classe é linearmente separável. Quando os registros de uma classe podem ser separados por uma função não linear, a classe é dita separável não linearmente. Na prática, na maioria dos casos, os registros não podem ser separados completamente por alguma função linear ou não linear. Nesse caso, as classes são ditas não separáveis uma vez que a informação coletada pelas variáveis de entrada não é suficiente para separar completamente as classes e, em consequência, o erro de classificação não pode ser nulo, por melhor que seja o modelo. O problema é explorado em detalhes no Capítulo 5, onde mostramos que é possível estabelecer um limite teórico do erro de um classificador. Os modelos lineares locais e os modelos de sistemas fuzzy e redes neurais podem tratar tanto problemas de classificação como problemas de regressão e serão apresentados no Capítulos 7, 8 e 9 respectivamente.

O problema de classificação tem sido estudado desde muito antes do surgimento das técnicas de inteligência computacional. Em estatística, o problema de classificação é chamado de reconhecimento de padrões [146]. As abordagens modernas para o problema reúnem técnicas estatísticas clássicas e técnicas de inteligência computacional [146]. Esse tipo de problema encontra inúmeras aplicações nos mais variados campos de negócio, como, por exemplo, diagnóstico médico, análise de crédito, classificação de documentos, classificação de imagens, bioinformática, entre outros.

1.2.3 Análise de agrupamentos

O objetivo da análise de agrupamentos é encontrar grupos de registros semelhantes no conjunto de dados. Na análise de agrupamentos, também chamada de classificação não supervisionada, não existe uma variável de saída ou variável alvo. O conjunto de dados contém, assim, apenas registros das variáveis de entrada.

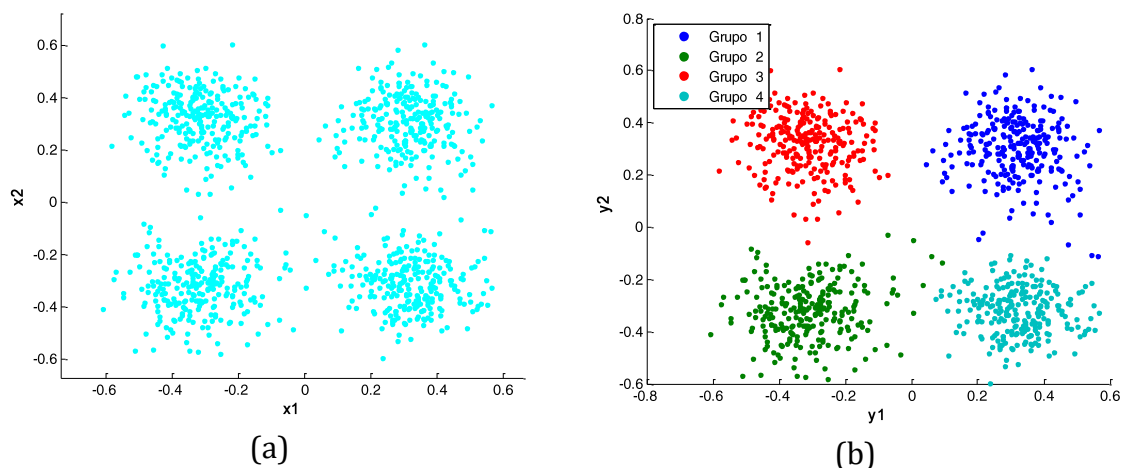


Figura 1.5: Problema de agrupamento: (a) conjunto de dados; (b) agrupamento.

Há diversas aplicações da análise de agrupamentos nos mais variados campos, como bioinformática, *marketing*, processamento de imagens etc. Uma das aplicações mais conhecidas é a segmentação de clientes, muito útil na definição de estratégias de relacionamento [530]. Nesse caso, as variáveis do conjunto de dados são características dos clientes em função das quais se deseja a análise.

Empresas com grandes bases de clientes, como operadoras de telefonia, utilizam análise de agrupamentos para segmentação de clientes, em que as variáveis representam a utilização dos serviços. Grupos de clientes podem ser alvo de campanhas de retenção ou promoções diferenciadas.

Um problema crítico na análise de agrupamentos é a avaliação da qualidade do resultado do agrupamento, pois a inexistência de uma variável de saída dificulta a definição de uma estatística de desempenho. Paradoxalmente, a definição do número de grupos é um dado de entrada para a maioria dos algoritmos. Há na literatura diversos índices que se propõem a realizar uma avaliação da qualidade do agrupamento. Entretanto, os índices refletem características geométricas desejáveis de um bom agrupamento e seu resultado deve ser interpretado por um analista. A experiência do analista e a participação do especialista do domínio são fundamentais para a análise de agrupamentos.

O problema de análise de agrupamentos possui diversas abordagens diferentes e um grande número de algoritmos. Há livros inteiros focados no assunto [573][194] e uma abordagem completa do problema foge ao escopo deste livro. As abordagens principais são discutidas no Capítulo 6.

Uma variante do problema de análise de agrupamento é chamada de classificação semi supervisionada [82], no qual um pequeno subconjunto dos dados anotados, que deve ser utilizado para a classificação de um conjunto maior de dados. A questão é tratada como um problema de agrupamentos em que os registros categorizados são utilizados para ajudar a ajustar o modelo.

1.3 Recursos Complementares

Esta seção apresenta um conjunto de recursos relacionados com o conteúdo deste livro e com a ciência de dados em geral.

1.3.1 Bibliografia de referência

A literatura na área de inteligência computacional com aplicação em ciência de dados é bastante diversificada. A abordagem estatística para a construção de modelos de dados está fartamente descrita em diversas referências. Um enfoque bastante completo que apresenta tanto os métodos mais avançados quanto dos métodos estatísticos clássicos é apresentado por Hastie, Tibshirani e Friedman¹⁰ [238].

O livro *Pattern Classification*,¹¹ de Duda e Hart [146], é um clássico em métodos estatísticos de classificação. O livro tem um volume correspondente com rotinas Matlab dos principais algoritmos [507]. O livro de Theodoris e Koutroumbas¹² [521], é outra referência em métodos estatísticos de classificação e análise de agrupamentos, que também tem um volume com rotinas em Matlab [522]. O livro de Bishop¹³ [52] é uma referência importante em método estatísticos e de inteligência computacional, trazendo um capítulo sobre métodos variacionais. Uma referência adicional para métodos estatísticos de classificação é o livro de Webb [553].

Em problemas de regressão, Montgomery [399] é uma referência clássica, em sua quarta edição. O clássico de modelos estatísticos de séries temporais e sistemas dinâmicos é o livro de Box, Jenkins e Reinsel [59], que também está na quarta edição. Outra obra de referência para o desenvolvimento de modelos dinâmicos de sistemas lineares

é o livro de Ljung¹⁴ [352], que serviu de base para o desenvolvimento da biblioteca *System Identification Toolbox*¹⁵ do Matlab. Schabenberger e Gotway [472] apresentam uma visão bastante abrangente sobre análise de dados espaciais.

Há diversas referências clássicas sobre as principais técnicas de inteligência computacional, como o livro de Haykin¹⁶[240][241] sobre redes neurais e os livros de Pedrycz e Gomide [432][433] sobre a teoria de conjuntos *fuzzy*. A obra de referência sobre máquinas de vetores de suporte e métodos baseados em funções de núcleo (*kernel*) é o livro de Schölkopf e Smola¹⁷[478], que apresenta uma abordagem completa dos métodos e suas formulações matemáticas e ainda inclui uma discussão sobre temas correlatos como regularização e otimização. Taylor e Cristianini¹⁸ [485], Huang, Kecman e Kopriva¹⁹ [269], e Kulkarni e Harman [326] apresentam uma excelente introdução à teoria de aprendizado estatístico. A obra de Vapnik [532]-[534] é a referência teórica mais completa. Em seu livro mais recente, Vapnik [533] traz uma apresentação simplificada dos principais conceitos da teoria.

O problema de construção de modelos a partir de dados é muito bem descrito por Cherkassky e Mulier [98], com uma abordagem que ressalta os pontos em comum dos vários algoritmos de inteligência computacional. Kecman²⁰ [301] apresenta os principais algoritmos de inteligência computacional, em que os métodos estatísticos clássicos são condensados no primeiro capítulo. A maioria dos livros de inteligência computacional é focada em sistemas *fuzzy*, redes neurais e algoritmos genéticos [152][318][520].

Há diversos livros sobre mineração de dados, o primeiro com este título foi editado por Fayyad e Piatetsky-Shapiro em 1996 [177]. O trabalho de Han e Kamber, publicado em três edições [232]-[234], também é uma referência importante e tem uma apresentação bastante completa. Han também participa em edições sobre a mineração de dados geográficos [386][387] e sobre mineração de dados em grafos [582]. O livro de Tan, Steinbach e Kumar²¹ [516] é outra referência clássica em mineração de dados. O livro escrito pelos autores do Weka²² [561] apresenta os principais algoritmos do *software*.

Um livro de Zaki e Meira [593] apresenta uma visão bem atual e completa do assunto com os principais algoritmos, além de mostrar aplicações em problemas complexos como mineração de grafos e sequências. O livro de Aggarwal [5] é uma referência mais completa e atualizada na área de ciência de dados, apresentando os fundamentos, técnicas e aplicações. Em suas 700 páginas, aborda o assunto de forma bem abrangente,

considerando o pré-processamento, detecção de *outliers*, mineração de dados estruturados, dados espaciais, séries temporais, sequências e dados não estruturados como textos e mineração da *web* e de redes sociais.

Mitchel²³ [392] é a referência clássica de aprendizado de máquinas. O livro de Murphy de 2012 [406] apresenta uma abordagem bem completa do tema sob a ótica probabilística. Alguns livros de aprendizado de máquina apresentam uma abordagem de mineração de dados [14][370] ou de análise inteligente de dados [44][45].

O livro *Deep Learning*,²⁴ de Googfellow, Bengio e Courville [212], publicado em 2016, é a referência atual sobre algoritmos de aprendizado profundo. O livro é bem completo e aborda desde os princípios básicos de álgebra linear, teoria das probabilidades e técnicas clássicas de aprendizado de máquinas até os mais avançados modelos que estão sendo utilizados atualmente em aplicações de inteligência artificial.

Liu²⁵ [348] é uma referência em problemas relacionados à internet. Os algoritmos de inteligência computacional têm sido utilizados em problemas com dados não estruturados como texto, grafos [1][43][178][502] e redes sociais [4][376]. Outras aplicações importantes são a bioinformática e sistemas financeiros[295].

Na literatura nacional, o livro editado por Solange Rezende em 2002 [451] reúne métodos de inteligência computacional, mineração de dados e aprendizado de máquina. A obra, assinada por 10 autores com a participação de vários colaboradores, permanece atual e inclui, além dos conceitos teóricos, uma série de aplicações práticas dos métodos apresentados. Alguns trabalhos na área de ciência de dados e inteligência computacional escritos em português apareceram recentemente [61][173][193][345]. Os modelos de redes neurais estão bem descritos na tradução do livro de Haykin [241] e no livro de Braga, Carvalho e Ludermir [62].

1.3.2 Recursos na internet

Em 2011, a Universidade de Stanford iniciou seu programa de oferta de cursos *online* gratuitos pela internet com três cursos de ciência da computação: banco de dados, aprendizado de máquina e inteligência artificial. Os mesmos cursos oferecidos aos alunos de Stanford foram disponibilizados livremente na internet, onde qualquer um poderia se inscrever e seguir aulas em vídeo. A grande diferença entre essa iniciativa e outras formas de educação *online* é que alunos inscritos podem responder perguntas,

realizar exercícios propostos e exames (dentro do mesmo ambiente computacional) e, ao final, desde que obtenham rendimento adequado, receber um certificado de participação assinado pelo professor. Os cursos atraíram mais de 100 mil inscritos em todo o mundo. A iniciativa gerou a criação de diversos repositórios de cursos *online* em universidades como Stanford²⁶ e MIT²⁷, além de empresas como Coursera²⁸ e Udacity²⁹, fundadas no início de 2012, que exploram este mercado. O curso de Machine Learning do Coursera,³⁰ apresentado por Andrew Ng, de Stanford, cobre a maioria dos assuntos tratados neste livro.

Aulas e conferências em vídeo também podem ser encontradas no VideoLectures³¹, que é uma espécie de YouTube acadêmico e contém vídeos sobre diversos assuntos. No VideoLectures é possível assistir a seminários sobre os temas de interesse apresentados pelos maiores especialistas da área. O Slide Share³² e o Scribd³³ também são repositórios de conteúdo geral, o primeiro no formato de *slides* de apresentações e o segundo no formato de documentos.

O principal portal de recursos de ciência de dados na internet é o KDnuggets,³⁴ que contém *links* para diversos recursos, como *softwares*, repositórios de dados, cursos etc., além de ofertas de empregos e também muita publicidade. O KDnuggets foi criado em 1997 e é mantido por Piatetsky-Shapiro, um dos principais pesquisadores da área e coeditor do primeiro livro de mineração de dados [177].

O repositório de dados que é referência em diversos trabalhos de ciência de dados é o UCI Machine Learning Repository,³⁵ mantido pela Universidade da Califórnia em Irvine. O UCI contém um acervo bastante completo de conjuntos de dados *benchmark* para avaliação de algoritmos. O UCI tem uma seção especial dedicada aos concursos de ciência de dados KDD Cup propostos nas edições do congresso Knowledge Discovery in Databases (KDD).³⁶ Atualmente, competições de algoritmos de ciência de dados se tornaram comuns através do *site* Kaggle.³⁷

O *site* do *software* de código aberto KEEL,³⁸ desenvolvido por um consórcio de seis grupos de pesquisa espanhóis, disponibiliza, além do próprio *software*, alguns conjuntos de dados *benchmark*. Os dados encontrados no *site* do KEEL são essencialmente os mesmos disponíveis no UCI, mas no KEEL eles podem ser encontrados em partições predefinidas, o que permite a comparação direta entre algoritmos. O *site* KEEL também

disponibiliza uma grande variedade de publicações e *links* para outros repositórios ligados à área de inteligência computacional.

Os resultados recentes nas áreas de inteligência computacional e ciência de dados são publicados em periódicos de circulação internacional. No Brasil, graças ao Portal de Periódicos Capes³⁹, o estudante de pós-graduação tem acesso aos principais periódicos científicos em todas as áreas. Na área de inteligência computacional e ciência de dados, os principais portais são o Science Direct⁴⁰ (Elsevier), o IEEEExplore⁴¹ (IEEE), o *site* da Springer⁴² e o portal da ACM Digital Library.⁴³ Os principais periódicos da área de ciência de dados estão listados no portal KDnuggets.⁴⁴

Pesquisadores da área de ciência de dados organizam anualmente diversos congressos internacionais sobre o tema. Os congressos mais importantes na área são o KDD, organizado pelo SIGKDD⁴⁵ e o ICDM,⁴⁶ organizado pelo IEEE. Na área de aprendizado de máquinas os congressos mais importantes são o NIPS⁴⁷ (NeurIPS a partir de 2018) e o ICML⁴⁸. Na área de inteligência computacional, os congressos mais importantes são organizados por especialidade. A cada quatro anos, é organizado o WCCI,⁴⁹ que reúne os principais congressos da área organizados pelo IEEE: FUZZ-IEEE de sistemas *fuzzy*; IJCNN de redes neurais; e IEEE CEC de computação evolutiva. Outras informações sobre eventos e publicações da área podem ser obtidas no *site* da Sociedade de Inteligência Computacional do IEEE.⁵⁰ No Brasil, a Associação Brasileira de Inteligência Computacional (ABRICOM),⁵¹ organiza o Congresso Brasileiro de Inteligência Computacional (CBIC), a cada dois anos.

Atualmente, é possível participar de discussões sobre diversos assuntos em *blogs*⁵² e perfis no Twitter⁵³ relacionados à área de ciência de dados. Há também diversos grupos de discussão em redes sociais,⁵⁴ principalmente no Facebook e no LinkedIn.

O desenvolvimento de modelos de inteligência computacional, principalmente redes neurais profundas, tem se beneficiado bastante de um ecossistema aberto, com *softwares* livres e linguagens de programação abertas, repositórios de artigos científicos como o arXiv⁵⁵ e repositórios de códigos como GitHub.⁵⁶

Finalmente, há na internet repositórios de conteúdo genérico que contêm material de diversos assuntos, incluindo tópicos relacionados à inteligência computacional e ciência de dados. É importante lembrar que artigos na internet não têm nenhum tipo de avaliação da qualidade da informação e devem ser consultados com cautela.

1.3.3 Softwares

Os *softwares* comerciais mais utilizados são o SAS (Statistical Analysis Software), o SPSS (Statistical Package for the Social Sciences) e o Statistica. Estes *softwares* foram desenvolvidos inicialmente como pacotes estatísticos que depois incorporaram outros módulos. O pacote de mineração de dados do SAS⁵⁷ fornece suporte para todo o processo, incluindo limpeza, transformações de dados e modelos. O SAS ainda tem uma customização de seus módulos para diversos setores de negócios. O IBM SPSS Modeler⁵⁸ (ex-PASW, ex-Clementine) e tem características similares ao concorrente SAS. O SAS e o SPSS oferecem uma solução integrada, considerando todas as etapas do processo, desde a aquisição de dados até a implantação dos modelos. A aquisição da SPSS pela IBM mostra claramente essa tendência. O Statistica Data Miner⁵⁹ também é integrado aos módulos de estatística e tem uma variedade maior de algoritmos, permitindo otimizar modelos para um problema específico. Os resultados dos modelos do Statistica também podem ser exportados em C, Java, Visual Basic e PMML. Os três pacotes oferecem, como módulos adicionais, funcionalidades de mineração de textos.

O Oracle Data Mining e o Microsoft SQL Server Data Mining são bibliotecas de mineração de dados que rodam dentro dos respectivos bancos de dados. A vantagem desse tipo de solução é que os modelos são desenvolvidos diretamente no banco de dados, podendo ser integrados à plataforma operacional. Os produtos da Oracle e da Microsoft oferecem uma variedade limitada de algoritmos, alguns proprietários que não são completamente explicados na documentação. Esses *softwares* têm poucos recursos nativos para visualização e caracterização estatística dos dados. A vantagem do Oracle Data Mining é sua presença no mercado e a integração com os outros produtos da Oracle e a possibilidade de ser diretamente integrado a programas Java e à linguagem PLSQL do Oracle. O Microsoft SQL Server Data Mining também tem a vantagem de ser totalmente integrado à plataforma .NET da Microsoft e o seu custo é bem inferior. Nas versões mais recentes, as duas plataformas têm recursos para mineração de textos.

O desenvolvimento de *softwares* comerciais segue a tendência crescente de oferta de serviços na nuvem. Nesse sentido, a plataforma Microsoft Azure Machine Learning⁶⁰ permite o desenvolvimento e o *deploy* de modelos de ciência de dados diretamente a partir da nuvem. O produto Amazon Machine Learning⁶¹ também segue essa tendência na plataforma de nuvem Amazon Web Services. A Prediction API⁶² da plataforma de nuvem do Google também é uma iniciativa nessa linha.

Para grandes corporações que atuam em mercados altamente competitivos e lidam com grandes massas de dados, o ganho potencial dos resultados obtidos com modelos de ciência de dados supera o custo de aquisição e manutenção de pacotes comerciais [439]. Para pequenas e médias empresas e, principalmente, o mundo acadêmico, o custo de uma licença de um pacote comercial como os mencionados pode ser proibitivo. Há no mercado outros *softwares* comerciais com custos mais acessíveis como o PolyAnalyst,⁶³ e sua variante para análises de textos TextAnalyst⁶⁴.

Há *softwares* livres que podem ser utilizados para o desenvolvimento de soluções sem uma linguagem de programação. O primeiro *software* livre de mineração de dados, desenvolvido em 1997 na Universidade de Waikato, foi o Weka.⁶⁵ Codificado em Java, o Weka tem código-fonte aberto, embora a documentação não seja das mais claras. O Weka contém uma variedade enorme de algoritmos, com diversas variedades de algoritmos de indução de árvores e regras de decisão. O Weka também tem uma interface gráfica intuitiva de fácil utilização. Em 2006, o Projeto Weka foi adquirido pela Pentaho⁶⁶ para ser integrado ao pacote de inteligência de negócios da empresa. O Weka continua sendo desenvolvido como projeto de *software* livre.

O RapidMiner⁶⁷ utiliza o mesmo modelo de negócios da Pentaho, fornecendo *softwares* livres para oferecer serviços de consultoria e treinamento. O RapidMiner também é desenvolvido em Java e incorpora algoritmos de outros *softwares* livres, como o Weka e o LibSVM. O RapidMiner tem recursos elaborados de visualização de dados e de resultados de modelos, além de permitir o desenvolvimento de *workflows* de modelagem complexos, com otimização de modelos por algoritmos genéticos. O KNIME⁶⁸ também é desenvolvido em Java e incorpora algoritmos de outros *softwares* livres, como o Weka e o LibSVM, numa interface gráfica de fácil utilização. O KNIME tem recursos de visualização melhores que o Weka, mas é equivalente em termos de algoritmos.

Uma alternativa ao padrão Java, utilizado pelos *softwares* livres citados anteriormente, é o Orange,⁶⁹ desenvolvido em C++ e Python. O Orange é um projeto acadêmico e apresenta uma boa variedade de algoritmos numa interface gráfica agradável e com bons recursos de visualização. O Orange é desenvolvido a partir de uma biblioteca que é fornecida junto com o *software*, permitindo o desenvolvimento de aplicações pelo usuário.

A utilização de *softwares* livres e linguagens de programação abertas têm impulsionado o desenvolvimento da ciência de dados. As principais bibliotecas do Python para

ciência de dados são o Pandas⁷⁰ e o Scikit Learn,⁷¹ que utilizam outras bibliotecas de referência do Python como NumPy e SciPy. O Python é desenvolvido por uma comunidade motivada e suas bibliotecas permitem o acesso a diversos algoritmos. Há uma grande variedade de recursos em Python na internet.⁷²

A linguagem R⁷³ é um projeto de *software* livre da comunidade estatística com um ambiente computacional e bibliotecas com uma grande variedade de rotinas para tratamento, processamento e visualização de dados. As rotinas são organizadas em pacotes para cada assunto e podem ser acessadas por uma linguagem de programação simples, que também é ser utilizada para o desenvolvimento de novos algoritmos. Há diversas extensões e interfaces gráficas para a linguagem, assim como empresas⁷⁴ que fornecem extensões e consultoria. A linguagem R tem sido incorporada em *softwares* comerciais como o SAS e em bancos de dados como Oracle.

O site KDnuggets realiza regularmente uma enquete com o objetivo de determinar os *softwares* mais utilizados pelos usuários do portal. Nos últimos anos, as linguagens R e Python têm se destacado como linguagens mais utilizadas.

Um ambiente computacional importante na área de engenharia é o Matlab.⁷⁵ O Matlab é um *software* comercial que tem um núcleo central, basicamente a janela de comando e bibliotecas numéricas básicas. Os recursos avançados do Matlab são comercializados como pacotes, chamados *Toolbox*, que contêm rotinas para as mais diversas áreas, como biologia, finanças, sistemas de controle etc. Uma configuração adequada para a área de inteligência computacional e ciência de dados deve conter essencialmente os pacotes de estatística,⁷⁶ redes neurais⁷⁷ e algoritmos genéticos.⁷⁸ Pacotes adicionais, como o de otimização⁷⁹ e de identificação de sistemas,⁸⁰ também são importantes. O pacote de programação paralela,⁸¹ que permite a execução de *scripts* Matlab em máquinas com processadores de múltiplos núcleos e *clusters*, pode ser necessário em aplicações computacionalmente intensivas. Os programas desenvolvidos em Matlab podem ser integrados a *softwares* corporativos através de compiladores para as plataformas .NET,⁸² Java⁸³ e Excel.⁸⁴ Além dos pacotes mantidos e desenvolvidos pela Mathworks, existe uma comunidade de usuários ativa que desenvolve rotinas e pacotes para o Matlab, organizados no repositório Matlab Central.⁸⁵ Há mais de 1200 livros escritos sobre o Matlab ou sobre aplicações utilizando o ambiente computacional.⁸⁶ As figuras neste livro foram geradas com Matlab.

1.4 Comentários Finais

Neste capítulo foram apresentados os conceitos iniciais inteligência computacional para ciência de dados e uma introdução dos assuntos abordados no livro. Os recursos bibliográficos representam um apanhado das principais fontes que podem servir de apoio a um estudo mais aprofundado nesses temas. É provável que alguma referência importante não tenha sido incluída e é provável que alguma informação esteja desatualizada, principalmente as referências da internet e *softwares*.

2 PRÉ-PROCESSAMENTO

O pré-processamento inclui a seleção, limpeza e transformação de dados. O objetivo dessa etapa é a preparação do conjunto de dados que será utilizado para o ajuste do modelo, chamado de **conjunto de treinamento**. Em geral, a etapa de pré-processamento consome de 60% a 80% do tempo de todo o processo de mineração de dados e fornece informações importantes que vão orientar a análise e o ajuste dos modelos.

Este capítulo introduz os conjuntos de dados encontrados em aplicações de ciência de dados. A seguir, são apresentados os principais conceitos estatísticos de métodos exploratórios necessários para o entendimento dos dados e, principalmente, para a interpretação adequada dos resultados da visualização. As técnicas de limpeza de dados e as transformações lineares mais importantes são apresentadas na sequência. Algumas aplicações que utilizam os conceitos discutidos no capítulo são apresentadas no final.

2.1 Conjuntos de Dados

Há uma diferença entre dado, informação e conhecimento [451]. Dado é o registro de um fato, informação é o fato analisado no seu contexto e conhecimento, característica inerentemente humana, é a interpretação da informação num contexto específico visando à sua aplicação. Por exemplo, o preço de uma ação num determinado instante na bolsa de valores é um dado; a variação do preço da ação é uma informação; e a interpretação dessa informação no contexto da conjuntura econômica do momento para a tomada de decisão de investimento é o conhecimento.

Uma metáfora bem-humorada que enfatiza a importância da apresentação da informação é mostrada na Figura 2.1.

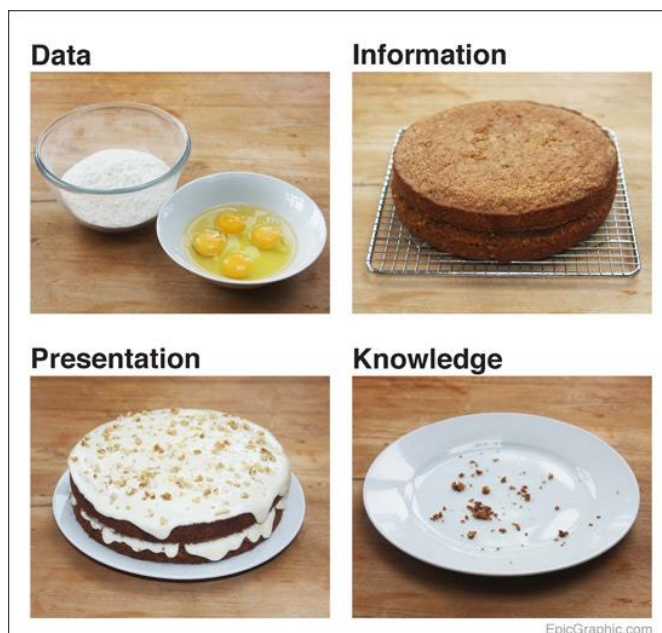


Figura 2.1: *Data Cake*. Fonte: epigraphics.com.⁸⁷

Os dados utilizados numa aplicação podem ser estruturados ou não estruturados. Dados estruturados são aqueles organizados em tabelas, nas quais as linhas são os **registros** que representam as observações e as colunas, as **variáveis** ou as características observadas em cada registro, frequentemente chamadas de **atributos**. Os dados não estruturados são aqueles que, na sua forma bruta, não são tabulados. Exemplos de dados não estruturados são imagens, sons, textos, estruturas gráficas etc. Neste livro serão considerados principalmente os modelos de dados estruturados. Modelos de dados não estruturados, principalmente de textos e de estruturas na forma de grafos (redes), têm sido cada vez mais importantes em ciência de dados [295].

Os dados estruturados são armazenados em bases de dados, de onde são selecionados para o ajuste dos modelos. Em sistemas corporativos, os dados podem estar armazenados em diversas bases diferentes. Nesse caso, a extração de dados pode se tornar extremamente complexa e é realizada por processos chamados ETL (*extract, transform and load*). Em ambientes mais simples, a seleção é feita diretamente a partir da base de dados e o conjunto de dados se apresenta na forma de uma tabela ou arquivo de dados.

Num conjunto de dados estruturados, cada linha ou registro contém os valores observados de um conjunto de variáveis. As variáveis podem ser contínuas, discretas ordenadas ou discretas não ordenadas. As variáveis contínuas armazenam valores numéricos do tipo real; as variáveis discretas ordenadas armazenam valores do tipo inteiro; enquanto as variáveis discretas não ordenadas armazenam valores categóricos ou nominais. As variáveis binárias são um tipo especial de variáveis discretas que tem apenas dois valores. Tipicamente, um conjunto de dados pode conter variáveis de diversos tipos. A maioria dos algoritmos de estatística e inteligência computacional trabalha com variáveis numéricas. Nesse caso, as variáveis categóricas são transformadas num conjunto de variáveis binárias chamadas variáveis indicadoras (ou *dummy*).^{*} Há algoritmos especializados em modelos de variáveis nominais – em geral, essa área é conhecida com aprendizado simbólico [392][451].

Neste livro, para facilitar o tratamento matemático dos algoritmos, as variáveis são consideradas numéricas (contínuas ou discretas) e as variáveis nominais são transformadas em variáveis indicadoras binárias no pré-processamento.

Um conjunto de dados é representado por $\mathcal{T} = \{(\mathbf{x}(t), \mathbf{y}(t)), t = 1 \dots N\}$. Cada registro é formado pelo par ordenado $(\mathbf{x}(t), \mathbf{y}(t))$, onde $\mathbf{x}(t)$ é o vetor[†] que contém os valores das variáveis de entrada e $\mathbf{y}(t)$, o vetor contendo os valores das variáveis alvo ou de saída. Sem perda de generalidade, os vetores de variáveis de entrada e saída são escritos como $\mathbf{x}(t) \in \mathcal{R}^p$ e $\mathbf{y}(t) \in \mathcal{R}^q$, onde p e q são, respectivamente, o número de variáveis de entrada e de saída. As variáveis nominais são codificadas como variáveis indicadoras que, assim como as variáveis inteiras, são um subconjunto das variáveis reais.

Problemas de saídas múltiplas ($q > 1$) podem ser representados como múltiplos problemas de saída simples se as variáveis de saída forem desacopladas, ou seja, o valor de uma saída não interfere no valor de outra. Nesse caso, um modelo independente é desenvolvido para cada saída. Em problemas descritivos, não há variáveis de saída e o conjunto de dados contém apenas variáveis de entrada.

^{*} Uma variável nominal com n valores é codificada com n variáveis binárias chamadas variáveis indicadoras, em que cada variável indica a observação de um valor nominal da variável original.

[†] Vetores e matrizes são grafados em negrito, vetores em letras minúsculas e matrizes em letras maiúsculas. Valores escalares são grafados em itálico. Letras maiúsculas gravadas em cursiva, como $\mathcal{A}, \mathcal{B}$ etc., indicam conjuntos. Letras gregas serão usadas para indicar parâmetros de modelos.

Alguns problemas apresentam características especiais que interferem na interpretação do conjunto de dados. Em problemas dinâmicos, existe uma dependência intrínseca entre uma observação e as observações precedentes. Nesse caso, as variáveis são séries temporais em que os registros são apresentados em sequência e o índice $t = 1 \dots N$ representa um instante de tempo. Os problemas em que a ordem das observações não é relevante são chamados estáticos e, nesse caso, qualquer permutação dos registros no conjunto de dados produz o mesmo resultado. Em problemas georreferenciados, existe uma dependência espacial entre as observações e cada registro está associado a duas (ou três) variáveis que representam as coordenadas da observação. Os problemas chamados espaço-temporais combinam essas duas características: contêm variáveis espaciais e os registros são armazenados em sequência. Este livro não aborda especificamente problemas espaciais ou espaço-temporais, apenas a modelagem de sistemas dinâmicos lineares no Capítulo 4.

Para a apresentação matemática dos algoritmos, o conjunto de dados é representado por matrizes. A matriz (de variáveis) de entrada é a matriz $\mathbf{X}$, de dimensão $N \times p$, e a matriz (de variáveis) de saída é a matriz $\mathbf{Y}$, de dimensão $N \times q$. As dimensões das matrizes indicam que os registros estão organizados em linhas e, para cada registro, os vetores* de variáveis de entrada e saída.

A notação apresentada anteriormente será utilizada ao longo do livro. Essa notação unificada facilita o entendimento dos algoritmos e a transposição de conceitos de modelos lineares para os modelos não lineares.

A próxima seção apresenta a primeira etapa na exploração de dados: a caracterização e visualização de dados.

2.2 Caracterização Estatística

A caracterização estatística de dados é uma das etapas mais importantes na ciência de dados. Essa fase permite o entendimento dos dados e facilita a análise dos resultados dos algoritmos.

* Matematicamente, um vetor é uma lista de valores e pode ser tratado como linha ou coluna na multiplicação com matrizes. Neste livro são utilizados vetores linha.

Estatisticamente, um conjunto de dados é tratado como uma amostra de um processo estocástico, ou seja, um processo cujo comportamento é imprevisível, mas pode ser descrito estatisticamente através de uma função de distribuição de probabilidades. O conceito de amostra é muito importante em ciência de dados, uma vez que os modelos serão ajustados a partir de uma amostra e não o processo estocástico. Essa é uma diferença importante entre a abordagem de inteligência computacional e a abordagem estatística, na qual em geral se busca a modelagem do processo [98], a partir de uma hipótese sobre a distribuição de probabilidades.

A maioria dos algoritmos de ciência de dados pressupõe implicitamente que o processo que gerou a amostra representada pelo conjunto de dados é estacionário, ou seja, não varia com o tempo. Dessa forma, podemos utilizar um conjunto de dados coletados num determinado instante para realizar previsões em outro momento uma vez que o processo permanece o mesmo entre o tempo de coleta da amostra e a realização da previsão. Entretanto, essa hipótese pode não ser verdadeira e, nesse caso, os modelos não terão bons resultados de previsão. Esse é tipicamente o caso da modelagem de processos altamente complexos como o mercado financeiro. A caracterização e modelagem de problemas dinâmicos será abordada no Capítulo 4. Neste capítulo, é apresentada a caracterização e visualização de dados de problemas estáticos.

2.2.1 Análise monovariável

O primeiro passo na caracterização estatística de uma amostra é o cálculo de estatísticas das variáveis. Uma estatística é qualquer função obtida a partir de uma amostra. As estatísticas da amostra, em geral, são calculadas para as variáveis de entrada do conjunto de dados, mas podem ser calculadas também para as variáveis de saída. As principais estatísticas são:

- **média**, uma medida do valor central da amostra:

$$\mu \approx \bar{x}_i = \frac{1}{N} \sum_{t=1}^N x_i(t), i = 1 \dots p \quad (2.1)$$

- **variância**, uma medida da dispersão da amostra:

$$Var(x_i) \approx \hat{\sigma}_i^2 = \frac{1}{N-1} \sum_{t=1}^N (x_i(t) - \bar{x}_i)^2, i = 1 \dots p \quad (2.2)$$

O **desvio padrão** estimado da variável $\hat{\sigma}_i \approx \sqrt{Var(x_i)}$ é comumente utilizado para a caracterização da dispersão de uma variável, pois seu valor tem a escala da variável.

Em amostras muito grandes de processos estacionários, a média da amostra é uma boa estimativa da média do processo. Esse resultado é conhecido na estatística como a “lei dos grandes números”. Da mesma forma, a variância da amostra converge para a variância do processo com o aumento do número de registros da amostra. Em conjuntos de dados pequenos, deve-se tomar cuidado ao utilizar essas estatísticas como estimativas de suas correspondentes do processo.

A média e a variância são os parâmetros que definem a função de densidade de probabilidades normal $N(\mu, \sigma^2)$, calculada como:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (2.3)$$

A Figura 2.2a apresenta exemplos da distribuição normal com diversos valores de média e desvio padrão (σ). Quanto maior o desvio padrão, maior a dispersão dos dados e, conseqüentemente, menor a probabilidade da média. Na Figura 2.2b, as funções de distribuição de probabilidades cumulativa dos exemplos mostrados na Figura 2.2a.

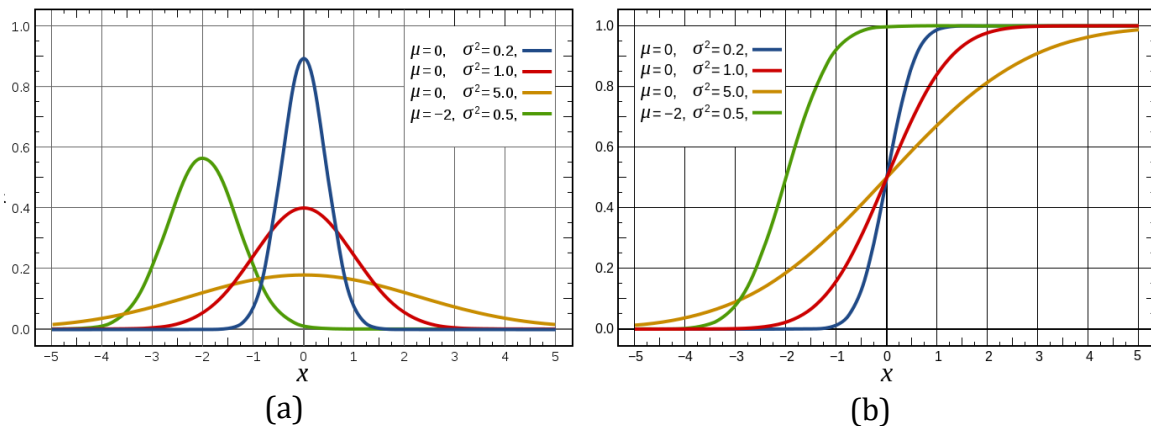


Figura 2.2: Distribuição normal: (a) função de densidade de probabilidades; (b) função de probabilidades cumulativa. Fonte: Wikipédia.⁸⁸

A função de distribuição de probabilidades normal, ou simplesmente distribuição normal, é muito importante na estatística. Um dos principais resultados envolvendo a distribuição normal é o teorema central do limite. O teorema estabelece que a distribuição da média de amostras de processos estacionários gerados com qualquer distribuição, de média e variância constantes, converge para uma distribuição normal com a média do processo.

A distribuição normal é representada matematicamente por uma função gaussiana que possui diversas propriedades e é utilizada em inúmeras aplicações. Em inteligência computacional, a função gaussiana aparece como uma função de base para o desenvolvimento de vários tipos de modelos, como será apresentado no Capítulo 7.

Outras estatísticas importantes para a caracterização de dados são:

- valores extremos: **mínimo** e **máximo**;
- a **mediana**: é o valor que divide o conjunto de dados em subconjuntos iguais, ou seja, 50% dos registros são maiores que a mediana, 50% são menores que a mediana. A mediana é o percentil 50 da amostra;
- o **percentil P** é o valor que divide o conjunto de dados de forma que $P\%$ dos registros são menores ou iguais a esse valor;
- a **faixa interquartil** é o intervalo compreendido entre o percentil 25 e o percentil 75, respectivamente chamados de primeiro quartil ($Q1$) e terceiro quartil ($Q3$), sendo a mediana o segundo quartil ($Q2$);
- a **moda**, calculada apenas para variáveis discretas, é o valor observado com maior frequência.

A mediana é uma medida de centralidade da amostra que é insensível a distribuições assimétricas. A média pode dar uma ideia equivocada do valor central de um conjunto de dados na presença de valores aberrantes (*outliers*) ou distribuições assimétricas. A comparação entre a média e a mediana permite detectar facilmente assimetrias na distribuição dos dados. Distribuições assimétricas podem gerar modelos tendenciosos. Da mesma forma, as medidas de dispersão dependem da forma da distribuição. A variância da amostra só tem sentido em distribuições simétricas; no caso de distribuições assimétricas, deve-se utilizar a faixa interquartil.

A Tabela 2.1 apresenta o resultado das estatísticas para as variáveis de entrada do conjunto de dados das flores Iris. Pela comparação entre os valores de média e mediana

(percentil 50), percebe-se uma assimetria na distribuição da variável x_3 . Ao avaliar as estatísticas, é importante observar a escala das variáveis. No caso das Flores Íris, todas as variáveis representam uma medida em centímetros e, portanto, é possível realizar a comparação direta de valores de diferentes variáveis. No caso geral, em que as variáveis representam medidas distintas, recomenda-se que a análise seja feita sobre as variáveis padronizadas (cf. seção 2.3.1).

Tabela 2.1: Estatísticas básicas do problema Íris

	x_1	x_2	x_3	x_4
Mínimos	43,0	20,0	10,0	1,0
Máximos	79,0	44,0	69,0	25,0
Médias	58,4	30,6	37,6	12,0
Desvios	8,3	4,4	17,7	7,6
Percentil 25	51,0	28,0	16,0	3,0
Percentil 50	58,0	30,0	43,5	13,0
Percentil 75	64,0	33,0	51,0	18,0

Em problemas de classificação, é interessante avaliar as variáveis por classe. O **gráfico de linhas** permite que se faça uma avaliação visual das diferenças entre os registros de cada variável por classe e pode ajudar a identificar as variáveis mais relevantes para a separação das classes. No gráfico de linhas, cada variável é um valor no eixo horizontal e cada registro é uma linha ligando as variáveis em seu valor correspondente. A Figura 2.3 mostra um exemplo de gráfico de linhas para o conjunto de dados Íris com variáveis padronizadas (cf. seção 2.3.1). Podemos observar que as classes podem ser mais bem separadas no plano formado pelas variáveis x_3 e x_4 . Essa mesma conclusão será confirmada por outros tipos de gráficos mostrados.

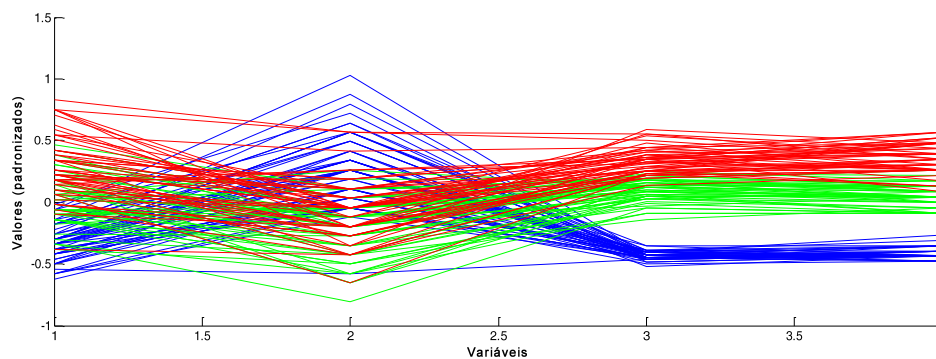


Figura 2.3: Gráficos de linha do problema Íris.

Há várias funções de distribuição de probabilidades⁸⁹, com aplicações em diversas áreas. A distribuição de cada variável pode ser avaliada qualitativamente através de **histogramas** ou diagramas de frequência. Um histograma é uma representação gráfica da distribuição de probabilidade da variável (cf. Figura 2.4). Há diversos métodos estatísticos para o ajuste da função de distribuição de probabilidades [146][238], bem como livros inteiros dedicados ao assunto [482][490].

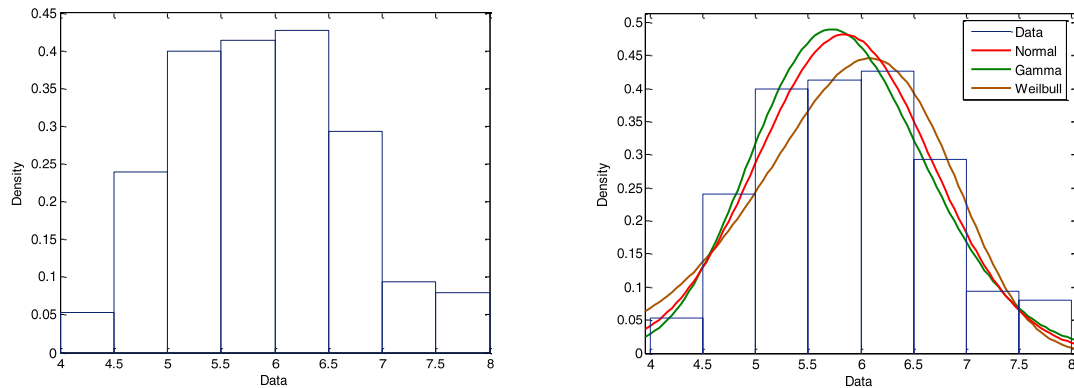


Figura 2.4: Histogramas e ajuste de distribuições.

As técnicas de inteligência computacional geralmente não levam em conta hipóteses sobre a distribuição de probabilidades que gerou a amostra. Entretanto, a análise dos histogramas é fundamental para o entendimento dos dados e para dar suporte à análise dos resultados do modelo. A avaliação visual dos histogramas pode revelar características indesejáveis na distribuição dos dados, como assimetria e distribuições multimodais (cf. Figura 2.5). Nesses casos, as estimativas de média e desvio padrão podem perder completamente o sentido.

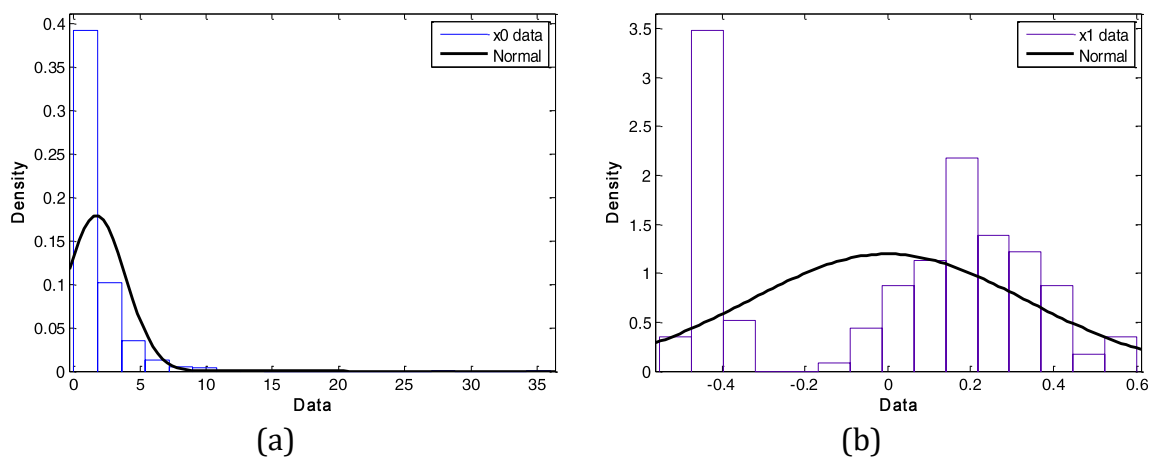


Figura 2.5: Histogramas: (a) distribuição assimétrica; (b) distribuição multimodal.

Em problemas de classificação, os histogramas podem ser calculados por classe, o que permite a análise da distribuição de cada classe no universo da variável. Os histogramas separados por classes das variáveis no conjunto de dados Íris são mostrados na Figura 2.6. Nesse exemplo, os dados são mais misturados nas variáveis x_1 e x_2 que nas variáveis x_3 e x_4 , como observado no gráfico de linhas da Figura 2.3. As variáveis x_3 e x_4 apresentam um comportamento multimodal. Entretanto, ao separar por classes, verifica-se que cada concentração de dados corresponde a uma classe, o que pode ser bastante útil para a construção de classificadores.

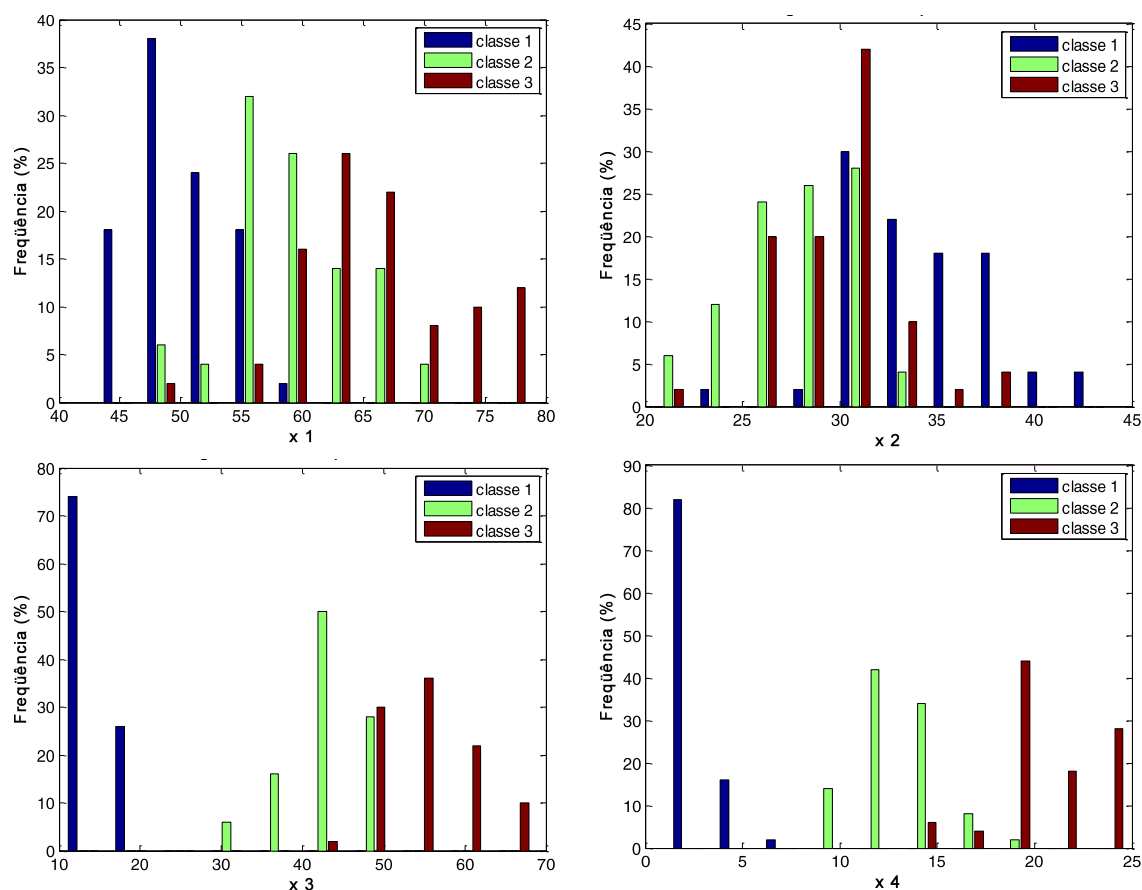


Figura 2.6: Histogramas das distribuições das variáveis por classe no problema Íris.

É possível verificar qualitativamente se uma amostra foi gerada por um processo de distribuição normal através do gráfico de probabilidade normal. Nesse tipo de gráfico, se a amostra foi gerada por um processo de distribuição normal, os pontos aparecem alinhados; no caso de uma distribuição não normal, há uma curvatura no gráfico. A Figura 2.7 apresenta os histogramas de uma amostra de duas variáveis e os respectivos gráficos de distribuição normal. As Figura 2.7a e 2.7c mostram a distribuição de uma

variável que pode ser representada por um processo de distribuição normal; as Figura 2.7b e 2.7d mostram a distribuição de uma variável gerada por um processo que não corresponde à distribuição normal.

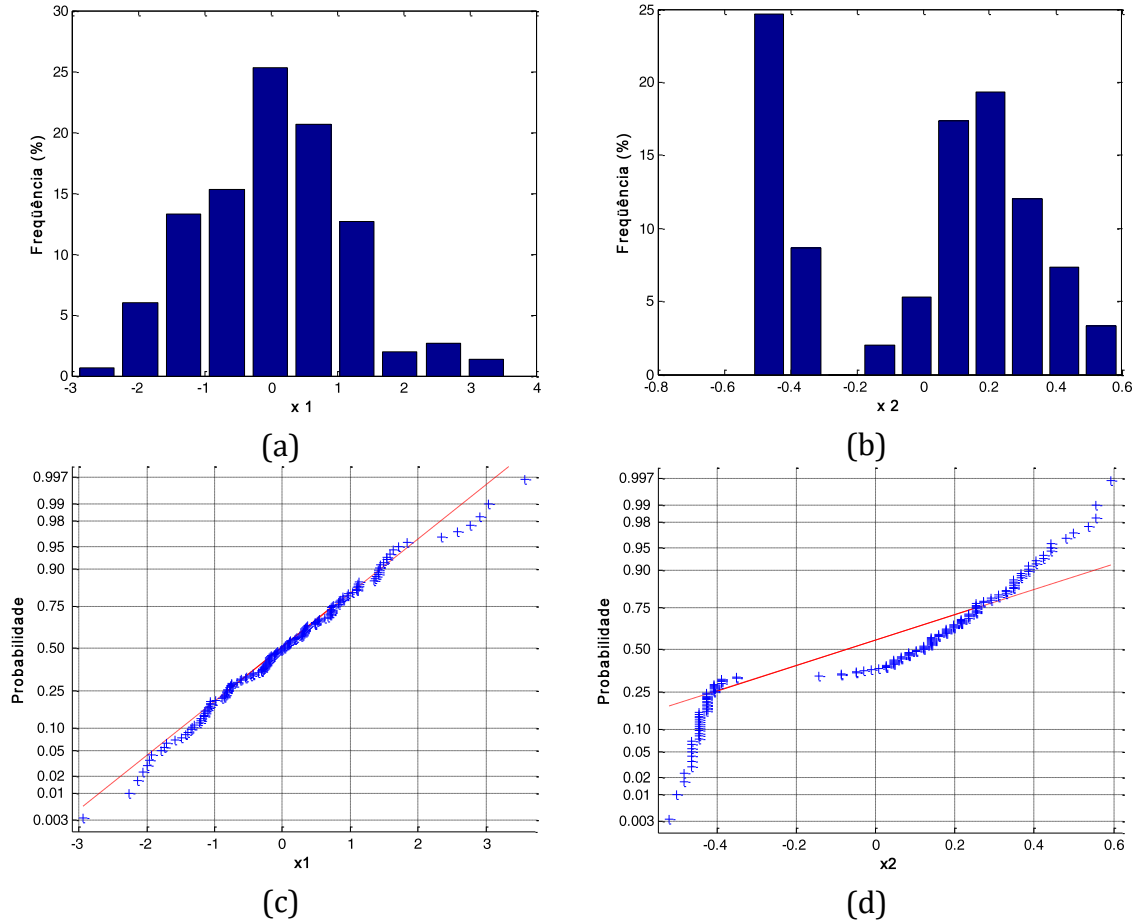


Figura 2.7: Histogramas e gráficos de probabilidade normal: (a) e (c) amostra gerada por distribuição normal; (b) e (d) amostra gerada por distribuição não normal.

2.2.2 Análise multivariada

A análise das características de cada variável é importante, mas é preciso também considerar igualmente as características multivariadas do processo.

A principal estatística multivariada estimada a partir de uma amostra é a **matriz de covariâncias** $\hat{\Sigma} \in \mathcal{R}^{p \times p}$, cujos elementos $\hat{\sigma}_{ij}^2 = Cov(x_i, x_j)$ são calculados como:

$$Cov(x_i, x_j) \approx \hat{\Sigma} = \frac{1}{N-1} \sum_{t=1}^N (x_i(t) - \bar{x}_i)(x_j(t) - \bar{x}_j), i = 1 \dots p, j = 1 \dots p \quad (2.4)$$

onde $\bar{x}_i$ e $\bar{x}_j$ são, respectivamente, as médias calculadas para cada variável.

A estimativa da matriz de covariâncias pode ser calculada diretamente pela expressão vetorial:

$$\hat{\Sigma} = \frac{1}{N-1} \sum_{t=1}^N (\mathbf{x}(t) - \bar{\mathbf{x}})^T (\mathbf{x}(t) - \bar{\mathbf{x}}) \quad (2.5)$$

onde $\bar{\mathbf{x}} \in \mathcal{R}^p$ é o vetor de médias das variáveis em que cada componente é a média de cada variável definida pela equação (2.1).

Assim como a variância, a matriz de covariâncias da amostra converge para a matriz de covariâncias do processo quando o número de registros da amostra aumenta. A matriz de covariâncias é simétrica ($\hat{\sigma}_{ij}^2 = \hat{\sigma}_{ji}^2$) e definida positiva;* os elementos na diagonal da matriz de covariâncias correspondem às variâncias das variáveis correspondentes, isto é, $\hat{\sigma}_{ii}^2 = \hat{\sigma}_i^2$, onde $\hat{\sigma}_i^2$ é calculado pela equação (2.2).

A matriz de covariâncias Σ e o vetor de médias das variáveis μ são os parâmetros que determinam a função de **distribuição normal multivariada** do processo:

$$f(\mathbf{x}) = \frac{1}{\sqrt{2\pi|\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)\Sigma^{-1}(\mathbf{x} - \mu)^T\right) \quad (2.6)$$

onde $|\Sigma|$ é o determinante da matriz de covariâncias. Na prática, a distribuição normal é calculada pela equação (2.6) com a matriz de covariâncias estimadas $\hat{\Sigma}$ e o vetor de médias estimadas $\bar{\mathbf{x}}$.

A matriz de covariâncias indica o grau de acoplamento de cada par de variáveis. Caso as variáveis sejam estatisticamente independentes, ou seja, a variação de uma variável não influencia na variação da outra, os termos fora da diagonal da matriz de covariâncias são nulos, isto é, $\hat{\sigma}_{ij}^2 = 0$, para $i \neq j$. Quando há dependência linear entre as variáveis, os elementos $\hat{\sigma}_{ij}^2 > 0$ indicam a dependência linear positiva, quando as variações são no mesmo sentido, e $\hat{\sigma}_{ij}^2 < 0$ indicam dependência linear negativa, quando as variações ocorrem em sentidos opostos.

A matriz de covariâncias depende da escala das variáveis e, em geral, é mais apropriado utilizar valores padronizados. O coeficiente de correlação de Pearson, ou simplesmente

* Uma matriz $\mathbf{A}$ é definida positiva se $\mathbf{x}\mathbf{A}\mathbf{x}^T > 0, \forall \mathbf{x}$.

coeficiente de correlação, é uma medida padronizada da covariância de duas variáveis e é calculado como:

$$\text{Corr}(x_i, x_j) \approx \hat{\rho}_{ij} = \frac{\text{Cov}(x_i, x_j)}{\sqrt{\text{Var}(x_i)\text{Var}(x_j)}} \quad (2.7)$$

O coeficiente de correlação varia no intervalo $[-1, 1]$ e quanto mais o valor absoluto é próximo de 1, mais as variáveis são correlacionadas, positiva ou negativamente. A **matriz de correlação** é a matriz cujos elementos são os coeficientes de correlação $\hat{\rho}_{ij}$.

Os gráficos de projeção, mostrados na Figura 2.8, ilustram o aspecto de variáveis independentes com correlação positiva ou negativa.

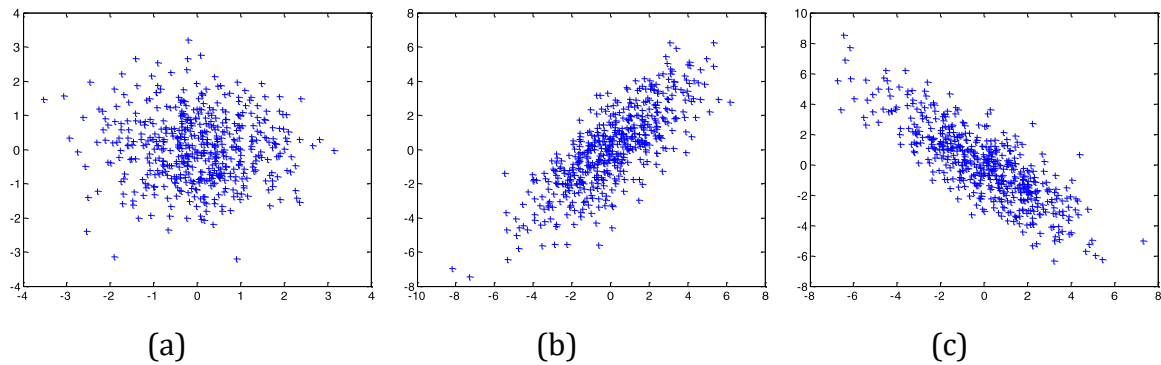


Figura 2.8: Gráfico de projeção das variáveis: (a) $\hat{\rho}_{ij} = 0$; (b) $\hat{\rho}_{ij} > 0$; (c) $\hat{\rho}_{ij} < 0$.

O coeficiente de correlação representa uma medida da redundância entre duas variáveis. Variáveis totalmente correlacionadas (positiva ou negativamente) representam essencialmente a mesma informação e uma delas poderia ser descartada. Entretanto, exceto no caso de correlação total, não é possível decidir sobre qual variável remover apenas a partir da informação de correlação. A análise de componentes principais (ACP), apresentada na seção 2.4.1, tem como objetivo projetar o conjunto de dados num espaço de menor dimensão de variáveis independentes.

É importante observar que o coeficiente de correlação mede apenas a dependência linear, portanto um resultado $\hat{\rho}_{ij} = 0$ não representa que não há relação entre as variáveis, apenas que a relação não é linear.

A correlação entre as variáveis também pode ser visualizada através dos **gráficos de projeção** das variáveis nos planos bidimensionais formados pela combinação de todas

as variáveis duas a duas. Esse gráfico, também chamado de matriz *scatter plot*, apresenta os histogramas na diagonal.

O gráfico de projeção do conjunto de dados Íris é apresentado na Figura 2.9, na qual as cores indicam as classes. O gráfico apresenta uma informação redundante uma vez que os gráficos da diagonal superior são os mesmos da diagonal inferior com os eixos trocados. O gráfico de projeção também confirma que as classes estão mais separadas no plano formado pelas variáveis x_3 e x_4 .

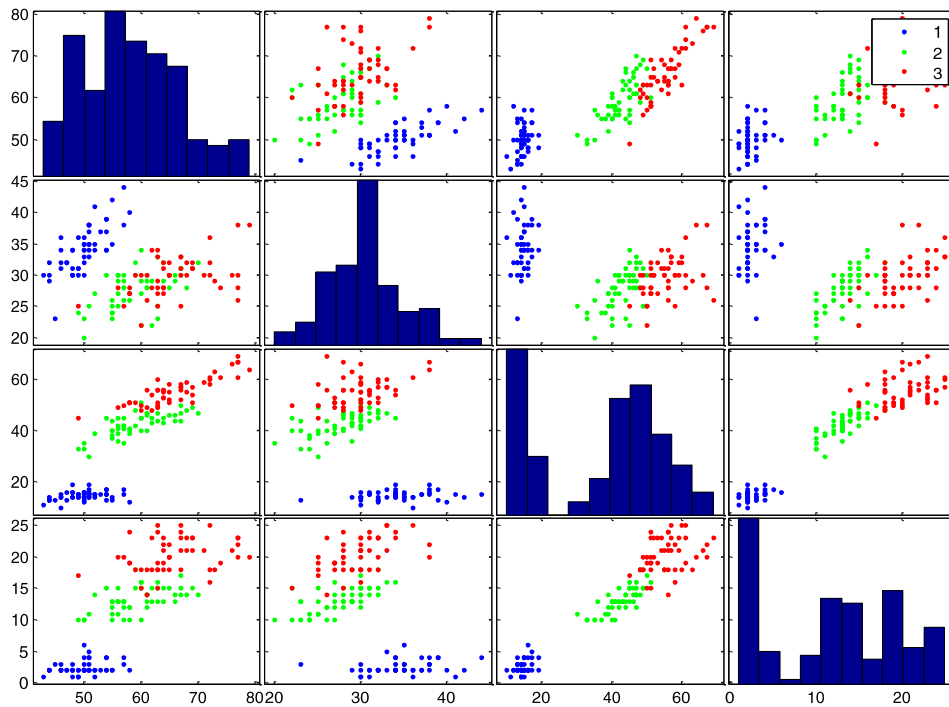


Figura 2.9: Gráfico de projeção do conjunto de dados Íris.

Os gráficos de projeção são uma importante ferramenta de visualização, mas sua utilização requer precauções. Inicialmente, é importante observar que cada gráfico apresenta uma projeção do espaço das variáveis num plano bidimensional. Dessa forma, os dados, principalmente em problemas de classificação, aparecem mais misturados do que são na realidade. Em problemas com um número muito grande de variáveis, a visualização do gráfico de projeção torna-se muito difícil. Em problemas desse tipo, é mais indicado utilizar uma visualização qualitativa da matriz de correlação, como mostra a Figura 2.10 para o caso do problema Ionosphere,⁹⁰ que tem $p = 32$ variáveis.

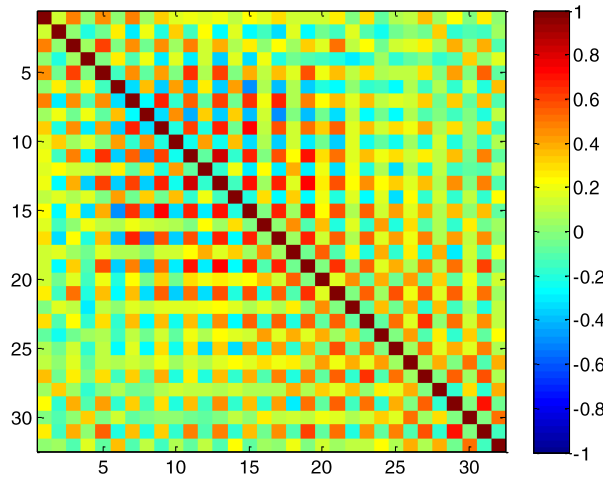


Figura 2.10: Visualização qualitativa da matriz de correlação no problema Ionosphere.

2.2.3 Matriz de distâncias e similaridades

A análise multivariada previamente apresentada é realizada principalmente a partir do cálculo da matriz de correlação entre as variáveis. A correlação entre duas variáveis pode ser entendida como uma medida de similaridade entre elas. A avaliação da similaridade (ou distância) entre os registros pode revelar características importantes do conjunto de dados, como a presença de *outliers*, por exemplo. As medidas de distância também são utilizadas para preenchimento de valores ausentes (cf. seção 2.3.3) e pela maioria dos algoritmos de análise de agrupamentos. Nesta seção, apresentamos uma introdução às métricas de distância no caso de variáveis numéricas.

Uma função de distância é uma métrica $dist: \mathcal{R}^p \times \mathcal{R}^p \rightarrow \mathcal{R}$ se satisfaz as seguintes propriedades [516]:

a) **não negatividade:**

$$dist(\mathbf{v}, \mathbf{u}) \geq 0, \forall \mathbf{v}, \mathbf{u} \in \mathcal{R}^p,$$

$$dist(\mathbf{v}, \mathbf{u}) = 0, \forall \mathbf{v} = \mathbf{u};$$

b) **simetria:**

$$dist(\mathbf{v}, \mathbf{u}) = dist(\mathbf{u}, \mathbf{v}), \forall \mathbf{v}, \mathbf{u} \in \mathcal{R}^p;$$

c) **desigualdade triangular:**

$$dist(\mathbf{v}, \boldsymbol{\xi}) \leq dist(\mathbf{v}, \mathbf{u}) + dist(\mathbf{u}, \boldsymbol{\xi}), \forall \mathbf{v}, \mathbf{u}, \boldsymbol{\xi} \in \mathcal{R}^p.$$

As funções de similaridade também são utilizadas em algoritmos de agrupamento e análise de *outliers*. Uma função de similaridade $sim: \mathcal{R}^p \times \mathcal{R}^p \rightarrow [0,1]$ satisfaz a propriedade de não negatividade, sendo que $sim(\mathbf{v}, \mathbf{u}) = 1$, apenas se $\mathbf{v} = \mathbf{u}$, e a propriedade

de simetria. Entretanto nem todas as funções de similaridade satisfazem a propriedade de desigualdade triangular, não podendo ser consideradas como métricas.

Diversas funções distância são utilizadas em ciência de dados. As mais importantes são:

- distância **euclidiana** (ou norma L^2): é a extensão da fórmula clássica da geometria para p dimensões:

$$dist_E(\mathbf{v}, \mathbf{u}) = \|\mathbf{v} - \mathbf{u}\| = \sqrt{\sum_{i=1}^p (v_i - u_i)^2} \quad (2.8)$$

- distância de **Mahalanobis**: é a distância geométrica ponderada pelo inverso da matriz de covariâncias estimada no conjunto de dados:

$$dist_{\Sigma}(\mathbf{v}, \mathbf{u}) = \sqrt{(\mathbf{v} - \mathbf{u})^T \hat{\Sigma}^{-1} (\mathbf{v} - \mathbf{u})} \quad (2.9)$$

- distância de **Minkowski**: é uma generalização da distância euclidiana:

$$dist_r(\mathbf{v}, \mathbf{u}) = \left(\sum_{i=1}^p |v_i - u_i|^r \right)^{\frac{1}{r}} \quad (2.10)$$

- distância de **Manhattan** (norma L^1): é um caso particular da distância de Minkowski com $r = 1$:

$$dist_1(\mathbf{v}, \mathbf{u}) = \sum_{i=1}^p |v_i - u_i| = \sum_{i=1}^p \text{abs}(v_i - u_i) \quad (2.11)$$

- distância de **Chebyshev** (norma L^∞): é a distância de Minkowski com $r \rightarrow \infty$:

$$dist_\infty(\mathbf{v}, \mathbf{u}) = \lim_{r \rightarrow \infty} \left(\sum_{i=1}^p |v_i - u_i|^r \right)^{\frac{1}{r}} = \max_i (|v_i - u_i|) \quad (2.12)$$

As funções de similaridade mais comumente empregadas são:

- similaridade do **cosseno**:

$$sim_c(\mathbf{v}, \mathbf{u}) = \frac{\langle \mathbf{v}, \mathbf{u} \rangle}{\|\mathbf{v}\| \|\mathbf{u}\|} \quad (2.13)$$

onde $\langle \mathbf{v}, \mathbf{v} \rangle = \mathbf{v}\mathbf{v}^T = \sum_{i=1}^p v_i v_i$ é o produto escalar e $\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}$ é a norma quadrática (L^2) de $\mathbf{v}$;

- similaridade **gaussiana**:

$$sim_G(\mathbf{v}, \mathbf{v}) = \exp\left(-\frac{\|\mathbf{v} - \mathbf{v}\|^2}{2\sigma^2}\right) \quad (2.14)$$

onde σ é um parâmetro de dispersão.

Para cada função de distância (ou similaridade) é possível calcular uma matriz de distâncias (ou similaridades) do conjunto de dados. Em geral, apenas as variáveis de entrada são utilizadas para detecção de *outliers*, preenchimento de valores ausentes ou análise de agrupamento. Dessa forma, a matriz de distâncias calculada sobre o conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$ é a matriz $\mathbf{D} \in \mathcal{R}^{N \times N}$, na qual cada elemento $d_{ij} = dist(\mathbf{x}(i), \mathbf{x}(j))$. Analogamente, a matriz de similaridades $\mathbf{A} \in [0,1]^{N \times N}$ e cada elemento $a_{ij} = sim(\mathbf{x}(i), \mathbf{x}(j))$.

Tendo em vista que a matriz de distâncias (ou similaridades) é simétrica, o cálculo dos elementos da matriz triangular envolve $N(N-1)/2$ operações. A complexidade computacional $\mathcal{O}(N^2)$ pode ser um problema no caso de N muito grande.

A visualização da matriz de similaridades (ou distâncias) permite uma análise de todo o conjunto de dados, independentemente do número de variáveis. O gráfico, chamado de **data image**, é uma matriz da dimensão do número de registros em que cada elemento é um valor numa escala de cores que representa a distância ou similaridade entre os registros correspondentes. O *data image* é empregado para detecção de *outliers* [365] e para avaliação da qualidade de agrupamentos [239].

A Figura 2.11 apresenta um exemplo de visualização do conjunto de dados Íris utilizando a distância euclidiana e a similaridade gaussiana. O conjunto de dados contém 50 registros de cada classe e todos os registros são ordenados pela classe. Observa-se na Figura 2.11 a presença de três grupos de registros fortemente similares entre si, representados pelos três quadrados azuis (vermelhos) na Figura 2.11a (Figura 2.11b), representando valores baixos (altos) de distâncias (similaridades). Observando o gráfico de projeção da Figura 2.9, é possível verificar que, no problema Íris, a classe 1 (*Iris setosa*) é separada das demais e que a classe 2 (*Iris versicolor*) e a classe 3 (*Iris virginica*)

são mais próximas entre si. O mesmo efeito pode ser verificado nos gráficos da Figura 2.11, em que as classes 2 e 3 formam praticamente um grupo de dados.

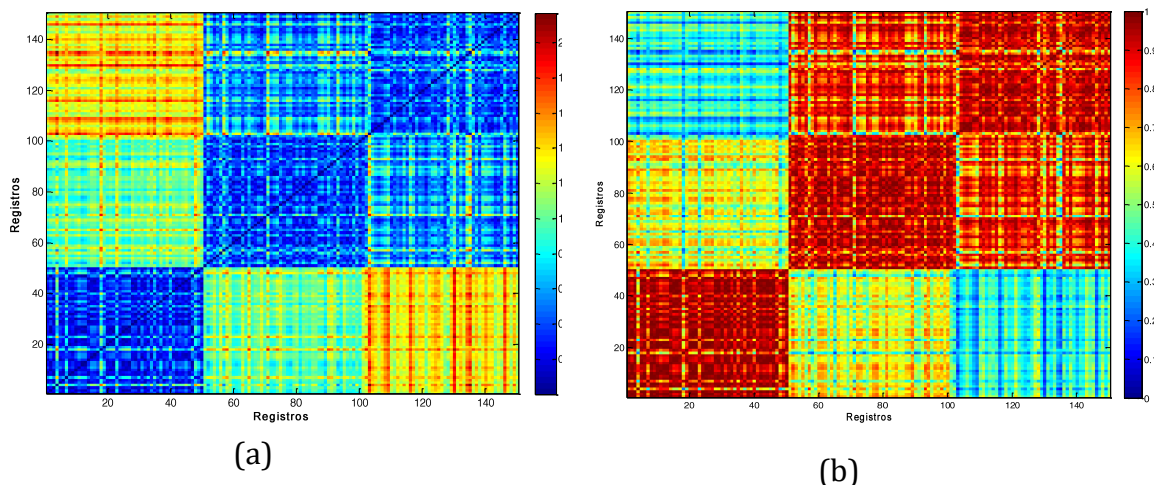


Figura 2.11: *Data image*. (a) Distância euclidiana; (b) similaridade gaussiana.

Há outros gráficos que permitem a análise de um conjunto de dados. Na verdade, a visualização de dados é uma área de pesquisa em grande atividade [490].

Após a caracterização estatística, o conjunto de dados deve ser pré-processado para formar o conjunto de treinamento para o ajuste dos modelos, como descrito a seguir.

2.3 Limpeza de Dados

O processo de mineração de dados (cf. Figura 1.1) começa com a seleção do conjunto de dados brutos da base de dados. O conjunto de dados brutos não é adequado para o ajuste dos modelos pois pode conter variáveis irrelevantes, valores espúrios (*outliers*) ou ausentes, variáveis redundantes ou algum outro tipo de problema específico.

Uma análise preliminar do conjunto de dados pode identificar se alguma variável de entrada não interfere no problema por ter todos os valores idênticos ($\hat{\sigma}_i = 0$). Nesse caso, essa variável (na verdade, uma constante) deve ser eliminada. Outro problema comum é a presença de uma variável que simplesmente identifica os registros, o que também não contribui para o modelo, de forma que deve ser descartada.

Nesta seção, apresentamos alguns procedimentos genéricos que são empregados como prática padrão em ciência de dados. Processamentos mais específicas podem ser necessárias dependendo da aplicação.

2.3.1 Padronização de variáveis

O conjunto de dados brutos frequentemente contém variáveis expressas em escalas diferentes, o que pode interferir nos resultados dos algoritmos. A padronização é uma transformação cujo objetivo é produzir variáveis de mesma escala.

Há diversas técnicas de padronização. No caso mais simples, os valores da variável são escalados no intervalo $[0,1]$, pelos valores de máximo e de mínimo:

$$\hat{x}_i(t) = \frac{x_i(t) - \min_{t=1\dots N}(x_i(t))}{\max_{t=1\dots N}(x_i(t)) - \min_{t=1\dots N}(x_i(t))} \quad (2.15)$$

Esse tipo de padronização, no entanto, não deve ser utilizado na presença de *outliers*, pois o máximo ou mínimo de alguma variável pode estar muito distante dos demais valores, gerando uma distorção na escala de valores padronizados. Na presença de *outliers*, o máximo e mínimo podem ser substituídos pelos valores do 95º percentil e do 5º percentil, respectivamente, ou por outros valores mais adequados para representar os limites superior e inferior da escala.

Há outros tipos de padronização que são robustas a *outliers*. A padronização pela estimativa do *z-score* é muito utilizada e calculada como:

$$\hat{x}_i(t) = \frac{x_i(t) - \bar{x}_i}{\hat{\sigma}_i} \quad (2.16)$$

Nesse caso, a variável padronizada tem média nula e desvio padrão unitário. Em alguns casos, partindo-se da hipótese de que a distribuição da variável pode ser modelada pela distribuição normal (2.3), aplica-se $3\hat{\sigma}_i$ no denominador. Com isso, caso a distribuição seja realmente normal, 99,7% dos valores da variável padronizada estarão dentro do intervalo $[-1, +1]$. Essa padronização é robusta a *outliers*, mas em distribuições altamente assimétricas pode gerar uma distorção de escala, pois utiliza as estatísticas de média e desvio padrão, que são mais apropriadas para distribuições simétricas.

Em distribuições assimétricas, como a da Figura 2.5a, é comum a aplicação da transformação pelo logaritmo para obtenção de uma distribuição simétrica, que é mais adequada para o desenvolvimento de modelos. A transformação pelo logaritmo não pode ser aplicada se a variável contém valores negativos ou nulos. Nesse caso, basta adicionar uma constante maior que o valor mínimo da variável a todos os valores.

As equações (2.15) e (2.16) são lineares e podem ser invertidas se necessário. Há fórmulas não lineares de padronização que não podem ser invertidas.

2.3.2 Detecção de *outliers*

A detecção e remoção de valores espúrios (*outliers*) é um procedimento importante uma vez que esses valores podem prejudicar o desempenho dos modelos. Entretanto, em problemas de detecção de anomalias, como fraudes [54], por exemplo, a descoberta de *outliers* é a solução do problema. Dessa forma, o analista deve avaliar a aplicação e verificar quando os *outliers* são realmente indesejados.

Há diversos métodos para a detecção de *outliers* e a literatura no assunto é extensa [54][270]. Nesta seção, são exploradas duas abordagens principais: a abordagem baseada em distribuições e a baseada em distâncias. A abordagem baseada em distribuições é a abordagem clássica da estatística, que requer uma hipótese sobre a função de distribuição de probabilidades para a amostra para a determinação dos *outliers*. Na abordagem baseada em distâncias, utiliza-se uma função de distância ou similaridade para determinar os registros mais afastados do conjunto de dados, sem considerações sobre a distribuição que gerou a amostra. A abordagem baseada em distribuição é geralmente empregada sobre cada variável independentemente. A abordagem baseada em distâncias considera todas variáveis, mas envolve o custo computacional do cálculo da matriz de distâncias.

A abordagem mais simples para detecção de *outliers* é estimar uma distribuição normal para cada variável. A validade da hipótese de distribuição normal pode ser verificada a partir de gráficos de distribuição normal (cf. Figura 2.7). Caso a hipótese seja confirmada, sabe-se que, numa distribuição normal, 99,7% dos registros encontram-se dentro do intervalo $[\mu - 3\sigma, \mu + 3\sigma]$. Os *outliers* são identificados como os registros contendo valores fora desse intervalo.

No caso de a distribuição das variáveis ser diferente da distribuição normal, pode-se utilizar a ferramenta gráfica conhecida como **box-plot** para a detecção de possíveis *outliers* a partir dos valores de percentil. O gráfico *box-plot* é formado por caixas, como mostra a Figura 2.12, que são calculadas para cada variável. A caixa representa a chamada distância interquartil (IRQ), que é a diferença entre o valor do 75º percentil ou terceiro quartil ($Q3$) e o valor do 25º percentil ou primeiro quartil ($Q1$). A distância

interquartil representa o intervalo de valores da variável em que se encontram 50% dos registros. A linha vermelha no interior da caixa representa a mediana (50º percentil) e permite identificar distribuições assimétricas caso não esteja localizada no centro da caixa. A linha à direita da caixa representa o intervalo $[Q3, Q3 + 1,5IRQ]$ e a linha à esquerda, o intervalo $[Q1 - 1,5IRQ, Q1]$. Pode acontecer que o maior (ou menor) valor observado seja inferior (ou superior) ao limite dos intervalos e as linhas à esquerda e à direita fiquem desiguais. Valores fora desses intervalos são assinalados em vermelho e podem ser considerados *outliers* para a variável correspondente. Dependendo do *software*, as caixas podem aparecer na vertical ou na horizontal e ainda com todas as variáveis no mesmo gráfico ou um gráfico para cada variável.

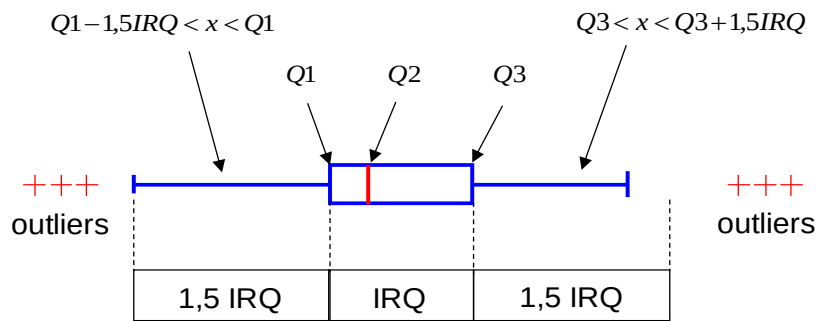


Figura 2.12: Valores para a construção do *box-plot*.

O *box-plot* depende da escala das variáveis e se o problema contém variáveis em escalas diferentes, é preferível sua avaliação após a padronização. A Figura 2.13 apresenta o *box-plot* do problema Íris antes e após a padronização das variáveis, em pode-se observar o efeito da padronização sobre a dispersão das variáveis.

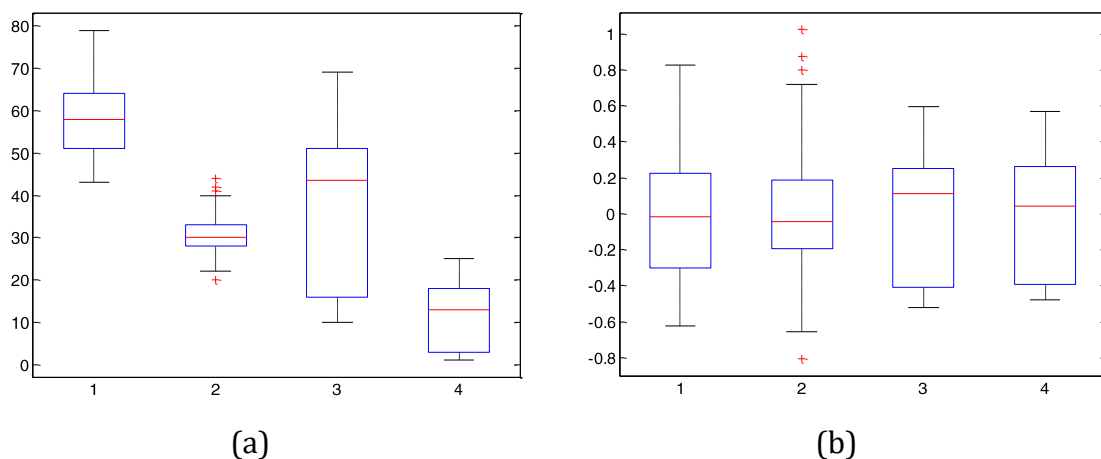


Figura 2.13: *Box-plot* para o problema Íris: (a) sem padronização; (b) com padronização.

O *box-plot* permite identificar apenas registros suspeitos de serem *outliers*, uma vez que a análise é feita para cada variável independentemente. Não se deve excluir os registros assinalados pelo *box-plot* como *outliers* em uma variável sem uma avaliação prévia dos valores nas outras variáveis.

A visualização do *box-plot* pode auxiliar na caracterização estatística do conjunto de dados, mas não é muito adequada para a detecção de *outliers*. Os métodos baseados em distâncias são mais efetivos, pois consideram todas as variáveis simultaneamente.

Um método simples e bastante eficaz identifica os *outliers* a partir da média ordenada $\hat{m}_i$ das distâncias de um registro $\mathbf{x}(i)$ para todos os outros, calculada a partir da matriz de distâncias $\mathbf{D}$. Os *outliers* são identificados por valores $\hat{m}_i > \delta$, onde o parâmetro δ é identificado por uma análise visual das distâncias médias ordenadas. A avaliação do limite para o corte pode ser evitada especificando-se uma porcentagem P_{out} de registros mais afastados para serem retirados, que é mais fácil de escolher. O procedimento é descrito no Algoritmo 2.1.

A detecção de *outliers* também pode ser realizada com a matriz de similaridades, podendo resultar diferente. Nesse caso, os valores médios m_i são calculados pela similaridade média, a ordenação é feita em ordem decrescente e o critério de detecção é $\hat{m}_i < \delta$ ou os P_{out} registros menos similares.

Algoritmo 2.1: Retirada de *outliers*

Entrada:	Matriz de distâncias $\mathbf{D}$, porcentagem P_{out} de <i>outliers</i> a ser retirada
Saída:	Índice de registros <i>outliers</i>
01	Início
02	Calcule das distâncias médias $m_i = \frac{1}{N} \sum_{j=1}^N d_{ij}$
03	Ordene as distâncias médias $\hat{\mathbf{m}} = \text{sort}(\mathbf{m})$, em ordem crescente
04	Retorna o índice dos P_{out} registros correspondentes aos maiores valores de $\hat{m}_i$
05	Fim

O procedimento de detecção de *outliers* pela distância é apresentado a seguir para o conjunto de dados *Pollution*.⁹¹ Esse é um problema clássico de regressão linear com 15

variáveis de entrada, uma variável de saída e apenas 60 registros. A presença de *outliers* no conjunto de dados é verificada na *data image* da Figura 2.14a, em que um registro (mais precisamente o registro $t = 8$) é bem diferente dos demais, uma vez que a distância desse registro para os demais apresenta uma tonalidade vermelha forte.

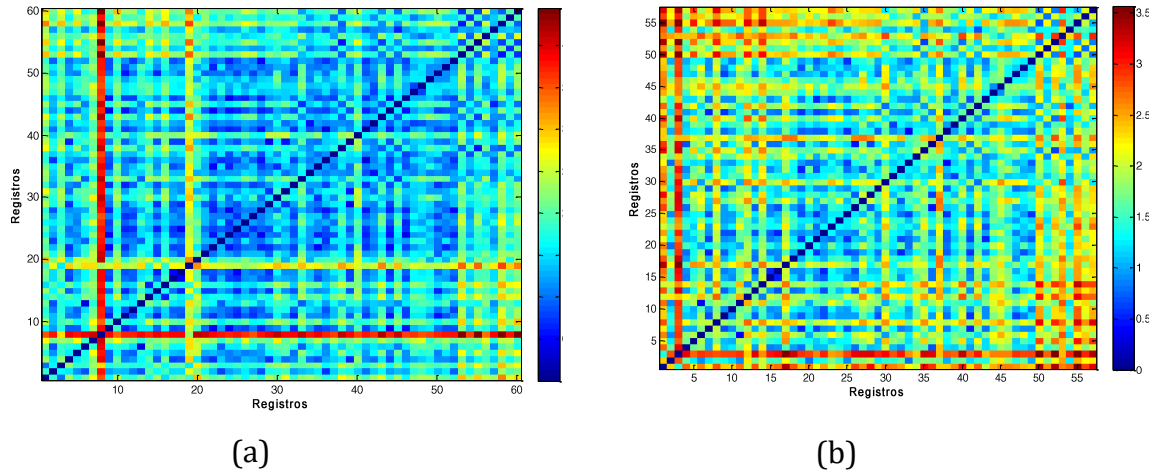


Figura 2.14: Matriz de distâncias euclidianas para o problema Pollution: (a) antes da retirada de outliers; (b) após a retirada de outliers.

A média ordenada das distâncias é mostrada na Figura 2.15a, na qual se verifica que quatro registros são identificados como *outliers*. A Figura 2.15b mostra os valores relativos de similaridade média calculados com a similaridade gaussiana. Após a retirada de *outliers* (pela distância euclidiana), a matriz de distâncias apresenta uma configuração mais homogênea, como mostrado no *data image* da Figura 2.14b.

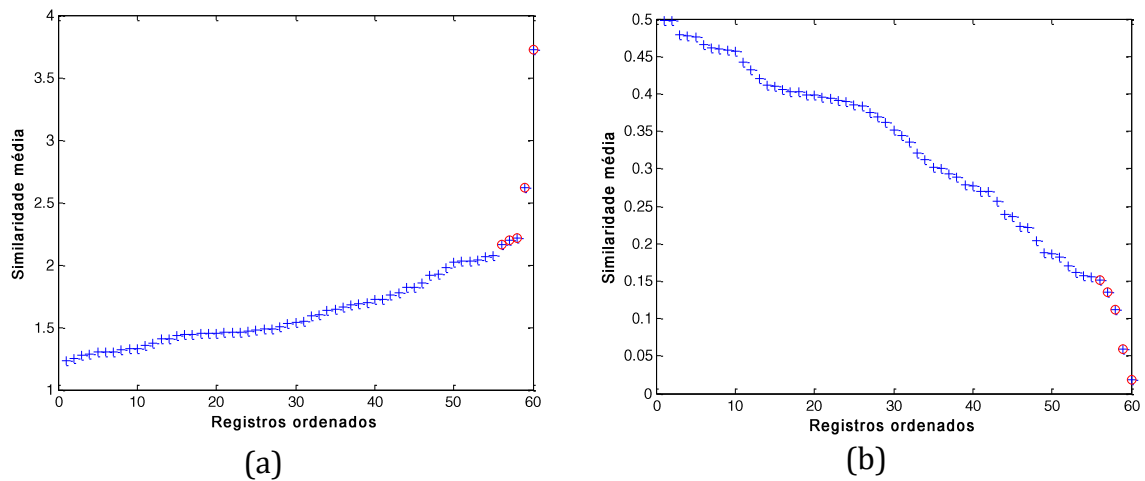


Figura 2.15: Detecção de *outliers*: (a) médias de distâncias euclidianas em ordem crescente; (b) médias de similaridade gaussiana em ordem decrescente.

2.3.3 Valores ausentes

Um valor ausente ocorre em alguma variável quando o valor não pôde ser observado ou foi observado, mas não armazenado. O tratamento de valores ausentes é um assunto importante em ciência de dados, uma vez que conjuntos de dados reais frequentemente apresentam valores ausentes [232]. É importante ressaltar que valor ausente é diferente de valor zero, em que o valor observado é conhecido e igual a zero. A codificação dos valores ausentes na base de dados deve ser feita de forma que o código utilizado para valor ausente não seja confundido com um valor possível da variável. Em muitos conjuntos de dados, os valores ausentes são codificados pelo caractere ‘?’.

Há duas estratégias para o tratamento de valores ausentes: a eliminação do registro contendo valor(es) ausente(s) ou o preenchimento (ou imputação) dos dados ausentes. Na primeira estratégia, deve-se considerar o impacto da eliminação dos registros contendo valores ausentes. Para isso, é importante que seja feito um levantamento do número de valores ausentes em cada variável. Pode ocorrer que a maioria dos valores ausentes esteja em apenas uma variável e, nesse caso, a eliminação da variável pode ser uma alternativa.

A visualização de valores ausentes num conjunto de dados pode revelar alguma falha na aquisição de dados quando os valores ausentes apresentam algum padrão característico. O gráfico da Figura 2.16 apresenta a posição dos valores ausentes das 22 variáveis de entrada no conjunto de dados *Water Treatment Plant*.⁹² O eixo horizontal representa os registros, o eixo vertical representa as variáveis e os valores ausentes são representados em preto. O número de valores ausentes em cada variável é apresentado no gráfico da Figura 2.17. O conjunto de dados contém 527 registros, dos quais 115 (aproximadamente 22%) contêm algum valor ausente.

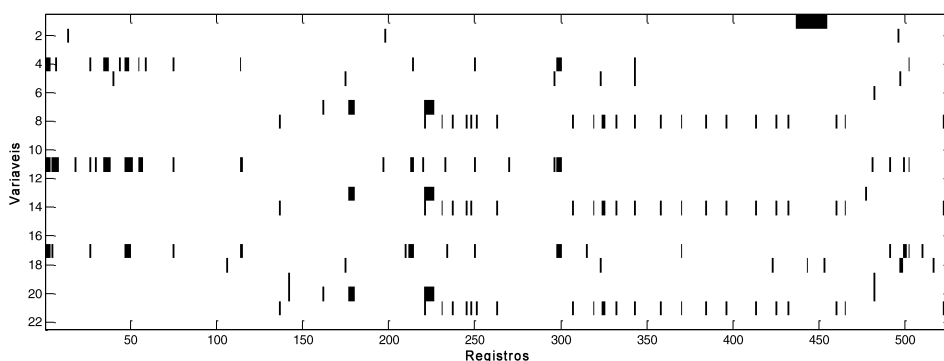


Figura 2.16: Visualização de valores ausentes.

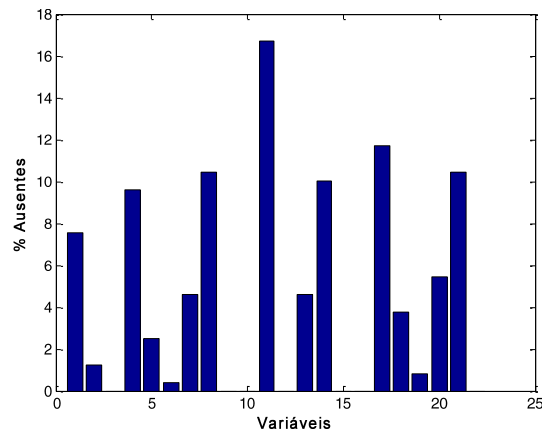


Figura 2.17: Valores ausentes por variável.

No conjunto de dados *Water Treatment Plant*, a redução de 22% dos registros não inviabiliza completamente a análise e pode-se optar por retirar os registros com valores ausentes ou realizar a imputação de dados. Entretanto, há problemas em que não há opção à imputação de dados.

A dificuldade da imputação de dados é a introdução de uma informação nova que não representa necessariamente a realidade. Introduce-se um erro que pode ter um impacto significativo no desempenho do modelo. O assunto é complexo e há diversas técnicas de imputação de dados ausentes [525][498].

A técnica mais ingênua para a imputação de dados é a substituição do valor ausente pela média. Apesar de simples, essa solução não apresenta bons resultados, principalmente se o número de valores ausentes for grande. O mais recomendado é utilizar algum método preditivo a partir dos valores das demais variáveis. Entretanto, o desenvolvimento de modelos muito elaborados para a imputação de dados não é muito interessante sendo mais indicado utilizar uma solução simples.

Uma solução simples e robusta para a imputação de dados é utilizar o método dos k -vizinhos mais próximos, apresentado no Algoritmo 2.2. O algoritmo utiliza como base para a imputação de dados o conjunto de registros sem valores ausentes definido no passo 02. A estimativa dos valores ausentes é calculada pela média dos k -vizinhos mais próximos do conjunto de dados completos. A média dos vizinhos, calculada no passo 08 do algoritmo, pode ser uma média simples ou ponderada por valores proporcionais ao inverso da distância aos vizinhos. O algoritmo depende da escolha do número k de vizinhos. Recomenda-se a utilização de $k \geq 3$ vizinhos.

Algoritmo 2.2: Imputação de dados pelo método dos k -vizinhos mais próximos

Entrada:	Variáveis de entrada $\{\mathbf{x}(t), t = 1 \dots N\}$ contendo valores ausentes, número de vizinhos k
Saída:	Variáveis de entrada sem valores ausentes
01	Início
02	Defina $\{\mathbf{z}(t), t = 1 \dots N'\}$ o conjunto de dados completos
03	Para cada registro i contendo algum valor ausente
04	Defina $\mathbf{x}^*(i) \subset \mathbf{x}(i)$ o vetor cujas componentes não contêm valor ausente no registro i
05	Defina $\mathbf{z}^*(j) \subset \mathbf{z}(j)$ o vetor de registros completos com as mesmas componentes de $\mathbf{x}^*(i)$
06	Calcule as distâncias $d_{ij} = \text{dist}(\mathbf{x}^*(i), \mathbf{z}^*(j)), j = 1 \dots N'$
07	Ordene as distâncias $\hat{d}_{ij} = \text{sort}(d_{ij})$
08	Calcule os valores ausentes $\mathbf{x}(i)$ pela média das componentes dos k -vizinhos mais próximos
09	Fim Para
10	Retorna $\{\mathbf{x}(t), t = 1 \dots N\}$ sem valores ausentes;
11	Fim

A aplicação do Algoritmo 2.2 permite obter um conjunto de dados completo. Entretanto, a imputação de dados altera ligeiramente as características dos dados. No exemplo do conjunto de dados *Water Treatment Plant*, a Figura 2.18 apresenta a comparação dos histogramas da variável x_{11} , que contém o maior número de valores ausentes (cf. Figura 2.17), com imputação de dados e com a retirada de valores ausentes. A diferença entre os dois histogramas é pequena.

O método dos k -vizinhos mais próximos pode ser empregado em problemas preditivos, como classificação e regressão e tem sido muito utilizado em sistemas de recomendação chamados de filtragem colaborativa.

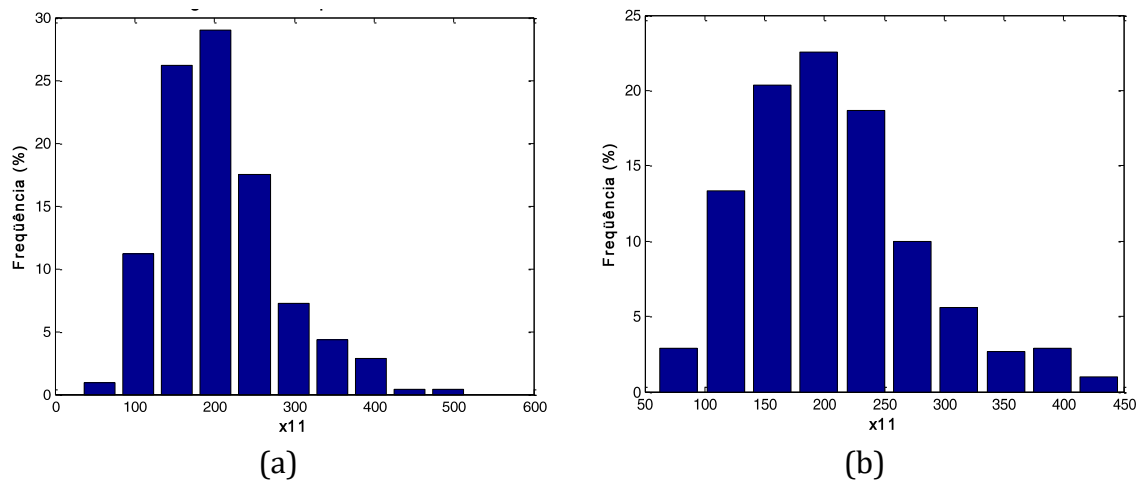


Figura 2.18: Histogramas da variável x_{11} do problema Waste Treatment Plant: (a) com imputação de dados; (b) com retirada de valores ausentes.

2.4 Transformações Lineares

Uma **transformação linear** [206] $\mathbf{T}: \mathcal{R}^n \rightarrow \mathcal{R}^m$ é definida por uma matriz de dimensão $n \times m$. A transformação $\mathbf{T}$, aplicada ao vetor $\mathbf{x} \in \mathcal{R}^n$, obtém como resultado (imagem) o vetor $\mathbf{z} \in \mathcal{R}^m$, onde $\mathbf{z} = \mathbf{x}\mathbf{T}$.

A utilização de matrizes para o cálculo de transformações lineares permite que uma transformação seja modificada por operações sobre a matriz. Por exemplo, se a matriz $\mathbf{T}$ admitir inversa, a matriz $\mathbf{T}^{-1}$ realiza a transformação inversa $\mathbf{x} = \mathbf{z}\mathbf{T}^{-1}$. A Figura 2.19 apresenta um exemplo de transformação linear e sua inversa.

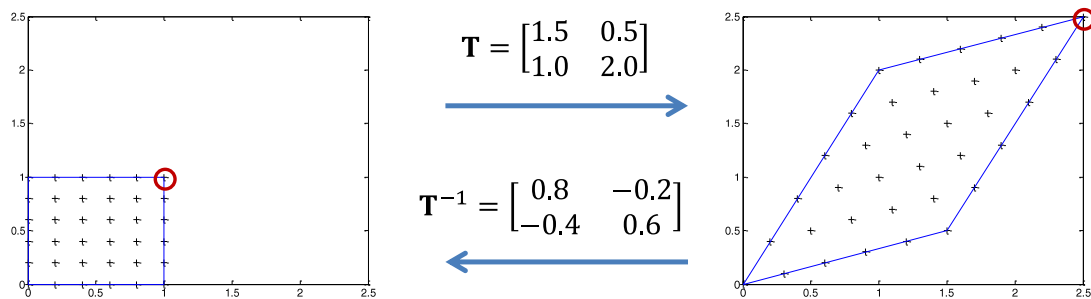


Figura 2.19: Exemplo de transformação linear.

Uma **transformação linear ortogonal** $\mathbf{U}: \mathcal{R}^n \rightarrow \mathcal{R}^n$ é um tipo particular de transformação linear que realiza uma rotação do espaço de entrada. A transformação ortogonal é realizada por uma matriz ortogonal, cuja inversa é sua transposta: $\mathbf{U}^{-1} = \mathbf{U}^T$. Adici-

onalmente, os vetores formados pelas colunas de uma de matriz ortogonal são ortogonais entre si, ou seja, o produto escalar é igual a zero. A Figura 2.20 apresenta um exemplo de transformação ortogonal e sua inversa.

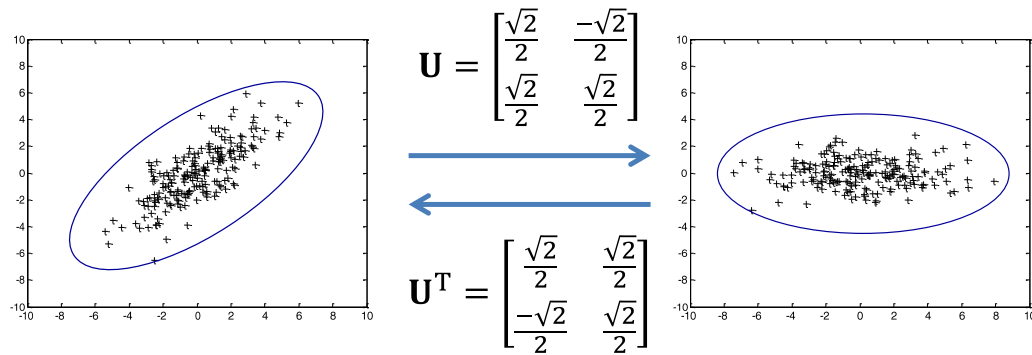


Figura 2.20: Exemplo de transformação linear ortogonal.

As transformações lineares têm inúmeras aplicações em engenharia. Em ciência de dados, uma transformação linear é utilizada para realizar uma mudança de coordenadas das variáveis de entrada, em geral de menor dimensionalidade, e com características mais apropriadas ao problema. As variáveis transformadas são combinações lineares das variáveis de entrada e, em geral, não podem ser interpretadas. Nesta seção, são apresentadas as principais técnicas de transformações lineares.

2.4.1 Análise de componentes principais

O objetivo da análise de componentes principais (ACP), também conhecida como transformação de Karhunen-Loève [521], é encontrar uma transformação linear que elimina a correlação entre as variáveis de entrada [366][449]. A ACP permite a redução da dimensionalidade do conjunto de dados pela eliminação da redundância provocada pela correlação entre as variáveis. A ACP é realizada por uma matriz de transformação linear ortogonal que, aplicada ao conjunto de dados, gera um novo conjunto de dados de variáveis transformadas não correlacionadas.

A formulação da ACP considera que todas as variáveis de entrada são numéricas, com distribuição normal. O cálculo da matriz de transformação deve ser realizado sobre dados centrados na origem do sistema de coordenadas, o que pode ser obtido pela padronização pelos *z-scores* (2.16). Nesse caso, a matriz de covariâncias estimada é igual à matriz de correlação e é calculada como $\hat{\Sigma} = \frac{1}{N-1} \mathbf{X}^T \mathbf{X}$, onde $\mathbf{X}$ é a matriz de dados de dimensão $N \times p$, contendo os registros das variáveis de entrada.

No conjunto de dados das variáveis transformados $\mathcal{Z} = \{\mathbf{z}(t), t = 1 \dots N\}$, cada registro é calculado como uma combinação linear dos registros correspondentes das variáveis originais, como $\mathbf{z}(t) = \mathbf{x}(t)\mathbf{P}$. A matriz de dados transformados é escrita como:

$$\mathbf{Z} = \mathbf{XP} \quad (2.17)$$

A matriz de transformação $\mathbf{P} \in \mathcal{R}^{p \times p}$ é uma matriz ortogonal calculada pela diagonalização da matriz de covariâncias das variáveis originais. Como resultado, a matriz de covariâncias das variáveis transformadas, $\mathbf{\Lambda} = \frac{1}{N-1}\mathbf{Z}^T\mathbf{Z}$, é diagonal. Escrevendo $\mathbf{\Lambda}$ com a matriz $\mathbf{Z}$ definida em (2.17), temos:*

$$\mathbf{\Lambda} = \frac{1}{N-1}\mathbf{Z}^T\mathbf{Z} = \frac{1}{N-1}(\mathbf{XP})^T(\mathbf{XP}) = \frac{1}{N-1}\mathbf{P}^T\mathbf{X}^T\mathbf{XP} = \mathbf{P}^T\mathbf{\hat{\Sigma}}\mathbf{P} \quad (2.18)$$

A matriz $\mathbf{P}$ é ortogonal, de forma que $\mathbf{P}^T\mathbf{P} = \mathbf{I}$, onde $\mathbf{I}$ é a matriz identidade. Multiplicando ambos os lados da equação $\mathbf{\Lambda} = \mathbf{P}^T\mathbf{\hat{\Sigma}}\mathbf{P}$ por $\mathbf{P}$, temos:

$$\mathbf{\hat{\Sigma}}\mathbf{P} = \mathbf{P}\mathbf{\Lambda} \quad (2.19)$$

A matriz $\mathbf{\Lambda}$ é diagonal, de forma que a equação (2.19) representa p equações $\mathbf{\hat{\Sigma}}\mathbf{p}_i = \lambda_i\mathbf{p}_i$, onde os vetores $\mathbf{p}_i$ são chamados de **autovetores**, e λ_i os **autovalores**. Na ACP, os autovetores da matriz de covariâncias das variáveis originais são chamados de **componentes principais**.

O problema de autovalores e autovetores como (2.19), também chamado de decomposição espectral, é um problema clássico que tem sido estudado desde o século XIX e tem aplicações em inúmeras áreas. Os algoritmos de solução do problema, em suas diversas variantes, estão disponíveis em bibliotecas como EISPACK⁹³ e ARPACK.⁹⁴

A decomposição espectral da matriz de covariâncias pela equação (2.19) fornece informações importantes sobre as características dos dados. Como a matriz de covariâncias é simétrica e definida positiva, os seus autovalores são todos positivos [206]. O maior autovalor λ_{max} é o valor máximo do cociente de Rayleigh:

$$\lambda_{max} = \frac{\mathbf{p}_{max}^T\mathbf{\hat{\Sigma}}\mathbf{p}_{max}}{\mathbf{p}_{max}^T\mathbf{p}_{max}} \geq \frac{\mathbf{v}^T\mathbf{\hat{\Sigma}}\mathbf{v}}{\mathbf{v}^T\mathbf{v}} \quad (2.20)$$

* Utilizando a propriedade de matrizes: $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$.

onde $\mathbf{v} \in \mathcal{R}^p$ é um vetor qualquer e $\mathbf{p}_{max}$ é o autovetor correspondente ao maior autovalor λ_{max} .

O cociente de Rayleigh mostra que o autovetor $\mathbf{p}_{max}$ representa a direção de maior variância da amostra (cf. Figura 2.21) e o autovalor λ_{max} mede a variância na direção de $\mathbf{p}_{max}$. Os demais autovetores são ortogonais e representam as outras direções de maior variância.

A matriz $\mathbf{P}$ realiza uma mudança de coordenadas no espaço de variáveis, transformando o conjunto de dados do problema em um novo conjunto de dados de variáveis transformadas pela equação (2.17). Não há correlação entre as variáveis transformadas, de forma que a matriz de covariâncias $\mathbf{\Lambda}$ é diagonal.

A Figura 2.21 apresenta um exemplo da transformação realizada pela matriz de transformação calculada como solução do problema (2.19). Nesse exemplo, o resultado obtido para a matriz $\mathbf{P}$ é a matriz de transformação mostrada na Figura 2.20.

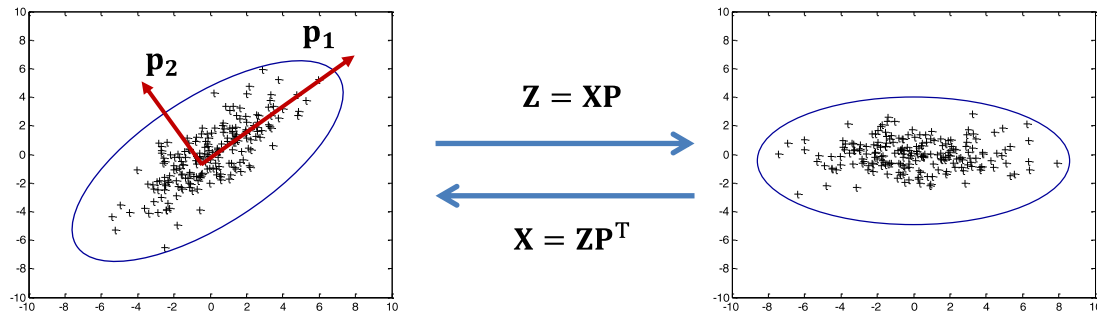


Figura 2.21: Exemplo de transformação linear realizada pela ACP.

Uma propriedade da álgebra linear garante que o traço, isto é, a soma dos elementos da diagonal principal de uma matriz, não se altera pela aplicação de uma transformação linear ortogonal, de forma que:

$$\sum_{i=1}^p \hat{\sigma}_i^2 = \sum_{i=1}^p \lambda_i \quad (2.21)$$

onde $\hat{\sigma}_i^2$ é um elemento da diagonal de $\hat{\mathbf{\Sigma}}$, representa a variância da variável x_i , e λ_i é um elemento da diagonal de $\mathbf{\Lambda}$ que representa a variância da variável transformada z_i .

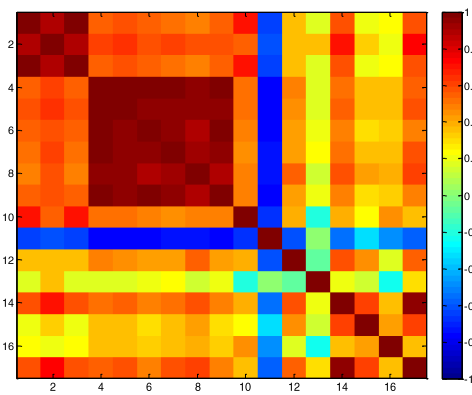
A equação (2.21) garante que a variância total do conjunto de dados não se altera com a transformação qualquer que seja o arranjo das colunas de $\mathbf{P}$. Na maioria das rotinas,

os autovetores são organizados de tal forma que $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$. Com isso, o primeiro autovetor $\mathbf{p}_1$ corresponde ao maior autovalor λ_1 , sendo, portanto, a direção de máxima variância dos dados e assim sucessivamente. Podemos, então, utilizar as $n < p$ primeiras colunas de $\mathbf{P}$ como a matriz de transformação $\hat{\mathbf{P}} \in \mathcal{R}^{p \times n}$ que vai produzir as variáveis transformadas $\hat{\mathbf{z}}(t) = \mathbf{x}(t)\hat{\mathbf{P}}$, $\hat{\mathbf{z}}(t) \in \mathcal{R}^n$, que explicam a maior parte da variância dos dados. O número n de componentes principais é escolhido de forma que sua variância total das variáveis transformadas, calculada como $\sum_{i=1}^n \lambda_i$, represente uma parcela importante (maior que 85%) da variância total das variáveis originais.

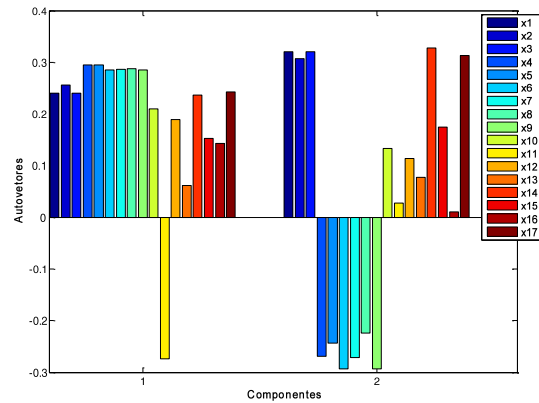
A ACP ajuda a entender o relacionamento entre as variáveis. A Figura 2.22 apresenta os resultados da ACP aplicados ao conjunto de dados Parkinson.⁹⁵ A Figura 2.22a apresenta a matriz de correlação e a Figura 2.22b, os dois componentes principais. Observe que o único elemento negativo do primeiro componente corresponde à variável x_{11} , que tem correlação negativa com todas as variáveis, sendo que as demais têm correlação positiva entre si. O segundo componente separa as variáveis $\{x_1, \dots, x_3\}$, com elementos positivos, das variáveis $\{x_4, \dots, x_9\}$, com elementos negativos, sendo que as variáveis dos dois grupos são bem correlacionadas entre si.

A Figura 2.22c apresenta os resultados dos autovalores no gráfico de Pareto, no qual os rótulos das barras são os valores dos autovalores. As variáveis foram padronizadas pelos *z-scores* de forma que $\hat{\sigma}_i^2 = 1$ e, portanto, $\sum_{i=1}^p \hat{\sigma}_i^2 = p$, ou seja, a variância total é igual ao número de variáveis. Na Figura 2.22c, observa-se que as $n = 7$ componentes principais representam mais que 95% da variância total dos dados. A Figura 2.22d apresenta o conjunto de dados transformados nas coordenadas transformadas pelos dois componentes principais.

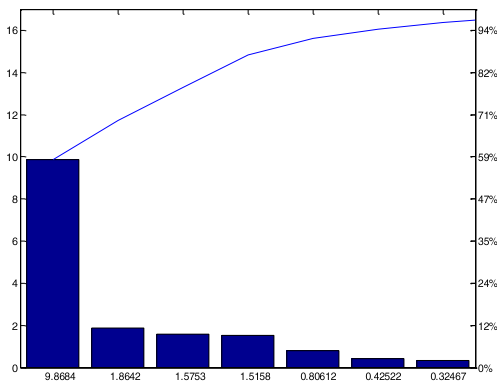
As variáveis transformadas são combinações lineares das variáveis originais. Em alguns casos, é possível interpretar as componentes principais ou as variáveis transformadas, mas esse não é o objetivo principal da ACP e nem sempre há uma interpretação razoável no domínio do problema. A obtenção de transformações interpretáveis no domínio do problema é o objeto da análise de fatores [449].



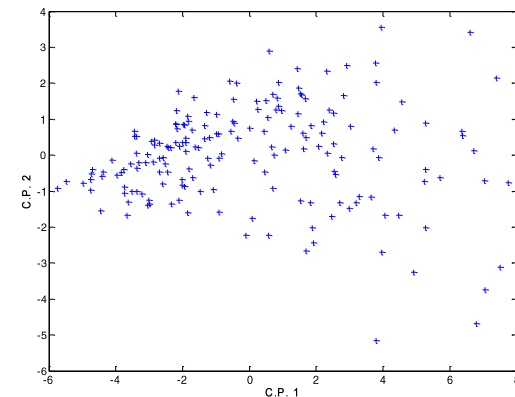
(a)



(b)



(c)



(d)

Figura 2.22: Resultados da ACP no conjunto de dados Parkinson: (a) matriz de correlação; (b) autovetores associados aos dois maiores autovalores; (c) gráfico de Pareto dos autovalores; (d) dados projetados nas duas componentes principais.

A transformação realizada pela ACP não leva em consideração a informação das variáveis de saída. Há transformações lineares que utilizam as variáveis de saída em problemas de classificação e regressão. Em problemas de classificação, a análise discriminante linear (ADL), descrita na seção 2.4.3, resulta em variáveis transformadas em que a separação das classes é máxima. Em problemas de regressão, a análise de correlação canônica, descrita na seção 2.4.4, resulta em transformações lineares que maximizam a correlação entre as variáveis de entrada e saída.

Na próxima seção, apresentamos a decomposição em valores singulares como um método de cálculo alternativo da ACP, que também pode ser aplicada em outros tipos de problemas.

2.4.2 Decomposição em valores singulares

A decomposição em valores singulares (SVD, na sigla em inglês) de uma matriz $\mathbf{A} \in \mathcal{R}^{n \times m}$ é definida como:

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T \quad (2.22)$$

onde $\mathbf{U} \in \mathcal{R}^{n \times n}$ e $\mathbf{V} \in \mathcal{R}^{m \times m}$ são matrizes ortogonais cujas colunas $\mathbf{u}_i, i = 1 \dots n$, e $\mathbf{v}_j, j = 1 \dots m$, são chamadas respectivamente de **vetores singulares** à esquerda e à direita; a matriz $\mathbf{S} \in \mathcal{R}^{n \times m}$ é diagonal e seus elementos são chamados de **valores singulares** de $\mathbf{A}$.

A matriz $\mathbf{S}$ tem $n - m$ (ou $m - n$) elementos nulos quando $n > m$ (ou $m > n$). As colunas de $\mathbf{U}$ (ou as linhas de $\mathbf{V}^T$) correspondentes aos elementos nulos de $\mathbf{S}$ formam a base do espaço nulo* da matriz $\mathbf{A}$ [206]. Alguns *softwares* apresentam o resultado do cálculo da SVD eliminando as linhas e colunas correspondentes aos valores singulares nulos, o que é chamado SVD reduzida. A Figura 2.23 apresenta uma ilustração da SVD nos casos de $n > m$ e $m < n$, onde as linhas tracejadas mostram as dimensões das matrizes resultantes da SVD reduzida.

Em geral, a SVD é calculada com os valores singulares em ordem não decrescente como $s_1 \geq s_2 \geq \dots \geq s_r \geq 0$, onde $r \leq \min(n, m)$ é o posto† da matriz $\mathbf{A}$. Uma aproximação de posto r da matriz $\mathbf{A}$ pode ser calculada utilizando apenas as r primeiras colunas de $\mathbf{U}$, $\mathbf{S}$ e $\mathbf{V}$.

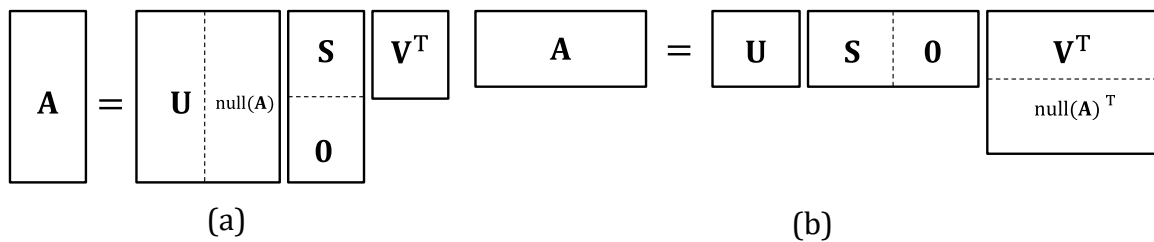


Figura 2.23: Ilustração da SVD: (a) com $n > m$; (b) com $m > n$.

O cálculo da SVD é equivalente ao cálculo de decomposição espectral de matrizes simétricas. Utilizando a SVD da matriz de dados do problema $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ para o cálculo da matriz de covariâncias, temos:

* O espaço nulo da matriz $\mathbf{A}$, denotado $\text{null}(\mathbf{A})$, é o conjunto de vetores $\{\mathbf{x}: \mathbf{A}\mathbf{x} = \mathbf{0}\}$ [206].

† O posto de uma matriz é o maior número de linhas ou colunas linearmente independentes [206].

$$\mathbf{X}^T \mathbf{X} = (\mathbf{U} \mathbf{S} \mathbf{V}^T)^T \mathbf{U} \mathbf{S} \mathbf{V}^T = \mathbf{V} \mathbf{S} \mathbf{U}^T \mathbf{U} \mathbf{S} \mathbf{V}^T = \mathbf{V} \mathbf{S}^2 \mathbf{V}^T \quad (2.23)$$

Comparando a equação (2.23) com a equação (2.19) escrita na forma $\hat{\Sigma} = \mathbf{P} \mathbf{A} \mathbf{P}^T$, pode-se concluir que os valores singulares $s_i = \sqrt{\lambda_i}$, $i = 1 \dots p$. Os vetores singulares à direita são iguais aos autovetores, isto é, $\mathbf{V} = \mathbf{P}$. A ACP pode ser calculada diretamente pela SVD da matriz $\mathbf{X}$, evitando o produto $\mathbf{X}^T \mathbf{X}$.

A SVD calcula uma decomposição ortogonal de uma matriz não quadrada de elementos reais. A SVD é utilizada para redução de dimensionalidade em recuperação de informação na técnica conhecida como índice de semântica latente [364].

2.4.3 Análise discriminante linear

ACP e SVD não utilizam a informação das variáveis de saída para o cálculo da transformação. Em problemas de classificação, é possível que a projeção realizada pela ACP não seja ótima no sentido de separação das classes. A Figura 2.24 apresenta um exemplo simples em que a projeção dos dados na direção do primeiro componente principal acaba misturando os dados.

A análise discriminante linear (ADL) calcula uma transformação linear que maximiza a separação entre as classes nas coordenadas das variáveis transformadas. A ADL também é chamada de discriminante linear de Fisher [146] devido aos trabalhos de Fisher em 1936 [183].

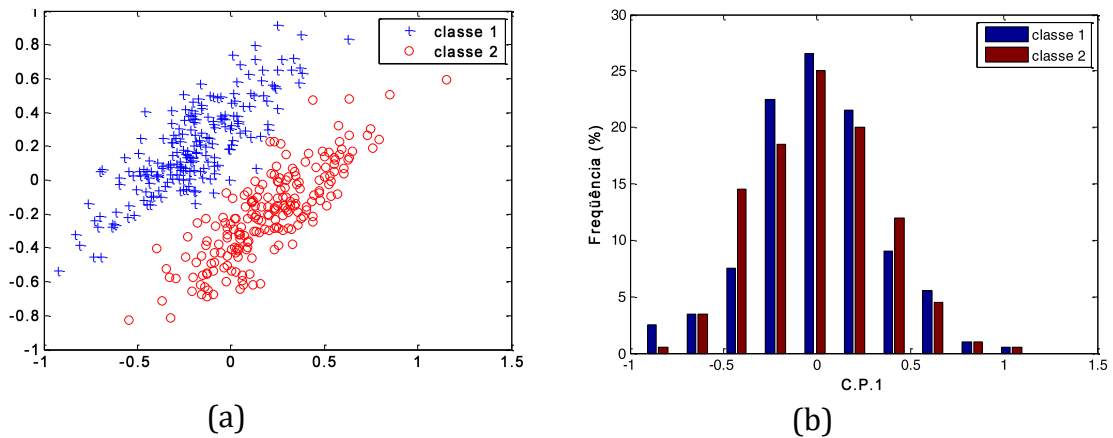


Figura 2.24: Exemplo de ACP num problema de classificação: (a) dados originais; (b) primeiro componente principal (CP1).

O cálculo da transformação é realizado a partir da decomposição da matriz de variâncias total em $\Sigma_T = \Sigma_B + \Sigma_W$, calculadas respectivamente como [146]:

$$\Sigma_{\mathbf{T}} = \sum_{t=1}^N (\mathbf{x}(t) - \bar{\mathbf{x}})^T (\mathbf{x}(t) - \bar{\mathbf{x}}) \quad (2.24)$$

$$\Sigma_{\mathbf{B}} = \sum_{i=1}^m N_i (\bar{\mathbf{x}}_i - \bar{\mathbf{x}})^T (\bar{\mathbf{x}}_i - \bar{\mathbf{x}}) \quad (2.25)$$

$$\Sigma_{\mathbf{W}} = \sum_{i=1}^m \sum_{\mathbf{x}(t) \in \mathcal{C}_i} (\mathbf{x}(t) - \bar{\mathbf{x}}_i)^T (\mathbf{x}(t) - \bar{\mathbf{x}}_i) \quad (2.26)$$

onde $\bar{\mathbf{x}} \in \mathcal{R}^p$ é o vetor de médias das variáveis, calculado para todos os registros; $\bar{\mathbf{x}}_i \in \mathcal{R}^p$ é o vetor de médias dos N_i registros da classe $\mathcal{C}_i$; $\mathcal{C}_i$ é o conjunto de registros da classe $\mathcal{C}_i$, isto é, $\mathcal{C}_i = \{\mathbf{x}(t) \in \mathcal{T} : y(t) = \mathcal{C}_i\}$, $i = 1 \dots m$, que contém N_i elementos; e m é o número de classes.

A matriz de variâncias total $\Sigma_{\mathbf{T}} = (N - 1)\hat{\Sigma}$, onde $\hat{\Sigma}$ é a estimativa da matriz de covariâncias definida pela equação (2.5). A matriz $\Sigma_{\mathbf{B}}$ mede a variância entre classes, ou seja, a separação entre as classes, e a matriz $\Sigma_{\mathbf{W}}$ mede a variância intraclases, ou seja, entre os registros da mesma classe.

A transformação da ADL é calculada de forma que o novo sistema de coordenadas produza dados com máxima variância entre classes e mínima variância intraclases. Em problemas de duas classes ($m = 2$), a direção $\mathbf{v}$ que maximiza a relação $\Sigma_{\mathbf{B}}/\Sigma_{\mathbf{W}}$ é obtida pela maximização do seguinte coeficiente de Rayleigh:

$$J(\mathbf{v}) = \frac{\mathbf{v}^T \Sigma_{\mathbf{B}} \mathbf{v}}{\mathbf{v}^T \Sigma_{\mathbf{W}} \mathbf{v}} \quad (2.27)$$

onde $\mathbf{v}$ é calculado como solução do problema de autovalor generalizado $\Sigma_{\mathbf{B}} \mathbf{v} = \lambda \Sigma_{\mathbf{W}} \mathbf{v}$. Esse resultado pode ser estendido a problemas de múltiplas classes ($m > 2$), resolvendo-se o seguinte problema de autovalor generalizado [146]:

$$\Sigma_{\mathbf{B}} \mathbf{V} = \Sigma_{\mathbf{W}} \mathbf{V} \mathbf{\Lambda} \quad (2.28)$$

onde $\mathbf{\Lambda}$ é a matriz diagonal dos autovalores generalizados, que contém apenas $m - 1$ elementos e $\mathbf{V}$ é a matriz dos $m - 1$ autovetores generalizados.

O conjunto de dados da variável transformada $\mathbf{Z}$, de dimensão $N \times m - 1$, é calculado pela transformação linear $\mathbf{Z} = \mathbf{XV}$.

A Figura 2.25 apresenta os resultados da ADL para o exemplo da Figura 2.24. Nesse caso, a direção calculada pela ADL é a mesma do segundo componente principal.

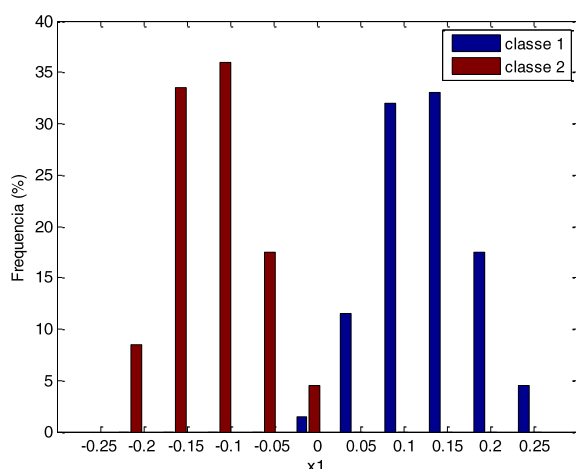


Figura 2.25: Resultado da ADL para os dados da Figura 2.24.

Em problemas linearmente separáveis, a ADL é uma solução ótima que permite uma boa aproximação com um número reduzido de componentes. O conjunto de dados *Wine*,⁹⁶ por exemplo, é um problema de classificação com 13 variáveis de entrada contendo os resultados de análises químicas em três categorias de vinhos de uma região da Itália. A Figura 2.26 mostra os resultados da ADL. A Figura 2.26a apresenta os dados transformados e os histogramas das $n = 2$ novas variáveis e a Figura 2.26b mostra os elementos de cada componente da matriz de transformação V .

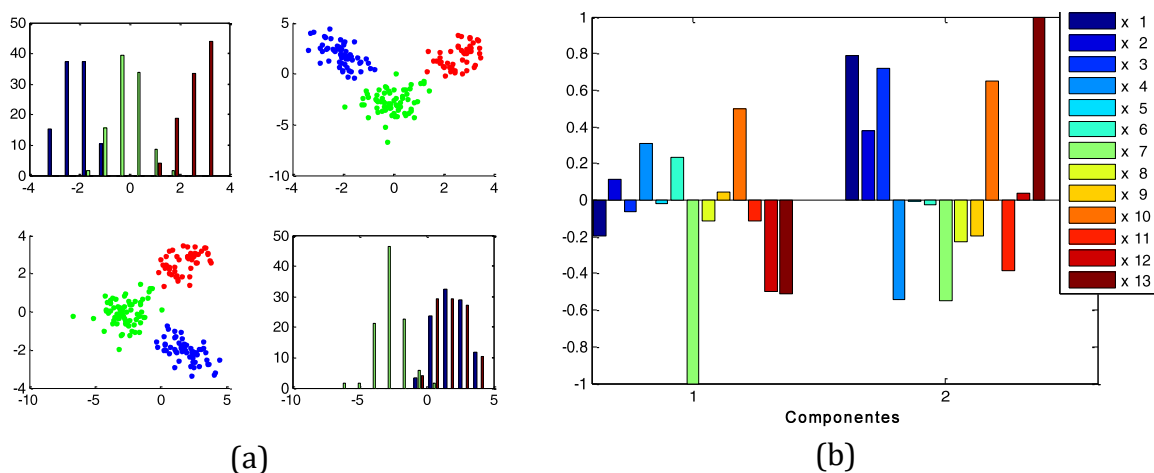


Figura 2.26: Resultado da ADL no conjunto de dados Wine: (a) dados transformados; (b) componentes da matriz de transformação.

2.4.4 Análise de correlação canônica

Em problemas de classificação, a ADL calcula uma transformação linear que leva em consideração a informação das classes para definir um espaço de coordenadas onde a separação das classes é ótima. Em problemas de regressão, é possível obter um resultado semelhante com a análise de correlação canônica (ACC).

A ACC considera o problema em que há dois conjuntos de variáveis, que representam duas formas diferentes de observar o mesmo fenômeno. O objetivo é calcular duas transformações lineares, uma para cada variável, a partir da correlação cruzada entre as variáveis. Quando aplicadas aos conjuntos de dados iniciais, as transformações produzem novas variáveis que representam a melhor a correlação do conjunto de dados.

A formulação da ACC será apresentada nesta seção utilizando as matrizes de dados de entrada $\mathbf{X} \in \mathcal{R}^{N \times p}$ e saída $\mathbf{Y} \in \mathcal{R}^{N \times q}$ do conjunto de dados de treinamento, mas a ordem dos conjuntos de dados não interfere no processamento. A ACC realiza uma aproximação de menor dimensão da entrada e da saída de um problema de regressão.

A ACC produz novas variáveis $\mathbf{Z}_x = \mathbf{X}\mathbf{U}_x$ e $\mathbf{Z}_y = \mathbf{Y}\mathbf{U}_y$, chamadas de **variáveis canônicas**, ambas de dimensão* $n \leq \min(p, q)$. As transformações lineares $\mathbf{U}_x$ e $\mathbf{U}_y$ são calculadas de forma a maximizar a correlação entre as variáveis canônicas e cada transformação é calculada com os dados das duas variáveis.

Assim como na ACP, as variáveis devem ser padronizadas pelos *z-scores*, resultando em variáveis de média nula e desvio padrão unitário. As transformações lineares ortogonais não alteram as médias e os desvios padrão das variáveis transformadas, de forma que a matriz de correlação cruzada entre as variáveis canônicas é escrita como:

$$\mathbf{Z}_x^T \mathbf{Z}_y = \mathbf{U}_x^T \mathbf{X}^T \mathbf{Y} \mathbf{U}_y \quad (2.29)$$

onde $\mathbf{Z}_x, \mathbf{Z}_y \in \mathcal{R}^{N \times n}$ são as matrizes de dados das variáveis canônicas.

O cálculo das matrizes de transformação na ACC foi proposto inicialmente por Hotelling em 1936 [261]. Nesta seção, utilizamos a formulação proposta por Golub e Zha [207], que é baseada na decomposição ortogonal QR das matrizes de dados:

$$\mathbf{X} = \mathbf{Q}_x \mathbf{R}_x \quad (2.30)$$

* Formalmente, $n = \min(\text{rank}(\mathbf{X}), \text{rank}(\mathbf{Y}))$, onde rank é o posto da matriz.

$$\mathbf{Y} = \mathbf{Q}_y \mathbf{R}_y \quad (2.31)$$

onde as matrizes $\mathbf{Q}_x$ e $\mathbf{Q}_y$ são ortogonais e as matrizes $\mathbf{R}_x$ e $\mathbf{R}_y$ são triangulares superiores. Nesse caso, utilizamos a decomposição QR reduzida, em que a matriz $\mathbf{Q}_x$ ($\mathbf{Q}_y$) tem dimensão $N \times p$ ($N \times q$) e a matriz $\mathbf{R}_x$ ($\mathbf{R}_y$) tem dimensão $p \times p$ ($q \times q$).

Aplicando as decomposições QR definidas nas equações (2.30) e (2.31) na equação (2.29), a correlação cruzada entre os dois conjuntos de variáveis é escrita como:

$$\mathbf{\Lambda} = \mathbf{U}_x^T \mathbf{X}^T \mathbf{Y} \mathbf{U}_y = \mathbf{U}_x^T \mathbf{R}_x^T \mathbf{Q}_x^T \mathbf{Q}_y \mathbf{R}_y \mathbf{U}_y \quad (2.32)$$

A matriz de correlação $\mathbf{\Lambda}$ é diagonal e seus elementos são os valores singulares de $\mathbf{Q}_x^T \mathbf{Q}_y$ [206], chamados de correlações canônicas. As transformações lineares $\mathbf{U}_x$ e $\mathbf{U}_y$ são escritas a partir da SVD $\mathbf{Q}_x^T \mathbf{Q}_y = \mathbf{V}_x \mathbf{S} \mathbf{V}_y^T$. Aplicando o resultado da SVD na equação (2.32):

$$\mathbf{U}_x^T \mathbf{R}_x^T \mathbf{Q}_x^T \mathbf{Q}_y \mathbf{R}_y \mathbf{U}_y = \mathbf{U}_x^T \mathbf{R}_x^T \mathbf{V}_x \mathbf{S} \mathbf{V}_y^T \mathbf{R}_y \mathbf{U}_y \quad (2.33)$$

A ACC calcula a correlação cruzada $\mathbf{\Lambda} = \mathbf{S}$ de tal forma que $\mathbf{U}_x^T \mathbf{R}_x^T \mathbf{V}_x = \mathbf{I}$ e $\mathbf{V}_y^T \mathbf{R}_y \mathbf{U}_y = \mathbf{I}$, onde $\mathbf{I}$ representa a matriz identidade de dimensão do número de componentes canônicos n . As transformações lineares da ACC são calculadas como:

$$\mathbf{U}_x = \mathbf{R}_x^{-1} \mathbf{V}_x^T \quad (2.34)$$

$$\mathbf{U}_y = \mathbf{R}_y^{-1} \mathbf{V}_y \quad (2.35)$$

A formulação anterior é equivalente à maximização do traço da matriz $\mathbf{\Lambda}$ [207], impondo a restrição que as matrizes de transformação devem ser ortogonais.

A aplicação da ACC é mostrada no exemplo de elevação de temperatura adiabática do concreto, que será discutido na seção 3.4.2. O conjunto de dados apresenta 14 variáveis de entrada divididas em dois tipos: cinco variáveis que representam a composição do cimento utilizado e outras nove com os demais aditivos [18]. A elevação de temperatura é parametrizada por 3 variáveis de saída.

Os resultados da ACC são exibidos na Figura 2.27. O número de componentes canônicos neste caso é $n \leq \min(p, q) = 3$. A Figura 2.27a mostra os elementos da correlação cruzada $\mathbf{\Lambda}$. As variáveis canônicas de maior correlação são calculadas como $\mathbf{z}_{x1} = \mathbf{X} \mathbf{u}_{x1}$ e $\mathbf{z}_{y1} = \mathbf{Y} \mathbf{u}_{y1}$, onde $\mathbf{u}_{x1}$ e $\mathbf{u}_{y1}$ são os primeiros componentes de $\mathbf{U}_x$ e $\mathbf{U}_y$. A Figura 2.27c mostra o gráfico de projeção entre as variáveis $\mathbf{z}_{x1}$ e $\mathbf{z}_{y1}$. A Figura 2.27c e a Figura 2.27c mostram respectivamente os elementos dos vetores $\mathbf{u}_{x1}$ e $\mathbf{u}_{y1}$.

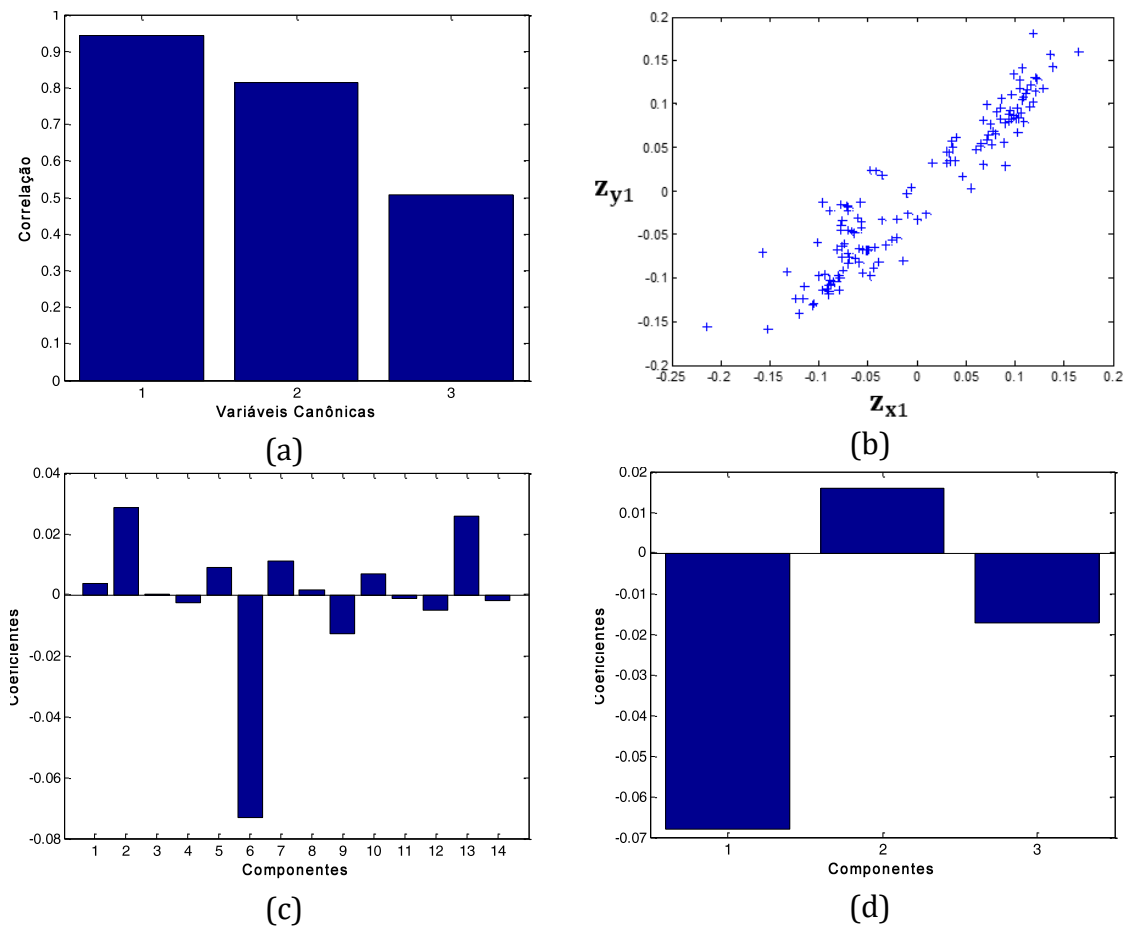


Figura 2.27: Resultados da ACC nos dados de elevação de temperatura do concreto:

(a) correlações canônicas; (b) gráfico de projeção das variáveis canônicas;

(c) primeiro componente de U_x ; (d) primeiro componente de U_y .

Esta seção apresentou os principais métodos para o cálculo de transformações lineares sobre o conjunto de dados. Esses métodos utilizam técnicas de cálculo matricial e mostram como pode ser útil a representação de um registro no conjunto de dados como um ponto do espaço vetorial.

As transformações lineares apresentadas nesta seção consideram que a amostra dos dados tem distribuição normal, o que nem sempre é verificado, e o resultado pode não ser representativo. Além disso, muitos problemas reais que contêm variáveis nominais e binárias, que também pode interferir nos resultados.

Aplicações de sumarização e visualização de dados são apresentadas a seguir, assim como a utilização de transformações lineares para validação de medidas.

2.5 Aplicações

2.5.1 Padrões de mobilidade

A universalização da telefonia celular e a difusão de dispositivos móveis conectados à internet transformaram o ser humano num sensor ambulante. O registro da atividade dos usuários fornece uma quantidade sem precedentes de dados, o que tem atraído o interesse de diversos pesquisadores de diferentes áreas [30][70].

Um projeto de pesquisa realizou o estudo de uma base de registros de chamadas por celular na região de código de área 21, que compreende toda a Região Metropolitana do Rio de Janeiro (RMRJ) (cf. detalhe da Figura 2.28). A base de dados utilizada do projeto continha apenas o registro de chamadas de voz realizadas, em que a identificação dos usuários foi criptografada. Há estudos que utilizam registros mais detalhados [53] como chamadas recebidas, mensagens de texto (SMS), registros de conexão com internet, dados processados diretamente pelas antenas ou registros de posicionamento via satélite (GPS) ou registros de redes sociais.

Os registros de chamadas de celulares (CDR)* permitem que o movimento das pessoas seja inferido pela posição da antena (célula) que processou a chamada. Em geral, pelo menos três antenas são posicionadas a 120° em cada torre, de forma que a cobertura por torre pode ser aproximada por um polígono de Voronoi.

A Figura 2.28 mostra a cobertura no Rio de Janeiro, com os polígonos de Voronoi das torres. O posicionamento das torres segue objetivos mercadológicos e técnicos, de forma que a distribuição resultante não é uniforme. Há maior concentração de antenas nas áreas de maior densidade populacional e nível socioeconômico, como o Centro, Tijuca e a Zona Sul. Outras áreas populosas, como Jacarepaguá, Campo Grande, Duque de Caxias e São Gonçalo, têm uma concentração menor de antenas. A geografia da cidade afeta a distribuição de antenas devido à presença de maciços montanhosos dentro da área urbana, como o maciço da Tijuca e o da Pedra Branca, visíveis na Figura 2.28.

* *Call Detail Records.*



Figura 2.28: Polígonos de Voronoi no Rio de Janeiro. No detalhe a cobertura da área 21.

Os dados foram coletados ao longo do ano de 2014, entre 31 de dezembro de 2013 e a 1º de janeiro de 2015, somando 2.1 bilhões de registros referentes a 2.9 milhões de assinantes. Apenas os dados de chamadas de voz realizadas foram disponibilizados para o estudo, o que limita consideravelmente a amostra.

Foi realizada uma modelagem da região de estudo pelo agrupamento de polígonos de Voronoi em áreas que aproximam unidades demográficas do IBGE. A identificação de residência presumida dos usuários permitiu a junção dos resultados de mobilidade com dados demográficos e socioeconômicos. Os dados de CDR foram utilizados para estimativa de matrizes origem-destino (OD) para cada dia para análise dos padrões de mobilidade na RMRJ [87][88]. Os resultados do processamento para cada dia estão disponíveis no Dataverse.⁹⁷

Os dois padrões de mobilidade mais comuns são mostrados na Figura 2.30 e na Figura 2.30 [88]. A Figura 2.30 apresenta o número de viagens estimadas entre o Centro (AP1) e Zona Sul (AP2) e representa o padrão casa-trabalho, com grande regularidade, maior número de viagens durante a semana e estimativas semelhantes na ida e na volta.

A Figura 2.30 mostra o número de viagens estimadas entre a Zona Sul (AP2) e Teresópolis, uma cidade turística na Região Serrana do Estado. Neste caso, o padrão é totalmente diferente, o maior número de viagens ocorre nos finais de semana, com picos nos feriados, e há uma defasagem entre a ida e a volta.

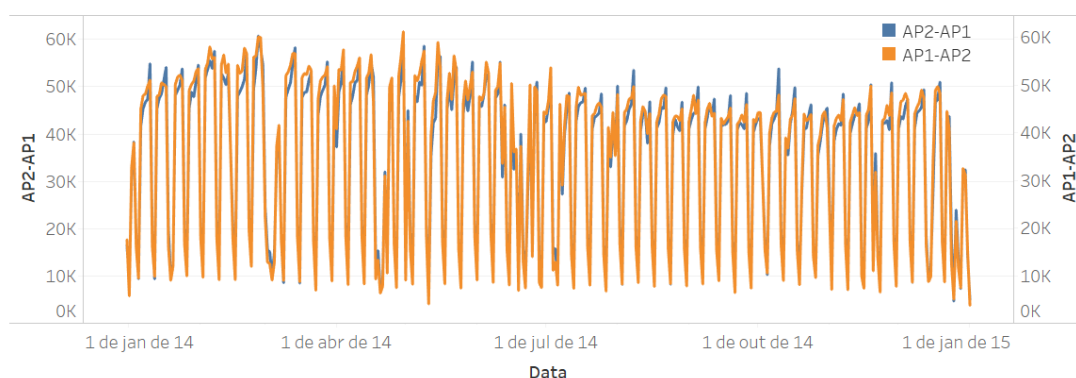


Figura 2.29: Número de viagens Centro (AP1) – Zona Sul (AP2). Fonte: reproduzido de [88].

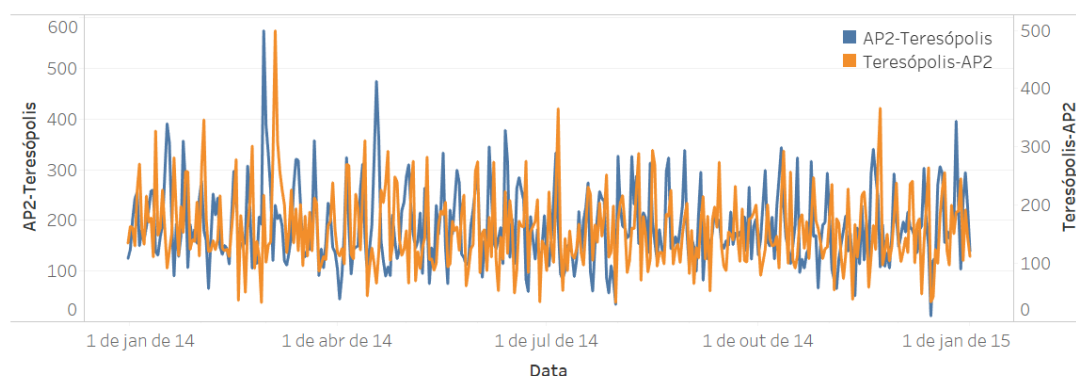


Figura 2.30: Número de viagens Zona Sul (AP2) – Teresópolis. Fonte: reproduzido de [88].

Há diversos estudos de mobilidade a indicadores sociais a partir de dados de telefonia celular [117][526]. Os resultados obtidos mostram que os dados de registros de chamadas de celular podem ser uma alternativa de baixo custo para a modelagem de problemas complexos das cidades modernas.

Atualmente, há uma grande oferta de dados públicos e muitas cidades no mundo mantêm portais de dados abertos. No Rio de Janeiro, o portal data.rio⁹⁸ mantém uma base de dados pública para consulta. Nesse portal, é possível acessar em tempo real os dados de localização (GPS) dos ônibus municipais urbanos.

Um estudo sobre a velocidade dos ônibus na cidade foi feito pela Coppe/UFRJ em parceria com o Centro de Pesquisas da Dell-EMC. O resultado da velocidade média diária nas vias da cidade é mostrado na Figura 2.32, na qual os valores de velocidade são representados por uma escala de cores e a largura do traço representa o número de linhas de ônibus que passam na via. Pode-se verificar que as vias expressas apresentam as maiores velocidades médias (verde), enquanto as vias em áreas com maior densidade urbana têm velocidade reduzida (vermelho). As principais áreas de concentração

da cidade podem ser identificadas no mapa: zona Sul, Centro, Tijuca, Meier, Madureira, Jacarepaguá, Bangu, Campo Grande e Santa Cruz.

A visualização e caracterização estatística dos dados é uma etapa fundamental para o entendimento do problema. Os exemplos apresentados nesta seção ilustram exemplos de visualização de dados georreferenciados utilizando sistemas de informação geográfica⁹⁹ em conjunto com servidores de mapas na internet.

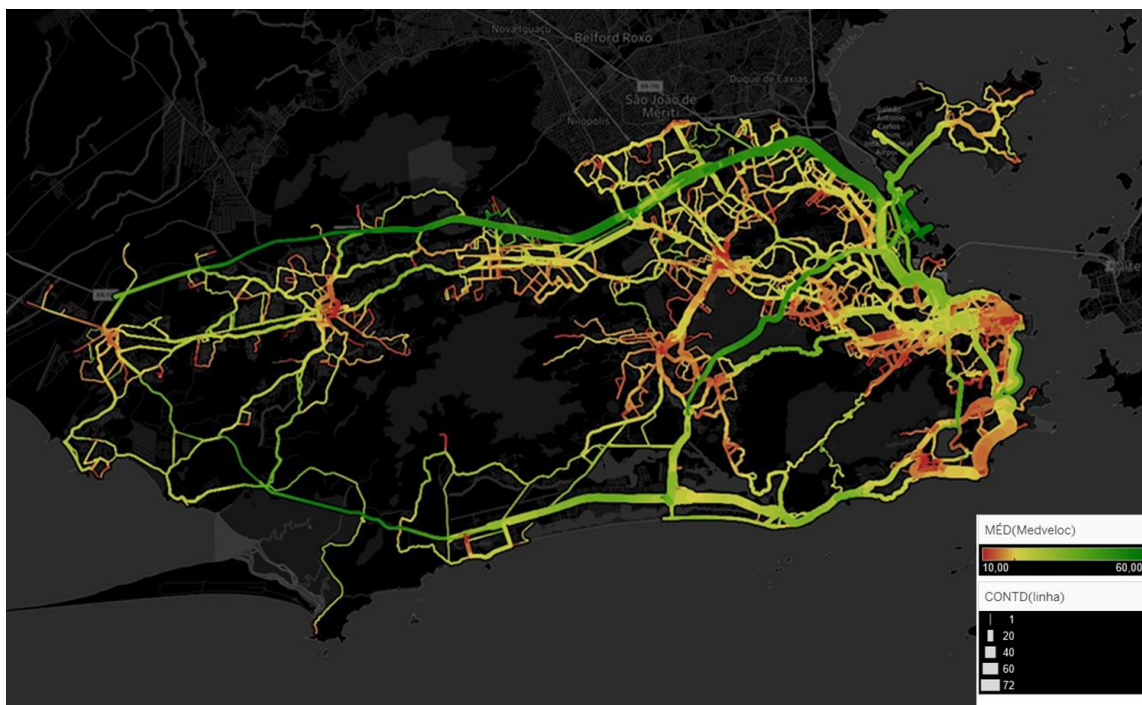


Figura 2.31: Velocidade média dos ônibus na cidade do Rio de Janeiro às 19:00.

A próxima seção mostra uma aplicação de transformações lineares para a validação de medidas em sistemas de diagnóstico.

2.5.2 Equações de paridade

O monitoramento de processos complexos é realizado com dados coletados por sensores cujo funcionamento deve ser continuamente verificado para evitar que alguma perturbação nas medidas seja confundida com uma falha no processo. O método de validação de medidas por equações de paridade utiliza a redundância de informações contida nas medidas para gerar indicadores de falha através de transformações lineares. A redundância pode ser direta, na qual existe mais de um sensor para uma mesma medida, ou indireta, quando as medidas produzidas por um conjunto de sensores devem

preservar alguma coerência entre seus valores. A análise de componentes principais (ACP) pode ser aplicada para a síntese de um sistema de validação de medidas por equações de paridade em sistemas lineares estáticos.

Considere um conjunto de p sensores, cujas medidas são representadas pelas componentes do vetor $\mathbf{x}(t) \in \mathcal{R}^p$. As medidas se relacionam com um conjunto de estados $\mathbf{z}(t) \in \mathcal{R}^n$ pela equação de medidas:

$$\mathbf{x}(t) = \mathbf{z}(t)\mathbf{C}^T + \Delta\mathbf{x}(t) + \mathbf{e}(t) \quad (2.36)$$

onde $\mathbf{C}$, de dimensão $p \times n$, é a matriz de medidas, $\Delta\mathbf{x}(t) \in \mathcal{R}^n$ representa falhas aditivas nos sensores e $\mathbf{e}(t) \in \mathcal{R}^n$ é um ruído de média nula e variância constante.

As medidas são padronizadas através dos *z-scores* (cf. equação (2.16)) e têm média nula e desvio padrão unitário. Há redundância de informações quando a dimensão do vetor de medidas $p > n$. Nesse caso, pode-se utilizar a informação redundante para detecção de falhas nos sensores.

Um método robusto e bastante conhecido para validação de sensores é baseado nas equações de paridade [201]. Essa técnica tem sido utilizada em diversas aplicações e consiste em projetar o vetor de medidas num espaço de dimensão $m = p - n$, chamado **espaço de paridades**. O vetor de paridades é calculado como:

$$\mathbf{u}(t) = \mathbf{x}(t)\mathbf{W} \quad (2.37)$$

onde $\mathbf{W}$, de dimensão $p \times m$, é a matriz de projeção e $\mathbf{u}(t) \in \mathcal{R}^m$, o vetor de paridade.

O vetor de paridades é um indicador de falhas que é nulo na ausência delas. Dessa forma, a matriz de projeção deve ser ortogonal à matriz de medidas:

$$\mathbf{C}^T\mathbf{W} = \mathbf{0} \quad (2.38)$$

Reescrevendo a equação (2.37) considerando a equação de medidas (2.36) e a restrição imposta pela equação (2.38), o vetor de paridade é calculado como:

$$\mathbf{u}(t) = (\mathbf{z}(t)\mathbf{C}^T + \Delta\mathbf{x}(t) + \mathbf{e}(t))\mathbf{W} = (\Delta\mathbf{x}(t) + \mathbf{e}(t))\mathbf{W} \quad (2.39)$$

Pela equação (2.39), pode-se perceber que o vetor de paridades representa apenas as falhas e incertezas (ruído) no processo, independentemente do valor dos estados. Assim, um evento externo que altera de forma consistente todas as medidas não altera significativamente o valor do vetor de paridade.

Cada linha da matriz $\mathbf{W}$ representa uma direção de falha no espaço de paridades. Uma perturbação aditiva numa medida provoca um acréscimo do vetor de paridade na direção correspondente. A identificação da falha é realizada comparando-se a direção do vetor de paridade com as direções definidas pela matriz de projeção.

Considere o exemplo de um sistema com redundância material em que $p = 4$ sensores são utilizados para observar os valores de $n = 2$ estados, com as seguintes matrizes de medidas e de projeção:

$$\mathbf{C}^T = \begin{bmatrix} 1.0 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & 1.0 \end{bmatrix}, \quad \mathbf{W}^T = \begin{bmatrix} 0.5 & -0.5 & 0.5 & -0.5 \\ 0.5 & -0.5 & -0.5 & 0.5 \end{bmatrix} \quad (2.40)$$

O espaço de paridades e as direções de falha associadas a cada medida neste exemplo são mostrados na Figura 2.32. O parâmetro ξ representa uma tolerância para o módulo do vetor de paridade a fim de acomodar o ruído do processo (cf. equação (2.37)), de tal forma que a detecção é feita somente se $\|\mathbf{u}(t)\| > \xi$.

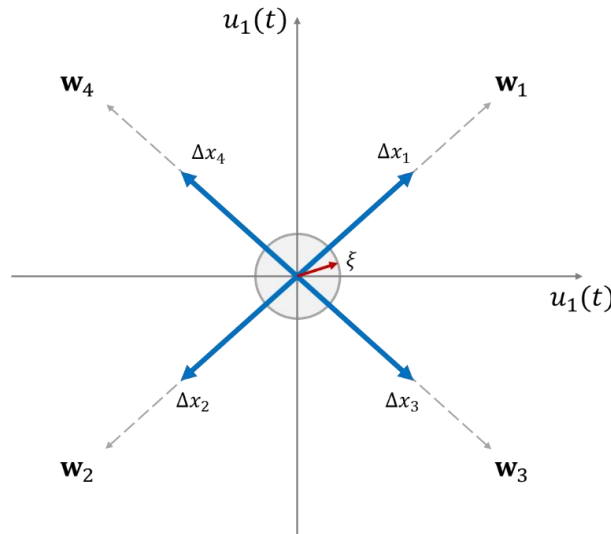


Figura 2.32: Direções de falha no espaço de paridades.

O método de equações de paridades garante a capacidade do sistema de detectar falhas simples, ou seja, falha aditiva em apenas uma medida. Em caso de falhas múltiplas, uma combinação de falhas pode anular o vetor de paridades e o vetor de perturbações deve representar assinaturas de falha. Dependendo da dimensão do vetor de paridades, pode não ser possível isolar todas as falhas. O método não se aplica em caso de falhas que afetam o comportamento do processo.

As matrizes de medida e de projeção do método de equações de paridade podem ser geradas a partir da ACP apresentada na seção 2.4.1. Considere um conjunto de registros do vetor de medidas $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$. A matriz de covariâncias estimada é calculada como $\hat{\Sigma} = \frac{1}{N-1} \mathbf{X}^T \mathbf{X}$, onde $\mathbf{X}$, de dimensão $N \times p$, contém os registros das variáveis de entrada. A ACP é calculada pela diagonalização da matriz $\hat{\Sigma}$ pela solução do problema de autovalor definido na equação (2.19):

$$\hat{\Sigma} \mathbf{P} = \mathbf{P} \mathbf{\Lambda} \quad (2.41)$$

onde as colunas da matriz $\mathbf{P}$ são os autovetores (componentes principais) e os autovalores da matriz diagonal $\mathbf{\Lambda}$ representam a variância da variável transformada.

As variáveis transformadas pela ACP podem ser utilizadas para o cálculo de equações de paridades, considerando a matriz de componentes principais composta pelas matrizes de medida e de projeção como $\mathbf{P} = [\mathbf{C} \mid \mathbf{W}]$. As colunas da matriz de componentes principais, de dimensão $p \times p$, são ortogonais, de forma que a condição (2.38) é satisfeita. A síntese do sistema de validação de medidas pela ACP consiste em escolher as dimensões n e m dos espaços de estados e de paridade. Tendo em vista que $p = n + m$, quanto maior a redundância das medidas, menor a dimensão dos estados n e maior a dimensão do vetor de paridades m .

O valor do parâmetro ξ para a detecção de falhas pode ser estimado pela variância total não representada pelo vetor de estados e calculado pela soma dos autovalores correspondentes às componentes do vetor de paridade:

$$\xi = \sqrt{\sum_{i=n+1}^p \lambda_i} \quad (2.42)$$

onde λ_i é o autovalor da matriz $\mathbf{\Lambda}$ correspondente às colunas $i = n + 1, \dots, p$.

O método de equações de paridade foi utilizado para validação de medidas de deformações da Usina Hidrelétrica (UHE) de Funil¹⁰⁰ por extensômetros de concreto. O conjunto de dados cobria o período entre 20/07/1967 e 30/11/2005, num total de 2.161 registros. Os registros com valores ausentes foram eliminados para o cálculo das matrizes de medida e projeção, que foi realizado com os $N = 380$ registros completos de 21 variáveis.

O método de equações de paridade é imune à presença de valores ausentes, uma vez que apenas as perturbações medidas são consideradas no cálculo do vetor de paridade (cf. equação (2.39)). O resultado do método de equações de paridade é mostrado na Figura 2.33, na qual os registros em cinza representam os valores ausentes e, em preto, os registros identificados com falha. Observe que, em alguns casos, a ocorrência de falha está associada à retirada do sensor de operação.

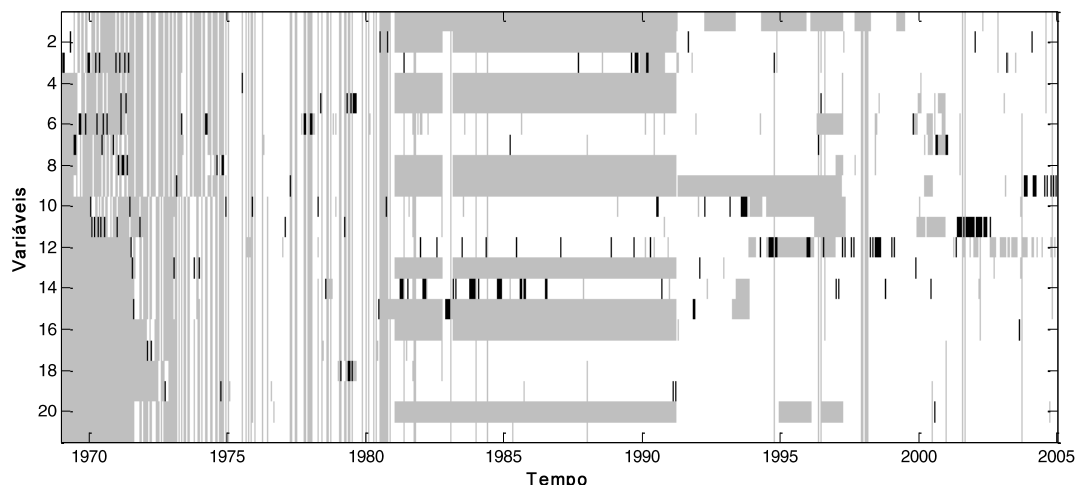


Figura 2.33: Resultado da validação de sensores.

2.6 Comentários Finais

Este capítulo apresentou técnicas básicas utilizadas na etapa de pré-processamento do conjunto de dados. Um aprofundamento sobre a teoria das probabilidades e álgebra linear é uma base importante de inteligência computacional [146][298]. Há diversos livros que abordam o assunto, como Montgomery [400] e Heath [244] que apresenta uma visão dos conceitos de cálculo matricial. O livro de Heath tem também uma página na internet¹⁰¹ que fornece vários recursos associados ao tema.

As medidas de caracterização estatística apresentadas são as mais usuais e que estão presentes na maioria dos *softwares* e linguagens de programação. O assunto continua sendo explorado na literatura [490] e diferentes soluções disponíveis.¹⁰²

Alguns métodos de redução de dimensionalidade não foram abordados na seção 2.4, cujo foco é apenas em transformações lineares. Outras transformações conhecidas, como o *Local Linear Embedding* (LLE), a análise de componentes independentes (ICA),

entre outras, não foram apresentadas pois envolvem algoritmos não lineares. Uma discussão de diversos métodos lineares e não lineares, incluindo uma extensa revisão bibliográfica no assunto, pode ser encontrada em [357], disponível para *download*,¹⁰³ onde é possível obter uma biblioteca Matlab contendo a implementação dos métodos.

As técnicas de cálculo matricial têm sido cada vez mais empregadas em ciência de dados, especialmente para análise de grandes bases de dados, como a internet, e bases de dados multidimensionais [493][106]. Os métodos lineares são estendidos para o caso não linear utilizando métodos baseados em função de núcleo [478]. As funções de núcleo serão apresentadas no Capítulo 7 no contexto de aproximação de funções com modelos SVM. Livros recentes apresentam as funções de núcleo e a matriz de núcleo nos capítulos iniciais como forma de representação dos dados [593], uma vez que funções de núcleo podem ser utilizadas em registros que não têm uma representação vetorial.

Neste livro, os capítulos iniciais abordam os fundamentos e, por isso, apenas os métodos mais simples são apresentados. O próximo capítulo discute o modelo de regressão linear e o método dos mínimos quadrados. Os algoritmos preditivos não lineares para regressão e classificação são apresentados nos Capítulos 7, 8 e 9.

3 REGRESSÃO LINEAR

A regressão linear é uma técnica clássica da estatística cujo objetivo é o ajuste de um modelo que relaciona uma ou mais variáveis de entrada, ou independentes, com uma ou mais variáveis de saída, ou dependentes. Em ciência de dados, a regressão (linear ou não linear) também é chamada de predição [232]. Em inteligência computacional, utiliza-se frequentemente o termo aproximação de funções [298].

O modelo de regressão linear permite que os desafios associados ao desenvolvimento de modelos complexos sejam apresentados de uma forma clara e analítica. Os conceitos introduzidos neste capítulo servirão de base para modelos não lineares apresentados nos Capítulos 7, 8 e 9. Na prática, o modelo linear é sempre o primeiro que o analista deve construir devido a sua simplicidade, baixo custo computacional e também para obter um limite inferior de desempenho para modelos não lineares. Não são raros os problemas em que a diferença de desempenho entre um modelo linear e outro modelo mais elaborado (como uma rede neural, por exemplo) seja menor que 5%. Entretanto, há casos em que a relação entre as variáveis de entrada e saída é intrinsecamente não linear e modelos mais complexos obtêm melhores resultados.

Neste capítulo, apresentamos o problema de regressão linear e sua solução através do método dos mínimos quadrados. Em seguida, abordamos o dilema entre *bias* e variância, que é essencial para o desenvolvimento de um bom modelo. As técnicas de regularização, ou suavização de modelos, são também discutidas neste capítulo. Finalmente, introduzimos as estatísticas para validação de modelos de regressão, que também serão utilizadas para avaliação de modelos não lineares.

3.1 O Problema de Regressão

Conforme apresentado no Capítulo 2, o conjunto de dados é representado neste livro por $\mathcal{T} = \{(\mathbf{x}(t), \mathbf{y}(t)), t = 1 \dots N\}$, onde cada registro é formado por pares ordenados $(\mathbf{x}(t), \mathbf{y}(t))$, sendo $\mathbf{x}(t) \in \mathcal{R}^p$ o vetor das variáveis de entrada e $\mathbf{y}(t) \in \mathcal{R}^q$ o vetor das variáveis de saída.

As variáveis de entrada não numéricas podem ser transformadas num conjunto de variáveis binárias chamadas variáveis indicadoras ou *dummy* [399]. As variáveis de saída não numéricas caracterizam o problema de classificação que será abordado no Capítulo 5. Nesse capítulo, a regressão linear é apresentada considerando variáveis de entrada numéricas e uma única variável de saída numérica, ou seja, $q = 1$ e $y(t) \in \mathcal{R}$. Os problemas de regressão de saídas múltiplas podem, na maioria dos casos, ser modelados por modelos independentes de saída simples.

No problema de regressão linear simples, uma única variável de entrada ($p = 1$) se relaciona linearmente com a variável de saída pela equação da reta $y(t) = ax(t) + b$ (cf. Figura 3.1). Na prática, a relação real não pode ser observada e os dados contêm perturbações de diversas origens, como erro de leitura, efeito de variáveis não observadas etc. Dessa forma, os registros observados, armazenados no conjunto de dados, contêm perturbações, como os pontos em azul na Figura 3.1. O modelo é ajustado a partir desses registros observados e, portanto, incorpora os erros de observação em seus parâmetros, sendo descrito por $\hat{y}(t) = \hat{a}x(t) + \hat{b}$, onde $\hat{y}(t)$ é o valor da variável de saída calculado pelo modelo e $\hat{a}$ e $\hat{b}$ são os parâmetros ajustados. O erro de ajuste ou **resíduo do modelo** é a diferença entre os valores observado e calculado pelo modelo da variável de saída, isto é, $e(t) = y(t) - \hat{y}(t)$, para cada registro do conjunto de treinamento.

O que se espera é que o modelo seja o mais próximo possível da relação real, pois sabemos que as perturbações têm origem aleatória. Assim, quanto mais próximo da relação real, melhor será o desempenho do modelo para a predição de novos dados, ou seja, registros diferentes dos que foram armazenados no conjunto de treinamento, mas gerados com a mesma distribuição.

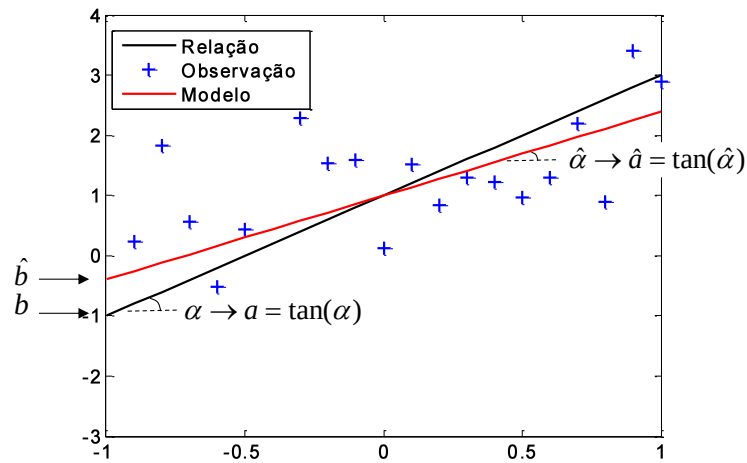


Figura 3.1: Exemplo de regressão linear.

Na hipótese da relação entre a variável de entrada e saída ser linear, os parâmetros do modelo podem ser facilmente ajustados pela minimização do erro total no conjunto de treinamento. O problema é que, na prática, não se conhece ou não há elementos para determinar o tipo de função que relaciona as variáveis de entrada e saída. A variedade de *softwares* e alternativas de modelagem é tão grande que, frequentemente, o analista pode ajustar um modelo cuja complexidade é maior do que a complexidade da relação, como mostra a Figura 3.2a. Nesse caso, diz-se que ocorreu sobre ajuste, ou **overfitting**, no jargão de ciência de dados.

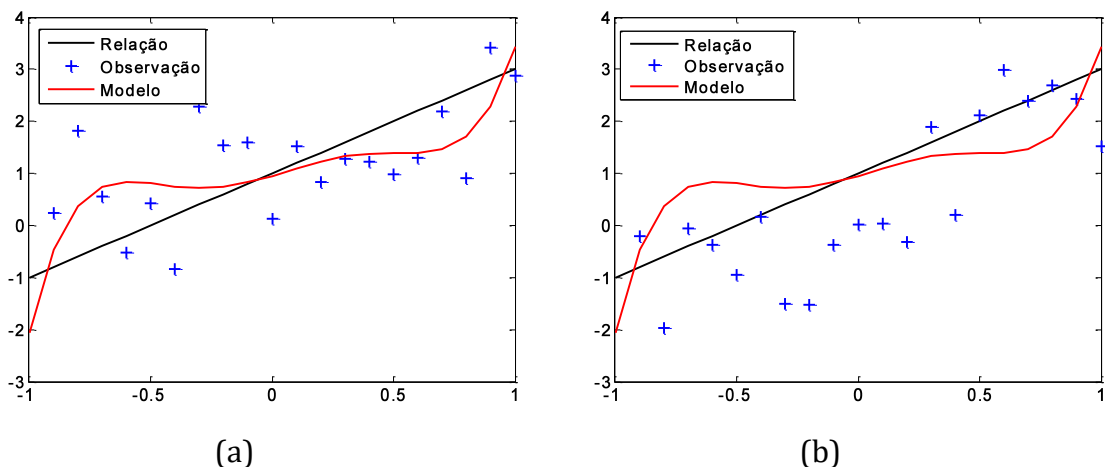


Figura 3.2: Exemplo de regressão linear com *overfitting*: (a) treinamento; (b) teste.

O *overfitting* é um efeito indesejado, pois, como as perturbações são aleatórias, uma nova amostra de teste pode produzir dados diferentes dos que foram utilizados para o

treinamento e, nesse caso, o erro produzido pelo modelo mais complexo é maior que aquele que seria obtido por um modelo mais próximo da relação ideal (cf. Figura 3.2b).

O raciocínio desenvolvido anteriormente para o modelo linear simples contém os elementos principais e resume o problema de regressão: como ajustar um modelo a partir de dados observados que seja o mais próximo possível da relação real entre as variáveis que geraram a amostra. A obtenção do modelo, portanto, depende da escolha adequada da forma da relação entre as variáveis de entrada e saída, que é a **estrutura** do modelo. Essa relação é geralmente representada por uma função matemática definida por um conjunto de parâmetros. O ajuste dos parâmetros do modelo é feito pela minimização do erro (ou resíduo) entre os valores observados e calculados pelo modelo da variável de saída no conjunto de treinamento. O método de ajuste de parâmetros da regressão linear é o método dos mínimos quadrados, apresentado na seção 3.1.2, mas antes definimos o modelo linear na próxima seção.

3.1.1 O Modelo linear

O modelo de regressão linear calcula uma estimativa $\hat{y}(t) \in \mathcal{R}$ da variável de saída a partir das variáveis de entrada $\mathbf{x}(t) \in \mathcal{R}^p$ e dos parâmetros do modelo como:

$$\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T = \sum_{i=1}^n \hat{x}_i(t)\theta_i \quad (3.1)$$

onde os elementos do vetor $\hat{\mathbf{x}}(t) \in \mathcal{R}^n$ são chamados **regressores** do modelo e o vetor de **parâmetros** $\boldsymbol{\theta} = (\theta_1, \dots, \theta_n)$. A notação $\hat{f}(\mathbf{x}(t), \boldsymbol{\theta})$ indica que a previsão de $\hat{y}(t)$ calculada no ponto $\mathbf{x}(t)$ depende dos parâmetros $\boldsymbol{\theta}$.

O modelo $\hat{f}(\mathbf{x}(t), \boldsymbol{\theta})$ é uma aproximação da função real $y(t) = f(\mathbf{x}(t))$, que relaciona as variáveis de entrada com a saída observada, que em geral é desconhecida.

O termo “linear” de regressão linear indica que o modelo é linear nos parâmetros, ou seja, que o valor de saída é calculado como uma combinação linear dos regressores. No modelo linear simples, ou linear de primeira ordem, os regressores são as próprias variáveis de entrada e o modelo é um hiperplano. Nesse caso, o vetor de regressores é $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$, onde o primeiro regressor é incluído para o ajuste do termo independente. O vetor de parâmetros tem dimensão $n = p + 1$ e o modelo escrito como:

$$\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}) = \theta_0 + \sum_{i=1}^p x_i(t) \theta_i \quad (3.2)$$

A formulação da regressão linear é geral o suficiente para acomodar outras estruturas de modelos mais complexos. Nesse caso, a formulação do modelo é uma combinação linear dos regressores, mas seu resultado é uma função não linear. De uma forma geral, o vetor de regressores é escrito como $\hat{\mathbf{x}}(t) = (1, h_1(\mathbf{x}(t)), \dots, h_n(\mathbf{x}(t)))$, onde $h_i(\mathbf{x}(t))$ são chamadas funções de base [238], como será visto no Capítulo 7.

Um exemplo de modelo linear que resulta em funções não lineares é o modelo polinomial, em que $h_i(\mathbf{x}(t)) = \mathbf{x}(t)^i$. O modelo linear polinomial de grau r é escrito na forma:

$$\hat{y}(t) = f(\mathbf{x}(t), \boldsymbol{\theta}) = \theta_0 + \sum_{i=1}^r \mathbf{x}(t)^i \boldsymbol{\theta}_i^T \quad (3.3)$$

onde o vetor de regressores $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t), \mathbf{x}(t)^2, \dots, \mathbf{x}(t)^r]$, desprezando-se os termos cruzados; $\boldsymbol{\theta}_i = (\theta_{i1}, \dots, \theta_{ip})$ é o vetor de parâmetros de cada termo do polinômio e o vetor de parâmetros do modelo $\boldsymbol{\theta} = [\theta_0, \boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_r]$, que tem $n = (r \cdot p) + 1$ elementos.

O ajuste dos parâmetros do modelo linear é realizado pela minimização do erro médio quadrático. O algoritmo de ajuste mais simples para regressão linear é o método dos mínimos quadrados, apresentado a seguir.

3.1.2 Método dos mínimos quadrados

O resultado do modelo geralmente não é igual ao valor observado da variável de saída e haverá sempre um erro ou resíduo $e(t) = y(t) - \hat{y}(t)$ (cf. Figura 3.1). Uma citação atribuída a George Box diz que “todos os modelos estão errados, alguns são úteis”.

A qualidade de um modelo de dados é medida por uma função de avaliação que mede a discrepância entre o valor observado e o valor calculado pelo modelo [98]. A função de avaliação utilizada em problemas de regressão linear é o erro médio quadrático (EMQ), calculado no conjunto de treinamento:

$$\text{EMQ}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t))^2 = \frac{1}{N} \sum_{t=1}^N (y(t) - \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}))^2 \quad (3.4)$$

O EMQ é uma medida de distância entre o valor observado e o valor predito pelo modelo em cada registro do conjunto de treinamento. Estatisticamente, o EMQ pressupõe que o resíduo $e(t) = y(t) - \hat{y}(t)$ tem distribuição normal, de média nula e variância constante. Essa hipótese considera implicitamente que o processo que gerou os dados é estacionário, ou seja, que as características estatísticas permanecem estáveis. Nesse caso, a minimização do EMQ corresponde a maximizar a função de verossimilhança entre o modelo e a variável de saída [84][399], como será visto na seção 9.2.2

Se as hipóteses são satisfeitas, o ajuste dos parâmetros do modelo é realizado pela minimização do EMQ, o que caracteriza o **método dos mínimos quadrados** (MMQ). O MMQ produz os parâmetros de mínima variância [238]. Entretanto, nem sempre todas as hipóteses são satisfeitas, mas, mesmo assim, o MMQ obtém bons resultados.

A apresentação do MMQ é mais elegante considerando a matriz de saída $\mathbf{Y}$, cujos elementos são os valores das variáveis de saída de todos os registros do conjunto de treinamento, e a matriz $\hat{\mathbf{X}}$, cujas linhas contêm os regressores dos registros do conjunto de treinamento. Nesse caso, o EMQ é escrito como:

$$EMQ(\boldsymbol{\theta}) = \frac{1}{N} (\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T)^T (\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T) \quad (3.5)$$

onde $\hat{\mathbf{X}} = \begin{bmatrix} \hat{x}_1(1) & \cdots & \hat{x}_n(1) \\ \vdots & \ddots & \vdots \\ \hat{x}_1(N) & \cdots & \hat{x}_n(N) \end{bmatrix}$, $\boldsymbol{\theta}^T = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$ e, no caso de saída simples, $\mathbf{Y} = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}$.

Os parâmetros que minimizam o EMQ são calculados a partir da derivada do EMQ em relação a $\boldsymbol{\theta}$, escrita como:

$$\frac{\partial EMQ(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -\frac{2}{N} \hat{\mathbf{X}}^T (\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T) \quad (3.6)$$

A solução é obtida igualando a derivada do EMQ a zero, o que resulta no seguinte sistema linear de equações, chamado de **equações normais** do MMQ:

$$\hat{\mathbf{X}}^T \hat{\mathbf{X}} \boldsymbol{\theta}^T = \hat{\mathbf{X}}^T \mathbf{Y} \quad (3.7)$$

Considerando que a matriz de regressores tem posto completo, isto é, $rank(\hat{\mathbf{X}}) = n$, a solução do sistema linear (3.6) pode ser calculada como

$$\boldsymbol{\theta}^* = (\hat{\mathbf{X}}^T \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^T \mathbf{Y} \quad (3.8)$$

onde a matriz $(\hat{\mathbf{X}}^T \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^T$ é chamada de **pseudoinversa** de $\hat{\mathbf{X}}$.

Os valores preditos pelo modelo para os registros do conjunto de treinamento podem ser calculados a partir da solução (3.8) como:

$$\hat{\mathbf{Y}} = \hat{\mathbf{X}} \boldsymbol{\theta}^* = \hat{\mathbf{X}} (\hat{\mathbf{X}}^T \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^T \mathbf{Y} = \mathbf{H} \mathbf{Y} \quad (3.9)$$

A matriz $\mathbf{H} = \hat{\mathbf{X}} (\hat{\mathbf{X}}^T \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^T$, de dimensão $N \times N$, é chamada de “matriz chapéu” [238][399], pois “coloca o chapéu” em $\mathbf{Y}$.

A solução do MMQ pela equação (3.8) é computacionalmente complexa, pois envolve o cálculo da inversa da matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$. Caso o posto da matriz de regressores não seja completo, a equação (3.8) pode se tornar instável numericamente. Há alternativas para o cálculo dos parâmetros do modelo linear que serão discutidas na seção 3.2.

3.1.3 Interpretação geométrica

O método dos mínimos quadrados tem uma interpretação geométrica mostrada na Figura 3.3. Suponha que cada registro observado da variável de saída representa uma coordenada no espaço N -dimensional, chamado de espaço amostral, de forma que o conjunto de registros da variável de saída define o vetor $\mathbf{Y} \in \mathcal{R}^N$. Cada coluna da matriz de regressores $\hat{\mathbf{X}} \in \mathcal{R}^{N \times n}$ também é um vetor no espaço amostral e as n colunas da matriz de regressores definem um subespaço de dimensão n , chamado subespaço do modelo, representado pela área em cinza na Figura 3.3.

Todos os modelos lineares $\hat{\mathbf{Y}} = \hat{\mathbf{X}} \boldsymbol{\theta}^T$, ou seja, todas as combinações lineares dos regressores, estão definidos no subespaço do modelo. A distância entre $\hat{\mathbf{Y}} = \hat{\mathbf{X}} \boldsymbol{\theta}^T$ e o vetor de observações $\mathbf{Y}$ é calculada como:

$$dist(\boldsymbol{\theta}) = \|\mathbf{Y} - \hat{\mathbf{X}} \boldsymbol{\theta}^T\| = \sqrt{(\mathbf{Y} - \hat{\mathbf{X}} \boldsymbol{\theta}^T)^T (\mathbf{Y} - \hat{\mathbf{X}} \boldsymbol{\theta}^T)} \quad (3.10)$$

A equação (3.10) mostra que o $EMQ(\boldsymbol{\theta}) = dist^2(\boldsymbol{\theta})/N$, de forma que a solução $\boldsymbol{\theta}^*$ que minimiza o EMQ produz o resíduo de menor norma L^2 . Dessa forma, o modelo deve ser ortogonal ao resíduo:

$$\hat{\mathbf{Y}}^T (\mathbf{Y} - \hat{\mathbf{X}} \boldsymbol{\theta}^*) = 0 \quad (3.11)$$

A equação (3.11) indica que o resíduo não deve ser correlacionado com os resultados do modelo. Na prática o EMQ não é nulo e a igualdade apresentada na equação (3.11)

não se verifica, mas esse resultado fornece um teste para validação do modelo de regressão linear. Outras estatísticas de validação serão discutidas na seção 3.3.

A Figura 3.3 mostra um exemplo da interpretação geométrica discutida no caso do de um modelo linear simples de uma amostra de dimensão $N = 3$ registros. O exemplo foi gerado com o conjunto de dados $\mathcal{T} = \{(0.2, 0.6), (0.5, 0.2), (0.7, 0.9)\}$. A matriz $\hat{\mathbf{X}} = [\mathbf{1}, \mathbf{X}]$ é a matriz de regressores que define o subespaço vetorial do modelo (em cinza), formado pelo vetor $\mathbf{1}$, cujos elementos são todos 1, e o vetor $\mathbf{X} = [0.2, 0.5, 0.7]^T$, cujos elementos são os registros da variável de entrada.

O resultado do ajuste do modelo pelo MMQ é o vetor de parâmetros $\boldsymbol{\theta}^* = [0.35, 0.44]^T$, que resulta na estimativa $\hat{\mathbf{Y}} = [0.44, 0.58, 0.67]^T$. Observe que o vetor de resíduos é ortogonal ao vetor de estimativas localizado no subespaço vetorial do modelo (em cinza).

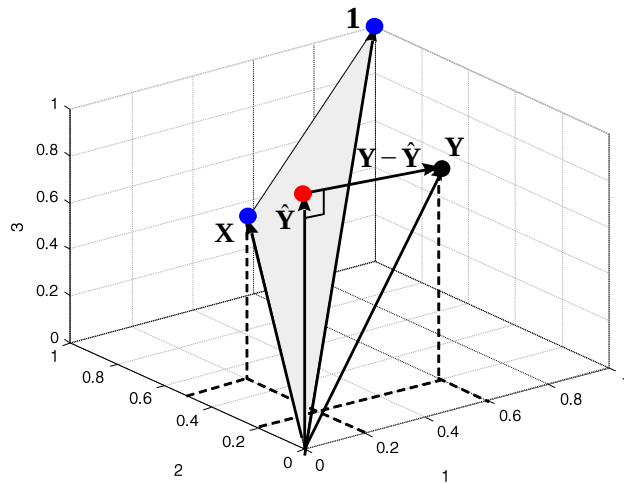


Figura 3.3: Interpretação geométrica do método dos mínimos quadrados.

A interpretação geométrica também mostra intuitivamente que a dimensão do espaço de modelo não pode ser maior que a dimensão do espaço amostral, isto é, $n < N$. Caso contrário, o modelo seria mais complexo que a própria amostra. A modelagem pode ser entendida como um processo de compactação da informação em que os N registros da amostra são representados pelos n parâmetros do modelo.

A dimensão da amostra em relação à dimensão do modelo é uma questão importante e que tem grande influência na qualidade do modelo. Quanto maior o tamanho da amostra, melhor é o desempenho do MMQ. Na prática, o número de registros de uma amos-

tra pode não ser muito grande quando há um custo associado à observação dos registros. Para obter uma amostragem relativamente densa em espaços de alta dimensionalidade, o número de registros necessários aumenta exponencialmente com o número de variáveis de entrada. Em espaços de alta dimensão, os dados se tornam altamente esparsos. Hastie, Tibshirani e Friedman [238] mostram um exemplo em que consideram uma amostragem uniforme num hipercubo de lado unitário. Nesse caso, para um problema de dimensão $p = 10$, é necessário utilizar 80% de cada dimensão para capturar 10% dos dados. Esse efeito é conhecido na literatura como a “maldição da dimensionalidade” [98][238].

Na próxima seção, apresentamos um exemplo simples no qual se discute o efeito da escolha da estrutura do modelo no resultado da modelagem.

3.1.4 Exemplo sintético

Neste exemplo, o conjunto de dados foi gerado a partir da função $y = x^2$, adicionando uma perturbação aleatória de média nula e desvio padrão 0.2. A Figura 3.4a mostra o conjunto de dados e a função $y = x^2$ no domínio da amostra.

Foram gerados três modelos a partir desse conjunto de dados: um modelo linear simples ou polinomial de grau $r = 1$; um polinomial de grau $r = 2$; e um polinomial de grau $r = 10$. Como nesse problema a variável de entrada tem apenas uma dimensão, a expressão do modelo polinomial da equação (3.3) se reduz a $\hat{y}(t) = \theta_0 + \sum_{i=1}^r x^i \theta_i$ e o modelo tem $n = r + 1$ parâmetros.

Os resultados do ajuste de cada um desses modelos são mostrados, respectivamente, nas Figura 3.4b, Figura 3.4c e Figura 3.4d. O modelo linear simples não representa bem a relação real, que é mais complexa. No caso do modelo polinomial $r = 10$, a complexidade do modelo é muito superior à da relação e seu resultado foi corrompido pela presença do ruído. No caso do modelo polinomial de grau $r = 2$, o resultado do modelo é muito próximo da função $y = x^2$ utilizada para gerar a amostra, pois o modelo foi ajustado com a estrutura correta.

A Tabela 3.1 mostra os parâmetros ajustados para cada um dos modelos. Podemos observar que o modelo linear simples é praticamente a média da variável de saída. No caso do modelo polinomial de grau $r = 2$, os parâmetros ajustados pelo MMQ correspondem aos da relação real até a primeira casa decimal. No caso do modelo polinomial

de grau $r = 10$, alguns parâmetros têm valores muito grandes em relação à escala da variável de saída, o indica que o ajuste realizado por esse modelo não foi adequado.

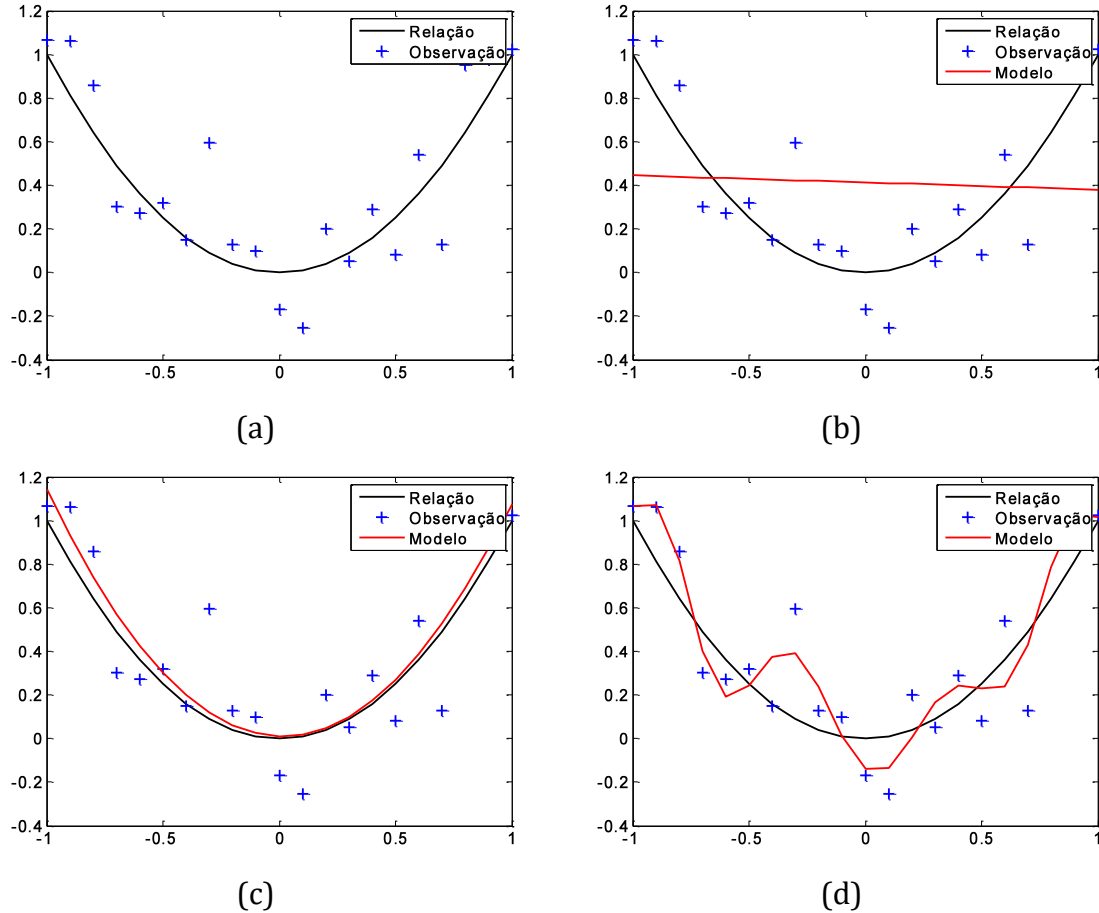


Figura 3.4: Exemplo de ajuste de parâmetros pelo MMQ: (a) conjunto de dados; (b) modelo linear puro; (c) modelo polinomial de grau 2; (d) modelo polinomial de grau 10.

O exemplo apresentado na Figura 3.4 mostra que o *overfitting* ocorre quando a complexidade do modelo é inadequada para a amostra. Entretanto, o MMQ pode ajustar adequadamente um modelo, mesmo de complexidade inadequada, quando o conjunto de dados for suficientemente grande. A Figura 3.5 mostra o resultado do modelo polinomial de grau 10 no caso de uma amostra com $N = 200$ e $N = 400$ registros.

Podemos observar que, mesmo com a complexidade do modelo sendo maior que a da relação real, um modelo adequado é ajustado com maior número de registros. Dessa forma, o *overfitting* ocorre quando a complexidade do modelo é maior que a da relação e, adicionalmente, o tamanho da amostra é pequeno em relação à complexidade do mo-

delo que se pretende ajustar. Há técnicas de combinação de modelos (chamadas de *ensembles*) como o *Boosting* [185][186] que exploram a ponderação do conjunto de dados de treinamento para o aumento de desempenho dos modelos. A relação entre a complexidade do modelo e o tamanho da amostra será explorada no Capítulo 7.

Tabela 3.1: Parâmetros ajustados dos modelos da Figura 3.4

θ	$r = 1$	$r = 2$	$r = 10$
θ_0	0.4131	0.0104	-0.1394
θ_1	-0.0325	-0.0325	-0.8151
θ_2		1.0983	8.455
θ_3			6.2056
θ_4			-53.2585
θ_5			-15.7255
θ_6			132.2678
θ_7			16.2839
θ_8			-133.545
θ_9			-5.9739
θ_{10}			47.2605

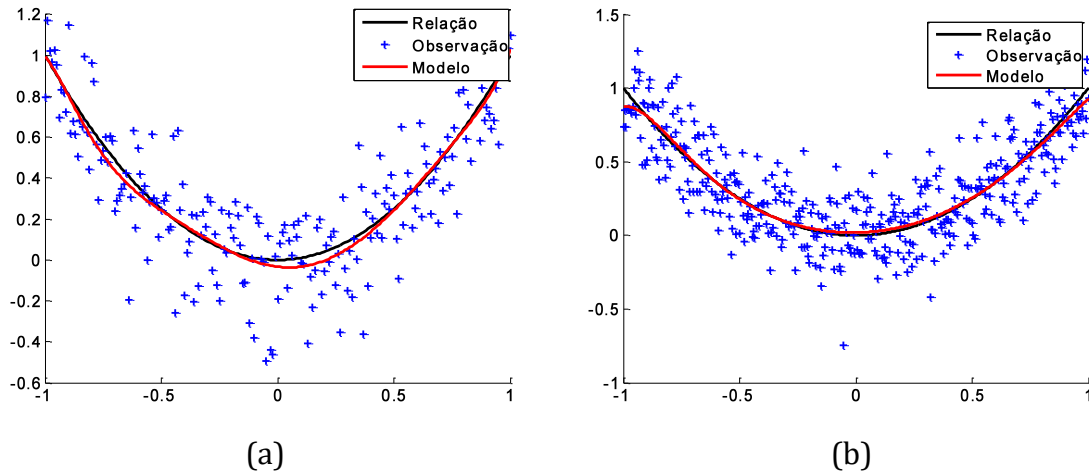


Figura 3.5: Efeito do número de registros no modelo de grau 10: (a) $N = 200$; (b) $N = 400$.

No MMQ, mesmo com uma amostra reduzida, é possível controlar a complexidade do modelo e encontrar uma solução estável para o sistema linear (3.7) através de técnicas de regularização, apresentadas a seguir.

3.2 Regularização

O desenvolvimento de um modelo em ciência de dados é, essencialmente, um processo de escolha, muitas vezes chamado de treinamento ou “aprendizado” [98], que pode ser resumido nas seguintes etapas:

1. Pré-seleção de um conjunto de funções $\mathcal{F}$ que pode conter um elemento capaz de ajustar o mapeamento entrada-saída representado pelo conjunto de treinamento. Em geral, esse passo é definido pela escolha do **tipo** de modelo.
2. Definição de um conjunto de hipóteses ou conhecimento *a priori* que estabelece restrições à família de funções $\mathcal{F}$ como solução do problema. Essas restrições visam limitar o espaço de soluções a um subconjunto $\mathcal{F}^* \subseteq \mathcal{F}$ e, em alguns casos, podem ser definidas através de parâmetros. Esse passo é chamado de seleção da **estrutura** do modelo.
3. Definição de uma medida (ou função) de avaliação da qualidade de uma instância $f \in \mathcal{F}^*$. A função de avaliação é utilizada na formulação de um **problema de otimização** cuja solução define a aproximação ótima f^* .
4. Utilização de um algoritmo de otimização para o problema formulado em 3) cuja solução são os **parâmetros ajustados** do modelo.

No caso do exemplo sintético da seção 3.1.4, o passo (1) corresponde à seleção do modelo polinomial de grau r . No passo (2), foram selecionadas três estruturas de modelos através da escolha do grau do polinômio. O processo indutivo associado ao MMQ é a minimização do EMQ e a solução é definida pela equação (3.8). Ao longo deste livro, alternativas para cada um desses passos serão apresentadas para que se possa encontrar a melhor solução em diferentes problemas.

Atualmente, o analista tem à sua disposição uma grande variedade de modelos em *softwares* livres e comerciais. Em muitos casos, o problema de *overfitting* não se mostra tão óbvio quanto no exemplo apresentado pela Figura 3.4d. Dessa forma, é importante entender as causas de *overfitting* para construir bons modelos, ou seja, modelos com boa capacidade de generalização.

A regularização é uma técnica que permite reduzir a complexidade do modelo para a obtenção de um modelo mais estável que permita melhor generalização para dados

diferentes dos que foram utilizados no ajuste de parâmetros. As técnicas de regularização estão fortemente associadas à relação *bias* e variância, apresentada a seguir.

3.2.1 *Bias* e variância

A estimativa de um modelo com *overfitting* depende da amostra utilizada para o ajuste do modelo. A parcela do EMQ (cf. equação (3.4)) calculada para um ponto qualquer $\mathbf{x}(t)$ do conjunto de treinamento é uma estimativa do valor esperado* do erro quadrático nesse ponto quando o número de amostras $N \rightarrow \infty$ [187]:

$$\lim_{N \rightarrow \infty} (y(t) - \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}))^2 = E \left((y_r(t) - \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}))^2 \right) \quad (3.12)$$

onde $y_r(t) = E(y(t))$ é o valor da função real em $\mathbf{x}(t)$.

A equação (3.12) é válida para qualquer registro do conjunto de treinamento e podemos escrever $y_r(t) = y_r$ e $f(\mathbf{x}(t), \boldsymbol{\theta}) = \hat{y}$ para simplificar a notação. O valor esperado do erro quadrático em $\mathbf{x}(t)$ pode ser decomposto em dois fatores [238] como:

$$\begin{aligned} E((y_r - \hat{y})^2) &= (y_r - E(\hat{y}))^2 + E \left((\hat{y} - E(\hat{y}))^2 \right) \\ &= \text{bias}^2 + \text{variância} \end{aligned} \quad (3.13)$$

O termo $(y_r - E(\hat{y}))$ é chamado ***bias*** teórico do modelo e o termo $E \left((\hat{y} - E(\hat{y}))^2 \right)$ é chamado de ***variância*** teórica do modelo. O *bias* é, então, a diferença entre o valor real da função em $\mathbf{x}(t)$ e o valor esperado da aproximação naquele ponto. A variância é o valor esperado da diferença entre uma estimativa do modelo e o valor esperado de todas as estimativas.

Uma aproximação do cálculo do valor esperado pode ser feita pela média em várias realizações da variável aleatória, o que permite uma visualização do efeito do *bias* e da variância nos resultados do ajuste dos modelos do exemplo da Figura 3.4. Para isso, 10 amostras de $N = 20$ registros foram geradas a partir da função $y_r = x^2 + \varepsilon$, onde o ruído ε foi gerado por uma distribuição normal $\varepsilon \sim N(0, 0.2)$. O *bias* e a variância em todos os pontos foram calculados pela decomposição (3.13). Para se ter uma ideia desses valores para cada modelo, foi calculada a média para todos os pontos, uma vez que os

* O valor esperado é definido como $E(x) = \int_{-\infty}^{\infty} xp(x)dx$, onde $p(x)$ é a função densidade de probabilidades da variável aleatória. O valor esperado pode ser aproximado pela média aritmética de todas as realizações da variável aleatória [400].

valores da variável de entrada permanecem os mesmos para todas as amostras. O resultado é mostrado na Figura 3.6a para o modelo linear simples e na Figura 3.6b para o modelo polinomial de grau 10. Nos dois gráficos da Figura 3.6, a curva verde apresenta a função real $y_r = x^2$, os pontos das amostras são mostrados em azul, o modelo ajustado $\hat{y}$ com cada amostra em vermelho e a média dos modelos entre todas as amostras em preto.

Os dois casos apresentam uma estrutura inadequada, como já foi discutido. O modelo linear simples apresenta o *bias* elevado, mas uma variância pequena, enquanto o modelo de grau 10 apresenta um *bias* pequeno e uma alta variância, mostrando que é mais sensível à amostra que o primeiro.

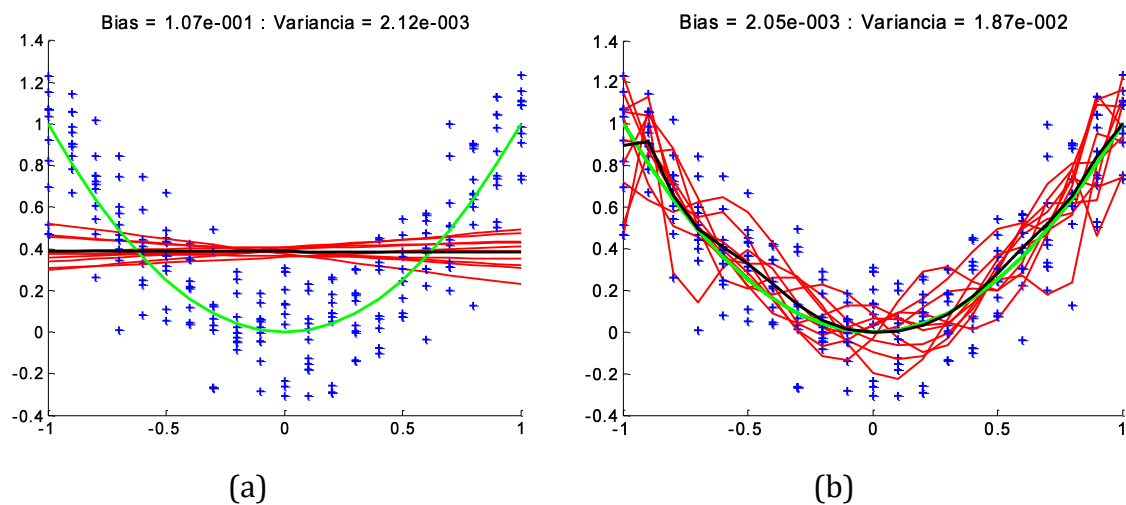


Figura 3.6: *Bias* e variância: (a) modelo linear simples; (b) modelo de grau 10.

Os resultados dos modelos ajustados com a estrutura correta, ou seja, um polinômio de grau 2, são mostrados na Figura 3.7. Podemos observar que o modelo com a estrutura correta é capaz de minimizar simultaneamente o *bias* e a variância e que a média de todos os modelos aproxima bem a função real.

A simulação anterior permite concluir que o *bias* acontece quando a complexidade do modelo é inferior à da função real e que a variância é o efeito da instabilidade de modelos muito complexos em relação ao tamanho da amostra. O objetivo das técnicas de regularização é a redução da variância de um modelo de alta complexidade para que o modelo se torne adequado à complexidade do problema, mantendo o *bias* baixo.

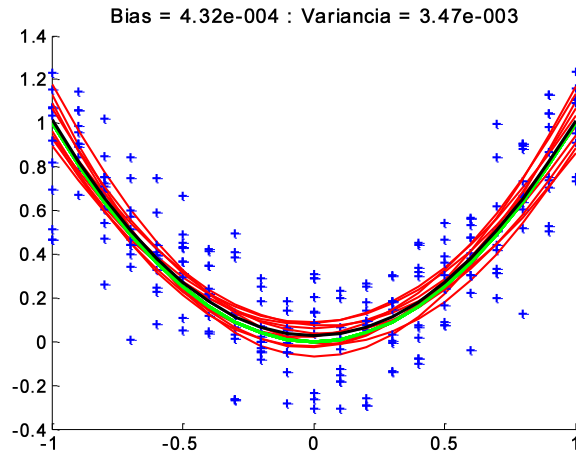


Figura 3.7: *Bias* e variância no modelo de grau 2.

A Figura 3.8 mostra o conceito de regularização como uma forma de reduzir a variância do modelo. A Figura 3.8 utiliza os elementos da decomposição do erro quadrático em *bias* e variância na equação (3.13). O círculo cinza no centro da figura representa o conjunto de valores possíveis da amostra, cujo centro é o valor real da função y_r . O valor observado na amostra difere do valor real devido ao erro de observação $e_o = y - y_r$. A partir de uma estrutura de modelo definida, o ajuste pelo MMQ produz uma estimativa $\hat{y}$, cuja diferença em relação ao valor observado é o resíduo do modelo $e = y - \hat{y}$. Considerando todas as amostras possíveis, a variância do modelo $E((\hat{y} - E(\hat{y}))^2)$ é representada pelo círculo laranja maior. Com essa estrutura, o modelo produzido pelo MMQ, considerando uma amostra infinita de dados, é o valor esperado do modelo $E(\hat{y})$. O *bias* do modelo é a diferença entre o valor real e o valor esperado do modelo.

A técnica de regularização visa a uma redução do espaço do modelo, que vai produzir uma estimativa de complexidade reduzida $\hat{y}'$ para o valor observado y . Essa estimativa, apesar de apresentar um resíduo do modelo $e = y - \hat{y}'$ maior, está mais próxima do resultado que seria obtido com uma amostra infinita $E(\hat{y}')$, de forma que a variância da estimativa do modelo reduzido $E((\hat{y}' - E(\hat{y}'))^2)$ é menor. O *bias* da estimativa do modelo reduzido pode ser maior, mas a soma EMQ que considera *bias* e variância pode ser menor para o resultado do modelo reduzido que para o resultado do modelo inicial.

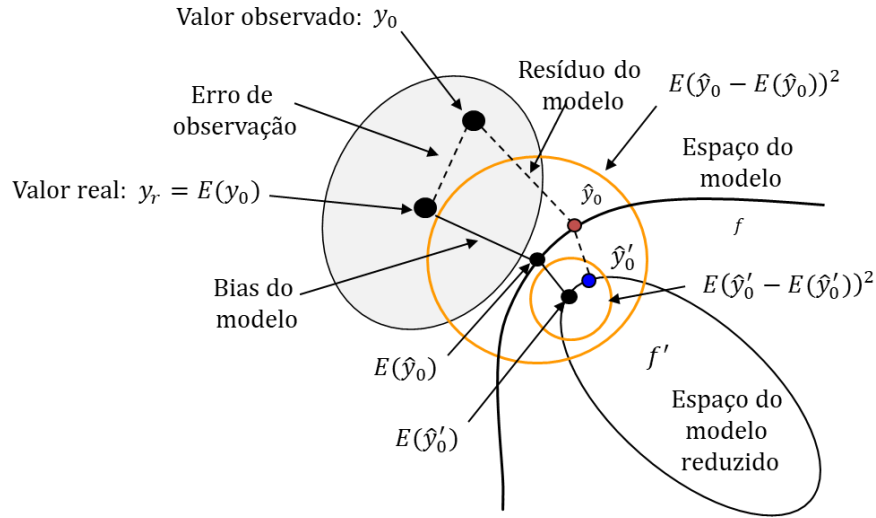


Figura 3.8: Conceito de regularização. Adaptado de [238].

O conceito de regularização descrito anteriormente será formalizado matematicamente em duas técnicas amplamente difundidas nas próximas seções.

3.2.2 Regularização de Tikhonov

Na regressão linear, o *overfitting* ocorre porque o modelo é muito complexo para o tamanho da amostra e o problema torna-se mal condicionado, ou seja, a solução do sistema linear (3.7) é muito sensível à amostra e torna-se instável. Para contornar esse problema, é necessário reduzir a complexidade do modelo, seja modificando diretamente a sua estrutura, seja aplicando algum tratamento numérico que permita o ajuste de um modelo menos complexo.

O método clássico de regularização foi proposto por Tikhonov para a solução de sistemas lineares mal condicionados [524]. Na estatística, o método de Tikhonov foi introduzido por Hoerl e Kennard [255] e chamado de *ridge regression*.

A regularização de Tikhonov é uma técnica de controle de complexidade do modelo através da aplicação de uma penalidade sobre o vetor de parâmetros. A solução é obtida pela minimização do erro médio quadrático regularizado (EMQR):

$$EMQR(\mu, \boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^N (y(t) - \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}))^2 + \mu \|\boldsymbol{\theta}\|^2 \quad (3.14)$$

onde $\mu \geq 0$ é o parâmetro de regularização que controla a aplicação da penalidade. Observe que, como $\mu \geq 0$, $EMQR(\mu, \boldsymbol{\theta}) \geq EMQ(\boldsymbol{\theta})$, o que é coerente com o raciocínio apresentado na Figura 3.8.

No modelo linear, o EMQR é escrito utilizando a notação vetorial como:

$$EMQR(\mu, \boldsymbol{\theta}) = \frac{1}{N} (\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T)^T (\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T) + \mu \boldsymbol{\theta}\boldsymbol{\theta}^T \quad (3.15)$$

onde $\hat{\mathbf{X}} = \begin{bmatrix} \hat{x}_1(1) & \cdots & \hat{x}_n(1) \\ \vdots & \ddots & \vdots \\ \hat{x}_1(N) & \cdots & \hat{x}_n(N) \end{bmatrix}$, $\boldsymbol{\theta}^T = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$ e $\mathbf{Y} = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}$, como na equação (3.5).

A minimização do EMQR seguindo o mesmo procedimento anterior resulta no sistema linear $(\hat{\mathbf{X}}^T \hat{\mathbf{X}} + \mu \mathbf{I}) \boldsymbol{\theta}^T = \hat{\mathbf{X}}^T \mathbf{Y}$, cuja solução é calculada como:

$$\boldsymbol{\theta}_\mu^* = (\hat{\mathbf{X}}^T \hat{\mathbf{X}} + \mu \mathbf{I})^{-1} \hat{\mathbf{X}}^T \mathbf{Y} \quad (3.16)$$

onde $\mathbf{I}$ é a matriz identidade.

A regularização de Tikhonov atua no condicionamento da matriz de coeficientes $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$ através do parâmetro de regularização μ .

O condicionamento de uma matriz $\mathbf{A}$ reflete a sensibilidade da solução de um sistema linear em que a matriz $\mathbf{A}$ é a matriz de coeficientes. Se a matriz $\mathbf{A}$ for mal condicionada, pequenas alterações no termo independente podem causar grandes variações na solução. O condicionamento de uma matriz quadrada $\mathbf{A}$ é medido pelo número de **condição**, definido como $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$, e calculado a partir dos seus autovalores [206]:

$$\kappa(\mathbf{A}) = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (3.17)$$

onde $\lambda_{\max}$ e $\lambda_{\min}$ são, respectivamente, o maior e o menor autovalor de $\mathbf{A}$.

A equação (3.17) mostra que, se $\lambda_{\min} \rightarrow 0$, o número de condição da matriz $\kappa(\mathbf{A}) \rightarrow \infty$ e a matriz tende a ser singular. O menor autovalor $\lambda_{\min} = 0$ indica que há colunas linearmente dependentes, ou seja, a matriz tem posto deficiente e o sistema linear é indeterminado. Um valor do número de condição grande, por exemplo, $\kappa(\mathbf{A}) > 1.000$, indica que o condicionamento da matriz é próximo do singular e que a solução do sistema linear é instável, sendo muito sensível a perturbações.

No caso da regressão linear, a matriz de coeficientes no MMQ (cf. equação (3.7)) é a matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$ e o termo independente, $\hat{\mathbf{X}}^T \mathbf{Y}$. A existência de regressores altamente correlacionados provoca o mau condicionamento da matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$ e resulta na instabilidade na solução, o que, no contexto do problema de regressão, significa *overfitting*. Em outras palavras, um modelo muito complexo (em relação ao tamanho da amostra) resulta em problemas de condicionamento na matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$ e instabilidade na solução. Esse efeito pode ser observado pelos valores do vetor de parâmetros $\boldsymbol{\theta}$ na solução para o modelo de grau 10 mostrado na Tabela 3.1.

Na regularização de Tikhonov, a adição de uma constante à diagonal principal da matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$ altera seu número de condição como:

$$\kappa(\mathbf{A}) = \frac{\lambda_{\max} + \mu}{\lambda_{\min} + \mu} \quad (3.18)$$

Dessa forma, mesmo que a matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$ seja singular e que $\lambda_{\min} = 0$, a matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}} + \mu \mathbf{I}$ será estável numericamente e a solução (3.16) poderá ser calculada adequadamente.

O controle do condicionamento da matriz de regressores foi a principal motivação para a introdução do método de Tikhonov na regressão linear [255]. Atualmente, o problema de regularização através da minimização do EMQR (3.14) tem sido tratado como uma técnica geral de controle da complexidade de modelos [98][238][478].

Na regularização de Tikhonov, o parâmetro de regularização deve ser informado ou calculado automaticamente [399]. Hoerl, Kennard e Baldwin [256] apresentam a seguinte proposta para o cálculo do parâmetro μ :

$$\mu = \frac{n}{(N - n)} \frac{\|\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T\|^2}{\|\boldsymbol{\theta}\|^2} \quad (3.19)$$

onde n é o número de parâmetros e $\boldsymbol{\theta}$ é a solução do MMQ sem regularização. O termo $\|\mathbf{Y} - \hat{\mathbf{X}}\boldsymbol{\theta}^T\|^2 / (N - n)$ é uma estimativa da variância do resíduo (cf. equação (3.30)).

A regularização de Tikhonov pode ser entendida num contexto de seleção de modelos através do ajuste do parâmetro de regularização μ , cuja variação permite uma regulação contínua de estrutura do modelo, o que pode ser interessante em alguns casos.

A Figura 3.9 apresenta o conjunto de dados do exemplo discutido na seção 3.1.4 com o resultado do ajuste do modelo polinomial de grau 10 pela minimização EMQR, com dois

valores para o parâmetro de regularização. A Figura 3.9a mostra o ajuste com o valor obtido pela equação (3.19) com $\mu = 1 \times 10^{-5}$ e a Figura 3.9b mostra o resultado obtido com o valor $\mu = 0.1$ escolhido de forma *ad hoc*.

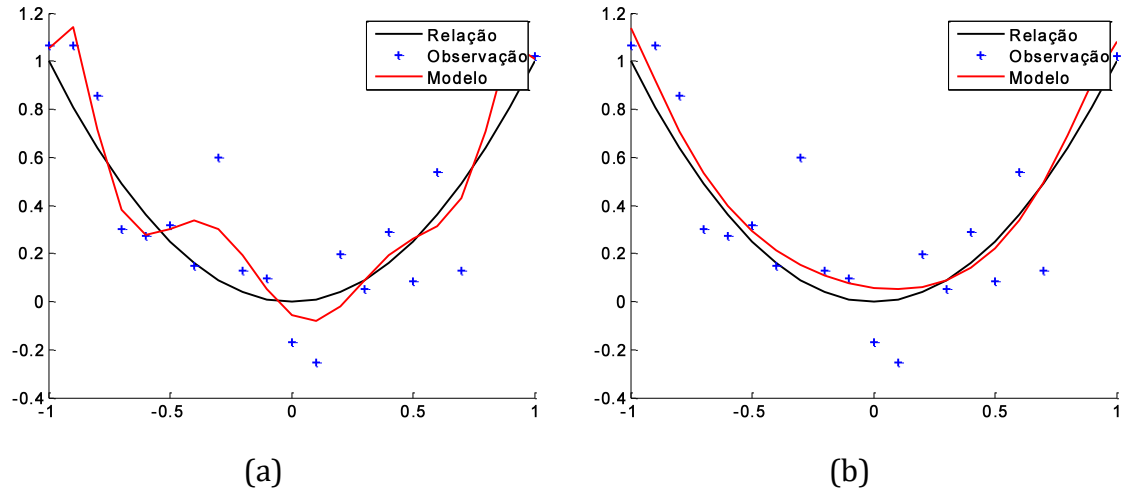


Figura 3.9: Modelo de grau 10 com regularização: (a) $\mu = 1 \times 10^{-5}$; (b) $\mu = 0.1$.

A Tabela 3.2 mostra os valores dos parâmetros. O resultado obtido sem regularização ($\mu = 0.0$) foi incluído para facilitar a comparação.

Tabela 3.2: Parâmetros ajustados dos modelos da Figura 3.9

θ	$\mu = 0.0$	$\mu = 1 \times 10^{-5}$	$\mu = 0.1$
θ_0	-0.1394	-0.0556	0.0567
θ_1	-0.8151	-0.7044	-0.1218
θ_2	8.4550	4.2082	0.6766
θ_3	6.2056	4.7143	0.1997
θ_4	-53.2585	-18.5496	0.4438
θ_5	-15.7255	-10.2949	0.0189
θ_6	132.2678	33.0619	0.2380
θ_7	16.2839	8.9882	-0.0538
θ_8	-133.5450	-18.2708	-0.0253
θ_9	-5.9739	-2.7254	-0.0693
θ_{10}	47.2605	0.6354	-0.2803

A solução robusta do MMQ também pode ser obtida evitando as componentes associadas aos autovalores próximos de zero, através do método das componentes principais.

3.2.3 Regularização por componentes principais

Na seção 2.4.2, vimos que a decomposição em valores singulares (SVD) permite eliminar colunas linearmente dependentes de uma matriz através de uma transformação linear ortogonal. A SVD permite uma solução robusta de sistemas lineares considerando apenas as colunas linearmente independentes da matriz de coeficientes.

Considere a SVD da matriz de regressores $\hat{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ aplicada ao ajuste de parâmetros do MMQ pela equação (3.8):

$$\boldsymbol{\theta}^* = (\hat{\mathbf{X}}^T \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^T \mathbf{Y} = (\mathbf{V}\mathbf{S}^2\mathbf{V}^T)^{-1} \mathbf{V}\mathbf{S}\mathbf{U}^T \mathbf{Y} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^T \mathbf{Y} \quad (3.20)$$

Uma solução estável do MMQ pode ser obtida utilizando apenas as $r \leq \text{rank}(\hat{\mathbf{X}})$ colunas linearmente independentes da matriz de regressores como:

$$\boldsymbol{\theta}_r^* = \sum_{i=1}^r \left(\frac{\mathbf{u}_i^T \mathbf{Y}}{s_i} \right) \mathbf{v}_i \quad (3.21)$$

onde $\mathbf{u}_i$ e $\mathbf{v}_i$ são as colunas de $\mathbf{U}$ e $\mathbf{V}$ e s_i é o valor singular correspondente.

A soma dos $r \leq \text{rank}(\hat{\mathbf{X}})$ termos associados aos maiores valores singulares evita a utilização de valores muito pequenos de s_i , que provocam a instabilidade da solução, como mostra a equação (3.21). Dessa forma, a solução (3.21) não utiliza todas as colunas de $\mathbf{U}$ e $\mathbf{V}$, tal forma que $\|\boldsymbol{\theta}_r^*\| < \|\boldsymbol{\theta}^*\|$. Ela pode, então, ser vista como a solução reduzida do MMQ e, portanto, uma técnica de regularização.

A decomposição SVD da matriz de regressores $\hat{\mathbf{X}}$ é equivalente à decomposição em autovalores e autovetores da matriz $\hat{\mathbf{X}}^T \hat{\mathbf{X}}$, como foi visto na seção 2.4.2. A solução da equação (3.21) corresponde à aplicação do MMQ aos componentes principais da matriz de regressores, o que dá nome ao método.

A regularização por componentes principais também pode ser utilizada como uma forma de seleção de modelos pela variação do parâmetro $r \leq \text{rank}(\hat{\mathbf{X}})$. Em muitos softwares, a solução (3.21) é utilizada como uma solução robusta do sistema linear (3.16) com $r \approx \text{rank}(\hat{\mathbf{X}})$, o que fica transparente para o usuário.

Para comparação com os resultados anteriores, a Figura 3.10 apresenta os resultados do ajuste do modelo polinomial de grau 10 pelo método de componentes principais. A Figura 3.10a mostra o ajuste com o valor obtido com $r = \text{rank}(\hat{\mathbf{X}}) = 9$ e a Figura 3.10b, o resultado obtido com o valor $r = 5$ escolhido de forma *ad hoc*.

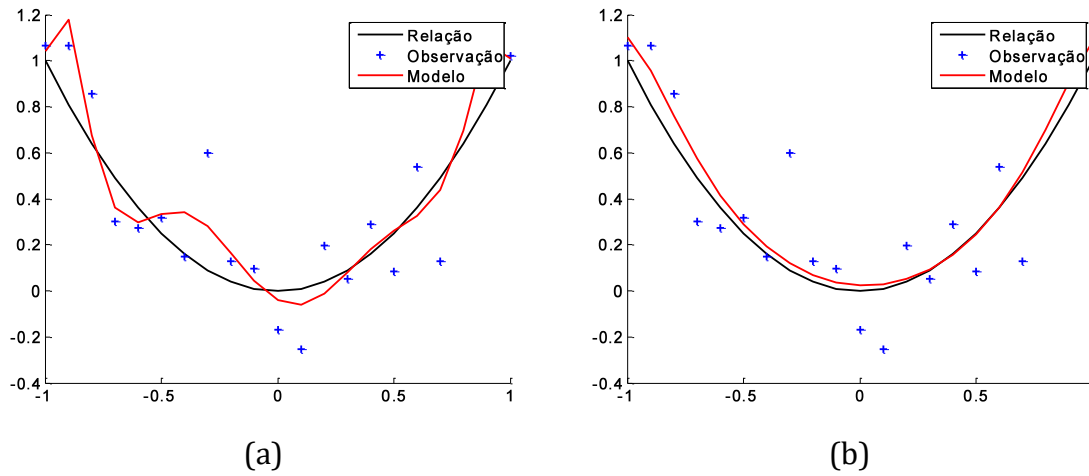


Figura 3.10: Modelo de grau 10 com regularização: (a) $r = 9$; (b) $r = 5$.

A Tabela 3.2 mostra os valores dos parâmetros. O resultado obtido sem regularização ($r = 11$) também foi incluído para facilitar a comparação.

Tabela 3.3: Parâmetros ajustados dos modelos da Figura 3.10

θ	$r = 11$	$r = 9$	$r = 5$
θ_0	-0.1394	-0.0408	0.0248
θ_1	-0.8151	-0.5505	-0.0447
θ_2	8.4550	3.3565	0.8601
θ_3	6.2056	2.6037	-0.0054
θ_4	-53.2585	-11.0903	0.4039
θ_5	-15.7255	-2.5260	0.0083
θ_6	132.2678	10.7787	0.1023
θ_7	16.2839	-1.5235	0.0146
θ_8	-133.5450	8.4170	-0.0871
θ_9	-5.9739	1.9788	0.0178
θ_{10}	47.2605	-10.3956	-0.2107

3.3 Validação de Modelos

A abordagem clássica para validação de modelos de regressão linear é o teste das hipóteses do MMQ [399]. O desempenho do modelo depende, principalmente, da escolha da melhor estrutura ou de parâmetros de regularização. Dessa forma, a seleção do modelo é feita a partir de estatísticas e procedimentos de validação.

As estatísticas de validação medem o desempenho do modelo num determinado conjunto de dados. A forma mais efetiva de avaliar a capacidade de predição do modelo é verificar as estatísticas de validação em dados diferentes daqueles utilizados para o ajuste dos parâmetros. A validação cruzada, utilizando as estatísticas de validação do modelo, é um método eficiente e prático para a avaliação da qualidade preditiva do modelo. As principais estatísticas de validação para regressão linear são apresentadas na próxima seção e a validação cruzada é apresentada na seção 3.3.2.

3.3.1 Estatísticas de validação

As estatísticas de validação são calculadas a partir do resíduo em cada registro:

$$e(t) = y(t) - \hat{y}(t), t = 1, \dots, N \quad (3.22)$$

A **análise de variância** do modelo é feita a partir da decomposição [399]:

$$\sum_{t=1}^N (y(t) - \bar{y})^2 = \sum_{t=1}^N (\hat{y}(t) - \bar{y})^2 + \sum_{t=1}^N (y(t) - \hat{y}(t))^2 \quad (3.23)$$

onde $\bar{y}$ representa a média da variável de saída.

A equação (3.23) pode ser escrita como $SS_T = SS_R + SS_{Res}$, onde SS_T é a soma de quadrados totais, que representa a variabilidade total da variável de saída; SS_R é a soma dos quadrados da regressão, que representa a porção da variabilidade total explicada pelo modelo; e SS_{Res} é a soma de quadrados dos resíduos, que representa a porção da variabilidade total que não pode ser explicada pelo modelo.

O **coeficiente de determinação** R^2 mede o percentual da variabilidade total que é explicada pelo modelo como [399]:

$$R^2 = \frac{SS_R}{SS_T} = \frac{\sum_{t=1}^N (\hat{y}(t) - \bar{y})^2}{\sum_{t=1}^N (y(t) - \bar{y})^2} = 1 - \frac{SS_{Res}}{SS_T} = 1 - \frac{\sum_{t=1}^N (y(t) - \hat{y}(t))^2}{\sum_{t=1}^N (y(t) - \bar{y})^2} \quad (3.24)$$

O coeficiente de determinação $R^2 \leq 1$ uma vez que SS_T é fixo e $SS_{Res} \leq SS_T$. O valor $R^2 \approx 1$ indica um resultado adequado pois a maior parte da variabilidade total é explicada pelo modelo. Intuitivamente, o coeficiente de determinação compara o erro do modelo estimado pelo erro da média da variável saída. Se o erro do modelo é da mesma ordem de grandeza do erro da média, o valor $R^2 \approx 0$ indica que o modelo não consegue

explicar as observações melhor que a média da variável de saída. Em alguns casos, o valor $R^2 < 0$ indica que o modelo é pior que a média da variável de saída.

O coeficiente de determinação é muito sensível aos valores mais altos da variável de saída uma vez que os erros são elevados ao quadrado. Para avaliação de valores menores, pode-se utilizar o coeficiente de determinação calculado sobre os logaritmos dos valores observados e preditos:

$$R_{log}^2 = 1 - \frac{\sum_{t=1}^N (\log(y(t)) - \log(\hat{y}(t)))^2}{\sum_{t=1}^N (\log(y(t)) - \overline{\log(y(t))})^2} \quad (3.25)$$

onde $\overline{\log(y(t))}$ é a média do logaritmo dos valores observados.

Uma estatística muito utilizada para avaliação de modelos é a raiz quadrada do EMQ, conhecida como RMS:*

$$RMS = \sqrt{\frac{1}{N} \sum_{t=1}^N e^2(t)} = \sqrt{\frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t))^2} \quad (3.26)$$

O resultado do RMS tem a mesma escala da variável de saída e pode ser analisado no contexto do problema.

Uma estatística de validação similar ao RMS é o erro absoluto médio (MAE)†:

$$MAE = \frac{1}{N} \sum_{t=1}^N |y(t) - \hat{y}(t)| \quad (3.27)$$

O erro absoluto médio percentual (MAPE)‡ é uma variante do MAE que apresenta os erros de forma relativa:

$$MAPE = \frac{1}{N} \sum_{t=1}^N \left| \frac{y(t) - \hat{y}(t)}{y(t)} \right| \quad (3.28)$$

A estatística MAPE é uma medida relativa e deve ser utilizada com cautela se a variável de saída for padronizada pela estimativa dos *z-scores*, pois pode resultar em erros de

* *Root Mean Square.*

† *Mean Absolute Error.*

‡ *Mean Absolute Percentual Error.*

arredondamento e, eventualmente, divisão por zero. Quando a variável de saída não contém o valor nulo, o resultado em forma percentual permite uma avaliação intuitiva da qualidade do modelo.

Uma estatística adimensional para avaliação do desempenho do modelo é a correlação entre os resultados do modelo e os valores observados da variável de saída:

$$\rho_{y\hat{y}} = \frac{\sum_{t=1}^N (y(t) - \bar{y})(\hat{y}(t) - \bar{\hat{y}})}{\sqrt{Var(y)}\sqrt{Var(\hat{y})}} \quad (3.29)$$

onde $\bar{\hat{y}} = \frac{1}{N} \sum_{t=1}^N \hat{y}(t)$ é a média dos resultados do modelo.

O valor de correlação $\rho_{y\hat{y}} \leq 1$; quanto mais próximo de 1, melhor a qualidade do modelo. O resultado de correlação também pode ser avaliado no gráfico de projeção entre o valor observado e o previsto da variável de saída.

Se a estrutura selecionada pelo modelo representa adequadamente a relação, uma estimativa da variância dos resíduos é calculada como:

$$Var(e) = \hat{\sigma}^2 = \frac{1}{N-n} \sum_{t=1}^N e^2(t) = \frac{1}{N-n} \sum_{t=1}^N (y(t) - \hat{y}(t))^2 \quad (3.30)$$

onde n é o número de regressores do modelo.

Quando a distribuição do resíduo pode ser representada adequadamente por uma distribuição normal, é possível calcular um intervalo de confiança do resultado do modelo para um valor de entrada arbitrário $\hat{\mathbf{x}}_0$ do vetor de regressores. A estimativa da variância do modelo no ponto $\hat{\mathbf{x}}_0$ é calculada como [399]:

$$Var(\hat{y}_0) = \hat{\sigma}^2 \hat{\mathbf{x}}_0^T (\hat{\mathbf{X}}^T \hat{\mathbf{X}})^{-1} \hat{\mathbf{x}}_0 \quad (3.31)$$

onde $\hat{y}_0$ é o valor do modelo no ponto $\hat{\mathbf{x}}_0$ e $\hat{\sigma}^2$ é a estimativa da variância do resíduo calculada pela equação (3.30).

O intervalo que contém o valor esperado da relação real $y_r = E(y_0)$, onde y_0 é o valor observado em $\hat{\mathbf{x}}_0$, com confiança estatística de $\xi\%$ é calculado como [399]:

$$\hat{y}_0 - t_{\xi,d} \sqrt{Var(\hat{y}_0)} \leq y_r \leq \hat{y}_0 + t_{\xi,d} \sqrt{Var(\hat{y}_0)} \quad (3.32)$$

onde $d = N - n$ é o número de graus de liberdade e $t_{\xi,d}$ é o valor da distribuição t de Student. A distribuição t pode ser aproximada pela normal para $d \geq 30$, de forma que o intervalo de confiança com 95% de significância pode ser aproximado por:

$$\hat{y}_0 - 2.0\sqrt{\text{Var}(\hat{y}_0)} \leq y_r \leq \hat{y}_0 + 2.0\sqrt{\text{Var}(\hat{y}_0)} \quad (3.33)$$

A Figura 3.11 mostra o resultado do modelo linear de duas das três variáveis de saída do problema *Contrete Slump*.¹⁰⁴ A Figura 3.11a mostra o resultado da variável de saída y_3 , em que o modelo tem um bom desempenho, e a Figura 3.11b, o resultado da variável y_1 , em que o desempenho do modelo não é bom.

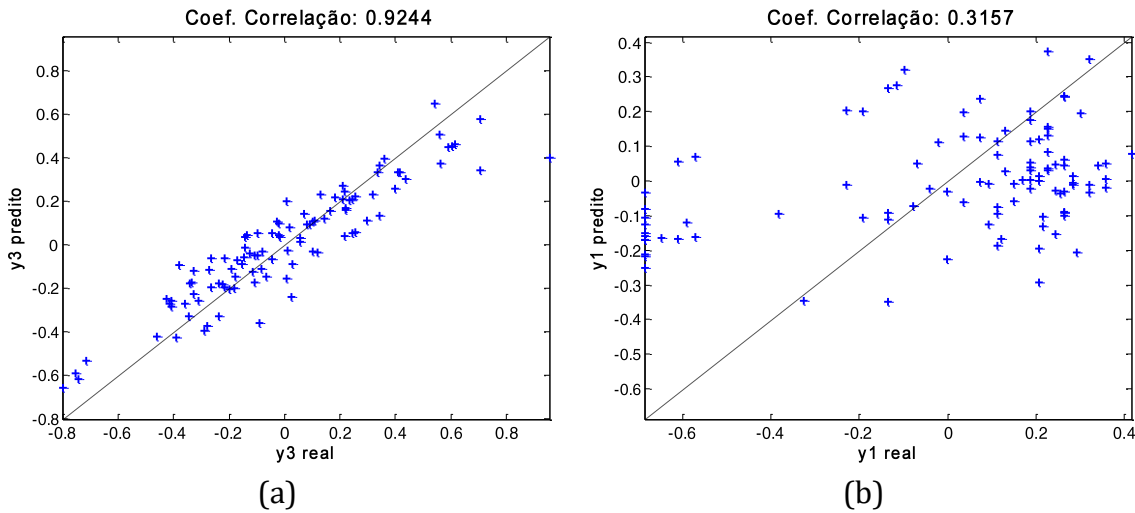
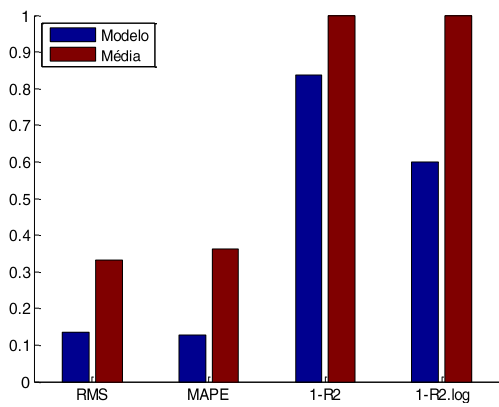


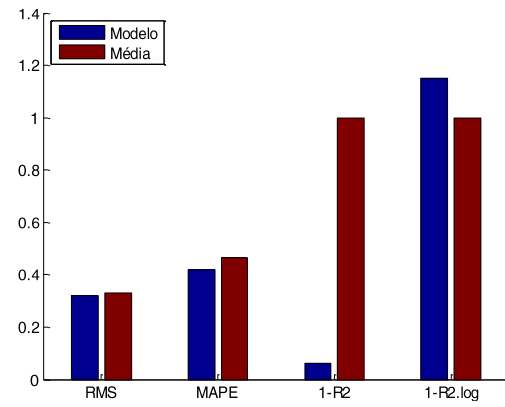
Figura 3.11: Resultados do modelo linear do problema *Contrete Slump*: (a) y_3 ; (b) y_1 .

A Figura 3.12 mostra as estatísticas de validação correspondentes. As estatísticas R^2 e R_{log}^2 são apresentadas subtraídas da unidade para que o melhor modelo tenha sempre o menor valor. As barras vermelhas indicam o resultado da previsão realizada pela média e servem como referência.

A comparação da distribuição dos resíduos com a distribuição normal $N(0, \hat{\sigma}^2)$ permite avaliar graficamente a qualidade do modelo. Resíduos com distribuição normal indicam que o ajuste realizado pelo MMQ foi de boa qualidade. A Figura 3.13 mostra os histogramas dos resíduos e os gráficos de probabilidade normal dos resultados apresentados na Figura 3.12 e na Figura 3.11.

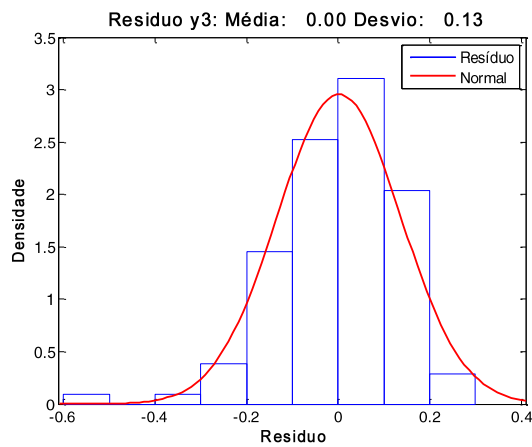


(a)

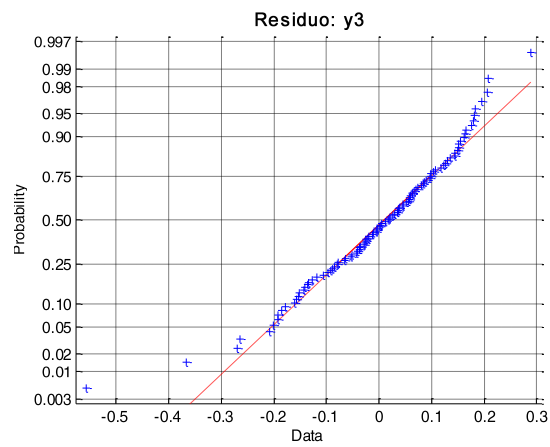


(b)

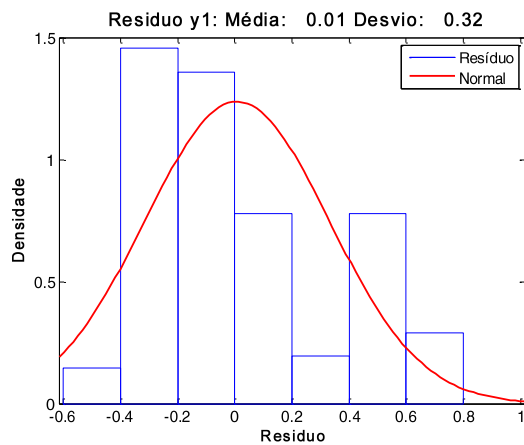
Figura 3.12: Resultados do modelo linear do problema *Concrete Slump*: (a) y_3 ; (b) y_1 .



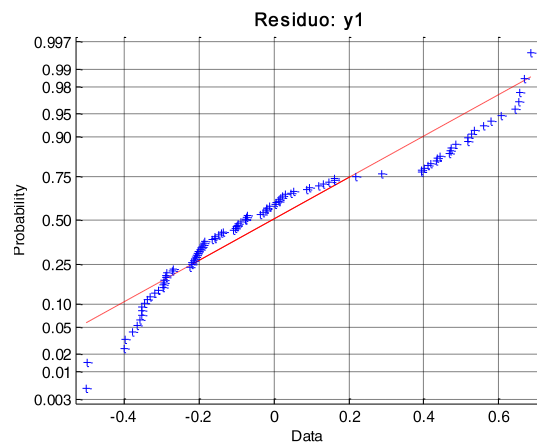
(a)



(b)



(c)



(d)

Figura 3.13: Distribuição do resíduo no problema *Slump*: (a) e (b) y_3 ; (c) e (d) y_1 .

As estatísticas de validação calculadas diretamente sobre o conjunto de treinamento não são úteis para a escolha do melhor modelo, pois um modelo com *overfitting* tem bom desempenho no conjunto de treinamento. A avaliação da capacidade de predição do modelo diretamente sobre o conjunto de treinamento pode ser avaliada por estatísticas como os critérios de Akaike (AIC) [7] e de Schwartz (BIC) [481], cujos resultados nem sempre correspondem ao desempenho efetivo do modelo. Na prática, utiliza-se o procedimento de validação cruzada para estimar a capacidade de predição do modelo, como descrito a seguir.

3.3.2 Validação cruzada

A maneira mais simples de estimar o desempenho do modelo num conjunto de dados diferente do que foi utilizado para o ajuste é dividir o conjunto de dados $\mathcal{T}$ em dois subconjuntos: o conjunto de treinamento $\mathcal{T}_1$ e o conjunto de teste $\mathcal{T}_2$, respectivamente com N_1 e N_2 registros (tipicamente $N_1 = 2N/3$ e $N_2 = N/3$). O conjunto de treinamento é utilizado para o ajuste de parâmetros e o conjunto de teste é utilizado para estimar o desempenho do modelo. Essa abordagem supõe que os dois conjuntos são amostras representativas do processo, o que pode não acontecer se o conjunto de dados não for grande o suficiente. Além disso, as estatísticas de validação ficam muito dependentes da partição e uma permutação dos registros pode gerar variações importantes nas estatísticas de validação.

A validação cruzada é um procedimento simples que permite avaliar adequadamente o desempenho do modelo em dados diferentes do conjunto de treinamento. Na validação cruzada, o conjunto de dados é dividido em K subconjuntos e a validação é realizada em K ciclos: para cada ciclo o modelo é ajustado utilizando $K - 1$ subconjuntos e avaliado no subconjunto remanescente. O procedimento é descrito pelo Algoritmo 3.1, mostrado esquematicamente na Figura 3.14.

As estatísticas de validação podem ser calculadas sobre o resultado de cada ciclo $\hat{\mathbf{Y}}_i$ e o resultado é apresentado em termos de média e desvio padrão dos resultados em todos os ciclos. Em conjuntos de dados pequenos, as estatísticas de validação não são significativas quando calculadas sobre um subconjunto com N/K registros. Dessa forma, é preferível concatenar os resultados em cada ciclo num vetor de resultados $\hat{\mathbf{Y}}$, que tem o mesmo número de registros que o conjunto de treinamento, e calcular as estatísticas

de validação sobre esses resultados. Observe que todos os elementos do vetor $\hat{\mathbf{Y}}$ foram calculados com registros diferentes daqueles utilizados para o ajuste dos modelos.

Algoritmo 3.1: Validação cruzada de K ciclos

Entrada:	Conjunto de dados $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$ e número de ciclos K
Saída:	Vetor com resultados do modelo $\hat{\mathbf{Y}} = [\hat{y}(1), \dots, \hat{y}(N)]^T$
01	Início
02	Dividir o conjunto $\mathcal{T}$ em K subconjuntos $\mathcal{T}_i, i = 1, \dots, K$ aproximadamente iguais
03	Para $i = 1 \dots K$:
04	Definir o conjunto de treinamento $\mathcal{T}_{TR} = \bigcup_{j \neq i} \mathcal{T}_j$
05	Ajustar os parâmetros do modelo utilizando $\mathcal{T}_{TR}$
06	Estimar o resultado do modelo no conjunto $\mathcal{T}_{TE} = \mathcal{T}_i$
07	Armazenar o resultado no vetor $\hat{\mathbf{Y}}_i$
08	Fim Para
09	Concatenar resultados $\hat{\mathbf{Y}} = [\hat{\mathbf{Y}}_1^T, \dots, \hat{\mathbf{Y}}_K^T]^T$
10	Fim

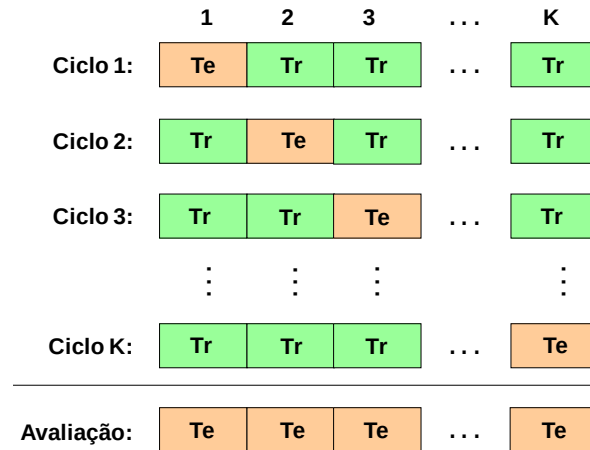


Figura 3.14: Validação cruzada de K ciclos: Tr são os subconjuntos de treinamento, Te são os subconjunto de teste

Uma escolha importante na validação cruzada é o número de ciclos K . No caso particular de $K = N$, chamada de *leave-one-out* (LOO), em cada ciclo, $N - 1$ registros são utilizados para o treinamento e a estimativa do modelo é feita com apenas um registro.

Apesar do custo computacional, a validação cruzada LOO é a melhor estimativa de desempenho do modelo em dados diferentes do conjunto de dados disponível, mas é muito sensível à amostra e pode ter resultados diferentes para amostras diferentes [238]. A escolha mais usual é $K = 5$ ou $K = 10$, número de registros [238][63].

3.4 Aplicações

3.4.1 Conjunto de dados *Pollution*

O conjunto de dados *Pollution*¹⁰⁵ é um problema utilizado frequentemente como exemplo da aplicação da regularização de Tikhonov [375]. O conjunto de dados, que relaciona poluição do ar com mortalidade, contém 60 registros com os valores de 15 variáveis observados entre 1959 e 1961 em áreas metropolitanas dos EUA. A variável de saída representa a média de mortalidade nas áreas observadas no mesmo período.

A Figura 3.15 mostra o *box-plot* das variáveis padronizadas pelas estimativas dos *z-scores* e a matriz de correlação. Podemos observar a presença de variáveis bastante correlacionadas (x_{12} e x_{13}).

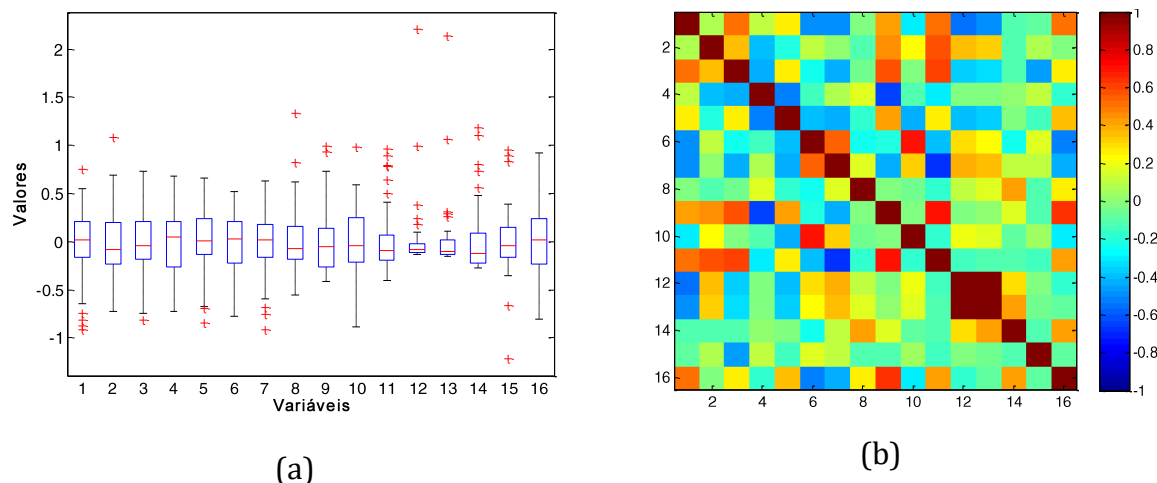


Figura 3.15: Conjunto de dados *Pollution*: (a) box-plot; (b) matriz de correlação.

A Figura 3.16 apresenta a visualização do gráfico de projeção e o *data image*. Podemos observar a existência de algumas variáveis com distribuições assimétricas (x_{12} a x_{13}) e a presença de pelo menos um registro *outlier*, representado pela linha e coluna em vermelho no *data image*.

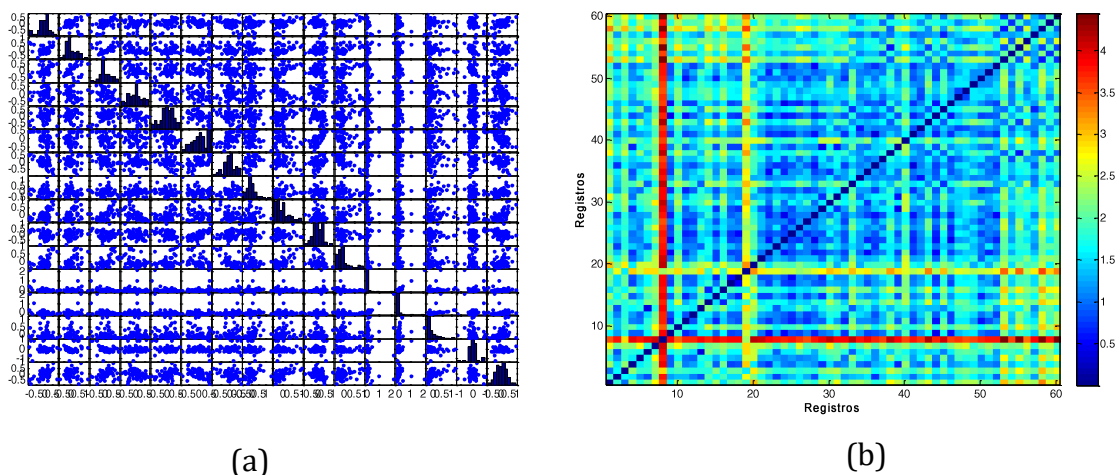


Figura 3.16: Conjunto de dados *Pollution*: (a) gráficos de projeção; (b) *data image*.

A Figura 3.17 apresenta o gráfico de projeção e o *data image* do conjunto de dados após a remoção de 12% (sete registros) da base, como descrito na seção 2.3.2. Podemos observar pela Figura 3.17b um padrão mais homogêneo da similaridade entre os registros, ainda que um registro se mostre discrepante em relação aos demais.

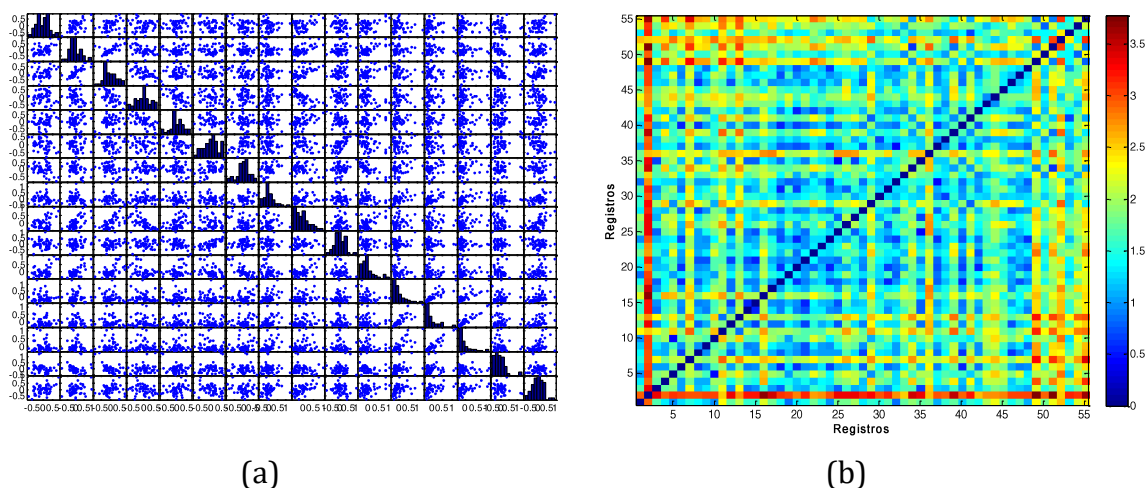


Figura 3.17: Conjunto de dados *Pollution* sem *outliers*: (a) gráficos de projeção; (b) *data image*.

A Figura 3.18 apresenta os resultados obtidos em validação cruzada do modelo linear simples com os parâmetros ajustados pelo MMQ sem regularização, sem a remoção de *outliers*. Podemos observar que a distribuição do resíduo não é bem ajustada pela distribuição normal, o que pode ser um indício de que o ajuste realizado pelo MMQ não foi adequado.

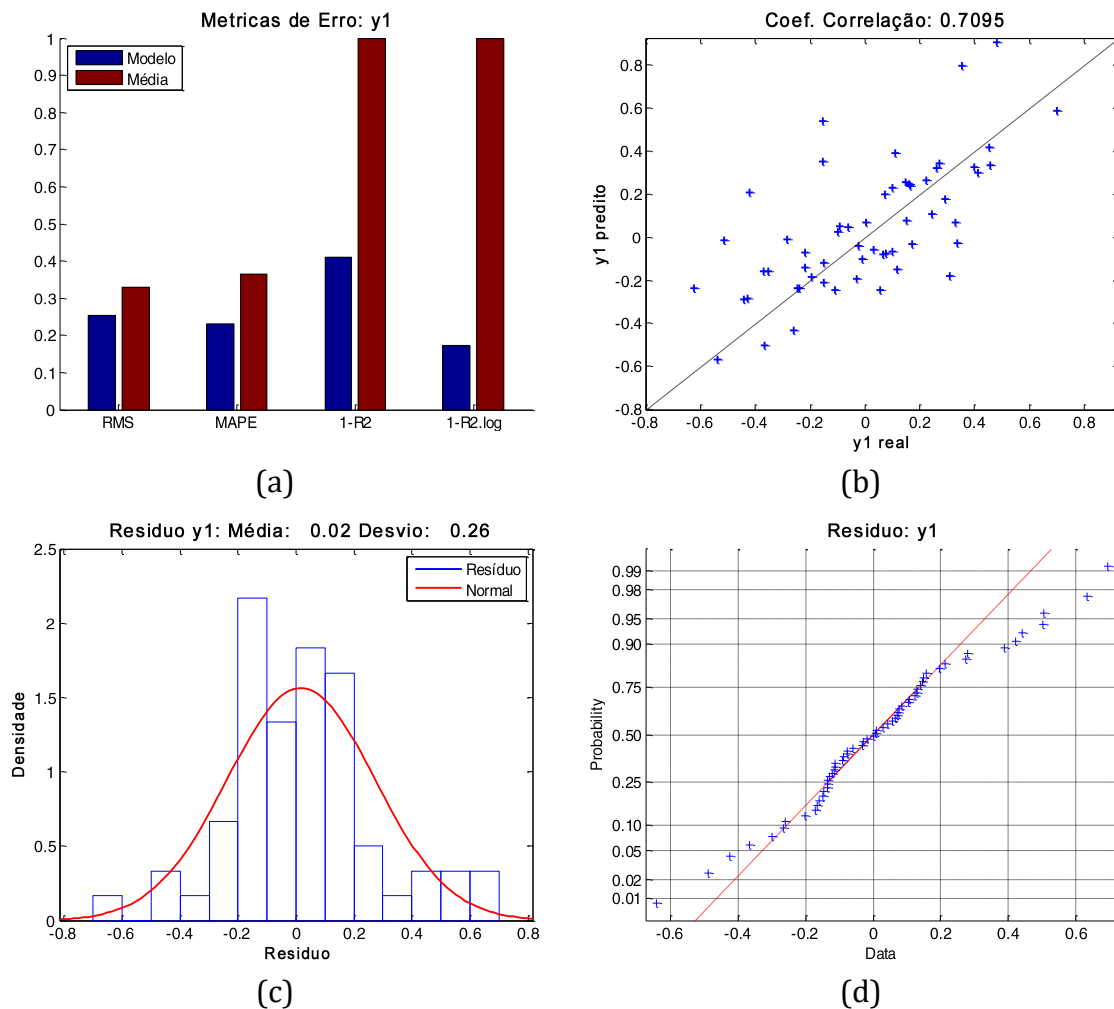


Figura 3.18: Resultados do MMQ no conjunto de dados Pollution. (a) Estatísticas de validação; (b) correlação real $\times$ predito; (c) histograma do resíduo; (d) probabilidade normal do resíduo.

A Figura 3.19 mostra os resultados do modelo linear aplicado ao conjunto de dados sem *outliers*. Podemos observar que o gráfico de probabilidade normal do resíduo indica que o ajuste dos dados sem *outliers* foi de melhor qualidade que o anterior. O coeficiente de correlação entre o valor real e o valor predito também demonstra o melhor desempenho do modelo nos dados sem *outliers*.

Os valores das estatísticas de validação são resumidos na

Tabela 3.4. O resultado também não melhora com a utilização de modelos polinomiais de ordem superior. O modelo linear regularizado com parâmetro calculado pela equação (3.19) ajustado nos dados sem *outliers* não resultou numa variação importante das

estatísticas. Finalmente, utilizando o valor $\mu = 0.01$ para o parâmetro de regularização nos dados sem *outliers*, obtemos uma pequena melhora nas estatísticas.

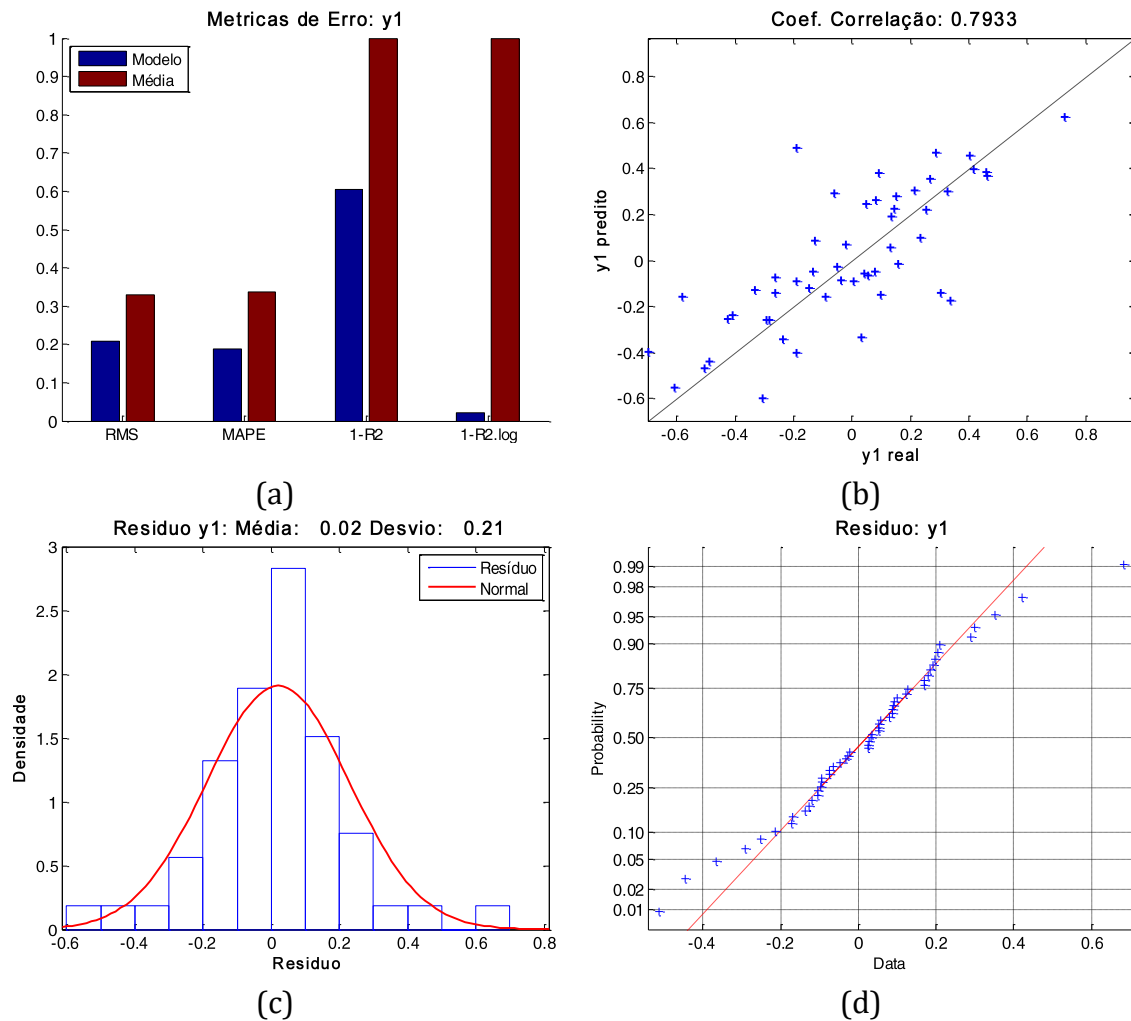


Figura 3.19: Resultados do conjunto de dados *Pollution* sem *outliers*: (a) estatísticas de validação; (b) correlação real $\times$ predito; (c) histograma do resíduo; (d) probabilidade normal do resíduo.

Tabela 3.4: Comparação das estatísticas de validação

Modelo	RMS	MAPE	R^2
MMQ	0.2537	22.98%	0.4107
MMQ s/ <i>outliers</i>	0.2078	18.93%	0.6039
MMQ Reg. c/ <i>outliers</i>	0.2316	20.52%	0.5091
MMQ Reg. s/ <i>outliers</i>	0.2094	18.99%	0.5976
MMQ Reg. $\mu = 0.01$	0.2057	18.51%	0.6118

Nesse exemplo, a regularização e a retirada de *outliers* aumentam o desempenho do modelo linear, mas o efeito das duas técnicas combinadas não produz uma alteração significativa no resultado. A retirada de *outliers* melhora os resultados das estatísticas de validação nesse exemplo, pois a remoção de *outliers* foi realizada sobre todo o conjunto de dados, antes da validação cruzada.

É importante observar que, mesmo para um exemplo simples, o número de opções de modelos diferentes pode aumentar muito se todas as alternativas forem exploradas durante o processo de validação de modelos.

3.4.2 Previsão da elevação adiabática de temperatura do concreto

O estudo apresentado nesta seção foi realizado em parceria com o Departamento de Engenharia Civil da Eletrobrás Furnas [174][166]. O trabalho foi realizado a partir dos registros de ensaios de elevação adiabática de temperatura realizados no Laboratório de Concretos de Furnas com corpos de prova de amostras de concretos utilizados na construção das principais barragens de usinas hidrelétricas brasileiras [18].

O principal componente do concreto é o cimento, que, em contato com a água, produz reação exotérmica, ganhando, assim, resistência mecânica. O ensaio de elevação adiabática de temperatura do concreto é um experimento controlado que mede a elevação de temperatura produzida por um corpo de prova de concreto. A temperatura final do ensaio é uma função principalmente da formulação e da concentração do cimento utilizado. Entretanto, os demais componentes da mistura exercem influência na forma da função de elevação de temperatura, que é não linear.

O conhecimento do calor gerado pelo concreto é importante na construção de estruturas de grande porte, como barragens, por exemplo. Os esforços gerados pela dilatação do material devido à transferência de calor podem ser excessivos, podendo causar rachaduras na estrutura já construída durante a concretagem de um novo bloco.

Os esforços gerados pela dilatação do concreto podem ser calculados a partir de modelos numéricos da estrutura. Entretanto, a previsão da quantidade de calor gerada por modelos analíticos é complexa e os modelos existentes para a previsão do calor gerado

só podem ser utilizados em condições limitadas [174]. Modelos de inteligência computacional podem ser utilizados para a previsão de calor gerado pelas misturas utilizadas na composição do concreto no Brasil [166].

Os dados para o estudo foram cedidos pelo Laboratório de Concreto de Furnas [174] e apresenta 136 registros de 14 variáveis de entrada do problema, apresentadas na Tabela 3.5. As variáveis são de dois tipos: cinco variáveis descrevem a composição química do cimento utilizado no concreto e nove variáveis, os demais elementos utilizados na mistura e a temperatura inicial do ensaio.

Cada registro do conjunto de dados original apresenta a evolução de temperatura do corpo de prova. Os registros representam observações a cada hora nas primeiras 24 horas e diariamente até o 28º dia, resultando 52 registros de tempo. Tendo em vista o grande número de valores ausentes e a dificuldade em modelar um problema com 52 variáveis de saída, a elevação de temperatura foi aproximada por uma função não linear de três parâmetros, que são utilizados como variáveis de saída no modelo de regressão [174].

Tabela 3.5: Descrição das variáveis de entrada

Variável	Descrição	
x_1	Concentração de CaO	(%)
x_2	Concentração de Si_2O_2	(%)
x_3	Concentração de Al_2O_3	(%)
x_4	Concentração de Fe_2O_3	(%)
x_5	Concentração de SO_3	(%)
x_6	Consumo de cimento	(Kg/m ³)
x_7	Finura Blaine	(cm ² /g)
x_8	Adição de pozolana	(Kg/m ³)
x_9	Adição de escória	(Kg/m ³)
x_{10}	Adição de cinza volante	(Kg/m ³)
x_{11}	Consumo de água	(Kg/m ³)
x_{12}	Consumo de aditivos	(Kg/m ³)
x_{13}	Consumo de agregados	(Kg/m ³)
x_{14}	Temperatura inicial	(°C)

A Figura 3.20a mostra a matriz de distâncias entre os registros. A Figura 3.20b apresenta a matriz de correlação das variáveis, onde são destacadas as variáveis de composição do cimento, as concentrações dos aditivos e as variáveis de saída. Os aditivos que têm maiores correlações com as variáveis de saída são: consumo de cimento (x_6), consumo de água (x_{11}) e consumo de agregados (x_{13}).

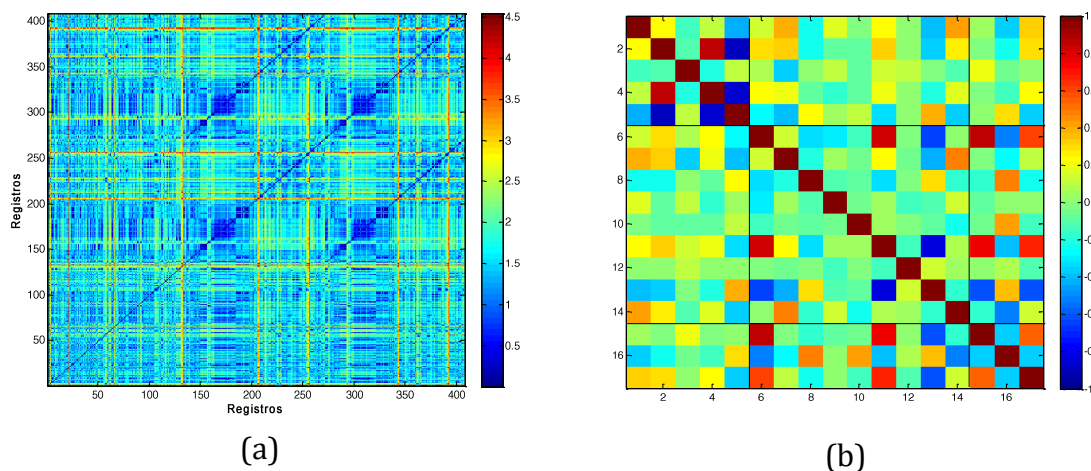
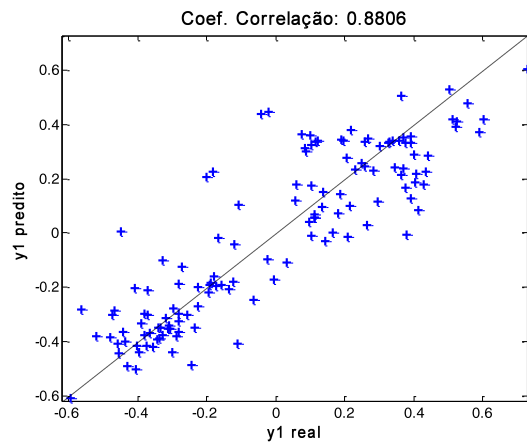


Figura 3.20: Variáveis de entrada. (a) matriz de distâncias; (b) matriz de correlação.

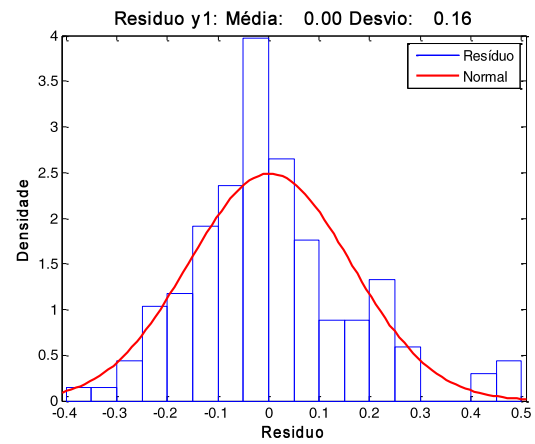
Os resultados do ajuste feito pelo MMQ simples e com regularização são mostrados na Tabela 3.6. O método com regularização apresenta resultados ligeiramente melhores, principalmente para a variável de saída y_3 . A variável de saída y_1 , que representa a temperatura final do concreto após a hidratação, é a que apresentou os melhores resultados, tendo em vista que seu comportamento é bastante correlacionado com o consumo de cimento, conforme apresentado no exemplo da análise de correlação canônica na seção 2.4.4. Os resultados do modelo MMQ com regularização são apresentados na Figura 3.21 e ilustram os comentários acima.

Tabela 3.6: Comparação das estatísticas de validação

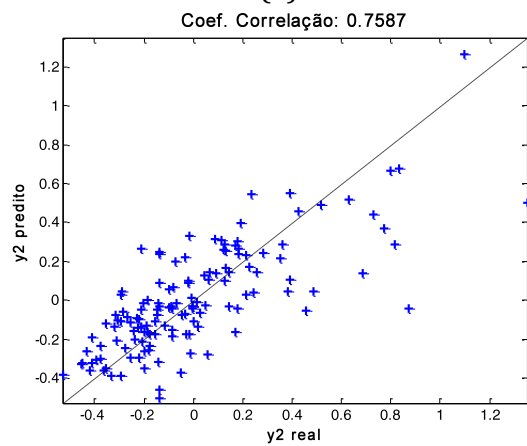
Saída	Modelo	RMS	MAPE	R^2
y_1	MMQ	0.1598	13.03%	0.7686
	MMQ Reg.	0.1573	13.07%	0.7756
y_2	MMQ	0.2178	16.21%	0.5700
	MMQ Reg.	0.2170	15.91%	0.5730
y_3	MMQ	0.2669	17.15%	0.3540
	MMQ Reg.	0.2591	16.45%	0.3911



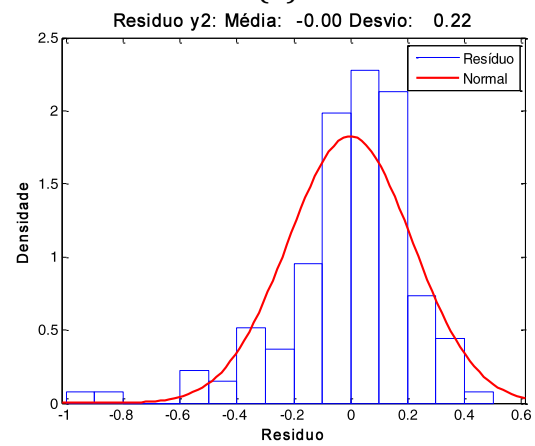
(a)



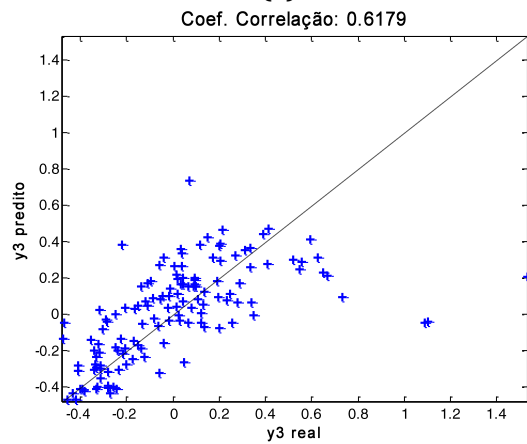
(b)



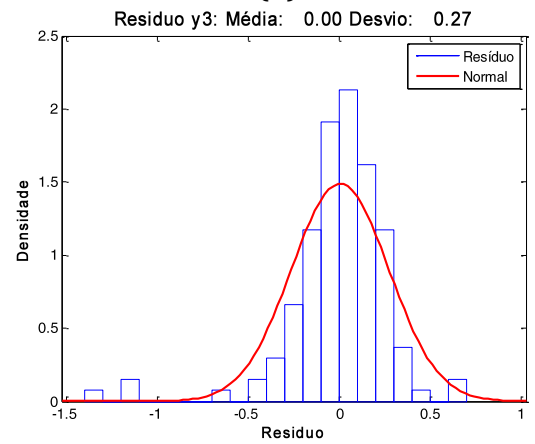
(c)



(d)



(e)



(f)

Figura 3.21: Resultados: (a) e (b) y_1 ; (c) e (d) y_2 ; (e) e (f) y_3 .

Os resultados foram obtidos com as variáveis de saída padronizadas e servem para a escolha da melhor estrutura e/ou abordagem de ajuste. As previsões do modelo devem

ser calculadas na escala original das variáveis. Além disso, as variáveis de saída representam os parâmetros de uma função de aproximação da elevação adiabática. A Figura 3.22 mostra o melhor e o pior ajuste realizados pelo modelo MMQ com regularização em relação aos valores observados de elevação de temperatura. A curva em azul mostra a aproximação realizada com os parâmetros utilizados como variável de saída e a curva em vermelho mostra a aproximação realizada com os parâmetros previstos pelo modelo. Podemos verificar que o pior ajuste é bem distante dos valores observados. Na Figura 3.22, o tempo é mostrado em escala logarítmica para facilitar a visualização da elevação de temperatura nas primeiras horas da reação de hidratação.

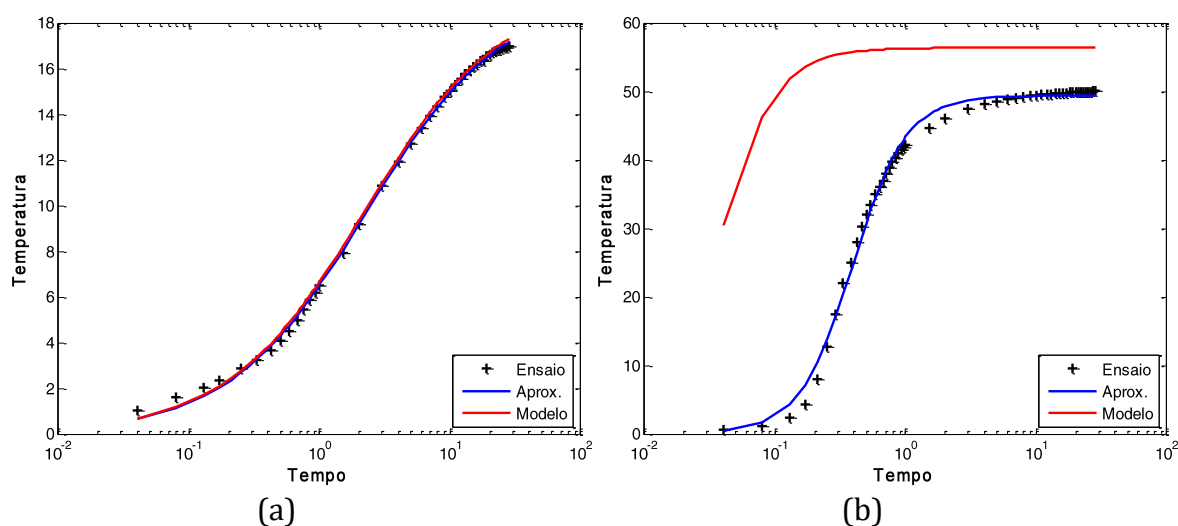


Figura 3.22: Resultados: (a) melhor ajuste; (b) pior ajuste.

O modelo de elevação adiabática de temperatura é complexo e altamente não linear [174]. Resultados mais precisos foram obtidos com modelos de redes neurais [174] e sistemas *fuzzy* [166].

3.5 Comentários Finais

Neste capítulo, apresentamos os principais conceitos de regressão linear pelo método dos mínimos quadrados. As principais características do processo de modelagem discutidas neste capítulo estão presentes em qualquer modelo preditivo.

O conceito mais importante introduzido neste capítulo é de que o processo de modelagem é, essencialmente, de escolha. O desenvolvimento do modelo segue três etapas principais: identificação da estrutura; ajuste de parâmetros e validação. A escolha da

estrutura é a etapa mais importante e a que tem maior influência sobre o resultado. O ajuste de parâmetros implica a solução de um problema de otimização, que, na regressão linear, é feito pelo método dos mínimos quadrados (MMQ). Tendo em vista os problemas numéricos que surgem da modelagem de sistemas complexos com um número reduzido de observações, as técnicas de regularização são úteis para garantir a estabilidade dos resultados. A validação do modelo visa predizer o seu comportamento em dados diferentes daqueles utilizados para o ajuste. O procedimento de validação cruzada é simples e efetivo para a obtenção de estatísticas de validação confiáveis.

Os resultados no problema *Pollution* mostram que a retirada de *outliers* resulta no aumento de desempenho dos modelos. É importante observar que esse procedimento deve ser utilizado com cautela, pois a retirada de valores extremos resulta na exclusão de valores com potencial para os maiores erros na regressão e, portanto, redução das estatísticas de validação. Entretanto, a retirada de *outliers* favorece que a distribuição do resíduo seja mais simétrica e, conseqüentemente, que as hipóteses do MMQ sejam confirmadas, resultando em modelos de melhor qualidade.

O problema de elevação adiabática de temperatura é complexo e ilustra como, na realidade, é importante realizar transformações nos dados existentes para a construção do conjunto de dados. Dessa forma, a capacidade do analista de encontrar alternativas é determinante para a obtenção de um bom resultado.

No próximo capítulo, os modelos lineares introduzidos neste capítulo serão utilizados para previsão de séries temporais e sistemas dinâmicos lineares.

4 MODELOS DINÂMICOS

Nos modelos de regressão linear discutidos no capítulo anterior os registros são considerados independentes, de forma que o resultado não se altera no caso de uma permutação entre os registros. Numa série temporal as observações realizadas em sequência no tempo em intervalos geralmente regulares. A característica intrínseca das séries temporais é que as observações são interdependentes no tempo. Essa dependência temporal entre observações permite o desenvolvimento de modelos lineares dinâmicos, para a predição de valores futuros da série.

Um sistema dinâmico é uma representação de um processo real em que as variáveis de saída estão relacionadas às variáveis de entrada pelo comportamento do sistema (cf. Figura 4.1). Numa série temporal, as variáveis de entrada não são observadas e o modelo pode ser ajustado a partir da variável de saída. Quando as variáveis de entrada e de saída são utilizadas para a modelagem, o modelo refere-se ao sistema dinâmico.



Figura 4.1: Sistema dinâmico.

Neste capítulo, apresentamos uma introdução à modelagem de séries temporais e sistemas dinâmicos focada nos modelos que podem ser formulados como modelos de regressão linear. O assunto é extenso e há obras clássicas na literatura, como os livros de Box, Jenkins e Reinsel [59] e de Ljung [352]. Os dois livros apresentam o assunto com

abordagens ligeiramente diferentes. Box, Jenkins e Reinsel [59] apresentam a abordagem estatística clássica e focam na análise e modelagem de séries temporais. Ljung [352] tem foco na modelagem de sistemas dinâmicos e utiliza uma terminologia da área de controle. Os algoritmos de identificação de sistemas descritos por Ljung estão implementados na *toolbox* de Identificação de Sistemas do Matlab.¹⁰⁶ Outras referências no assunto são os livros de Brockwell e R. A. Davis [65], Hamilton [230], que traz um enfoque voltado para econometria, e Pollock [441], que utiliza a terminologia de processamento de sinais e apresenta uma série de algoritmos codificados em Pascal e C. Em português, a obra de referência é o livro de Morentin e Toloi [404]. Este capítulo apresenta um apanhado das referências clássicas, buscando uma linguagem mais acessível ao cientista de dados não necessariamente especialista em séries temporais. Algumas especificidades técnicas foram suprimidas para facilitar o entendimento.

A modelagem de séries temporais e sistemas dinâmicos segue o fluxograma geral de mineração de dados apresentado na Figura 1.1. A caracterização estatística de séries temporais é realizada através da análise da função de autocorrelação, apresentada a seguir. Os modelos de séries temporais são descritos na seção 4.2 e os modelos de sistemas dinâmicos lineares, na seção 4.3. A seção 4.4 apresenta algumas aplicações.

4.1 Autocovariância e Autocorrelação

Os processos reais acontecem de forma contínua no tempo. Entretanto, os sistemas digitais de aquisição de dados realizam amostras em intervalos regulares de tempo, chamado de período de amostragem T . A série temporal é gerada por registros da variável do processo em múltiplos do período de amostragem T , como $y(t) = f(kT)$, onde k é um número natural, conforme ilustrado na Figura 4.2.

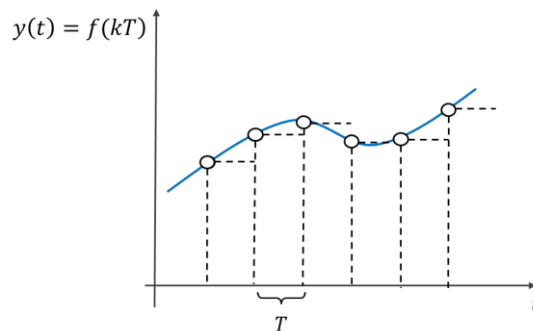


Figura 4.2: Amostragem de processo com período T .

Neste capítulo, consideramos o tratamento clássico de séries temporais e sistemas dinâmicos, em que o período de amostragem é constante [59]. Grande parte dos processos reais têm amostragem regular ou que pode ser considerada regular. Entretanto, a amostragem irregular acontece em algumas aplicações [11].

O período de amostragem do modelo não precisa ser o mesmo dos dados brutos, deve ser considerado em relação ao horizonte de previsão. Os modelos preditivos de séries temporais e sistemas dinâmicos realizam a previsão para alguns passos adiante. Para realizar previsões em horizontes maiores, é necessário realizar uma subamostragem ou calcular a média da série temporal para o ajuste do modelo na escala temporal representativa do tempo de resposta do sistema.

A análise de séries temporais desenvolvida nesta seção considera apenas processos **estacionários**. Um processo é estacionário se suas características estatísticas não se alteram em amostras distintas. Em processos estacionários, a distribuição de probabilidade de uma sequência $\{y(t), t = 0, \dots, N\}$ é idêntica à distribuição da sequência defasada de k registros $\{y(t + k), t = k, \dots, N + k\}$ [59]. Processos estacionários são também chamados de sistemas dinâmicos invariantes no tempo [352].

Uma série temporal é uma amostra do processo em estudo representada pelo conjunto de dados $\mathcal{T} = \{y(t), t = 1, \dots, N\}$, ou seja, não contém variáveis de entrada, apenas a variável de saída. A característica fundamental de uma série temporal é que as observações apresentam uma dependência linear medida pela covariância entre um registro $y(t)$ e outro registro $y(t + k)$ chamada de **autocovariância**. Em processos estacionários, a autocovariância entre dois registros separados de k instantes de tempo é a mesma para qualquer instante t e depende apenas do valor de k , chamado de atraso.

A autocovariância é uma característica do processo, mas que pode ser estimada a partir de uma amostra de dados $\mathcal{T}$. Em processos estacionários, a autocovariância é estimada para o atraso k como [59]:

$$\hat{\gamma}_y(k) = \frac{1}{N} \sum_{t=1}^{N-k} (y(t) - \bar{y})(y(t + k) - \bar{y}) \quad (4.1)$$

onde $\bar{y} = \frac{1}{N} \sum_{t=1}^N y(t)$ é a estimativa da média da série temporal.

Analogamente, a **autocorrelação** entre dois registros $y(t)$ e $y(t + k)$ é um valor adimensional, calculada para o atraso k como:

$$\hat{\rho}_y(k) = \frac{\hat{\gamma}_y(k)}{\hat{\gamma}_y(0)} = \frac{\sum_{t=1}^{N-k} (y(t) - \bar{y})(y(t+k) - \bar{y})}{\sum_{t=1}^N (y(t) - \bar{y})^2} \quad (4.2)$$

O conjunto dos valores de autocorrelação para todos os valores de atraso é chamado função de autocorrelação (ACF)* ou **correlograma**. Os valores de autocorrelação são simétricos em relação à $k = 0$, isto é, $\hat{\rho}_y(k) = \hat{\rho}_y(-k)$ e $\hat{\rho}_y(0) = 1$. Dessa forma, apenas os valores positivos da função de autocorrelação são apresentados em gráficos como o da Figura 4.3.

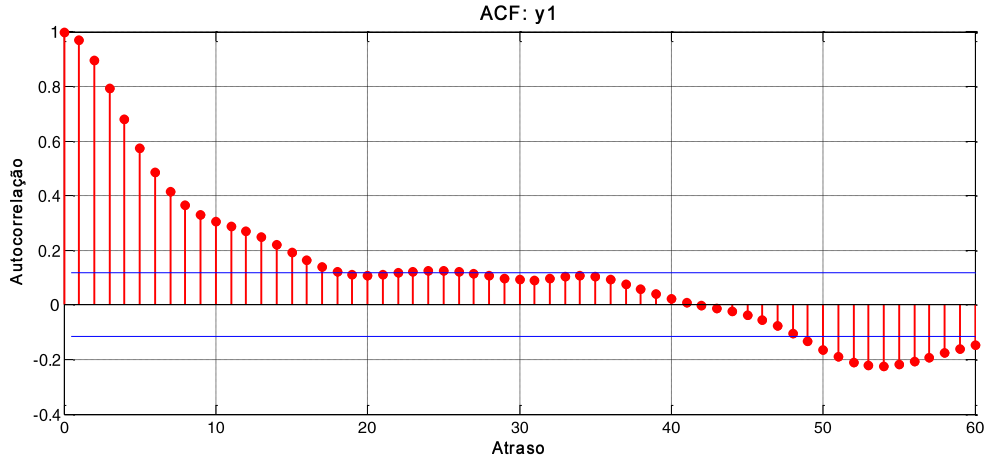


Figura 4.3: Função de autocorrelação.

As linhas azuis que aparecem no gráfico da Figura 4.3 representam o intervalo de confiança em que o valor de autocorrelação pode ser considerado nulo. Com 95% de confiança, os limites são calculados como $\pm 2/\sqrt{N}$.

Em sistemas dinâmicos em que as entradas e saídas do sistema são observadas, é possível calcular a função de **covariância cruzada** [59]:

$$\hat{\gamma}_{xy}(k) = \begin{cases} \frac{1}{N} \sum_{t=1}^{N-k} (x(t) - \bar{x})(y(t+k) - \bar{y}), & k = 0, 1, 2, \dots \\ \frac{1}{N} \sum_{t=1-k}^N (y(t) - \bar{y})(x(t+k) - \bar{x}), & k = 0, -1, -2, \dots \end{cases} \quad (4.3)$$

* *AutoCorrelation Function.*

Observe que $\hat{\gamma}_{xy}(k) \neq \hat{\gamma}_{yx}(k)$, mas como $\hat{\gamma}_{xy}(k) = \hat{\gamma}_{yx}(-k)$, a equação (4.3) utiliza a parte positiva e negativa da escala de atrasos para mostrar, respectivamente, $\hat{\gamma}_{xy}(k)$ e $\hat{\gamma}_{yx}(k) = \hat{\gamma}_{xy}(-k)$.

Analogamente à equação (4.2), a **função de correlação cruzada (XCF)** * é calculada como:

$$\hat{\rho}_{xy}(k) = \frac{\hat{\gamma}_{xy}(k)}{\sqrt{\hat{\gamma}_x(0)}\sqrt{\hat{\gamma}_y(0)}} = \frac{\hat{\gamma}_{xy}(k)}{\hat{\sigma}_x\hat{\sigma}_y}, k = 0, \pm 1, \pm 2, \dots \quad (4.4)$$

onde $\hat{\sigma}_x = \frac{1}{N}\sqrt{\hat{\gamma}_x(0)} = \frac{1}{N}\sqrt{\sum_{t=1}^N(x(t) - \bar{x})^2}$ e $\hat{\sigma}_y = \frac{1}{N}\sqrt{\hat{\gamma}_y(0)} = \frac{1}{N}\sqrt{\sum_{t=1}^N(y(t) - \bar{y})^2}$.

A função de correlação cruzada mostra a correlação entre as variáveis de entrada e saída do sistema em relação aos valores de atraso. A Figura 4.4 mostra as variáveis do problema *Gas Furnance*¹⁰⁷ [59] e a Figura 4.5, a função de correlação cruzada.

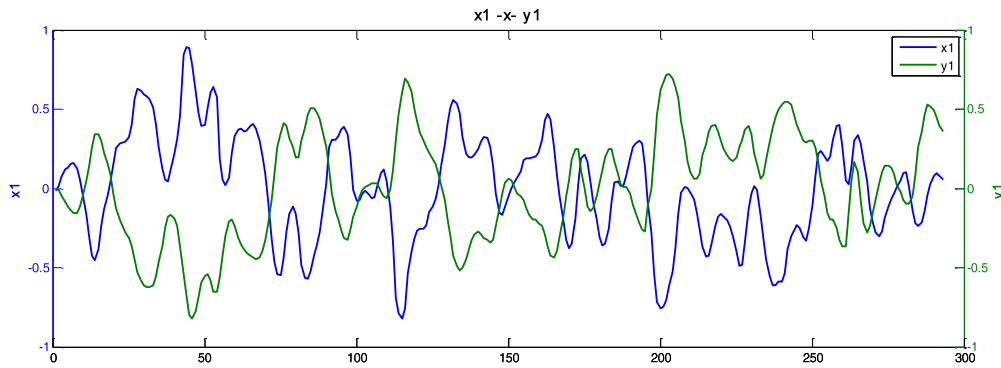


Figura 4.4: Séries temporais de variáveis de entrada e saída do problema *Gas Furnance*.

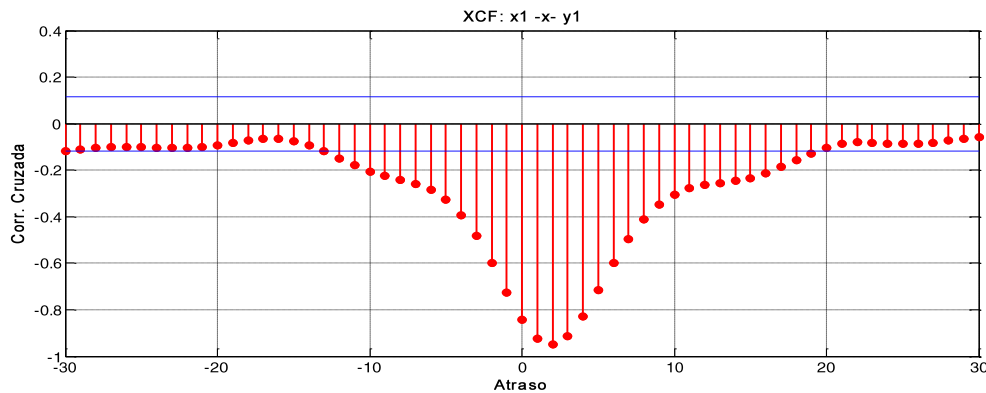


Figura 4.5: Função de correlação cruzada do problema *Gas Furnance*.

* Cross Correlation Function.

Podemos observar que a função não é simétrica em relação a $k = 0$ e que o valor mínimo de correlação acontece em $k = 2$, que é o atraso da saída em relação à entrada. Além disso, a correlação entre as variáveis de entrada e saída é negativa, como pode ser observado na Figura 4.4.

As funções de autocorrelação e de correlação cruzada permitem identificar as características dinâmicas do processo em estudo. Os modelos de séries temporais e sistemas dinâmicos são descritos a seguir.

4.2 Modelos de Séries Temporais

O modelo mais simples de processo estacionário é o **ruído branco**, que é uma variável aleatória independente, de média nula e variância constante $\hat{\sigma}_e$. Para o desenvolvimento desta seção, vamos supor que o ruído branco tenha distribuição normal $N(0, \hat{\sigma}_e)$. No ruído branco, os registros da série temporal são completamente independentes, de forma que a sua função de autocorrelação é mostrada na Figura 4.6.

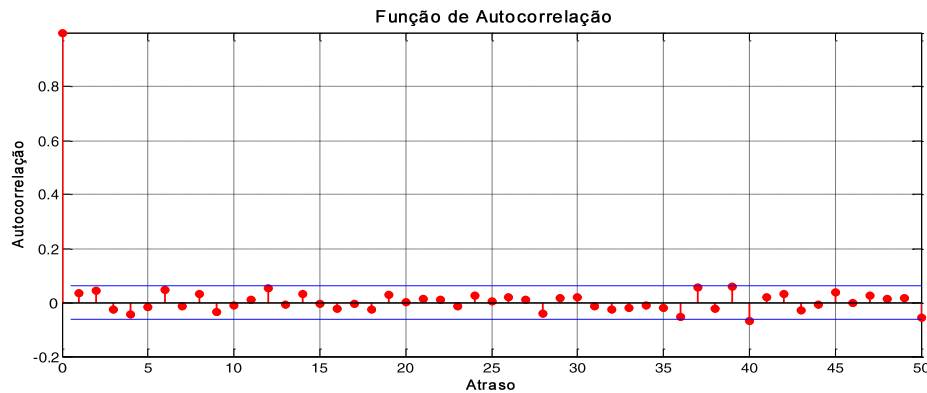


Figura 4.6: Função de autocorrelação do ruído branco.

O ruído branco tem um papel fundamental na modelagem de séries temporais e sistemas dinâmicos lineares.

Um sistema dinâmico é linear se a resposta do sistema a uma entrada formada pela soma de duas variáveis aleatórias é igual à soma das respostas do sistema a cada variável aleatória isolada:

$$f(x(t)) = f(x_1(t)) + f(x_2(t)) \quad (4.5)$$

onde $x(t) = x_1(t) + x_2(t)$.

Essa propriedade é chamada de **princípio da superposição** e permite a representação de uma série temporal estacionária de média nula como a saída de um filtro linear cuja entrada é um ruído branco:

$$y(t) = \sum_{k=1}^{\infty} \psi_k e(t - k) + e(t) \quad (4.6)$$

onde $e(t)$ é um ruído branco e $\psi_k, k = 1, \dots, \infty$ são coeficientes constantes tais que $\sum_{k=1}^{\infty} |\psi_k| < \infty$.

Analogamente, uma série temporal estacionária de média nula pode ser representada como uma combinação linear de valores anteriores:

$$y(t) = \sum_{k=1}^{\infty} \phi_k y(t - k) + e(t) \quad (4.7)$$

onde $\phi_k, k = 1, \dots, \infty$ são coeficientes constantes e $\sum_{k=1}^{\infty} |\phi_k| < \infty$.

As representações (4.6) e (4.7) são equivalentes e muito importantes teoricamente, pois servem de base para o desenvolvimento de modelos de séries temporais.

4.2.1 Modelo AR

A equação (4.7) não é muito útil na prática, pois envolve uma soma infinita. O modelo **autorregressivo** (AR) considera apenas um número finito de termos da equação (4.7) e o número de termos utilizados é chamado de **ordem** do modelo. No modelo autorregressivo de ordem n , abreviado $AR(n)$, a série temporal é escrita como:

$$y(t) = \sum_{k=1}^n a_k y(t - k) + e(t) \quad (4.8)$$

O cálculo com equações a diferenças como (4.8) torna-se mais elegante utilizando o operador *shift* q , que realiza o atraso de uma série temporal como um polinômio em q , de forma que $q^k y(t) = y(t - k)$. A equação (4.8) é escrita com o operador *shift* como:

$$A(q)y(t) = e(t) \quad (4.9)$$

onde $A(q) = 1 - a_1 q^1 - a_2 q^2 - \dots - a_n q^n$ é chamado de **polinômio característico** da equação a diferenças (4.8) e os coeficientes $(a_1, \dots, a_n)$ são os **parâmetros do modelo**.

O modelo AR apresenta um padrão característico de função de autocorrelação. A Figura 4.7 mostra as funções de autocorrelação de dois modelo $AR(1)$, sem o termo de ruído da equação (4.8).

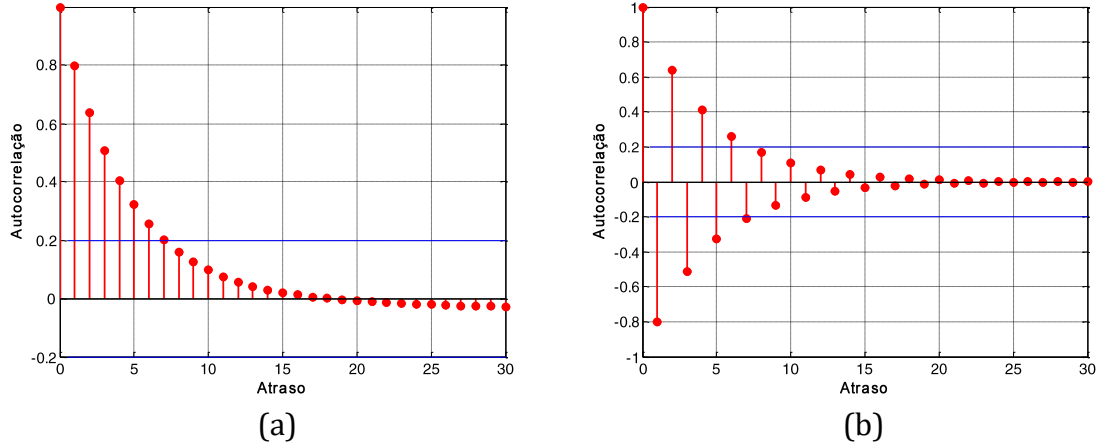


Figura 4.7: Função de autocorrelação do modelo $AR(1)$: (a) $a_1 = 0.8$; (b) $a_1 = -0.8$.

Os parâmetros do modelo autorregressivo na equação (4.8) se relacionam com os coeficientes da função de autocorrelação como:

$$\hat{\rho}_y(k) = \sum_{i=1}^n a_i \hat{\rho}_y(k-i), k > 0 \quad (4.10)$$

A partir da equação (4.10), os parâmetros do modelo autorregressivo podem ser estimados diretamente a partir dos valores da função de autocorrelação pela solução das equações de Yule-Walker [545][584]:

$$\begin{aligned} \hat{\rho}_y(1) &= a_1 \hat{\rho}_y(0) & + a_2 \hat{\rho}_y(1) & + \dots + a_n \hat{\rho}_y(n-1) \\ \hat{\rho}_y(2) &= a_1 \hat{\rho}_y(1) & + a_2 \hat{\rho}_y(2) & + \dots + a_n \hat{\rho}_y(n-2) \\ \vdots &= \vdots & + \vdots & + \dots + \vdots \\ \hat{\rho}_y(n) &= a_1 \hat{\rho}_y(n-1) + a_2 \hat{\rho}_y(n-2) + \dots + a_n \hat{\rho}_y(0) \end{aligned} \quad (4.11)$$

Uma estimativa robusta dos parâmetros do modelo AR pode ser obtida diretamente pelo método dos mínimos quadrados (MMQ) [474]. Escrevendo o resíduo da equação (4.8) como $e(t) = y(t) - \hat{y}(t)$ e resolvendo para $\hat{y}(t)$, o modelo AR é escrito na forma de um modelo de regressão linear:*

* Observe que $A(q) = 1 - a_1 q^1 - a_2 q^2 - \dots - a_n q^n$ e, portanto, $1 - A(q) = a_1 q^1 + a_2 q^2 + \dots + a_n q^n$.

$$\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T = (1 - A(q))y(t) = \sum_{k=1}^n a_k y(t - k) \quad (4.12)$$

onde o vetor de regressores $\hat{\mathbf{x}}(t) = (y(t - 1), \dots, y(t - n))$ e $\boldsymbol{\theta} = (a_1, \dots, a_n)$. A estrutura do modelo AR é representada na Figura 4.8.

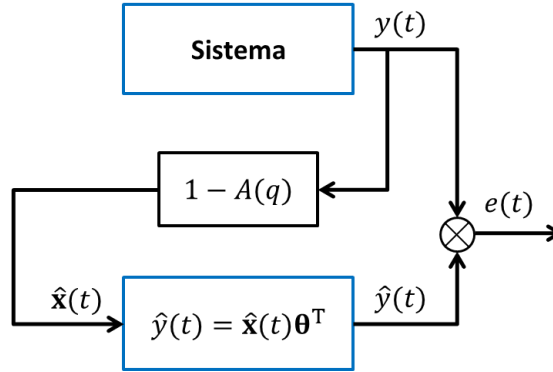


Figura 4.8: Estrutura do modelo AR.

A montagem da matriz de regressores para o ajuste dos parâmetros do modelo AR implica que n registros sejam descartados. A Figura 4.9 apresenta um exemplo da matriz de regressores e do vetor de saídas para o modelo $AR(2)$.

		$y(1)$
	$y(1)$	$y(2)$
$y(1)$	$y(2)$	$y(3)$
$y(2)$	$y(3)$	$y(4)$
$y(3)$	$y(4)$	$y(5)$
$\vdots$	$\vdots$	$\vdots$
$y(t - 2)$	$y(t - 1)$	$y(t)$
$\vdots$	$\vdots$	$\vdots$
$y(N - 2)$	$y(N - 1)$	$y(N)$
$y(N - 1)$	$y(N)$	
$y(N)$		

Figura 4.9: Montagem da matriz de regressores no modelo $AR(2)$.

O vetor de regressores do modelo AR como valores defasados da variável de saída permite que a relação temporal entre os registros seja capturada pelos regressores do modelo e o ajuste de parâmetros seja calculado pelo MMQ. A escolha da estrutura do modelo AR está relacionada à escolha na ordem do modelo.

A Figura 4.10 apresenta o resultado do modelo $AR(2)$ aplicado à variável de saída do problema *Gas Furnance*. Os resultados apresentados foram obtidos em validação cruzada de 10 ciclos.

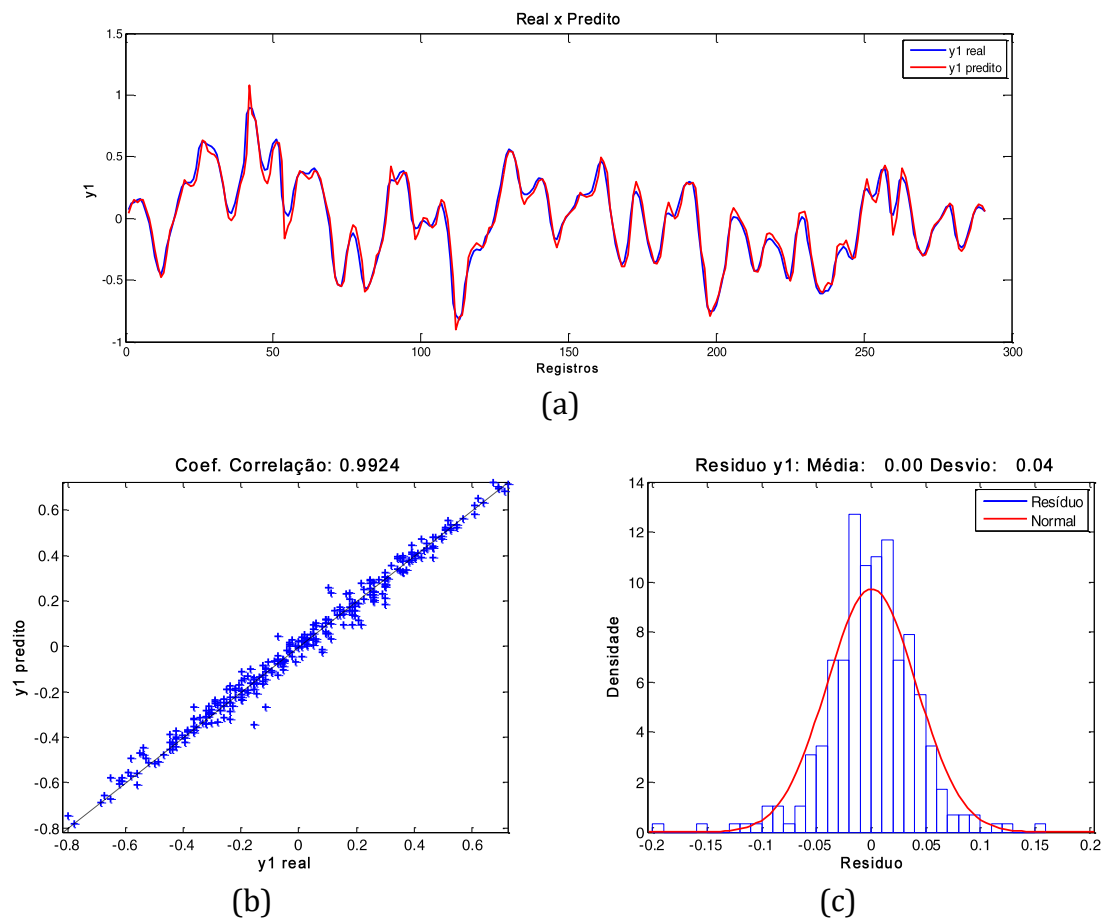


Figura 4.10: Resultados do modelo $AR(2)$ do problema Gas Furnance: (a) séries temporais real e predita; (b) correlação real x predito; (c) histograma do resíduo.

O modelo AR é ajustado utilizando os valores observados da variável de saída como regressores do modelo. Por isso, esse tipo de modelo só pode ser utilizado para a previsão do próximo registro da série, chamada de previsão de um passo adiante. A previsão de múltiplos passos não pode ser realizada utilizando os valores previamente calculados, pois os erros do modelo são reinseridos na entrada.

A Figura 4.11 mostra um exemplo do modelo $AR(2)$ do problema *Gas Furnance* em que, a partir do instante $t = 120$ assinalado, os resultados do modelo nos instantes anteriores são utilizados no vetor de regressores no lugar dos valores observados. Podemos observar que o comportamento do modelo difere consideravelmente do comportamento observado.

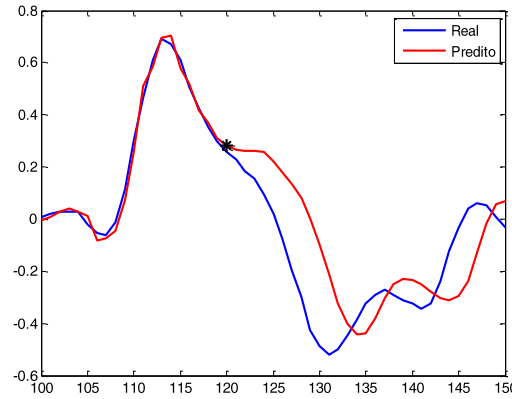


Figura 4.11: Modelo $AR(2)$ utilizando a predição como regressor.

4.2.2 Modelo MA

A equação (4.6) mostra que uma série temporal de média nula pode ser escrita como um filtro linear cuja entrada é um ruído branco. Assim como no modelo AR, a soma de infinitos termos é inviável e, na prática, o modelo de **média móvel** (MA)* de ordem m , abreviado $MA(m)$, é escrito como:

$$y(t) = C(q)e(t) = e(t) + \sum_{k=1}^m c_k e(t - k) \quad (4.13)$$

onde $C(q) = 1 + c_1 q^1 + c_2 q^2 + \dots + c_m q^m$ é o polinômio característico da equação de diferenças e os coeficientes $(c_1, \dots, c_m)$ são os parâmetros do modelo.†

Assim como o modelo AR, o modelo MA também apresenta um padrão característico de função de autocorrelação. A Figura 4.12 mostra as funções de autocorrelação para dois modelos $MA(1)$.

* *Moving Average*.

† O sinal dos coeficientes é arbitrário e a escolha de coeficientes negativos para $A(q)$ e positivos para $C(q)$ serve apenas para facilitar a formulação dos modelos.

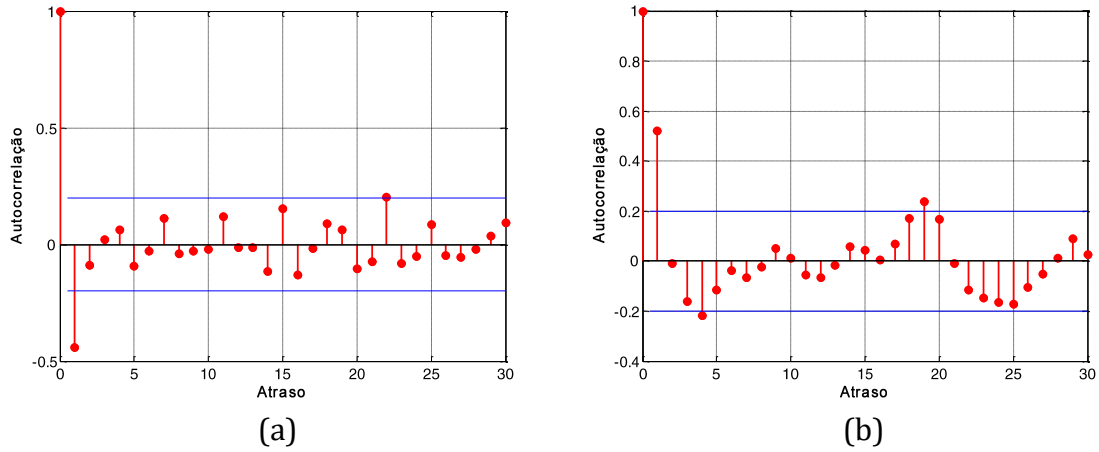


Figura 4.12: Função de autocorrelação do modelo $MA(1)$: (a) $c_1 = 0.8$; (b) $c_1 = -0.8$.

A função de autocorrelação de uma série temporal descrita por um modelo $MA(m)$ tem coeficientes nulos para $k > m$. Os demais coeficientes podem ser escritos em função dos parâmetros do modelo como:

$$\hat{\rho}_y(k) = \frac{c_k + c_1 c_{k-1} + \dots + c_{m-k} c_m}{1 + c_1^2 + \dots + c_m^2}, 1 < k \leq m \quad (4.14)$$

Diferentemente de um modelo AR, os parâmetros de um modelo MA não podem ser estimados pela solução de um sistema linear, pois a equação (4.14) é não linear. A solução da equação (4.14) para $m > 1$ é realizada por processos iterativos [59]. Pollock [441] apresenta os algoritmos para estimação de parâmetros de modelos MA codificados em Pascal e C.

O modelo MA na equação (4.13) pode ser escrito num formato semelhante ao modelo linear chamado **pseudolinear**. Separando o resíduo $e(t) = y(t) - \hat{y}(t)$, que não é conhecido no instante t , dos resíduos observados nos instantes anteriores $\hat{e}(t - k) = y(t - k) - \hat{y}(t - k)$, $1 < k < m$, a equação (4.13) pode ser reescrita como:

$$y(t) = e(t) + \sum_{k=1}^m c_k \hat{e}(t - k) \quad (4.15)$$

onde $e(t) = y(t) - \hat{y}(t)$ é o erro de predição ou **inovação**.

Resolvendo a equação (4.15) para $\hat{y}(t)$:

$$\hat{y}(t) = \hat{\mathbf{x}}(t) \boldsymbol{\theta}^T = (\mathcal{C}(q) - 1) \hat{e}(t) = \sum_{k=1}^m c_k \hat{e}(t - k) \quad (4.16)$$

onde o vetor de regressores $\hat{\mathbf{x}}(t) = (\hat{e}(t-1), \dots, \hat{e}(t-n))$ e $\boldsymbol{\theta} = (c_1, \dots, c_m)$ é o vetor de parâmetros. A estrutura do modelo MA é mostrada na Figura 4.13.

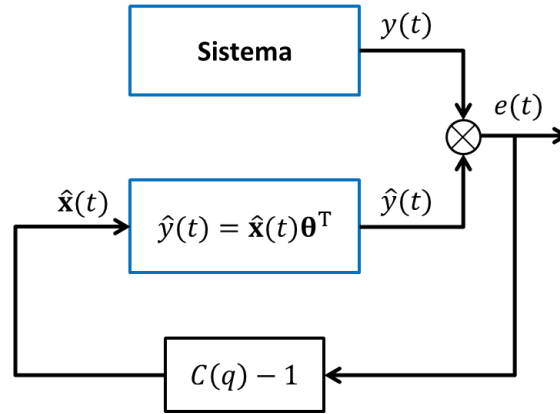


Figura 4.13: Estrutura do modelo MA.

A equação (4.16) é chamada pseudolinear porque, apesar de sua forma ser semelhante ao modelo linear, o vetor de regressores depende dos parâmetros do modelo de forma recursiva, uma vez que $\hat{e}(t-k) = y(t-k) - \hat{y}(t-k)$.

Existe uma dualidade entre os modelos AR e MA que pode ser verificada aplicando-se a equação (4.13) à equação (4.8):

$$e(t) = A(q)y(t) = A(q)C(q)e(t) \quad (4.17)$$

A equação (4.17) implica que $A(q)C(q) = 1$. O cálculo de $1/A(q)$ é feito utilizando-se a representação da série temporal como um filtro (cf. equação (4.6)) na equação (4.9):

$$y(t) = \frac{1}{A(q)}e(t) = \left(1 + \sum_{i=1}^{\infty} \psi_i q^i\right)e(t) \quad (4.18)$$

Analogamente para $1/C(q)$, utilizando-se as equações (4.7) e (4.13).

$$e(t) = \frac{1}{C(q)}y(t) = \left(1 - \sum_{j=1}^{\infty} \phi_j q^j\right)y(t) \quad (4.19)$$

Dessa forma, o modelo $MA(m)$ é equivalente a um modelo $AR(\infty)$ e, reciprocamente, um modelo $AR(n)$ corresponde a um modelo $MA(\infty)$.

4.2.3 Estruturas mistas

O modelo **autorregressivo com médias móveis** (ARMA)* é uma representação de séries temporais que contém simultaneamente as características dos modelos AR e MA. O modelo ARMA de ordem n para a parte AR e m para a parte MA, abreviado $ARMA(n, m)$, é escrito como:

$$A(q)y(t) = C(q)e(t) \quad (4.20)$$

onde $A(q) = 1 - a_1q^1 - a_2q^2 - \dots - a_nq^n$ e $C(q) = 1 + c_1q^1 + c_2q^2 + \dots + c_mq^m$ são os polinômios característicos.

Seguindo a formulação anterior, escrevendo o resíduo no instante de predição (inovação) como $e(t) = y(t) - \hat{y}(t)$ e o resíduo nos instantes anteriores como $\hat{e}(t - k) = y(t - k) - \hat{y}(t - k)$, $k > 1$, a equação (4.20) é reescrita como:

$$A(q)y(t) = e(t) + (C(q) - 1)\hat{e}(t) \quad (4.21)$$

onde $e(t) = y(t) - \hat{y}(t)$.

Resolvendo para $\hat{y}(t)$:

$$\hat{y}(t) = (1 - A(q))y(t) + (C(q) - 1)\hat{e}(t) \quad (4.22)$$

A equação (4.22) pode ser escrita na forma pseudolinear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, onde $\hat{\mathbf{x}}(t) = (y(t - 1), \dots, y(t - n), e(t - 1), \dots, e(t - m))$ e $\boldsymbol{\theta} = (a_1, \dots, a_n, c_1, \dots, c_m)$ é o vetor de parâmetros que contém os coeficientes dos polinômios $A(q)$ e $C(q)$. A estrutura do modelo ARMA é mostrada na Figura 4.14.

Assim como no modelo MA, os parâmetros do modelo ARMA não podem ser identificados pelo MMQ, pois também aparecem no vetor de regressores. Há algoritmos na literatura para a estimativa dos parâmetros do modelo ARMA a partir de uma estimativa inicial e métodos iterativos [59][441].

O modelo ARMA só pode ser utilizado em séries estacionárias. No caso de séries não estacionárias, há técnicas para a extração de padrões específicos como tendências e sazonalidades [404], que são muito utilizadas em econometria. Algumas variantes do modelo ARMA permitem a modelagem de séries temporais não estacionárias.

* *AutoRegressive-Moving Average*.

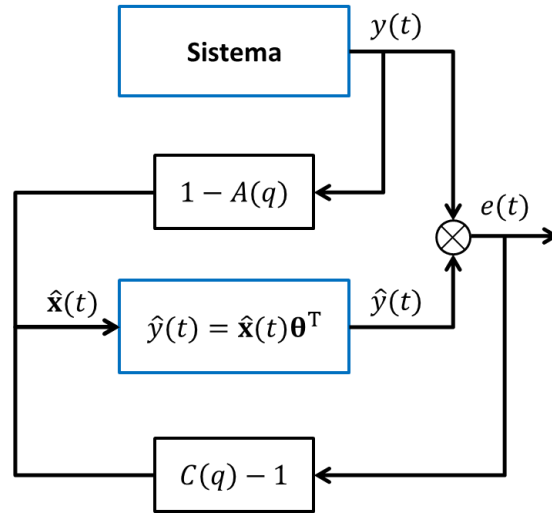


Figura 4.14: Estrutura do modelo ARMA.

O modelo **autorregressivo, integrado, de médias móveis** (ARIMA)* é utilizado em séries com tendências que se tornam estacionárias quando diferenciadas [59]. O modelo $ARIMA(n, d, m)$ é escrito como:

$$A(q)\Delta^d y(t) = C(q)e(t) \quad (4.23)$$

onde $\Delta^d = (1 - q)^d$ é o operador de diferenciação calculado pela expansão binomial¹⁰⁸ e $\Delta^d y(t) = \alpha_0 + (-1)^{d-1} \alpha_1 y(t-1) + (-1)^{d-2} \alpha_2 y(t-2) + \dots + \alpha_d y(t-d)$, onde os coeficientes α_i correspondem à linha d do triângulo de Pascal.¹⁰⁹ Por exemplo, para $d = 2$, $\Delta^d y(t) = 1 - 2y(t-1) + y(t-2)$.

O modelo ARIMA é equivalente a um modelo ARMA utilizando a transformação de variáveis $w(t) = \Delta^d y(t)$:

$$\hat{A}(q)w(t) = C(q)e(t) \quad (4.24)$$

onde $\hat{A}(q)$ é o polinômio característico da variável transformada.

O modelo ARIMA descrito pela equação (4.23) representa a estrutura mais geral para a modelagem de séries temporais. Diversas outras estruturas podem ser derivadas do modelo ARIMA pela supressão de componentes, como mostra a Tabela 4.1.

* AutoRegressive Integrated Moving Average.

Uma variante do modelo ARIMA para a representação de processos com sazonalidade é obtida através de uma transformação $w(t) = (1 - q^h)y(t) = y(t) - y(t - h)$, onde h é o período da sazonalidade considerada [59][404].

Tabela 4.1: Modelos de séries temporais

Modelo	Nome da Estrutura
$A(q)y(t) = e(t)$	$AR(n)$
$A(q)\Delta^d y(t) = e(t)$	$ARI(n, d)$
$y(t) = C(q)e(t)$	$MA(m)$
$\Delta^d y(t) = C(q)e(t)$	$IMA(d, m)$
$A(q)y(t) = C(q)e(t)$	$ARMA(n, m)$
$A(q)\Delta^d y(t) = C(q)e(t)$	$ARIMA(n, d, m)$

O modelo ARIMA é citado como modelo Box & Jenkins em algumas fontes, devido aos resultados publicados na primeira edição do livro [59] de 1970. Na verdade, os autores apresentam uma metodologia para identificação de modelos de séries temporais que segue, essencialmente, os passos descritos na seção 3.3: identificação da estrutura, ajuste de parâmetros e validação do modelo.

Modelos lineares de séries temporais são úteis em aplicações em que as variáveis de entrada do sistema não são observadas. Há problemas em que as variáveis de entrada são observadas e o conhecimento destas pode contribuir com os resultados da modelagem, como descrito a seguir.

4.3 Sistemas Dinâmicos Lineares

Um sistema linear monovariável discreto e invariante no tempo pode ser caracterizado pelos coeficientes (resposta ao impulso) φ_k como:

$$y(t) = \sum_{k=1}^{\infty} \varphi_k x(t - k) \quad (4.25)$$

onde $x(t)$ é a entrada e $y(t)$ a saída do sistema como na Figura 4.1 e $\sum_{k=1}^{\infty} |\varphi_k| < \infty$.

Incluindo as perturbações aleatórias na equação (4.25), a resposta do sistema dinâmico é escrita como:

$$y(t) = \sum_{k=1}^{\infty} \varphi_k x(t-k) + \sum_{k=1}^{\infty} \xi_k e(t-k) + e(t) \quad (4.26)$$

onde $e(t)$ é um ruído branco de média nula e variância constante e $\sum_{k=1}^{\infty} |\xi_k| < \infty$.

O modelo do sistema dinâmico linear com perturbações da equação (4.26) pode ser escrito de forma sintética utilizando o operador *shift* q como:

$$y(t) = G(q)x(t) + H(q)e(t) \quad (4.27)$$

onde o polinômio $G(q) = \sum_{k=1}^{\infty} \varphi_k q^k$ é chamado de **função de transferência** do sistema e $H(q) = 1 + \sum_{k=1}^{\infty} \xi_k q^k$ é a função de transferência do ruído.

Escrevendo o erro de predição no instante t como $e(t) = y(t) - \hat{y}(t)$ na equação (4.27) temos:

$$y(t) = G(q)x(t) + H(q)(y(t) - \hat{y}(t)) \quad (4.28)$$

Resolvendo a equação (4.28) para $\hat{y}(t)$:

$$\begin{aligned} H(q)\hat{y}(t) &= G(q)x(t) + (H(q) - 1)y(t) \\ \hat{y}(t) &= H^{-1}(q)G(q)x(t) + (1 - H^{-1}(q))y(t) \end{aligned} \quad (4.29)$$

O modelo $\hat{y}(t)$ na equação (4.29) é calculado a partir dos valores das variáveis de entrada e saída nos instantes anteriores, isto é, $\{x(t-k), y(t-k), k > 1\}$. Dessa forma, a previsão $\hat{y}(t)$ é chamada predição a um passo.

O erro de previsão é escrito como (cf. equações (4.27) e (4.29)):

$$e(t) = y(t) - \hat{y}(t) = -H^{-1}(q)G(q)x(t) + H^{-1}(q)y(t) \quad (4.30)$$

O erro de previsão no instante t é a parcela da saída que não pode ser descrita pelo modelo a partir dos dados passados, chamada de **inovação** [352]. Dessa forma, como anteriormente, podemos decompor o erro do modelo em duas parcelas: a inovação ou erro de predição no instante t , $e(t) = y(t) - \hat{y}(t)$, e o resíduo do modelo nos instantes anteriores $\hat{e}(t-k) = y(t-k) - \hat{y}(t-k), k > 1$.

A equação (4.29) não é útil na prática para a descrição do sistema, pois requer o conhecimento de todo o histórico de dados e um grande número de parâmetros precisam ser

ajustados. Modelos de sistemas dinâmicos podem ser escritos como aproximações das funções de transferência $G(q)$ e $H(q)$ com um número finito de parâmetros. Os mesmos passos definidos pelas equações (4.27) a (4.29) são repetidos para a obtenção da expressão do modelo nas diferentes estruturas apresentadas a seguir.

4.3.1 Modelo ARX

O modelo mais simples para a descrição da relação entrada-saída de um sistema dinâmico linear é o **autorregressivo em entrada exógena** (ARX) escrito como:

$$A(q)y(t) = B(q)x(t) + e(t) \quad (4.31)$$

onde os polinômios $A(q) = 1 - a_1q^1 - \dots - a_{na}q^{na}$; $B(q) = b_1q^1 + \dots + b_{nb}q^{nb}$; na é a ordem de entrada e nb a ordem de saída.

Escrevendo o erro de predição no instante t como $e(t) = y(t) - \hat{y}(t)$ e resolvendo para $\hat{y}(t)$ na equação (4.31), temos:

$$\hat{y}(t) = (1 - A(q))y(t) + B(q)x(t) = \sum_{i=1}^{na} a_i y(t-i) + \sum_{j=1}^{nb} b_j x(t-j) \quad (4.32)$$

A equação (4.32) pode ser escrita como um modelo linear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, com o vetor de regressores $\hat{\mathbf{x}}(t) = (y(t-1), \dots, y(t-na), x(t-1), \dots, x(t-nb))$ e o vetor de parâmetros $\boldsymbol{\theta} = (a_1, \dots, a_{na}, b_1, \dots, b_{nb})$. A Figura 4.15 mostra a estrutura do modelo ARX.

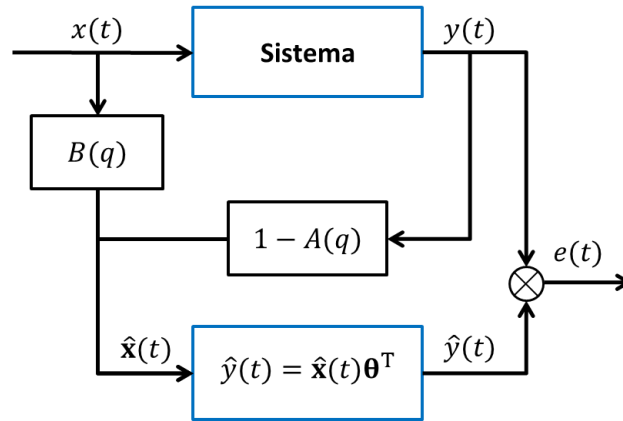


Figura 4.15: Estrutura do modelo ARX.

O modelo ARX pode ser estendido para sistemas de múltiplas entradas como:

$$\hat{y}(t) = (1 - A(q))y(t) + \sum_{i=1}^p B_i(q)x_i(t) \quad (4.33)$$

onde p é o número de entradas.

O modelo ARX de múltiplas entradas é um modelo linear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, em que $\hat{\mathbf{x}}(t) = (y(t-1), \dots, y(t-na), x_1(t-1), \dots, x_1(t-nb_1), \dots, x_p(t-1), \dots, x_p(t-nb_p))$ é o vetor de regressores, $\boldsymbol{\theta} = (a_1, \dots, a_{na}, b_{1,1}, \dots, b_{1,nb_1}, \dots, b_{p,1}, \dots, b_{p,nb_p})$ é o vetor de parâmetros e $nb_i, i = 1 \dots p$ são as ordens das variáveis de entrada.

No modelo $ARX(n, m)$, os parâmetros ocorrem linearmente na expressão do modelo, de forma que ele pode ser ajustado pelo MMQ. A seleção de registros para a matriz de regressores implica o descarte de $\max(na, nb_1, \dots, nb_p)$ registros, como mostra a Figura 4.16 para o caso do modelo $ARX(2,3)$. Uma vez realizada a montagem da matriz de regressores, o problema de modelagem de um sistema dinâmico por um modelo ARX torna-se um problema estático, uma vez que toda a memória necessária ao modelo é armazenada no vetor de regressores.

					$y(1)$
		$x(1)$		$y(1)$	$y(2)$
	$x(1)$	$x(2)$	$y(1)$	$y(2)$	$y(3)$
$x(1)$	$x(2)$	$x(3)$	$y(2)$	$y(3)$	$y(4)$
$x(2)$	$x(3)$	$x(4)$	$y(3)$	$y(4)$	$y(5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$x(t-3)$	$x(t-2)$	$x(t-1)$	$y(t-2)$	$y(t-1)$	$y(t)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$x(N-3)$	$x(N-2)$	$x(N-1)$	$y(N-2)$	$y(N-1)$	$y(N)$
$x(N-2)$	$x(N-1)$	$x(N)$	$y(N-1)$	$y(N)$	
$x(N-1)$	$x(N)$		$y(N)$		
$x(N)$					

Figura 4.16: Montagem das matrizes do MMQ no modelo ARX.

A Figura 4.17 apresenta os resultados obtidos em validação cruzada de 10 ciclos do modelo $ARX(3,2)$ aplicado ao problema *Gas Furnance*. Comparando aos resultados do modelo $AR(2)$ da Figura 4.10, observamos um pequeno aumento do desempenho com a introdução da variável de entrada.

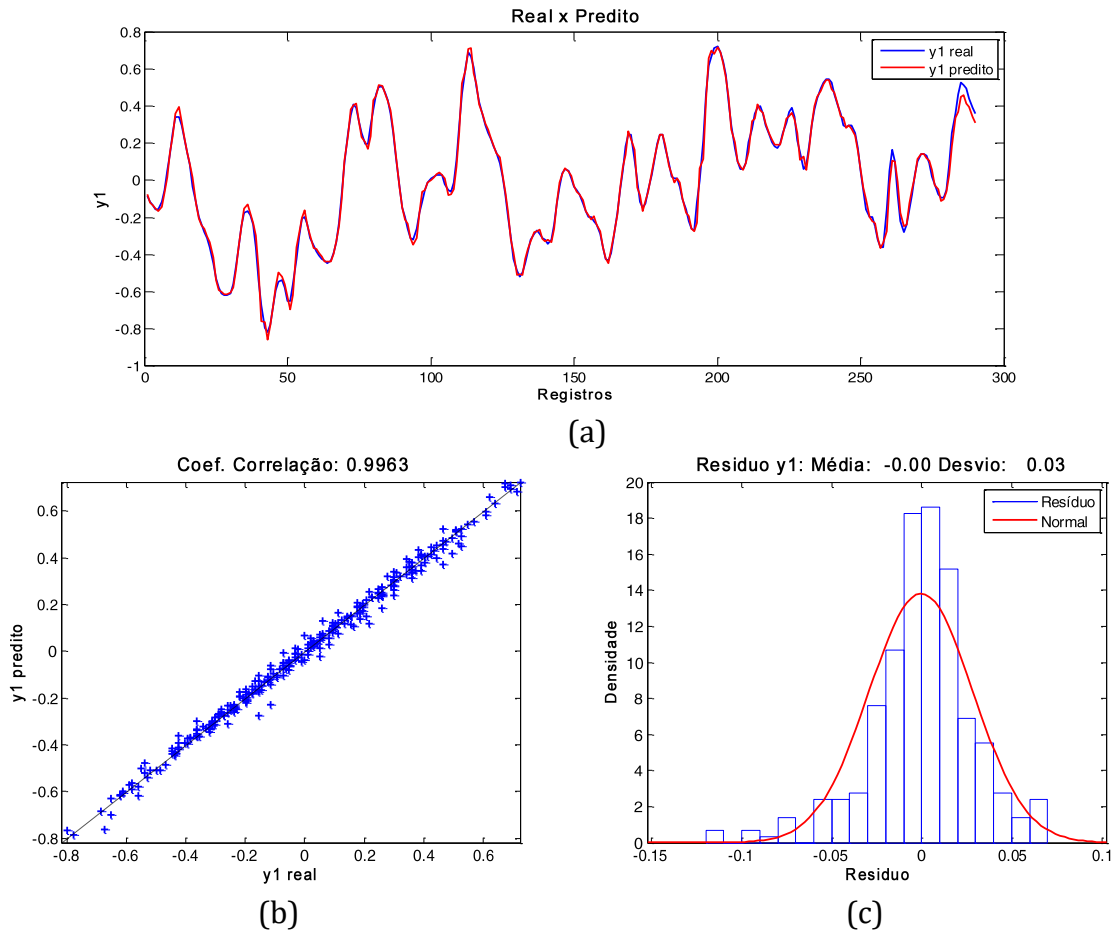


Figura 4.17: Resultados do modelo $ARX(3,2)$ para o problema *Gas Furnance*:
 (a) séries temporais real e predita; (b) correlação real x predito; (c) histograma do resíduo.

4.3.2 Modelo ARMAX

O modelo **autorregressivo em entrada exógena e média móvel** (ARMAX) é uma extensão da estrutura ARX incluindo uma representação em média móvel do resíduo:

$$A(q)y(t) = B(q)x(t) + C(q)e(t) \quad (4.34)$$

onde os polinômios $A(q)$ e $B(q)$ são definidos como no modelo ARX e o polinômio $C(q) = 1 + c_1q^1 + \dots + c_{nc}q^{nc}$ representa a influência da média móvel do resíduo sobre a variável de saída. A equação (4.34) representa o modelo $ARMAX(na, nb, nc)$, onde na e nb são, respectivamente, as ordens de entrada e saída e nc é a ordem do termo de média móvel.

Assim como o modelo ARMA, o modelo ARMAX tem uma representação pseudolinear que pode ser obtida escrevendo o erro de predição como $e(t) = y(t) - \hat{y}(t)$ e o resíduo do modelo nos instantes anteriores como $\hat{e}(t - k) = y(t - k) - \hat{y}(t - k), k > 1$:

$$\hat{y}(t) = B(q)x(t) + (1 - A(q))y(t) + (C(q) - 1)\hat{e}(t) \quad (4.35)$$

O modelo ARMAX da equação (4.35) é escrito na forma pseudolinear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, $\hat{\mathbf{x}}(t) = (y(t - 1), \dots, y(t - na), x(t - 1), \dots, x(t - nb), \hat{e}(t - 1), \dots, \hat{e}(t - nc))$ é o com o vetor de regressores e $\boldsymbol{\theta} = (a_1, \dots, a_{na}, b_1, \dots, b_{nb}, c_1, \dots, c_{nc})$ o vetor de parâmetros. A Figura 4.15 apresenta a estrutura do modelo ARMAX, que também pode ser calculado em sistemas de múltiplas variáveis de entrada considerando o efeito das entradas como na formulação do modelo ARX de múltiplas variáveis (4.33).

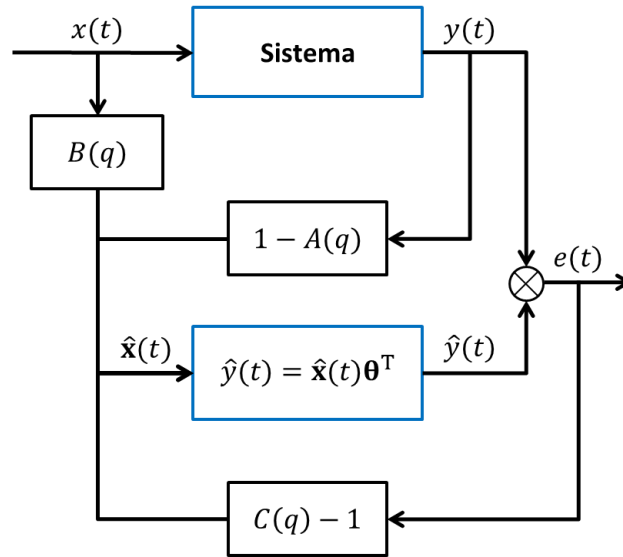


Figura 4.18: Estrutura do modelo ARMAX.

4.3.3 Modelo OE

Nas estruturas ARX e ARMAX, os valores anteriores da variável de saída são necessários para o cálculo do modelo e a variável de saída deve ser observada. Dessa forma, esse tipo de modelo é chamado de **predição**. Em geral, os modelos de predição são utilizados para a previsão de apenas alguns passos adiante, uma vez que os resultados anteriores do modelo não podem ser utilizados em substituição à saída observada em sua entrada. No modelo de **simulação**, também chamado de **erro de saída (OE)**,* o

* Output Error.

modelo é ajustado com seus resultados anteriores, o que permite a previsão de múltiplos passos adiante. Dessa forma, o modelo OE realiza a previsão da variável de saída sem utilizar os valores observados anteriores e por isso pode ser utilizado para a simulação da variável de saída a partir de uma simulação da variável de entrada.

Na estrutura OE, o modelo é calculado a partir de uma previsão da saída do modelo (sem perturbação) e os erros são modelados como um ruído branco aditivo:

$$y(t) = \frac{B(q)}{F(q)}x(t) + e(t) \quad (4.36)$$

onde o polinômio $B(q)$ é como no modelo ARX e $F(q) = 1 - f_1q^1 - \dots - f_{nf}q^{nf}$ representa os resultados anteriores do modelo.

O modelo OE também tem uma representação pseudolinear, seguindo o mesmo procedimento anterior, definindo $e(t) = y(t) - \hat{y}(t)$:

$$F(q)\hat{y}(t) = B(q)x(t) \quad (4.37)$$

e resolvendo para $\hat{y}(t)$:

$$\hat{y}(t) = B(q)x(t) + (1 - F(q))\hat{y}(t) \quad (4.38)$$

A equação (4.38) está na forma pseudolinear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, com o vetor de regressores $\hat{\mathbf{x}}(t) = (x(t-1), \dots, x(t-nb), \hat{y}(t-1), \dots, \hat{y}(t-nf))$ e o vetor de parâmetros $\boldsymbol{\theta} = (b_1, \dots, b_{nb}, f_1, \dots, f_{nf})$. A Figura 4.19 apresenta a estrutura do modelo OE.

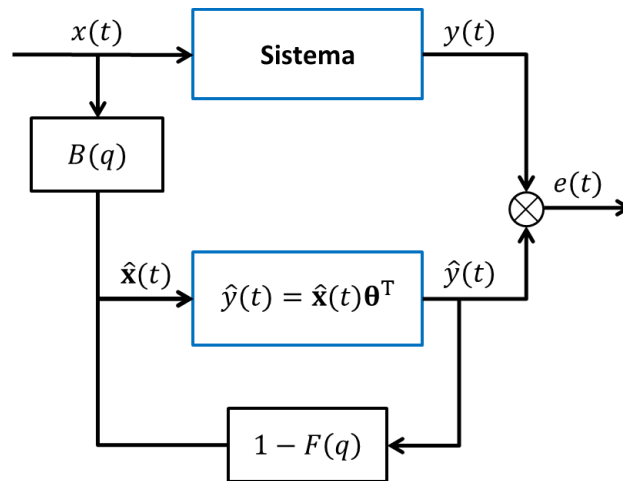


Figura 4.19: Estrutura do modelo OE.

Os valores calculados pelo modelo nos instantes anteriores aparecem explicitamente no vetor de regressores, de forma que os parâmetros do modelo não podem ser ajustados pelo MMQ.

4.3.4 Modelo Box & Jenkins

A estrutura mais complexa, que combina as características de modelos de predição e simulação, é chamada de estrutura Box & Jenkins por Ljung [352]. A estrutura Box & Jenkins (BJ) utiliza a previsão da saída sem perturbações do modelo OE com uma descrição mais elaborada do ruído:

$$y(t) = \frac{B(q)}{F(q)}x(t) + \frac{C(q)}{D(q)}e(t) \quad (4.39)$$

onde $B(q)$ e $C(q)$ são os mesmos do modelo ARMAX, o polinômio $F(q)$ é o mesmo do modelo OE e $D(q) = 1 + d_1q^1 + \dots + d_{nd}q^{nd}$.

A estrutura BJ também pode ser colocada na forma pseudolinear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, mas requer uma transformação de variáveis:

$$y(t) = w(t) + v(t) \quad (4.40)$$

onde $w(t) = B(q)/F(q)x(t)$ é o modelo da saída e $v(t) = C(q)/D(q)e(t)$, o modelo de perturbações. Separando o erro de predição $e(t) = y(t) - \hat{y}(t)$ do resíduo do modelo nos instantes anteriores $\hat{e}(t-k) = y(t-k) - \hat{y}(t-k)$, $k > 1$, temos:

$$\begin{aligned} w(t) &= B(q)x(t) + (1 - F(q))w(t) \\ v(t) &= e(t) + (C(q) - 1)\hat{e}(t) + (1 - D(q))v(t) \end{aligned} \quad (4.41)$$

Aplicando (4.41) em (4.40) e resolvendo para $\hat{y}(t)$:

$$\hat{y}(t) = B(q)x(t) + (1 - F(q))w(t) + (C(q) - 1)\hat{e}(t) + (1 - D(q))v(t) \quad (4.42)$$

A equação (4.40) pode ser escrita na forma pseudolinear $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, com o vetor $\hat{\mathbf{x}}(t) = (x(t-1), \dots, x(t-nb), w(t-1), \dots, w(t-nf), \hat{e}(t-1), \dots, \hat{e}(t-nc), v(t-1), \dots, v(t-nd))$ e $\boldsymbol{\theta} = (b_1, \dots, b_{nb}, f_1, \dots, f_{nf}, c_1, \dots, c_{nc}, d_1, \dots, d_{nd})$.

A Tabela 4.2 resume as estruturas de modelos de sistemas dinâmicos lineares. Apenas a estrutura ARX pode ter seus parâmetros ajustados pelo MMQ. As demais estruturas,

apesar de representarem modelos de sistemas lineares na forma pseudolinear, os modelos resultantes são não lineares nos parâmetros e o ajuste deve ser feito pelo método do gradiente [352]. Os algoritmos para estimação de parâmetros dos modelos na Tabela 4.2 estão implementados na *toolbox* de identificação de sistemas do Matlab.¹¹⁰

Tabela 4.2: Modelos de sistemas dinâmicos

Modelo	Nome da estrutura
$A(q)y(t) = B(q)x(t) + e(t)$	$ARX(na, nb, nc)$
$A(q)y(t) = B(q)x(t) + C(q)e(t)$	$ARMAX(na, nb, nc)$
$y(t) = \frac{B(q)}{F(q)}x(t) + e(t)$	$OE(nb, nf)$
$y(t) = \frac{B(q)}{F(q)}x(t) + \frac{C(q)}{D(q)}e(t)$	$BJ(nb, nc, nd, nf)$

4.3.5 Equações de estado

A representação de modelos de sistemas dinâmicos na forma de equações a diferenças é bastante flexível e permite a representação de diversos tipos de modelos. Entretanto, sua formulação para sistemas de múltiplas variáveis torna-se excessivamente complexa. Uma representação de sistemas dinâmicos amplamente utilizada na teoria de controle é a representação por equações de estado. As equações de estado podem ser diretamente obtidas a partir de equações diferenciais do modelo analítico e permitem a representação de sistemas de múltiplas variáveis com vários graus de liberdade.

O modelo de um sistema linear discreto de saída simples (MISO)* é escrito na forma de equações de estado como:

$$\begin{cases} \mathbf{z}(t+1) = \mathbf{z}(t)\mathbf{A} + \mathbf{x}(t)\mathbf{B} \\ y(t) = \mathbf{z}(t)\mathbf{C} \end{cases} \quad (4.43)$$

* Multiple Inputs Single Output.

onde $\mathbf{z}(t) \in \mathcal{R}^n$ é o **vetor de estados** do modelo de ordem n ; a matriz $\mathbf{A} \in \mathcal{R}^{n \times n}$ é chamada de **matriz de transição**; $\mathbf{B} \in \mathcal{R}^{p \times n}$ é a **matriz de entrada**; e $\mathbf{C} \in \mathcal{R}^n$ é a **matriz de saída** ou de medidas. A matriz de saída normalmente apenas identifica os estados que são observáveis.

A representação de sistemas dinâmicos por equações de estado foi introduzida na década de 1960. O conceito fundamental de variáveis de estado é definido por Kalman [293] como “a menor quantidade de dados sobre o comportamento passado de um sistema necessária para prever o seu comportamento futuro”. As equações de estado são uma descrição da dinâmica do sistema linear em termos de transformações lineares das variáveis de estado.

Ao contrário da representação por função de transferência, a representação de estados não é única e qualquer transformação linear ortogonal das matrizes $\mathbf{A}$, $\mathbf{B}$ e $\mathbf{C}$ da equação (4.43) também é uma representação válida do sistema.

A conversão de representações entrada-saída de modelos dinâmicos e séries temporais é direta para sistemas monovariáveis (SISO),* mas um pouco mais complexa para sistemas multivariados (MISO) [352]. Em sistemas MISO, a representação por equações de estado é mais indicada, pois favorece o ajuste de parâmetros do modelo por métodos de identificação à base de subespaços [402][298].

4.4 Aplicações

4.4.1 *Wolf's sunspot numbers*

Este conjunto de dados¹¹¹ apresenta o registro do fenômeno de manchas solares¹¹² na superfície do sol, investigado inicialmente por Rudolf Wolf. A série temporal apresenta a média anual de manchas solares observadas entre 1700 e 1988.

O fenômeno é cíclico e atinge o valor máximo a cada 10 anos aproximadamente. A Figura 4.20 apresenta a série temporal e a sua função de autocorrelação. Podemos verificar que o valor de autocorrelação é oscilatório, com amplitude que decresce com o aumento do atraso, o que caracteriza o processo como autorregressivo.

* *Single Input Single Output*.

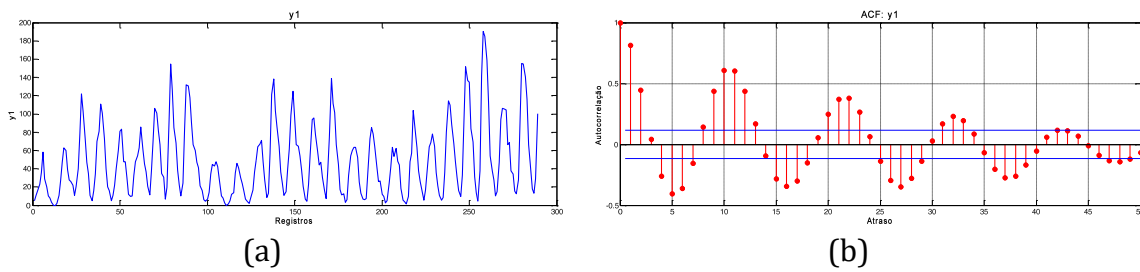


Figura 4.20: Conjunto de dados Sunspot: (a) série temporal; (b) função de autocorrelação.

A estrutura de melhor desempenho em validação cruzada foi obtida pelo modelo $AR(9)$, cujos resultados são apresentados na Figura 4.21 e na Figura 4.22. O fenômeno apresenta características que permitem sua predição com boa precisão com o modelo AR. Entretanto, como já foi discutido, o modelo AR não poderia ser utilizado para simular esse fenômeno.

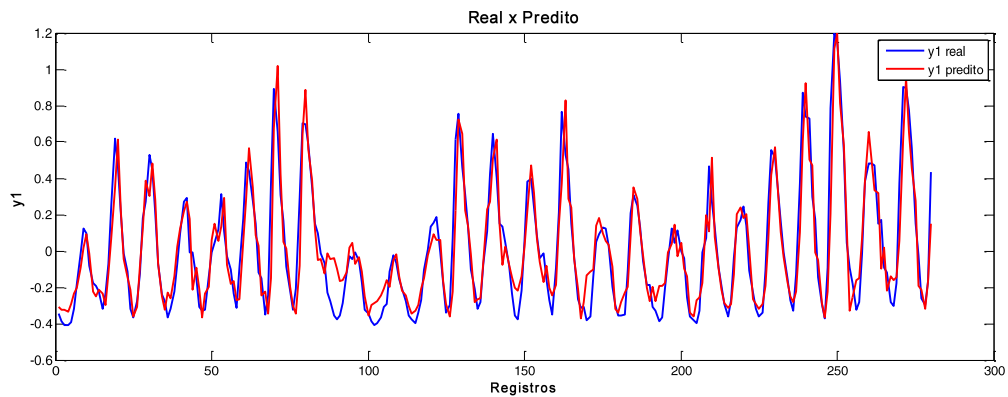


Figura 4.21: Resultados do modelo $AR(9)$ sobre o conjunto de dados Sunspot.

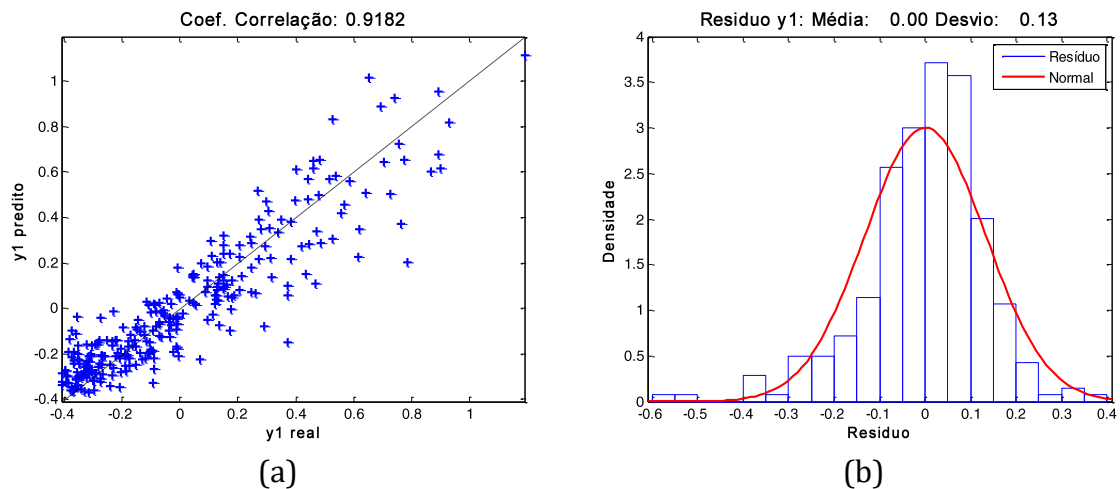


Figura 4.22: Resultados do modelo $AR(9)$ sobre o conjunto de dados Sunspot:
(a) correlação real x predito; (b) histograma do resíduo.

4.4.2 Modelo chuva-vazão

O desenvolvimento de modelos precisos de previsão de vazões é de grande importância para o planejamento operacional das usinas hidrelétricas do Sistema Interligado Nacional¹¹³ (SIN). No final de 2004, o Operador Nacional do Sistema Elétrico (ONS), responsável pelo planejamento operacional do SIN, lançou uma demanda para o desenvolvimento de modelos de previsão de vazões com incorporação de previsões de precipitações [222]. O objetivo dos modelos era calcular uma previsão semanal de vazões 12 dias à frente utilizando as previsões de precipitações fornecidas pelo modelo regional ETA [103] do Centro de Previsão do Tempo e Estudos Climáticos (CPE-TEC/Inpe).¹¹⁴ Esta seção descreve um modelo linear da bacia do rio Iguaçu.

Os dados do problema são apresentados na Figura 4.23, onde as variáveis de entrada são as estações de observação de chuva, chamadas pluviômetros, assinaladas em verde e as variáveis de saída são os aproveitamentos hidrelétricos, em vermelho. Os limites das sub-bacias e a grade do modelo regional ETA também são mostrados na Figura 4.23. O conjunto de dados fornecido continha o histórico de vazões naturais* de cinco aproveitamentos hidrelétricos, vazões incrementais das sub-bacias e os registros de precipitação observada de 25 pluviômetros. Para o desenvolvimento do projeto também foi fornecido o histórico de previsões do modelo ETA. Os resultados do projeto são descritos em [222].

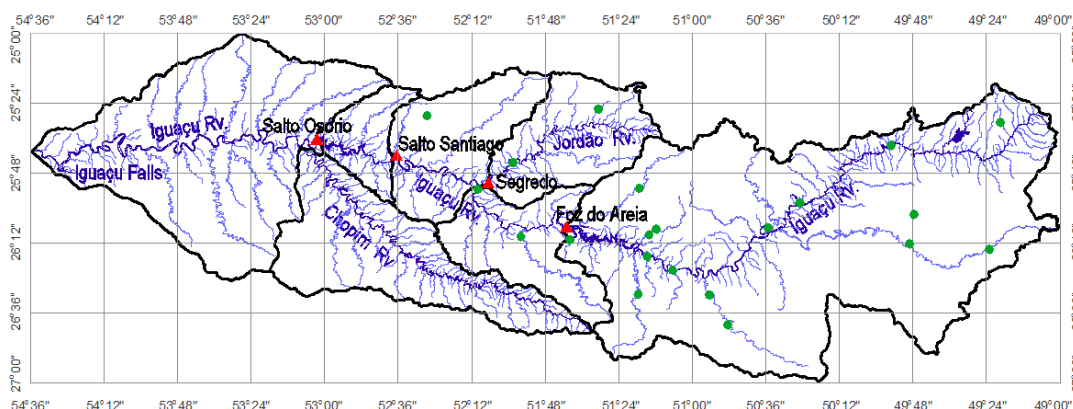


Figura 4.23: Bacia do rio Iguaçu, com sub-bacias e grade do modelo ETA. Fonte: [169].

* Vazões naturais numa seção são as vazões observadas descontadas de todas as ações realizadas pelo homem a montante daquela seção.

Nesta seção, vamos apresentar os resultados de um modelo ARX para a previsão da vazão em Foz do Areia, que é a maior sub-bacia do rio Iguaçu. Para facilitar o desenvolvimento do modelo, a chuva acumulada na bacia foi utilizada como variável de entrada, calculada pela soma das séries de todos os pluviômetros.

O conjunto de dados contém o histórico de 16 anos da bacia com o registro de chuva e vazão diária entre 1992 e 2008, totalizando 5842 registros. A Figura 4.24 mostra a série temporal de entrada e saída do conjunto de dados e um *zoom* em um período de aproximadamente dois anos em que é possível verificar a dinâmica de cheia e vazante.

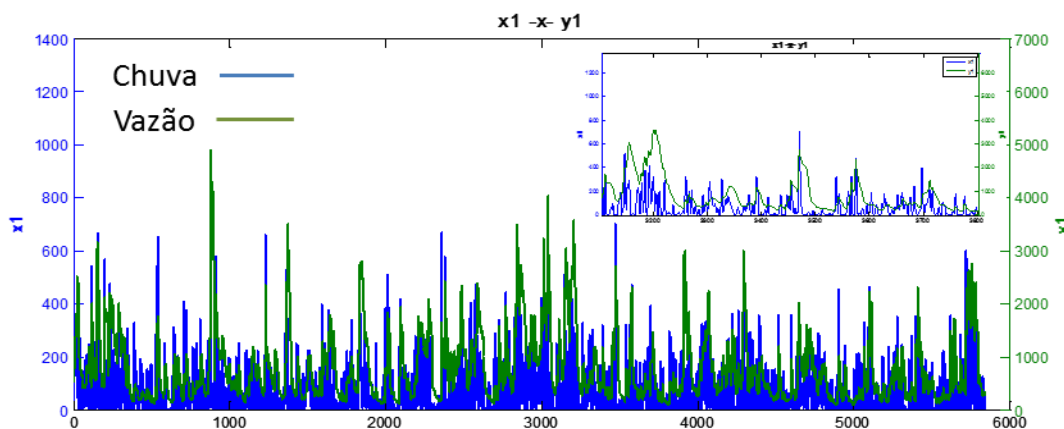
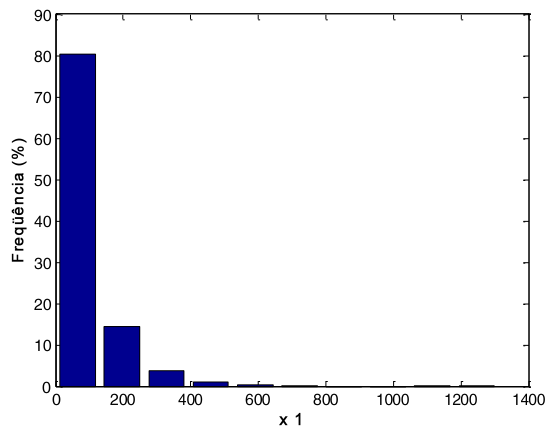


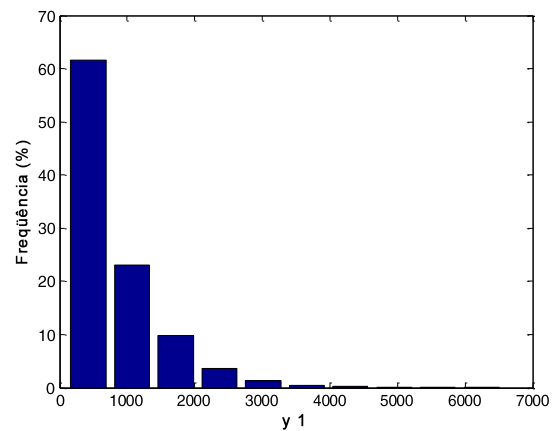
Figura 4.24: Série temporal de chuva e vazão.

Durante uma chuva forte, a vazão do rio Iguaçu aumenta rapidamente, permanecendo alta enquanto houver eventos de chuva. A vazante é bem mais lenta que a cheia, o que mostra um comportamento dinâmico diferente entre cheia e vazante, dificultando a modelagem do processo como um sistema linear. Uma característica importante desse problema é que as duas séries têm uma distribuição altamente assimétrica, como mostra a Figura 4.25.

A análise das funções de autocorrelação na Figura 4.26 mostra que a série de chuva tem um comportamento de processo de média móvel, enquanto a série de vazão tem um comportamento de processo autorregressivo com memória longa. A função de correlação cruzada não reflete atrasos significativos entre as séries de chuva e de vazão.

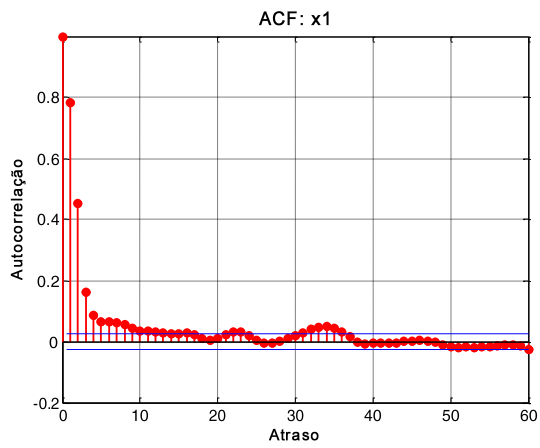


(a)

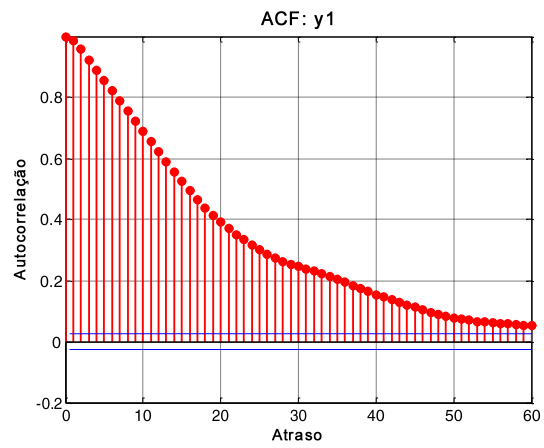


(b)

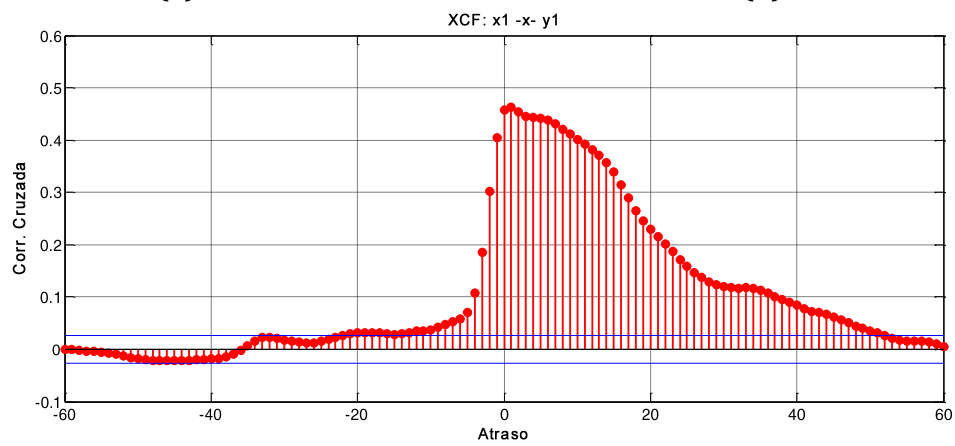
Figura 4.25: Histogramas: (a) chuva acumulada; (b) vazão natural.



(a)



(b)



(c)

Figura 4.26: Autocorrelação: (a) chuva acumulada; (b) vazão natural; (c) correlação cruzada.

Os resultados do modelo $ARX(1,2)$ em validação cruzada de 10 ciclos são apresentados na Figura 4.27 e na Figura 4.28. Podemos observar que o modelo obtém um excelente resultado para a previsão da vazão do próximo dia. Entretanto, o comportamento do modelo se degrada rapidamente caso a vazão observada não esteja disponível.

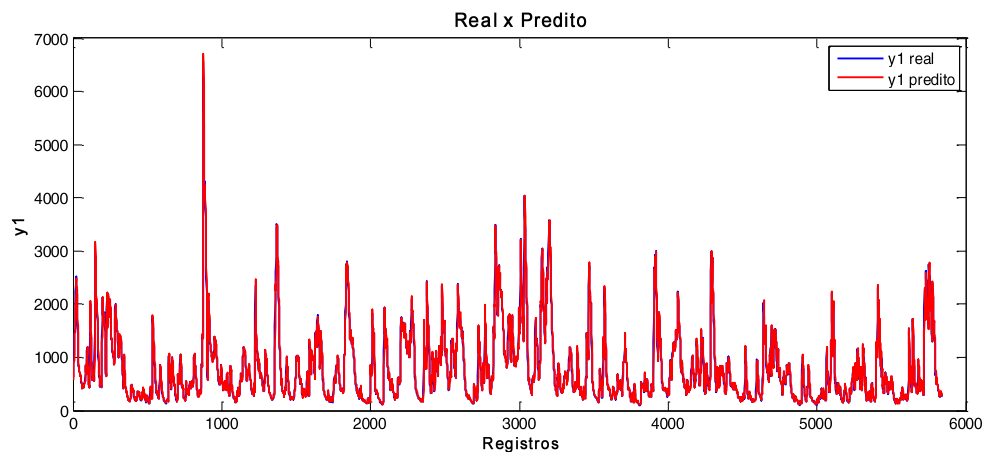


Figura 4.27: Séries temporais real e predita do modelo $ARX(1,2)$.

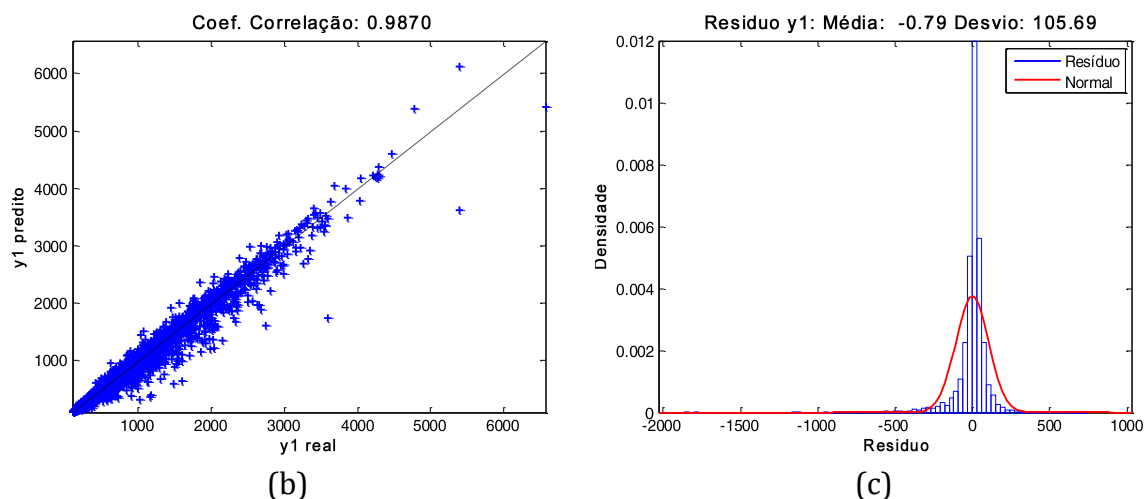


Figura 4.28: Resultados do modelo $ARX(1,2)$. (a) Correlação real x predito; (b) histograma do resíduo.

A Figura 4.29 mostra as estatísticas de validação para cada dia de previsão numa comparação entre os resultados do modelo de predição ARX e um modelo de “simulação” no qual os resultados do modelo ARX são utilizados como regressores. Nos dois casos, os valores observados da variável de entrada são utilizados no modelo. Podemos perceber que o desempenho do modelo piora muito à medida que se afasta do valor observado da variável de saída.

Os resultados da Figura 4.29 ainda são uma visão otimista do problema, pois utiliza os valores observados de chuva. Na realidade, a previsão deveria ser feita com a chuva prevista pelo modelo meteorológico, cujo erro aumenta com o dia de previsão, causando um efeito semelhante [169].

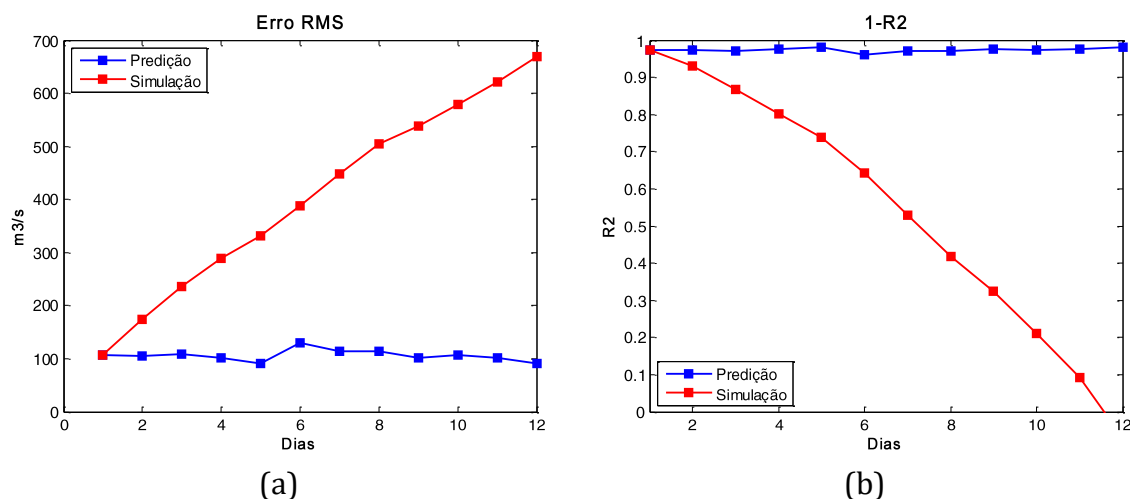


Figura 4.29: Resultados do modelo $ARX(1,2)$. (a) RMS; (b) $1 - R_2$.

Os resultados apresentados nesta seção ilustram a aplicação e as limitações do modelo de previsão ARX num problema complexo. Uma solução desse problema utilizando um modelo de sistemas *fuzzy* recorrente é apresentada na seção 8.4.4.

4.5 Comentários Finais

A modelagem de sistemas dinâmicos é um assunto complexo que tem sido bem explorado na literatura. Na abordagem clássica ou analítica [109][304], o modelo é uma representação do sistema por um conjunto de equações diferenciais em que os parâmetros têm significado físico. A modelagem de sistemas complexos pela abordagem analítica depende do conhecimento dos fenômenos físicos envolvidos e, em geral, é realizada a partir de hipóteses simplificadoras que eliminam efeitos secundários de difícil representação. O ajuste dos modelos é feito a partir de ensaios com o sistema real para a determinação dos parâmetros.

Com o desenvolvimento dos métodos estatísticos e da teoria de controle, principalmente na primeira metade do século XX, a modelagem de sistemas passou a focar no comportamento entrada-saída do sistema. Nessa abordagem, o objetivo da modelagem

é a obtenção de um modelo capaz de prever corretamente o valor de saída correspondente a um determinado valor de entrada. O ajuste do modelo é feito a partir de um conjunto de dados de entrada-saída e os parâmetros não têm correspondência com as características físicas do sistema real. Esse tipo de abordagem, também chamada de “caixa-preta”, é capaz de representar sistemas complexos cuja formulação matemática é muito complexa ou não conhecida.

Diversas aplicações envolvem a modelagem de sistemas dinâmicos, o que justifica o grande interesse pelo assunto. Em economia, a análise de modelos de regressão e séries temporais com dados econômicos é chamada de econometria [138] e utiliza ferramentas semelhantes às que foram discutidas neste capítulo. Em ciência de dados, modelos de dados temporais incluem outras aplicações, como agrupamento e descoberta de padrões em sequência [192], e outros tipos de dados temporais [264][393]. Recentemente, um grande avanço tem sido obtido no processamento de linguagem natural, e reconhecimento de fala, como será discutido no Capítulo 8.

5 CLASSIFICAÇÃO

O objetivo da classificação supervisionada, ou simplesmente classificação, é o desenvolvimento de um **classificador**, isto é, um modelo capaz de prever a classe correta de um registro a partir dos valores das variáveis de entrada. A classificação pode ser entendida como um problema de decisão em que uma classe é escolhida de acordo com os valores dos atributos do problema, que são as variáveis de entrada.

Um modelo de classificação divide de forma explícita ou implícita o espaço de variáveis de entrada em regiões que são associadas às classes. A fronteira entre as regiões é chamada de **superfície de decisão**. A predição da classe é feita a partir da verificação da região correspondente ao registro que se deseja classificar. Em problemas reais, registros com valores similares de variáveis de entrada podem pertencer a classes diferentes devido ao efeito de variáveis não observadas, ruído etc. Esse tipo de registro é tratado como erro de classificação e o modelo é ajustado para minimizar esse erro. Em alguns casos, chamados desbalanceados, as classes não têm o mesmo número de registros, o que pode levar o modelo a realizar previsões enviesadas.

Este capítulo apresenta o problema de classificação a partir da teoria bayesiana de decisão, que permite uma formalização matematicamente simples e bastante elucidativa sobre as principais características do problema. Em seguida, apresentamos o método de discriminante linear em que a classificação é tratada como um problema de predição análogo à regressão linear e o modelo de regressão logística, muito conhecido na literatura. Este capítulo também apresenta as estatísticas mais utilizadas para avaliação da qualidade do modelo de classificação e alguns exemplos de aplicação.

5.1 Classificação Bayesiana

O teorema (ou regra) de Bayes¹¹⁵ foi criado pelo pastor Thomas Bayes e publicado em 1764, após sua morte, pelo amigo e filósofo Richard Price. O teorema de Bayes introduz a interpretação bayesiana da probabilidade como uma medida do “grau de conhecimento” sobre um evento em contraste com a interpretação baseada na frequência que mede a propensão de um evento ocorrer.

A apresentação do teorema de Bayes nesta seção é focada no problema de classificação e utiliza uma notação que evidencia os valores de probabilidade como função das variáveis do problema.

5.1.1 Decisão bayesiana

O problema de classificação é essencialmente uma decisão, uma vez que uma das classes deve ser escolhida em função dos valores das variáveis de entrada. A tomada de decisão no problema de classificação pode ser formalizada pelo **teorema de Bayes** [146][521], que relaciona o conhecimento prévio do problema na forma:

$$P(C_i|\mathbf{x}) = \frac{p(\mathbf{x}|C_i)P(C_i)}{p(\mathbf{x})} \quad (5.1)$$

onde $P(C_i|\mathbf{x})$ é a probabilidade de observar a classe C_i conhecendo a observação (evidência) dos valores das variáveis $\mathbf{x}$, chamada **probabilidade a posteriori**; $p(\mathbf{x}|C_i)$ representa a distribuição de probabilidades das variáveis $\mathbf{x}$ quando a classe observada é C_i , chamada de distribuição* de **probabilidade condicional**; e $P(C_i)$ é a probabilidade de ocorrência da classe C_i na ausência de qualquer observação, chamada de **probabilidade a priori**. O denominador $p(\mathbf{x}) = \sum_{i=1}^m p(\mathbf{x}|C_i)P(C_i)$ é a probabilidade de observar valores das variáveis $\mathbf{x}$, que funciona como um fator de padronização para que o resultado permaneça no intervalo $[0,1]$.

A equação (5.1) mostra de forma simples como a probabilidade de um evento é alterada pelo conhecimento de evidências relacionadas. O teorema de Bayes realiza uma “inversão” do conhecimento representado pela probabilidade condicional, o que mostra o raciocínio abdutivo¹¹⁶ do problema de classificação.

* Adotamos a notação com p minúsculo para representar uma função distribuição de probabilidades e P maiúsculo para representar um valor de probabilidades.

Considere, por exemplo, o raciocínio de um médico para o diagnóstico de uma doença a partir dos sintomas observados. O resultado do diagnóstico é determinado pela probabilidade *a posteriori*, isto é, o médico vai escolher, dentre as hipóteses plausíveis para o quadro observado, aquela que julgar mais provável. O conhecimento do médico é representado pela probabilidade condicional que estabelece a probabilidade de observação de um conjunto de sintomas $\mathbf{x}$ na presença de alguma doença C_i . A probabilidade *a priori* influi na decisão do médico, que, na observação dos mesmos sintomas, vai ter a tendência de decidir pela doença mais frequente.

A interpretação da equação (5.1) no contexto do problema de classificação fornece os elementos para o correto entendimento das características do problema e serve para o desenvolvimento de modelos de classificação.

Vamos considerar um exemplo simples, com apenas uma variável de entrada e duas classes. Inicialmente, vamos supor que as duas classes ocorrem com a mesma frequência, ou seja, têm a mesma probabilidade *a priori* $P(C_1) = P(C_2) = 0.5$. Vamos considerar também que as duas classes têm distribuição de probabilidade condicional normal $N(\mu, \sigma)$ de média μ e desvio padrão σ . Para o exemplo discutido a seguir, foram utilizadas as distribuições $p(x|C_1) = N(2.5, 2.5)$ e $p(x|C_2) = N(-1.5, 2.0)$.

A distribuição de probabilidade condicional para cada uma das classes e os valores de probabilidade *a posteriori* calculados pela regra de Bayes são mostrados na Figura 5.1. A função de probabilidade condicional $p(x|C_i)$ informa a probabilidade de observar um valor x sabendo-se que a classe é C_i . No exemplo, a média da distribuição $p(x|C_1)$ é maior que a média de $p(x|C_2)$.

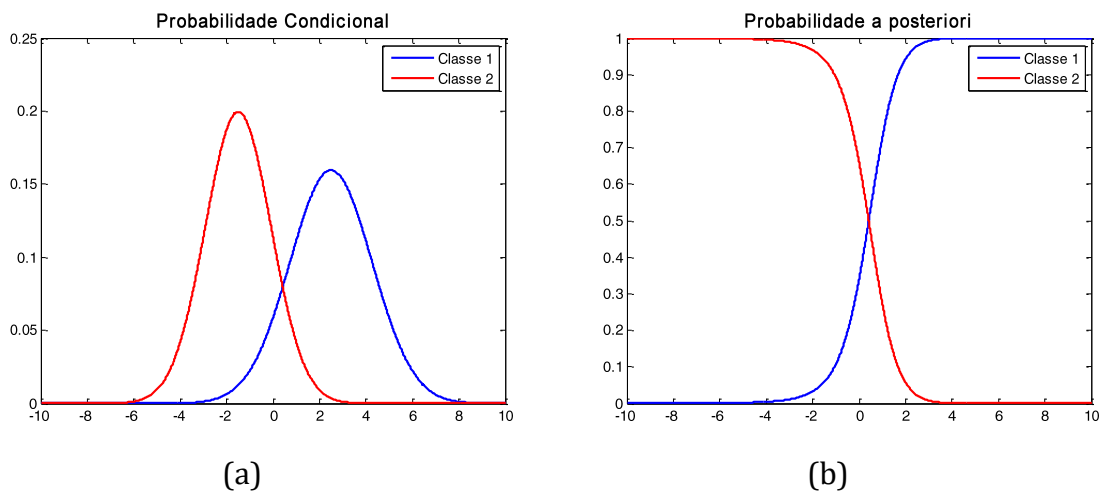


Figura 5.1: Regra de Bayes: (a) probabilidade condicional; (b) probabilidade *a posteriori*.

No classificador bayesiano, o resultado do problema de classificação é obtido por uma **regra de decisão**, definida sobre os valores de probabilidade *a posteriori* das classes calculados pelo teorema de Bayes.

5.1.1.1 Decisão pelo mínimo erro de classificação

A decisão mais simples para a classificação bayesiana é feita pelo valor máximo de probabilidade *a posteriori*, que resulta na divisão do espaço das variáveis de entrada em duas regiões, cada uma associada a uma das classes, da seguinte forma:

$$\begin{aligned}\mathcal{R}_1 &= \{x \in \mathcal{R}, P(C_1|x) \geq P(C_2|x)\} \\ \mathcal{R}_2 &= \{x \in \mathcal{R}, P(C_2|x) > P(C_1|x)\}\end{aligned}\tag{5.2}$$

A Figura 5.1a mostra que valores da variável na região de sobreposição das funções de densidade condicional podem pertencer simultaneamente às duas classes com probabilidades diferentes. Dessa forma, considerando essa única variável, haverá um **erro de classificação**, calculado pela probabilidade de classificação incorreta como:

$$P(E) = \int_{\mathcal{R}_1} p(x|C_2)P(C_2)dx + \int_{\mathcal{R}_2} p(x|C_1)P(C_1)dx\tag{5.3}$$

O primeiro termo à direita da equação (5.3) representa o percentual de registros da classe C_2 que serão classificados como C_1 por estarem na região R_1 . Analogamente, o segundo termo representa o percentual de registros da classe C_1 que serão classificados como C_2 . O resultado da equação (5.3) para o exemplo da Figura 5.1 é mostrado na Figura 5.2, na qual as distribuições condicionais $p(x|C_i)$ são mostradas em linhas tracejadas e as distribuições $p(x|C_i)P(C_i)$, em linhas cheias.

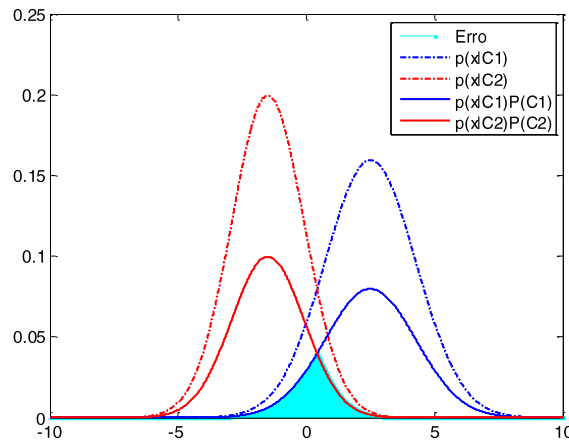


Figura 5.2: Probabilidade de classificação incorreta.

É fácil mostrar que a classificação realizada pelas regiões $\mathcal{R}_1$ e $\mathcal{R}_2$ definidas nas equações (5.2) minimizam o erro definido pela equação (5.3) [521]. Observando a Figura 5.2, uma regra de decisão diferente de (5.2) resulta num aumento da área correspondente à equação (5.3), ou seja, da probabilidade de erro de classificação.

As regiões associadas às classes pela regra de decisão (5.2) seriam as mesmas se a decisão fosse realizada diretamente sobre a função de distribuição condicional, pois, no exemplo da Figura 5.2, as classes são equiprováveis. Quando a frequência relativa das classes é diferente, esse resultado não é válido e a região da classe mais frequente aumenta em relação à da menos frequente para refletir o efeito das classes mais frequentes. A Figura 5.3 mostra o resultado da probabilidade *a posteriori* quando as probabilidades *a priori* são diferentes. Observe o deslocamento do ponto de decisão em relação ao ponto de interseção das distribuições de probabilidade condicionais.

No ponto de decisão, ou seja, no valor limite entre as regiões $\mathcal{R}_1$ e $\mathcal{R}_2$, os valores de probabilidade *a posteriori* são iguais e $p(x|C_1)P(C_1) = p(x|C_2)P(C_2)$. Dessa forma, podemos escrever:

$$\frac{p(x|C_1)}{p(x|C_2)} = \frac{P(C_2)}{P(C_1)} \quad (5.4)$$

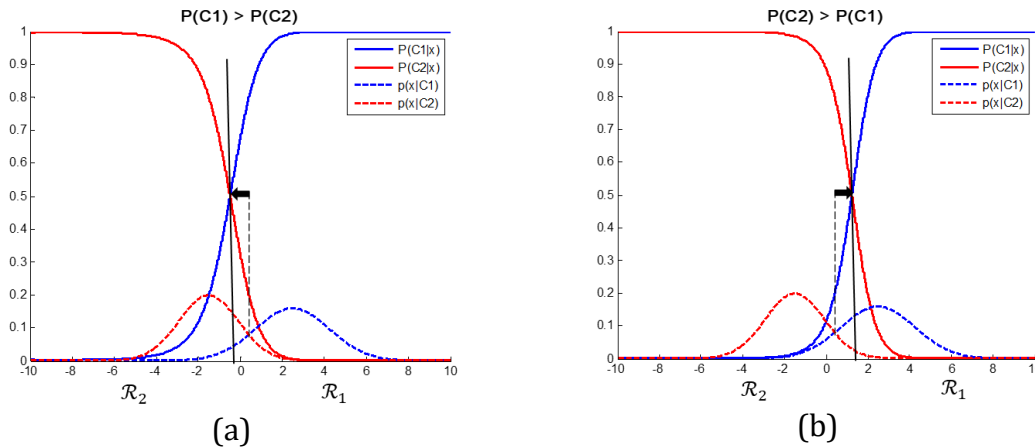


Figura 5.3: Regra de Bayes: (a) $P(C_1) > P(C_2)$; (b) $P(C_2) > P(C_1)$.

A equação (5.4) mostra que, no caso de problemas desbalanceados, a superfície de decisão é obtida quando a probabilidade condicional da classe menos frequente supera a probabilidade condicional da mais frequente na mesma razão do cociente das probabilidades *a priori*.

5.1.1.2 Decisão pelo mínimo risco condicional

No problema de duas classes em que uma é considerada positiva e outra, negativa, o resultado de um modelo de classificação pode ser apresentado pela **matriz de confusão** da Tabela 5.1. O resultado é chamado de verdadeiro positivo (VP) quando a classe positiva é predita corretamente pelo modelo ou verdadeiro negativo (VN) quando a classe negativa é predita corretamente. Os erros de classificação ocorrem de duas formas: se a classe positiva é predita quando a classe observada é negativa, o resultado é falso positivo (FP) (ou erro do Tipo I na nomenclatura de testes de hipóteses); o falso negativo (FN) (ou erro do Tipo II) acontece se a classe negativa é predita quando a classe observada é positiva.

Tabela 5.1: Matriz de confusão

	C_1 Predita	C_2 Predita
C_1 Real	VP	FN
C_2 Real	FP	VN

A matriz de confusão também será utilizada na seção 5.3 para avaliação de classificadores. Nesta seção, a matriz de confusão é utilizada para apresentar o conceito de risco de classificação incorreta.

Em muitas aplicações, principalmente quando as classes são desbalanceadas, o custo da classificação incorreta é diferente de acordo com o tipo de erro. Considere, por exemplo, um problema de diagnóstico médico a partir de um exame em que a classe positiva representa a presença de uma determinada doença grave. O erro FP consiste em alarmar o paciente desnecessariamente, enquanto o erro FN pode ter impacto importante no tratamento por atrasar a detecção da doença. Considerando que a probabilidade *a priori* de pacientes saudáveis é maior que a de pacientes portadores da doença, a classificação realizada de acordo com a regra (5.2) só detectaria a doença em casos nos quais as evidências fossem muito fortes.

O formalismo de decisão bayesiana permite representar o custo associado a cada decisão. O risco condicional associado a escolher uma classe C_i a partir da observação das evidências $\mathbf{x}$ é escrito como [146]:

$$R(C_i|\mathbf{x}) = \sum_{j=1}^m \lambda_{ij}P(C_j|\mathbf{x}) \quad (5.5)$$

onde λ_{ij} é o custo associado a escolher a classe C_i quando a classe correta é C_j .

No caso do problema de duas classes considerado anteriormente, o risco condicional associado a cada classe é calculado como:

$$\begin{aligned} R(C_1|x) &= \lambda_{11}P(C_1|x) + \lambda_{12}P(C_2|x) \\ R(C_2|x) &= \lambda_{21}P(C_1|x) + \lambda_{22}P(C_2|x) \end{aligned} \quad (5.6)$$

A decisão é calculada pelo menor risco condicional, que não é uma probabilidade. Considerando o risco de classificação correta nulo, $\lambda_{11} = \lambda_{22} = 0$, as regiões associadas às classes são calculadas como:

$$\begin{aligned} \mathcal{R}_1 &= \{x \in \mathcal{R}, \lambda_{21}P(C_1|x) \geq \lambda_{12}P(C_2|x)\} \\ \mathcal{R}_2 &= \{x \in \mathcal{R}, \lambda_{12}P(C_2|x) > \lambda_{21}P(C_1|x)\} \end{aligned} \quad (5.7)$$

A Figura 5.4 mostra a diferença da decisão de classificação calculada a partir do valor das probabilidades *a posteriori* (cf. equações (5.2)) ou pelo risco condicional (cf. equações (5.7)). No exemplo, as classes foram consideradas equiprováveis para facilitar a interpretação. Na Figura 5.4, as linhas tracejadas representam a distribuição de valores de probabilidade *a posteriori* e as linhas cheias, os valores de risco de classificação incorreta calculado pela equação (5.6).

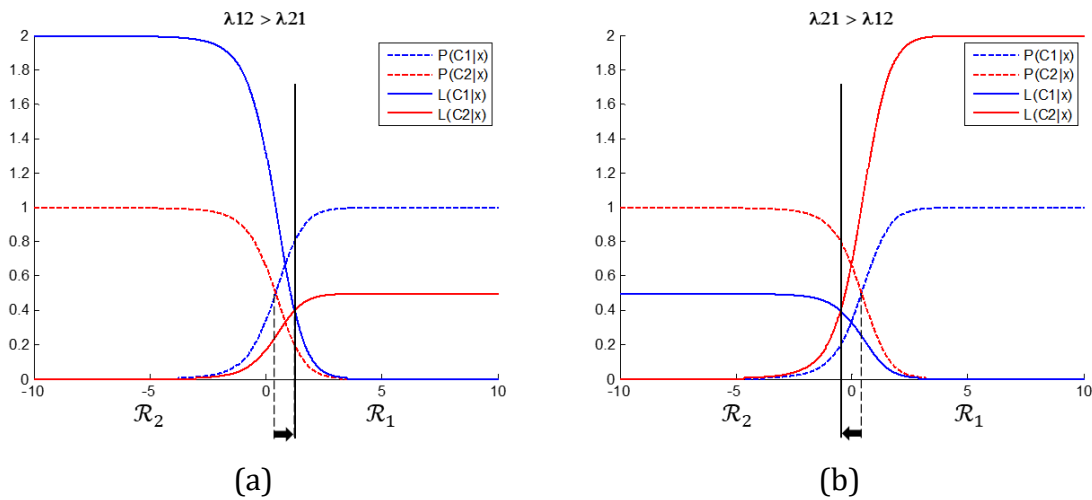


Figura 5.4: Decisão pelo risco de classificação: (a) $\lambda_{12} > \lambda_{21}$; (b) $\lambda_{21} > \lambda_{12}$.

Quando o custo de classificação incorreta $\lambda_{12} > \lambda_{21}$, ou seja, o erro em favor da classe C_1 é mais grave, a região $\mathcal{R}_1$ calculada pela minimização do risco condicional (cf. equações (5.7)) é menor que a correspondente calculada pela minimização do erro de classificação (cf. equações (5.2)). Dessa forma, o limite de decisão se desloca visando reduzir a região associada à classe com maior custo de classificação incorreta. Caso contrário, quando $\lambda_{21} > \lambda_{12}$, o limite de decisão se desloca em favor da classe C_1 uma vez que o custo de classificação incorreta dessa classe é menor.

O erro de classificação calculado pela equação (5.3) é maior caso a decisão seja realizada pelo risco condicional, uma vez que a decisão pela probabilidade *a posteriori* é a que minimiza o erro de classificação. Entretanto, embora globalmente o erro seja maior, ele será menor na classe de maior custo de classificação incorreta.

A aplicação da decisão pelo mínimo risco condicional depende de estimativas dos valores de custo λ_{ij} . Em problemas em que o custo pode ser quantificado, os parâmetros são definidos facilmente. Entretanto, na maioria dos casos, os custos não podem ser especificados diretamente e os valores são estimados de forma *ad hoc*. Em problemas desbalanceados, os custos de classificação incorreta podem ser calculados para compensar o efeito do desbalanceamento.

O formalismo da classificação bayesiana é aplicado diretamente sobre dois modelos de classificação que são apresentados a seguir.

5.1.2 Classificador bayesiano simples

No problema de classificação, o conjunto de dados $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$, onde cada registro t é formado por pares ordenados $(\mathbf{x}(t), y(t))$, sendo $\mathbf{x}(t) \in \mathcal{R}^p$ o vetor das variáveis de entrada e a variável de saída $y(t) \in \mathcal{N}$, em que valor $y(t) = j$ associa o registro t à classe $C_j, j = 1, \dots, m$. É importante observar que os índices não representam uma relação de ordem entre as classes, apenas facilitam a representação das classes de forma simbólica.

O classificador bayesiano utiliza diretamente a regra de Bayes (5.1) para o cálculo da probabilidade *a posteriori* e realiza a decisão por uma das formas apresentadas anteriormente, geralmente pelo mínimo erro de classificação. No classificador **bayesiano simples** ou ingênuo (*naïf*), a distribuição de probabilidade condicional dos registros

do conjunto de treinamento é calculada, para cada classe, por uma estimativa que considera as variáveis de entrada independentes. Dessa forma, o classificador bayesiano simples não leva em consideração a covariância entre as variáveis e a distribuição de probabilidade condicional conjunta para cada classe é calculada como:

$$p(\mathbf{x}(t)|C_j) = \prod_{i=1}^p p(x_i(t)|C_j), \quad j = 1, \dots, m \quad (5.8)$$

No modelo mais comum, a probabilidade condicional de cada variável é estimada como uma distribuição normal monovariável $p(x_i(t)|C_j) \approx N(\hat{\mu}_{ij}, \hat{\sigma}_{ij}^2)$. Os parâmetros de média $\hat{\mu}_{ij}$ e de variância $\hat{\sigma}_{ij}^2$ são estimados no conjunto de treinamento como a média e a variância da variável $x_i(t)$ nos registros da classe C_j . Há variantes que utilizam outros modelos de distribuição condicional.¹¹⁷

A hipótese de independência entre as variáveis de entrada reduz consideravelmente o número de parâmetros a serem ajustados pelo classificador. A estimativa da matriz de covariâncias da distribuição normal multivariada (cf. equação (2.6)) requer $p(p + 1)/2$ parâmetros para cada classe, o que resulta num total de $p + p(p + 1)/2$ parâmetros, incluindo as médias. No classificador bayesiano simples, a hipótese de independência reduz o número de parâmetros a serem ajustado a apenas $2p$ para cada classe.

O ajuste de parâmetros do classificador bayesiano simples é muito eficiente. Para cada classe, para cada variável, calcula-se a média $\hat{\mu}_{ij}$ e a variância $\hat{\sigma}_{ij}^2$ estimadas – se a variável for numérica – ou então diretamente a frequência de cada valor nominal – quando a variável for categórica. A probabilidade *a priori* também é calculada diretamente pela frequência da classe.

Na avaliação, o resultado do classificador bayesiano simples é a probabilidade *a posteriori*, calculada diretamente pelo teorema de Bayes (cf. equação (5.1)), em que a probabilidade condicional conjunta é calculada pela equação (5.8). A decisão de classificação pode ser feita pela regra de mínimo erro de classificação ou de mínimo risco.

A Figura 5.5a mostra um exemplo sintético de um problema linearmente separável em $p = 2$ dimensões. A Figura 5.5b apresenta a superfície de decisão e as regiões associadas a cada classe pelo valor máximo de probabilidade *a posteriori*. As Figuras 5.5c e 5.5d mostram, respectivamente, as distribuições de probabilidade condicional normal ajustadas para cada uma das variáveis. Podemos perceber que, apesar de existir correlação

entre as variáveis, a diferença das médias das distribuições de probabilidade condicional permite ajustar um classificador capaz de gerar uma superfície de decisão eficiente.

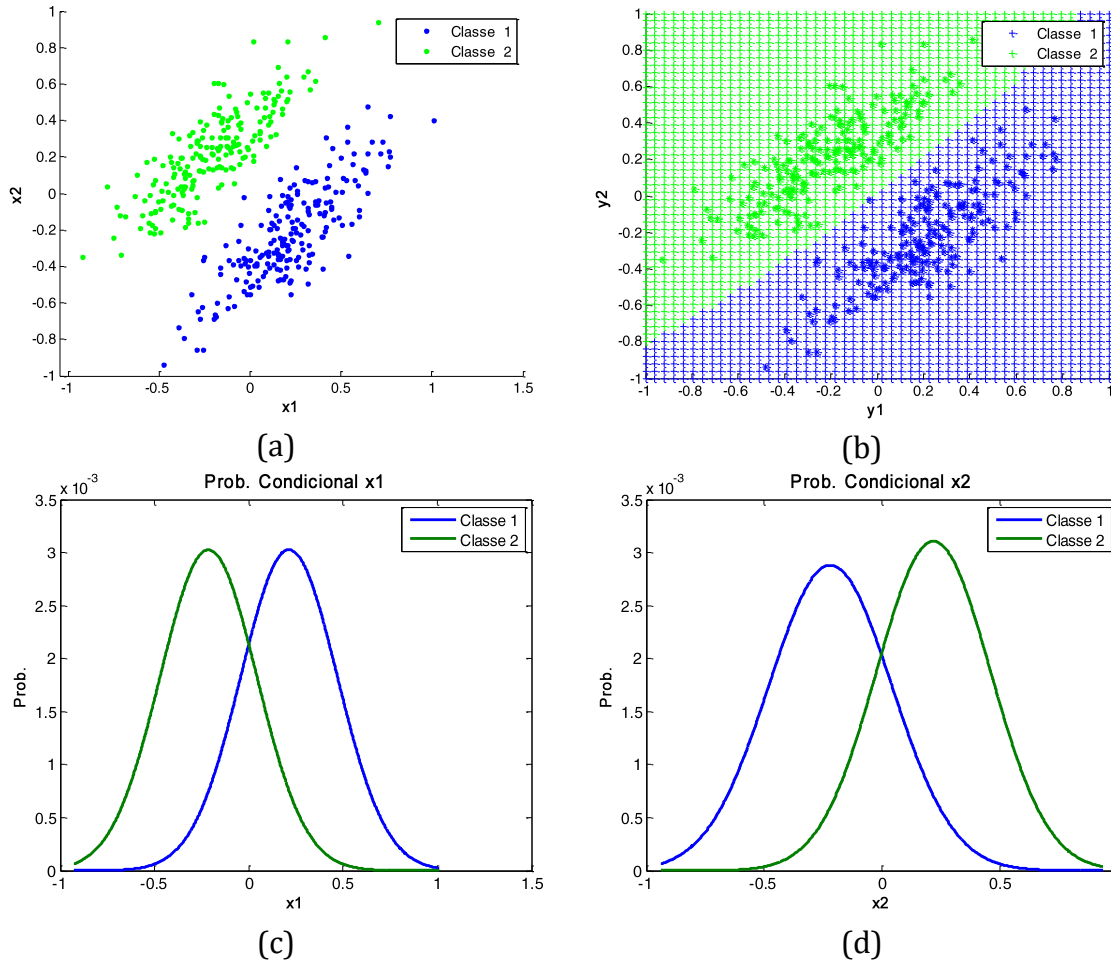


Figura 5.5: Problema linearmente separável: (a) Conjunto de treinamento; (b) superfície de decisão; (c) distribuição $p(x_1|C_j)$; (d) distribuição $p(x_2|C_j)$.

A “economia” de parâmetros e a simplicidade do classificador bayesiano simples têm efeitos no seu desempenho. A Figura 5.6a mostra um exemplo um pouco mais complexo, no qual as classes não podem ser separadas por uma superfície linear – são, assim, chamadas de não separáveis linearmente. Nesse caso, a superfície de decisão gerada pelo classificador bayesiano simples, mostrada na Figura 5.6b, não permite separar as duas classes. O resultado pode ser entendido observando-se as distribuições condicionais nas Figura 5.6c e 5.6d, que mostram que a projeção dos dados em cada variável resulta em valores similares das médias e das variâncias das classes e, portanto, a distribuição conjunta das duas classes é muito parecida, de forma que a superfície de decisão gerada não é eficiente para separá-las.

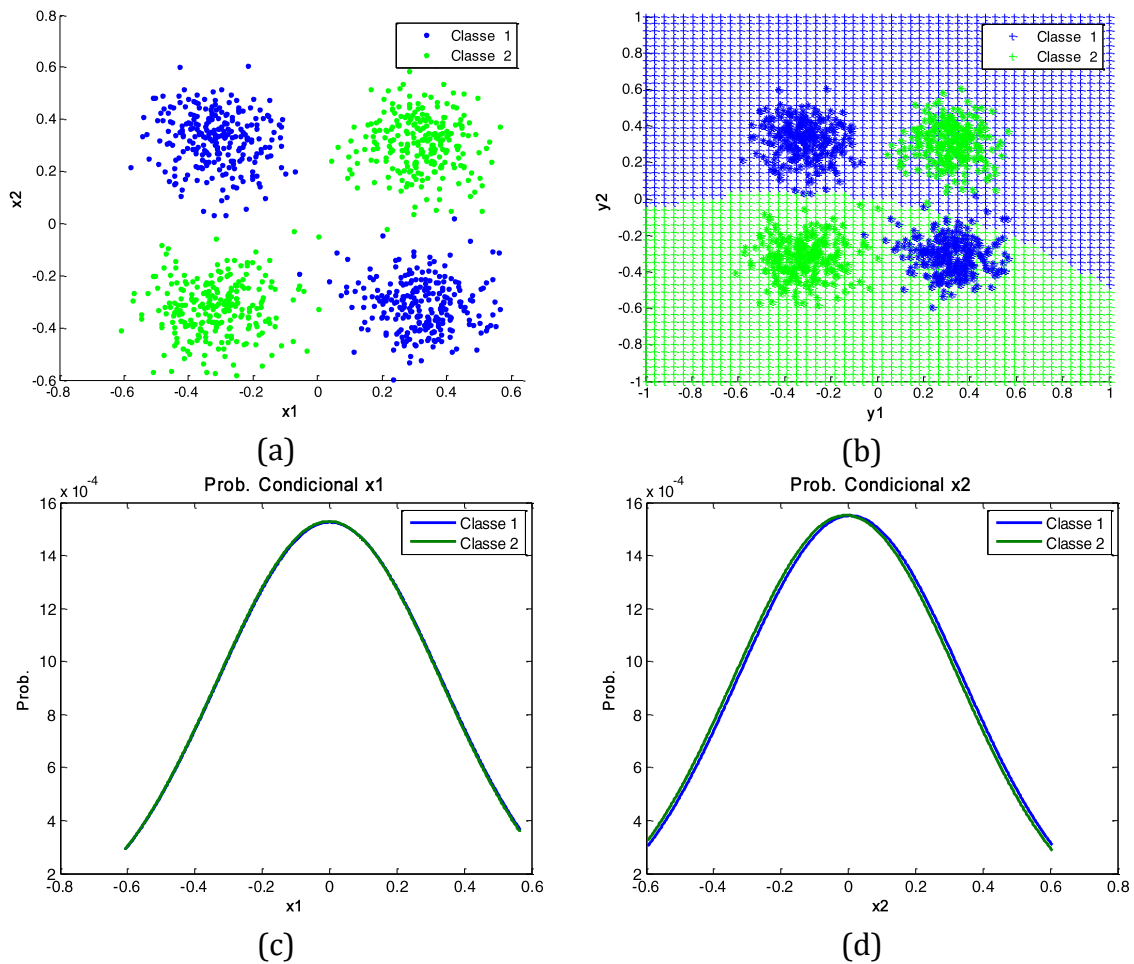


Figura 5.6: Problema não separável linearmente: (a) conjunto de treinamento; (b) superfície de decisão; (c) distribuição $p(x_1|C_j)$; (d) distribuição $p(x_2|C_j)$.

A distribuição normal não é adequada para a representação da distribuição dos dados de cada classe no exemplo da Figura 5.6a. Entretanto, o problema principal é que o classificador bayesiano simples não é capaz de capturar a correlação existente entre as variáveis no conjunto de dados de cada classe. A Figura 5.7a mostra o mesmo conjunto de dados da Figura 5.6a após a transformação realizada pela análise de componentes principais (ACP), que elimina a correlação entre as variáveis. Nesse caso, o classificador bayesiano é capaz de separar as classes (cf. Figura 5.7b), mesmo que a distribuição normal não seja a melhor representação da distribuição das classes, como pode ser observado na Figura 5.7c e Figura 5.7d. Nesse caso, a diferença de variâncias das distribuições de probabilidade condicional permite gerar uma superfície de decisão correta.

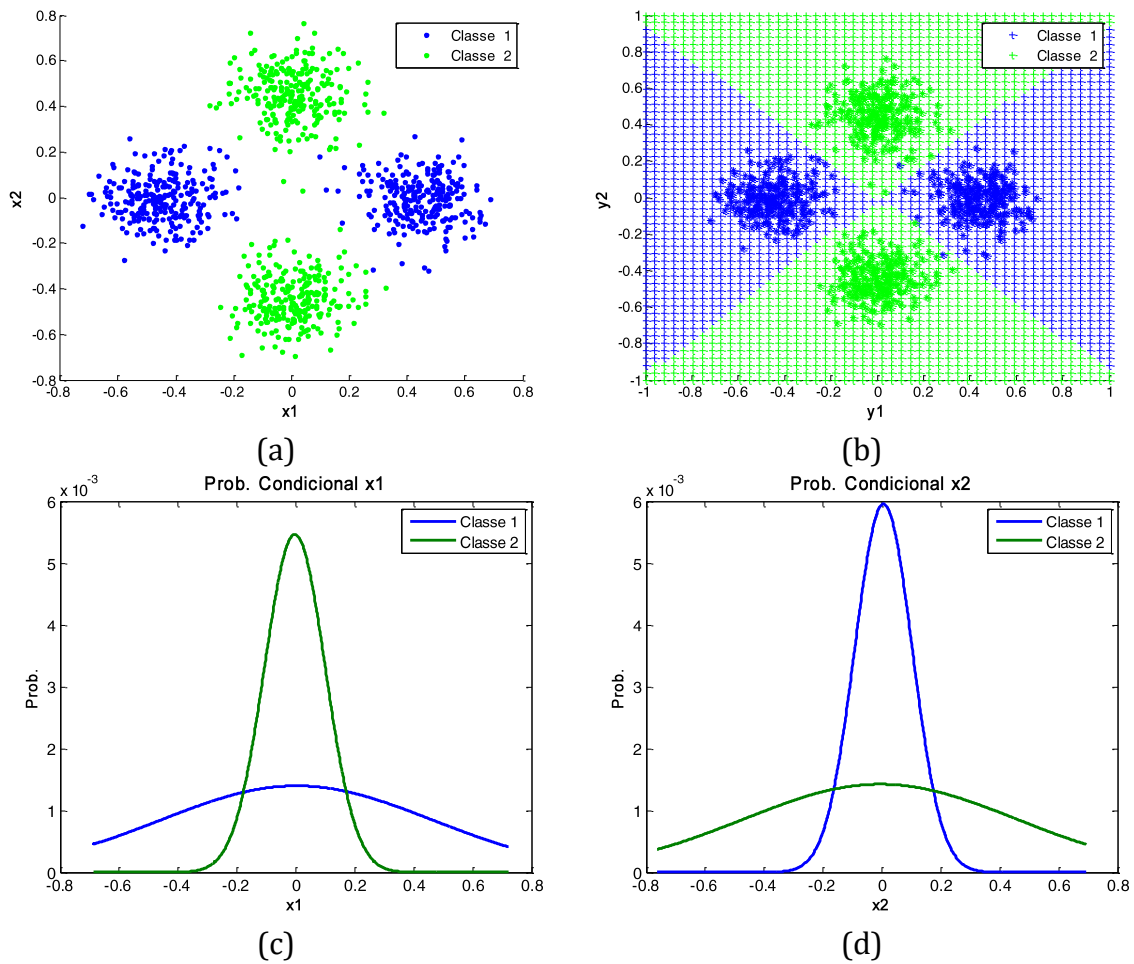


Figura 5.7: Problema transformado pela ACP: (a) conjunto de treinamento; (b) superfície de decisão; (c) distribuição $p(x_1|C_j)$; (d) distribuição $p(x_2|C_j)$.

A utilização da ACP para eliminar a correlação dos dados nem sempre produz bons resultados, mas é um recurso que pode ser empregado na busca pelo melhor modelo de classificação. Na seção 5.4.1, os resultados do classificador bayesiano simples são comparados aos resultados de outros classificadores apresentados neste capítulo.

O classificador bayesiano simples é um classificador computacionalmente eficiente, que pode ser aplicado em problemas de grande dimensionalidade e permite combinar facilmente variáveis numéricas e categóricas. Um classificador bayesiano simples pode facilmente acomodar valores ausentes na avaliação, o que é um problema em diversos outros tipos de classificadores. O algoritmo básico, com algumas variações, pode ser encontrado em praticamente todos os *softwares* de ciência de dados. Apesar do seu desempenho regular, o classificador bayesiano tem muitas aplicações em ciência de dados [289], tendo se mostrado útil na classificação de documentos, em filtros de *spam*

[115] e na análise de sentimentos [348][427]. O bom desempenho do classificador bayesiano simples também o qualifica para utilização em *ensembles* [325], que combinam os resultados de vários modelos simples para melhorar o desempenho do classificador.

Em conjuntos de dados com registros suficientes para estimação das distribuições de probabilidade condicionais conjuntas, é possível utilizá-la na síntese do classificador, como descrito a seguir.

5.1.3 Classificador bayesiano quadrático

Um classificador realiza a predição da classe de um novo registro pela sua localização em relação às regiões associadas às classes. No caso de um problema com m classes, o resultado do classificador vai particionar o domínio das variáveis de entrada em m regiões. É possível generalizar regras de decisão como (5.2) e (5.7) pela definição de **funções discriminantes** em problemas de múltiplas classes. A classificação é feita pelo valor máximo da função discriminante como:

$$\mathbf{x}(t) \in C_i \rightarrow g_i(\mathbf{x}(t)) > g_j(\mathbf{x}(t)), \quad j = 1, \dots, m \quad (5.9)$$

No classificador bayesiano, a função discriminante pode ser calculada diretamente como $g_i(\mathbf{x}(t)) = P(C_i|\mathbf{x}(t))$ no caso de minimização da probabilidade de classificação incorreta ou como $g_i(\mathbf{x}(t)) = -R(C_i|\mathbf{x}(t))$ no caso de minimização do risco condicional, onde $R(C_i|\mathbf{x}(t))$ é definido em (5.5) e o sinal negativo é para manter coerência com a equação (5.9), uma vez que a decisão é tomada pelo menor risco de classificação. A escolha da função discriminante não é única e qualquer função monótona crescente de uma função discriminante $f(g_i(\mathbf{x}(t)))$ é também uma função discriminante.

A questão fundamental para a síntese de um classificador bayesiano é a escolha da função de distribuição de probabilidade condicional. No classificador bayesiano simples, a hipótese de independência resulta num classificador computacionalmente eficiente, mas de precisão apenas regular. O desempenho do classificador pode melhorar bastante se a distribuição condicional for mais bem estimada. Há métodos estatísticos, paramétricos e não paramétricos, para a estimativa de uma função de distribuição condicional [553]. A abordagem clássica é utilizar a distribuição normal multivariada como uma aproximação da distribuição condicional.

A hipótese de a probabilidade condicional ser uma distribuição normal permite a definição do classificador **bayesiano quadrático** [146][405], cuja função discriminante para a decisão pela minimização do erro de classificação é calculada como:

$$g_i(\mathbf{x}(t)) = \ln P(C_i|\mathbf{x}(t)) = \ln p(\mathbf{x}(t)|C_i) + \ln P(C_i), \quad i = 1, \dots, m \quad (5.10)$$

onde $p(\mathbf{x}(t)|C_i) = N(\hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i)$ é a distribuição normal multivariada de médias $\hat{\boldsymbol{\mu}}_i$ e matriz de covariâncias $\hat{\boldsymbol{\Sigma}}_i$ estimadas a partir dos registros da classe C_i .

Substituindo a expressão da distribuição normal multivariada na equação (5.10):

$$g_i(\mathbf{x}(t)) = -\frac{1}{2}(\mathbf{x}(t) - \hat{\boldsymbol{\mu}}_i)\hat{\boldsymbol{\Sigma}}_i^{-1}(\mathbf{x}(t) - \hat{\boldsymbol{\mu}}_i)^T - \frac{1}{2}\ln 2\pi - \frac{1}{2}\ln|\hat{\boldsymbol{\Sigma}}_i| + \ln P(C_i) \quad (5.11)$$

O termo $-\frac{1}{2}\ln 2\pi$ é o mesmo para todas as classes e, por isso, não exerce influência na função discriminante, podendo ser eliminado.

A expressão da função discriminante calculada pela equação (5.11) é dominada por um termo quadrático e, por isso, o classificador bayesiano com distribuições normais multivariadas é chamado de classificador bayesiano quadrático. A Figura 5.8 mostra dois exemplos sintéticos de classes geradas por variáveis de distribuição normal e a superfície de decisão calculada pela função discriminante (5.11).

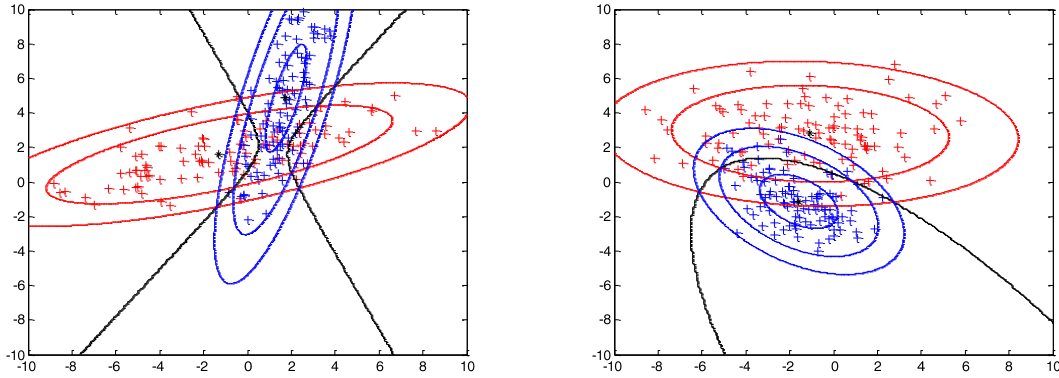


Figura 5.8: Exemplos de superfícies de decisão quadráticas.

O resultado do classificador bayesiano quadrático no conjunto de dados da Figura 5.6 é mostrado na Figura 5.9, na qual podemos observar que função discriminante é eficiente para a separação das classes.

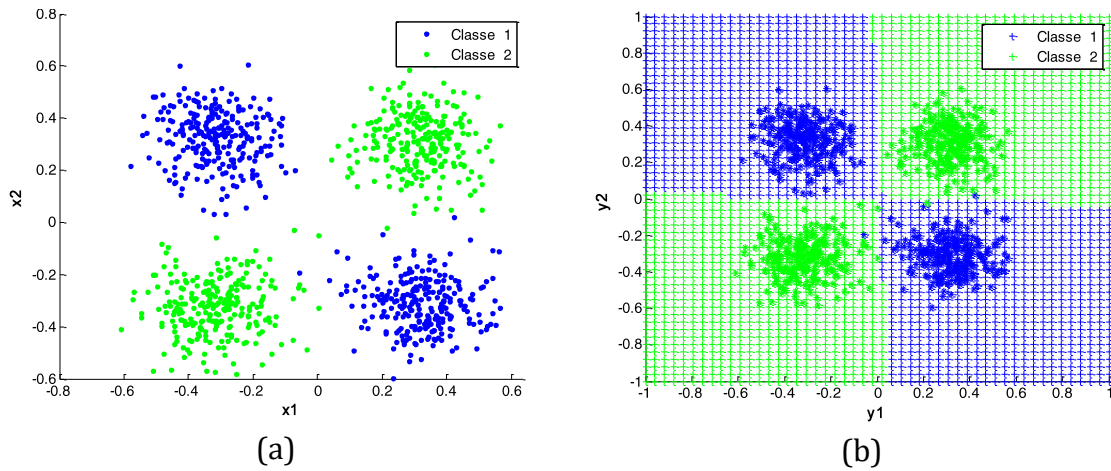


Figura 5.9: Problema não separável linearmente: (a) conjunto de treinamento; (b) superfície de decisão pelo classificador bayesiano quadrático.

A estimativa dos parâmetros da equação normal para o cálculo da função discriminante como (5.11) pode se tornar inviável em problemas de alta dimensionalidade, uma vez que são necessários $p + p(p + 1)/2$ parâmetros para cada classe. Considere, por exemplo, o conjunto de dados *Wine*, com $p = 13$ variáveis de entrada, o que resulta em 104 parâmetros a serem estimados para cada classe. O conjunto de dados conta apenas 178 registros, dos quais 59 da classe C_1 , 71 da classe C_2 e 48 da classe C_3 , o que prejudica as estimativas de média e matriz de covariâncias.

Em problemas de alta dimensionalidade com reduzido número de registros, é possível utilizar a redução de dimensionalidade produzida por transformações lineares como a ACP e a ADL (análise discriminante linear). Aplicando a ADL no conjunto de dados *Wine*, o número de variáveis é reduzido para $\hat{p} = m - 1 = 2$, o que significa estimar apenas três parâmetros para a média e a matriz de covariâncias. A Figura 5.10 apresenta o resultado da superfície de decisão calculada pelo classificador bayesiano quadrático sobre o conjunto de dados *Wine* 2D, obtido após a transformação pela ADL do conjunto *Wine*. Podemos observar o contorno quadrático das funções discriminantes de cada classe.

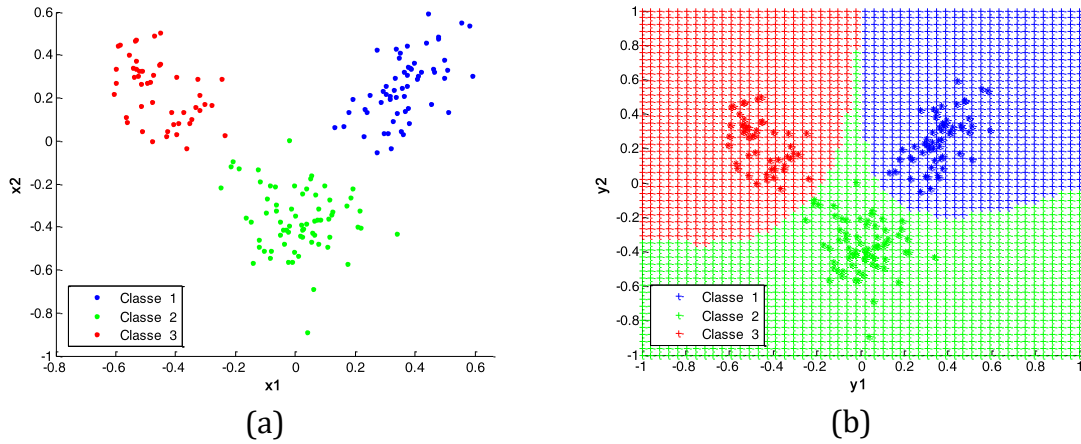


Figura 5.10: Conjunto de dados *Wine* 2D: (a) conjunto de treinamento; (b) superfície de decisão pelo classificador bayesiano quadrático.

O classificador bayesiano com distribuições normais multivariadas gera uma função discriminante linear quando a matriz de covariâncias é a mesma para todas as classes e é estimada como a matriz de covariâncias dos dados, isto é, $\hat{\Sigma}_i = \hat{\Sigma}, i = 1, \dots, m$. Nesse caso, o termo quadrático da equação (5.11) pode ser ignorado (por ser o mesmo para todas as classes) e a função discriminante é escrita como:

$$g_i(\mathbf{x}(t)) = \mathbf{x}(t)\hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}_i^T - \frac{1}{2}\hat{\boldsymbol{\mu}}_i\hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}_i^T + \ln P(C_i) \quad (5.12)$$

A equação (5.12) pode ser escrita na forma de uma função linear como:

$$g_i(\mathbf{x}(t)) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}_i^T \quad (5.13)$$

onde $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ é o vetor de regressores e $\boldsymbol{\theta}_i = [\theta_{i0}, \boldsymbol{\theta}_{i1}]$ é o vetor de parâmetros, cujos componentes $\theta_{i0} = -\frac{1}{2}\hat{\boldsymbol{\mu}}_i\hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}_i^T + \ln P(C_i)$ e $\boldsymbol{\theta}_{i1} = \hat{\Sigma}^{-1}\hat{\boldsymbol{\mu}}_i^T$.

O classificador bayesiano linear não requer um número considerável de parâmetros a serem estimados, pois apenas uma matriz de covariâncias é estimada para todas as classes. Entretanto, na prática, o desempenho pode ser pior que o classificador bayesiano simples.

Do ponto de vista conceitual, a equação (5.13) mostra que um classificador pode ser estimado diretamente por sua função discriminante, sem a necessidade de hipóteses sobre a função de distribuição de probabilidades condicional. Dessa forma, os modelos lineares podem ser aplicados à classificação, como apresentado a seguir.

5.2 Modelos Lineares de Classificação

A equação (5.13) mostra que um modelo linear pode ser utilizado para a aproximação da função discriminante num problema de classificação. Dessa forma, os problemas de classificação e de regressão são similares, diferindo apenas no tipo de variável de saída.

A síntese do classificador pode ser realizada diretamente pelo ajuste dos parâmetros de um modelo linear pelo método dos mínimos quadrados (MMQ), como apresentado nas seções 5.2.1 e 5.2.2. Entretanto, um modelo linear não permite a interpretação da função discriminante como um valor de probabilidade. Na regressão logística, apresentada na seção 5.2.3, o resultado do modelo linear é transformado por uma função exponencial que permite a interpretação do resultado como um valor de probabilidade.

5.2.1 Mínimos quadrados

O ajuste de parâmetros do modelo linear é feito a partir do conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$, onde $\mathbf{x}(t) \in \mathcal{R}^p$ o vetor das variáveis de entrada e a saída $y(t) \in \mathcal{N}$, o índice das classes.

Para o ajuste da função discriminante, a variável de saída é transformada para refletir o resultado da função discriminante em (5.9). Essa transformação é feita pela definição de uma **variável indicadora** (“dummy”) bipolar $v_i(t) \in \{-1, 1\}$, como:*

$$v_i(t) = \begin{cases} +1, & \text{se } y(t) = i \\ -1, & \text{se } y(t) \neq i \end{cases} \quad i = 1, \dots, m \quad (5.14)$$

O vetor $\mathbf{v}(t) \in \{-1, 1\}^m$ é chamado de vetor de variáveis indicadoras, pois a posição do valor 1 no vetor indica a classe a que pertence o registro.

As funções discriminantes são calculadas pelo modelo linear como aproximações das variáveis indicadoras como:

$$\hat{v}_i(t) = g_i(\mathbf{x}(t), \boldsymbol{\theta}_i) = \hat{\mathbf{x}}(t) \boldsymbol{\theta}_i^T, \quad i = 1, \dots, m \quad (5.15)$$

As variáveis indicadoras estimadas $\hat{\mathbf{v}}(t) \in \mathcal{R}^m$ podem ser utilizadas para a classificação através da regra (5.9). Os parâmetros $\boldsymbol{\theta}_i$ são calculados, para cada classe, pela minimização do erro médio quadrático (EMQ):

* A variável indicadora binária $v_i(t) \in \{0, 1\}$ também pode ser utilizada.

$$EMQ(\theta_i) = \frac{1}{N} \sum_{t=1}^N (v_i(t) - \hat{v}_i(t))^2, \quad i = 1, \dots, m \quad (5.16)$$

A minimização do EMQ para cada classe é feita pelo MMQ da mesma forma que apresentada na seção 3.1.2. A relação *bias* x variância do modelo discutida na seção 3.2.1 e as técnicas de regularização apresentadas nas sessões 3.2.2 e 3.2.3 se aplicam igualmente ao modelo linear de classificação. A avaliação de classificadores, entretanto, é feita por estatísticas adicionais, discutidas na seção 5.3.

O modelo linear de classificação permite a síntese de um classificador por uma função discriminante linear nos parâmetros. No modelo linear polinomial, descrito na equação (3.3), a estrutura é definida pelo grau do polinômio. A Figura 5.11 apresenta dois exemplos de modelos polinomiais de classificação executados sobre os mesmos dados do exemplo mostrado na Figura 5.10: o conjunto de dados *Wine* após transformação pela ADL. A Figura 5.11a mostra o resultado do modelo linear, ou polinomial de grau 1, e a Figura 5.11b mostra o resultado do modelo quadrático ou polinomial de grau 2.

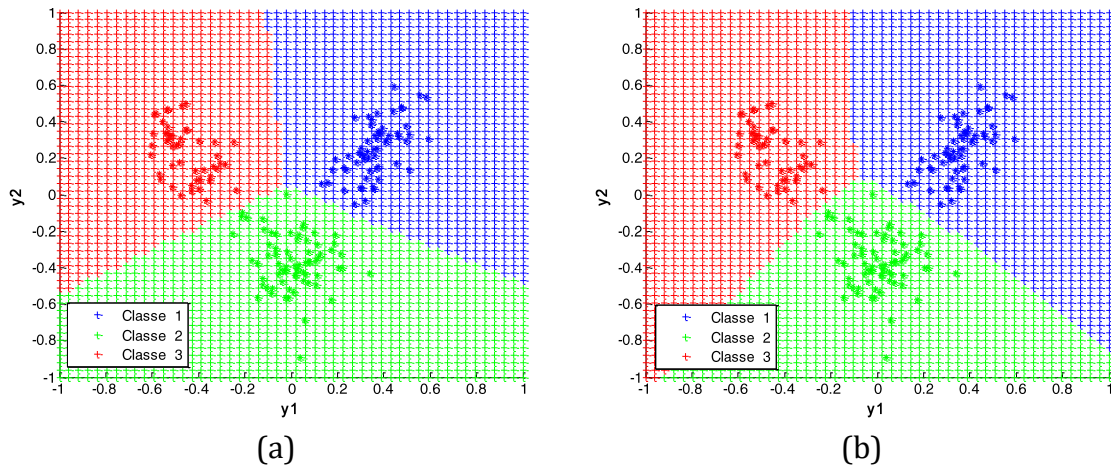


Figura 5.11: Superfícies de decisão no problema *Wine* 2D: (a) modelo linear; (b) modelo quadrático.

Os resultados da Figura 5.11 são ligeiramente diferentes entre si, mas há uma grande diferença entre o resultado da Figura 5.11b e o do classificador bayesiano quadrático, mostrado na Figura 5.10b. Em ambos os casos, o modelo é quadrático, mas, no classificador linear, a estrutura do modelo foi definida pela escolha do grau do polinômio, enquanto no classificador bayesiano, a estrutura do modelo foi definida pela hipótese de distribuição normal. O modelo linear foi ajustado pelo MMQ, enquanto o classificador bayesiano foi ajustado pelas médias e matrizes de covariâncias de cada classe.

O ajuste de parâmetros pelo MMQ obtém um classificador que realiza a decisão pela minimização do erro de classificação. Entretanto, em alguns problemas, é importante realizar a decisão pela minimização do risco condicional, que implica ponderar uma classe em relação à outra. É possível ponderar os registros de uma classe em relação a outra através do método dos mínimos quadrados ponderados descrito a seguir.

5.2.2 Mínimos quadrados ponderados

O método dos mínimos quadrados ponderados (MMQP) permite ajustar os parâmetros de um modelo linear na situação em que alguns registros são mais “importantes” que outros. O MMQP é utilizado na regressão linear para estabilizar a variância do conjunto de dados e reduzir a influência de *outliers* [399]. Em modelos lineares de classificação, o MMQP pode atribuir maior peso aos registros da classe de maior risco, permitindo que a decisão de classificação seja feita pela minimização do risco condicional.

A formulação do MMQP é feita a partir da definição de um vetor de pesos $\mathbf{w} \in \mathcal{R}^N$, em que cada elemento $w(t)$ representa o peso do registro $t \in \mathcal{T}$. Os parâmetros do modelo para cada classe $\boldsymbol{\theta}_i$ são calculados pela minimização do **erro médio quadrático ponderado** (EMQP):

$$EMQP(\boldsymbol{\theta}_i) = \frac{1}{N} \sum_{t=1}^N w(t) (v_i(t) - \hat{v}_i(t))^2, \quad i = 1, \dots, m \quad (5.17)$$

onde os valores de ponderação são os mesmos para o cálculo dos parâmetros de todas as classes, uma vez que a ponderação é aplicada aos registros.

Adotando a representação vetorial em que $\mathbf{V}_i = [v_i(1), \dots, v_i(N)]^T$ é a coluna i da matriz de saída, cujos elementos são os valores da variável indicadora da classe C_i , $\hat{\mathbf{X}}$ é a matriz de regressores (cf. equação (3.5)) e $\mathbf{W} = \text{diag}(\mathbf{w})$, a matriz diagonal em que cada elemento é o peso do registro correspondente, o EMQP pode ser escrito como:

$$EMQP(\boldsymbol{\theta}_i) = \frac{1}{N} (\mathbf{V}_i - \hat{\mathbf{X}}\boldsymbol{\theta}_i^T)^T \mathbf{W} (\mathbf{V}_i - \hat{\mathbf{X}}\boldsymbol{\theta}_i^T), \quad i = 1, \dots, m \quad (5.18)$$

Derivando-se a expressão do EMQP na equação (5.18) e igualando a zero, obtemos a expressão das equações normais do MMQP, semelhante à equação (3.7):

$$\hat{\mathbf{X}}^T \mathbf{W} \hat{\mathbf{X}} \boldsymbol{\theta}_i^T = \hat{\mathbf{X}}^T \mathbf{W} \mathbf{V}_i, \quad i = 1, \dots, m \quad (5.19)$$

A solução do sistema linear (5.19), considerando que a matriz* $\hat{\mathbf{X}}^T \mathbf{W} \hat{\mathbf{X}}$ tem posto completo, é calculada pela pseudoinversa como:

$$\theta_i^* = (\hat{\mathbf{X}}^T \mathbf{W} \hat{\mathbf{X}})^{-1} \hat{\mathbf{X}}^T \mathbf{W} \mathbf{v}_i, \quad i = 1, \dots, m \quad (5.20)$$

Os métodos de regularização discutidos na seção 3.2 se aplicam-se igualmente à solução do sistema linear da equação (5.19).

Na seção 3.1.3, foi visto que a solução calculada pelo EMQ produz melhor aproximação possível do conjunto de dados. Dessa forma, a solução calculada pelo MMQP terá um erro maior. Em problemas de classificação, a solução do MMQ corresponde à decisão pela minimização do erro de classificação na abordagem bayesiana, enquanto a solução do MMQP corresponde à minimização do risco condicional. Analogamente, o MMQP produz um erro global maior, mas terá um erro menor da classe mais importante.

Num problema de classificação desbalanceado, a classe minoritária é, em geral, a mais importante. Nesse caso, podemos definir a ponderação de cada registro pelo complemento do valor de probabilidade *a priori* da classe correspondente:

$$w(t) = 1 - \frac{N_i}{N} \quad \forall \mathbf{x}(t) \in C_i, \quad i = 1, \dots, m \quad (5.21)$$

onde N_i é o número de registros da classe C_i .

A ponderação realizada pela equação (5.21) é equivalente a eliminar o efeito da probabilidade *a priori* no classificador bayesiano. A probabilidade *a priori* influencia indiretamente o classificador ajustado pelo EMQ (cf. equação (5.16)), uma vez que a minimização do EMQ produz uma superfície de decisão que vai ser ajustada para incluir um número maior de registros da classe majoritária.

A Figura 5.12 mostra uma comparação dos resultados obtidos por um modelo quadrático ajustado pelo MMQ e pelo MMQP, com ponderações calculadas pela equação (5.21), nos dados do problema de Diabetes das índias Pima.¹¹⁸ Esse problema tem duas classes que representam o diagnóstico, positivo ou negativo, de diabetes em mulheres da etnia Pima a partir de atributos como número de filhos, pressão sanguínea etc. O

* A matriz de coeficientes $\hat{\mathbf{X}}^T \mathbf{W} \hat{\mathbf{X}} = (\mathbf{W} \hat{\mathbf{X}})^T \hat{\mathbf{X}}$. A matriz $\mathbf{W} \hat{\mathbf{X}}$ pode ser calculada diretamente a partir do vetor de pesos $\mathbf{w}^T$, multiplicando-se, elemento a elemento, as colunas da matriz de regressores $\hat{\mathbf{X}}$ pelo vetor de pesos $\mathbf{w}^T$, evitando-se, assim, o custo computacional e de alocação de memória do processamento dos elementos nulos da matriz diagonal $\mathbf{W}$.

conjunto de dados é desbalanceado e a classe negativa (C_1) tem 65% dos registros, enquanto a classe positiva (C_2) é atribuída a 35% dos registros. O conjunto de dados tem oito variáveis, mas a Figura 5.12 mostra os resultados calculados sobre as duas componentes principais para facilitar a visualização. Observe o aumento da região atribuída à classe positiva (em verde) no caso da solução calculada pelo MMQP.

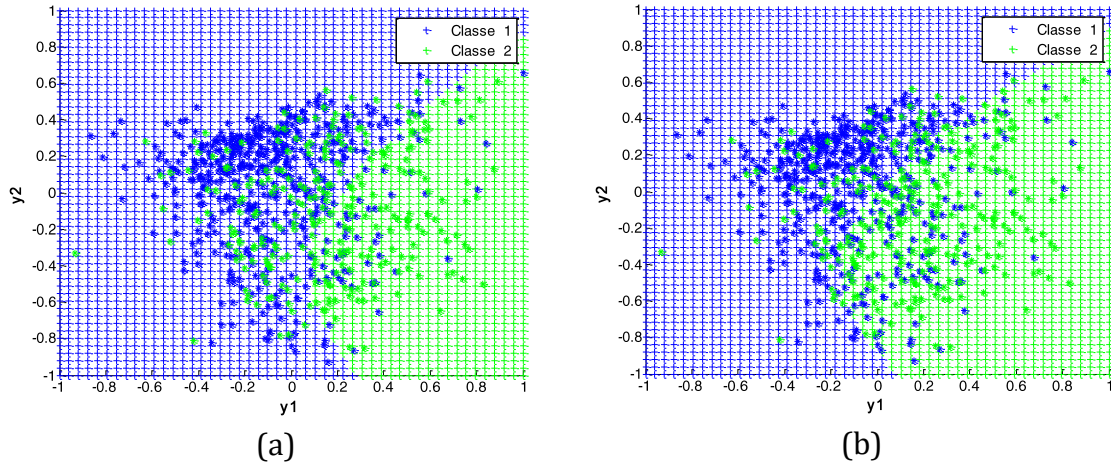


Figura 5.12: Superfícies de decisão problema de Diabetes das índias Pima 2D: (a) MMQ; (b) MMQP.

Nos modelos lineares ajustados pelo MMQ e MMQP, a variável de saída é uma variável indicadora $v_i(t) \in \{-1, 1\}$, enquanto a aproximação do modelo linear $\hat{v}_i(t) \in \mathcal{R}$, que não pode ser interpretada a probabilidade *a posteriori* da classe. No modelo de regressão logística, descrito a seguir, o modelo linear é transformado para que resultado possa ser interpretado como um valor de probabilidade.

5.2.3 Regressão logística

A regressão logística, também conhecida como **modelo logit**, é um modelo de classificação encontrado em diversos *softwares* livres e comerciais, e em bibliotecas de linguagens de programação como R¹¹⁹ e Python.¹²⁰

A formulação clássica da regressão logística é obtida para o problema de classificação binária em que a saída observada é representada pela variável indicadora $v(t) \in \{0, 1\}$:

$$v(t) = \begin{cases} 1, & \text{se } \mathbf{x}(t) \in C \\ 0, & \text{se } \mathbf{x}(t) \notin C \end{cases} \quad (5.22)$$

O modelo é uma aproximação da probabilidade *a posteriori*, isto é, $g(\mathbf{x}, \boldsymbol{\theta}) \approx P(C|\mathbf{x})$, e a função discriminante é calculada pela função sigmoide, também conhecida por função logística ou logit (de onde o método deriva seu nome), aplicada ao modelo linear:

$$g(\mathbf{x}(t), \boldsymbol{\theta}) = \frac{1}{1 + \exp(-\hat{\mathbf{x}}(t)\boldsymbol{\theta}^T)} \quad (5.23)$$

onde $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ é o vetor de regressores e $\boldsymbol{\theta}$ é o vetor de parâmetros.

A Figura 5.13 mostra o comportamento da função sigmoide $g(z)$, na qual podemos observar que $\lim_{z \rightarrow +\infty} g(z) = 1$ e $\lim_{z \rightarrow -\infty} g(z) = 0$.

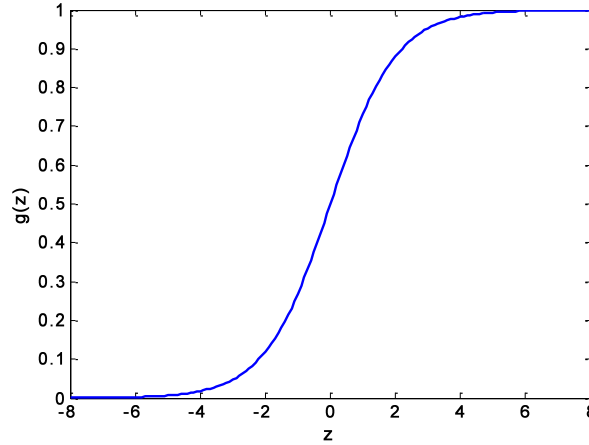


Figura 5.13: Função sigmoide ou logística.

A regressão logística é um modelo de classificação binária, cuja função discriminante é definida pela probabilidade *a posteriori* como:

$$\begin{aligned} P(v(t) = 1 | \mathbf{x}(t), \boldsymbol{\theta}) &= g(\mathbf{x}(t), \boldsymbol{\theta}) \\ P(v(t) = 0 | \mathbf{x}(t), \boldsymbol{\theta}) &= 1 - g(\mathbf{x}(t), \boldsymbol{\theta}) \end{aligned} \quad (5.24)$$

A equação acima pode ser escrita como uma distribuição de Bernoulli¹²¹:

$$P(v(t) | \mathbf{x}(t), \boldsymbol{\theta}) = \hat{v}(t)^{v(t)} (1 - \hat{v}(t))^{1-v(t)} \quad (5.25)$$

onde $\hat{v}(t) = g(\mathbf{x}(t), \boldsymbol{\theta})$ e $v(t) \in \{0, 1\}$.

O modelo de regressão logística é ajustado para maximizar a probabilidade (ou a função de verossimilhança) do modelo em representar os valores de observados:

$$L(\boldsymbol{\theta}) = \prod_{t=1}^N P(v(t) | \mathbf{x}(t), \boldsymbol{\theta}) \quad (5.26)$$

Aplicando a expressão (5.25) em (5.26), temos a expressão da função de verossimilhança da regressão logística:

$$L(\theta) = \prod_{t=1}^N \hat{v}(t)^{v(t)} (1 - \hat{v}(t))^{1-v(t)} \quad (5.27)$$

O produtório da equação (5.27) dificulta o ajuste de parâmetros e o resultado não se altera se o negativo do logaritmo da função de verossimilhança for utilizado para o ajuste de parâmetros como um problema de minimização:

$$l(\theta) = -\log(L(\theta)) = -\sum_{t=1}^N v(t) \log(\hat{v}(t)) + (1 - v(t)) \log(1 - \hat{v}(t)) \quad (5.28)$$

A Figura 5.14 mostra o resultado da função (5.28) calculada para um determinado registro. Quando $v(t) = 1$, mostrado na Figura 5.14a, $l(\theta) = -\log(\hat{v}(t))$, que aumenta exponencialmente à medida que $\hat{v}(t) \rightarrow 0$. No caso de $v(t) = 0$, mostrado na Figura 5.14b, $l(\theta) = -\log(1 - \hat{v}(t))$ que aumenta exponencialmente quando $\hat{v}(t) \rightarrow 1$.

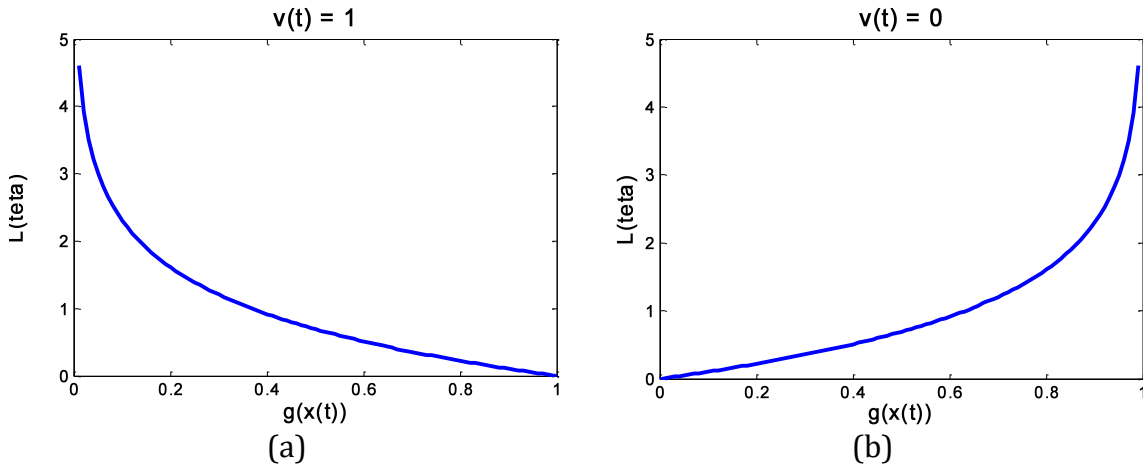


Figura 5.14: Função de custo (5.28): (a) $v(t) = 1$; (b) $v(t) = 0$.

O negativo do logaritmo da função de verossimilhança (cf. equação (5.28)) é mais adequada para o ajuste de modelos de classificação que o EMQ pela solução de um problema de minimização. Os resultados da regressão logística e do modelo linear ajustado pelo MMQ são equivalentes, mas a regressão logística tem uma interpretação probabilística, que é mais adequada para a tomada de decisão. Entretanto, embora os parâmetros do modelo ocorram não linearmente na expressão da função sigmoide (5.23). Hastie e colaboradores [238] apresentam a solução baseada no método de Newton-Raphson, que resulta no algoritmo conhecido como mínimos quadrados iterativo ponderado [406]. O modelo de regressão logística também pode ser ajustado pelo método do gradiente como será apresentado na seção 9.1.2.

O modelo de regressão logística pode ser estendido para problemas de classificação de múltiplas classes [238][521] e para funções discriminantes não lineares [578]. No caso de múltiplas classes, o modelo é conhecido como **regressão softmax** e será discutido no Capítulo 9.

A Figura 5.15 apresenta o resultado de dois modelos de classificação sobre o conjunto de dados *Wine* 2D, formado pelo conjunto *Wine* após a ADL, discutido na Figura 5.10 e na Figura 5.11. A Figura 5.15a apresenta o resultado do classificador bayesiano simples; a Figura 5.15b mostra o resultado da regressão logística. Pode-se observar que o resultado da regressão logística é idêntico ao resultado do modelo de classificação linear ajustado pelo MMQ mostrado na Figura 5.11a.

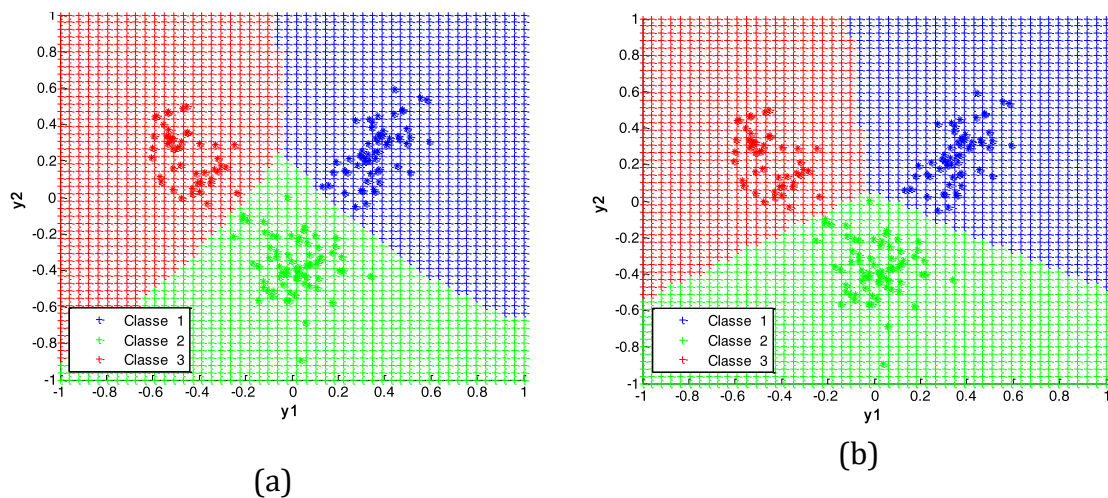


Figura 5.15: Dados *Wine* 2D: (a) classificador bayesiano simples; (b) regressão logística.

Assim como na regressão, a escolha do melhor modelo deve ser feita a partir de estatísticas de validação calculadas em validação cruzada, como apresentado a seguir.

5.3 Avaliação de Classificadores

O problema de classificação é preditivo e, como no caso da regressão linear, é necessário evitar o *overfitting*. O processo de desenvolvimento de um modelo de classificação segue os mesmos passos delineados na seção 3.2, em que o objetivo da etapa de validação é a seleção do tipo e da estrutura do modelo mais adequados ao problema. As estatísticas são calculadas em validação cruzada para gerar um modelo com boa capacidade de generalização.

O modelo linear ajustado pelo MMQ segue as mesmas hipóteses da regressão linear e o classificador poderia ser avaliado pelas mesmas estatísticas de validação apresentadas na seção 3.3.1. Entretanto, o problema de classificação tem saídas discretas e, geralmente, não ordenadas que requerem estatísticas específicas. Esta seção apresenta uma descrição das estatísticas mais utilizadas para avaliação de classificadores.

5.3.1 Estatísticas de validação

As estatísticas de avaliação de classificadores são calculadas a partir da matriz de confusão, citada na seção 5.1.1 e apresentada na Tabela 5.2 para duas classes. A matriz de confusão é utilizada em estatística para representar as possíveis soluções de um teste de hipóteses. Em problemas de classificação, as linhas representam as classes observadas e as colunas, as classes preditas pelo modelo.

As estatísticas de validação são definidas para problemas de duas classes em que uma classe é chamada arbitrariamente de classe positiva e a outra, de classe negativa. Na Tabela 5.2, a classe C_1 é a classe positiva e a classe C_2 , a negativa.

Tabela 5.2: Matriz de confusão

	$\hat{C}_1$ (Predita)	$\hat{C}_2$ (Predita)
C_1 (Real)	VP	FN
C_2 (Real)	FP	VN

A matriz de confusão é preenchida com o número de registros correspondentes, obtidos, em geral, em validação cruzada, seguindo a seguinte terminologia:

- **Verdadeiros Positivos (VP):** é o número de registros da classe C_1 corretamente classificados pelo modelo como classe $\hat{C}_1$.
- **Falsos Negativos (FN):** é o número de registros da classe C_1 classificados pelo modelo como classe $\hat{C}_2$.
- **Falsos Positivos (FP):** é o número de registros da classe C_2 classificados pelo modelo como classe $\hat{C}_1$.
- **Verdadeiros Negativos (VN):** é o número de registros da classe C_2 corretamente classificados pelo modelo como classe $\hat{C}_2$.

A avaliação global do classificador é feita pela **acurácia** do modelo, que é sua taxa de classificação correta, calculada como:

$$ACC = \frac{VP + VN}{VP + FP + FN + VN} \quad (5.29)$$

O **erro global** de classificação é o complemento da acurácia:

$$ERR = 1 - ACC = \frac{FP + FN}{VP + FP + FN + VN} \quad (5.30)$$

O erro global destaca a diferença de desempenho de dois classificadores. Por exemplo, um modelo com acurácia de 96% tem o dobro do erro de outro modelo com 98% de acurácia.

As estatísticas globais de acurácia e erro não informam sobre o erro que o modelo está realizando para cada classe. Duas medidas frequentemente utilizadas em recuperação da informação podem ser utilizadas para avaliar o desempenho de classificadores [364]: **precisão** (PRE) e **recuperação** (REC), também chamada de sensibilidade ou *recall*. Estas medidas são definidas para cada classe como:

$$PRE(C_1) = \frac{VP}{VP + FP} \quad PRE(C_2) = \frac{VN}{VN + FN} \quad (5.31)$$

$$REC(C_1) = \frac{VP}{VP + FN} \quad REC(C_2) = \frac{VN}{VN + FP} \quad (5.32)$$

A precisão mede a capacidade de predição do modelo e a recuperação é sua capacidade de reconhecer os registros da classe correspondente. Em problemas de classificação com múltiplas classes, as medidas de precisão e *recall* são calculadas para cada classe considerando as demais como a classe negativa. As medidas de precisão e recuperação podem ser combinadas na chamada **medida F**, definida para a classe C_i como:

$$F(C_i) = \frac{2 PRE(C_i). REC(C_i)}{PRE(C_i) + REC(C_i)} \quad (5.33)$$

A medida F representa a média harmônica entre precisão e *recall*, e indica o desempenho em termos das duas estatísticas simultaneamente, podendo ser generalizada para variar entre a precisão e o *recall* através de um parâmetro [516].

Em problemas desbalanceados, as estatísticas de cada classe devem ser consideradas em conjunto para não levar a conclusões equivocadas. Por exemplo, considere um problema de duas classes, em que a probabilidade *a priori* $P(C_1) = 90\%$. Um classificador

que classifique todos os registros na classe majoritária (C_1) vai obter $PRE(C_1) = 0.9$ e $REC(C_1) = 1.0$, mas $PRE(C_2) = REC(C_2) = 0.0$ para a classe minoritária (C_2).

A estatística **kappa** é uma medida global que relaciona a probabilidade de concordância observada com a probabilidade de concordância esperada $P(E)$ [110]. Em modelos de classificação, a concordância observada é medida pela acurácia do classificador (5.29) e a concordância esperada é o resultado que seria obtido por um modelo aleatório em uma amostra de tamanho infinito. O coeficiente kappa (κ) é calculado como:

$$\kappa = \frac{ACC - P(E)}{1 - P(E)} \quad (5.34)$$

O valor $\kappa \leq 1$, sendo que $\kappa = 1$, indica a concordância completa; $\kappa = 0$ indica que $ACC = P(E)$, isto é, a acurácia é igual à concordância esperada. Um valor $\kappa < 0$ indica que o classificador tem um desempenho pior que um resultado puramente aleatório. Landis e Koch [327] sugerem que um valor de $\kappa \geq 0,8$ representa boa concordância e um resultado $0,6 \leq \kappa < 0,8$ representa uma concordância razoável. A probabilidade de concordância esperada $P(E)$ é calculada a partir da matriz de confusão, como:

$$P(E) = P(C_1)P(\hat{C}_1) + P(C_2)P(\hat{C}_2) \quad (5.35)$$

onde $P(C_1)$ é a probabilidade *a priori* da classe C_1 , $P(C_2)$ é a probabilidade *a priori* da classe C_2 , $P(\hat{C}_1)$ é o percentual de registros preditos como sendo da classe $\hat{C}_1$ pelo modelo e $P(\hat{C}_2)$, o percentual de registros preditos como classe $\hat{C}_2$.

Os termos da equação (5.35) são calculados como:

$$P(C_1) = \frac{VP + FN}{VP + FP + FN + VN} = \frac{N_1}{N} \quad (5.36)$$

$$P(C_2) = \frac{FP + VN}{VP + FP + FN + VN} = \frac{N_2}{N} \quad (5.37)$$

$$P(\hat{C}_1) = \frac{VP + FP}{VP + FP + FN + VN} = \frac{\hat{N}_1}{N} \quad (5.38)$$

$$P(\hat{C}_2) = \frac{FN + VN}{VP + FP + FN + VN} = \frac{\hat{N}_2}{N} \quad (5.39)$$

Os valores totais observados e preditos de cada classe são mostrados na Tabela 5.3.

Tabela 5.3: Totais observados e preditos de cada classe

	$\hat{C}_1$	$\hat{C}_2$	Total
C_1	VP	FN	N_1
C_2	FP	VN	N_2
Total	$\hat{N}_1$	$\hat{N}_2$	N

A avaliação de classificadores também é feita graficamente utilizando as taxas de verdadeiros positivos e de falsos positivos no chamado de espaço ROC, descrito a seguir.

5.3.2 Espaço ROC e AUC

Na seção 5.1.1, foi visto que, em caso de problemas desbalanceados, a decisão pelo mínimo erro de classificação favorece a classe majoritária. Em muitos problemas, a classe minoritária é a mais importante e, nesse caso, o erro global não pode ser utilizado para a escolha do classificador. A escolha do classificador em problemas desbalanceados deve favorecer a classe mais importante, aumentando a recuperação de registros de maior custo.

O efeito do desbalanceamento do problema sobre o resultado do classificador ajustado pelo mínimo erro de classificação é mostrado na Figura 5.16 que mostra o valor de $p(x|C_i)P(C_i)$ de cada para um problema de uma variável e duas classes. A situação em que as probabilidades *a priori* são iguais é mostrada na Figura 5.16a. Nesse caso, as taxas de FP e FN são definidas pela forma e posição das distribuições condicionais. No caso de problemas desbalanceados (Figura 5.16b), o ajuste do modelo pelo mínimo erro de classificação favorece a classe mais frequente (C_1), resultando num aumento da taxa de VP e consequente diminuição da taxa de VN. Quando a classe minoritária é a mais importante, o classificador deve favorecer a taxa de VN. Entretanto, o aumento da taxa de VN só pode ser obtido à custa de um aumento da taxa de FN, o que aumenta também o erro global, como foi discutido na seção 5.2.2.

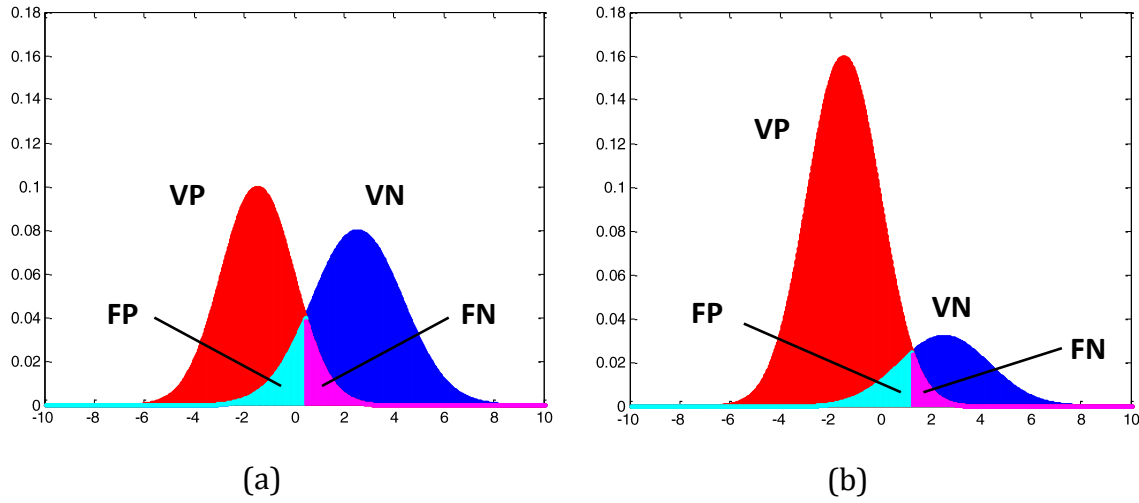


Figura 5.16: Problemas desbalanceados: (a) $P(C_1) = P(C_2)$; (b) $P(C_1) > P(C_2)$.

O espaço ROC (do inglês *receiver operating characteristic*) foi desenvolvido por operadores de radar para caracterização do *trade-off* entre a taxa de detecção e a de falsos alarmes em ambientes com ruído [516][561]. Para avaliação de classificadores, as taxas de FP e VP obtidas por um classificador são utilizadas como coordenadas do espaço ROC, mostrado na Figura 5.17.

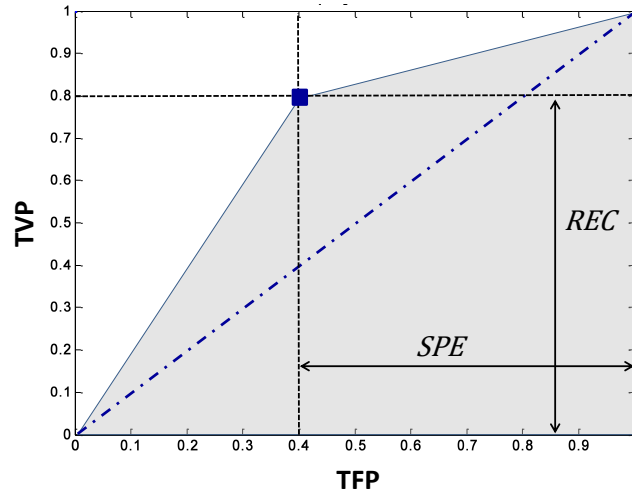


Figura 5.17: Espaço ROC.

A taxa de VP (TVP) é o *recall* da classe positiva $REC(C_1)$ em (5.32) e a taxa de FP (TFP) é o complemento da taxa de VN ($REC(C_2)$) chamada de **especificidade** (SPE). O valor de TFP é calculado como:

$$TFP = 1 - SPE = 1 - \frac{VN}{FP + VN} = \frac{FP}{FP + VN} \quad (5.40)$$

O espaço ROC mostra o desempenho do classificador relacionando o quanto ele acerta da classe positiva e o quanto deixa de errar da classe negativa. O modelo ideal é o ponto (0,1) do gráfico, com $TFP = 0$, ou seja, $SPE = REC(C_2) = 1$ e $REC(C_1) = 1$. Nesse caso o modelo tem 100% de aproveitamento, pois recupera todos os exemplos de ambas as classes. Classificadores na diagonal do espaço ROC têm $REC(C_1) = 1 - REC(C_2)$, são modelos com $\kappa = 0$, ou seja, têm desempenho equivalente a um modelo aleatório. Os classificadores abaixo da diagonal representam modelos com $\kappa < 0$.

A área sombreada na Figura 5.17 é uma medida de avaliação de classificadores chamada de área sob a curva ROC (AUC, na sigla em inglês). A medida AUC é calculada pela área do quadrilátero obtido ao se ligar as coordenadas do classificador aos pontos (0,0) e (1,1) (cf. Figura 5.17). Após algumas simplificações, o valor de AUC é calculado como a média entre a taxa de VP e VN:

$$AUC = \frac{REC(C_1) + REC(C_2)}{2} \quad (5.41)$$

O valor de AUC é calculado pela equação (5.41) para problemas de duas classes. Em problemas de múltiplas classes, um valor de AUC é calculado para cada classe, considerada a classe positiva, e o valor de AUC total do classificador é calculado pela média do AUC de cada classe ponderada pela estimativa de probabilidade *a priori* da classe correspondente [176]:

$$AUC = \sum_{i=1}^m AUC(C_i)P(C_i) \quad (5.42)$$

onde $AUC(C_i)$ é o valor calculado pela equação (5.41) considerando a classe C_i como positiva e $P(C_i)$ é a frequência relativa da classe C_i . Outras abordagens para o cálculo do valor total de AUC foram avaliadas por Hand e Till [235].

Um método gráfico de avaliação de modelos no espaço ROC é chamado de curva ROC, que apresenta uma curva contínua no espaço ROC para cada classe. O valor AUC nesse caso é calculado integrando-se a curva ROC pelo método dos trapézios. Entretanto, a curva ROC é independente da frequência das classes [176]. A medida de AUC obtida pela equação (5.41), além de mais eficiente, é sensível às frequências das classes sendo portanto mais adequada em problemas desbalanceados.

O resultado do valor AUC em problemas desbalanceados pode ser observado no conjunto de dados Diabetes das índias Pima, discutido na seção 5.2.2. Nesse problema, a classe positiva é a minoritária, representada por C_2 , com 35% dos registros. A Figura 5.18 mostra os resultados calculados em validação cruzada para os modelos quadráticos ajustados pelo MMQ e pelo MMQP utilizando todas as variáveis.

Na coluna da esquerda, a Figura 5.18a apresenta os resultados de precisão e *recall* de cada classe do classificador ajustado com o MMQ e a Figura 5.18c mostra o espaço ROC e o valor de AUC calculado pela equação (5.41) do mesmo modelo. Na coluna da direita, os mesmos resultados são apresentados para o modelo ajustado com o MMQP.

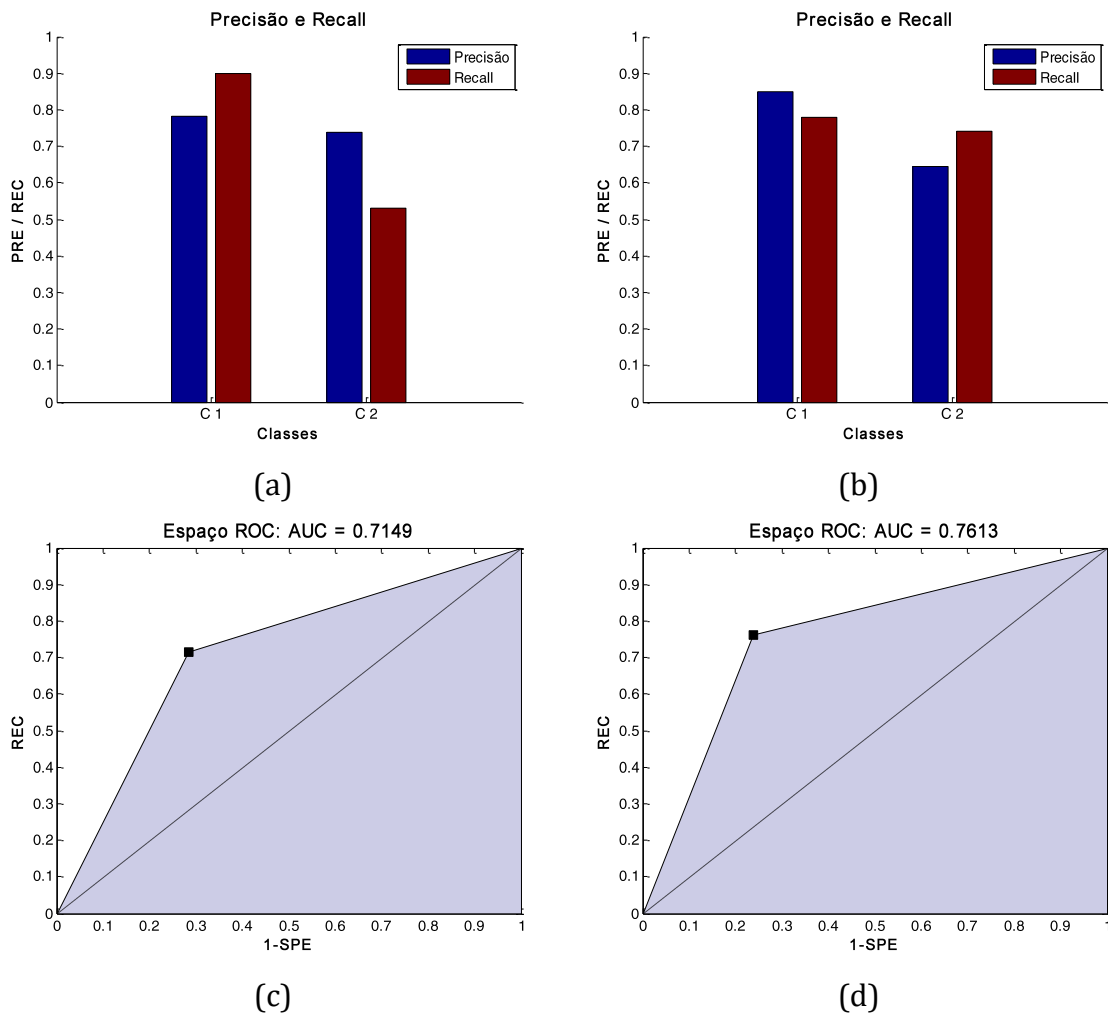


Figura 5.18: Resultados do problema Diabetes das índias Pima: (a) e (c) Modelo ajustado com MMQ; (b) e (d) modelo ajustado com MMQP.

No modelo ajustado pelo MMQP, a recuperação da classe positiva (C_2) é maior, resultando num valor maior de AUC. O modelo ajustado pelo MMQ obtém menor erro global, influenciado pelo desbalanceamento entre as classes, resultando numa taxa de recuperação maior da classe negativa (C_1).

5.4 Aplicações

Os modelos de classificação apresentados neste capítulo serão discutidos nesta seção em dois contextos diferentes. Na seção 5.4.1, os algoritmos são comparados numa série de conjuntos de dados *benchmark* para uma avaliação global do desempenho do algoritmo. Esse mesmo tipo de teste será repetido nos Capítulos 7 e 8 com os modelos de classificação de inteligência computacional. Na seção 5.4.2, apresentamos um estudo de caso sobre a análise de cancelamentos em um operador de telefonia.

5.4.1 Testes *benchmark*

O repositório da Universidade da Califórnia em Irvine (UCI)¹²² é uma referência na área de ciência de dados e contém diversos conjuntos de dados para avaliação de algoritmos em problemas reais.

Um conjunto de 10 *benchmarks* foi selecionado para a avaliação dos modelos apresentados neste capítulo:

- Bayes N: Classificador bayesiano simples (*naif*)
- Bayes Q: Classificador bayesiano multivariado (quadrático)
- Logit: Regressão logística
- M. Lin. 1: Modelo linear de primeira ordem
- M. Lin. 2: Modelo linear de segunda ordem

Os conjuntos de dados foram selecionados de forma a cobrir uma grande variedade de tipos de problemas de dimensões diferentes, tanto em termos de número de registros quanto de número de variáveis. A Tabela 5.4 apresenta o número de variáveis (p), o número de classes (m) e o número de registros (N) de cada problema.

A maioria dos *benchmarks* são problemas conhecidos na literatura. Os *benchmarks* *Wine* e *Diabetes das índias Pima* já foram utilizados neste capítulo como exemplos de aplicação dos modelos. Nesta avaliação, todos os *benchmarks* foram utilizados como dados brutos, ou seja, não foi feito nenhum pré-processamento que pudesse interferir

no desempenho dos modelos. Os algoritmos de ajuste de parâmetros também foram executados em sua forma padrão, ou seja, sem utilizar nenhuma ponderação para compensar o desbalanceamento das classes. Todos os testes foram realizados em validação cruzada de 10 ciclos, conforme o algoritmo descrito na seção 3.3.2.

Tabela 5.4: Conjuntos de dados *benchmark*

<i>Benchmark</i>	<i>p</i>	<i>m</i>	<i>N</i>
Íris ¹²³	4	3	150
Balance ¹²⁴	4	3	625
Diabetes ¹²⁵	8	2	768
Cancer ¹²⁶	9	2	286
Statlog Heart ¹²⁷	13	2	270
Wine ¹²⁸	13	3	178
Parkinson ¹²⁹	17	2	186
Ionosphere ¹³⁰	32	2	351
Spambase ¹³¹	57	2	4601
Sonar ¹³²	60	2	208

O desempenho dos modelos pode ser avaliado quantitativamente através das métricas de acurácia (ACC), definida na equação (5.29), e da área sobre a curva ROC (AUC), definida na equação (5.41). Os resultados são apresentados na Tabela 5.5, na qual os valores em negrito correspondem aos maiores valores de cada métrica para o respectivo *benchmark*.

Os resultados apresentados na Tabela 5.5 mostram que nenhum modelo é melhor em todos os testes, mas indicam que o modelo linear de segunda ordem (M. Lin. 2) obtém os melhores resultados na maioria das vezes. Além disso, os modelos quadráticos (Bayes Q e M. Lin. 2) obtiveram resultados melhores que seus correspondentes mais simples (Bayes N e M. Lin. 1) na maioria dos casos. O modelo de regressão logística (Logit) apresentou resultados próximos aos do modelo linear simples (M. Lin. 1).

É importante verificar se o melhor desempenho representa realmente um comportamento consistente do modelo ou é um resultado obtido em função de fatores aleatórios. Essa avaliação é feita por testes estatísticos não paramétricos [125][195] disponíveis em *softwares* estatísticos e no *software* livre KEEL,¹³³ utilizado nesta seção.

Tabela 5.5: Resultados dos testes *benchmark*

<i>Benchmark</i>	Bayes N		Bayes Q		Logit		M. Lin. 1		M. Lin. 2	
	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>
Íris	94.67	0.9600	98.00	0.9850	82.67	0.8697	83.33	0.8747	95.33	0.9650
Balance	90.40	0.9079	91.68	0.9469	87.04	0.8755	87.04	0.8755	91.36	0.9171
Diabetes	71.88	0.7260	74.61	0.7054	76.69	0.7145	76.30	0.7089	77.21	0.7151
Cancer	94.58	0.9574	95.02	0.9559	96.05	0.9512	96.05	0.9512	97.22	0.9709
Statlog Heart	83.70	0.8392	82.59	0.8225	83.33	0.8275	84.07	0.8358	84.44	0.8375
Wine	97.19	0.9766	99.44	0.9958	98.88	0.9919	98.31	0.9881	99.44	0.9958
Parkinson	79.49	0.7026	80.51	0.8427	86.15	0.7889	86.15	0.7889	86.15	0.8099
Ionosphere	60.68	0.6933	86.89	0.8803	85.75	0.8138	86.04	0.8178	89.46	0.8602
Spambase	90.18	0.8911	82.53	0.8474	88.63	0.8686	88.63	0.8684	91.37	0.9006
Sonar	72.60	0.7159	71.15	0.6992	76.92	0.7656	75.96	0.7579	80.29	0.8010

O teste de hipóteses é uma técnica clássica de estatística que permite avaliar se um conjunto de evidências observadas confirma ou refuta uma hipótese. Nos testes de hipótese paramétricos, há um modelo da distribuição do processo e o teste é realizado sobre os parâmetros da distribuição. Nos testes chamados não paramétricos, não há um modelo da distribuição dos dados [501]. Na avaliação de classificadores, as hipóteses são formuladas da seguinte forma:

- Hipótese nula $\mathcal{H}_0$: os modelos apresentam resultados equivalentes;
- Hipótese alternativa $\mathcal{H}_1$: os modelos não apresentam resultados equivalentes.

A aceitação da hipótese nula significa que as diferenças de desempenho são devidas às particularidades de cada *benchmark* [195]. O resultado do teste de hipóteses (paramétrico ou não paramétrico) é o chamado **valor- p** , que mede a significância estatística das diferenças observadas na avaliação. O valor- p é interpretado como a probabilidade de observar o resultado obtido considerando que a hipótese nula é verdadeira. Em outras palavras, o valor- p representa a probabilidade do chamado erro Tipo I (ou falso negativo), de rejeitar a hipótese nula quando, de fato, ela é verdadeira. Quanto menor o valor- p , menor é a probabilidade de que a hipótese nula seja verdadeira. Tipicamente, a

hipótese nula é rejeitada se o valor- $p < 0.05$, que representa confiança estatística de 95% na hipótese alternativa.

Garcia e colaboradores [195] sugerem que os testes não paramétricos são mais indicados para avaliação de classificadores. O teste de Friedman, particularmente o chamado *Friedman aligned rank*, é recomendado quando um número relativamente pequeno de modelos (menor que cinco) são comparados. O teste ordena os resultados obtidos por cada modelo e utiliza as diferenças de ordem para o mesmo *benchmark* e as diferenças de ordem para o mesmo modelo em *benchmarks* diferentes [195].

Nesta seção, discutimos apenas os resultados do teste *Friedman aligned rank*. Uma discussão detalhada deste e outros métodos é feita por Garcia e colaboradores [195]. Os autores são do grupo de pesquisa responsável pelo desenvolvimento do *software* livre KEEL [9][10], utilizado para a realização dos testes. O *software* tem um módulo de testes não paramétricos, com várias opções de testes.

A Tabela 5.6 mostra os *rankings* de cada modelo, calculados pelo teste de Friedman a partir dos resultados da Tabela 5.5, e o valor- p correspondente a cada métrica de avaliação. O classificador calculado pelo discriminante linear de segunda ordem (M. Lin. 2) obteve o melhor *ranking* pelo teste de Friedman pelas duas métricas. O valor- p indica que as probabilidades de erro em considerar as diferenças significativas são, respectivamente, de 8,26% e 7,95% para as métricas de ACC e AUC.

Tabela 5.6: Resultados de teste *Friedman aligned rank*

	ACC	AUC
Bayes N	33.30	31.95
Bayes Q	28.55	23.80
Logit	26.45	31.25
M. Lin. 1	26.75	27.35
M. Lin. 2	12.45	13.15
Valor- p	0.0826	0.0795

O grande problema dos testes estatísticos é que, na grande maioria dos casos, não se pode ter 100% de certeza dos resultados. O que o teste estatístico mostra, com uma confiança estatística em torno de 92%, é que o modelo discriminante linear de segunda ordem apresenta um desempenho melhor que os demais. Esse resultado é coerente com o argumento teórico de que o MMQ é mais robusto que a estimativa de parâmetros

do classificador bayesiano. O teste de hipóteses quantifica esse resultado, que não fica evidente observando-se apenas o melhor resultado de cada *benchmark* na Tabela 5.5.

5.4.2 Análise de cancelamento em telefonia

Empresas que operam em ambiente competitivo estão sempre preocupadas em promover a fidelização dos clientes de maior valor em sua base, uma vez que a manutenção de um cliente tem menor custo que a captação de novos clientes. Em empresas que oferecem serviços de assinatura, como telefonia e TV paga, um termo muito difundido é a taxa de *churn*,¹³⁴ que se refere ao percentual de cancelamentos de assinaturas num determinado período. O desenvolvimento de modelos preditivos de cancelamentos é uma das áreas de aplicação mais ativas de ciência de dados [422][530].

Um exemplo interessante de análise de cancelamentos num operador de telefonia foi apresentado por Cister [107]. A base de dados é uma amostra de clientes considerados de alto valor: empresas com média de quatro linhas móveis e alto consumo. A base de dados original é um conjunto de dados em três dimensões: 1.171 clientes, 10 variáveis de consumo e 12 meses. A amostra de dados foi realizada entre agosto de 2004 e julho de 2005 [107].

A análise apresentada nesta seção foi realizada sobre a média das variáveis de consumo por cliente, uma vez que as variáveis de consumo representavam o faturamento de cada tipo de ligação realizado pelo cliente durante o mês. Dessa forma, foi possível eliminar a dimensão de consumo e trabalhar com uma tabela de clientes por mês, que, após a retirada de 2,5% de *outliers*, ficou com 1.141 registros. Uma análise preliminar do gráfico de linhas aplicado às médias e desvios padrão de cada variável (Figura 5.19) revelou que o comportamento nos seis primeiros meses da amostra foi praticamente constante, não havendo interesse para o modelo de predição. Com isso, foram selecionadas as variáveis correspondentes aos seis últimos meses, assinaladas na Figura 5.19, em que a variável x_1 representa o consumo médio seis meses antes do final da amostra (fevereiro de 2005) e a variável x_6 representa o consumo médio no último mês do estudo (julho de 2005).

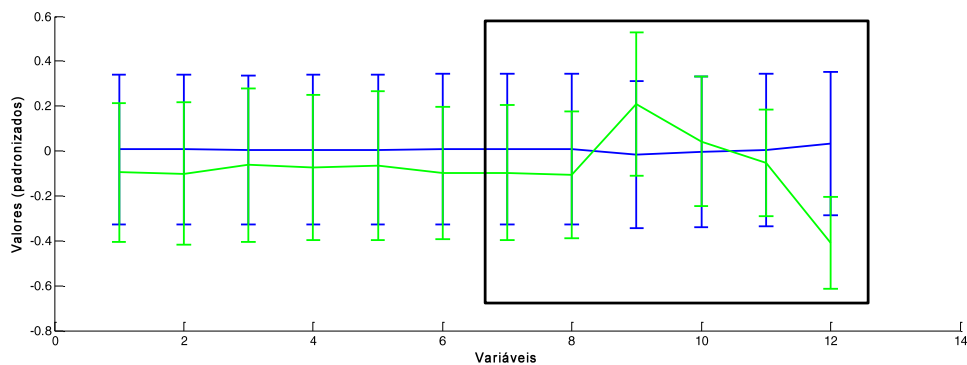


Figura 5.19: Gráfico de linhas: médias e desvios padrão de cada variável por classe.

O objetivo do modelo é a predição dos clientes que abandonaram a companhia após o período da amostra. A classe C_1 representa os clientes que continuaram a assinatura e C_2 os clientes que cancelaram a assinatura que, nesse caso, é a classe de maior relevância. A Figura 5.19 mostra que o comportamento dos clientes que vieram a cancelar mudou consideravelmente a partir do nono mês da amostra, aumentando bruscamente e depois diminuindo até valores inferiores aos registrados até o mês oito. As causas dessa mudança de comportamento não foram divulgadas [107].

A Figura 5.20 mostra as principais características da base de dados. As distribuições das variáveis de consumo médio são razoavelmente assimétricas e as variáveis são bem correlacionadas entre si, à exceção da variável x_6 , que representa o consumo no último mês da amostra e que tem um comportamento bem diferente das demais variáveis.

Uma das principais características desse problema é o alto grau de desbalanceamento da base, uma vez que a classe dos clientes que cancelaram a assinatura (C_2) tem apenas 7,17% dos registros.

O objetivo nesta seção é avaliar os efeitos da ponderação para compensar o desbalanceamento das classes nos resultados dos modelos. Os modelos bayesianos e lineares discutidos na seção 5.4.1 foram utilizados na avaliação. Os modelos foram executados de duas formas: sem ponderação para compensar o efeito da probabilidade *a priori* e com ponderação. No caso dos modelos bayesianos, a ponderação corresponde a retirar o valor de probabilidade *a priori* do cálculo do valor de probabilidade *a posteriori*.

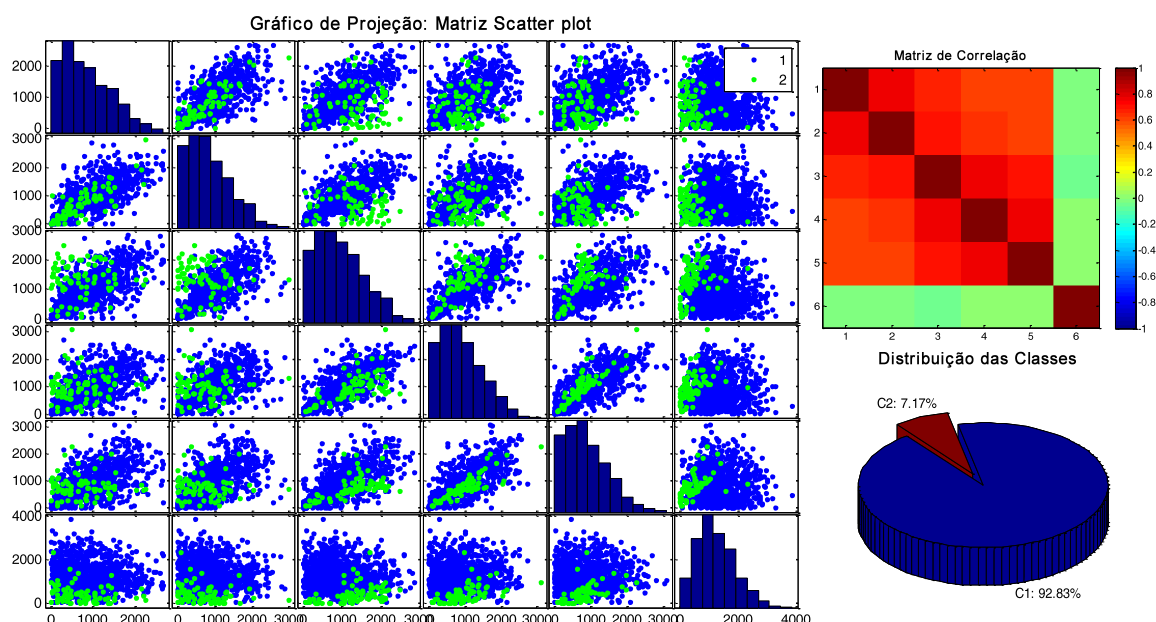


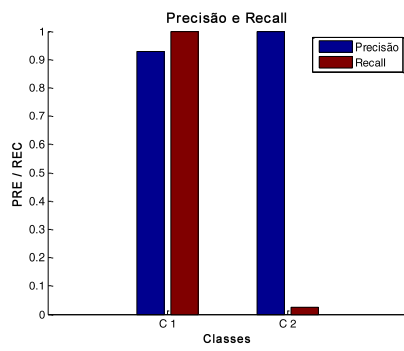
Figura 5.20: Características da base de cancelamentos.

Os resultados de acurácia (ACC) e AUC são apresentados na Tabela 5.7, na qual os valores máximos de cada métrica em cada tipo de teste estão em negrito. Podemos observar que, em todos os modelos, a ponderação resultou em diminuição de ACC e aumento de AUC.

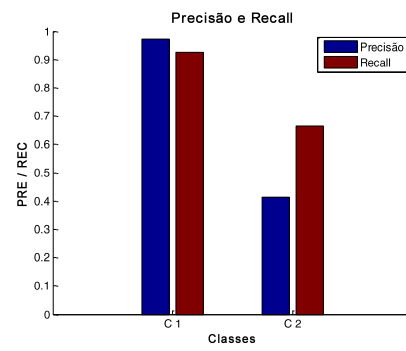
Tabela 5.7: Resultados dos modelos

Modelo	Bayes N		Bayes Q		M. Lin. 1		M. Lin. 2	
	ACC	AUC	ACC	AUC	ACC	AUC	ACC	AUC
S. Pond.	92.82	0.5119	96.41	0.8272	92.91	0.5179	94.40	0.6300
C. Pond.	90.63	0.7960	88.18	0.8814	85.29	0.8658	90.19	0.8868

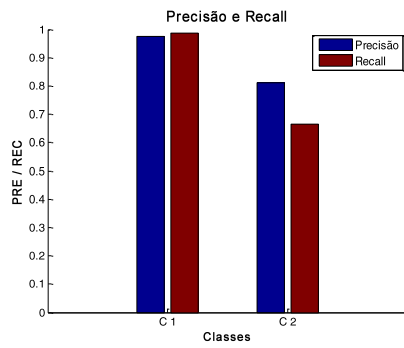
Os gráficos de precisão e *recall* para cada modelo com e sem ponderação são mostrados na Figura 5.21, na qual a coluna da esquerda representa os resultados sem ponderação e a coluna da direita, os resultados com ponderação. Podemos observar que a ponderação aumenta a taxa de recuperação (*recall*) da classe minoritária C_2 em todos os modelos, que provoca o aumento de AUC.



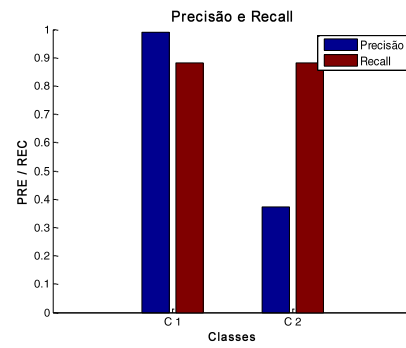
(a)



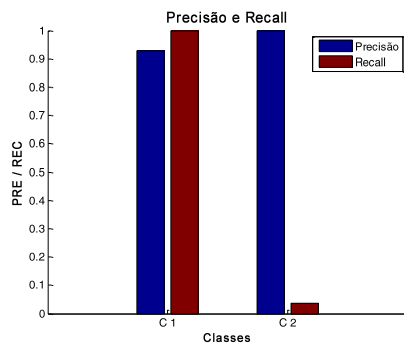
(b)



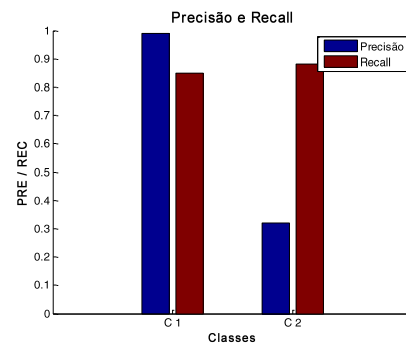
(c)



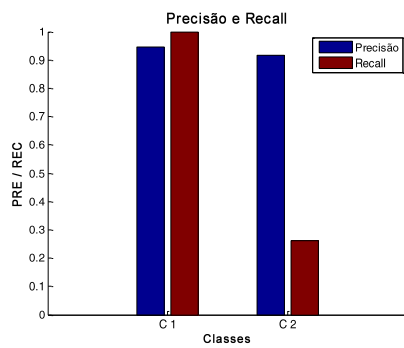
(d)



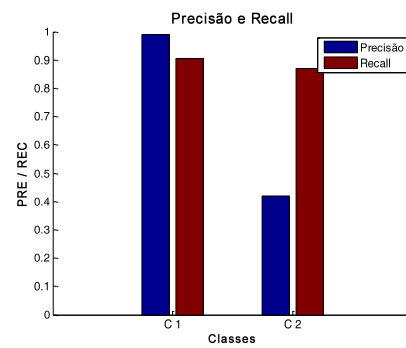
(e)



(f)



(g)



(h)

Figura 5.21: Precisão e *recall*, coluna da esquerda sem ponderação e coluna da direita com ponderação: (a) e (b) Bayes N; (c) e (d) Bayes Q; (e) e (f) M. Lin. 1; (g) e (h) M. Lin. 2.

Em problemas altamente desbalanceados como este, é importante utilizar as métricas adequadas para avaliação dos algoritmos. O classificador bayesiano simples e o modelo linear de primeira ordem, sem ponderação, obtiveram resultados de acurácia próximos à probabilidade *a priori* da classe majoritária C_1 (92,83%). Consequentemente, esses modelos obtiveram taxas de recuperação muito pequenas da classe minoritária C_2 que, neste exemplo, é a de maior relevância. Esse resultado foi identificado pelo valor de AUC próximo a 0.5 nos dois modelos.

5.5 Comentários Finais

O problema de reconhecimento de padrões é um problema clássico e há vários livros dedicados ao assunto [43][146][189][521][553]. É muito difícil condensar os tópicos mais importantes num único capítulo. O critério utilizado para a seleção dos assuntos apresentados foi a importância para a formulação do problema de classificação e a relevância para a fundamentação das técnicas de inteligência computacional que serão discutidas nos Capítulos 7, 8 e 9.

Alguns assuntos importantes não foram incluídos. Os modelos de árvore de decisão, por exemplo, que estão presentes em praticamente todos os livros de ciência de dados [232][516][561], não foram incluídos. Optou-se por uma fundamentação do problema de classificação baseada no teorema de Bayes. O leitor vai encontrar uma apresentação teórica completa dos modelos de classificação sob a ótica de aprendizado de máquina no excelente livro de Mitchel [392]. Uma discussão mais detalhada dos modelos de árvore de decisão é apresentada por Quinlan [446].

A formulação da regra de Bayes é bastante adequada para o entendimento do problema de classificação. Mesmo que o classificador não utilize diretamente o teorema de Bayes na sua formulação, os efeitos da probabilidade condicional e, principalmente, da probabilidade *a priori* estarão presentes em todo classificador que seja ajustado pela minimização do erro de classificação.

Nas redes bayesianas [121][125][247][263], a distribuição de probabilidades é definida para pares de variáveis selecionadas e o modelo é representado através de um grafo. O modelo de redes bayesianas busca um compromisso entre as propriedades teóricas do teorema de Bayes e a dificuldade de estimação de parâmetros em proble-

mas de alta dimensionalidade. Uma apresentação bastante completa do tema é encontrada no livro de Koller e Friedman [316] e no curso do Coursera.¹³⁵ A obtenção da estrutura do grafo não é tarefa trivial, embora existam algoritmos eficientes [125].

Em geral, para uma amostra de tamanho finito, o problema de identificar a distribuição de probabilidades do processo que gerou a amostra é mais complexo que encontrar uma função discriminante [98]. Dessa forma, os modelos de classificação mais modernos buscam diretamente encontrar a função discriminante com base nas observações disponíveis. A minimização do erro de classificação no conjunto de treinamento para um modelo de estrutura fixa é a abordagem mais comum. Nos modelos de máquina de vetor de suporte (SVM, na sigla em inglês) [478][532], o erro de classificação é limitado e o ajuste de parâmetros busca um modelo maximizando a margem de separação entre as classes, o que corresponde à minimização da estrutura do modelo. Esse tipo de modelo, que começou a se difundir na área de ciência de dados a partir de meados da década de 1990, será apresentado no Capítulo 7. Recentemente, redes neurais têm se destacado em problemas de classificação, principalmente em aplicações complexas como visão computacional e processamento de linguagem natural. As redes neurais serão apresentadas no Capítulo 9.

Há diversas abordagens para o tratamento de problemas desbalanceados na literatura [90][273]. A abordagem mais comum é a reamostragem de dados para compensar o desbalanceamento. O algoritmo SMOTE [89] realiza essa abordagem de forma eficiente e os resultados são bastante satisfatórios, principalmente quando utilizado com um modelo de árvore de decisão.

A avaliação de algoritmos por testes estatísticos não paramétricos tem se difundido nas áreas de inteligência computacional e ciência de dados [195]. Entretanto, é importante observar que o desempenho de um algoritmo sobre um conjunto de testes *benchmark* vale apenas para aquele conjunto e não significa que o algoritmo terá um desempenho melhor em todos os problemas. Na prática, a melhor abordagem é sempre a busca do melhor desempenho para o problema em questão, evitando alguma preferência *ad hoc* sobre um tipo particular de modelo.

6 ANÁLISE DE AGRUPAMENTOS

O objetivo da análise de agrupamentos é a divisão do conjunto de dados em subconjuntos chamados de grupos. A análise de agrupamentos é um problema de aprendizado não supervisionado e frequentemente é chamada de classificação não supervisionada, uma vez que não existe uma variável de saída (ou variável alvo) a ser predita como na regressão ou na classificação, que são problemas de aprendizado supervisionado. A falta de supervisão dificulta a definição de uma função de avaliação da qualidade do resultado. Dessa forma, a análise de agrupamentos é essencialmente uma atividade descritiva, em que o papel do analista é preponderante para a qualidade do resultado.

As aplicações da análise de agrupamentos são bastante variadas. Na área de negócios, a segmentação da base de clientes é muito comum em grandes corporações [530]. Há também aplicações em praticamente todas as áreas com uso intensivo de dados, como, por exemplo, bioinformática [290][357], processamento de imagens [528], recuperação da informação [319], entre outras [284].

A análise de agrupamentos tem sido estudada em diferentes comunidades científicas e a literatura sobre algoritmos de agrupamento é muito diversificada. Há obras clássicas na estatística [146][194][238][521], em inteligência computacional [572][573] e ciência de dados [232][516]. Neste capítulo, apresentamos os algoritmos clássicos, que podem ser encontrados na maioria dos *softwares* e abordagens mais recentes baseadas na teoria de grafos.

6.1 O Problema de Análise de Agrupamentos

Um grupo é um conjunto de objetos com alguma característica comum que os identifica como elementos do grupo. O objetivo da análise de agrupamentos é identificar padrões que permitam separar um conjunto de dados em grupos. Os algoritmos de agrupamento encontram grupos ou subconjuntos de dados da amostra com características similares. Frequentemente, o problema envolve a análise e interpretação dos grupos encontrados.

Matematicamente, um grupo é um conjunto definido por uma propriedade característica compartilhada pelos seus elementos. Assim, dado um universo $\mathcal{X}$, os elementos do conjunto $\mathcal{A} \subseteq \mathcal{X}$ são definidos pela propriedade $\chi_{\mathcal{A}}(x)$, que, para todo elemento do universo $x \in \mathcal{X}$:

$$\chi_{\mathcal{A}}(x) = \begin{cases} 1, & \text{se } x \in \mathcal{A} \\ 0, & \text{se } x \notin \mathcal{A} \end{cases} \quad (6.1)$$

A equação (6.1) é a base do chamado axioma da separação [77], que estabelece as duas leis (ou axiomas) fundamentais da teoria dos conjuntos:

- a) a lei do **terceiro excluído**: $\mathcal{A} \cup \bar{\mathcal{A}} = \mathcal{X}$;
- b) a lei da **não contradição**: $\mathcal{A} \cap \bar{\mathcal{A}} = \emptyset$.

A noção de conjunto é estendida ao conceito de **partição**, particularmente útil em ciência de dados, em que o universo é subdividido em K subconjuntos disjuntos.

Na análise de agrupamentos, a partição do conjunto de dados (de treinamento) $\mathcal{T}$ em K subconjuntos é definida como $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, de tal forma que as seguintes condições sejam satisfeitas:

- i. A união de todos os subconjuntos é o universo: $\bigcup_{i=1}^K \mathcal{C}_i = \mathcal{T}$;
- ii. Os subconjuntos não têm elementos em comum: $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$, $i \neq j$.

A descoberta de uma partição num conjunto de dados é computacionalmente complexa; a partição de um conjunto de dados com N registros em apenas dois subconjuntos, com N_1 e N_2 registros respectivamente, implica a avaliação de $N!/(N_1! N_2!)$ partições distintas. Por exemplo, um conjunto de dados de $N = 100$ registros pode ser dividido em dois subconjuntos de $N_1 = N_2 = 50$ registros de 10^{29} formas diferentes! Além da complexidade inerente ao problema, a da qualidade do agrupamento não é bem definida e, em geral, utiliza-se índices de validação (cf. seção 6.5).

Na análise de agrupamentos, um grupo é definido como um conjunto de registros similares entre si e dissimilares dos registros alocados em outros grupos. A escolha do melhor agrupamento num conjunto de dados é uma tarefa analítica que depende muito da interpretação dos grupos no contexto da aplicação. O conjunto de dados da Figura 6.1a pode ser dividido em dois grupos, como mostrado na Figura 6.1b, mas também em quatro ou seis grupos, como nas Figura 6.1c e d [516]. A decisão sobre a melhor solução vai depender muito da aplicação.

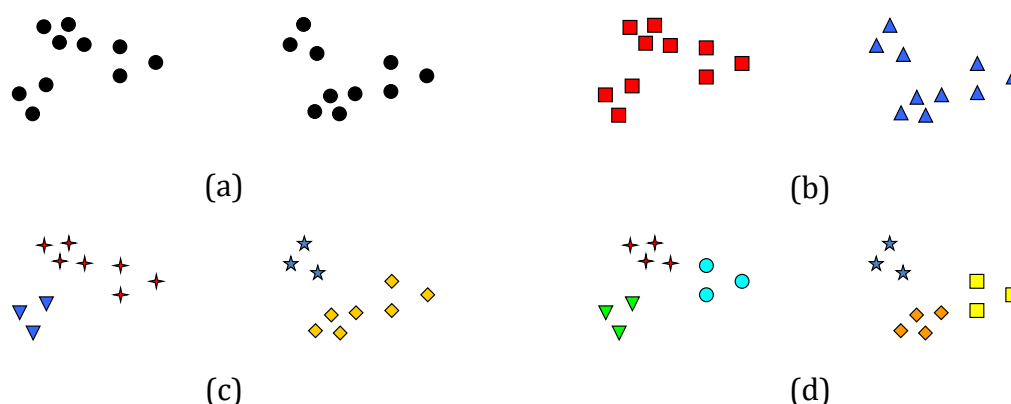


Figura 6.1: Diferentes agrupamentos sobre o mesmo conjunto de dados: (a) conjunto de dados; (b) dois grupos; (c) quatro grupos; (d) seis grupos. Fonte: [516].

Observando a Figura 6.1, percebemos que os grupos da Figura 6.1d estão contidos na Figura 6.1c, que, por sua vez, estão contidos na partição da Figura 6.1b. Dessa forma, uma solução para a dificuldade de avaliação da qualidade do agrupamento é realizá-lo de forma hierárquica, como nos algoritmos apresentados na seção 6.3.

Outra característica importante da análise de agrupamentos é que os grupos encontrados podem não corresponder às classes de um problema supervisionado. Isso não significa que o algoritmo de agrupamento tenha falhado, apenas que os padrões encontrados não correspondem às classes. Isso acontece porque os algoritmos de agrupamento utilizam informações dos registros e a classificação é usualmente uma informação externa aos registros. Dessa forma, o entendimento da análise de agrupamentos como “classificação não supervisionada” deve ser utilizado com cautela.

A Figura 6.2a mostra um conjunto de dados contendo claramente quatro grupos, embora esse mesmo conjunto tenha sido apresentado como um problema de duas classes no exemplo da Figura 5.6.

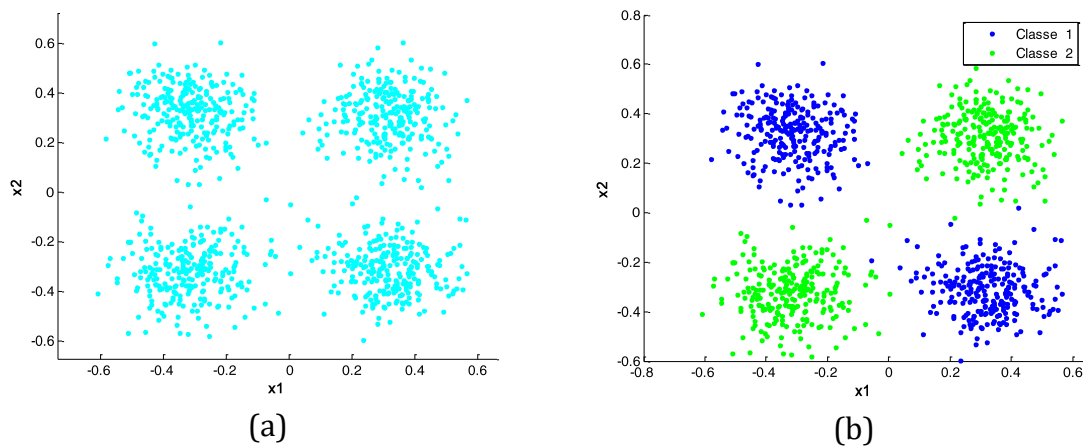


Figura 6.2: Exemplo de agrupamento: (a) conjunto de dados; (b) dados classificados.

Os algoritmos apresentados neste capítulo utilizam métricas de distância, como as definidas na seção 2.2.3, para variáveis numéricas. Há diversas outras, além de métricas para variáveis binárias e categóricas [194]. A escolha das métricas de distância ou similaridade pode tornar o problema ainda mais complexo, pois não há um critério universal para guiar tal escolha [232]. A abordagem mais simples para o tratamento de variáveis nominais é a sua transformação em variáveis indicadoras (cf. equação (5.14)) e a utilização de métricas definidas para as variáveis numéricas.

O algoritmo clássico em análise de agrupamentos é o algoritmo k-médias, em que os grupos são definidos a partir da distância de cada elemento ao centro do grupo. O k-médias é um algoritmo dos métodos de partição. Neste tipo de método, a métrica de distância determina o formato do grupo. A Figura 6.3 mostra um conjunto de pontos e regiões métricas de distância em relação ao centro, onde as regiões escuras representam valores menores. A distância euclidiana (norma L^2), mostrada na Figura 6.3b, gera grupos de forma esférica; a distância de Mahalanobis induz grupos elipsoides (Figura 6.3c); as distâncias de Manhattan (norma L^1) e de norma infinita (L^∞) geram hipercubos (Figura 6.3a e d). Dessa forma, a escolha da função de distância ou similaridade influencia o resultado do algoritmo de agrupamento.

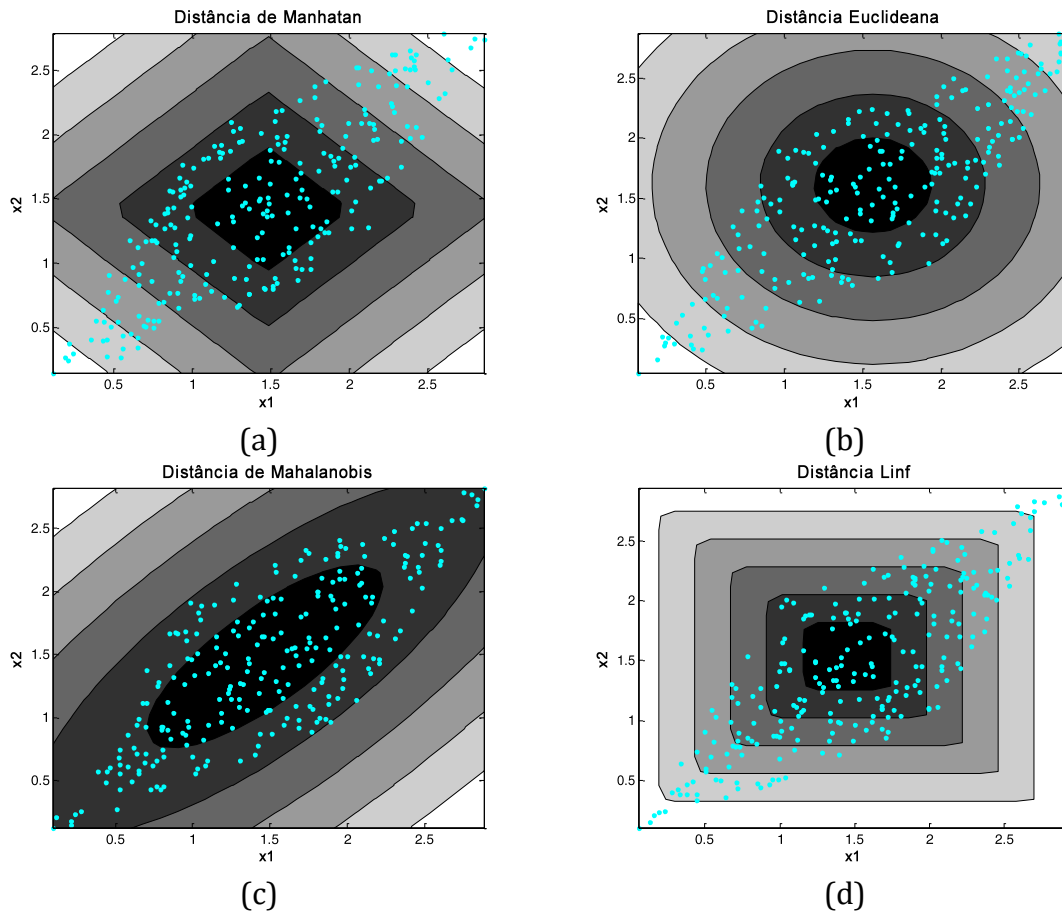


Figura 6.3: Métricas de distância: (a) distância de Manhattan (L^1); (b) distância euclidiana (L^2); (c) distância de Mahalanobis; (d) norma infinita (L^∞).

A interseção entre dois grupos calculados na análise de agrupamentos também forma uma superfície de decisão, como no problema de classificação. Um conjunto de dados pode conter grupos que formam superfícies de decisão convexas ou não convexas, como ilustrado na Figura 6.4. Em grupos convexas, como o da Figura 6.4a, as superfícies de decisão podem ser aproximadas por funções lineares. Em agrupamentos não convexas, como na Figura 6.4b, a superfície de decisão é uma função não linear.

Algoritmos mais simples, como os métodos de partição apresentados na seção 6.2, geram agrupamentos convexas. Algoritmos capazes de gerar agrupamentos não convexas podem representar melhor as características do conjunto de dados. Os métodos hierárquicos, apresentados na seção 6.3, e os métodos baseados em grafos, descritos na seção 6.4, encontram grupos não convexas.

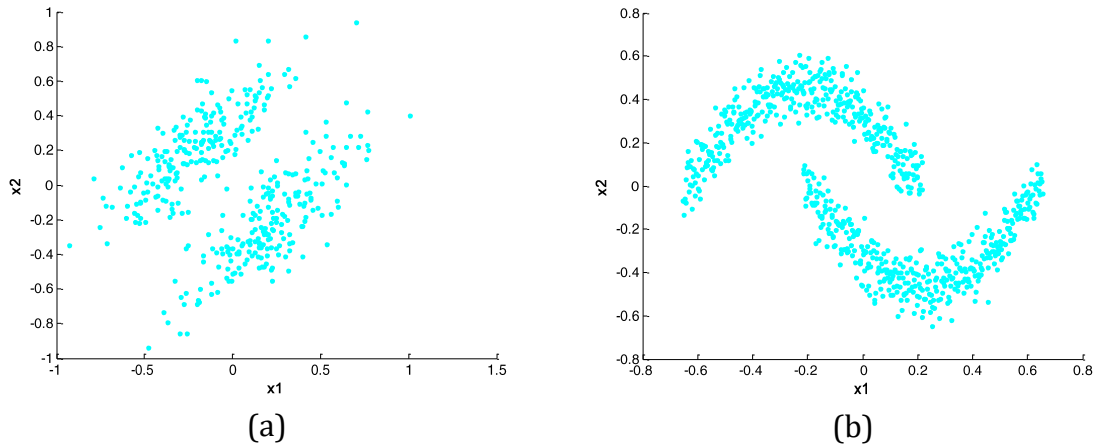


Figura 6.4: Forma de grupos: (a) convexa; (b) não convexa.

6.2 Métodos de Partição

Os métodos de partição buscam diretamente uma partição do conjunto de dados através da minimização de um critério de avaliação. O método de partição (e de agrupamento) mais amplamente utilizado é o algoritmo k-médias, em que o critério de custo é calculado em função do quadrado da distância entre os registros e as coordenadas dos centros, conforme descrito a seguir.

6.2.1 Algoritmo k-médias

O algoritmo k-médias é um algoritmo simples e eficiente que geralmente converge num número reduzido de iterações na maioria dos problemas. O algoritmo original foi proposto há mais de 50 anos* [285], mas continua sendo utilizado em diversas áreas e pode ser encontrado nos principais *softwares* e bibliotecas.

O objetivo do algoritmo k-médias é encontrar uma partição no conjunto de dados em K grupos $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ através da minimização da seguinte função de avaliação:

$$J(\mathbf{\Omega}) = \sum_{t=1}^N \sum_{i=1}^K u_i(\mathbf{x}(t)) \|\mathbf{x}(t) - \mathbf{\omega}_i\|^2 \quad (6.2)$$

* O algoritmo é atribuído ao trabalho proposto por Lloyd em 1957 para representação digital de sinais analógicos, que só foi publicado em 1982 [353]. Entretanto, é muito difícil reconhecê-lo nesse artigo. O algoritmo na sua forma atual foi proposto por MacQueen em 1967 [359].

onde $\boldsymbol{\omega}_i = \frac{1}{\hat{N}_i} \sum_{\mathbf{x}(t) \in \mathcal{C}_i} \mathbf{x}(t)$ é o vetor com as coordenadas do centro do grupo $\mathcal{C}_i$ e a matriz $\boldsymbol{\Omega} = [\boldsymbol{\omega}_1^T, \dots, \boldsymbol{\omega}_K^T]$ representa os centros de todos os grupos.

A função $u_i(\mathbf{x}(t))$ na equação (6.2) define a pertinência de um registro $\mathbf{x}(t)$ ao grupo $\mathcal{C}_i$ (cf. equação (6.1)) como:

$$u_i(\mathbf{x}(t)) = \begin{cases} 1, & \text{se } \mathbf{x}(t) \in \mathcal{C}_i \\ 0, & \text{se } \mathbf{x}(t) \notin \mathcal{C}_i \end{cases} \quad (6.3)$$

A cardinalidade do grupo $\mathcal{C}_i$ é o número de registros do grupo, isto é, $|\mathcal{C}_i| = \hat{N}_i$, pode ser calculada pela soma dos valores de pertinência ao grupo como $\hat{N}_i = \sum_{t=1}^N u_i(\mathbf{x}(t))$.

A função de avaliação (6.2) pode ser interpretada pela decomposição da variância total dos dados como $\boldsymbol{\Sigma}_T = \boldsymbol{\Sigma}_W + \boldsymbol{\Sigma}_B$ [146][573], apresentada na seção 2.4.3.

A matriz de variância total $\boldsymbol{\Sigma}_T = (N - 1)\hat{\boldsymbol{\Sigma}}$, onde $\hat{\boldsymbol{\Sigma}}$ é a matriz de covariâncias estimada (cf. equação (2.5)), é calculada como:

$$\boldsymbol{\Sigma}_T = \sum_{t=1}^N (\mathbf{x}(t) - \bar{\mathbf{x}})^T (\mathbf{x}(t) - \bar{\mathbf{x}}) \quad (6.4)$$

onde $\bar{\mathbf{x}} = \frac{1}{N} \sum_{t=1}^N \mathbf{x}(t)$ é a média dos registros do conjunto de treinamento, que também pode ser calculada pela média ponderada dos centros de grupos como $\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^K \hat{N}_i \boldsymbol{\omega}_i$.

A matriz de variâncias intragrupos $\boldsymbol{\Sigma}_W$ e a matriz de variância entre grupos $\boldsymbol{\Sigma}_B$ são calculadas como (cf. seção 2.4.3):

$$\boldsymbol{\Sigma}_B = \sum_{i=1}^K N_i (\boldsymbol{\omega}_i - \bar{\mathbf{x}})^T (\boldsymbol{\omega}_i - \bar{\mathbf{x}}) \quad (6.5)$$

$$\boldsymbol{\Sigma}_W = \sum_{t=1}^N \sum_{i=1}^K u_i(\mathbf{x}(t)) (\mathbf{x}(t) - \boldsymbol{\omega}_i)^T (\mathbf{x}(t) - \boldsymbol{\omega}_i) \quad (6.6)$$

A função de avaliação (6.2) é soma dos elementos da diagonal principal (traço) da matriz de variância intragrupos $\boldsymbol{\Sigma}_W$, cuja minimização resulta na minimização da variância intragrupos $\boldsymbol{\Sigma}_W$ e, portanto, em grupos mais compactos. A variância total $\boldsymbol{\Sigma}_T$ não é função do agrupamento e se mantém constante, de forma que a minimização da variância intragrupos implica a maximização da variância entre grupos $\boldsymbol{\Sigma}_B$, ou seja, em grupos mais separados.

A minimização da função (6.2) é um problema NP-difícil [12][360]. O algoritmo k-médias aplica uma heurística gulosa para a solução eficiente do problema. Entretanto, o resultado depende muito da estrutura dos dados e uma solução de ótimo local pode ser encontrada, o que torna o resultado dependente da inicialização.

O algoritmo k-médias é descrito no Algoritmo 6.1. Sua execução é feita a partir de uma matriz de centros fornecida na inicialização que, em geral, é aleatória. A cada iteração, cada registro do conjunto de dados é alocado ao grupo mais próximo. Embora outras métricas de distância possam ser utilizadas dependendo da implementação, o algoritmo k-médias clássico utiliza a distância euclidiana. Após uma passagem por todo o conjunto de dados, o centro de cada grupo é atualizado como a média das coordenadas dos registros alocados ao grupo.

Algoritmo 6.1: Algoritmo k-médias

Entrada:	Conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, estimativa inicial de centros $\mathbf{\Omega} = [\omega_1^T, \dots, \omega_K^T]$
Saída:	Matriz de centros $\mathbf{\Omega} = [\omega_1^T, \dots, \omega_K^T]$
01	Início
02	$\varepsilon \leftarrow \infty$
03	Enquanto $\varepsilon > tol$
04	Para $t = 1, \dots, N$
05	$\mathbf{x}(t) \in \mathcal{C}_i \Leftrightarrow \omega_i = \underset{j}{\operatorname{argmin}} \left(\ \mathbf{x}(t) - \omega_j\ ^2 \right)$
06	Fim Para
07	$\mathbf{\Omega}_0 \leftarrow \mathbf{\Omega}$
08	Atualizar $\mathbf{\Omega}$ com $\omega_i = \frac{\sum_{t=1}^N u_i(\mathbf{x}(t))\mathbf{x}(t)}{\sum_{t=1}^N u_i(\mathbf{x}(t))}, i = 1, \dots, K$
09	Calcular critério de parada $\varepsilon = \ \mathbf{\Omega}_0 - \mathbf{\Omega}\ $
10	Fim Enquanto
11	Retorna $\mathbf{\Omega} = [\omega_1^T, \dots, \omega_K^T]$
12	Fim

O critério de parada do algoritmo, calculado no passo 09, é a norma L^2 da diferença entre as matrizes dos centros calculados em duas iterações sucessivas. O algoritmo é

interrompido se não há mudanças significativas dos centros. O número máximo de iterações também pode ser utilizado como critério de parada.

A cada iteração, uma partição do conjunto de dados deve ser encontrada, de forma que, se algum grupo não tiver nenhum registro alocado no passo 05, uma ação precisa ser realizada para evitar erro de divisão por zero no passo 08. Dependendo do *software*, diferentes ações podem ser realizadas. No Matlab,¹³⁶ por exemplo, o usuário pode escolher entre uma mensagem de erro (*default*), a eliminação do grupo ou a geração de novas coordenadas aleatoriamente. A possibilidade de redefinição de centros aleatoriamente causa instabilidades ao algoritmo.

Em geral, o algoritmo k-médias converge rapidamente e a complexidade de cada iteração do algoritmo é estimada em $O(NKp)$, onde p é o número de variáveis, K o número de grupos e N o número de registros. Em problemas em que $Kp \ll N$, a complexidade do algoritmo é dominada por $O(N)$. Em problemas de grandes dimensões, a busca por um agrupamento com muitos grupos resulta em $Kp \approx N$ e a complexidade do algoritmo é $O(N^2)$. Em alocação de memória, a complexidade do algoritmo é definida pelo armazenamento do conjunto de dados como $O(Np) \approx O(N)$.

Há diversas variantes do algoritmo k-médias [194][229][285][573]. O algoritmo ISODATA,* proposto em 1967 [27], é uma variante do k-médias que, muitas vezes, é confundido com o próprio. O algoritmo ISODATA ajusta automaticamente o número de grupos, agrupando-os e subdividindo-os de acordo com limites especificados, e elimina aqueles com poucos registros. No algoritmo ISODATA, a escolha do número de grupos é substituída pela escolha dos parâmetros do algoritmo.

O resultado do algoritmo k-médias é muito sensível à presença de *outliers* e ruído no conjunto de dados. Um *outlier* será associado a algum grupo e contabilizado no cálculo da média, o que pode afastar os centros calculados dos centros do conjunto mais denso de registros. Algumas variantes do k-médias visam maior robustez a *outliers*, como os algoritmos k-medoids [300], PAM† e CLARA.‡ Nesse tipo de algoritmo, o centro do grupo é um registro do conjunto de treinamento, chamado *medoid*, que tem a menor distância média para todos os registros do grupo. Os algoritmos PAM e CLARA aplicam a mesma ideia de k-medoids, mas o segundo utiliza apenas uma amostra do conjunto de dados

* *Iterative Self-Organizing Data Analysis Technique*

† *Partition Around Medoids*

‡ *Clustering LARge Applications*

para reduzir a complexidade computacional [521]. O algoritmo CLARANS* [416] é uma variante dos *k-medoids* mais complexa que o CLARA que obtém melhores resultados [521]. Outra abordagem para a solução do problema de *outliers* é obtida pelos algoritmos chamados *k-medianas* [83][378], cujos centros de grupos também são registros do conjunto de treinamento.

Há trabalhos mais recentes com outras variantes do *k-médias*. No algoritmo *x-médias* [434], o número de grupos é otimizado a partir dos critérios AIC e BIC, utilizados para seleção de estrutura de modelos de regressão [238]. O algoritmo *k-médias esférico*¹³⁷ [126][602] utiliza a similaridade do cosseno como métrica, sendo indicado para problemas de alta dimensionalidade. Zhao e Karypis [600] discutem algumas alternativas à função de avaliação quadrática para o agrupamento de documentos. Algumas abordagens têm sido propostas para a inicialização do *k-médias*, como PCA-Part [508], CCIA [302] e o algoritmo *k-médias++* [21][421].

O algoritmo *fuzzy c-médias* é uma variante importante do *k-médias*, em que o conceito pertinência ao grupo é estendido por funções de pertinência *fuzzy*. O algoritmo *fuzzy c-médias* será apresentado no Capítulo 8.

O *k-médias* é um algoritmo simples e eficiente, mas apresenta algumas características e limitações, discutidas nos exemplos a seguir.

6.2.2 Exemplos

A forma dos grupos encontrados pelo algoritmo *k-médias* depende da métrica de distância utilizada (cf. Figura 6.3). Com a distância euclidiana, os grupos encontrados pelo *k-médias* têm formato hiper esférico, de modo que a superfície de decisão entre grupos é aproximada por uma função linear. Dessa forma, as regiões associadas aos grupos formam um diagrama de Voronoi¹³⁸ no espaço de variáveis de entrada. A Figura 6.5 apresenta os resultados do algoritmo *k-médias* sobre os dados de Diabetes das índias Pima, projetado no plano definido pela ACP. Podemos observar claramente as superfícies de decisão lineares entre os grupos.

* *Clustering LARge Applications based on RANdomized Search*

O algoritmo k-médias tem algumas limitações, tais como encontrar grupos de tamanhos muito diferentes [516] e grupos de forma não convexa. Entretanto, é possível utilizar o algoritmo para encontrar um número maior de grupos e, posteriormente, agregar grupos para obter formas não convexas.

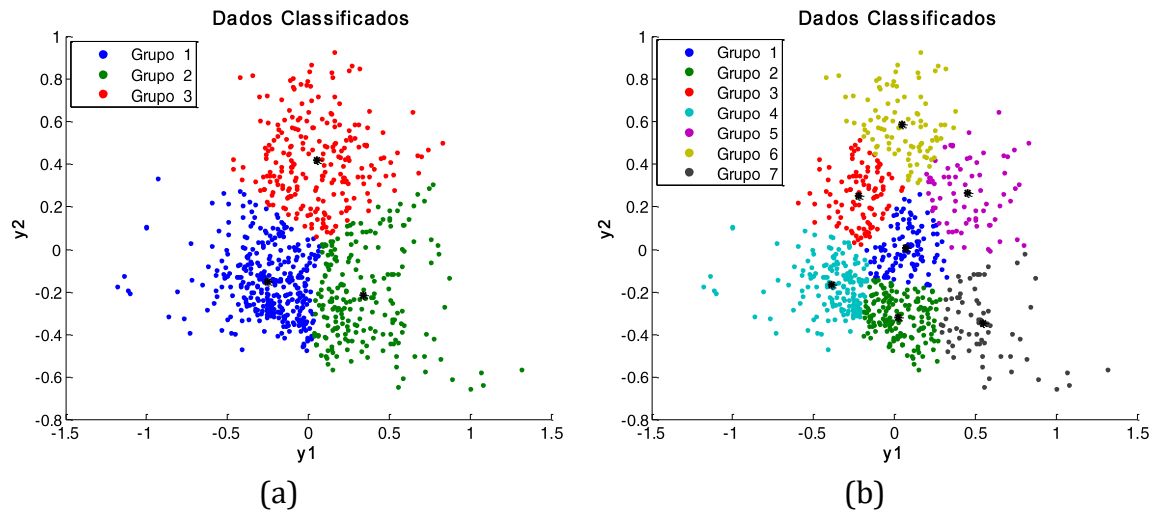


Figura 6.5: Resultados no conjunto de dados Diabetes projetados em 2D pela ACP:

(a) $K = 3$ grupos; (b) $K = 7$ grupos.

A Figura 6.6 mostra os resultados do k-médias no conjunto de dados das duas luas. Na Figura 6.6a, o resultado do algoritmo com $K = 2$ mostra que não foi possível identificar os dois grupos. O resultado com $K = 6$ grupos é mostrado na Figura 6.6b permite verificar que é possível encontrar os grupos pela agregação de subgrupos. Essa solução é aplicada no desenvolvimento de modelos de base radial, apresentados no Capítulo 7.

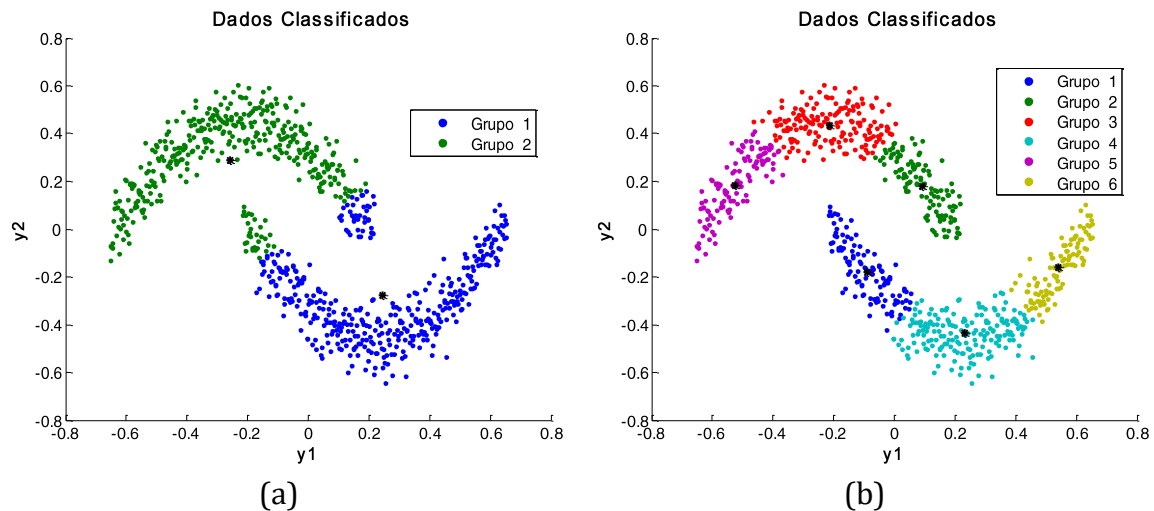


Figura 6.6: Resultados no conjunto de dados duas luas: (a) $K = 2$ grupos; (b) $K = 6$ grupos.

A principal limitação do algoritmo k-médias é a dependência da estimativa inicial dos centros de grupos, que ocorre principalmente em problemas em que o número de grupos não corresponde à estrutura dos dados. A Figura 6.7 mostra os resultados sobrepostos dos centros de grupos calculados em 100 execuções do algoritmo k-médias sobre os dados do problema de quatro grupos. Com $K = 2$ grupos, a Figura 6.7a mostra que o algoritmo particiona os dados horizontalmente ou verticalmente. Com $K = 3$ ou $K = 5$ (Figuras 6.7b e d), soluções diferentes são encontradas em cada execução. No caso de $K = 4$ grupos (Figura 6.7c), todas as execuções convergem para a mesma solução, que corresponde aos centros dos grupos.

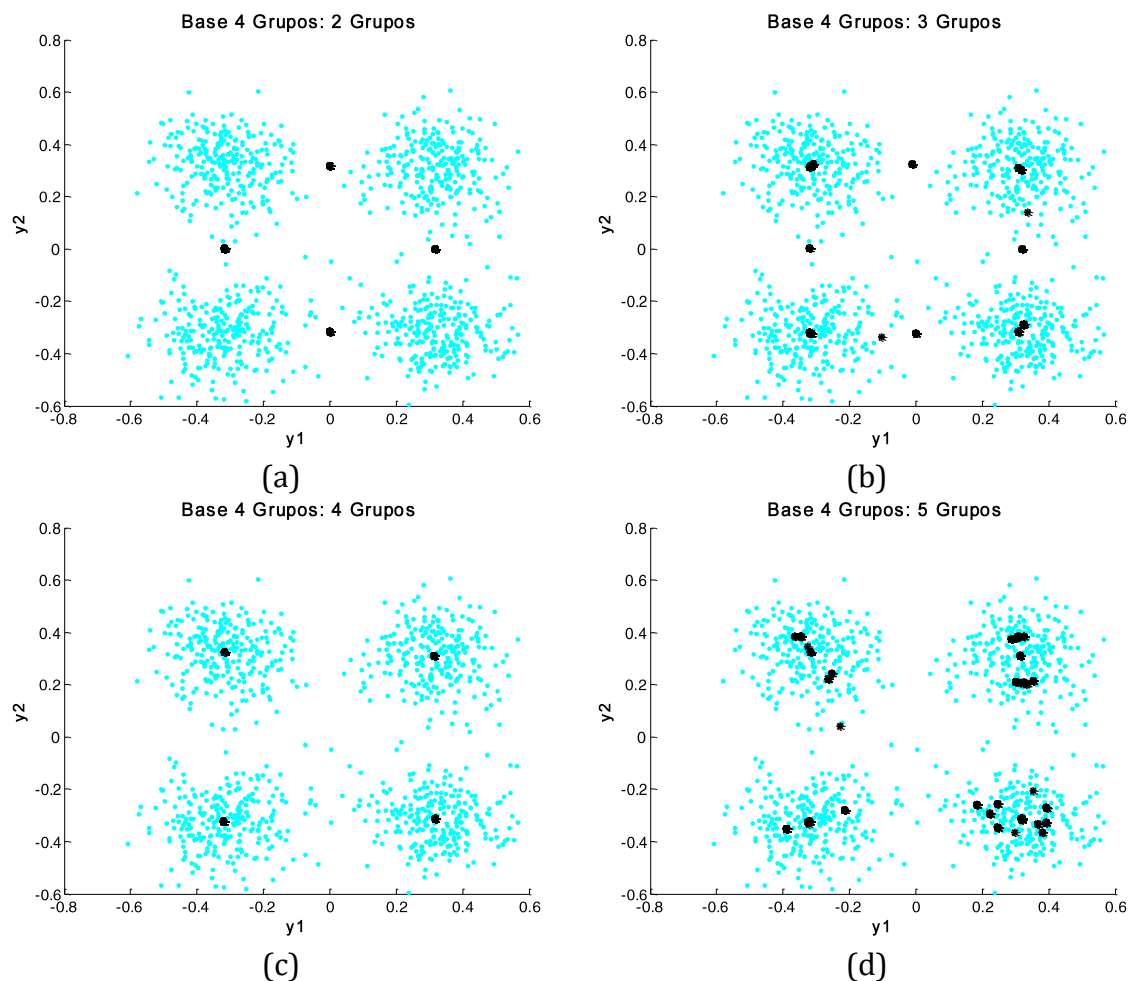


Figura 6.7: Resultados de centros encontrados pelo k-médias no conjunto de dados de quatro grupos: (a) $K = 2$ grupos; (b) $K = 3$ grupos; (c) $K = 4$ grupos; (d) $K = 5$ grupos.

A variação dos resultados do k-médias em execuções diferentes do algoritmo pode ser utilizada como um indicativo da qualidade do agrupamento. No exemplo da Figura 6.7, os dados têm quatro grupos muito bem definidos, de forma que o algoritmo k-médias

encontra sempre o mínimo global, independentemente da inicialização. Quando o número de grupos não corresponde às características do conjunto de dados, o algoritmo encontra um mínimo local e o resultado depende da inicialização. Celebi e colaboradores [80] apresentam um estudo sobre os principais métodos de inicialização do k-médias, mas não discutem a influência da escolha do número de grupos. Algumas medidas de validação para escolha do melhor agrupamento são apresentadas na seção 6.5.

6.3 Métodos Hierárquicos

Os algoritmos de agrupamento hierárquicos geram uma estrutura hierárquica de agrupamentos em que cada nível representa uma partição do conjunto de dados definida na seção 6.1. A estrutura de agrupamentos hierárquicos em diferentes níveis é representada como uma árvore binária, chamada **dendrograma***.

Os métodos hierárquicos podem ser classificados em aglomerativos ou divisivos. Nos algoritmos aglomerativos, a estrutura hierárquica é construída a partir dos registros, sendo cada registro definido como um grupo *singleton*. Dessa forma, a partição do nível mais baixo da hierarquia é definida como $\mathbb{C}_0 = \mathcal{T}$, com $K = N$ grupos. A partir dos grupos *singletons*, o algoritmo constrói a estrutura hierárquica $\mathbb{C}_0 \subseteq \mathbb{C}_1 \subseteq \dots \subseteq \mathbb{C}_{N-1}$, iterativamente, fundindo grupos das partições dos níveis anteriores até formar a partição $\mathbb{C}_{N-1}$, que corresponde a um único grupo contendo todo o conjunto de dados. Nos algoritmos divisivos, a estratégia é de cima para baixo; a partição inicial $\mathbb{C}_0$ corresponde a um único grupo e a estrutura hierárquica $\mathbb{C}_{N-1} \subseteq \mathbb{C}_{N-2} \subseteq \dots \subseteq \mathbb{C}_0$ [522].

Nesta seção, são apresentados os algoritmos hierárquicos aglomerativos, que são mais comuns e podem ser encontrados na maioria dos *softwares*. Algoritmos divisivos são obtidos aplicando-se recursivamente em cada grupo e diferentes abordagens são encontradas na literatura [194][573].

* A palavra dendrograma é de origem grega, pela composição de *dendron*, que significa árvore, e *gramma*, desenho ou escrita.

6.3.1 Algoritmos aglomerativos

Os algoritmos hierárquicos aglomerativos seguem o procedimento geral descrito pelo Algoritmo 6.2, cujo resultado é uma estrutura hierárquica, utilizada para gerar o dendrograma que representa os agrupamentos formados em cada nível.

Algoritmo 6.2: Algoritmo hierárquico (adaptado de [522])

Entrada:	Conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$
Saída:	Partições hierárquicas $\mathbb{C}_0 \subseteq \mathbb{C}_1 \subseteq \dots \subseteq \mathbb{C}_{N-1}$
01	Início
02	Calcular a matriz de distâncias $\mathbf{D}$
03	Definir $\mathbb{C}_0 = \{\mathcal{C}_i = \mathbf{x}(i), i = 1, \dots, N\}$
04	$k = 1$
05	Enquanto $k < N - 1$
06	Selecionar $\{\mathcal{C}_i, \mathcal{C}_j\} \in \mathbb{C}_{k-1}$ como $\operatorname{argmin}(L(\mathcal{C}_i, \mathcal{C}_j))$
07	Gerar a nova partição $\mathbb{C}_k = \{\mathbb{C}_{k-1} - \{\mathcal{C}_i, \mathcal{C}_j\}\} \cup \{\mathcal{C}_i \cup \mathcal{C}_j\}$
08	$k \leftarrow k + 1$
09	Fim Enquanto
10	Retorna $\{\mathbb{C}_0, \mathbb{C}_1, \dots, \mathbb{C}_{N-1}\}$
11	Fim

Em cada iteração k do Algoritmo 6.2, no passo 07, um par de grupos $\{\mathcal{C}_i, \mathcal{C}_j\}$ é retirado da partição $\mathbb{C}_{k-1}$ e a união $\{\mathcal{C}_i \cup \mathcal{C}_j\}$ é incluída na partição $\mathbb{C}_k$. A função de ligação $L(\mathcal{C}_i, \mathcal{C}_j)$ no passo 06 é utilizada para selecionar os grupos a serem unidos. Diferentes funções de ligação resultam em diferentes agrupamentos. Na literatura, as principais estratégias para função de ligação são:

- Vizinho mais próximo (**single-link**): a distância entre os grupos é calculada pelos elementos mais próximos [495]:

$$L_{\min}(\mathcal{C}_i, \mathcal{C}_j) = \min_{\substack{\mathbf{v} \in \mathcal{C}_i \\ \mathbf{v} \in \mathcal{C}_j}} \operatorname{dist}(\mathbf{v}, \mathbf{v}) \quad (6.7)$$

- Vizinho mais distante (**complete-link**): a distância entre os grupos é calculada pela distância máxima entre seus elementos:

$$L_{max}(\mathcal{C}_i, \mathcal{C}_j) = \max_{\substack{\mathbf{v} \in \mathcal{C}_i \\ \mathbf{u} \in \mathcal{C}_j}} dist(\mathbf{v}, \mathbf{u}) \quad (6.8)$$

- Distâncias médias (**UPGMA**):* a distância entre os grupos é calculada pela distância média entre os elementos [499]:

$$L_{med}(\mathcal{C}_i, \mathcal{C}_j) = \frac{1}{|\mathcal{C}_i||\mathcal{C}_j|} \sum_{\mathbf{v} \in \mathcal{C}_i} \sum_{\mathbf{u} \in \mathcal{C}_j} dist(\mathbf{v}, \mathbf{u}) \quad (6.9)$$

onde $|\mathcal{C}_i|$ representa a cardinalidade, isto é, o número de elementos do grupo $\mathcal{C}_i$;

- Centroides (**UPGMC**):† a distância entre os grupos é calculada pela distância entre os centroides [499]:

$$L_{cen}(\mathcal{C}_i, \mathcal{C}_j) = dist(\bar{\mathbf{v}}, \bar{\mathbf{u}}) \quad (6.10)$$

onde $\bar{\mathbf{v}} = \frac{1}{|\mathcal{C}_i|} \sum_{\mathbf{v} \in \mathcal{C}_i} \mathbf{v}$ e $\bar{\mathbf{u}} = \frac{1}{|\mathcal{C}_j|} \sum_{\mathbf{u} \in \mathcal{C}_j} \mathbf{u}$.

Há também as variantes WPGMA e WPGMC, que são semelhantes, respectivamente, aos critérios de distância média (UPGMA) e de centroide (UPGMC), em que as distâncias são ponderadas pelo número de elementos em cada grupo [194]. Finalmente, o critério de Ward minimiza a variância dos grupos [551].

Os algoritmos hierárquicos aglomerativos, em geral, trabalham com a matriz de distâncias na memória, o que significa armazenar, pelo menos, $N(N - 1)/2$ valores.‡ A aplicação desse tipo de algoritmo em conjuntos de dados com número muito grande de registros pode causar problemas de memória.

6.3.2 Exemplos

Nesta seção, os algoritmos anteriores são avaliados em conjuntos de dados sintéticos e *benchmark*. Todos os exemplos foram executados utilizando a distância euclidiana, variando apenas a função de ligação.

A Figura 6.8 mostra os dendrogramas e as partições resultantes dos quatro algoritmos hierárquicos discutidos previamente no conjunto de dados sintético de quatro grupos (cf. Figura 6.2a). O nível inferior foi limitado a 25 grupos para facilitar a visualização.

* *Unweighted Pair Group Method with Average linkage*

† *Unweighted Pair Group Method with Centroid linkage*.

‡ Considerando que a matriz de distâncias é simétrica e armazenando apenas a matriz triangular.

Observe que, mesmo num exemplo simples como este, os resultados são ligeiramente diferentes, tanto em termos do dendrograma quanto da partição encontrada.

A Figura 6.9 apresenta os resultados dos algoritmos no conjunto de dados de duas luas apresentado (cf. Figura 6.4b). Este exemplo mostra a capacidade do algoritmo do vizinho mais próximo (*single-link*) de encontrar partições não convexas, o que não acontece no caso das demais funções de ligação.

A observação dos resultados na Figura 6.8 e na Figura 6.9 mostra que o dendrograma pode indicar o número de grupos presentes num conjunto de dados pelo número de ramos (traços verticais) entre os dois patamares (traços horizontais) mais distantes. O resultado dessa análise qualitativa é bem visível na Figura 6.8, indicando a presença de quatro grupos (exceto para o algoritmo *single-link*), e na Figura 6.9, indicando a presença de dois grupos.

6.4 Particionamento de grafos

A teoria dos grafos [55][129] é o ramo da matemática que estuda problemas que podem ser representados na forma de um **grafo**, isto é, um conjunto de vértices conectados por um conjunto de arestas. A teoria dos grafos é a base matemática para uma série de disciplinas na ciência da computação e a análise de rede complexas [414].

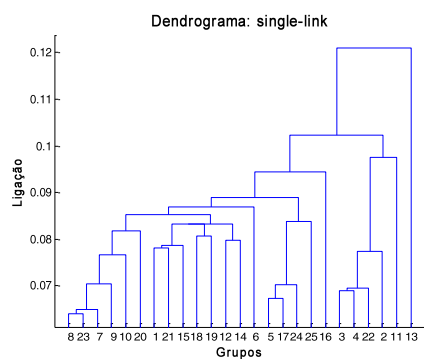
Um conjunto de dados pode ser representado como um grafo $G(\mathcal{V}, \mathcal{E})$, no qual o conjunto de vértices $\mathcal{V}$ representa os registros, isto é, $\mathcal{V} \equiv \mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, e o conjunto de arestas é definido pela matriz de adjacências* $\mathbf{A} \in \mathcal{R}^{N \times N}$, em que cada elemento representa a similaridade entre um par de registros como:

$$a_{ij} = \begin{cases} \text{sim}(\mathbf{x}(i), \mathbf{x}(j)), & \text{se } i \neq j \\ 0, & \text{se } i = j \end{cases} \quad (6.11)$$

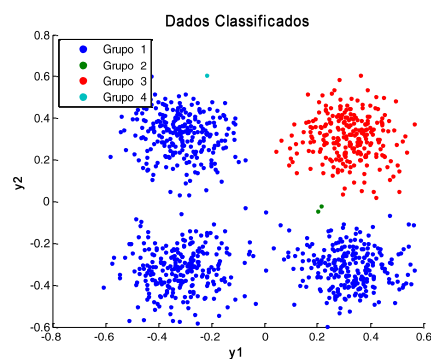
A matriz de similaridades é simétrica e, portanto, as arestas do grafo correspondente a um conjunto de dados são ponderadas e não direcionadas.

O grau de um vértice é o número de vizinhos do vértice. Em grafos ponderados, o grau é a soma do peso das arestas conectadas ao vértice.

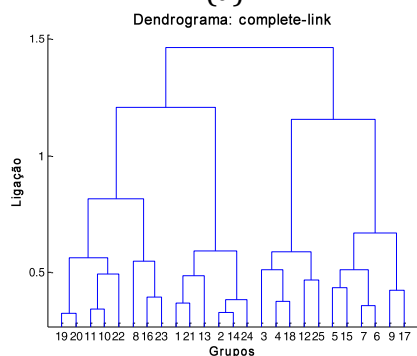
* Aqui, seguimos a notação proposta por Newman [414], em que a matriz de adjacências binária é um caso particular da matriz de adjacências ponderada.



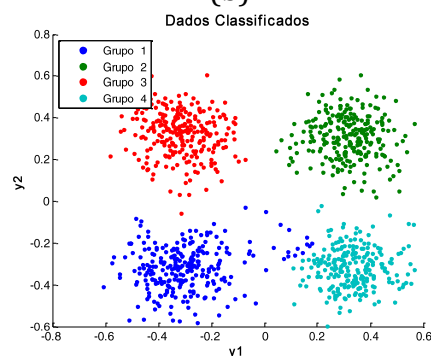
(a)



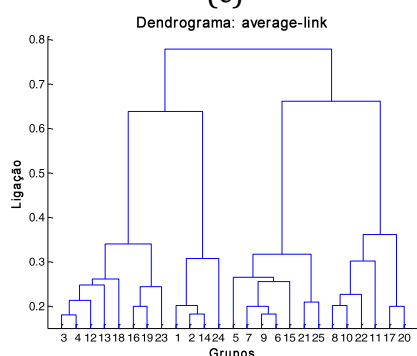
(b)



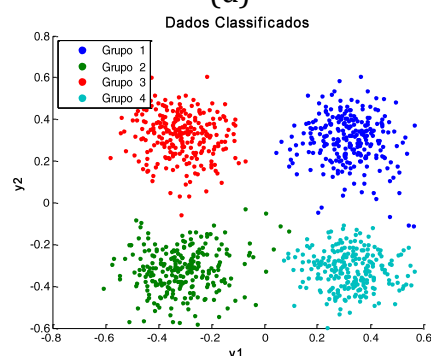
(c)



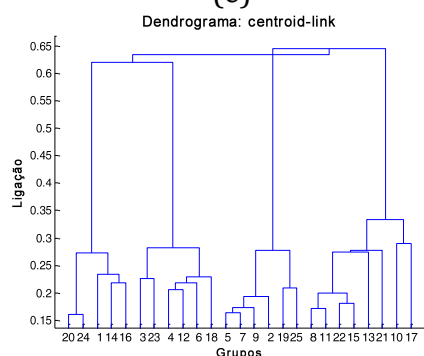
(d)



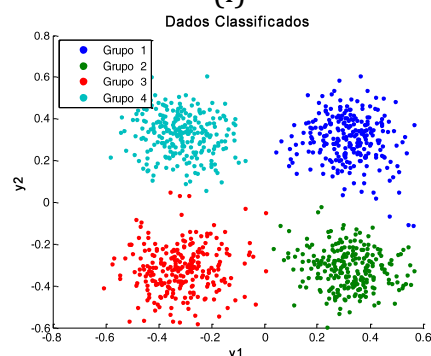
(e)



(f)

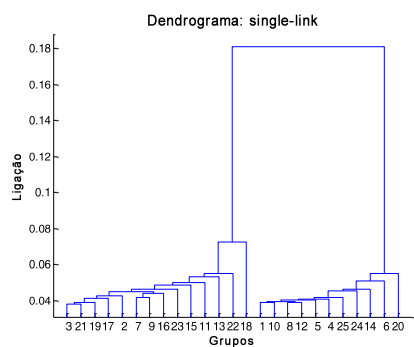


(g)

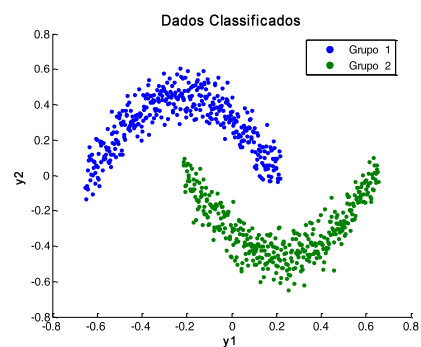


(h)

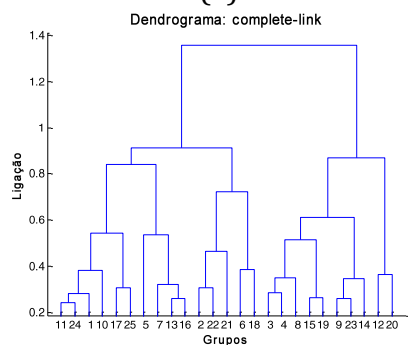
Figura 6.8: Resultados do agrupamento hierárquico nos dados de quatro grupos: (a) e (b) *single-link*; (c) e (d) *complete-link*; (e) e (f) distância média; (g) e (h) centroide.



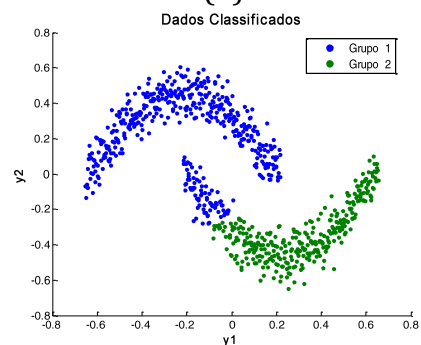
(a)



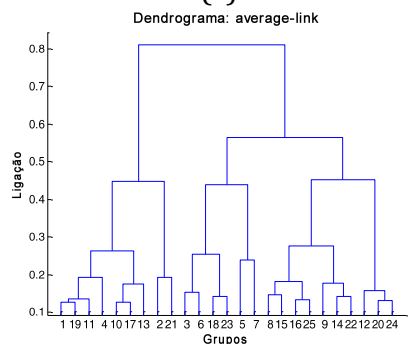
(b)



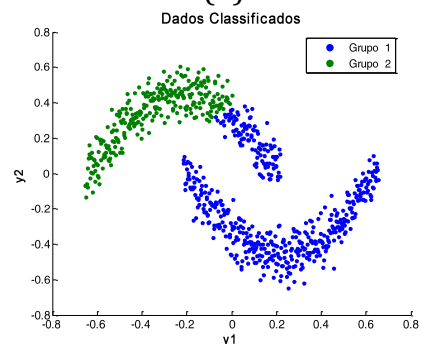
(c)



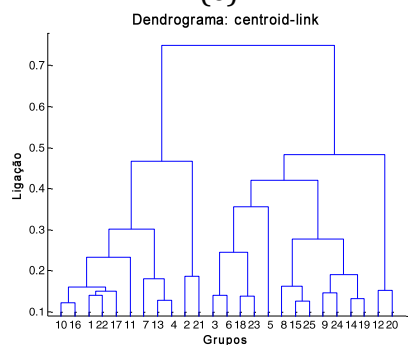
(d)



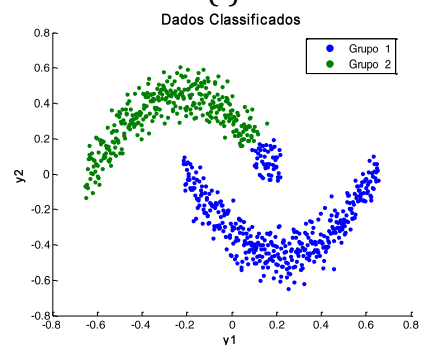
(e)



(f)



(g)



(h)

Figura 6.9: Resultados do agrupamento hierárquico nos dados duas luas: (a) e (b) *single-link*; (c) e (d) *complete-link*; (e) e (f) distância média; (g) e (h) centroide.

A função de similaridade gaussiana é muito utilizada para representação de um conjunto de dados como um grafo e é definida como (cf. equação (2.14)):

$$sim_G(\mathbf{x}(i), \mathbf{x}(j)) = \exp\left(-\frac{\|\mathbf{x}(i) - \mathbf{x}(j)\|^2}{2\sigma^2}\right) \quad (6.12)$$

A Figura 6.10 mostra o exemplo de um grafo gerado a partir de um conjunto de dados de quatro grupos pela função de similaridade gaussiana (6.12) com $\sigma = 1/3$. Na Figura 6.10a, o grafo formado pela conexão dos registros em que os tons de cinza representam os valores das arestas; na Figura 6.10b, a matriz de adjacências.

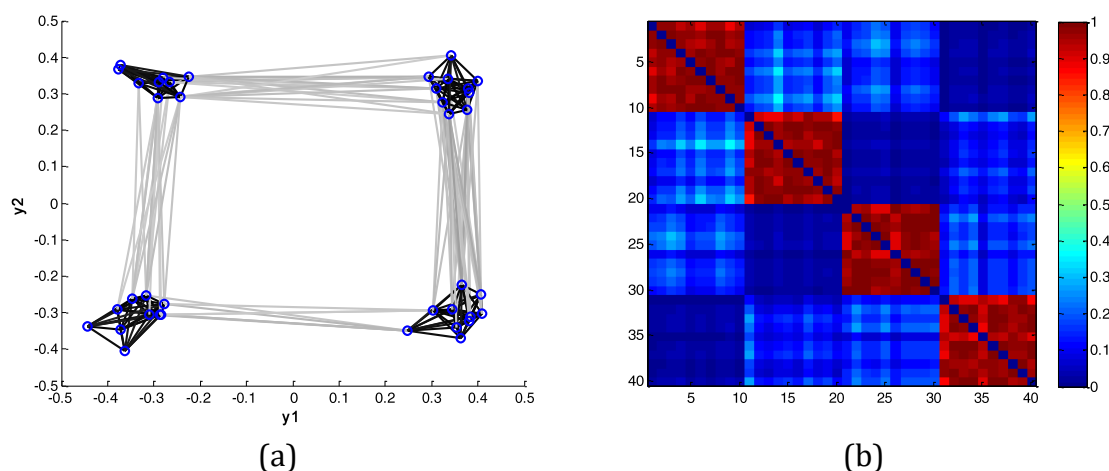


Figura 6.10: Conjuntos de dados quatro grupos: (a) Grafo; (b) matriz de adjacências.

A função de similaridade do cosseno (cf. equação (2.13)) também é frequentemente utilizada, principalmente em aplicações de mineração de textos [468]. Há outras formas de representar um conjunto de dados como um grafo [356][473]. A função de similaridade gaussiana relaciona o particionamento em grafos com métodos baseados em funções de núcleo [182], discutidos no Capítulo 7.

Um corte num grafo conexo é um conjunto de arestas (ou vértices) cuja remoção desconecta um determinado par de vértices [414]. O corte em grafos é estudado principalmente em problemas em que o grafo representa um sistema onde há algum fluxo (de matéria, informação etc.) ao longo das arestas e a capacidade de fluxo é representada pelo valor (ou peso) da aresta. O corte mínimo é o conjunto de arestas que, se retiradas, desconecta um par de vértices e cuja soma dos pesos das arestas retiradas é mínima. O corte mínimo corresponde ao fluxo máximo entre o par de vértices [6].

O problema do corte mínimo num grafo pode ser associado ao problema de agrupamento para encontrar uma partição $\mathbb{C} = \{\mathcal{C}_1, \mathcal{C}_2\}$, tal que $\mathcal{C}_1 \cap \mathcal{C}_2 = \emptyset$ e $\mathcal{C}_1 \cup \mathcal{C}_2 = \mathcal{V}$, através da minimização da soma dos pesos das arestas entre os dois grupos calculada pela função de corte:

$$cut(\mathcal{C}_1, \mathcal{C}_2) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N a_{ij} u_1(\mathbf{x}(i)) u_2(\mathbf{x}(j)) \quad (6.13)$$

onde $a_{ij} \in \mathbf{A}$ são os elementos da matriz de adjacências, $u_i(\mathbf{x}(t))$ é a pertinência de um registro $\mathbf{x}(t)$ ao grupo $\mathcal{C}_i$ (cf. equação (6.3)) e o fator $\frac{1}{2}$ é incluído para evitar a contagem dupla da mesma aresta, uma vez que a matriz de adjacências $\mathbf{A}$ é simétrica.

Em grafos não direcionados, o critério de corte definido pela equação (6.13) admite uma solução trivial que corresponde a separar apenas um vértice cuja soma das suas arestas é mínima, isto é, o vértice de menor grau. Num grafo correspondente a um conjunto de dados, essa solução trivial significa separar um *outlier*. Para evitar esse problema, Shi e Malik [489] propõem minimizar a função de corte normalizado:

$$Ncut(\mathcal{C}_1, \mathcal{C}_2) = \frac{cut(\mathcal{C}_1, \mathcal{C}_2)}{cut(\mathcal{C}_1, \mathcal{V})} + \frac{cut(\mathcal{C}_2, \mathcal{C}_2)}{cut(\mathcal{C}_2, \mathcal{V})} \quad (6.14)$$

onde $cut(\mathcal{C}_k, \mathcal{V}) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N a_{ij} u_k(\mathbf{x}(i))$ e $\mathcal{V}$ é o conjunto de vértices do grafo.

A função de avaliação definida pela equação (6.14) evita a obtenção de um grupo em que as arestas do corte representam uma parcela importante da sua conectividade.

O corte mínimo pode ser ilustrado na Figura 6.10a, em que o agrupamento é definido pelo corte das arestas mais claras. Nesse caso, a medida de corte da equação (6.13) é generalizada para o caso de uma partição genérica $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ como [468]:

$$cut(\mathcal{C}_1, \dots, \mathcal{C}_K) = \sum_{i=1}^K cut(\mathcal{C}_i, \bar{\mathcal{C}}_i) \quad (6.15)$$

onde $\bar{\mathcal{C}}_i = \mathcal{V} - \mathcal{C}_i$ é o complemento de $\mathcal{C}_i$, i.e. $\bar{\mathcal{C}}_i = \bigcup_{j \neq i}^K \mathcal{C}_j$.

Na literatura encontram-se outras variantes da medida de corte em grafos com características ligeiramente diferentes [130][468].

A teoria dos grafos tem sido explorada para o estudo de redes complexas [414]. O problema de particionamento de grafos na área de redes complexas é chamado de **detecção de comunidades** [412][414][468]. Numa rede social, por exemplo, os indivíduos representam os vértices do grafo e as arestas, as relações entre eles. As comunidades são formadas por grupos de indivíduos que compartilham características em comum e, por isso, têm mais relações entre si do que com indivíduos de outros grupos. Essa tendência é chamada de homofilia.

Newman apresenta uma medida para aferir o grau de homofilia de uma rede, chamada modularidade [411]-[413]. A medida de modularidade segue um raciocínio bem diferente da medida de corte em grafos (6.13). Em vez de avaliar o peso das arestas em grupos diferentes, a ideia é avaliar o peso das arestas no mesmo grupo e a medida reflete a fração de arestas que ligam vértices desse grupo. Entretanto, para evitar a solução trivial de considerar toda a rede como um só grupo, a medida de modularidade subtrai a fração de arestas esperada de uma rede cujas ligações ocorrem de forma aleatória. Em outras palavras, a modularidade mede o quanto a estrutura de comunidades encontrada na rede é diferente de um arranjo aleatório.

Matematicamente, a modularidade de uma partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ é definida como:

$$Q(\mathcal{C}_1, \dots, \mathcal{C}_K) = \frac{1}{2E} \sum_{i=1}^N \sum_{j=1}^N \left(a_{ij} - \frac{d_i d_j}{2E} \right) \mathbf{u}_i \mathbf{u}_j^T \quad (6.16)$$

onde $d_i = \sum_j a_{ij}$ é o grau do vértice associado ao registro $\mathbf{x}(i)$ e $E = \sum_i d_i$ é o número total de arestas. O vetor $\mathbf{u}_i = (u_1(\mathbf{x}(i)), \dots, u_K(\mathbf{x}(i)))$ é aquele em que cada elemento é a pertinência do registro $\mathbf{x}(i)$ a cada um dos grupos (cf. equação (6.3)). O produto $\mathbf{u}_i \mathbf{u}_j^T = 1$ se os registros $\mathbf{x}(i)$ e $\mathbf{x}(j)$ estão no mesmo grupo; caso contrário, $\mathbf{u}_i \mathbf{u}_j^T = 0$.

Se os registros $\mathbf{x}(i)$ e $\mathbf{x}(j)$ estão no mesmo grupo, a modularidade da partição calculada pela equação (6.16) é a diferença entre o número total de arestas $\sum_{ij} a_{ij}$ e o número esperado de arestas numa rede aleatória $\sum_{ij} d_i d_j / 2E$ [413].

A modularidade pode ser escrita diretamente a partir da definição da matriz de modularidade $\mathbf{B}$, calculada como:

$$\mathbf{B} = \mathbf{A} - \frac{\mathbf{d}^T \mathbf{d}}{2E} \quad (6.17)$$

onde $\mathbf{d} = (d_1, \dots, d_N)$ é o vetor dos graus dos vértices (registros) correspondentes, em que cada elemento é calculado como $d_i = \sum_j a_{ij}$. Observe que, em geral, a matriz $\mathbf{A}$ é esparsa, mas $\mathbf{B}$ é uma matriz densa.

De uma forma geral, o problema de particionamento em grafos é um problema NP-difícil por razões análogas à complexidade do problema de particionamento de um conjunto de dados. Dessa forma, uma solução computacionalmente tratável para o problema é obtida por uma formulação aproximada, baseada na decomposição espectral de uma matriz derivada da matriz de adjacências do grafo $\mathbf{A}$, chamada de laplaciano [356][408]. Essa abordagem dá origem a diferentes algoritmos, apresentados a seguir.

6.4.1 Algoritmos de agrupamento espectral

Uma partição $\mathbb{C} = \{\mathcal{C}_1, \mathcal{C}_2\}$ num grafo pode ser representada por um vetor $\mathbf{v} \in \mathcal{R}^N$, que representa a partição através da variável indicadora:

$$v_i = \begin{cases} +1, & \text{se } \mathbf{x}(i) \in \mathcal{C}_1 \\ -1, & \text{se } \mathbf{x}(i) \in \mathcal{C}_2 \end{cases} \quad (6.18)$$

onde $\mathcal{C}_1 \cap \mathcal{C}_2 = \emptyset$ e $\mathcal{C}_1 \cup \mathcal{C}_2 =$.

A variável indicadora permite identificar se dois registros $\mathbf{x}(i)$ e $\mathbf{x}(j)$ estão no mesmo grupo como $u_1(\mathbf{x}(i))u_2(\mathbf{x}(j)) = \frac{1}{2}(1 - v_i v_j)$. Se os registros estão no mesmo grupo, $\frac{1}{2}(1 - v_i v_j) = 0$; caso contrário, $\frac{1}{2}(1 - v_i v_j) = 1$. Dessa forma, a medida de corte (6.13) pode ser escrita como:

$$cut(\mathcal{C}_1, \mathcal{C}_2) = \frac{1}{4} \sum_{i=1}^N \sum_{j=1}^N a_{ij} (1 - v_i v_j) \quad (6.19)$$

Considerando $\sum_{i=1}^N \sum_{j=1}^N a_{ij} = \sum_{i=1}^N d_i = \sum_{i=1}^N \sum_{j=1}^N d_i \delta_{ij}$ [414], onde $d_i = \sum_j a_{ij}$ é o grau do vértice e δ_{ij} , a função δ de Kronecker,* a equação (6.19) é reescrita como:

$$cut(\mathcal{C}_1, \mathcal{C}_2) = \frac{1}{4} \sum_{i=1}^N \sum_{j=1}^N (d_i \delta_{ij} - a_{ij}) v_i v_j \quad (6.20)$$

* A função δ de Kronecker é definida como: $\delta_{ij} = \begin{cases} 1, & \text{se } i = j \\ 0, & \text{se } i \neq j \end{cases}$.

A medida de corte escrita na forma (6.20) evidencia os valores $l_{ij} = (d_i \delta_{ij} - a_{ij})$, que são os elementos do **laplaciano** do grafo, definido como:

$$\mathbf{L} = \mathbf{D} - \mathbf{A} \quad (6.21)$$

onde $\mathbf{D}$ é uma matriz diagonal em que cada elemento $d_{ii} = \sum_j a_{ij}$ é o grau do vértice (registro) correspondente.

O laplaciano do grafo recebe esse nome devido a uma analogia com o operador laplaciano da equação da difusão [414]. O laplaciano tem grande importância nas propriedades topológicas de grafos [104][381].

A medida de corte (6.20) é escrita em função do laplaciano e da variável indicadora da partição $\mathbb{C} = \{\mathcal{C}_1, \mathcal{C}_2\}$ pela seguinte equação vetorial:

$$cut(\mathcal{C}_1, \mathcal{C}_2) = \frac{1}{4} \mathbf{v}^T \mathbf{L} \mathbf{v} \quad (6.22)$$

A minimização da função de corte definida pela equação (6.22) não é uma tarefa simples, considerando a restrição $v_i \in \{-1, 1\}$ imposta pela definição da variável indicadora na equação (6.18). Entretanto, uma solução aproximada $\hat{\mathbf{v}} \in \mathcal{R}^N$ pode ser calculada pela solução do problema de autovalor:

$$\mathbf{L} \mathbf{W} = \mathbf{W} \mathbf{\Lambda} \quad (6.23)$$

onde $\mathbf{\Lambda}$ é a matriz diagonal dos autovalores cujos elementos são ordenados de forma que $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_N$; $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_N]$ é a matriz dos autovetores em que cada coluna $\mathbf{w}_i \in \mathcal{R}^N$ é o autovetor correspondente ao autovalor λ_i .

O laplaciano é uma matriz positiva semi-definida,* de modo que todos os seus autovalores são números reais e positivos [206][414]. O laplaciano tem sempre um autovalor nulo, isto é, $\lambda_1 = 0$, que corresponde ao autovetor unitário $\mathbf{w}_1 = [1/\sqrt{N}, \dots, 1/\sqrt{N}]^T$ [414]. O número de autovalores nulos corresponde ao número de componentes conexas do grafo. Assim, se $\lambda_2 > 0$, o grafo é conexo, isto é, há apenas um componente e existe um caminho entre quaisquer pares de vértices. Quanto maior o autovalor λ_2 , maior a conectividade do grafo [179]. O segundo autovetor do laplaciano $\mathbf{w}_2$, correspondente ao segundo autovalor λ_2 , é conhecido como o autovetor de Fiedler [408], em homenagem ao matemático Miroslav Fiedler [179][180].

* Uma matriz $\mathbf{A}$ é positiva semi-definida se $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0, \forall \mathbf{x}$.

Uma partição genérica $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, segundo o corte mínimo (6.15), é obtida aplicando-se o algoritmo k-médias aos K autovetores correspondentes aos K menores autovaleiros da solução do problema (6.23). O algoritmo de agrupamento espectral básico é descrito pelo Algoritmo 6.3.

Algoritmo 6.3: Algoritmo de agrupamento espectral

Entrada:	Conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, parâmetro de dispersão σ e número de centros K
Saída:	Partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$
01	Início
02	Calcular a matriz $\mathbf{A}$, pela equação (6.11)
03	Calcular o laplaciano
04	Calcular a decomposição espectral do laplaciano
05	Calcular a matriz $\mathbf{Z}$ a partir dos autovetores do laplaciano
06	Calcular a partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ pelo k-médias em $\mathbf{Z}$
07	Retorna $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$
08	Fim

Von Luxburg [356] sugere que a matriz de adjacências binária possa ser utilizada no passo 02, em que uma aresta é definida por um valor mínimo de similaridade ou por um número especificado de vizinhos. Entretanto, a forma mais usual é utilizar diretamente a matriz de similaridade nos algoritmos [408].

A partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ obtida no passo 06 do Algoritmo 6.3 é definida no espaço de variáveis transformadas $\mathbf{Z}$ e, devido à transformação não linear realizada pelo cálculo da similaridade, o Algoritmo 6.3 pode encontrar uma partição não convexa no espaço de variáveis originais. O problema da inicialização aleatória do k-médias também é resolvido no algoritmo de agrupamento espectral. A matriz identidade pode ser utilizada na inicialização dos centros, uma vez que o algoritmo realiza uma partição de K grupos no espaço de variáveis transformadas, de K dimensões.

O algoritmo de agrupamento espectral descrito pelo Algoritmo 6.3 foi formulado em função da medida de corte mínimo, mas algumas variantes do algoritmo básico são obtidas segundo a medida de corte utilizada. As principais variantes são apresentadas a

seguir, começando pela formalização do próprio algoritmo de corte mínimo. Para facilitar a apresentação, a mesma notação será utilizada para denotar a matriz de autovalores $\mathbf{\Lambda}$ e a matriz de autovetores $\mathbf{W}$ em cada variante do algoritmo espectral. Essas matrizes são diferentes em função do problema formulado em cada algoritmo.

6.4.1.1 Algoritmo de corte mínimo

No algoritmo de corte mínimo [356][408], os passos 03 e 04 são calculados pelas equações (6.21) e (6.23). A matriz de dados transformados $\mathbf{Z} = [\mathbf{w}_1, \dots, \mathbf{w}_K]$, onde $\mathbf{w}_i$ são os autovetores calculados pela equação (6.23), ordenados pela ordem crescente de seus autovalores [356][408]. Os passos 03-05 do Algoritmo 6.3 para o corte mínimo são formulados no Algoritmo 6.3^a.

Algoritmo 6.3a: Corte mínimo

03	$\mathbf{L} = \mathbf{D} - \mathbf{A}$
04	$\mathbf{LW} = \mathbf{W}\mathbf{\Lambda}$
05	Ordenar $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_N$, $\mathbf{Z} = [\mathbf{w}_1, \dots, \mathbf{w}_K]$

6.4.1.2 Algoritmo de corte mínimo normalizado de Shi e Malik

Uma variante do Algoritmo 6.3a para a minimização do corte mínimo normalizado (6.14) foi proposta por Shi e Malik [488][489]. Nesse caso, a decomposição espectral do passo 04 é calculada pela solução do problema de autovalor generalizado:

$$\mathbf{LW} = \mathbf{DWA} \quad (6.24)$$

onde $\mathbf{L}$ é o laplaciano (6.21) e $\mathbf{D}$ é a matriz diagonal de graus.

Shi e Malik [488] mostram que o sinal do segundo menor autovalor de (6.24) é uma aproximação da solução pelo corte mínimo normalizado (6.14). Para uma partição genérica em K grupos, os autores apresentam uma abordagem hierárquica divisiva em que, a cada iteração, uma nova partição pode ser encontrada pela aplicação recursiva do problema de autovalor (6.24) [489].

Na abordagem hierárquica divisiva, a definição do número de grupos é substituída por um critério de parada do algoritmo. Entretanto, essa abordagem pode se tornar instável, além de computacionalmente complexa [489].

Uma partição de K grupos pode ser obtida pela aplicação do Algoritmo 6.3, em que a matriz das variáveis transformadas é definida pelos autovetores correspondentes aos menores autovalores $\mathbf{Z} = [\mathbf{w}_1, \dots, \mathbf{w}_K]$. O algoritmo do corte mínimo normalizado é apresentado pelo Algoritmo 6.3b.

Algoritmo 6.3b: Corte mínimo normalizado [489]

03	$\mathbf{L} = \mathbf{D} - \mathbf{A}$
04	$\mathbf{LW} = \mathbf{DWA}$
05	Ordenar $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_N$, $\mathbf{Z} = [\mathbf{w}_1, \dots, \mathbf{w}_K]$

O algoritmo de corte mínimo normalizado de Shi e Malik (Algoritmo 6.3b) tem o mesmo resultado do algoritmo de corte mínimo (Algoritmo 6.3a).

A equação (6.24) também pode ser escrita como:

$$\hat{\mathbf{L}}\hat{\mathbf{W}} = \hat{\mathbf{W}}\mathbf{A} \quad (6.25)$$

onde $\hat{\mathbf{L}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{1} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ e $\hat{\mathbf{W}} = \mathbf{D}^{1/2}\mathbf{W}$ [489]. A matriz $\hat{\mathbf{L}}$ é chamada de laplaciano simétrico [356] ou, ainda, laplaciano normalizado [408].

6.4.1.3 Algoritmo de corte mínimo normalizado de Ng, Jordan e Weiss

Uma variante do algoritmo de corte mínimo normalizado foi apresentada por Ng, Jordan e Weiss [415]. O laplaciano no passo 03 é substituído pela matriz $\hat{\mathbf{A}} = \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, chamada de matriz de adjacências normalizada [104] ou matriz de adjacências reduzida [414]. Alguns autores sugerem que a decomposição espectral seja calculada com o Laplaciano normalizado $\hat{\mathbf{L}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$ [356][408]. No artigo original [415], a decomposição espectral do passo 04 é calculada como:

$$\hat{\mathbf{A}}\mathbf{W} = \mathbf{W}\hat{\mathbf{\Lambda}} \quad (6.26)$$

A decomposição espectral (6.26) produz os mesmos autovetores $\mathbf{W}$ que (6.25) e autovalores $\hat{\mathbf{\Lambda}} = \mathbf{1} - \mathbf{\Lambda}$, onde $\mathbf{\Lambda}$ é a matriz de autovalores calculada em (6.25). Dessa forma, a solução do problema (6.26) corresponde à minimização de corte mínimo normalizado (6.14) com os autovalores em ordem não crescente $\hat{\lambda}_1 \geq \hat{\lambda}_2 \geq \dots \geq \hat{\lambda}_N$.

A matriz $\hat{\mathbf{A}}$ tem outras propriedades interessantes. Num grafo definido por uma matriz de adjacências binária, isto é, $a_{ij} \in \{0,1\}$, o maior autovalor $\hat{\lambda}_1 = 1$. O número de autovalores unitários corresponde ao número de componentes do grafo [104][342] e, em

grafos conexos, $\hat{\lambda}_2 < 1$ [104]. Num grafo com K componentes que definem uma partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, o valor do corte mínimo $cut(\mathcal{C}_1, \dots, \mathcal{C}_K) = 0$ (cf. (6.15)) e a matriz $\hat{\mathbf{A}}$ tem K autovalores unitários: $\hat{\lambda}_i = 1, i = 1, \dots, K$. Essa propriedade permite utilizar o resultado da decomposição espectral (6.26) para uma estimativa do número de grupos de um conjunto de dados [342][168][170].

As variáveis transformadas são calculadas, para cada registro, pelos autovetores normalizados como:

$$\hat{\mathbf{z}}(t) = \left[\frac{w_1(t)}{\|\mathbf{z}(t)\|}, \dots, \frac{w_K(t)}{\|\mathbf{z}(t)\|} \right] \quad (6.27)$$

onde $\mathbf{z}(t) = [w_1(t), \dots, w_K(t)]$ e $w_i(t)$ é o elemento t do autovetor correspondente ao autovalor $\hat{\lambda}_i$, calculado na decomposição espectral (6.26).

A utilização das variáveis transformadas normalizadas no k-médias é equivalente à utilização do k-médias esférico nas variáveis transformadas sem normalização [468]. Na prática, a normalização (6.27) também pode ser aplicada aos demais algoritmos de corte mínimo. O algoritmo do corte normalizado de Ng, Jordan e Weiss é descrito no Algoritmo 6.3c.

Algoritmo 6.3c: Corte mínimo normalizado [415]

03	$\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$
04	$\hat{\mathbf{A}} \mathbf{W} = \mathbf{W} \hat{\mathbf{A}}$
05	Ordenar $1 = \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N$, $\hat{\mathbf{z}}(t) = \left[\frac{w_1(t)}{\ \mathbf{z}(t)\ }, \dots, \frac{w_K(t)}{\ \mathbf{z}(t)\ } \right]$

A complexidade computacional do cálculo da matriz de similaridade é $O(N^2)$ e a complexidade do algoritmo de agrupamento espectral é dominada pelo cálculo do problema de autovalor, que é de $O(N^3)$. Em problemas que $K \ll N$, a complexidade pode ser reduzida a $O(N^2)$, uma vez que apenas os primeiros autovetores são necessários.

O algoritmo de agrupamento espectral pode separar grupos não convexos no espaço de variáveis de entrada do problema e, por isso, tem bons resultados em aplicações de agrupamento. Essas características motivaram o desenvolvimento de outras variantes do algoritmo [23][356][408][554]. Os algoritmos de agrupamento espectral são relacionados aos algoritmos baseados em função de núcleo [127][128][182][596].

6.4.1.4 Algoritmo de maximização da modularidade de Newman e Girvan

O problema de maximização da modularidade (cf. equação (6.16)) também tem solução aproximada pela decomposição espectral da matriz de modularidade $\mathbf{B}$. No caso de uma partição binária $\mathbb{C} = \{\mathcal{C}_1, \mathcal{C}_2\}$, o produto $\mathbf{u}_i \mathbf{u}_j^T$ é escrito em função variável indicadora (6.18) como $\mathbf{u}_i \mathbf{u}_j^T = \frac{1}{2}(v_i v_j + 1)$, de modo que $\frac{1}{2}(v_i v_j + 1) = 1$ se dois registros $\mathbf{x}(i)$ e $\mathbf{x}(j)$ estão no mesmo grupo e $\frac{1}{2}(v_i v_j + 1) = 0$ caso contrário.

A função de modularidade (6.16) é reescrita como:

$$Q(\mathcal{C}_1, \mathcal{C}_2) = \frac{1}{4E} \sum_{i=1}^N \sum_{j=1}^N b_{ij} (v_i v_j + 1) = \frac{1}{4E} \left(\sum_{i=1}^N \sum_{j=1}^N b_{ij} v_i v_j + \sum_{i=1}^N \sum_{j=1}^N b_{ij} \right) \quad (6.28)$$

onde b_{ij} são os elementos da matriz de modularidade $\mathbf{B}$ definida em (6.17). A soma das linhas (ou colunas) da matriz $\mathbf{B}$ é zero, de forma que o termo $\sum_{ij} b_{ij} = 0$. A função de modularidade pode ser escrita vetorialmente como:

$$Q(\mathcal{C}_1, \mathcal{C}_2) = \frac{1}{4E} \mathbf{v}^T \mathbf{B} \mathbf{v} \quad (6.29)$$

Uma aproximação para a solução do problema de maximização da modularidade (6.29) é obtida pela solução do problema de autovalor $\mathbf{B} \mathbf{w} = \lambda \mathbf{w}$, em que λ é o maior autovalor e $\hat{\mathbf{v}}$, o autovetor correspondente. O sinal dos elementos do autovetor correspondente ao maior autovalor é utilizado para a obtenção da partição $\mathbb{C} = \{\mathcal{C}_1, \mathcal{C}_2\}$:

$$\hat{v}_i = \begin{cases} +1, & \text{se } w_i \geq 0 \\ -1, & \text{se } w_i < 0 \end{cases} \quad (6.30)$$

Newman e Girvan [413] apresentam uma abordagem hierárquica divisiva para o particionamento do grafo em que, a cada iteração, o valor do ganho da função modularidade é avaliado e utilizado para a decisão sobre um novo particionamento. Vieira e colaboradores [541] apresentam um estudo de desempenho e o código dos principais algoritmos de detecção de comunidades baseados na função de modularidade.

6.4.2 Exemplos

A Figura 6.11 mostra, em cada linha, os resultados dos algoritmos 6.3a-c no conjunto de dados de duas luas utilizando o parâmetro $\sigma = 0.1$ no cálculo da função de similaridade. Na coluna da direita são apresentados os resultados do agrupamento e, na coluna da esquerda, as variáveis transformadas.

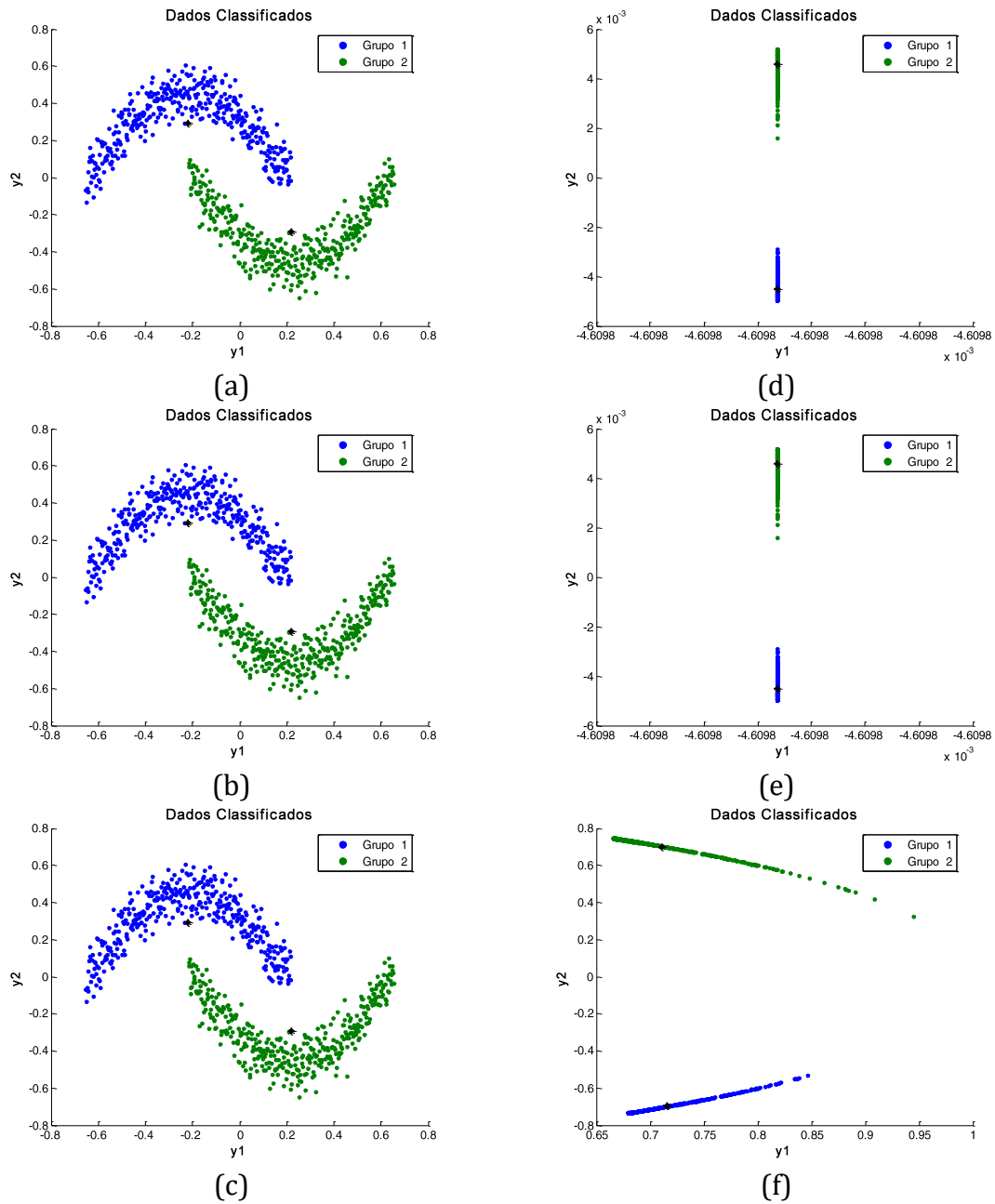


Figura 6.11: Resultados no conjunto de dados duas luas: à esquerda, os resultados do agrupamento e, à direita, as variáveis transformadas; nas linhas, os Algoritmos 6.3a-c.

Podemos observar que os algoritmos espectrais baseados no corte mínimo podem gerar grupos de forma não convexa e as três formulações produzem os mesmos resultados. Os resultados das variáveis transformadas utilizadas nos algoritmos de corte mínimo (6.3a) e corte mínimo normalizado (6.3b) também são idênticos. Uma aplicação

do algoritmo de maximização da modularidade para agrupamento de documentos será apresentada na seção 6.6.2.

Os resultados dos algoritmos de agrupamento espectral dependem do valor do parâmetro de dispersão σ no cálculo da função de similaridade gaussiana (6.12), além da informação do número de grupos K para o algoritmo k-médias. Entretanto, a inicialização do algoritmo k-médias é definida pela matriz identidade e o resultado do algoritmo espectral é determinístico.

Na análise de agrupamentos, a decisão sobre o melhor número de grupos depende de alguma medida de validação, como apresentado a seguir.

6.5 Validação de Grupos

Encontrar uma partição num conjunto de dados é um problema mal definido e diferentes agrupamentos podem ser obtidos do mesmo conjunto (cf. Figura 6.1). Não há uma medida absoluta de qualidade para o problema e o resultado do agrupamento deve ser avaliado no contexto da aplicação.

As medidas de validação de agrupamentos visam avaliar as características geométricas do agrupamento formado, sabendo-se que um bom agrupamento deve produzir grupos compactos e bem separados. É comum encontrar na literatura a divisão das medidas de validação de agrupamentos em três tipos [194][227][228][573], cujos principais índices são apresentados na sequência:

- **Medidas externas:** avaliam a qualidade do agrupamento em relação a uma informação externa sobre a estrutura de grupos (ou classes) do problema;
- **Medidas internas:** avaliam a qualidade do agrupamento a partir dos registros do conjunto de dados, em geral utilizando critérios geométricos;
- **Medidas relativas:** produzem índices calculados pela combinação de critérios (internos) que avaliam a qualidade de um agrupamento em relação a outros.

6.5.1 Medidas externas

As medidas externas avaliam a qualidade do agrupamento a partir de uma matriz de confusão como na avaliação de classificadores (cf. Tabela 5.2). Para avaliação de grupos, a matriz de confusão relaciona os grupos encontrados pelo algoritmo de agrupa-

mento com as classes do problema. A Tabela 6.1 mostra a matriz de confusão para avaliação de agrupamentos, onde $\{C_1, \dots, C_m\}$ é o conjunto de classes e $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, uma partição de K grupos. Cada elemento n_{ij} na Tabela 6.1 representa o número de registros da classe C_i alocados no grupo $\mathcal{C}_j$. A soma das linhas corresponde ao número de elementos das classes e a soma das colunas, o número de elementos dos grupos.

Quando o número de grupos é igual ao número de classes, qualquer estatística de avaliação de classificadores (cf. seção 5.3.1) pode ser utilizada como medida de validação externa. Entretanto, é comum que o número de grupos seja diferente do número de classes e, nesse caso, as medidas externas avaliam o quanto o agrupamento encontrado corresponde à estrutura de classes conhecida.

Tabela 6.1: Matriz de confusão

	$\mathcal{C}_1$	...	$\mathcal{C}_K$	Total
C_1	n_{11}	...	n_{1K}	N_1
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
C_m	n_{m1}	...	n_{mK}	N_m
Total	$\hat{N}_1$	...	$\hat{N}_K$	N

A **pureza** de um grupo é a porção dos registros de um grupo que pertence à classe que contém a maioria dos registros daquele grupo. A pureza é uma medida de homogeneidade do agrupamento: quanto maior a pureza, mais homogêneos são os grupos. A pureza de um grupo é calculada como:

$$pur(\mathcal{C}_j) = \max_{i=1, \dots, m} \left(\frac{n_{ij}}{\hat{N}_j} \right) \quad (6.31)$$

A pureza total ou pureza do agrupamento é calculada como a média da pureza dos grupos ponderada pelo número de registros de cada grupo:

$$PUR(\mathbb{C}) = \sum_{j=1}^K \frac{\hat{N}_j}{N} pur(\mathcal{C}_j) \quad (6.32)$$

A entropia de Shannon é uma medida clássica da teoria da informação que pode ser utilizada para avaliar a heterogeneidade do agrupamento: quanto menor a entropia, melhor o agrupamento. A **entropia** de um grupo é calculada como (cf. Tabela 6.1):

$$ent(\mathcal{C}_j) = -\frac{1}{\log_2 m} \sum_{i=1}^m \left(\frac{n_{ij}}{\hat{N}_j} \right) \log_2 \left(\frac{n_{ij}}{\hat{N}_j} \right) \quad (6.33)$$

A entropia do agrupamento é calculada como a média da entropia dos grupos ponderada pelo número de registros de cada grupo:

$$ENT(\mathbb{C}) = \sum_{j=1}^K \frac{\hat{N}_j}{N} ent(\mathcal{C}_j) \quad (6.34)$$

Há outras medidas externas para avaliação da qualidade de agrupamentos baseadas em contagem de pares e informação mútua [544]. Entretanto, nem sempre uma informação de classes é conhecida para a avaliação do agrupamento. Além disso, a estrutura de grupos pode não coincidir com a estrutura de classes, o que não significa que o agrupamento esteja inadequado (cf. Figura 6.2). Medidas internas e relativas são mais comuns em problemas de agrupamento.

6.5.2 Medidas internas

Medidas internas avaliam um agrupamento utilizando apenas as informações dos registros do conjunto de dados e o resultado do agrupamento. A função de avaliação de um algoritmo de agrupamento, por exemplo, é uma medida interna de avaliação. Em algoritmos hierárquicos, o coeficiente de correlação cofenética [572] é empregado para avaliação de agrupamentos.

Uma medida interna muito utilizada é o índice de partição (PI, na sigla em inglês), proposta por Bensaid e colaboradores [41] para avaliação de agrupamentos *fuzzy*. O índice PI, quando calculado com funções de pertinência binárias, é a razão entre o critério quadrático do k-médias e a soma ponderada das distâncias entre centros:

$$PI(\mathbb{C}) = \frac{\sum_{i=1}^K \sum_{t=1}^N u_i(\mathbf{x}(t))^2 \|\mathbf{x}(t) - \boldsymbol{\omega}_i\|^2}{\sum_{i=1}^K \sum_{j=1}^K N_i \|\boldsymbol{\omega}_i - \boldsymbol{\omega}_j\|^2} = \frac{tr(\boldsymbol{\Sigma}_W)}{\sum_{i=1}^K \sum_{j=1}^K N_i \|\boldsymbol{\omega}_i - \boldsymbol{\omega}_j\|^2} \quad (6.35)$$

onde $tr(\boldsymbol{\Sigma}_W)$ é o traço da matriz de variância intragrupos (cf. equação (6.6)).

Na avaliação de agrupamentos obtidos pelo algoritmo k-médias e suas variantes, o índice PI permite a escolha do melhor agrupamento de um conjunto de soluções obtidas com diversas inicializações aleatórias, mas com o mesmo número de grupos K .

Uma proposta para avaliação de agrupamentos é a visualização qualitativa da tendência de agrupamento (VAT)* [49][239]. A VAT é basicamente o *data image* do conjunto de dados com os registros ordenados segundo o resultado do agrupamento. Um bom agrupamento mostra um contraste mais intenso que um agrupamento de pior qualidade, para o mesmo conjunto de dados.

A Figura 6.12 mostra os resultados do agrupamento do conjunto de dados Íris com $K = 4$ grupos. Foram realizados 25 agrupamentos distintos e selecionados os de menor e maior valor de PI, que são apresentados, respectivamente, na coluna da esquerda e na da direita da Figura 6.12. Podemos observar que o resultado com menor valor de PI (à esquerda) resulta num contraste ligeiramente maior que o resultado com maior valor de PI (à direita).

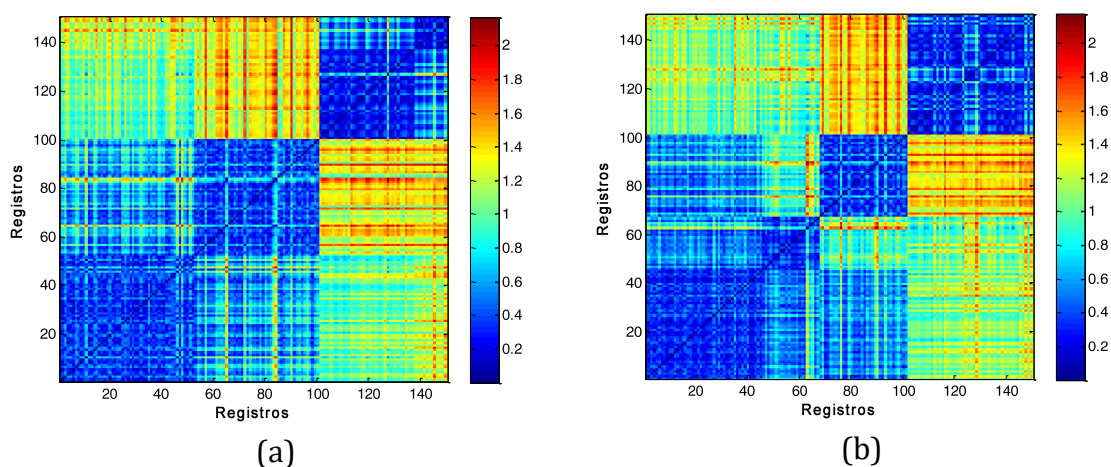


Figura 6.12: *Data image* do resultado do agrupamento do conjunto de dados Iris com $K = 4$:

(a) resultado com menor PI; (b) resultado com maior PI.

A avaliação qualitativa pelo *data image* é muito subjetiva, sendo preferível a utilização de uma métrica quantitativa para a escolha do melhor agrupamento. As medidas relativas permitem avaliar o número de grupos, principalmente em algoritmos de partição.

* *Visual Assessment of (cluster) Tendency.*

6.5.3 Medidas relativas

Os métodos de partição, principalmente o k-médias e suas variantes, são os mais utilizados na prática e, nesse caso, as medidas relativas são empregadas para a descoberta do número de grupos. As medidas relativas são índices utilizados para comparação de diversas soluções do mesmo algoritmo, variando-se o número de grupos. No algoritmo k-médias, o resultado depende da inicialização e as medidas relativas podem variar para o mesmo número de grupos, sendo, portanto, uma função do agrupamento gerado pelo algoritmo e não somente do número de grupos.

Num estudo realizado em 1985, Milligan e Cooper [388] compararam 30 medidas de avaliação de agrupamento existentes na época. A medida que obteve o melhor resultado foi a proposta por Calinski e Harabasz [71] em 1974.

O índice Calinski-Harabasz (CH) de uma partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ é calculado como:

$$CH(\mathbb{C}) = \frac{N - K}{K - 1} \frac{tr(\mathbf{\Sigma}_B)}{tr(\mathbf{\Sigma}_W)} \quad (6.36)$$

onde, $tr(\mathbf{\Sigma}_B)$ e $tr(\mathbf{\Sigma}_W)$ são, respectivamente, os traços das matrizes de variância entre grupos e intragrupos, definidas nas equações (6.5) e (6.6).

O índice CH é um critério de maximização, uma vez que o valor pequeno de $tr(\mathbf{\Sigma}_W)$ indica grupos coesos e um valor grande de $tr(\mathbf{\Sigma}_B)$, grupos separados.

Um índice semelhante, proposto por Pakhira, Bandyopadhyay e Maulik [423] em 2006, também é um critério de maximização calculado pelo produto de três termos:

$$PBM(\mathbb{C}) = \left(\frac{1}{K} \frac{E_1}{E_K} \frac{1}{D_{\max}} \right)^2 \quad (6.37)$$

onde $E_1 = \sum_{t=1}^N \|\mathbf{x}(t) - \bar{\mathbf{x}}\|$ é constante para um determinado conjunto de dados. O termo, $E_K = \sum_{t=1}^N \sum_{i=1}^K u_i(\mathbf{x}(t)) \|\mathbf{x}(t) - \mathbf{\omega}_i\|$, diminui com o aumento do número de grupos K e o termo $D_{\max} = \max_{i,j=1 \dots K} (\|\mathbf{\omega}_i - \mathbf{\omega}_j\|^2)$ aumenta com o número de grupos.

O índice PBM utiliza os valores de distâncias nas somas de E_1 e E_K , ao passo que as matrizes de variância utilizadas no índice CH são calculadas pela soma das diferenças ao quadrado (cf. equações (6.4)-(6.6)). A razão E_1/E_K favorece agrupamentos cuja variância intragrupos é a menor porção da variância total. O primeiro e o terceiro fatores

do índice PBM favorecem, respectivamente, agrupamentos com pequeno número de grupos e com centros separados.

Uma medida de avaliação relativa muito conhecida é o coeficiente de silhueta [462], que avalia a qualidade do agrupamento para cada registro como:

$$sw_i(t) = - \frac{(b_i(t) - a_i(t))}{\max_{t=1, \dots, N} (a_i(t), b_i(t))} \quad (6.38)$$

onde $a_i(t)$ é a média das distâncias entre o registro $\mathbf{x}(t)$ e todos os outros registros alocados ao mesmo grupo $\mathcal{C}_i$ e $b_i(t)$ é a menor distância média entre $\mathbf{x}(t)$ e os registros alocados aos demais grupos $\mathcal{C}_j \neq \mathcal{C}_i$.

O valor $a_i(t)$ é calculado para cada registro $\mathbf{x}(t) \in \mathcal{C}_i$ como a distância média para os demais elementos do grupo:

$$a_i(t) = \frac{1}{\bar{N}_i} \sum_{k=1}^N dist(\mathbf{x}(t), \mathbf{x}(k)), \quad \mathbf{x}(k) \in \mathcal{C}_i \quad (6.39)$$

O valor $b_i(t) = \min_j (\bar{a}_j(t))$, onde $\bar{a}_j(t)$ é a distância média de $\mathbf{x}(t)$ para os registros dos demais grupos $\mathcal{C}_j, j = 1, \dots, K, j \neq i$, calculada como:

$$\bar{a}_j(t) = \frac{1}{\bar{N}_j} \sum_{k=1}^N dist(\mathbf{x}(t), \mathbf{x}(k)), \quad \mathbf{x}(t) \in \mathcal{C}_j, j \neq i \quad (6.40)$$

O coeficiente de silhueta é definido no intervalo $-1 < sw_i(t) < 1$, sendo que o valor $sw_i(t) \approx 1$ é obtido quando $a_i(t) \approx 0$, o que indica que o registro está no centro de um agrupamento compacto. Quanto maior o valor do coeficiente de silhueta, melhor a posição do registro no agrupamento. O coeficiente de silhueta também pode ser utilizado numa etapa de refinamento, verificando-se seu valor caso o registro seja alocado em outro grupo.

O coeficiente de silhueta da partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$ é calculado como o valor médio do coeficiente de silhueta para todos os registros e todos os grupos:

$$SW(\mathbb{C}) = \frac{1}{KN} \sum_{i=1}^K \sum_{t=1}^N sw_i(t) \quad (6.41)$$

O índice proposto por Xie e Beni [570] para avaliação de agrupamentos *fuzzy* é uma medida de minimização que avalia a razão entre a variância intragrupos e a menor separação entre os grupos. O índice XB é similar ao PI e é calculado como:

$$XB(\mathbb{C}) = \frac{1}{N} \frac{\sum_{i=1}^K \sum_{t=1}^N u_i(\mathbf{x}(t))^2 \|\mathbf{x}(t) - \boldsymbol{\omega}_i\|^2}{D_{\min}} = \frac{1}{N} \frac{tr(\boldsymbol{\Sigma}_W)}{D_{\min}} \quad (6.42)$$

onde $D_{\min} = \min_{i,j=1,\dots,K, i \neq j} (\|\boldsymbol{\omega}_i - \boldsymbol{\omega}_j\|^2)$.

As medidas relativas são geralmente aplicadas para a avaliação do número de grupos em algoritmos de partição. No caso do algoritmo k-médias e suas variantes, a questão da inicialização aleatória deve ser levada em consideração e uma avaliação estatística em diferentes execuções com o mesmo número de grupos é a abordagem mais indicada para uma avaliação isenta da influência da inicialização.

A Figura 6.13 apresenta *boxplot* dos resultados das medidas de validação aplicadas sobre os resultados de 10 execuções do algoritmo k-médias no conjunto de dados quatro grupos (cf. Figura 6.7). A linha azul representa a média das medidas em todas as simulações para cada valor de K . Neste exemplo, todas as medidas indicam o número de grupos $K = 4$, que é o valor com menor dispersão de resultados.

Os resultados apresentados na Figura 6.13 mostram o desempenho das medidas de avaliação num conjunto de dados com $p = 2$ variáveis e com uma estrutura de grupos muito bem definida (cf. Figura 6.7). Na prática, as medidas de validação podem apresentar resultados conflitantes.

A Figura 6.14 apresenta os resultados de 100 simulações das medidas de validação no conjunto de dados de Diabetes das índias Pima, que tem $p = 8$ variáveis. Nesse caso, as medidas CH e XB apontam uma estrutura com $K = 3$ grupos por apresentar os melhores valores dos índices e a menor dispersão. Os resultados do índice PBM e SW não são conclusivos com relação ao número de grupos, mas o agrupamento com $K = 3$ grupos também gera a menor dispersão dessas medidas.

Observe que o conjunto de dados de Diabetes das índias é um problema de classificação com $m = 2$ classes, mas é desbalanceado com 65% dos registros numa das classes. Este exemplo mostra que nem sempre a estrutura de grupos presente nos dados corresponde às classes do problema.

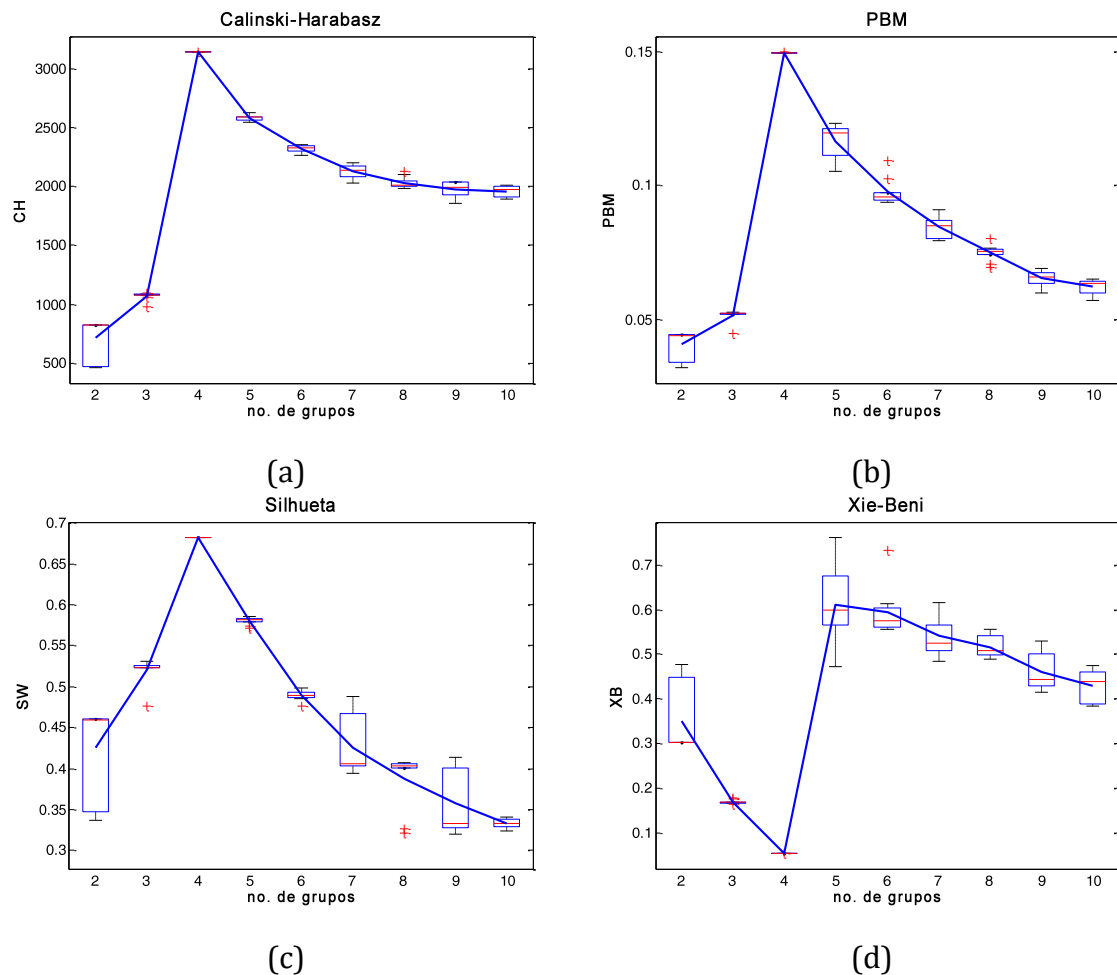


Figura 6.13: Resultados das medidas de validação: (a) CH; (b) PBM; (c) SW; (d) XB.

Mesmo um algoritmo simples como o k-médias acarreta num alto custo computacional quando executado com diferentes inicializações e para vários valores do parâmetro K , o que limita a análise em problemas de grande dimensionalidade. Entretanto, o algoritmo k-médias pode ser executado em paralelo [58][601] para viabilizar a busca do melhor particionamento utilizando medidas de validação. Modenesi e colaboradores [395]-[397] apresentam uma abordagem que integra a versão *fuzzy* do algoritmo k-médias (cf. Capítulo 8) com o cálculo do índice PBM de avaliação de grupos em processamento paralelo.

Um estudo abrangente sobre medidas relativas de validação de agrupamentos foi apresentado recentemente por Vendramin, Campello e Hruschka [538]. No artigo, os autores analisam e comparam 40 índices de validação em 1.080 conjuntos de dados diferentes. O artigo apresenta a descrição das medidas de validação acompanhada de análises de complexidade assintótica. A análise experimental é feita em duas partes: na

primeira, é reproduzido o mesmo procedimento empregado por Milligan e Cooper [388]; na segunda, a análise é estendida com conjuntos de dados maiores, com maior diversidade de número de grupos e de variáveis. Os resultados mostram que os índices CH, PBM e SW são os mais efetivos para encontrar o número de grupos presentes num conjunto de dados.

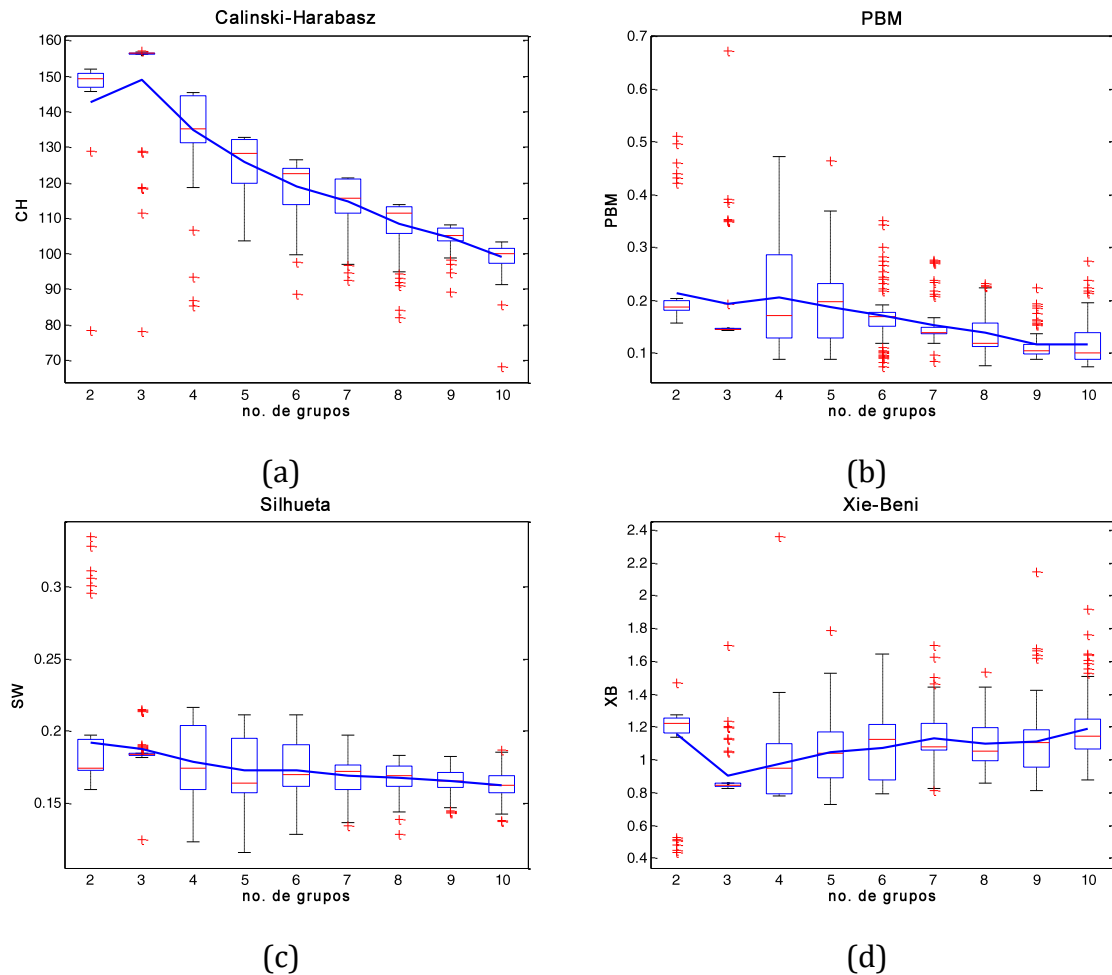


Figura 6.14: Resultados das medidas de validação: (a) CH; (b) PBM; (c) SW; (d) XB.

A validação de agrupamentos é um assunto recorrente na literatura e diversas medidas são encontradas na literatura [592][599]. Em geral, as medidas são avaliadas em conjuntos de dados sintéticos e de baixa dimensionalidade. A própria característica do problema de análise de agrupamentos dificulta a definição de uma média absoluta para avaliar a melhor solução.

6.6 Aplicações

A análise de agrupamentos tem aplicações em diversas áreas. Nesta seção, apresentamos duas aplicações em problemas reais em trabalhos de pesquisa realizados na Copppe/UFRJ [32][468].

6.6.1 Segmentação de imagens de satélite

Segmentação de uma imagem é a partição da imagem em regiões representadas por conjuntos de *pixels*. Essencialmente, a segmentação de imagens é uma aplicação de análise de agrupamentos na qual os registros são os *pixels* e as variáveis são características extraídas destes.

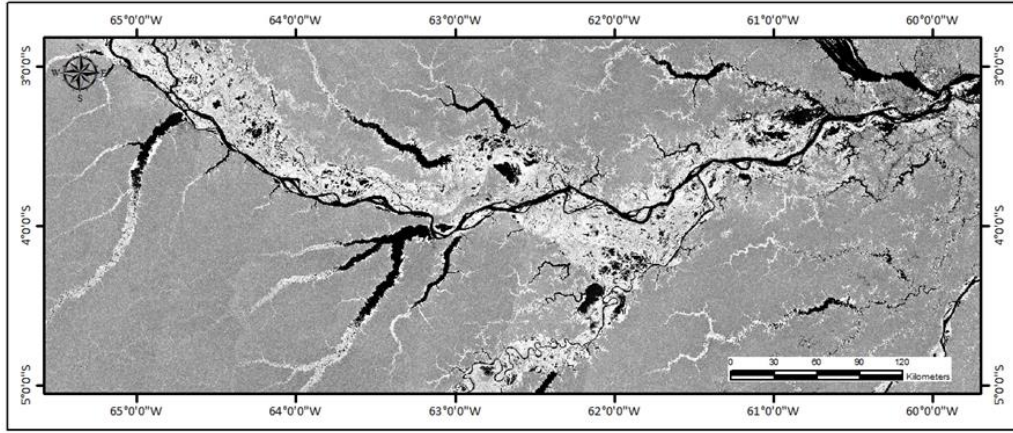
Nesta seção, apresentamos uma aplicação de segmentação de imagens para classificação das zonas inundáveis na Amazônia a partir de imagens de satélite obtidas por radares de abertura sintética (SAR, na sigla em inglês). As imagens SAR são úteis para monitorar florestas tropicais úmidas, pois são imunes à cobertura de nuvens. A interpretação de imagens SAR permite a discriminação de áreas de cobertura vegetal, florestas inundadas e corpos d'água.

Beisl [32] utilizou imagens SAR da região Amazônica registradas pelo satélite JERS-1 e obtidas no projeto de Mapeamento Global de Florestas Tropicais (GRFM, na sigla em inglês) em duas épocas distintas: a época de cheia, captadas em outubro de 1995 (Figura 6.15a), e a época de seca, em maio de 1996 (Figura 6.15b). Uma parte do trabalho foi feita para a determinação do traçado do poliduto Coari-Manaus, da Petrobras.

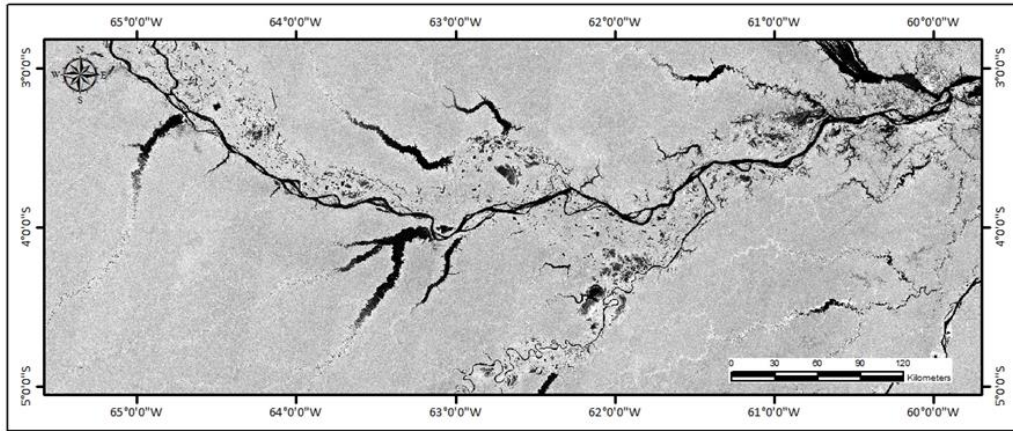
Beisl realizou o mapeamento das mudanças em toda a extensão da bacia do Solimões em função das variações no ciclo hidrológico do rio. O estudo relacionou os padrões de mudança com as características do ambiente, que haviam sido identificadas em estudos anteriores, para obter um mapa de sensibilidade da região ao derramamento de óleo.

Após o pré-processamento, os *frames* foram combinados para produzir as imagens mostradas na Figura 6.15. A imagem cobre boa parte da bacia do Solimões, com cerca de 600 km de extensão, onde Manaus aparece no alto à direita das imagens. As regiões mais escuras indicam a reflexão total do sinal do radar e indicam corpos d'água. As regiões de cinza mais escuro indicam o espalhamento do sinal pela copa das árvores e indicam regiões de floresta. As regiões de cinza mais claro indicam regiões de dupla

reflexão, em que o sinal reflete na água, mas parte do sinal retorna após refletir novamente em algum obstáculo, como árvores, o que caracteriza floresta inundada.



(a)



(b)

Figura 6.15: Imagens SAR da Amazônia: (a) em época de cheia (outubro de 1995); (b) em época de seca (maio de 1996). Fonte: [32].

Em imagens SAR, o conteúdo é apresentado em um canal e a imagem é representada pelo conjunto de dados $Z = \{z(t) \in \mathcal{R}, t = 1, \dots, N\}$, onde $z(t)$ é o valor do *pixel*. Beisl [32] utilizou uma transformação do conjunto de dados Z pela função de semi vario-grama, que relaciona um *pixel* a seus vizinhos pela relação:

$$x_h(t) = \frac{1}{2n} \sum_{i=1}^n (v_h(t, i) - z(t))^2 \quad (6.43)$$

onde h é um parâmetro que regula a extensão da vizinhança e n , o número de *pixels* de vizinhança e $v_h(t, i) = h(t, i)$ uma função de vizinhança.

A função de vizinhança é calculada num círculo de raio h centrado no *pixel* correspondente, como mostra a Figura 6.16 para $h = 6 \text{ pixels}$.

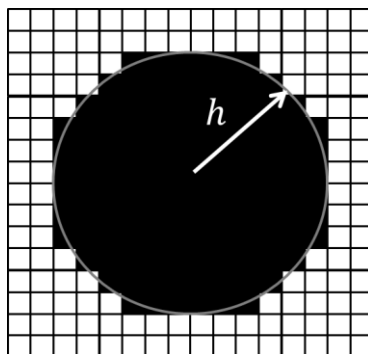


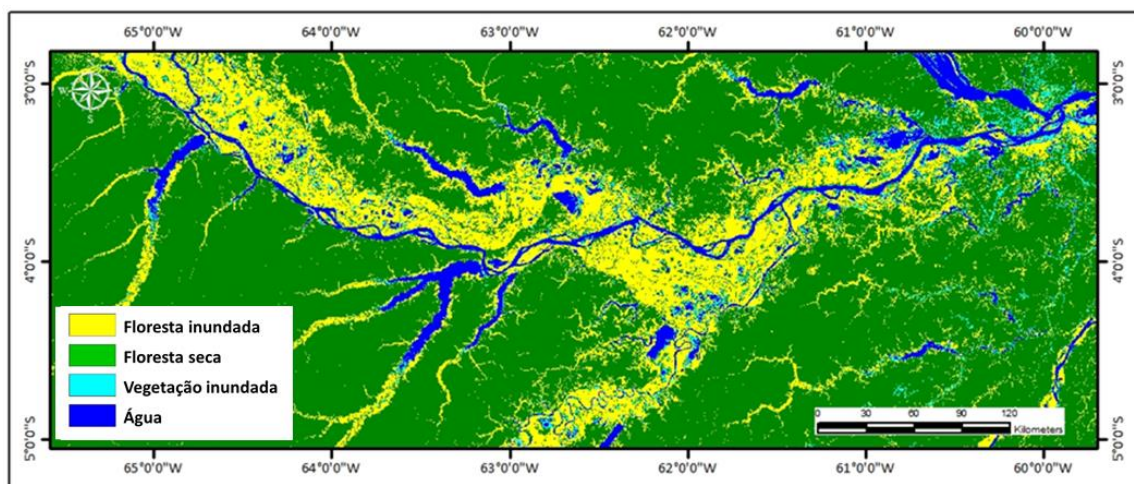
Figura 6.16: Função de vizinhança com $h = 6$.

As imagens foram segmentadas pelo algoritmo textural não supervisionado por semi variograma (USTC),* que utiliza a função semi variograma como um descritor de textura [390][391]. No método USTC, o agrupamento é realizado pelo algoritmo ISODATA [27] aplicado ao conjunto $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, em que o vetor de variáveis tem dimensão $h + 2$ e as componentes são calculados pela função de semi variograma (6.43) como $\mathbf{x}(t) = [z(t), x_1(t), \dots, x_h(t), \sigma_h^2(t)]$, incluindo o valor do pixel central $z(t)$ e $\sigma_h^2(t)$ que é a variância calculada na região da função de vizinhança $h(t, i), i = 1 \dots n$.

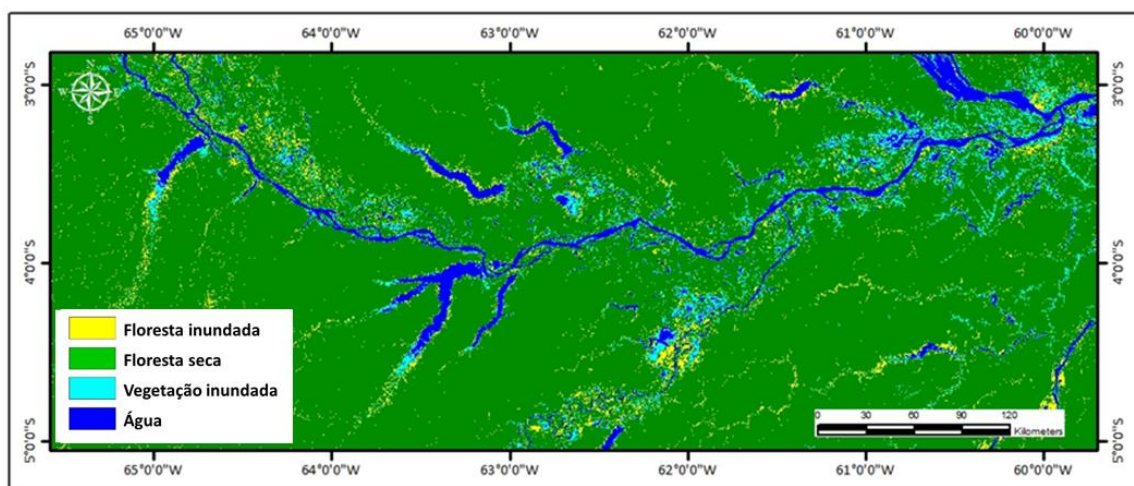
O resultado obtido pelo algoritmo ISODATA é classificado por um especialista e os grupos são definidos para representar um significado temático. Beisl [32] utilizou quatro classes: água, vegetação inundada e floresta seca ou inundada. O resultado final é mostrado na Figura 6.17, na qual a região em verde representa floresta seca, a região em azul representa corpos d'água, a região em amarelo representa floresta inundada e a região em azul-claro, vegetação inundada.

Beisl utiliza o resultado da classificação de imagens obtida pelo método USTC para produzir mapas de sensibilidade ao derramamento de óleo [32]. O trabalho utiliza sistemas *fuzzy* para o cálculo do índice e os resultados são apresentados no Capítulo 8.

* *Unsupervised Semivariogram Textural Classifier*.



(a)



(b)

Figura 6.17: Resultados do USTC: (a) em época de cheia; (b) em época de seca. Fonte: [32].

6.6.2 Agrupamento de documentos

Estima-se que 85% das informações estão armazenadas na forma não estruturada, o que inclui imagens, gravações e, principalmente, texto. A mineração de textos utiliza técnicas de recuperação de informações, processamento de linguagem natural e algoritmos de inteligência computacional para a extração de conhecimento de coleções de documentos. A análise de agrupamentos de uma coleção de documentos permite a determinação dos principais assuntos da coleção e tem diversas aplicações, tendo em vista que a maior parte dos documentos não é categorizada.

A abordagem mais comum para a mineração de textos é aplicar uma etapa de pré-processamento para transformar a coleção de documentos numa tabela de dados, de forma que os algoritmos de mineração de dados possam ser aplicados [43][178][502]. O pré-processamento básico inclui a retirada de palavras irrelevantes (*stopwords*) e a redução ao termo raiz (*stemming*). Outras etapas mais elaboradas podem incluir o tratamento de polissemia (palavras diferentes utilizadas com o mesmo significado) e de homonímia (palavras iguais com significados diferentes), além da identificação de sintagmas nominais [437] e outras técnicas de processamento de linguagem natural [294].

Após o pré-processamento, cada documento é representado como um vetor em que os componentes representam os termos (palavras após o pré-processamento) presentes na coleção. Essa representação é chamada de modelo de espaço vetorial e é muito utilizada na área de recuperação de informações [43][467]. A coleção de documentos é, assim, transformada no conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, onde N é o número total de documentos e cada documento é representado pelo vetor $\mathbf{x}(t) \in \mathcal{R}^p$, onde p é o número total de termos da coleção.

As coordenadas associadas a cada documento no modelo de espaço vetorial podem ser calculadas pela frequência do termo no documento. Em geral, utiliza-se o valor chamado *tf-idf*, em que a frequência do termo no documento é ponderada pelo logaritmo do inverso da frequência do termo na coleção, o que evita que palavras muito citadas em toda a coleção tenham peso muito alto. Cada coordenada é calculada como:

$$x_i(t) = f_i(t) \log \frac{N}{N_i}, \quad i = 1, \dots, p \quad (6.44)$$

onde $f_i(t)$ é a frequência do termo i no documento t e N_i é o total de ocorrências do termo i em toda a coleção.

A ponderação pelo logaritmo do inverso da frequência reduz a importância de termos que aparecem em muitos documentos da coleção e, portanto, são menos importantes para diferenciar os documentos entre si. Há algumas variantes¹³⁹ ao *tf-idf* que podem apresentar resultados em aplicações específicas [372].

A representação pelo modelo de espaço vetorial gera um conjunto de dados de altíssima dimensionalidade. Dependendo do assunto, uma coleção pode conter milhares de termos, o que pode dificultar o cálculo de similaridade entre documentos. Há algumas abordagens de redução da dimensionalidade, das quais a mais conhecida é o índice de

semântica latente (LSI)* [43], cujo cálculo é semelhante à análise de componentes principais (ACP), apresentada na seção 2.4.1.

A aplicação de algoritmos de detecção de comunidades para o agrupamento de coleções de documentos foi estudada por Santos [468][469]. Nesta seção, reproduzimos os resultados obtidos no conjunto de resumos das teses e dissertações do Programa de Engenharia Civil¹⁴⁰ (PEC) da Coppe/UFRJ, publicados entre janeiro de 2004 e dezembro de 2008, totalizando 585 resumos. Após o pré-processamento, a coleção foi representada por um conjunto de 265 termos [468].

A coleção de documentos foi transformada num grafo para a aplicação do algoritmo de maximização da modularidade de Newman e Girvan [413]. A similaridade entre os registros foi utilizada para a definição da matriz de adjacências $\mathbf{A} \in \mathcal{R}^{N \times N}$, cujos elementos foram calculados como:

$$a_{ij} = \begin{cases} 1, & \text{se } i \neq j \text{ e } \text{sim}(\mathbf{x}(i), \mathbf{x}(j)) > \varepsilon \\ 0, & \text{caso contrário} \end{cases} \quad (6.45)$$

onde a função de similaridade $\text{sim}(\mathbf{x}(i), \mathbf{x}(j))$ é calculada pelo cosseno (cf. equação (2.13)) e $\varepsilon \in [0,1]$ é um parâmetro que determina um limite de similaridade para dois registros serem considerados próximos.

Cada valor do parâmetro ε na equação (6.45) define um grafo diferente que resulta num número diferente de comunidades [413]. A Tabela 6.2 mostra os resultados de modularidade (Q) e número de grupos (K) para diversos valores de ε na definição do grafo [468]. O resultado com $\varepsilon = 0.20$ foi escolhido porque o número de grupos gerados é coerente com a estrutura de áreas de pesquisa do PEC. O algoritmo foi aprimorado por uma etapa de refinamento que, a cada nova divisão, verifica se o resultado da modularidade pode ser melhorado alterando-se a alocação de cada documento a outro grupo [541]. Com essa etapa de refinamento, Santos encontrou sete grupos com $\varepsilon = 0.20$ com modularidade $Q = 0.5334$ [468].

O resultado do agrupamento é mostrado na Figura 6.18, gerada com o *software* livre Cytoscape.¹⁴¹ A Figura 6.18 foi gerada de forma que os vértices similares do grafo fiquem mais próximos, representando a coesão dos grupos.

* *Latent Semantic Index.*

Tabela 6.2: Modularidade para vários valores de ε . Fonte: [468]

ε	Q	K
0.10	0.3232	4
0.15	0.4236	5
0.20	0.4998	6
0.25	0.5468	9
0.30	0.6588	12
0.35	0.7167	13

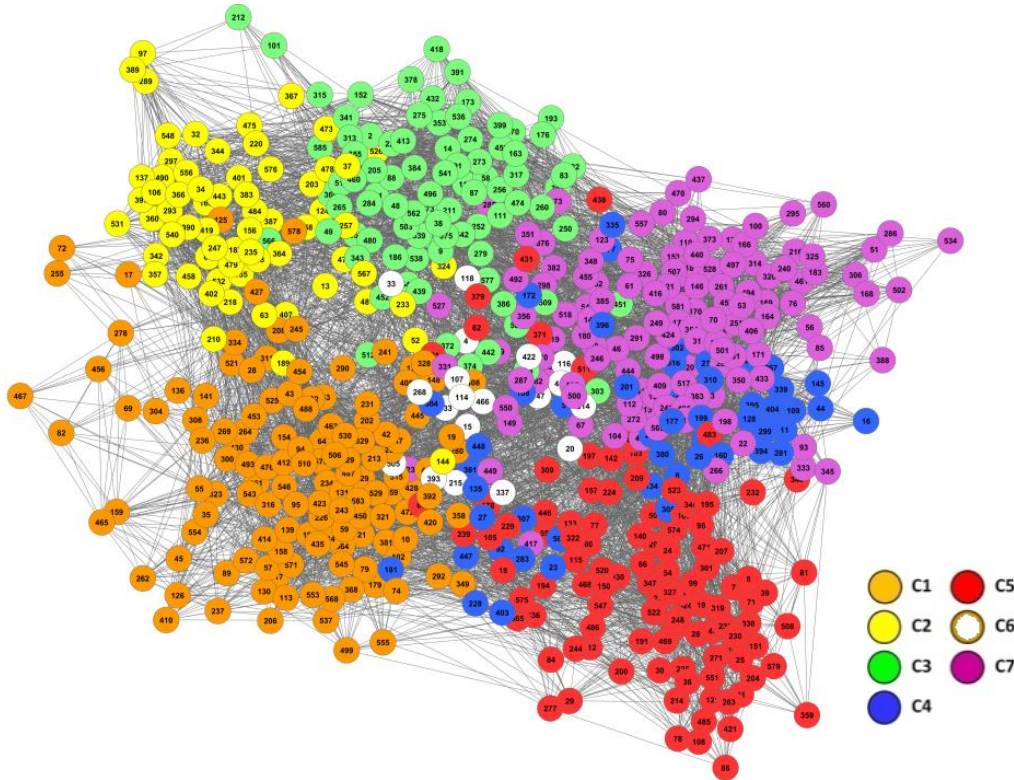


Figura 6.18: Rede de documentos resultante da análise de agrupamentos. Fonte: [468][469].

Uma interpretação do resultado do agrupamento pode ser obtida observando-se os termos mais importantes em cada grupo. A seleção de termos é um caso particular do problema de seleção de variáveis [350][351]. Santos [468] utilizou uma abordagem simples, na qual a relevância de cada termo para cada grupo C_i é calculada como:

$$rel(i, j) = \sum_{t=1}^N x_j(t) u_i(\mathbf{x}(t)) \quad (6.46)$$

onde $u_i(\mathbf{x}(t))$ é a função de pertinência do documento t ao grupo $\mathcal{C}_i$.

Os 10 termos de maior relevância para cada grupo segundo o índice (6.46) são mostrados na Tabela 6.3 [468], na qual os grupos foram interpretados em relação às áreas de pesquisa do PEC. Observa-se que o agrupamento recuperou os assuntos das principais áreas de pesquisa do Programa. A área de Petróleo e Gás foi dividida em dois agrupamentos, que representam as duas linhas de pesquisa mais importantes da área: Geoquímica e Geofísica.

Tabela 6.3: Palavras de cada grupo. Fonte: [468]

Grupo	Termos Relevantes	Área de Pesquisa
$\mathcal{C}_1$	elemento, <i>riser</i> , estaca, linha, estrutura, finito, duto, acoplamento, método, contorno	Mecânica Computacional
$\mathcal{C}_2$	concreto, fibra, reforço, viga, agregado, misto, pavimento, resistência, armado, liga	Estrutura e Materiais
$\mathcal{C}_3$	aterro, resíduo, solo, ensaio, contaminar, sólido, geotecnia, rejeitar, argila, liga	Geotecnia / Meio Ambiente
$\mathcal{C}_4$	óleo, geoquímica, migrar, biomarcador, hidrocarboneto, maturar, biodegradável, bacia, amostra, sísmica	Petróleo e Gás (Geoquímica)
$\mathcal{C}_5$	classificação, redes, virtual, texto, dados, <i>fuzzy</i> , mineração, algoritmo, regra, agrupamento	Sistemas Computacionais
$\mathcal{C}_6$	reservatório, poço, fluxo, simulação, petróleo, fluido, produção, estimativa, otimização, rocha	Petróleo e Gás (Geofísica)
$\mathcal{C}_7$	bacia, hídrico, água, recursos, regra, gestão, cobrança, urbano, modelo, cheia	Recursos Hídricos / Meio Ambiente

6.7 Comentários Finais

O problema de análise de agrupamentos é extenso e alguns livros tratam exclusivamente do assunto [194][573]. O problema é mal definido, o que dificulta a formulação de estatísticas de validação. Há diversas abordagens distintas, sendo muito difícil uma cobertura completa do assunto.

Os tópicos incluídos neste capítulo foram escolhidos para servir como introdução ao tema e aos assuntos que serão tratados nos próximos capítulos. O algoritmo k-médias e os algoritmos hierárquicos aglomerativos apresentados, respectivamente, nas seções

6.2 e 6.3, são as abordagens clássicas, presentes em praticamente todos os *softwares* e bibliotecas. Os algoritmos baseados em grafos, apresentados na seção 6.4, têm sido utilizados em aplicações de mineração de textos e processamento de imagens.

A avaliação de agrupamentos depende muito do contexto e, quando disponível, a avaliação de grupos como classes deve ser feita com cuidado, pois nem sempre os dois conceitos são equivalentes. A relação entre classes e grupos fez surgir métodos chamados de classificação semi supervisionada [8][269][605], em que o conhecimento da classe de alguns registros é utilizado para guiar o algoritmo de agrupamento. Esses métodos têm aplicações em mineração de textos e bioinformática, onde um pequeno número de registros anotados é utilizado para a classificação de um conjunto maior não anotado.

As aplicações de mineração de textos e bioinformática trouxeram novos desafios para os algoritmos de análise de agrupamentos, principalmente devido ao problema da dimensionalidade. Nesse tipo de aplicação, os registros são descritos por vetores de grande dimensão, o que dificulta a análise de similaridade entre os registros, fundamental para o agrupamento. No estudo de caso apresentado na seção 6.6.2, a função cosseno é utilizada como métrica de similaridade e um algoritmo de detecção de comunidades em redes complexas realiza o agrupamento dos documentos. A combinação de técnicas de mineração de dados e de textos com análise de redes complexas também tem sido aplicada para a análise de redes sociais [4][376].

Este capítulo representa a transição entre os métodos clássicos e os métodos mais avançados, que serão apresentados nos próximos capítulos.

7 MODELOS LINEARES LOCAIS

Os modelos preditivos lineares foram discutidos nos capítulos anteriores com a apresentação de métodos estatísticos clássicos para regressão linear e classificação. Este capítulo discute o problema de aproximação de funções e através da apresentação de dois modelos de inteligência computacional: funções de base radial (RBF)* e máquinas de vetor de suporte (SVM).† Esses modelos aproximam uma função qualquer por uma combinação linear de funções de base. Em geral, utiliza-se funções de base chamadas de suporte compacto, que valem zero em todo domínio menos na vizinhança do ponto central e, por isso, podem ser chamadas de funções locais.

O modelo RBF é um modelo de aproximação por funções locais utilizado em diferentes aplicações como computação gráfica e simulação numérica, além de estatística e aprendizado de máquinas. Em geral, utiliza-se uma função gaussiana como função de base radial e a função aproximada é uma combinação linear de funções gaussianas. Este tipo de modelo funciona bem em aplicações com pequeno número de variáveis e grande número de registros. O ajuste de parâmetros é realizado de forma eficiente pela combinação de um algoritmo de agrupamento e o ajuste de um modelo linear da dimensão do número de RBFs.

O modelo SVM utiliza funções de núcleo como funções de base, o que faz com que a formulação do modelo seja feita em relação aos registros do conjunto de treinamento

* *Radial Basis Function.*

† *Support Vector Machine.*

e não diretamente como funções de base definidas explicitamente no espaço de variáveis de entrada. O número de parâmetros do modelo SVM é igual ao número de registros e o ajuste é realizado pela **teoria de aprendizado estatístico**. O resultado é um modelo adequado a problemas com grande número de variáveis e número de registros não muito grande. Em problemas separáveis, esta abordagem encontra a estrutura ótima do modelo. Em problemas não separáveis, há um parâmetro para ajustar a estrutura do modelo, como uma forma de regularização.

Modelos RBF e SVM são comumente encontrados na literatura de redes neurais. Entretanto, têm mais características em comum entre si que com outros modelos de redes neurais. O ajuste de parâmetros dos modelos RBF e SVM resultam em problemas de otimização quadráticos, que são convexos e possuem solução única. Os modelos de redes neurais, que serão discutidos no Capítulo 9, são exemplos de representação distribuída, em que as funções de base não têm suporte compacto e resultam em problemas de otimização não convexos. Os sistemas *fuzzy*, que serão discutidos no próximo capítulo, são modelos RBF que permitem a representação de conceitos como funções de base e o modelo relaciona estes conceitos.

A próxima seção discute o problema de ajuste de parâmetros como um problema de otimização, revisitando os modelos lineares apresentados nos capítulos 3 e 5. A seção 7.2 introduz o problema de aproximação de funções, os modelos de aproximação por funções de base radial (RBF) e seu algoritmo de ajuste de parâmetros. A seção 7.3 descreve o modelo de máquinas de vetor de suporte (SVM) a partir de uma introdução às funções de núcleo e à abordagem de minimização de risco estrutural que leva à formulação do problema de otimização quadrática para o ajuste de parâmetros do modelo. A seção 7.4 apresenta a aplicação dos modelos nos testes benchmark e uma proposta para determinação do número ótimo de funções de base no modelo RBF baseada no particionamento de grafos.

7.1 Aprendizado de Máquinas

Pedro Domingos [137] discute o aprendizado por máquinas como o resultado de três elementos: a representação do modelo, uma função de avaliação e um algoritmo de otimização para solução do problema de ajuste de parâmetros. A definição de Domingos é similar às etapas enunciadas por Eyckhoff em 1974 [172] para identificação de sistemas, apresentadas na seção 3.2:

1. Escolha do **tipo do modelo** pela definição de um conjunto de funções $\mathcal{F}$ que pode conter um elemento capaz de ajustar o conjunto de treinamento;
2. Ajuste ou seleção da **estrutura do modelo** pela definição de um conjunto de restrições para limitar o espaço de soluções a um subconjunto $\mathcal{F}^* \subset \mathcal{F}$;
3. Definição de uma **função de avaliação** e formulação de um **problema de otimização** cuja solução $f^* \in \mathcal{F}^*$ é a instância que determina o modelo ajustado ao conjunto de treinamento;
4. Utilização de um algoritmo de otimização para solução do problema de otimização formulado em (3) para o **ajuste de parâmetros** do modelo aos dados do conjunto de treinamento.

A representação do modelo, segundo Domingos [137], corresponde à escolha do tipo e estrutura do modelo na definição de Eyckhoff [172].

No aprendizado de máquinas, os modelos são definidos como funções matemáticas (ou “máquinas”) e o aprendizado é realizado pela solução de um problema de otimização, para o ajuste de parâmetros definido como:

$$\begin{aligned} \min_{\theta} \quad & l(\theta) \\ \text{subj. } & r(\theta) \leq 0 \end{aligned} \tag{7.1}$$

onde o vetor $\theta \in \mathcal{R}^n$ representa as variáveis do problema que, no caso, são os **parâmetros do modelo**; $l: \mathcal{R}^n \rightarrow \mathcal{R}$ é a **função de custo** ou **função de avaliação**; e a função vetorial $r(\theta)$ define o conjunto de **restrições**. A função de avaliação é uma medida da qualidade do modelo [98] e as restrições definem a **região viável**, que contém as possíveis soluções do problema.

A forma da função de avaliação depende das características do problema. Quando o problema define uma região viável convexa,* a solução é única e representa o **mínimo global**. Se o problema define uma região viável não convexa, pode ocorrer algum **mínimo local**, e a solução do problema de otimização é mais difícil. A Figura 7.1 apresenta exemplos de funções convexas e não convexas com mínimos global θ^* e local θ' .

O problema de aproximação de funções a partir de um conjunto de dados é mal formulado, uma vez que não há uma solução única e diversas funções podem aproximar um

* Uma região é convexa quando qualquer reta a intercepta sua fronteira em apenas dois pontos.

conjunto de dados qualquer. Desta forma, o ajuste de parâmetros do modelo depende da escolha do tipo e estrutura (representação) do modelo. Diferentes representações de modelos foram apresentadas nos Capítulos 3 e 5 para modelos lineares de regressão e classificação.

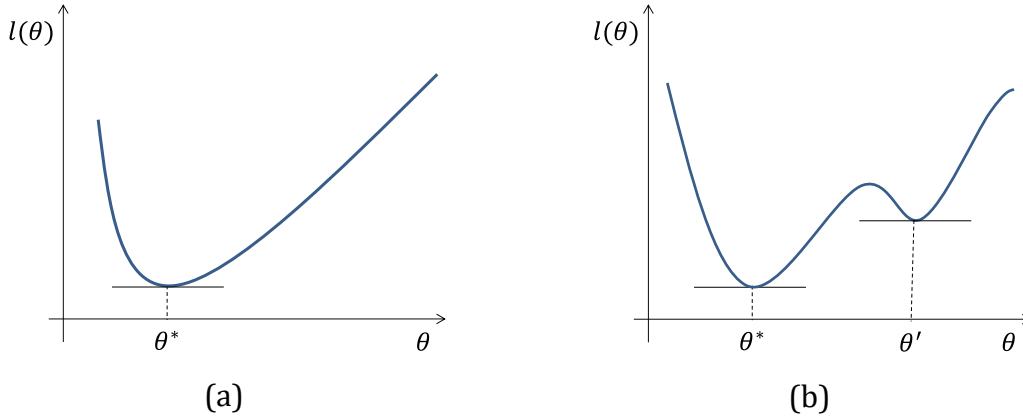


Figura 7.1: Exemplos de função: (a) convexa; (b) não convexa.

Na regressão linear, a estrutura do modelo é definida por seus regressores. No caso do modelo linear simples, a estrutura é fixa e o modelo é uma combinação linear das variáveis de entrada. No modelo linear polinomial, a estrutura do modelo é definida pelo grau do polinômio. Nesse caso, o comportamento do modelo é não linear, mas o modelo continua sendo linear nos parâmetros. O ajuste dos parâmetros do modelo é feito pela minimização do erro médio quadrático (EMQ) sem restrições (cf. equação (3.4)):

$$l(\boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t))^2 \quad (7.2)$$

onde $\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta})$ é o resultado do modelo para cada registro do conjunto de treinamento.

O método dos mínimos quadrados (MMQ) é uma solução eficiente do problema de ajuste de parâmetros quando a função do modelo é linear nos parâmetros, podendo ser escrita como uma combinação linear dos regressores:

$$\hat{f}(\mathbf{x}(t), \boldsymbol{\theta}) = \hat{\mathbf{x}}(t) \boldsymbol{\theta}^T \quad (7.3)$$

onde $\hat{\mathbf{x}}(t)$ são os regressores do modelo e $\boldsymbol{\theta}$, o vetor de parâmetros.

Em muitas aplicações, a função aproximada pelo modelo deve ser suave. Funções suaves tendem a ter melhor capacidade de generalização e evitam o *overfitting*. O método

de regularização de Tikhonov introduz uma restrição sobre a norma L^2 dos parâmetros do modelo, na forma de uma penalidade, que suaviza a função do modelo. A função de avaliação é o erro médio quadrático regularizado (EMQR) (cf. equação (3.14)):

$$l(\mu, \boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^N (y(t) - \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}))^2 + \frac{\mu}{2} \|\boldsymbol{\theta}\|^2 \quad (7.4)$$

onde μ é o parâmetro de regularização.

A técnica de regularização tem um papel importante no ajuste de modelos, pois permite que a seleção da estrutura seja incorporada na formulação do problema de otimização e ajustada através de um parâmetro. Este parâmetro regula a suavidade do modelo e, conseqüentemente, sua capacidade de generalização.

No modelo linear de classificação, as classes são representadas por variáveis indicadoras binárias:

$$v(t) = \begin{cases} 1, & \text{se } \mathbf{x}(t) \in C \\ 0, & \text{se } \mathbf{x}(t) \notin C \end{cases} \quad (7.5)$$

O problema de múltiplas classes é tratado como múltiplos problemas de classificação binária. Os parâmetros do modelo de classificação podem ser calculados, para cada classe, pela minimização do EMQ substituindo de $y(t)$ por $v(t)$ na equação (7.2). A função do modelo é contínua, definida pela função discriminante $\hat{v}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta})$.

No modelo de regressão logística, a função do modelo é uma estimativa da probabilidade *a posteriori* da classe, calculada pela função sigmoide aplicada sobre o modelo de classificação linear (cf. equação (5.23)):

$$\hat{v}(t) = \frac{1}{1 + \exp(-\hat{\mathbf{x}}(t)\boldsymbol{\theta}^T)} \quad (7.6)$$

A função $\hat{v}: \mathcal{R} \rightarrow [0,1]$ é uma estimativa da probabilidade de classificação $P(C|\mathbf{x})$. A decisão de classificação é feita pela regra:

$$\begin{cases} \text{Se } \hat{v}(t) \geq 1 - \hat{v}(t) \rightarrow \mathbf{x}(t) \in \hat{C} \\ \text{Se } \hat{v}(t) < 1 - \hat{v}(t) \rightarrow \mathbf{x}(t) \notin \hat{C} \end{cases} \quad (7.7)$$

onde $\hat{C}$ é a classe predita pelo modelo.

Na regressão logística, o ajuste de parâmetros é calculado pela minimização do negativo do logaritmo da função de verossimilhança (cf. equação (5.28)):

$$l(\theta) = - \sum_{t=1}^N v(t) \log(\hat{v}(t)) + (1 - v(t)) \log(1 - \hat{v}(t)) \quad (7.8)$$

A maximização da função de verossimilhança é uma formulação mais geral que a minimização do EMQ. Apesar do modelo de regressão logística ser linear, o logaritmo da verossimilhança é uma função de avaliação não linear nos parâmetros, de forma que o MMQ não pode ser aplicado e o ajuste de parâmetros é feito pelo método do gradiente, que será apresentado na seção 9.1.2.

Modelos lineares nos parâmetros, ajustados pela minimização do EMQ são interessantes pois a função de avaliação é convexa e a solução do problema é única e representa um **mínimo global**. Os modelos lineares locais, que utilizam as funções de base radial (RBF) e funções de núcleo como representação, também são lineares nos parâmetros e admitem solução única, como apresentado nas próximas seções.

7.2 Modelos de Base Radial (RBF)

O problema de aproximação de funções a partir de um conjunto de pontos é um dos problemas mais estudados na matemática e os primeiros resultados remontam a Euler, no século XVIII [505], para a solução de problemas ligados à geometria da Terra.

A aproximação por **funções de base radial** (RBF) gera funções contínuas, diferenciáveis e são muito utilizadas em diversas aplicações [68][454][579]. Na área de inteligência computacional, os modelos de aproximação por funções de base radial começaram a ser empregados no final da década de 1980 [66][440]. Na década de 1990, os modelos RBF com função gaussiana foram introduzidos no contexto de sistemas *fuzzy* [286][287], como o modelo fuzzy-RBF, apresentado na seção 8.3.4. Em 1995 o modelo SVM, apresentado na seção 7.3.3, obteve o melhor desempenho até então no problema de reconhecimento de dígitos manuscritos.¹⁴²

Os modelos RBF são apresentados pela formalização geral do problema de aproximação de funções. Em seguida, discutimos os principais algoritmos de ajuste de parâmetros de modelos RBF. Os modelos SVM e sua formulação do problema de ajuste de parâmetros são discutidos na seção 7.3.

7.2.1 Aproximação de funções

Na teoria de aproximação de funções, um espaço vetorial de funções $\mathcal{F}$ é formado pelo conjunto de todas as funções contínuas $f: \mathcal{R}^p \rightarrow \mathcal{R}$ que satisfaz as propriedades características de um espaço vetorial. É possível definir uma norma para o espaço de funções $\mathcal{F}$ numa região fechada $\mathcal{X} \subset \mathcal{R}^p$ que contenha a origem através do produto interno:

$$\langle f(\mathbf{x}), g(\mathbf{x}) \rangle = \int_{\mathcal{X}} f(\mathbf{x})g(\mathbf{x})d\mathcal{X} \quad (7.9)$$

onde $\mathbf{x} \in \mathcal{R}^p$, $f, g \in \mathcal{F}$ e a integral $\int_{\mathcal{X}} d\mathcal{X}$ é calculada na região fechada $\mathcal{X} \subset \mathcal{R}^p$. No caso unidimensional ($p = 1$), $\mathcal{X} = [a, b]$ e $\langle f(x), g(x) \rangle = \int_a^b f(x)g(x)dx$.

O produto interno (7.9) define a norma L^2 de $\mathcal{F}$ como:

$$\|f(\mathbf{x})\|^2 = \langle f(\mathbf{x}), f(\mathbf{x}) \rangle = \int_{\mathcal{X}} f(\mathbf{x})f(\mathbf{x})d\mathcal{X} \quad (7.10)$$

Um espaço vetorial é chamado de espaço de Hilbert quando é dotado de uma norma e é completo, de modo que as técnicas de cálculo diferencial podem ser utilizadas nos vetores. O espaço de Hilbert é uma generalização do espaço geométrico $\mathcal{R}^n$ em que um ponto é formado por um vetor descrito por suas coordenadas em relação a uma base vetorial. Espaços de Hilbert foram desenvolvidos no início do século XX e são fundamentais em diversas teorias.¹⁴³

Na teoria de aproximação de funções, uma função $f \in \mathcal{F}$ é descrita por um vetor de coordenadas e uma base vetorial. Um espaço vetorial de funções de dimensão N é gerado pela base $\boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_N(\mathbf{x}))$, onde as componentes $\phi_i(\mathbf{x}), i = 1, \dots, N$ são chamadas de **funções de base**. Qualquer função $f \in \mathcal{F}$ pode ser escrita como combinação linear das funções de base:

$$f(\mathbf{x}) = \sum_{i=1}^N \phi_i(\mathbf{x})f_i \quad (7.11)$$

onde os elementos do vetor $\mathbf{f} = (f_1, \dots, f_N)$ representam as coordenadas que descrevem a função f através da base $\boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_N(\mathbf{x}))$.

Em aplicações de ciência de dados, o vetor de base $\boldsymbol{\phi}(\mathbf{x})$ define o **vetor de características** do modelo $\mathbf{u}(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})$, de forma que o espaço vetorial de funções $\mathcal{F}$ é chamado de **espaço de características** (*feature space*).

A região fechada $\mathcal{X}$ é definida pelo conjunto de registros $\mathcal{X} = \{\mathbf{x}(t) \in \mathcal{R}^p, t = 1 \dots N\}$, considerando que os registros são distintos e contêm a origem. Um conjunto de dados com N registros define o produto interno das funções $f, g \in \mathcal{F}$, analogamente ao caso contínuo (cf. equação (7.9)), como:

$$\langle f(\mathbf{x}), g(\mathbf{x}) \rangle = \sum_{t=1}^N f(\mathbf{x}(t))g(\mathbf{x}(t)) \quad (7.12)$$

A norma L^2 definida pelo produto interno (7.12) é escrita como:

$$\|f(\mathbf{x})\|^2 = \langle f(\mathbf{x}), f(\mathbf{x}) \rangle = \sum_{t=1}^N f(\mathbf{x}(t))^2 \quad (7.13)$$

O conjunto de dados de treinamento $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$ representa uma amostra, ou seja, uma realização do processo, cujos valores da variável de saída foram gerados por uma função específica $f^* \in \mathcal{F}$ desconhecida, como $y(t) = f^*(\mathbf{x}(t))$. A função f^* não é exatamente a função do processo que está sendo modelado, uma vez que a variável de saída pode conter ruído e o efeito de variáveis de entrada não observadas.

Uma base $\boldsymbol{\phi}(\mathbf{x}) = [\phi_1(\mathbf{x}), \dots, \phi_N(\mathbf{x})]$ gera um espaço vetorial de funções $\mathcal{F}$ de dimensão N formado pelo conjunto de todas as funções $f: \mathcal{X} \rightarrow \mathcal{R}$ capazes de reproduzir qualquer realização do conjunto de dados $\mathcal{T}$.

Uma função $f^* \in \mathcal{F}$ desconhecida pode ser aproximada por $\hat{y}(t) = \hat{f}(\mathbf{x}(t))$, cujas coordenadas são escritas na base $\boldsymbol{\phi}(\mathbf{x})$ pelo vetor de parâmetros $\boldsymbol{\theta} = (\theta_1, \dots, \theta_N)$, como:

$$\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}) = \sum_{i=1}^N \phi_i(\mathbf{x}(t))\theta_i = \mathbf{u}(t)\boldsymbol{\theta}^T, \quad t = 1, \dots, N \quad (7.14)$$

A equação (7.14) mostra que o vetor de características do modelo linear forma uma base do espaço vetorial de funções como $\mathbf{u}(t) = (\phi_1(\mathbf{x}(t)), \dots, \phi_N(\mathbf{x}(t)))$ e o vetor de parâmetros é uma aproximação das coordenadas de f^* como $\boldsymbol{\theta} \approx \mathbf{f} = (f_1, \dots, f_N)$.

O ajuste de parâmetros da aproximação $\hat{f}(\mathbf{x}, \boldsymbol{\theta})$, descrita pela equação (7.14), é feito pela definição de uma função de avaliação $l(\boldsymbol{\theta})$ que tem valor mínimo na solução $\boldsymbol{\theta}^*$, ou

seja: $l(\boldsymbol{\theta}^*) \leq l(\boldsymbol{\theta}), \forall \boldsymbol{\theta}$. No método dos mínimos quadráticos (MMQ), a solução do problema é calculada pela minimização da norma L^2 do resíduo ao quadrado, que corresponde à função de erro quadrático:

$$l(\boldsymbol{\theta}) = \|f^*(\mathbf{x}) - \hat{f}(\mathbf{x}, \boldsymbol{\theta})\|^2 = \sum_{t=1}^N \left(y(t) - \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}) \right)^2 \quad (7.15)$$

No MMQ, o conjunto de parâmetros $\boldsymbol{\theta}^*$ que minimiza a função de custo (7.15) é calculado pela solução das equações normais (cf. equação (3.7)):

$$\mathbf{U}^T \mathbf{U} \boldsymbol{\theta}^T = \mathbf{U}^T \mathbf{Y} \quad (7.16)$$

onde $\mathbf{U} = \begin{bmatrix} \phi_1(\mathbf{x}(1)) & \cdots & \phi_N(\mathbf{x}(1)) \\ \vdots & \ddots & \vdots \\ \phi_1(\mathbf{x}(N)) & \cdots & \phi_N(\mathbf{x}(N)) \end{bmatrix}$, $\boldsymbol{\theta}^T = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_N \end{bmatrix}$ e $\mathbf{Y} = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}$.

Observe que cada elemento da matriz $\mathbf{U}^T \mathbf{U}$ corresponde ao produto interno $\langle \phi_i(\mathbf{x}), \phi_j(\mathbf{x}) \rangle$, de acordo com (7.12), de tal forma que as equações normais do MMQ podem ser escritas a partir do produto interno das funções de base como:

$$\begin{bmatrix} \langle \phi_1(\mathbf{x}), \phi_1(\mathbf{x}) \rangle & \cdots & \langle \phi_1(\mathbf{x}), \phi_N(\mathbf{x}) \rangle \\ \vdots & \ddots & \vdots \\ \langle \phi_N(\mathbf{x}), \phi_1(\mathbf{x}) \rangle & \cdots & \langle \phi_N(\mathbf{x}), \phi_N(\mathbf{x}) \rangle \end{bmatrix} \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_N \end{bmatrix} = \begin{bmatrix} \langle \phi_1(\mathbf{x}), \mathbf{Y} \rangle \\ \vdots \\ \langle \phi_N(\mathbf{x}), \mathbf{Y} \rangle \end{bmatrix} \quad (7.17)$$

onde $\langle \phi_i(\mathbf{x}), \phi_j(\mathbf{x}) \rangle = \sum_{t=1}^N \phi_i(\mathbf{x}(t)) \phi_j(\mathbf{x}(t))$.

Supondo que a função $f^*(\mathbf{x})$ é conhecida apenas nos pontos do conjunto $\mathcal{T}$, a determinação exata da função $f^*(\mathbf{x})$ é mal formulada, no sentido que não tem uma solução única. Diferentes bases do espaço vetorial podem gerar diferentes aproximações que são iguais nos registros do conjunto de treinamento, mas que podem ser diferentes para novos registros. A solução calculada pela equação (7.14) é chamada de **interpolação** do conjunto de dados pela base $\boldsymbol{\phi}(\mathbf{x})$.

A solução do problema de aproximação consiste na escolha de funções de base compatíveis com as características do problema e que produza um sistema de equações (7.17) não singular. Essa abordagem de solução é encontrada em diversos métodos de interpolação que utilizam diferentes funções de base [454].

O espaço vetorial de funções $\mathcal{F}$ de dimensão N pode reproduzir “qualquer” realização de dimensão N do processo que gerou a amostra. Entretanto, a reprodução exata de todos os pontos do conjunto de treinamento não é adequada, pois, além de ruído, há o

efeito de variáveis não observadas. O importante é realizar uma boa **aproximação** da função que gerou o conjunto de dados, isto é, $\hat{y}(t) \approx f^*(\mathbf{x}(t))$, evitando o *overfitting*.

Uma base de dimensão $n \ll N$ gera o subespaço vetorial $\mathcal{F}^* \subset \mathcal{F}$, que pode aproximar bem os pontos observados no conjunto de dados e generalizar para novas amostras. A dimensão n determina o número de graus de liberdade do modelo e define a sua estrutura, que está relacionada à capacidade de generalização do modelo. A aproximação é escrita como:

$$\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta}) = \sum_{i=1}^n \phi_i(\mathbf{x}(t))\theta_i = \mathbf{u}(t)\boldsymbol{\theta}^T, \quad t = 1, \dots, N \quad (7.18)$$

Um exemplo desse tipo de aproximação foi discutido na seção 3.1.1 com o modelo linear polinomial de grau r , cuja base $\boldsymbol{\phi}(\mathbf{x}) = [1, \mathbf{x}, \mathbf{x}^2, \dots, \mathbf{x}^r]$ descreve uma função $f \in \mathcal{F}^*$ com um vetor de características de dimensão $n = rp + 1$. O vetor de regressores é o vetor de características do modelo.

A escolha de uma base $\boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_n(\mathbf{x}))$, de dimensão $n \ll N$, não altera o cálculo da norma de $\mathcal{F}^*$, definida pela equação (7.13), que está relacionada ao número de registros do conjunto de dados de treinamento.

A solução do problema de aproximação pelo MMQ é calculada pela solução do sistema linear de equações de dimensão n escrito como:

$$\begin{bmatrix} \langle \phi_1(\mathbf{x}), \phi_1(\mathbf{x}) \rangle & \cdots & \langle \phi_1(\mathbf{x}), \phi_n(\mathbf{x}) \rangle \\ \vdots & \ddots & \vdots \\ \langle \phi_n(\mathbf{x}), \phi_1(\mathbf{x}) \rangle & \cdots & \langle \phi_n(\mathbf{x}), \phi_n(\mathbf{x}) \rangle \end{bmatrix} \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix} = \begin{bmatrix} \langle \phi_1(\mathbf{x}), \mathbf{Y} \rangle \\ \vdots \\ \langle \phi_n(\mathbf{x}), \mathbf{Y} \rangle \end{bmatrix} \quad (7.19)$$

onde $\langle \phi_i(\mathbf{x}), \phi_j(\mathbf{x}) \rangle = \sum_{t=1}^N \phi_i(\mathbf{x}(t))\phi_j(\mathbf{x}(t))$.

O sistema linear (7.19) pode ser escrito na forma das equações normais (7.16), onde a matriz de características $\mathbf{U}$ tem dimensão $N \times n$ e o vetor de parâmetros $\boldsymbol{\theta} \in \mathcal{R}^n$.

A equação (7.19) pode ser vista como uma projeção do espaço original $\mathcal{F}$, de dimensão N , no espaço de menor dimensionalidade $\mathcal{F}^* \subset \mathcal{F}$, como na interpretação geométrica do MMQ, discutida na seção 3.1.3.

As quatro etapas para a construção de um modelo apresentadas na seção 7.1 podem ser interpretadas formalmente no contexto de aproximação de funções. O tipo de modelo é definido por uma função de base para gerar o espaço vetorial de funções $\mathcal{F}$. A estrutura do modelo é definida pela escolha da dimensão $n \ll N$, limitando a solução

ao subespaço vetorial $\mathcal{F}^* \subset \mathcal{F}$. O ajuste de parâmetros é realizado pela definição da função de avaliação $l(\boldsymbol{\theta})$ e, eventualmente, um conjunto de restrições. Finalmente, um algoritmo de otimização é utilizado para encontrar a solução $f^* \in \mathcal{F}^*$.

O modelo RBF é uma aproximação que utiliza funções de base radial, de dimensão $n \ll N$, como descrito a seguir.

7.2.2 Modelos RBF

Em problemas de aproximação de funções, como a solução numérica de equações diferenciais, os valores da função a ser aproximada são calculados numa malha regular em que a borda da região $\mathcal{X}$ é bem definida (cf. Figura 7.2a). Em problemas de ciência de dados, em geral, não há controle sobre a amostra e os dados estão dispostos aleatoriamente dentro da região $\mathcal{X}$, que é definida pelo conjunto de dados, como na Figura 7.2b.

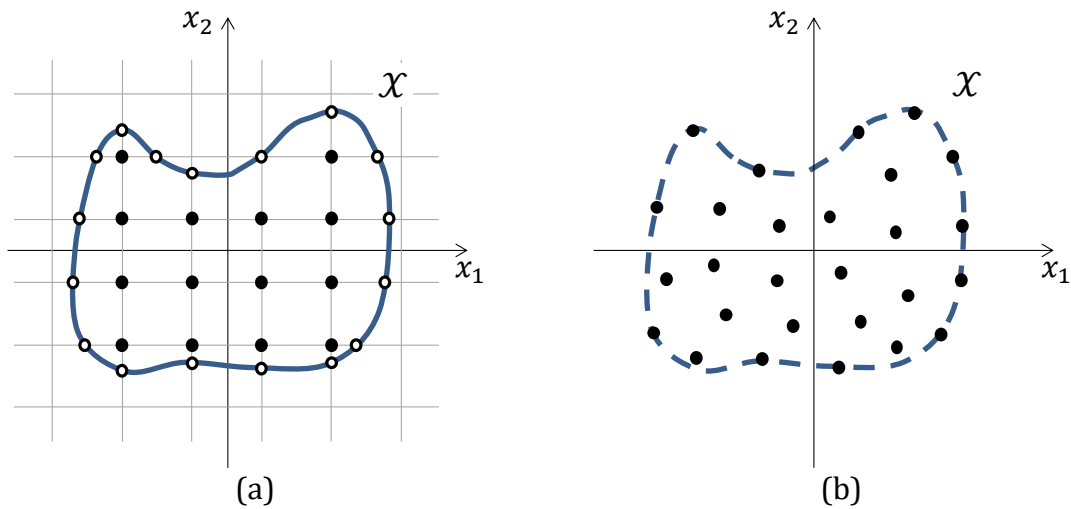


Figura 7.2: Amostragem de pontos numa região $\mathcal{X}$: (a) com malha; (b) sem malha.

As funções de base radial (RBF) são funções simétricas em relação ao centro escritas como $\phi_i(\mathbf{x}) = h(\mathbf{x} - \boldsymbol{\omega}_i)$, onde o vetor $\boldsymbol{\omega}_i$ define as coordenadas do centro. As RBF são frequentemente utilizadas na aproximação de funções sem malha, pois dependem apenas da distância entre o ponto a ser estimado e o centro e geram aproximações de funções suaves de derivada contínua [349].

A matriz de coeficientes $\mathbf{U}^T \mathbf{U}$ na equação (7.19) (ou na equação (7.16)) é não singular para as seguintes funções de base radial (entre outras) [379][440]:

- Função linear:

$$h(\mathbf{x} - \boldsymbol{\omega}) = \|\mathbf{x} - \boldsymbol{\omega}\| \quad (7.20)$$

- Função multiquadrática com parâmetro $\beta > 0$:

$$h(\mathbf{x} - \boldsymbol{\omega}) = (1 + \beta \|\mathbf{x} - \boldsymbol{\omega}\|)^{\frac{1}{2}} \quad (7.21)$$

- Função gaussiana com parâmetro $\sigma > 0$:

$$h(\|\mathbf{x} - \boldsymbol{\omega}\|) = \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\omega}\|^2}{2\sigma^2}\right) \quad (7.22)$$

O modelo RBF é uma aproximação de dimensão n , em que a saída é calculada como:

$$\hat{y}(\mathbf{x}(t)) = \theta_0 + \sum_{i=1}^n h(\mathbf{x}(t) - \boldsymbol{\omega}_i) \theta_i = \mathbf{u}(t) \boldsymbol{\theta}^T \quad (7.23)$$

onde o vetor de características $\mathbf{u}(t) = (1, \phi_1(t), \dots, \phi_n(t))$; $\phi_i(t) = h(\mathbf{x}(t) - \boldsymbol{\omega}_i)$, são as funções de base radial e $\boldsymbol{\theta} = (\theta_0, \dots, \theta_n)$ é o vetor de parâmetros.

Teoricamente, a aproximação RBF é invariante à translação e o parâmetro θ_0 seria desnecessário no caso de a variável de saída ter média nula. Na prática, o parâmetro θ_0 é incluído no modelo para que a variável de saída possa ser utilizada na sua escala original e corrigir distorções em função de distribuições assimétricas, variáveis binárias etc.

A extensão dos modelos RBF para problemas de múltiplas variáveis de saída é direta pela adição de novas variáveis de saída e o respectivo conjunto de parâmetros. O modelo RBF pode ser utilizado em problemas de regressão e de classificação, em que as classes são representadas por variáveis indicadoras e o modelo aproxima as funções discriminantes, como no modelo linear de classificação (cf. seção 5.2).

A maioria dos modelos RBF em inteligência computacional utilizam a função gaussiana (cf. equação (7.22)), que define um **modelo local** ou de suporte compacto, como uma aproximação suave nas vizinhanças do centro. No modelo RBF com funções gaussianas, o valor das funções de base decai exponencialmente a zero para registros afastados do conjunto de treinamento. Em alguns casos, é conveniente utilizar o vetor de características normalizado, que pode realizar estimativas fora da região definida pelo conjunto de treinamento:

$$\hat{u}_i(t) = \frac{u_i(t)}{\sum_{i=1}^n u_i(t)} \quad (7.24)$$

Um modelo RBF realiza um mapeamento não linear das variáveis de entrada no espaço de características. Cada registro de entrada corresponde a um ponto no espaço de características definido pelas coordenadas do vetor de características.

A Figura 7.3 mostra o exemplo do problema da função XOR (disjunção exclusiva) com sua a tabela de valores e o gráfico correspondente. Podemos observar que esse problema é não separável linearmente, uma vez que não há uma função linear capaz de separar as classes, como para as funções AND e OR (cf. Figura 9.3).

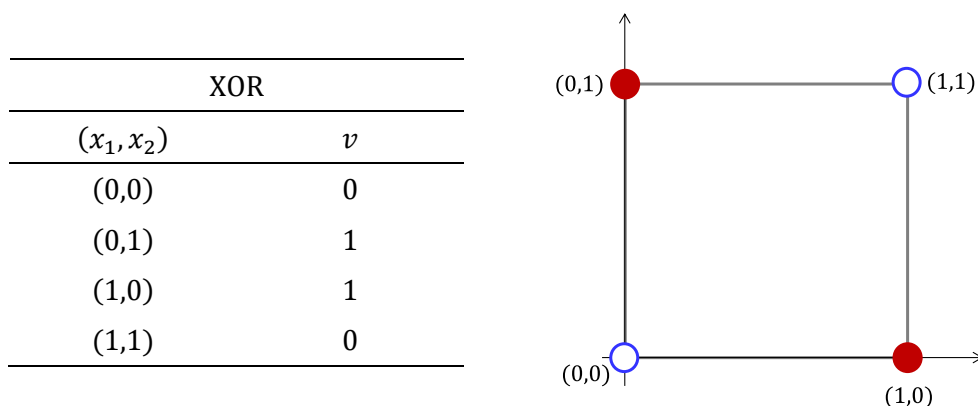


Figura 7.3: Problema XOR.

O problema XOR pode ser resolvido por um modelo RBF com duas unidades RBF gaussianas com centros $\omega_1 = (0,0)$ e $\omega_2 = (1,1)$. Os valores do vetor de características correspondentes aos registros são mostrados na Tabela 7.1.

Tabela 7.1: Valores da função XOR e das unidades RBF

(x_1, x_2)	v	u_1	u_2
(0,0)	0	0.1353	1.0000
(0,1)	1	0.3678	0.3678
(1,0)	1	0.3678	0.3678
(1,1)	0	1.0000	0.1353

O mapeamento dos registros no plano formado pelas unidades RBF é mostrado na Figura 7.4. Observa-se que os registros são linearmente separáveis no espaço transformado gerado pelas RBF (Figura 7.4b).

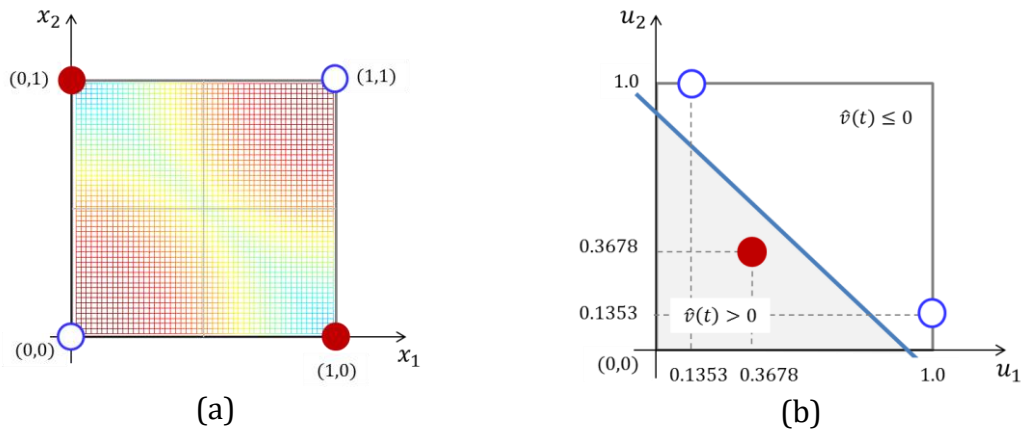


Figura 7.4: O problema XOR: (a) variáveis de entrada; (b) variáveis transformadas.

Os modelos RBF com funções gaussianas podem gerar funções arbitrariamente complexas pela adição de um número suficiente de unidades RBF e pelo ajuste do parâmetro de dispersão σ de cada função gaussiana. A Figura 7.5a mostra o resultado da aproximação por um modelo RBF do conjunto de dados das duas espirais, que é um *benchmark* clássico de classificação não linear com $N = 378$ registros. O resultado mostrado na Figura 7.5b foi obtido com modelo RBF de topologia 2:64:1, com RBF gaussianas com parâmetro $\sigma = 0.1$.

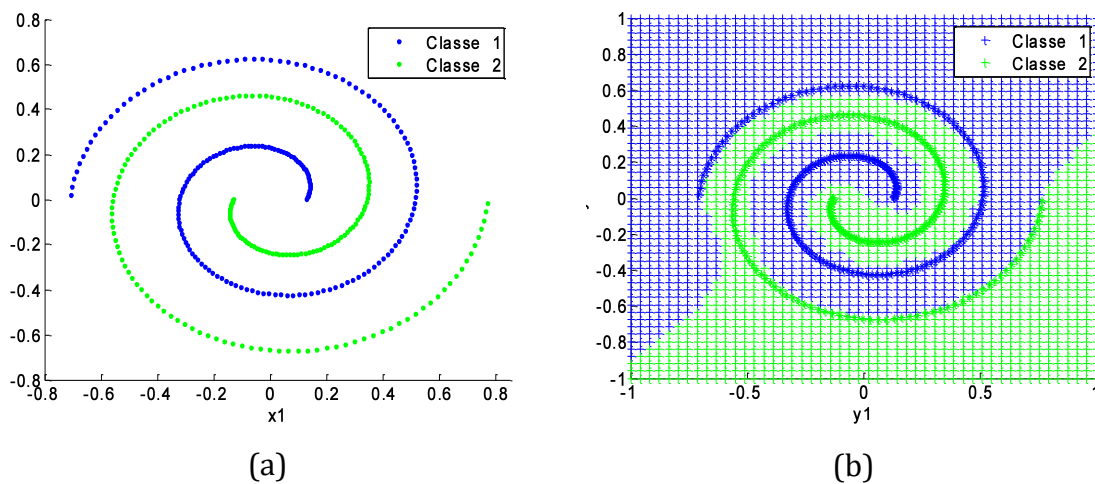


Figura 7.5: O problema Espiral: (a) conjunto de treinamento; (b) superfície de decisão.

Os parâmetros de saída dos modelos RBF são lineares e podem ser ajustados pelo MMQ, como discutido na próxima seção.

7.2.3 Ajuste de parâmetros

A estrutura do modelo RBF é definida pelo número n de funções de base. Os parâmetros do modelo RBF com funções gaussianas são:

- A matriz de centros: $\mathbf{\Omega} = [\mathbf{\omega}_1^T, \dots, \mathbf{\omega}_n^T]$, onde $\mathbf{\omega}_i = (\omega_{i1}, \dots, \omega_{ip})$ é o vetor de coordenadas dos centros das funções gaussianas $i = 1, \dots, n$;
- O vetor de dispersões das funções gaussianas: $\boldsymbol{\sigma} = (\sigma_1, \dots, \sigma_n)$;
- O vetor de parâmetros de saída: $\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_n)$.

A seleção do número de funções de base pode ser tratada como um problema de seleção de características [224][350]. A solução clássica é obtida pelo algoritmo de mínimos quadrados ortogonais (OLS)*. No algoritmo OLS, a cada iteração, um registro de treinamento é selecionado como um novo centro de RBF e os parâmetros são ajustados através da ortogonalização de Gram-Schmidt [206].

Selecionar os centros das RBF como um subconjunto dos registros de treinamento é uma solução interessante, uma vez que o conjunto de registros representa toda a informação disponível para o modelo. Entretanto, a seleção de n centros de um conjunto de N registros envolve a avaliação de $N!/(n!(N-n)!)$ combinações, o que é inviável mesmo para N pequeno. A solução do algoritmo OLS foi aprimorada com a utilização de algoritmos genéticos para otimização da posição dos centros de RBF por Barreto e colaboradores [31].

O processo de seleção da estrutura do modelo por validação cruzada garante a escolha do número de unidades RBF com melhor capacidade de generalização. A minimização do EMQ em relação às coordenadas dos centros e dispersões das RBF é um problema de otimização não linear, que pode ser resolvido pelo método do gradiente.

Uma alternativa ao gradiente para o ajuste das coordenadas da matriz de centros é a aplicação de algum algoritmo de análise de agrupamentos, como o k-médias [401]. O algoritmo de agrupamento posiciona os centros nas regiões com maior densidade de registros, de modo que cada função de base representa um modelo local de uma vizinhança densamente amostrada.

Conhecendo o número e as coordenadas dos centros das RBF, os parâmetros de dispersão podem ser ajustados por diferentes estratégias. O mais simples é utilizar um

* *Orthogonal Least Squares.*

mesmo valor para todas as unidades. Um valor para cada unidade pode ser calculado a partir da distância do centro da unidade até os demais centros como, por exemplo, pela menor distância:

$$\sigma_k = \frac{1}{2} \|\hat{\omega}_k - \omega_k\|, \quad \hat{\omega}_k = \underset{\substack{i=1, \dots, n \\ i \neq k}}{\operatorname{argmin}} \|\omega_i - \omega_k\|, \quad k = 1, \dots, n \quad (7.25)$$

O modelo definido pela equação (7.16) é apropriado para altas dimensionalidades. Em problemas de baixa dimensionalidade e dados em alta densidade, o parâmetro de dispersão pode ser ajustado pela média ou o máximo. Eventualmente, um ajuste fino com o método do gradiente pode ser realizado a partir dos centros iniciais encontrados pelo algoritmo de agrupamento e o ajuste do parâmetro de dispersão.

O ajuste de parâmetros dos modelos RBF é feito pelo MMQ através da solução do sistema de equações normais (cf. equação (7.16)):

$$\mathbf{U}^T \mathbf{U} \boldsymbol{\theta}^T = \mathbf{U}^T \mathbf{Y} \quad (7.26)$$

$$\text{onde } \mathbf{U} = \begin{bmatrix} 1, u_1(1) & \cdots & u_n(1) \\ \vdots & \ddots & \vdots \\ 1, u_1(N) & \cdots & u_n(N) \end{bmatrix}, \boldsymbol{\theta}^T = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_n \end{bmatrix} \text{ e } \mathbf{Y} = \begin{bmatrix} y(1) \\ \vdots \\ y(N) \end{bmatrix}.$$

A equação (7.26) pode ser escrita na forma da equação (7.19) como:

$$\mathbf{H} \boldsymbol{\theta}^T = \mathbf{C}^T \quad (7.27)$$

$$\text{onde } \mathbf{H} = \mathbf{U}^T \mathbf{U} = \begin{bmatrix} \langle \phi_0(\mathbf{x}), \phi_0(\mathbf{x}) \rangle & \cdots & \langle \phi_0(\mathbf{x}), \phi_n(\mathbf{x}) \rangle \\ \vdots & \ddots & \vdots \\ \langle \phi_n(\mathbf{x}), \phi_0(\mathbf{x}) \rangle & \cdots & \langle \phi_n(\mathbf{x}), \phi_n(\mathbf{x}) \rangle \end{bmatrix}; \quad \mathbf{C}^T = \mathbf{U}^T \mathbf{Y} = \begin{bmatrix} \langle \phi_0(\mathbf{x}), \mathbf{Y} \rangle \\ \vdots \\ \langle \phi_n(\mathbf{x}), \mathbf{Y} \rangle \end{bmatrix}; \quad \text{o}$$

produto interno $\langle \phi_i(\mathbf{x}), \phi_j(\mathbf{x}) \rangle = \sum_{t=1}^N \phi_i(\mathbf{x}(t)) \phi_j(\mathbf{x}(t))$ e $\phi_0(\mathbf{x}) = 1$.

O ajuste de parâmetros pela minimização do EMQR (cf. equação (3.14)) é muito frequente nos modelos RBF [238][546]. A regularização impõe uma restrição à complexidade do modelo através de uma penalidade à norma L^2 do vetor de parâmetros. Nesse caso, as equações normais são escritas como:

$$\hat{\mathbf{H}} \boldsymbol{\theta}^T = \mathbf{C}^T \quad (7.28)$$

onde $\hat{\mathbf{H}} = \left(\mathbf{U}^T \mathbf{U} + \frac{\mu}{2} \mathbf{I} \right)$, sendo $\mathbf{I}$ a matriz identidade de ordem $n + 1$ e μ , o parâmetro de regularização. O termo $\mathbf{C}^T = \mathbf{U}^T \mathbf{Y}$ como anteriormente.

Um modelo regularizado também pode ser obtido pela solução de um problema de programação quadrática (QP)* [202][478], com restrições à norma do vetor de parâmetros. Nesse caso, a função de avaliação é o EMQ escrito na forma vetorial:

$$l(\boldsymbol{\theta}) = [\mathbf{Y} - \mathbf{U}\boldsymbol{\theta}^T]^T[\mathbf{Y} - \mathbf{U}\boldsymbol{\theta}^T] \quad (7.29)$$

onde $\mathbf{Y} = [y(1), \dots, y(N)]^T$ é o vetor de saídas observadas; a matriz $\mathbf{U}$, de dimensão $N \times n + 1$, é a matriz de características, como na equação (7.26), e $\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_n)$ é o vetor de parâmetros.

Expandindo a equação (7.29) o EMQ é calculado como:

$$l(\boldsymbol{\theta}) = \mathbf{Y}^T\mathbf{Y} - 2\mathbf{Y}^T\mathbf{U}\boldsymbol{\theta}^T + \boldsymbol{\theta}^T\mathbf{U}^T\mathbf{U}\boldsymbol{\theta} \quad (7.30)$$

Na formulação do problema QP, o de ajuste de parâmetros é escrito pela minimização do EMQ definido na equação (7.30) com restrições à norma L^2 do vetor de parâmetros de forma que $\|\boldsymbol{\theta}\|^2 < \psi$ como:

$$\begin{aligned} \min_{\boldsymbol{\theta}} \quad & \frac{1}{2}\boldsymbol{\theta}^T\mathbf{H}\boldsymbol{\theta} - \mathbf{C}^T\boldsymbol{\theta} \\ \text{sujeito a} \quad & -\sqrt{\psi} \leq \theta_i \leq \sqrt{\psi}, \quad i = 0, \dots, n \end{aligned} \quad (7.31)$$

onde $\mathbf{H} = \mathbf{U}^T\mathbf{U}$, $\mathbf{C}^T = \mathbf{U}^T\mathbf{Y}$ e ψ um parâmetro de regularização.

A solução do problema QP (7.31) realiza uma regularização do modelo, para soluções com $\|\boldsymbol{\theta}\|^2 < \psi$. Em [167], por exemplo, foi utilizado $\psi = n$. O problema QP (7.31) também pode ser formulado com outras restrições. No LASSO,[†] a restrição é definida sobre o valor absoluto (ou norma L^1) dos parâmetros [238][523].

A seleção da estrutura do modelo RBF é geralmente realizada por validação cruzada. O cálculo da estrutura do modelo como solução de um problema de otimização é obtido pelo modelo de máquina de vetor de suporte (SVM), apresentado na próxima seção.

* Quadratic Programming.

† Least Absolute Shrinkage and Selection Operator.

7.3 Máquinas de Vetor de Suporte

O modelo RBF pode ser entendido como um mapeamento não linear do conjunto de dados em outro espaço onde a função a ser estimada é linear (cf. Figura 7.4). O resultado do mapeamento é chamado de **vetor de características**, que é definido no **espaço de características** [521].

O mapeamento realizado pelo modelo RBF no espaço de características é explícito e definido pelas RBF. É possível obter uma representação indireta do espaço de características a partir do produto interno do espaço vetorial de funções. Esse tipo de representação permite o mapeamento em espaços vetoriais de funções de dimensão infinita e pode ser utilizado em problemas em que os registros são objetos complexos que podem não ter uma representação vetorial, como grafos, textos e sequências de DNA.

A representação indireta do espaço de características é realizada por **funções de núcleo** (*kernel function*), apresentadas a seguir. O modelo de máquina de vetor de suporte (SVM), discutido na 7.3.3, utiliza a representação por funções de núcleo.

7.3.1 Funções de núcleo

As funções de núcleo permitem realizar operações com os dados no espaço de características sem a necessidade de calcular o mapeamento explicitamente. Essa representação é chamada de **representação dual**, uma vez que a representação explícita do vetor de características é chamada de primal.

Considere um conjunto não vazio $\mathcal{X}$ sobre o qual o vetor de características é definido pela função vetorial $\boldsymbol{\varphi}(\mathbf{x}) = (\varphi_1(\mathbf{x}), \dots, \varphi_M(\mathbf{x}))$, onde $\varphi_i: \mathcal{X} \rightarrow \mathcal{R}$, $i = 1, \dots, M$. O número de funções $M \leq \infty$, isto é, o espaço de características pode ter dimensão infinita.

Uma função de núcleo é uma medida de similaridade $k: \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{R}^+$, calculada pelo produto interno:

$$k(\mathbf{x}', \mathbf{x}) = \langle \boldsymbol{\varphi}(\mathbf{x}'), \boldsymbol{\varphi}(\mathbf{x}) \rangle \quad (7.32)$$

onde $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$ são elementos do conjunto $\mathcal{X}$ e o produto interno é calculado como:

$$\langle \boldsymbol{\varphi}(\mathbf{x}'), \boldsymbol{\varphi}(\mathbf{x}) \rangle = \sum_{i=1}^M \varphi_i(\mathbf{x}') \varphi_i(\mathbf{x}) \quad (7.33)$$

A representação por funções de núcleo não depende de uma representação vetorial dos registros, de forma que $\mathcal{X}$ pode ser qualquer conjunto não vazio [478]. Diferentes funções características podem ter a mesma função de núcleo. Por exemplo, suponha que $\boldsymbol{\varphi}(x) = x$ e $\hat{\boldsymbol{\varphi}}(x) = (x/\sqrt{2}, x/\sqrt{2})$, $x \in \mathcal{R}$, em ambos os casos $k(x', x) = x'x$.

O mapeamento no espaço de características pode ter dimensão muito grande, ou até infinita. Na prática, o mapeamento não precisa ser calculado explicitamente e o produto interno é calculado através da função de núcleo, pela equação (7.32) [478].

Há várias funções de núcleo descritas na literatura [478], as mais conhecidas são:

- Núcleo linear:

$$k(\mathbf{x}', \mathbf{x}) = \langle \mathbf{x}', \mathbf{x} \rangle = \mathbf{x}' \mathbf{x}^T \quad (7.34)$$

- Núcleo polinomial homogêneo de grau $r > 0$:

$$k(\mathbf{x}', \mathbf{x}) = (\mathbf{x}' \mathbf{x}^T)^r \quad (7.35)$$

- Núcleo polinomial não homogêneo de grau $r > 0$:

$$k(\mathbf{x}', \mathbf{x}) = (\mathbf{x}' \mathbf{x}^T + \beta)^r \quad (7.36)$$

- Núcleo gaussiano com parâmetro $\sigma > 0$:

$$k(\mathbf{x}', \mathbf{x}) = \exp\left(-\frac{\|\mathbf{x}' - \mathbf{x}\|^2}{2\sigma^2}\right) \quad (7.37)$$

Funções de núcleo também podem ser definidas para problemas em que os registros tenham estruturas mais complexas, como sequências e grafos [478][479][593].

Em ciência de dados, o conjunto de dados $\mathcal{X} = \{\mathbf{x}(t) \in \mathcal{R}^p, t = 1, \dots, N\}$, cujo mapeamento no espaço de características é calculado pela matriz de núcleo, também conhecida por **matriz de Gram**:

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}(1), \mathbf{x}(1)) & \cdots & k(\mathbf{x}(1), \mathbf{x}(N)) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}(N), \mathbf{x}(1)) & \cdots & k(\mathbf{x}(N), \mathbf{x}(N)) \end{bmatrix} \quad (7.38)$$

Por definição, a matriz de Gram é simétrica e positiva semi-definida, de forma que:

$$\mathbf{v} \mathbf{K} \mathbf{v}^T \geq 0, \quad \forall \mathbf{v} \neq \mathbf{0} \quad (7.39)$$

A matriz de Gram é uma representação implícita do conjunto $\mathcal{X}$ no espaço de características sem que as funções características sejam calculadas. Um modelo de aproximação pode, assim, ser obtido implicitamente a partir da matriz de Gram, mesmo que as variáveis de entrada não tenham uma representação vetorial.

No caso da função de núcleo linear, a matriz de Gram $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ tem dimensão $N \times N$, enquanto a matriz de correlação* $\hat{\Sigma} = \frac{1}{N-1}\mathbf{X}^T\mathbf{X}$, tem dimensão $p \times p$. A posição da matriz transposta se refere à dimensão do produto interno. Na representação por funções de núcleo, o produto interno é realizado na dimensão do vetor de características (cf. equação (7.33)) e, na matriz de correlação, o produto interno é calculado na dimensão do número de registros (cf. equações (7.12) e (7.19)).

Em alguns problemas, o conjunto de entrada e o conjunto de registros têm dimensões comparáveis. A Figura 7.6 mostra a matriz de Gram com função de núcleo linear e a matriz de correlação para o conjunto de dados Sonar, que tem $p = 60$ variáveis de entrada e $N = 208$ registros. No caso em que $p > N$, a representação por funções de núcleo permite uma representação mais compacta do conjunto de dados.

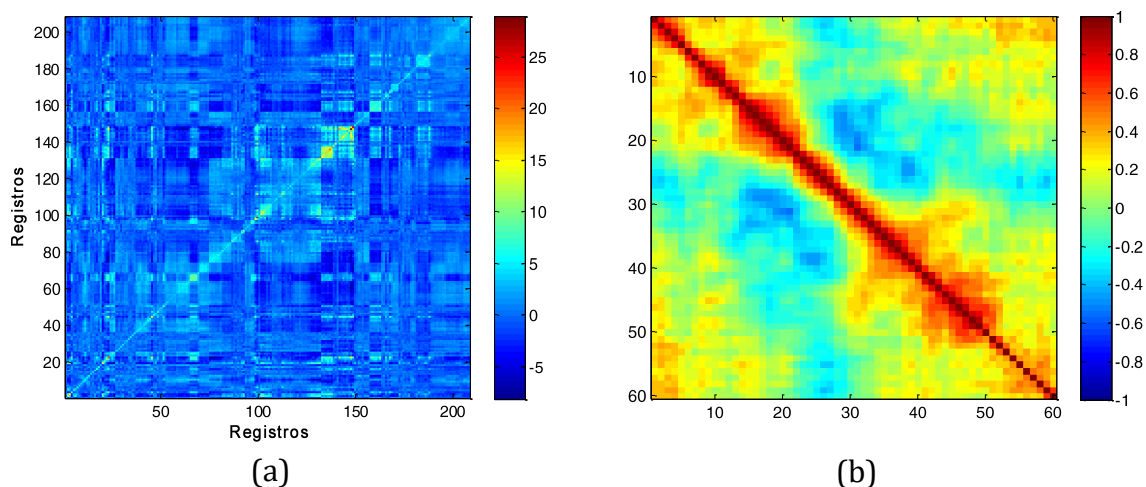


Figura 7.6: Problema Sonar: (a) matriz de núcleo linear; (b) matriz de correlação.

Funções de núcleo permitem realizar operações lineares no espaço de características que correspondem a operações não lineares no espaço de entrada. Essa técnica é conhecida como o “truque do núcleo” permite a adaptação de algoritmos ao espaço de características [118][257][485]. As transformações lineares discutidas na seção 2.4 têm correspondentes diretos em transformações não lineares através de funções de

* Considerando variáveis de média nula e desvio padrão unitário.

núcleo [476][519]. O algoritmo *kernel k-means*, por exemplo, resulta num particionamento de grupos não convexos no espaço das variáveis de entrada [476].

Funções de núcleo também são utilizadas para aproximação de funções. Considere o espaço vetorial de funções $\mathcal{F}$ discutido na seção 7.2, formado pelo conjunto de todas as funções contínuas $f: \mathcal{X} \rightarrow \mathcal{R}$, como um espaço de Hilbert. Qualquer função $f \in \mathcal{F}$ pode ser escrita em função de uma base $\Phi(\mathbf{x})$ como (cf. equação (7.11)):

$$f(\mathbf{x}) = \sum_{i=1}^N \phi_i(\mathbf{x}) f_i = \Phi(\mathbf{x}) \mathbf{f}^T \quad (7.40)$$

onde $\mathbf{f} = (f_1, \dots, f_N)$.

O espaço de funções $\mathcal{F}$ é chamado de espaço de Hilbert reproduzido por uma função de núcleo (RKHS)* quando qualquer função $f \in \mathcal{F}$ pode ser escrita como:

$$f(\mathbf{x}) = \langle \mathbf{f}, k(\mathbf{x}, \cdot) \rangle \quad (7.41)$$

onde $k(\mathbf{x}, \cdot)$ representa a aplicação da função de núcleo sobre todos os elementos do conjunto $\mathcal{X}$. No caso de $\mathcal{X} = \{\mathbf{x}(t) \in \mathcal{R}^p, t = 1, \dots, N\}$, $k(\mathbf{x}, \cdot) = \{k(\mathbf{x}, \mathbf{x}(i)), i = 1, \dots, N\}$.

Uma função de núcleo gera um único RKHS e cada RKHS tem sua função de núcleo associada, chamada de função de **núcleo reprodutor**. Uma função de núcleo reprodutor pode ser escrita como:

$$k(\mathbf{x}', \mathbf{x}) = \langle k(\mathbf{x}', \cdot), k(\cdot, \mathbf{x}) \rangle \quad (7.42)$$

O teorema de Mercer, de 1909 [377], mostra que funções de núcleo reprodutor geram bases ortonormais para um espaço de Hilbert, de modo que qualquer função $f \in \mathcal{F}$ pode ser escrita como (cf. equação (7.41)):

$$f(\mathbf{x}) = \sum_{i=1}^N k(\mathbf{x}, \mathbf{x}(i)) f_i \quad (7.43)$$

A norma $\|f(\mathbf{x})\|^2$ definida pela função de núcleo reprodutor é calculada como:

$$\|f(\mathbf{x})\|^2 = \sum_{i=1}^N \sum_{j=1}^N k(\mathbf{x}(i), \mathbf{x}(j)) f_i f_j = \mathbf{f} \mathbf{K} \mathbf{f}^T \quad (7.44)$$

* *Reproducing Kernel Hilbert Space.*

onde $\|f(\mathbf{x})\|^2 \geq 0$, uma vez que a matriz de núcleo $\mathbf{K}$ é positiva semi-definida (cf. equação (7.39)).

Comparando as equações (7.40) e (7.43), podemos concluir que uma função de núcleo aplicada a um conjunto de pontos forma uma base do espaço de funções $\mathcal{F}$. As duas representações podem ser equivalentes, como no caso da função de núcleo gaussiano, mas são formulações distintas. As funções de base na equação (7.40) são definidas explicitamente e podem ser ajustadas pelo modelo, enquanto a matriz de Gram é uma representação do conjunto de dados, que geralmente não é ajustada pelo modelo.

Uma função $f \in \mathcal{F}$ pode ser aproximada por uma função de núcleo reproduzidor como:

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^N k(\mathbf{x}, \mathbf{x}(i)) \theta_i \quad (7.45)$$

onde $\boldsymbol{\theta} = (\theta_1, \dots, \theta_N)$ é o conjunto de parâmetros da aproximação.

Um resultado importante estabelece que a aproximação definida pela equação (7.45) é uma solução da forma generalizada da função de custo com regularização:

$$l(\mu, \boldsymbol{\theta}) = \frac{1}{N} \sum_{t=1}^N l(y(t), \hat{y}(t)) + \frac{\mu}{2} \|\hat{f}(\mathbf{x})\|^2 \quad (7.46)$$

onde $l(y(t), \hat{y}(t))$ é uma função de custo genérica que determina a qualidade da aproximação; $\|\hat{f}(\mathbf{x})\|^2 = \boldsymbol{\theta} \mathbf{K} \boldsymbol{\theta}^T$ (cf. equação (7.44)) e μ um parâmetro de regularização.

Funções de núcleo têm sido extensivamente estudadas na matemática. Em 1950, Aronszajn [20] generaliza diversos resultados anteriores e apresenta a teoria de espaços de Hilbert reproduzidos por funções de núcleo. A aplicação das funções de núcleo em algoritmos de ciência de dados passou a ser mais intensa a partir de 1995. Atualmente, essa teoria tem sido bastante utilizada, sobretudo em problemas em que os registros não têm uma representação vetorial explícita. Uma descrição completa de funções de núcleo e sua aplicação no contexto de máquinas de vetor de suporte (SVM) pode ser encontrada no livro de Schölkopf e Smola [478].

Os modelos SVM utilizam a formulação de funções de núcleo, mas o ajuste de parâmetros é realizado pelo princípio de minimização do risco estrutural, descrito a seguir.

7.3.2 Minimização do risco estrutural

O ajuste de modelos RBF depende da escolha adequada do número de unidades RBF que determina a estrutura do modelo. Na prática, a seleção da melhor estrutura é realizada a partir dos resultados do modelo em validação cruzada. Esse procedimento visa realizar uma avaliação “experimental” do modelo e selecionar aquele com melhor capacidade de generalização. O ajuste de parâmetros em cada ciclo de validação cruzada é realizado minimizando o EMQ no conjunto de treinamento. Essa abordagem é chamada de minimização do **risco empírico**. No modelo SVM, o ajuste de parâmetros é realizado pela minimização do chamado **risco estrutural**, que está diretamente relacionado à capacidade de generalização do modelo.

A capacidade de generalização de um modelo pode ser avaliada pelo seu desempenho em relação a todos os valores possíveis das variáveis de entrada e saída, isto é, o processo estocástico que representa o problema. O **risco esperado total** do modelo é definido pela distribuição de probabilidades conjunta $p(\mathbf{x}, y)$ que representa o processo:

$$R(\hat{f}) = E \left(l(f(\mathbf{x}), \hat{f}(\mathbf{x}, \boldsymbol{\theta})) \right) \triangleq \int l(f(\mathbf{x}), \hat{f}(\mathbf{x}, \boldsymbol{\theta})) p(\mathbf{x}, y) d\mathbf{x} dy \quad (7.47)$$

onde E é a esperança matemática; $l(f(\mathbf{x}), \hat{f}(\mathbf{x}, \boldsymbol{\theta}))$ é uma função de custo de forma que $l(f(\mathbf{x}), \hat{f}(\mathbf{x}, \boldsymbol{\theta})) = 0$ se $f(\mathbf{x}) = \hat{f}(\mathbf{x}, \boldsymbol{\theta})$; $p(\mathbf{x}, y)$ é a função de densidade de probabilidade da distribuição conjunta e $y = f(\mathbf{x})$, o valor observado de saída.

No problema de regressão, o conjunto de dados $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$ é uma amostra independente e identicamente distribuída (iid) do processo com distribuição de probabilidades conjunta $p(\mathbf{x}, y)$. Nos modelos discutidos no Capítulo 3, o ajuste de parâmetros é realizado pela minimização do risco empírico do modelo $\hat{R}(\hat{f})$, calculado sobre o conjunto de treinamento como:

$$\hat{R}(\hat{f}) = l(\boldsymbol{\theta}) = \sum_{t=1}^N l(y(t), \hat{y}(t)) \quad (7.48)$$

onde $\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta})$ é o valor estimado pelo modelo e $y(t)$ é o valor observado de cada registro do conjunto de treinamento $t = 1, \dots, N$.

A função $l(\boldsymbol{\theta})$ na equação (7.48) generaliza diversas funções de custo utilizadas no ajuste de parâmetros de modelos preditivos, como discutido na seção 7.1. Frequentemente, mesmo na solução ótima, há um resíduo $e(t)$ que representa o efeito de variáveis ausentes e erros de medida, de forma que $y(t) = \hat{y}(t) + e(t)$.

O risco empírico converge para o risco total quando $N \rightarrow \infty$:

$$R(\hat{f}) = \lim_{N \rightarrow \infty} \hat{R}(\hat{f}). \quad (7.49)$$

Os modelos que realizam o ajuste de parâmetros pela minimização do risco empírico terão melhor desempenho quanto maior for o tamanho da amostra. Esse comportamento foi discutido na seção 3.1.4. A Figura 3.4d e a Figura 3.5a, reproduzidas na Figura 7.7, mostram os resultados do modelo linear polinomial de grau 10 ajustados em duas amostras oriundas da mesma distribuição de probabilidades conjunta. Na Figura 7.7a, o modelo foi ajustado com uma amostra de $N = 20$ registros e, na Figura 7.7b, com uma amostra de $N = 200$ registros. A função real $f(\mathbf{x})$, um polinômio de grau 2, é bem diferente da aproximação $\hat{f}(\mathbf{x}, \boldsymbol{\theta})$, um polinômio de grau 10, mas os parâmetros calculados pela minimização do EMQ na amostra de $N = 200$ registros permitem uma boa aproximação na região representada pelo conjunto de treinamento.

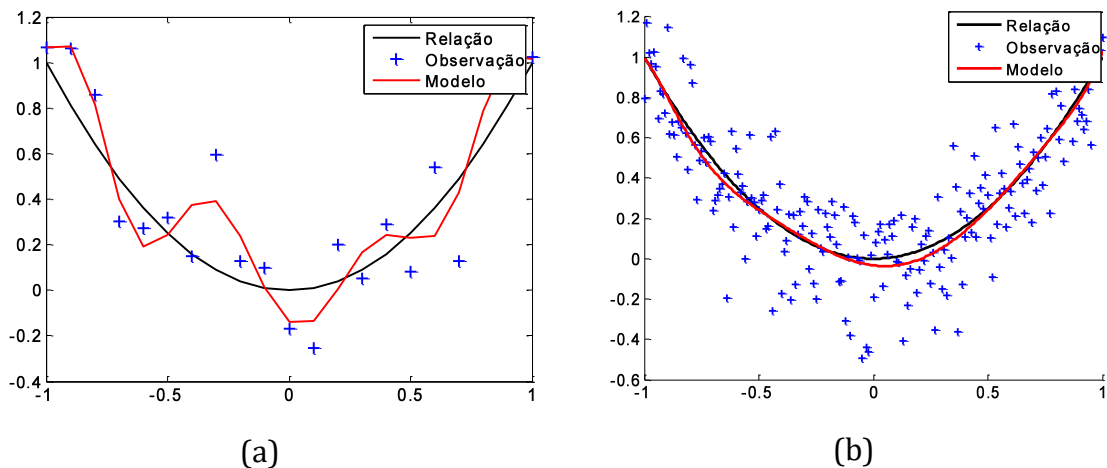


Figura 7.7: Efeito do tamanho da amostra n modelo polinomial de grau 10: (a) $N = 20$; (b) $N = 200$.

Num problema real, a distribuição de probabilidades conjunta $p(\mathbf{x}, y)$ é desconhecida e a função real $f(\mathbf{x})$ é observada apenas nos registros do conjunto de treinamento, frequentemente contaminada por erros de medida e pelo efeito de variáveis não observadas. Em problemas em que o número de registros do conjunto de treinamento é pequeno em relação à complexidade do modelo, o ajuste de parâmetros pela minimização

do risco empírico pode resultar em *overfitting*, sendo necessária a utilização de alguma técnica de regularização.

O efeito do tamanho da amostra do conjunto de treinamento depende da **capacidade** do modelo de representar funções arbitrariamente complexas. No exemplo sintético discutido na seção 3.1.4, o modelo linear polinomial de grau 2 realiza um bom ajuste, mesmo com a amostra de $N = 20$ registros (cf. Figura 3.4c).

A capacidade do modelo está relacionada à sua estrutura e, intuitivamente, pode ser associada ao número de parâmetros a serem ajustados. Entretanto, o número de parâmetros do modelo não é uma medida adequada para se estimar o risco esperado total, pois dois modelos com o mesmo número de parâmetros podem ter comportamentos diferentes com amostras diferentes, como mostra a Figura 7.7.

Na teoria do aprendizado estatístico [532][533][534], a capacidade do modelo é definida por uma medida abstrata chamada de **dimensão VC**, em homenagem aos autores Vapnik e Chervonenkis. A dimensão VC é uma medida teórica definida para problemas de classificação como a capacidade discriminante de uma família de funções. A dimensão VC não considera a distribuição de probabilidades $p(\mathbf{x}, y)$ de algum problema particular e, teoricamente vale para todas as combinações possíveis de atribuição das classes. A dimensão VC é definida pelo número máximo de pontos que podem ser separados pela família de funções.

A dimensão VC é uma medida teórica e seu cálculo é complexo. No caso de funções lineares e registros $\mathbf{x} \in \mathcal{R}^p$, a dimensão VC é calculada como $\zeta = p + 1$. Por exemplo, no caso de funções lineares e $p = 2$ variáveis, $\zeta = 3$, isto é, no máximo três registros podem ser separados por uma função linear em $\mathcal{R}^2$. A Figura 7.8 mostra as $2^3 = 8$ combinações possíveis de atribuições de classe a três registros em $\mathcal{R}^2$. Observe que quatro pontos não podem ser separados por uma função linear em $\mathcal{R}^2$, como, por exemplo, no caso do problema XOR (cf. Figura 7.3).

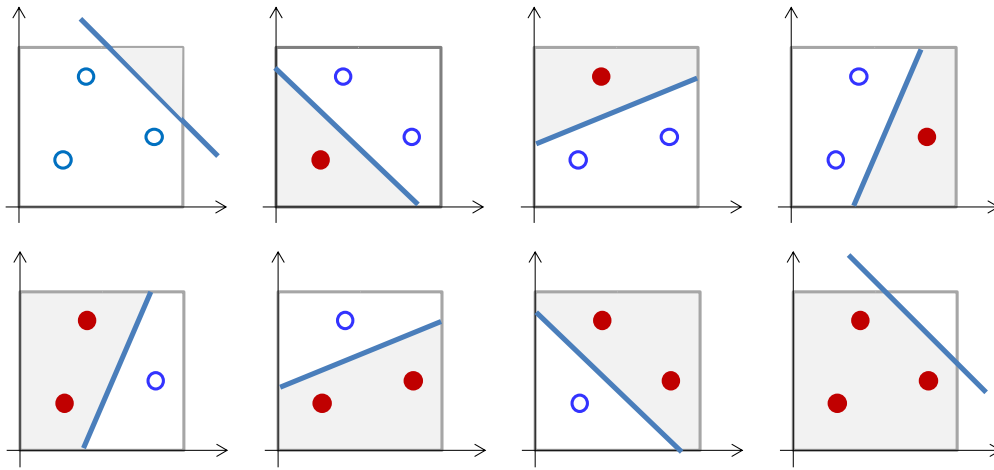


Figura 7.8: Combinações possíveis de classificação de três pontos em $\mathcal{R}^2$.

A teoria de aprendizado estatístico permite a definição de limites para o risco esperado total de modelos definidos por sua dimensão VC [533][534]. Um dos resultados mais conhecidos estabelece que o risco esperado total de um classificador com dimensão VC igual a ζ num problema com N registros é calculado por:

$$R(\hat{f}) \leq \hat{R}(\hat{f}) + \sqrt{\frac{\zeta(\log(2N/\zeta) + 1) - \log(\eta/4)}{N}} \quad (7.50)$$

onde η é a confiança estatística do resultado.

A inequação (7.50) estabelece um limite para o desempenho do modelo nos registros não observados, isto é, o risco estrutural $R_s(\hat{f}) \geq R(\hat{f}) - \hat{R}(\hat{f})$, permitindo uma explicação formal do efeito de *overfitting*. Esse limite pode ser observado na Figura 7.9, que representa o risco estrutural definido pela inequação (7.50) como função da dimensão VC e do número de registros do conjunto de treinamento N . Para amostras grandes, o risco estrutural $R_s(\hat{f})$ é baixo, mesmo para valores grandes da dimensão VC, e quanto menor a capacidade do modelo, menor o risco estrutural. Para amostras pequenas, o risco estrutural aumenta muito para modelos com maior capacidade, isto é, valores altos de ζ , e é menor para modelos de menor capacidade. Dessa forma, o *overfitting* acontece em modelos complexos com amostras pequenas.

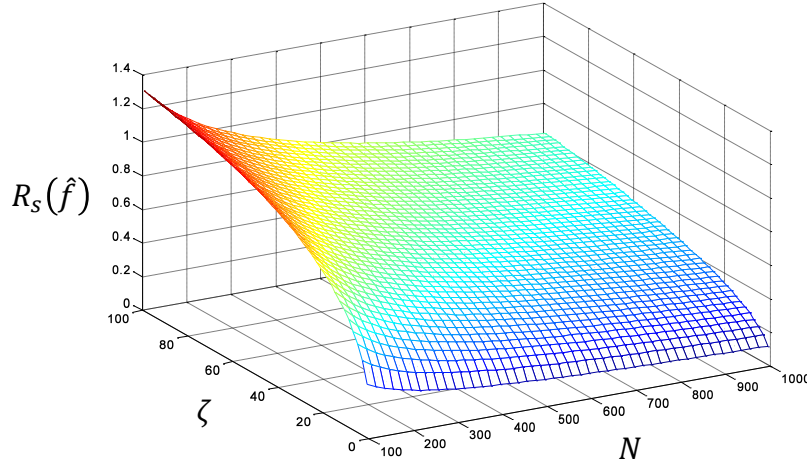


Figura 7.9: Limite do risco estrutural $R_s(\hat{f})$ como função da dimensão VC ζ e do número de registros do conjunto de treinamento N .

A inequação (7.50) é um resultado teórico que define formalmente limites para a capacidade de generalização de modelos, mas não é utilizada diretamente para o ajuste de modelos. O conceito de minimização de risco estrutural é aplicado nos modelos SVM pela maximização da margem de separação, descritos a seguir.

7.3.3 Classificadores de máxima margem

A seção 7.1 apresenta um resumo dos problemas de ajuste de parâmetros utilizados pelos modelos de regressão e classificação linear, que seguem o princípio de minimização do risco empírico. Na regressão linear, o ajuste de parâmetros é realizado pela minimização do EMQ calculado no conjunto de treinamento (cf. equação (7.4)). No modelo linear regularizado, a função de custo é o EMQR (cf. equação (7.8)) em que a norma dos parâmetros é uma restrição, controlada pelo parâmetro de regularização μ , que penaliza modelos mais complexos. No ajuste de parâmetros de modelos RBF pela solução do problema quadrático (7.31), a norma L^2 dos parâmetros também é utilizada para controlar a complexidade da solução, de modo que $\|\theta\|^2 < \psi$.

No modelo SVM, o ajuste de parâmetros é feito pela minimização do risco estrutural, em que a função de custo representa a estrutura do modelo e as restrições visam garantir seu desempenho no conjunto de treinamento. Em problemas de classificação, a estrutura do modelo tem uma interpretação geométrica que corresponde à **margem de separação** entre os registros de duas classes.

Considere o conjunto de dados de treinamento $\mathcal{T} = \{(\mathbf{x}(t), v(t)), t = 1 \dots N\}$ de um problema de classificação de $m = 2$ classes onde $v(t)$ representa a variável indicadora:

$$v(t) = \begin{cases} +1, & \text{se } \mathbf{x}(t) \in C_0 \\ -1, & \text{se } \mathbf{x}(t) \in C_1 \end{cases} \quad (7.51)$$

Suponha que o problema seja linearmente separável e que a função discriminante é um hiperplano definido* como:

$$g(\mathbf{x}) = \langle \mathbf{x}, \mathbf{w} \rangle + b \quad (7.52)$$

onde $\mathbf{x} \in \mathcal{R}^p$ é um ponto qualquer no espaço de variáveis de entrada; o vetor $\mathbf{w} \in \mathcal{R}^p$ representa a direção ortogonal ao hiperplano e o produto interno $\langle \mathbf{x}, \mathbf{w} \rangle = \mathbf{x}\mathbf{w}^T$. O parâmetro b é o termo independente. A função discriminante $g(\mathbf{x}) > 0$ na região da classe positiva C_0 e $g(\mathbf{x}) < 0$ na região da classe negativa C_1 .

No exemplo com $\mathbf{x}(t) \in \mathcal{R}^2$, mostrado na Figura 7.10, os registros da classe C_0 são mostrados em azul e os registros da classe C_1 em vermelho, de modo que o plano $g(\mathbf{x})$ é negativo na região cinza. A interseção entre o plano discriminante $g(\mathbf{x})$ e o plano $\mathcal{R}^2$, que contém os registros, define a reta discriminante $g(\mathbf{x}) = 0$ que separa as classes.

O vetor unitário $\mathbf{w}/\|\mathbf{w}\|$ é ortogonal à reta $g(\mathbf{x}) = 0$, de modo que a distância de um ponto qualquer do conjunto $\mathcal{T}$ até a reta discriminante pode ser calculada como:

$$\delta(t) = \frac{v(t)g(\mathbf{x}(t))}{\|\mathbf{w}\|} \quad (7.53)$$

Em particular, a distância da origem até a reta discriminante é $b/\|\mathbf{w}\|$. A margem de separação é definida como a menor distância de um ponto do conjunto de treinamento até a reta discriminante e é calculada como (cf. Figura 7.10):

$$\delta^* = \min_{t=1, \dots, N} \left(\frac{v(t)g(\mathbf{x}(t))}{\|\mathbf{w}\|} \right) \quad (7.54)$$

A equação (7.54) pode ter infinitas soluções multiplicando-se $g(\mathbf{x}(t))$ por qualquer constante. Para garantir que a solução seja única, utiliza-se o hiperplano canônico, que é normalizado de modo que a margem de separação $\delta^* = 1/\|\mathbf{w}\|$.

* Nesta seção, vamos seguir a notação usual da literatura. Posteriormente, os parâmetros do modelo SVM serão definidos utilizando pelo vetor $\boldsymbol{\theta}$, como nos capítulos anteriores.

A Figura 7.10 mostra duas soluções possíveis num problema linearmente separável. A solução da Figura 7.10a tem maior margem de separação que a da Figura 7.10b e, segundo a teoria de aprendizado estatístico, melhor capacidade de generalização.

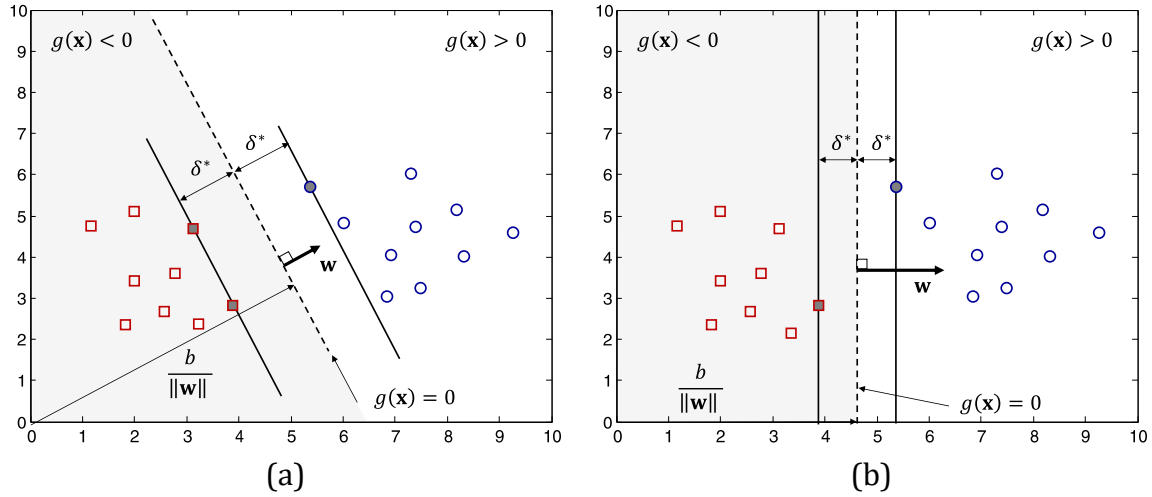


Figura 7.10: Interpretação geométrica da margem de separação.

Teoricamente, o controle de $\|\mathbf{w}\|$ corresponde ao controle da dimensão VC. Num modelo linear, se $\|\mathbf{w}\| < \Delta$, então a dimensão VC $\zeta < r^2 \Delta^2$, onde r é o raio da maior esfera contendo todos os registros do conjunto de treinamento [478].

Os registros do conjunto de treinamento sobre a margem de separação são chamados de **vetores de suporte**, pois são os pontos que “apoiam” o hiperplano da função discriminante. Nos vetores de suporte $\mathbf{x}(t^*)$, o valor do hiperplano canônico $g(\mathbf{x}(t^*)) = 1$ se $\mathbf{x}(t^*) \in C_0$, e $g(\mathbf{x}(t^*)) = -1$ se $\mathbf{x}(t^*) \in C_1$.

O ajuste de parâmetros do modelo SVM é feito pela maximização da margem de separação, garantindo o desempenho do modelo nos registros do conjunto de treinamento, como descrito a seguir.

7.3.3.1 Problemas separáveis

Num problema de classificação com $m = 2$ classes linearmente separáveis, a solução do modelo SVM linear é o hiperplano canônico com maior margem de separação. O problema de ajuste de parâmetros é escrito como:

$$\begin{aligned} \min_{\mathbf{w}} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{sujeito a} \quad & v(t)g(\mathbf{x}(t)) \geq 1, \quad t = 1, \dots, N \end{aligned} \quad (7.55)$$

onde $v(t)$ é a variável indicadora definida na equação (7.51).

O problema (7.55) é convexo, pois a função de avaliação é quadrática, e possui N restrições lineares que garantem que todos os registros do conjunto de treinamento sejam adequadamente classificados. O problema (7.55) é chamado **primal** porque é formulado diretamente sobre a margem de separação $\mathbf{w}$.

A solução do problema (7.55) é obtida através da definição do lagrangeano:

$$L(b, \mathbf{w}, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{t=1}^N \alpha(t) (v(t)g(\mathbf{x}(t)) - 1) \quad (7.56)$$

onde $\boldsymbol{\alpha} = (\alpha(1), \dots, \alpha(N))$, em que $\alpha(t) \geq 0, t = 1, \dots, N$, é o vetor dos **multiplicadores de Lagrange**.

A formulação do lagrangeano é uma técnica clássica de otimização que transforma o problema de otimização com restrições (7.55) num problema sem restrições cuja função de avaliação é definida na equação (7.56). A solução é obtida pelas condições de Karush Kuhn Tucker (KKT) [478], igualando-se a derivada do lagrangeano com relação aos parâmetros a zero:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{t=1}^N \alpha(t) v(t) \mathbf{x}(t) = \mathbf{0} \quad (7.57)$$

$$\frac{\partial L}{\partial b} = \sum_{t=1}^N \alpha(t) v(t) = 0 \quad (7.58)$$

Resolvendo (7.57), obtém-se a solução para $\mathbf{w}$ como uma combinação linear dos registros do conjunto de treinamento:

$$\mathbf{w} = \sum_{t=1}^N \alpha(t) v(t) \mathbf{x}(t) \quad (7.59)$$

As restrições do problema primal (7.55) e a formulação do lagrangeano (7.56) determinam que, na solução, os multiplicadores de Lagrange correspondentes às restrições não ativas devem ser nulos. Na solução, os registros que correspondem às restrições ativas estão no limite da margem de separação, e os multiplicadores de Lagrange correspondentes são positivos. Desta forma, a seguinte equação é satisfeita:

$$\alpha(t) (v(t)g(\mathbf{x}(t)) - 1) = 0, \quad t = 1, \dots, N \quad (7.60)$$

A equação (7.60) mostra que, nos registros correspondentes aos vetores de suporte, $v(t^*)g(\mathbf{x}(t^*)) = 1$ e $\alpha(t^*) > 0$. Nos demais registros $\alpha(t) = 0$. Desta forma, os vetores de suporte $\mathbf{x}(t^*)$ são os únicos registros do conjunto de treinamento que são considerados no cálculo dos coeficientes da função discriminante pela equação (7.59); todos os outros registros correspondem a multiplicadores de Lagrange nulos e são considerados na função discriminante. Os vetores de suporte também são utilizados para o cálculo do parâmetro b pela média das soluções da equação $v(t^*)g(\mathbf{x}(t^*)) = 1$ para todos os vetores de suporte.

O lagrangeano (7.56) é um ponto de sela, que deve ser minimizado em relação ao vetor $\mathbf{w}$ e maximizado em relação ao vetor de multiplicadores de Lagrange $\boldsymbol{\alpha}$. O problema de ajuste de parâmetros pode ser escrito em função apenas dos multiplicadores de Lagrange substituindo as equações (7.57) e (7.58) na equação (7.56), que resulta na formulação **dual** do modelo SVM linear:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} L(\boldsymbol{\alpha}) &= \sum_{t=1}^N \alpha(t) - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha(i) \alpha(j) v(i) v(j) \langle \mathbf{x}(i), \mathbf{x}(j) \rangle \\ \text{subj. } \sum_{t=1}^N \alpha(t) v(t) &= 0 \text{ e } \alpha(t) \geq 0, \quad t = 1, \dots, N \end{aligned} \quad (7.61)$$

onde $L_d(\boldsymbol{\alpha})$ é o lagrangeano dual e $\langle \mathbf{x}(i), \mathbf{x}(j) \rangle = \mathbf{x}(i) \mathbf{x}(j)^T$ é o produto interno definido no espaço de variáveis de entrada, que corresponde à função de núcleo linear.

A solução do problema de otimização dual (7.61) é o vetor $\boldsymbol{\alpha}^*$ que identifica os registros correspondentes aos vetores de suporte $\mathbf{x}(t^*)$ pelos multiplicadores de Lagrange positivos, isto é, $\alpha^*(t^*) > 0$. A função discriminante pode ser escrita diretamente a partir dos multiplicadores de Lagrange como:

$$g(\mathbf{x}) = \sum_{t=1}^N \alpha^*(t) v(t) \langle \mathbf{x}, \mathbf{x}(t) \rangle + b \quad (7.62)$$

A formulação dual do ajuste de parâmetros do modelo SVM linear em problemas separáveis pode ser facilmente estendida ao caso não linear através do “truque” do núcleo, substituindo o produto interno $\langle \mathbf{x}(i), \mathbf{x}(j) \rangle$ linear por uma função de núcleo não linear $k(\mathbf{x}(i), \mathbf{x}(j))$, como as definidas nas equações (7.35)-(7.37).

Definindo-se o vetor de parâmetros $\boldsymbol{\theta} = (\alpha(1)v(1), \dots, \alpha(N)v(N))$, a formulação dual do de ajuste de parâmetros do modelo SVM não linear pode ser escrita como um problema de programação quadrática (QP):

$$\begin{aligned} \min_{\boldsymbol{\theta}} \hat{L}(\boldsymbol{\theta}) &= \frac{1}{2} \boldsymbol{\theta} \mathbf{K} \boldsymbol{\theta}^T - \boldsymbol{\theta} \mathbf{V}^T \\ \text{sujeito a } \mathbf{1} \boldsymbol{\theta}^T &= 0 \text{ e } v(t)\theta(t) \geq 0, \quad t = 1, \dots, N \end{aligned} \quad (7.63)$$

onde $\mathbf{K}$ é a matriz de núcleo, $\mathbf{V} = (v(1), \dots, v(N))$ é o vetor com as variáveis indicadoras para todos os registros do conjunto de treinamento e $\mathbf{1} = (1, 1, \dots, 1)$, o vetor de coordenadas unitárias de dimensão N , de forma que $\mathbf{1} \boldsymbol{\theta}^T = \sum_{t=1}^N \alpha(t)v(t) = 0$.

Observe que a variável indicadora $v(t) \in \{-1, 1\}$, então $v(t)\theta(t) = \alpha(t)v(t)^2 = \alpha(t)$ e o produto $\boldsymbol{\theta} \mathbf{V}^T = \sum_{t=1}^N \alpha(t)$.

O lagrangeano no problema de otimização (7.63) $\hat{L}(\boldsymbol{\theta}) = -L(\boldsymbol{\alpha})$ (cf. (7.61)) e, por isso, o problema é de minimização. O problema QP (7.63) é semelhante ao do ajuste de parâmetros de modelos RBF (cf. (7.31)), mas com restrições diferentes. No ajuste de parâmetros de modelos RBF, a matriz de coeficientes $\mathbf{H}$ tem dimensão $n \times n$, onde n é o número de RBF. No caso do ajuste de parâmetros do modelo SVM, a matriz de núcleo $\mathbf{K}$ tem dimensão $N \times N$, onde N é o número de registros. Em problemas N grande, o ajuste de parâmetros se torna computacionalmente complexo.

A função discriminante não linear é escrita diretamente a partir da função de núcleo e da solução do problema (7.63) como:

$$g(\mathbf{x}) = \sum_{t=1}^N \theta^*(t) k(\mathbf{x}, \mathbf{x}(t)) + b \quad (7.64)$$

onde $\theta^*(t)$ é a solução do problema, em que $\theta^*(t^*) = \alpha^*(t^*)v(t^*)$ nos vetores de suporte e, nos demais registros, $\theta^*(t) = 0, \forall t \neq t^*$; $k(\mathbf{x}, \mathbf{x}(t))$ uma função de núcleo e $\mathbf{x}$ o vetor a ser predito pelo modelo.

A equação (7.64) define a função discriminante do modelo SVM como uma aproximação gerada pela função de núcleo reprodutor (cf. equação (7.45)). O primeiro termo do lagrangeano dual $\hat{L}(\boldsymbol{\theta})$ no problema de otimização (7.63) é a norma da função discriminante, ou seja, $\boldsymbol{\theta} \mathbf{K} \boldsymbol{\theta}^T = \|g(\mathbf{x})\|^2$, de forma que a solução do modelo SVM busca uma função discriminante de menor norma.

A solução do problema de otimização (7.63) pode ser obtida diretamente pelo método do gradiente [593], o que é computacionalmente complexo em problemas com grande número de registros. Há, na literatura, soluções mais eficientes, como o algoritmo da otimização de sequência mínima (SMO)* [477], que subdivide o problema e, a cada iteração, resolve um problema de menor dimensão. Há também soluções por algoritmos paralelos para solução do problema QP [594]. O modelo SVM é mais adaptado a problemas com pequeno número de registros.

A Figura 7.11 apresenta dois exemplos de aplicação do modelo SVM em problemas separáveis de $p = 2$ variáveis, mostrando os registros das classes, a função discriminante e os vetores de suporte (em azul). A Figura 7.11a é um exemplo de problema linearmente separável e o resultado foi obtido com a função de núcleo linear. Na Figura 7.11b, o problema é separável, mas a função discriminante é não linear e, nesse caso, o resultado foi obtido com a função de núcleo gaussiano.

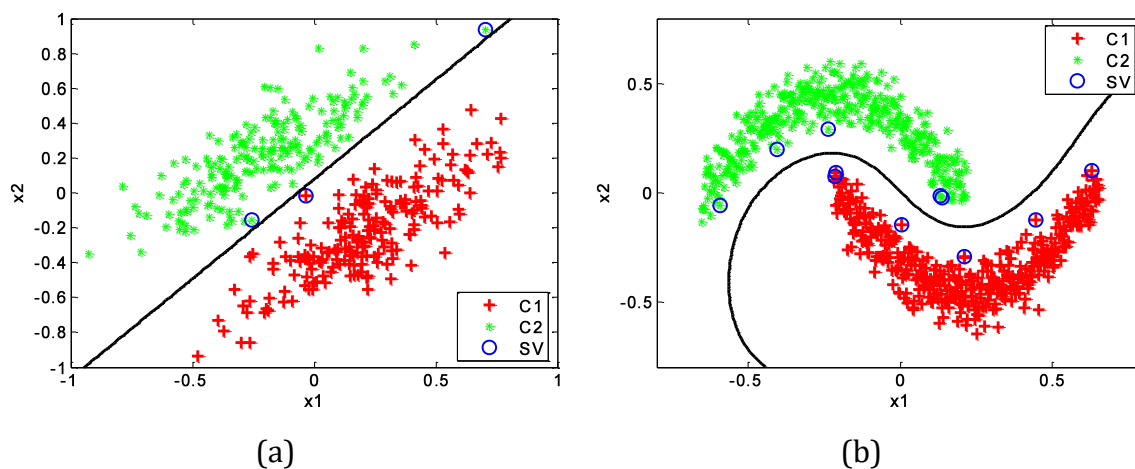


Figura 7.11: Exemplos do modelo SVM: (a) núcleo linear; (b) núcleo gaussiano.

O modelo SVM para problemas separáveis é bem fundamentado teoricamente e não tem nenhum parâmetro livre, uma vez que a estrutura do modelo é determinada como solução do problema de ajuste de parâmetros e as restrições garantem erro nulo em todos os registros do conjunto de treinamento. Entretanto, na maioria das aplicações, os problemas são não separáveis. A extensão do modelo SVM em problemas não separáveis é feita por classificadores de **margem flexível** (*soft margin*), descritos a seguir.

* *Sequential Minimal Optimization*.

7.3.3.2 Problemas não separáveis

Em problemas não separáveis, alguns registros serão classificados incorretamente pelo modelo. Nesse caso, o ajuste de parâmetros do modelo SVM é formulado a partir da introdução do vetor $\xi = (\xi(1), \dots, \xi(N))$, chamado de **variáveis de folga** (*slack variables*). Cada elemento $\xi(t) \geq 0$ representa o quanto o registro correspondente viola a restrição de classificação correta no problema de otimização (7.55) como:

$$v(t)g(\mathbf{x}(t)) \geq 1 - \xi(t), \quad t = 1, \dots, N \quad (7.65)$$

Como a variável indicadora $v(t) \in \{-1, 1\}$ e $\xi(t) \geq 0$, as variáveis de folga representam o erro de classificação, e a equação (7.65) pode ser rescrita como:

$$\xi(t) = \begin{cases} \max(0, v(t) - \hat{v}(t)), & \text{se } v(t) = +1 \\ \max(0, \hat{v}(t) - v(t)), & \text{se } v(t) = -1 \end{cases} \quad (7.66)$$

onde $\hat{v}(t) = g(\mathbf{x}(t))$.

A equação (7.66) é ilustrada na Figura 7.12. Para os registros classificados corretamente, $\xi(t) = 0$ para ambas as classes. Considerando o hiperplano canônico em que $\hat{v}(t^*) = \pm 1$ nos vetores de suporte, as variáveis de folga $0 \leq \xi(t) \leq 1$ se o registro correspondente for classificado corretamente, mas estiver no interior da margem. Nos pontos sobre a superfície discriminante $\hat{v}(t) = 0$ e $\xi(t) = 1$ para ambas as classes. Nos casos de registros classificados incorretamente, $\xi(t) \geq 1$ para ambas as classes.

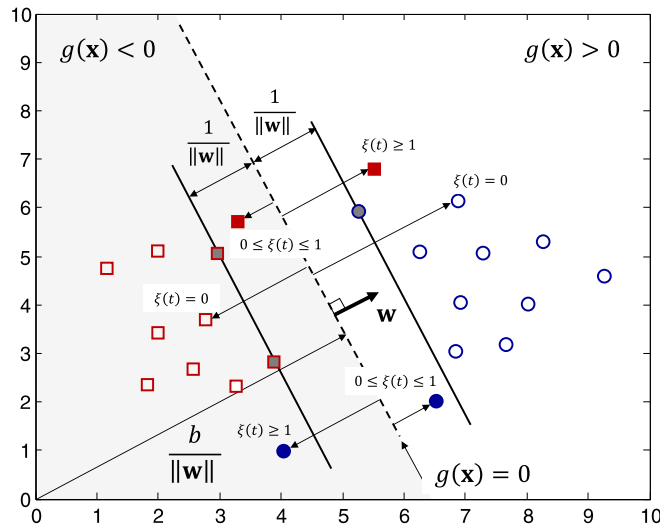


Figura 7.12: Modelo SVM não separável.

As variáveis de folga podem ser interpretadas como restrições para a formulação do problema de ajuste de parâmetros do modelo SVM em problemas não separáveis:

$$\begin{aligned} \min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 + \frac{\lambda}{N} \sum_{t=1}^N \xi(t)^\kappa \\ \text{sujeito a } \xi(t) \geq 0, v(t)g(\mathbf{x}(t)) \geq 1 - \xi(t), \quad t = 1, \dots, N \end{aligned} \quad (7.67)$$

O termo adicional da função objetivo representa o custo de classificação incorreta e o expoente κ permite a definição de diferentes funções de avaliação, tipicamente $\kappa = 1$ ou $\kappa = 2$ [593]. O *trade-off* entre a maximização da margem, que representa a suavidade do modelo, e a penalidade sobre a classificação incorreta é ajustado pelo parâmetro* de regularização λ . Quando $\lambda \rightarrow 0$, o termo de penalização perde importância e o objetivo é basicamente a maximização da margem. Quando $\lambda \rightarrow \infty$, o termo da margem perde o efeito e o objetivo passa a ser a minimização do custo de classificação incorreta, como na abordagem de minimização do risco empírico. Schölkopf e Smola indicam o valor $\lambda/N = 10$ pode ser usado como *default* [478].

No caso do expoente $\kappa = 1$ em (7.67), a formulação dual do problema de otimização resulta no problema QP escrito como [593]:

$$\begin{aligned} \min_{\boldsymbol{\theta}} L_d(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta} \mathbf{K} \boldsymbol{\theta}^T - \boldsymbol{\theta} \mathbf{V}^T \\ \text{sujeito a } \mathbf{1} \boldsymbol{\theta}^T = 0 \text{ e } 0 \leq v(t)\theta(t) \leq \frac{\lambda}{N}, \quad t = 1, \dots, N \end{aligned} \quad (7.68)$$

onde, como no caso separável, o vetor de parâmetros $\boldsymbol{\theta} = (\alpha(1)v(1), \dots, \alpha(N)v(N))$, $\mathbf{K}$ é a matriz de núcleo e $\mathbf{V} = (v(1), \dots, v(N))$, o vetor com as variáveis indicadoras para todos os registros do conjunto de treinamento. A diferença entre os problemas de otimização (7.68) e (7.63) são as restrições adicionais $\alpha(t) \leq \lambda/N$, $t = 1, \dots, N$ sobre os multiplicadores de Lagrange, escritas como $(t)\theta(t) \leq \lambda/N$.

No caso do expoente $\kappa = 2$, temos $\xi(t)^2 = (v(t) - \hat{v}(t))^2$ (cf. equação (7.66)). Dividindo-se os dois termos da função objetivo por λ , o problema (7.67) é escrito como:

* Na literatura, o parâmetro λ é geralmente representado por C e o modelo é chamado de C -SVM. Esse parâmetro também é identificado dessa forma na maioria dos *softwares* que implementam o modelo.

$$\begin{aligned} \min_{\mathbf{w}} \frac{1}{N} \sum_{t=1}^N (v(t) - \hat{v}(t))^2 + \frac{\mu}{2} \|\mathbf{w}\|^2 \\ \text{sujeito a } v(t)g(\mathbf{x}(t)) \geq 1 - \xi(t), \quad t = 1, \dots, N \end{aligned} \quad (7.69)$$

Podemos observar que a função objetivo do problema (7.69) é análoga ao EMQR, onde o parâmetro de regularização $\mu = 1/\lambda$. Dessa forma, o parâmetro λ atua de forma inversa ao parâmetro de regularização do EMQR. Na formulação (7.69), a restrição $\xi(t) \geq 0$ é desnecessária, uma vez que $\xi(t)^2 \geq 0$. A formulação dual do problema de ajuste de parâmetros é escrita como [593]:

$$\begin{aligned} \min_{\boldsymbol{\theta}} L_d(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T \hat{\mathbf{K}} \boldsymbol{\theta} - \boldsymbol{\theta}^T \mathbf{V}^T \\ \text{sujeito a } \mathbf{1} \boldsymbol{\theta}^T = 0 \text{ e } v(t)\theta(t) \geq 0, \quad t = 1, \dots, N \end{aligned} \quad (7.70)$$

onde $\hat{\mathbf{K}} = \mathbf{K} + \frac{\mu}{2} \mathbf{I}$ é a matriz de núcleo ajustada pela adição do valor $\frac{\mu}{2}$ nos elementos da diagonal principal e $\mathbf{I}$ é a matriz identidade, como na regularização de Tikhonov (cf. equação (7.28)).

O valor do parâmetro de regularização λ no modelo SVM tem grande influência no resultado do modelo, como mostra a Figura 7.13. Um valor grande de λ/N prioriza a minimização do erro de classificação e um valor pequeno de λ/N prioriza a maximização da margem. Como resultado do processo de otimização, um valor pequeno de λ/N produz um maior número de vetores de suporte, em azul na Figura 7.13.

Os algoritmos de solução do modelo SVM com margem flexível são essencialmente os mesmos discutidos acima para o modelo SVM. Entretanto, no caso não separável, há a necessidade de se ajustar o parâmetro λ , o que, em geral, é feito em validação cruzada, como na seleção de estrutura de modelos RBF. Dessa forma, apesar das suas características teóricas, o modelo SVM, na prática, depende da seleção do parâmetro de regularização λ , que determina a capacidade do modelo.

Há diversas extensões do modelo SVM que seguem a ideia de utilizar o termo correspondente ao erro na função de custo do problema de ajuste de parâmetros. No caso da regressão, o modelo ε -SVM busca uma função de aproximação que esteja, no máximo, a uma distância ε dos registros do conjunto de treinamento [494]. Há algumas variantes do modelo SVM para regressão de acordo com a função de custo escolhida para penalizar registros fora da “margem” definida pelo parâmetro ε [478].

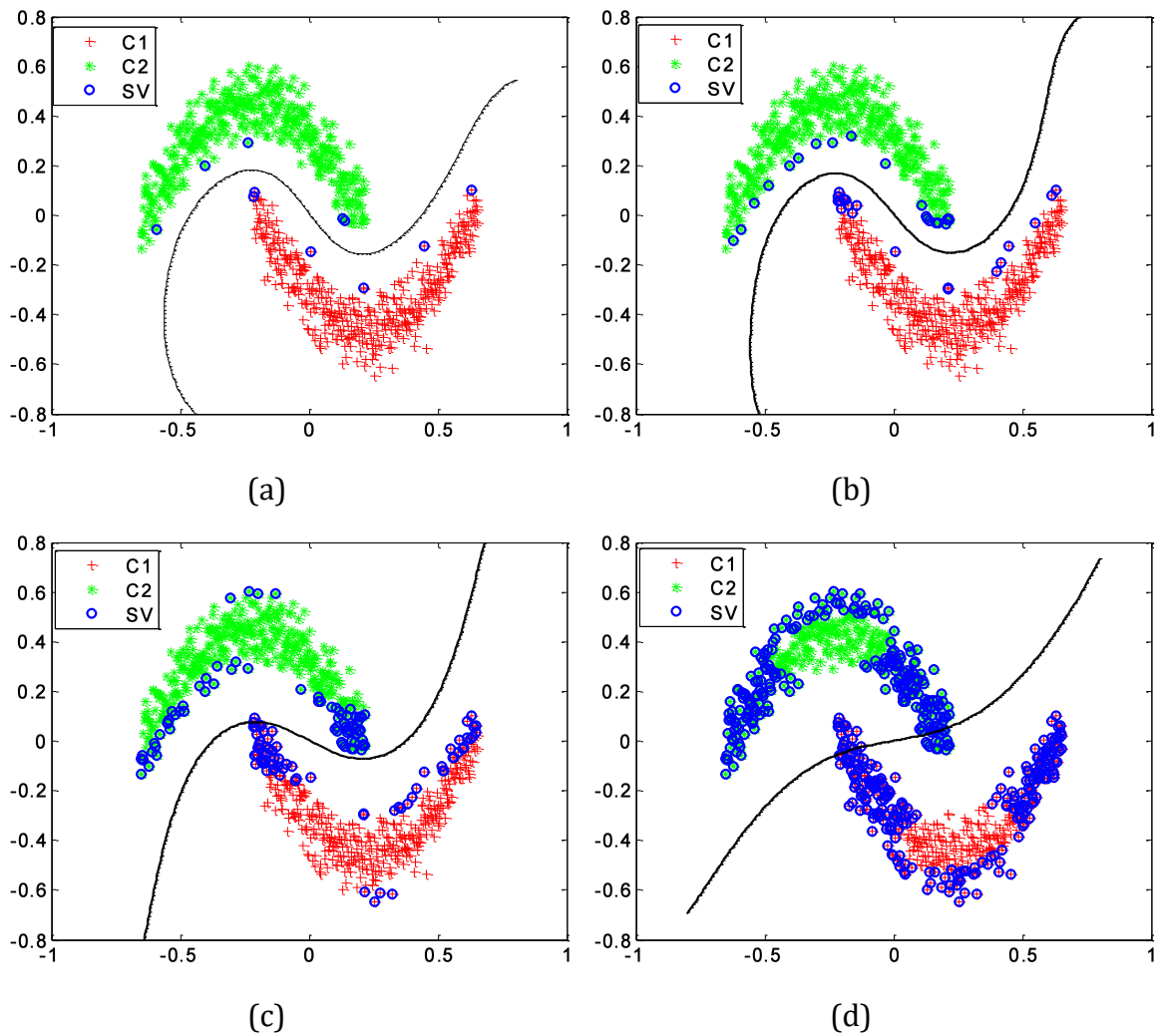


Figura 7.13: Efeito do parâmetro de regularização no modelo SVM: (a) $\lambda/N = 10$; (b) $\lambda/N = 1$; (c) $\lambda/N = 0.1$; (d) $\lambda/N = 0.01$.

O modelo SVM também é utilizado para estimativa de densidades e detecção de *outliers* no problema de classificação de uma classe [303] [531]. Nesse caso, o modelo busca estabelecer os limites de uma região fechada que contenha a maior parte dos pontos do conjunto de treinamento. Na formulação do problema de ajuste de parâmetros, uma função de erro que penaliza os registros fora da região definida pelo modelo é incluída na função de custo [478].

O modelo SVM transdutivo utiliza registros não classificados para melhorar o desempenho do modelo [533][534]. Esta abordagem é chamada de aprendizado semi supervisionado, uma vez que utiliza registros classificados e não classificados para o ajuste

[82]. Esse tipo de abordagem tem aplicações em mineração de textos e em bioinformática, uma vez que, nesses casos, há poucos registros categorizados e muitos registros sem classificação.

Há uma extensa bibliografia sobre modelos à base de funções de núcleo e SVM. A referência mais completa é o livro de Schölkopf e Smola [478]. Outras referências mais recentes são Huang e colaboradores [269] e Steinwart e Christmann [504]. Nesta seção, apresentamos apenas os conceitos mais importantes do modelo SVM e a formulação do problema de ajuste de parâmetros com uma abordagem similar aos modelos RBF.

7.4 Aplicações

Modelos RBF e SVM têm sido extensivamente utilizados em diversas aplicações. Nesta seção apresentamos os resultados com os testes benchmark e uma abordagem para determinação ótima do número de funções de base em modelos RBF.

7.4.1 Testes *benchmark*

Os conjuntos de dados dos testes *benchmark* apresentados na seção 5.4.1 foram utilizados para avaliação dos modelos RBF e SVM. Para garantir a compatibilidade com os resultados anteriores, foram utilizados os mesmos conjuntos de dados e as mesmas partições no processo de validação cruzada.

Oito configurações dos modelos RBF e SVM foram avaliadas, mas apenas o melhor resultado para cada tipo de modelo é apresentado na Tabela 7.2. O número de unidades RBF foi variado entre oito e 64 com parâmetro de dispersão fixo em $\sigma = 1.0$. No modelo SVM, foi utilizada a função de núcleo gaussiano com quatro opções para o parâmetro λ/N : (0.1, 1.0, 10, 100). Os resultados dos modelos bayesiano simples e do modelo linear de segunda ordem são os mesmos apresentados na seção 5.4.1 e foram incluídos na Tabela 7.2 para comparação.

A Tabela 7.2 mostra que todos os algoritmos foram predominantes em algum exemplo e não há nenhum algoritmo que supera todos os demais na maioria dos testes. Esse é um exemplo do que se conhece pela expressão “não existe almoço grátis”, em que um algoritmo não pode ser essencialmente superior a outro em todos os problemas [565].

Na prática, o analista deve explorar o maior número de soluções possível até encontrar a que obtém o melhor desempenho para os objetivos que o modelo se propõe.

Tabela 7.2: Resultados dos testes *benchmark*

<i>Benchmark</i>	Bayesiano		M. Lin. 2		RBF (8-64)		SVM - RBF	
	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>
Íris	98.00	0.9850	95.33	0.9650	95.33	0.9650	96.67	0.9750
Balance	91.68	0.9469	91.36	0.9171	88.32	0.8908	89.28	0.9112
Diabetes	72.92	0.7271	77.21	0.7151	75.91	0.7120	73.18	0.7334
Cancer	95.02	0.9559	97.22	0.9709	97.36	0.9768	96.19	0.9688
Stat. Heart	84.07	0.8433	84.44	0.8375	83.70	0.8342	77.78	0.7800
Wine	99.44	0.9958	99.44	0.9958	94.94	0.9599	82.58	0.8697
Parkinson	83.08	0.8325	86.15	0.8099	90.77	0.8616	91.28	0.8686
Ionosphere	86.89	0.8803	89.46	0.8602	93.45	0.9303	88.60	0.9041
Spambase*	90.11	0.8916	91.37	0.9006	88.40	0.8727	74.80	0.6953
Sonar	72.60	0.7166	80.29	0.8010	81.25	0.8048	57.69	0.5464

7.4.2 Algoritmo da decomposição espectral

Uma questão fundamental para a definição do modelo RBF é a determinação do número de funções de base utilizadas pelo modelo. O algoritmo de agrupamento espectral de Ng, Jordan e Weiss [415], apresentado na seção 6.4.1, pode ser utilizado para o cálculo de uma estimativa do número de RBF [170].

Os algoritmos de agrupamento espectral têm uma relação interessante com funções de núcleo, uma vez que a matriz de adjacências no agrupamento baseado em grafos é uma matriz de núcleo.

Na seção 6.4.1, foram apresentados três algoritmos de agrupamento espectral a partir da matriz de adjacências $\mathbf{A} \in [0,1]^{N \times N}$. No caso de função de similaridade gaussiana, os elementos da matriz de adjacências são calculados como (cf. equações (6.11) e (6.12)):

$$a_{ij} = \begin{cases} \exp\left(-\frac{\|\mathbf{x}(i) - \mathbf{x}(j)\|^2}{2\sigma^2}\right), & \text{se } i \neq j \\ 0, & \text{se } i = j \end{cases} \quad (7.71)$$

onde σ é o parâmetro de dispersão do cálculo da similaridade.

No algoritmo do corte mínimo normalizado [415] (cf. Algoritmo 6.3c), o agrupamento é calculado pelo algoritmo k-médias aplicado aos autovetores da matriz de adjacências normalizada $\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, onde $\mathbf{D}$ é uma matriz diagonal em que cada elemento $d_{ii} = \sum_j a_{ij}$ é o grau do vértice (registro) correspondente. No caso de um grafo com K componentes, definido por uma matriz de adjacências binária, isto é, $a_{ij} \in \{0,1\}$, a matriz $\hat{\mathbf{A}}$ tem K autovalores unitários: $\hat{\lambda}_i = 1, i = 1, \dots, K$. Essa propriedade pode ser estendida para o caso geral em que $a_{ij} \in [0,1]$ e o número de autovalores positivos define o número de regiões densas de dados, que pode ser usado como um indicativo do número de regras a ser utilizado pelo modelo RBF [168][170].

Um exemplo sintético é apresentado na Figura 7.14 [168], onde a coluna da esquerda mostra um conjunto de dados com quatro grupos bem separados entre si e a coluna da direita, um conjunto de dados com quatro grupos menos distinguíveis.

A Figura 7.14b e a Figura 7.14e mostram o *data image* das matrizes $\hat{\mathbf{A}}$ calculadas para os dois exemplos pela similaridade gaussiana com $\sigma = 1$. Podemos observar que, no exemplo da esquerda, o contraste é maior, indicando maior variação da similaridade entre os registros no mesmo grupo em relação aos de grupos diferentes.

A Figura 7.14c e a Figura 7.14f mostram os autovalores da matriz $\hat{\mathbf{A}}$. No exemplo da esquerda, a matriz $\hat{\mathbf{A}}$ tem quatro autovalores; no exemplo da direita, seis, embora os dois últimos sejam muito pequenos. Dessa forma, o número de autovalores da matriz de adjacências reduzida representa uma estimativa do número de regiões densas num conjunto de dados e, portanto, pode ser utilizado para definir o número de funções de base para um modelo RBF.

O Algoritmo 7.1 calcula o número e os centros de funções gaussianas pelo método da decomposição espectral. O parâmetro δ define um limite para a identificação de autovalores positivos a fim de evitar autovalores muito pequenos. Em [168][170] foi utilizado $\delta = 0.01$. O número de funções de base é definido no passo 05 pelo número de autovalores maiores que δ . No passo 06, uma partição dos registros é calculada por um algoritmo de agrupamento, que define os centros de funções de base gaussianas pela média aritmética dos registros alocados ao mesmo grupo.

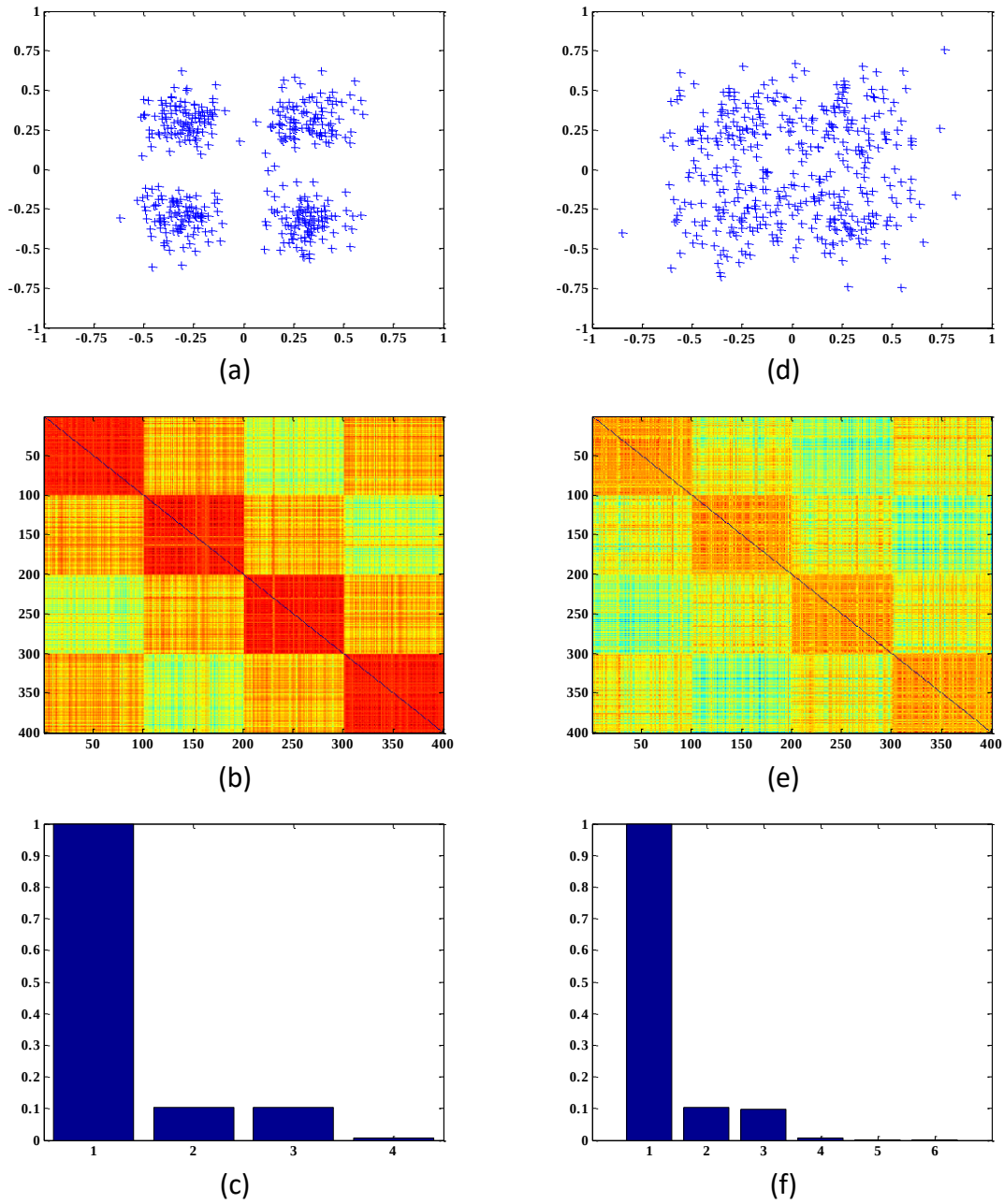


Figura 7.14: Exemplos sintéticos em 2D: (a) (d) conjuntos de dados; (b) (e) *data image* da matriz $\hat{\mathbf{A}}$ correspondente; (c) (f) autovalores positivos. Fonte: [168]

O cálculo da partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$ no passo 06 do Algoritmo 7.1 pode ser feito por diferentes algoritmos de agrupamento gerando variantes. Em [170] as seguintes opções foram consideradas:

- Algoritmo 7.1a (SM): em que a partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$ é calculada pelo algoritmo de agrupamento espectral (Algoritmo 6.3c);

- Algoritmo 7.1b (KM): em que os centros $\mathbf{\Omega} = [\mathbf{\omega}_1^T, \dots, \mathbf{\omega}_K^T]$ são calculados diretamente pelo algoritmo k-médias (Algoritmo 6.1).

Algoritmo 7.1: Algoritmo de indução de regras por decomposição espectral. Fonte: [170]

Entrada:	Conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, parâmetro de dispersão σ , <i>threshold</i> δ
Saída:	Matriz de centros $\mathbf{\Omega} = [\mathbf{\omega}_1^T, \dots, \mathbf{\omega}_K^T]$
01	Início
02	Calcular a matriz $\mathbf{A}$, como (7.71)
03	Calcular $\hat{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, onde $d_{ii} = \sum_i a_{ij}$
04	Calcular $\hat{\mathbf{A}} \hat{\mathbf{U}} = \hat{\mathbf{U}} \hat{\mathbf{\Lambda}}$, onde $1 = \hat{\lambda}_1 \geq \hat{\lambda}_2 \geq \dots \geq \hat{\lambda}_N$
05	Definir $n \leftarrow K: \hat{\lambda}_i \geq \delta, i = 1, \dots, K$
06	Calcular partição $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$ e centros $\mathbf{\Omega} = [\mathbf{\omega}_1^T, \dots, \mathbf{\omega}_K^T]$
07	Retorna $\mathbf{\Omega}$
08	Fim

No caso do Algoritmo 7.1a, o algoritmo k-médias é aplicado nos registros definidos pela matriz $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_n]$, formada pelos n autovetores associados aos n maiores autovalores da matriz $\hat{\mathbf{A}}$, calculados no passo 04. É possível utilizar os autovetores normalizados pela equação (6.27). Tendo em vista que, nesse caso, o algoritmo k-médias é aplicado num conjunto de dimensão n para encontrar n grupos, a inicialização pode ser proporcional à matriz identidade, eliminando os problemas de inicialização aleatória do k-médias. No Algoritmo 7.1b é o modelo RBF padrão apresentado na seção 7.2.3 em que o algoritmo k-médias é aplicado diretamente às variáveis de entrada.

Em [170], os modelos RBF foram executadas com diferentes configurações de estrutura e com a estimativa de número centros pelo Algoritmo 7.1. Os modelos foram comparados em diversos testes benchmark de problemas de classificação e regressão.

A Figura 7.15, adaptada de [170], que mostra a comparação da solução apresentada no Algoritmo 7.1 com uma busca exaustiva pelo número de funções de base. Os resultados mostram que o aumento do número de funções de base nem sempre resulta em aumento de desempenho e diferentes resultados podem acontecer em diferentes conjuntos de dados. O resultado do modelo com número de funções de base pelo número de autovalores positivos (cf. passo 04 do Algoritmo 7.1) é mostrado na Figura 7.15 com

um asterisco vermelho. A exceção do conjunto de dados *Heart*, na Figura 7.15b, nos demais exemplos a solução definida pelos autovalores é próxima da melhor solução.

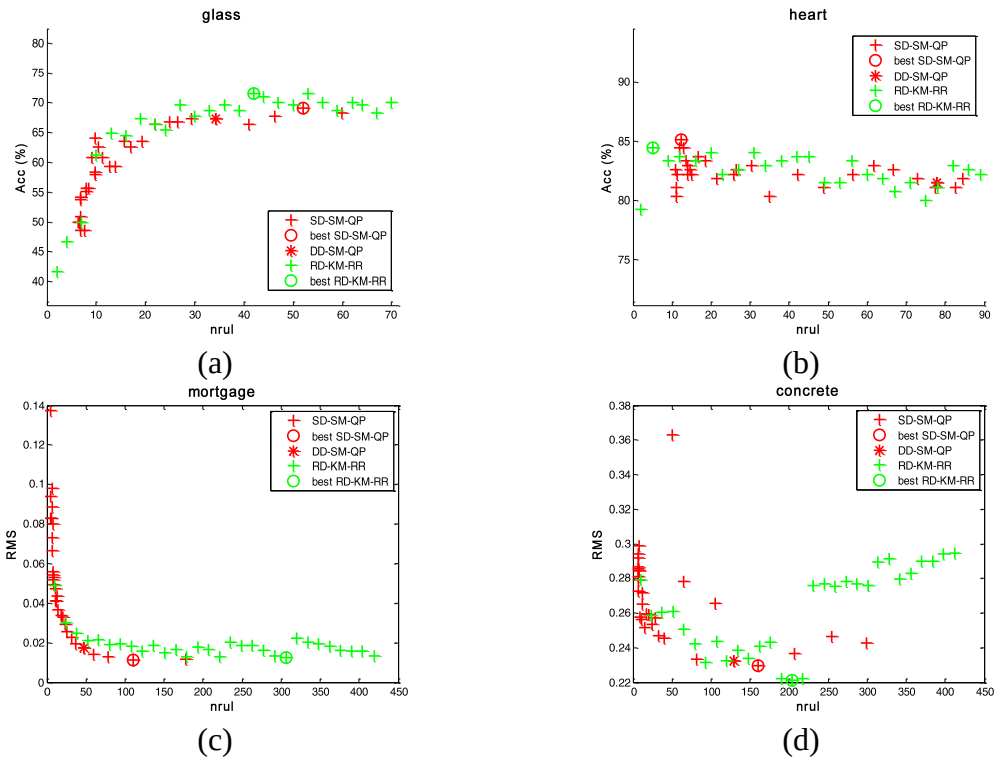


Figura 7.15: Resultados dos modelos com diferentes combinações de configurações de parâmetros em função do número de RBF (nrul): (a) e (b) problemas de classificação; (c) e (d) problemas de regressão. Fonte: [170].

7.5 Comentários Finais

Segundo Domingos [137], um modelo de aprendizado de máquina é obtido pela combinação de três componentes: representação, função de avaliação e otimização.

A representação de modelos foi discutida neste capítulo através da formulação do problema de aproximação de funções por duas abordagens diferentes. A representação explícita por funções de base é utilizada no modelo RBF e a representação implícita por funções de núcleo é empregada no modelo SVM. Modelos funcionalmente equivalentes podem ser obtidos com as duas representações. O ajuste de parâmetros nos dois casos é feito pela solução de um problema de otimização quadrática com uma função de avaliação similar, mas de dimensão diferente.

A aproximação de funções por funções de base radial (RBF) é uma composição de modelos locais em que a saída é uma combinação linear do vetor de características, cujos elementos são os valores das RBF. O ajuste de parâmetros pode ser feito eficientemente combinando uma análise de agrupamentos e um problema de otimização convexo para o ajuste de parâmetros de saída.

O modelo SVM utiliza a representação por funções de núcleo e o ajuste de parâmetros é feito pela minimização do risco estrutural, estimado pela margem de separação entre as classes. O modelo SVM em problemas separáveis encontra a estrutura ótima. Em problemas não separáveis, a função de custo para o ajuste dos parâmetros do modelo SVM com margem flexível é similar ao EMQR.

O ajuste de parâmetros do modelo SVM utiliza o mesmo algoritmo de otimização quadrática (QP) que formulação do ajuste de parâmetros dos modelos RBF. A diferença é a dimensão do vetor de parâmetros que, no caso do modelo SVM, é o número de registros do conjunto de treinamento e, no caso do modelo RBF, é a dimensão do vetor de características. O modelo SVM trabalha com variáveis de entrada de alta dimensionalidade, mas o ajuste de parâmetros pode ser lento em amostras muito grandes. O modelo RBF é mais robusto ao tamanho da amostra, mas perde desempenho em problemas de alta dimensionalidade.

O tema de aproximação de funções é extenso e há diversos modelos similares e relacionados aos apresentados neste capítulo como, por exemplo, o modelo de misturas de gaussianas [154] e o algoritmo de maximização da expectativa (EM)* [124]. Como nos capítulos anteriores, o objetivo não foi uma abordagem exaustiva sobre o tema, mas uma visão balanceada dos métodos mais encontrados em *softwares* e bibliotecas de aprendizado de máquinas.

* *Expectation-Maximization*

8 SISTEMAS *FUZZY*

Um conjunto identifica seus elementos por uma propriedade característica, que pode ser simplesmente o fato de pertencer ao conjunto. Os axiomas do terceiro excluído e da não contradição, apresentados no Capítulo 6, estabelecem que a interseção entre um conjunto e o seu complemento é vazia e a união dos dois é o próprio universo. Esses axiomas formam a base da teoria dos conjuntos e estão intrinsecamente ligados à definição da noção de pertinência binária, definida pela função característica.

No mundo real, certos conceitos não podem ser modelados por valores binários. O matemático e filósofo inglês Bertrand Russel foi o primeiro a explorar formalmente as limitações da teoria de conjuntos para a representação do mundo real no início do século XX. Em determinadas circunstâncias, a representação binária de pertinência pode levar a um paradoxo. Há várias versões para o paradoxo formalizado por Russel, a mais conhecida é a do barbeiro, descrita a seguir:

“Numa cidade havia um único barbeiro e todos os homens seguiam estritamente duas regras mutuamente exclusivas: a) fazem a barba em si mesmos ou b) fazem a barba no barbeiro.”

O paradoxo ocorre quando se pergunta “quem faz a barba do barbeiro?” Sendo o barbeiro um homem, ele satisfaz simultaneamente às duas regras, que são mutuamente exclusivas. O barbeiro pertence, então, ao conjunto dos homens que se barbeiam e ao seu complemento, o conjunto dos homens que fazem a barba no barbeiro, infringindo, assim, o axioma da não contradição e criando o paradoxo.

O paradoxo de Russel seria apenas uma anedota se não trouxesse à tona a incapacidade da teoria dos conjuntos de representar situações simples do mundo real. Alguns anos após os trabalhos de Russel, o filósofo polonês Jan Łukasiewicz apresenta a sua lógica de múltiplos valores para permitir a representação de conceitos indeterminados entre o verdadeiro e o falso. Em 1965, Zadeh cria a **teoria de conjuntos fuzzy*** com a publicação do artigo intitulado “*fuzzy sets*” [585].

O artigo de Zadeh apresenta uma extensão do conceito de pertinência binária, definido pela função característica na equação (6.1), ao conceito de pertinência *fuzzy*, no qual a pertinência é um valor no intervalo $[0, 1]$. O artigo também apresenta as operações básicas de conjuntos estendidas ao novo conceito de pertinência e o operador composição-projeção, que é o método básico de dedução da lógica *fuzzy*.

O conceito de pertinência gradual é essencial para a representação de conceitos vagos ou imprecisos como “João é alto”. O artigo de Zadeh e o conceito de conjuntos *fuzzy* permaneceram circunscritos aos meios acadêmicos até meados da década de 1970, quando houve um grande desenvolvimento na teoria de conjuntos *fuzzy*. Em 1973, Zadeh introduz o conceito de variáveis linguísticas [586], que teria grande influência no desenvolvimento dos sistemas de controle *fuzzy*. Em 1978, no primeiro artigo da revista *Fuzzy Sets and Systems*, Zadeh apresenta a **teoria das possibilidades** como uma teoria para tratamento de incertezas e informações incompletas [590]. Apesar de ser uma teoria formal [140], a teoria das possibilidades gerou controvérsias [139][591].

Mamdani e Assilian [363] apresentam a primeira aplicação prática da teoria de conjuntos *fuzzy* em 1975, para controle de uma planta com uma caldeira e uma máquina a vapor. O sistema de controle *fuzzy* reproduz o raciocínio humano e o modelo é representado por regras. A partir da década de 1980, diversas aplicações industriais de controle *fuzzy* começam a aparecer, principalmente no Japão, como o sistema de controle do metrô de Sendai, desenvolvido pela Hitachi, e o sistema de foco automático de câmeras Canon. Em 1990, os sistemas de controle *fuzzy*, ou simplesmente sistemas *fuzzy*, já estavam consolidados na literatura [335][336].

* Há muitas traduções para o termo *fuzzy* em português, como “difuso” ou “nebuloso”. O termo *fuzzy* se consolidou na comunidade brasileira de inteligência computacional.

Em 1985, Takagi e Sugeno apresentam o modelo que veio a ser conhecido como sistema *fuzzy* de Takagi-Sugeno (TS)* [515]. O sistema *fuzzy* TS é amplamente empregado em inteligência computacional, sendo utilizado em diversas aplicações, como diagnóstico [277][431] e ciência de dados [271]. O sistema *fuzzy* TS são um tipo de modelo RBF, em que se pode interpretar as unidades RBF como conceitos linguísticos, permitindo uma interpretação do modelo.

A teoria de conjuntos *fuzzy* tem outros desdobramentos além dos sistemas *fuzzy*, que podem ser encontrados no livro de Klir e Yuang [310], bem como em referências clássicas como Pedrycz e Gomide [432][433] e Ross [461]. O foco deste capítulo é a utilização de sistemas *fuzzy* em problemas de ciência de dados, principalmente em análise de agrupamentos, classificação e regressão. A apresentação é limitada aos principais algoritmos, buscando uma formalização que unifique os conceitos discutidos nos capítulos anteriores. O capítulo termina com a discussão de aplicações reais de sistemas *fuzzy*.

8.1 Conjuntos Fuzzy

Um conjunto *fuzzy* $\mathcal{A}$ é definido por sua **função de pertinência**:

$$\mu_{\mathcal{A}}: \mathcal{X} \rightarrow [0,1] \quad (8.1)$$

onde o conjunto $\mathcal{X}$ é chamado de domínio ou universo de discurso.

Um conjunto *fuzzy* também é definido pela enumeração de todos os seus elementos e os respectivos valores de pertinência como $\mathcal{A} = \{(x, \mu_{\mathcal{A}}(x)), x \in \mathcal{X}\}$, onde $\mu_{\mathcal{A}}(x)$ é a função de pertinência como definida em (8.1).

Na maioria das aplicações, um conjunto *fuzzy* é definido sobre um universo de discurso numérico, discreto ou contínuo, no qual o conceito correspondente ao conjunto *fuzzy* é representado. Por exemplo, a expressão “João é alto”, representa o fato que João pertence ao conjunto de pessoas altas. O conceito de “alto” é definido pelo valor da altura e, portanto, o universo de discurso é o conjunto de valores de altura de um ser humano, que pode ser representado (em metros) por $\mathcal{X} = \{x \in \mathcal{R}, 0 < x < 2,5\}$. Entretanto, um conjunto *fuzzy* também pode ser definido sobre um domínio não numérico, como será visto na seção 8.2.3.

* Em algumas referências, esse modelo também é citado como Takagi-Sugeno-Kang (ou TSK) em função do colaborador G. T. Kang.

O conceito “alto” não pode ser adequadamente representado por uma função característica binária, pois “alto” é um conceito vago e relativo. A Figura 8.1a apresenta a função característica binária correspondente ao conjunto “alto”, considerando o limite de 1,70 m para a caracterização de pessoas altas. Observe que, nesse caso, há uma mudança brusca em função de uma variação pequena do valor de altura. Por exemplo, alguém com 1,69 m não é alto, enquanto uma pessoa com 1,71 m é alta. Na Figura 8.1b, a representação do conceito “alto” por um conjunto *fuzzy* permite uma variação gradual da pertinência ao conjunto correspondente ao conceito “alto”.

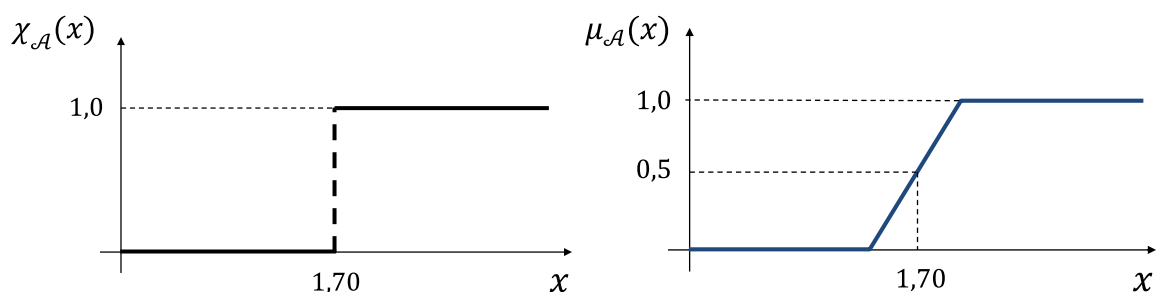


Figura 8.1: Representação do conceito de “alto”:
(a) função característica; (b) função de pertinência. Adaptado de [433].

Há três interpretações principais para a função de pertinência de um conjunto *fuzzy* [141]: similaridade, incerteza e preferência. As três interpretações são complementares e um conjunto *fuzzy* representa simultaneamente as três visões.

A interpretação de similaridade é a mais intuitiva e foi apresentada no capítulo anterior com modelos RBF. Nesse caso, a função de pertinência representa a similaridade entre os elementos do universo e os valores mais representativos do conceito associado ao conjunto *fuzzy*, que têm pertinência mais alta. No caso de conjuntos *fuzzy* normalizados, em que ao menos um elemento do universo tem pertinência igual a 1, os valores de pertinência unitária são os **protótipos** do conjunto *fuzzy*. Protótipos são muito úteis no desenvolvimento de sistemas *fuzzy*, pois permitem que as relações sejam definidas em função dos valores dos protótipos e o sistema *fuzzy* (cf. seção 8.2) realiza uma aproximação dos valores intermediários.

Na interpretação de incerteza, um conjunto *fuzzy* representa uma medida imprecisa em que os valores com maior pertinência são mais plausíveis. Nesse caso, a função de pertinência representa um **número fuzzy** [433].

Na interpretação de preferência, um conjunto *fuzzy* representa uma restrição flexível aos valores de uma variável e sua função de pertinência denota uma preferência desses valores. Essa interpretação é utilizada em aplicações de apoio à decisão [575].

A representação de conceitos através de conjuntos *fuzzy* também pode ser explorada através de cores. A Figura 8.1 mostra a paleta de 256 cores utilizada para representação dos valores de pertinência nas figuras deste capítulo. As cores variam gradualmente do azul (valor 0.0) ao vermelho (valor 1.0), passando pelo verde (valor 0.5).

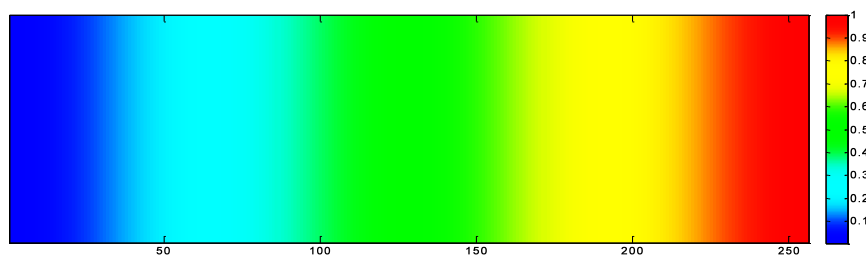


Figura 8.2: Paleta de 256 cores para representação de valores de pertinência.

A função característica binária de um conjunto é o caso particular de uma função de pertinência *fuzzy* em que apenas os extremos $\{0,1\}$ são utilizados. Esse fato demonstra que a capacidade de representação de um conjunto *fuzzy* é muito maior do que a capacidade de representação de um conjunto clássico.

Conjuntos *fuzzy* definidos em universos discretos podem ter uma representação vetorial no hipercubo unitário, cujas coordenadas representam a pertinência a cada elemento do universo [320]. Os vértices do hipercubo representam o *power set*^{*} do universo de discurso. Essa representação mostra um conjunto *fuzzy* como um ponto no interior do hipercubo. O conjunto de todos os conjuntos *fuzzy* que podem ser definidos no universo é chamado de *fuzzy power set*.

Suponha um problema de classificação com $m = 2$ classes representadas pelos elementos do universo $\mathcal{X} = \{C_1, C_2\}$. A classificação de um registro pode ser representada pelo conjunto *fuzzy* $\mathcal{A} = \{(C_1, 0.3), (C_2, 0.7)\}$ (cf. Figura 8.3a). Nesse caso, o conjunto *fuzzy* $\mathcal{A}$ é representado pelo vetor $(0.3, 0.7)$ no plano (hipercubo) mostrado na Figura 8.3b, cujos vértices são os elementos do *power set* $2^{\mathcal{X}} = \{\emptyset, \{C_1\}, \{C_2\}, \{C_1, C_2\}\}$ e a linha diagonal é o lugar geométrico dos pontos cuja soma das coordenadas é igual a 1.

^{*} O *power set* é o conjunto de todos os subconjuntos.

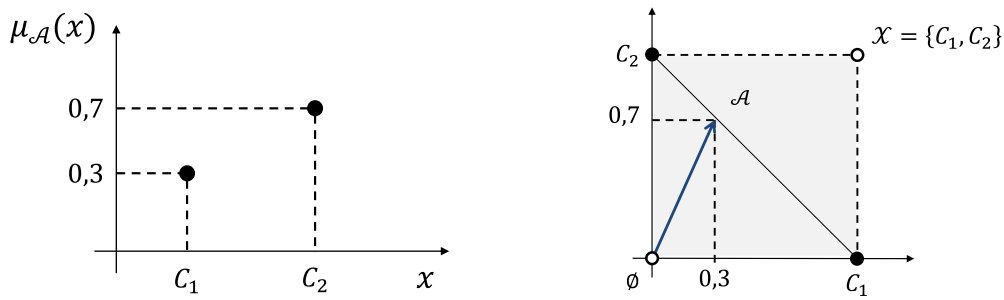


Figura 8.3: Representação de conjuntos *fuzzy* em universos discretos: (a) função de pertinência; (b) representação vetorial.

Há diversos tipos de funções de pertinência; as mais utilizadas em sistemas *fuzzy* para ciência de dados são apresentadas a seguir.

8.1.1 Funções de pertinência

Um conjunto *fuzzy* é definido por sua função de pertinência, mas a escolha da função não é preponderante em relação à sua capacidade de representação. Na prática, a função de pertinência geralmente é definida com base em requisitos computacionais.

Há várias funções de pertinência descritas na literatura [433]. As funções mais utilizadas representam bem a noção de similaridade ao protótipo e podem ser consideradas como funções de base para aproximação de funções em sistemas *fuzzy*:

- **Função gaussiana:**

$$gaus(x) = \exp\left(-\frac{(x - \omega)^2}{\rho^2}\right) \quad (8.2)$$

- **Função triangular:**

$$tri(x) = \max\left(\min\left(\frac{x - \omega_1}{\omega_2 - \omega_1}, \frac{\omega_3 - x}{\omega_3 - \omega_2}\right), 0\right) \quad (8.3)$$

Na função gaussiana, o parâmetro ω representa o valor central e ρ a dispersão. Na função triangular, os elementos do conjunto de parâmetros $\{\omega_1, \omega_2, \omega_3\}$ representam os vértices do triângulo. A Figura 8.4 mostra exemplos de funções de pertinência triangular e gaussiana. A função gaussiana é sempre simétrica em relação ao centro e a triangular pode ser assimétrica. As funções definidas na equações (8.2) e (8.3) são normalizadas, isto é, o elemento central (protótipo) tem valor de pertinência igual a 1.

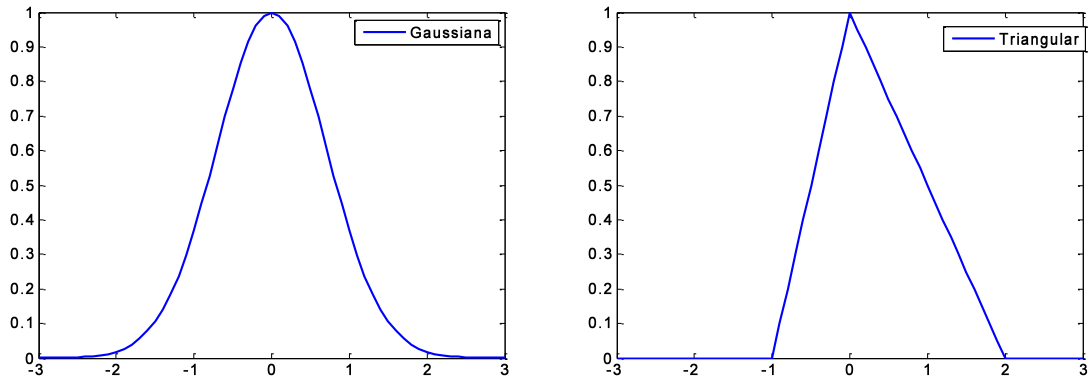


Figura 8.4: Funções de pertinência: (a) gaussiana; (b) triangular.

A representação de uma sequência ordenada de valores através de conjuntos *fuzzy* é feita por uma **partição fuzzy** $\mathbb{A} = \{\mathcal{A}_1, \dots, \mathcal{A}_n\}$, onde n é o número de conjuntos *fuzzy* da partição. Numa partição *fuzzy*, para todo elemento do universo, há ao menos um conjunto *fuzzy* cuja função de pertinência é maior que zero:

$$\forall x \in \mathcal{X}, \exists \mathcal{A}_i \in \mathbb{A}, \mu_{\mathcal{A}_i}(x) > 0 \quad (8.4)$$

Os conjuntos *fuzzy* de uma partição *fuzzy* representam conceitos ordenados que traduzem os valores numéricos em conceitos linguísticos. Por exemplo, se uma variável representa a temperatura, uma partição *fuzzy* pode representar uma sequência de valores linguísticos como $\{\text{"baixa"}, \text{"média"}, \text{"alta"}\}$.

Uma partição *fuzzy* é uma função vetorial que calcula o vetor de características de um sistema *fuzzy*, chamado de **descrição fuzzy** [190]:

$$\mu_{\mathbb{A}}: \mathcal{X} \rightarrow [0,1]^n \quad (8.5)$$

onde $\mu_{\mathbb{A}}(x) = (\mu_{\mathcal{A}_1}(x), \dots, \mu_{\mathcal{A}_n}(x))$ é a descrição *fuzzy* de um valor $x \in \mathcal{X}$.

Numa **partição fuzzy forte** todos os conjuntos *fuzzy* são normalizados e a soma dos valores de pertinência é sempre igual a 1:

$$\forall \mathcal{A}_i \in \mathbb{A}, \sum_{i=1}^n \mu_{\mathcal{A}_i} = 1 \quad (8.6)$$

Qualquer partição *fuzzy* pode ser padronizada para uma partição *fuzzy* forte pela transformação sobre cada função de pertinência como:

$$\hat{\mu}_{\mathcal{A}_i}(x) = \frac{\mu_{\mathcal{A}_i}(x)}{\sum_{j=1}^n \mu_{\mathcal{A}_j}(x)}, \quad i = 1, \dots, n \quad (8.7)$$

Uma partição *fuzzy* pode ser definida por diversos tipos de funções de pertinência. As partições *fuzzy* mais usuais são as seguintes:

▪ **Partição *fuzzy* gaussiana:**

$$gaus_i(x) = \exp\left(-\frac{(x - \omega_i)^2}{\rho_i^2}\right), \quad i = 1, \dots, n \quad (8.8)$$

▪ **Partição *fuzzy* triangular:**

$$tri_i(x) = \begin{cases} \max\left(\min\left(1, \frac{\omega_{i+1} - x}{\omega_{i+1} - \omega_i}\right), 0\right), & i = 1 \\ \max\left(\min\left(\frac{x - \omega_{i-1}}{\omega_i - \omega_{i-1}}, \frac{\omega_{i+1} - x}{\omega_{i+1} - \omega_i}\right), 0\right), & i = 2, \dots, n-1 \\ \max\left(\min\left(\frac{x - \omega_{i-1}}{\omega_i - \omega_{i-1}}, 1\right), 0\right), & i = n \end{cases} \quad (8.9)$$

▪ **Partição *fuzzy* FCM:**

$$fcm_i(x) = \frac{(x - \omega_i)^{\frac{2}{\gamma-1}}}{\sum_{j=1}^n (x - \omega_j)^{\frac{2}{\gamma-1}}}, \quad i = 1, \dots, n \quad (8.10)$$

A partição *fuzzy* gaussiana é definida pelo vetor $\boldsymbol{\omega} = (\omega_1, \dots, \omega_n)$ dos centros das funções gaussianas e $\boldsymbol{\rho} = (\rho_1, \dots, \rho_n)$ das dispersões. A Figura 8.5 apresenta um exemplo de partição *fuzzy* gaussiana com vetor de centros $\boldsymbol{\omega} = (-10, -5, 0, 5, 10)$. Os elementos do vetor de dispersões foram calculados de forma que as funções adjacentes se cruzem no valor 0.5, como:

$$\rho_i = -\frac{1}{4 \log(0.5)} \frac{(\omega_n - \omega_1)}{n}, \quad i = 1, \dots, n \quad (8.11)$$

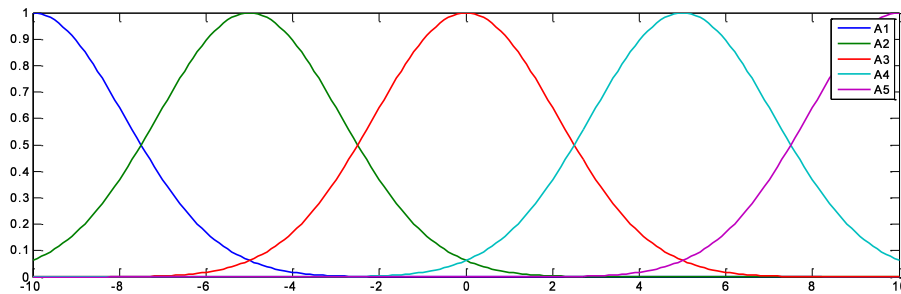


Figura 8.5: Partição *fuzzy* de funções gaussianas.

Uma partição triangular é uma partição *fuzzy* forte definida pelo vetor de protótipos $\omega = (\omega_1, \dots, \omega_n)$, onde ω_i representa o protótipo do conjunto *fuzzy* $\mathcal{A}_i$, que coincide com os vértices dos triângulos vizinhos. As bordas de uma partição *fuzzy* triangular são trapezoidais, de forma que, pela equação (8.9), $tri_i(x) = 1$ para $x < \omega_1$ ou $x > \omega_n$.

A Figura 8.6 mostra um exemplo de partição *fuzzy* de funções triangulares não simétricas com $\omega = (-8, -3, 0, 3, 8)$.

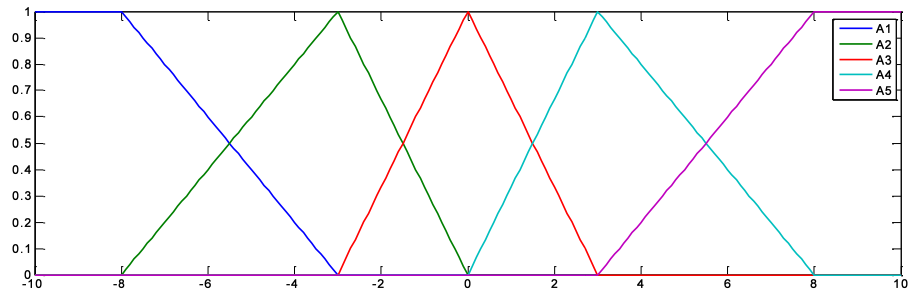


Figura 8.6: Partição *fuzzy* de funções triangulares.

A partição *fuzzy* FCM, definida pela equação (8.10), é calculada pelo algoritmo de agrupamentos *fuzzy* c-médias (FCM), apresentado na seção 8.3.1. A partição *fuzzy* FCM é ajustada por um parâmetro adicional $\gamma > 1$, que determina a forma das funções de pertinência. A Figura 8.7 apresenta a partição *fuzzy* FCM com $\omega = (-10, -5, 0, 5, 10)$ e $\gamma = 1.2$. O efeito do parâmetro de forma será discutido na seção 8.3.1.

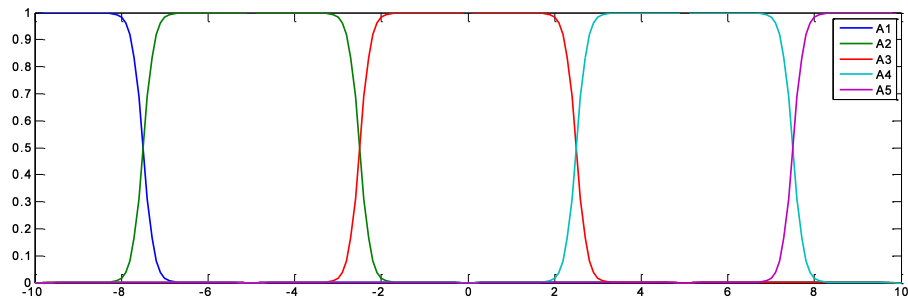


Figura 8.7: Partição *fuzzy* FCM com $\gamma = 1.2$.

A seguir, apresentamos as operações sobre conjuntos *fuzzy* que são realizadas sobre as funções de pertinência. Na seção 8.2.1, discutimos o conceito de variáveis linguísticas, que permite realizar o mapeamento de conceitos simbólicos.

8.1.2 Operadores *fuzzy*

Operações entre conjuntos *fuzzy* são realizadas sobre as funções de pertinência. As operações básicas são ilustradas na Figura 8.8 e calculadas pelos seguintes operadores:

- **Complemento:** o conjunto $\bar{\mathcal{A}} = \mathcal{X} - \mathcal{A}$ é calculado como:

$$\mu_{\bar{\mathcal{A}}}(x) = 1 - \mu_{\mathcal{A}}(x) \quad (8.12)$$

- **União:** o conjunto $\mathcal{C} = \mathcal{A} \cup \mathcal{B}$ é calculado como:

$$\mu_{\mathcal{C}}(x) = \max(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x)) \quad (8.13)$$

- **Interseção:** o conjunto $\mathcal{C} = \mathcal{A} \cap \mathcal{B}$ é calculado como:

$$\mu_{\mathcal{C}}(x) = \min(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x)) \quad (8.14)$$

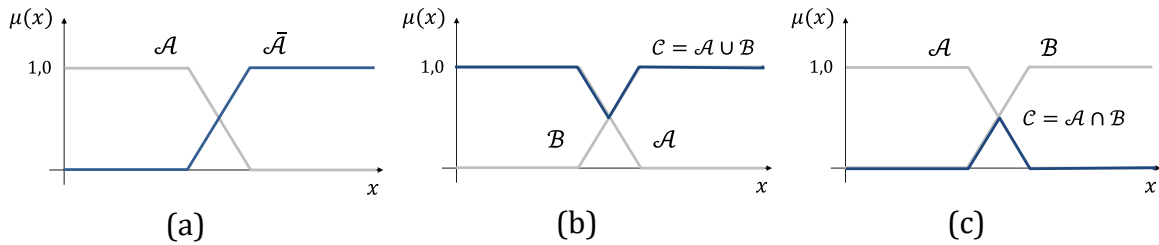


Figura 8.8: Operações com conjuntos *fuzzy*: (a) complemento; (b) união; (c) interseção.

Os operadores *fuzzy* definidos pelas equações (8.12) a (8.14) satisfazem às propriedades das operações entre conjuntos apresentadas na Tabela 8.1 e, por isso, são comumente chamados de operadores *standard*. As únicas propriedades das operações entre conjuntos que não são satisfeitas pelos operadores *fuzzy standard* são justamente a do terceiro excluído e da não contradição (cf. seção 6.1), uma vez que, por definição, os conjuntos *fuzzy* não satisfazem essas propriedades.

A flexibilidade criada pela violação dos axiomas do terceiro excluído e da não contradição permite a definição de operadores *fuzzy* generalizados como funções que têm semântica de complemento, união e interseção.

Os operadores generalizados para interseção são chamados de normas triangulares ou **t-normas** e os operadores generalizados para união são chamados de conormas triangulares ou **t-conormas** [310]. Os operadores generalizados são definidos a seguir.

- **T-norma:** a função monótona $T: [0,1] \times [0,1] \rightarrow [0,1]$ é uma t-norma se satisfaz as propriedades 1 a 3 da Tabela 8.1. A t-norma calcula a interseção generalizada $\mathcal{C} = \mathcal{A} \cap \mathcal{B}$ como:

$$\mu_{\mathcal{C}}(x) = T(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x)) \quad (8.15)$$

- **T-conorma:** a função monótona $S: [0,1] \times [0,1] \rightarrow [0,1]$ é uma t-conorma se satisfaz as propriedades 1 a 3 da Tabela 8.1. A t-conorma calcula a união generalizada $\mathcal{C} = \mathcal{A} \cup \mathcal{B}$ como:

$$\mu_{\mathcal{C}}(x) = S(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x)) \quad (8.16)$$

Tabela 8.1: Propriedades das operações básicas de conjuntos *fuzzy*

1. Condições de contorno	$\mathcal{A} \cup \emptyset = \mathcal{A}$ e $\mathcal{A} \cup \mathcal{X} = \mathcal{X}$ $\mathcal{A} \cap \emptyset = \emptyset$ e $\mathcal{A} \cap \mathcal{X} = \mathcal{A}$
2. Comutatividade	$\mathcal{A} \cup \mathcal{B} = \mathcal{B} \cup \mathcal{A}$ $\mathcal{A} \cap \mathcal{B} = \mathcal{B} \cap \mathcal{A}$
3. Associatividade	$(\mathcal{A} \cup \mathcal{B}) \cup \mathcal{C} = \mathcal{A} \cup (\mathcal{B} \cup \mathcal{C})$ $(\mathcal{A} \cap \mathcal{B}) \cap \mathcal{C} = \mathcal{A} \cap (\mathcal{B} \cap \mathcal{C})$
4. Distributividade	$\mathcal{A} \cup (\mathcal{B} \cap \mathcal{C}) = (\mathcal{A} \cup \mathcal{B}) \cap (\mathcal{A} \cup \mathcal{C})$ $\mathcal{A} \cap (\mathcal{B} \cup \mathcal{C}) = (\mathcal{A} \cap \mathcal{B}) \cup (\mathcal{A} \cap \mathcal{C})$
5. Idempotência	$\mathcal{A} \cup \mathcal{A} = \mathcal{A}$ $\mathcal{A} \cap \mathcal{A} = \mathcal{A}$
6. Dualidade (De Morgan)	$\overline{\mathcal{A} \cup \mathcal{B}} = \bar{\mathcal{A}} \cap \bar{\mathcal{B}}$ $\overline{\mathcal{A} \cap \mathcal{B}} = \bar{\mathcal{A}} \cup \bar{\mathcal{B}}$

Os operadores *standard* para a interseção (mínimo) e união (máximo) também são t-normas e t-conormas. Além dos operadores *standard*, os operadores generalizados mais utilizados em sistemas *fuzzy* são:

- **Produto:** o conjunto $\mathcal{C} = \mathcal{A} \cap \mathcal{B}$ é calculado pela t-norma:

$$\mu_{\mathcal{C}}(x) = T(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x)) = \mu_{\mathcal{A}}(x) \cdot \mu_{\mathcal{B}}(x) \quad (8.17)$$

- **Soma limitada:** o conjunto $\mathcal{C} = \mathcal{A} \cup \mathcal{B}$ é calculado pela t-conorma:

$$\mu_{\mathcal{C}}(x) = S(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(x)) = \min(1, (\mu_{\mathcal{A}}(x) + \mu_{\mathcal{B}}(x))) \quad (8.18)$$

A Figura 8.9 apresenta as funções definidas pelos operadores *standard* e pelos operadores generalizados definidos pelas equações (8.17) e (8.18).

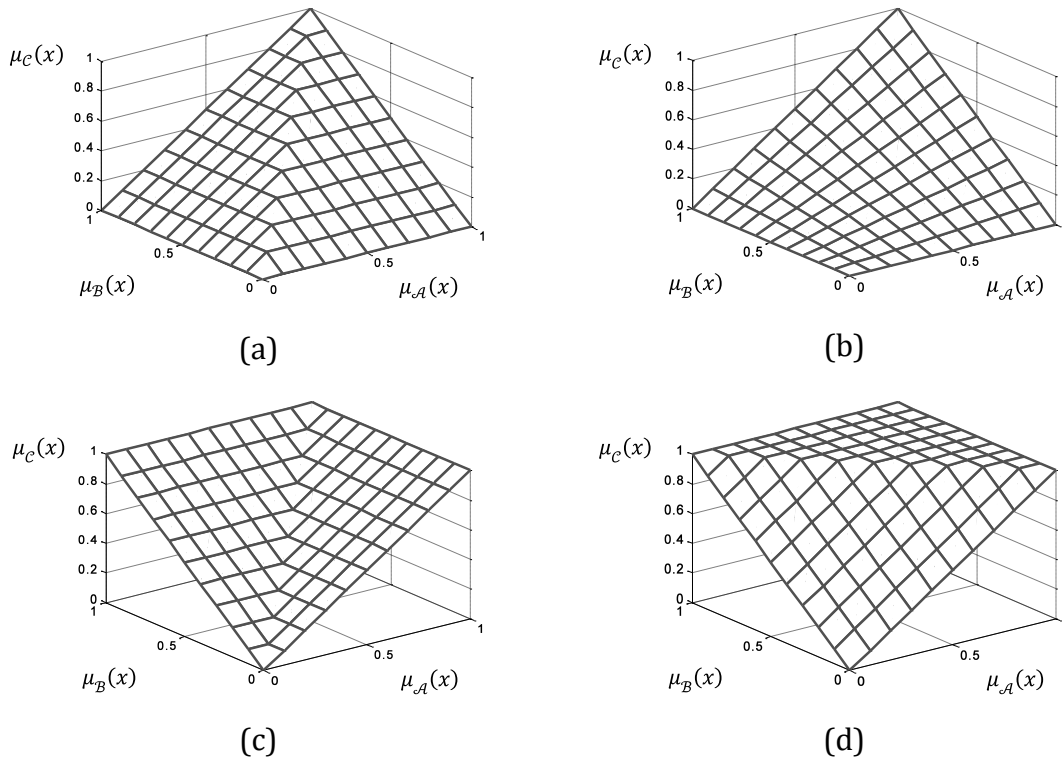


Figura 8.9: Operações *fuzzy* generalizadas: (a) t-norma mínimo; (b) t-norma produto; (c) t-conorma máximo; (d) t-conorma soma limitada.

Os operadores produto e soma limitada não são duais no sentido da propriedade de Morgan (cf. Tabela 8.1, item 7), mas são distributivos (cf. Tabela 8.1, item 4). Com isso, sua aplicação gera uma operação análoga à multiplicação matricial, que é muito útil para o cálculo de sistemas *fuzzy* (cf. seção 8.2).

A violação do axioma da não contradição e do terceiro excluído permite a definição de operadores intermediários entre união e interseção, chamados de operadores de **agregação**. A média aritmética, por exemplo, é intermediária entre o mínimo e o máximo de dois valores de pertinência. O operador ordenado ponderado (OWA, na sigla em inglês), proposto por Yager em 1988 [575][576], é muito utilizado em aplicações de apoio a decisão, pois o seu resultado pode variar continuamente entre o mínimo (maior de todas as t-normas) e o máximo (menor de todas as t-conormas).

Um operador *fuzzy* muito útil em aplicações de ciência de dados é o *sigma-count* [311], cujo resultado é a **cardinalidade** de um conjunto *fuzzy*, calculado como:

$$\Sigma(\mathcal{A}) = \sum_{t=1}^N \mu_{\mathcal{A}}(x(t)) \quad (8.19)$$

onde $\mu_{\mathcal{A}}(x(t))$ é a função de pertinência do conjunto *fuzzy* $\mathcal{A}$ no universo de discurso, representado pelo conjunto de dados $\mathcal{T} = \{x(t), t = 1 \dots N\}$.

O operador *sigma-count* sobre cada conjunto *fuzzy* de uma partição, definida pela função $\mathbf{\mu}_{\mathbb{A}}(\mathbf{x}) = (\mu_{\mathcal{A}_1}(x), \dots, \mu_{\mathcal{A}_n}(x))$, sobre conjunto de dados $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, é a cardinalidade da partição $\Sigma(\mathbb{A})$, calculado como:

$$\Sigma(\mathbb{A}) = (\Sigma(\mathcal{A}_1), \dots, \Sigma(\mathcal{A}_n)) \quad (8.20)$$

O resultado de $\Sigma(\mathbb{A})$ é semelhante ao de um histograma em que os valores correspondentes aos protótipos dos conjuntos *fuzzy* são interpolados pela função de pertinência, como em métodos de estimativa de densidades [238][546].

A Figura 8.10 mostra a aplicação do *sigma-count* em cada um dos conjuntos *fuzzy* de uma partição triangular com $n = 10$ conjuntos *fuzzy* na base Íris. O resultado do *sigma-count* é sobreposto ao resultado do histograma.

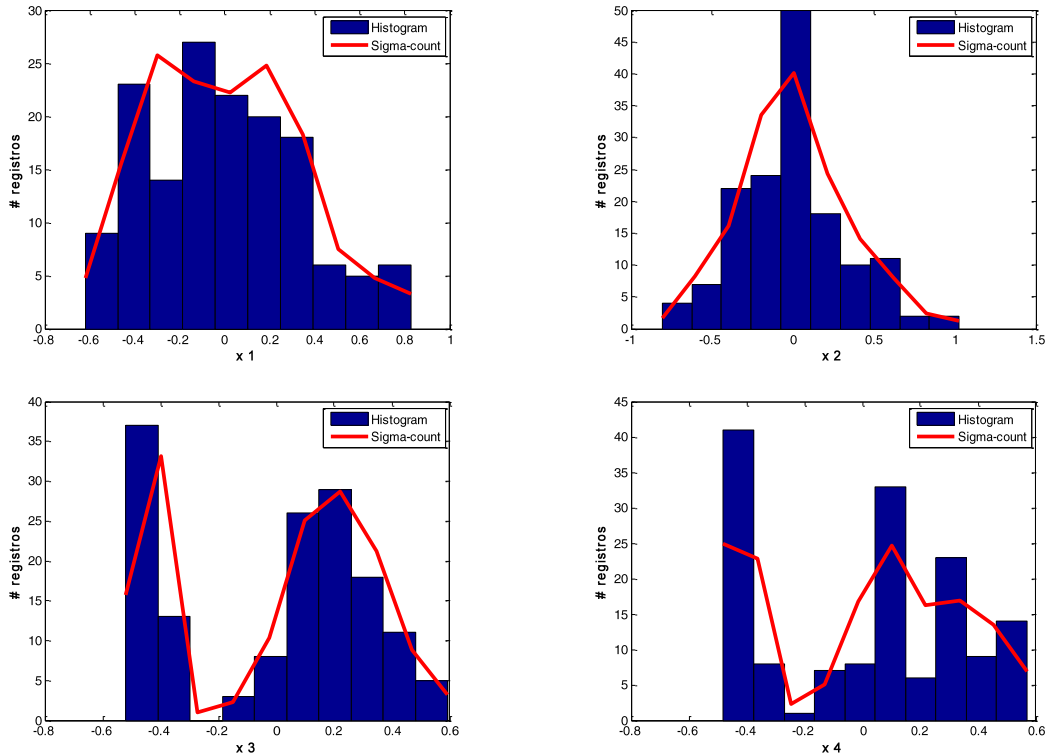


Figura 8.10: *Sigma-count* gerado por uma partição *fuzzy* sobre a base Íris.

O operador *sigma-count* relativo $\Sigma(\mathcal{A}|\mathcal{B})$ entre dois conjuntos $\mathcal{A}$ e $\mathcal{B}$ representa a proporção dos registros descritos por $\mathcal{A}$ e que também podem ser descritos por $\mathcal{B}$.

$$\Sigma(\mathcal{A}|\mathcal{B}) = \frac{\Sigma(\mathcal{A} \cap \mathcal{B})}{\Sigma(\mathcal{B})} = \frac{\sum_{t=1}^N T(\mu_{\mathcal{A}}(x(t)), \mu_{\mathcal{B}}(x(t)))}{\sum_{t=1}^N \mu_{\mathcal{B}}(x(t))} \quad (8.21)$$

No caso em que as funções de pertinência se reduzem às funções características binárias, o *sigma-count* relativo representa a probabilidade condicional $P(\mathcal{A}|\mathcal{B})$. Linguisticamente, a estatística (8.21) representa o grau em que a expressão “muitos elementos de $\mathcal{A}$ são também elementos de $\mathcal{B}$ ” é válida no conjunto de dados.

As operações *fuzzy* definidas anteriormente são todas realizadas sobre o mesmo universo. Operadores *fuzzy* aplicados a conjuntos *fuzzy* definidos em universos diferentes são as relações *fuzzy*, apresentadas a seguir.

8.1.3 Relações fuzzy

A relação *fuzzy* é uma extensão da relação na teoria clássica de conjuntos. Formalmente, uma relação *fuzzy* $\mathcal{A}$ é um conjunto *fuzzy* definido num universo multidimensional pela função de pertinência:

$$\mu_{\mathcal{A}}: \mathcal{X}_1 \times \dots \times \mathcal{X}_p \rightarrow [0,1] \quad (8.22)$$

onde $\mathcal{X}_1 \times \dots \times \mathcal{X}_p$ é o produto cartesiano dos universos $\mathcal{X}_1 \dots \mathcal{X}_p$.

Uma relação *fuzzy* é um conjunto *fuzzy* multidimensionais e denota uma propriedade expressa por mais de um valor. Por exemplo, o conceito de “conforto térmico” pode ser expresso em função dos valores de temperatura e umidade [373].

Todos os operadores descritos na seção 8.1.2 podem ser aplicados a relações *fuzzy* [433]. Uma relação *fuzzy* importante no cálculo de sistemas *fuzzy* é o produto cartesiano *fuzzy* $\mathcal{A} \times \mathcal{B}$, calculado como:

$$\mu_{\mathcal{A} \times \mathcal{B}}(x, y) = T(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(y)), \quad \forall x \in \mathcal{X}, \forall y \in \mathcal{Y} \quad (8.23)$$

onde o conjunto *fuzzy* $\mathcal{A} \subset \mathcal{X}$ e o conjunto *fuzzy* $\mathcal{B} \subset \mathcal{Y}$.

Tendo em vista que as t-normas são distributivas, o produto cartesiano pode ser generalizado para múltiplos universos de discurso incluindo-se outros termos no argumento da equação (8.23). A Figura 8.11 mostra os resultados do produto cartesiano

fuzzy calculados pela t-norma *standard* e pela t-norma produto sobre dois conjuntos *fuzzy* com funções de pertinência triangulares e gaussianas.

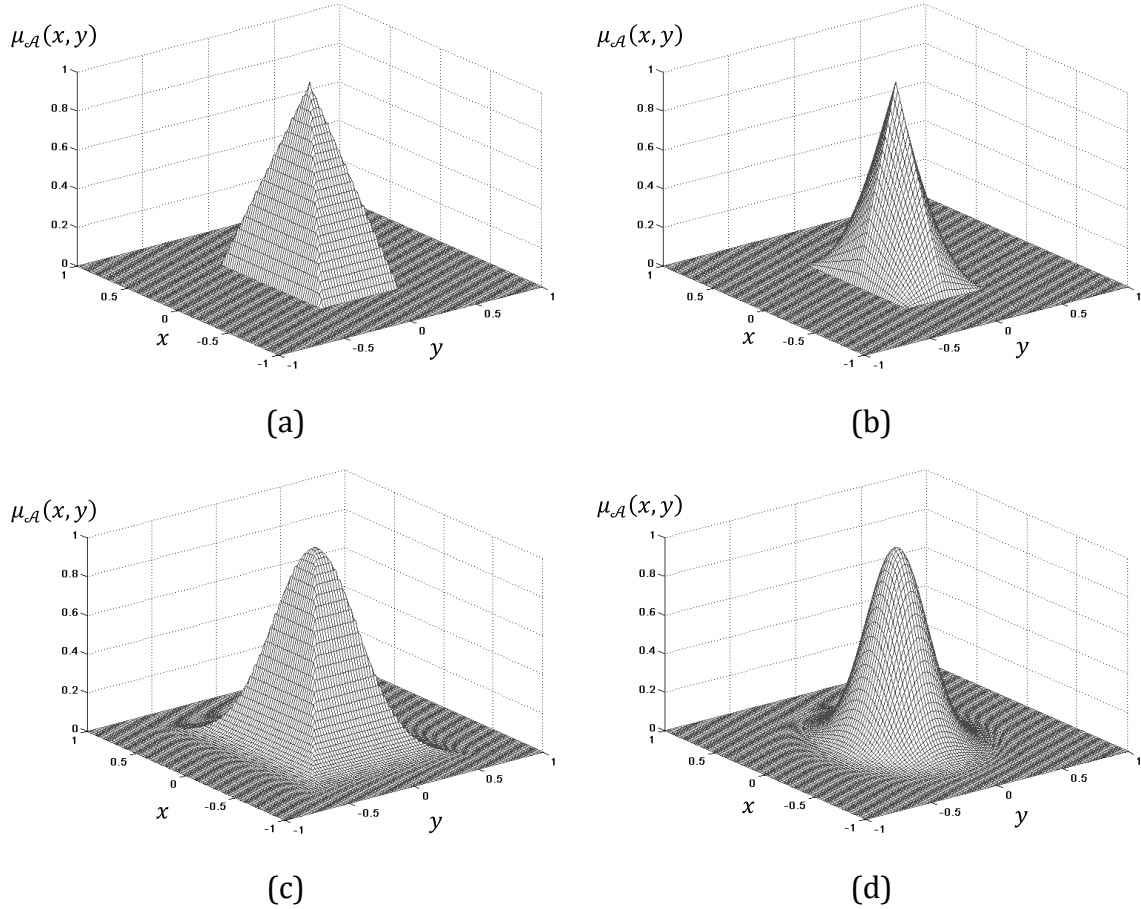


Figura 8.11: Produto cartesiano *fuzzy*: (a) e (b) funções triangulares; (c) e (d) funções gaussianas; (a) e (c) t-norma *standard*; (b) e (d) t-norma produto.

Há operadores *fuzzy* específicos para relações *fuzzy* como a composição-projeção, que é utilizada no cálculo de regras *fuzzy*, descrito a seguir.

8.2 Modelos de Regras *Fuzzy*

Sistemas *fuzzy* estão bem descritos na literatura [301][433]. A apresentação neste capítulo foca na representação simbólica, formalizada pelo conceito de variáveis linguísticas [586], introduzido na seção 8.2.1. As variáveis linguísticas são a base do cálculo de regras *fuzzy*, apresentado na seção 8.2.2, onde são definidos os modelos de Mamdani [363] e Takagi-Sugeno [515]. A seção 8.2.3 apresenta os sistemas *fuzzy* simbólicos, utilizados nas aplicações de ciência de dados descritas nas seções 8.3 e 8.4.

8.2.1 Variáveis linguísticas

O conceito de variáveis linguísticas foi introduzido por Zadeh em 1973 [586] e formalizado em 1975 [587]-[589]. Uma variável linguística é uma “variável cujos valores são palavras ou sentenças de uma linguagem natural ou artificial” [587]. O conceito de variável linguística é a base para a lógica *fuzzy*, cujos valores de verdade são tratados como conjuntos *fuzzy*.

Uma variável linguística* é definida como uma tupla:

$$\mathcal{V} = \langle \mathcal{X}, \mathcal{L}, \mathcal{M} \rangle \quad (8.24)$$

onde $\mathcal{X}$ é o universo de discurso da variável, $\mathcal{L}$ é um conjunto de símbolos (ou termos linguísticos) e $\mathcal{M}$ é uma regra semântica que atribui um significado a cada símbolo em $\mathcal{L}$ através de um conjunto *fuzzy*.

Uma variável linguística é caracterizada por uma partição *fuzzy* $\mathbb{A} = \{\mathcal{A}_1, \dots, \mathcal{A}_n\}$ em que cada conjunto *fuzzy* define o significado do elemento correspondente do conjunto de símbolos $\mathcal{L}(\mathbb{A}) = \{A_1, \dots, A_n\}$. Dessa forma, uma variável linguística formaliza a diferença entre o conjunto *fuzzy* e o símbolo a ele associado.

Para um valor $x \in \mathcal{X}$, a **descrição fuzzy** $\mu_{\mathbb{A}}(x)$ é um vetor em que cada elemento representa o grau de compatibilidade entre o valor x e o símbolo correspondente do conjunto $\mathcal{L}(\mathbb{A})$. Uma descrição *fuzzy* é calculada por uma partição *fuzzy* como:

$$\mu_{\mathbb{A}}(x) = (\mu_{\mathcal{A}_1}(x), \dots, \mu_{\mathcal{A}_n}(x)) \quad (8.25)$$

A descrição *fuzzy* também representa um conjunto *fuzzy* definido sobre o conjunto de símbolos $\mathcal{L}(\mathbb{A})$ como: $x = \{(A_i, \mu_x(A_i)), A_i \in \mathcal{L}(\mathbb{A})\}$, cuja função de pertinência também é calculada pela partição *fuzzy* como $\mu_x(A_i) = \mu_{\mathcal{A}_i}(x)$ [190].

Partições *fuzzy* distintas associadas ao mesmo conjunto de símbolos resultam em descrições *fuzzy* diferentes para o mesmo valor numérico. A Figura 8.12 mostra dois exemplos de descrição *fuzzy* do valor $x = -4$ obtidas por uma partição *fuzzy* triangular na Figura 8.12a e uma partição *fuzzy* gaussiana forte na Figura 8.12b.

* A definição original de Zadeh inclui também uma gramática para a geração dos valores linguísticos, que não será utilizada no desenvolvimento dos modelos de regras *fuzzy* neste capítulo.

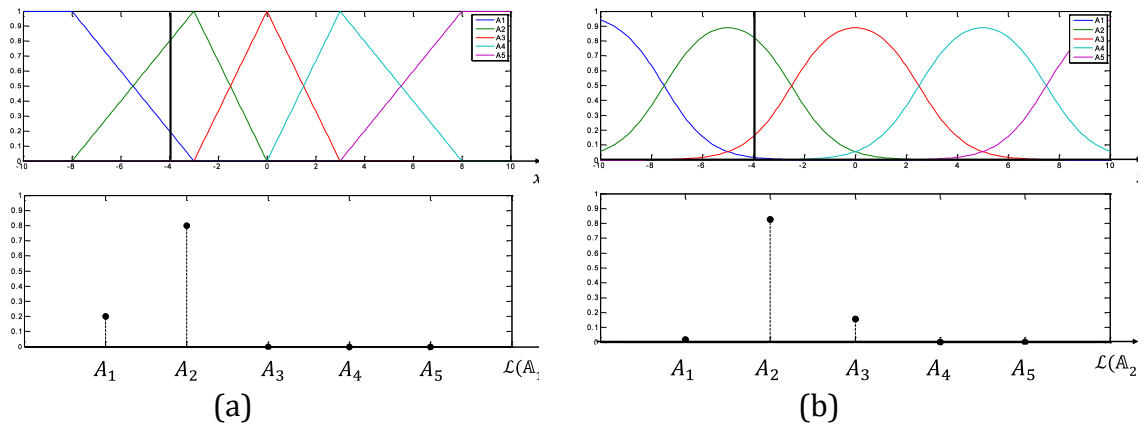


Figura 8.12: Exemplos de descrição *fuzzy*: (a) partição triangular; (b) partição gaussiana forte.

Conjuntos *fuzzy* em universos multidimensionais são relações *fuzzy* definidas no produto cartesiano dos universos. A descrição *fuzzy* de um vetor $\mathbf{x} = (x_1, \dots, x_p)$ é calculada pelo produto tensorial de Kronecker como*:

$$\mu_{\mathbb{B}}(\mathbf{x}) = \mu_{A_1}(x_1) \otimes \dots \otimes \mu_{A_p}(x_p) \quad (8.26)$$

onde a partição *fuzzy* A_i , com n_i conjuntos *fuzzy*, é utilizada para a descrição *fuzzy* da variável x_i e cada elemento da partição *fuzzy* $\mathbb{B} = \{B_1, \dots, B_M\}$ é um conjunto *fuzzy* definido no universo $Z = X_1 \times \dots \times X_p$. Nos sistemas *fuzzy*, a equação (8.26) pode ser generalizada utilizando outros operadores t-norma no lugar do produto.

O número de elementos da partição $\mathbb{B}$ calculada pela equação (8.26) é $M = \prod_{i=1}^p n_i$, o que gera uma explosão combinatória do número de símbolos com o aumento do número de variáveis. Por exemplo, em um problema com $p = 10$ variáveis em que cada variável é representada por uma partição *fuzzy* de $n = 5$ conjuntos *fuzzy*, o produto tensorial de Kronecker (8.26) gera $n^p = 9.765.625$ combinações!

A Figura 8.13 mostra um exemplo em que $A_1 = \{A_{11}, \dots, A_{15}\}$ e $A_2 = \{A_{21}, \dots, A_{23}\}$ são partições *fuzzy* definidas respectivamente nos universos X_1 e X_2 . As partições são combinadas para gerar $\mathbb{B} = \{B_1, \dots, B_{15}\}$, que é definida no universo $Z = X_1 \times X_2$. Na Figura 8.13, as cores representam os valores de pertinência (cf. Figura 8.2) e cada conjunto *fuzzy* da partição $\mathbb{B}$, que é calculado por uma t-norma produto (cf. Figura 8.11b).

* O resultado do produto tensorial de Kronecker de dois vetores $\mathbf{a} = (a_1, \dots, a_n)$ e $\mathbf{b} = (b_1, \dots, b_m)$ é o vetor $\mathbf{a} \otimes \mathbf{b} = (a_1 \mathbf{b}, \dots, a_n \mathbf{b}) = (a_1 b_1, a_1 b_2, \dots, a_n b_m)$.

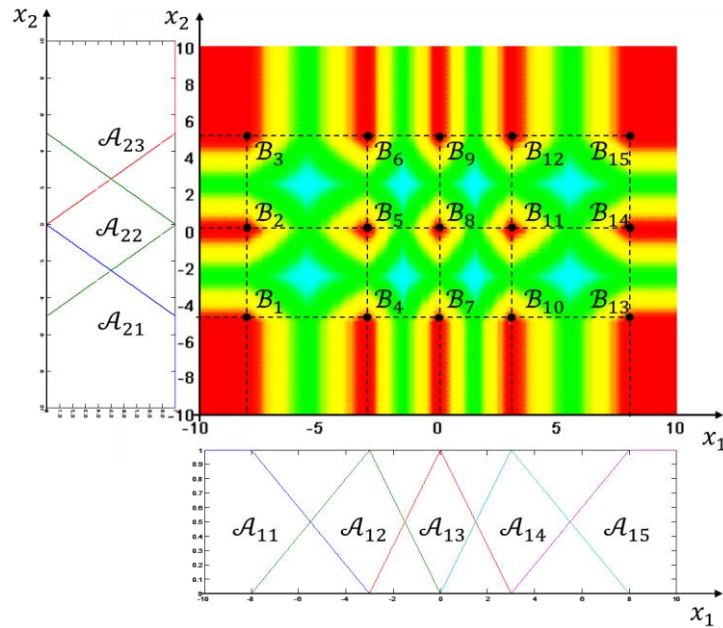


Figura 8.13: Combinação de partições *fuzzy*.

A Figura 8.14 mostra a combinação de $\mathcal{L}(\mathbb{A}_1) = \{A_{11}, \dots, A_{15}\}$ e $\mathcal{L}(\mathbb{A}_2) = \{A_{21}, \dots, A_{23}\}$ para a construção do conjunto de símbolos $\mathcal{L}(\mathbb{B}) = \mathcal{L}(\mathbb{A}_1) \times \mathcal{L}(\mathbb{A}_2) = \{B_1, \dots, B_{15}\}$.

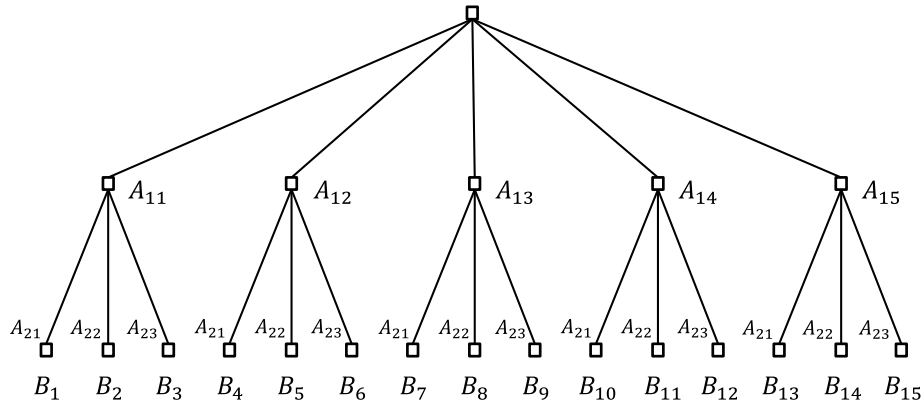


Figura 8.14: Combinação do conjunto de termos.

Conjuntos *fuzzy* também podem ser definidos diretamente sobre universos multidimensionais a partir da análise de agrupamentos. Há algoritmos na literatura para a síntese de bases de regras *fuzzy* que evitam a explosão combinatória de símbolos, como será apresentado na seção 8.3. Entretanto, quando o número de variáveis é pequeno a combinação de partições *fuzzy* pode ser útil.

Na próxima seção, a combinação de partições *fuzzy* será apresentada no contexto de regras *fuzzy*, que são a base do modelo de inferência realizado pelos sistemas *fuzzy*.

8.2.2 Cálculo de regras fuzzy

O conceito de variável linguística estabelece a representação de um símbolo como um conjunto *fuzzy*, de forma que a expressão “ x é A ” é traduzida pelo valor de pertinência $\mu_{\mathcal{A}}(x)$. Uma regra fuzzy $A \rightarrow B$ é uma expressão escrita como:

$$\text{Se } x \text{ é } A, \text{ então } y \text{ é } B \quad (8.27)$$

Uma regra fuzzy é uma relação fuzzy $\mathcal{F} = \{(x, y), \mu_{\mathcal{F}}(x, y)\}, (x, y) \in \mathcal{X} \times \mathcal{Y}$, cuja função de pertinência é calculada por um operador de implicação* fuzzy [310][433]. Em sistemas fuzzy, a função de pertinência da regra fuzzy é calculada pelo operador t-norma, como no produto cartesiano fuzzy (8.23):

$$\mu_{\mathcal{F}}(x, y) = T(\mu_{\mathcal{A}}(x), \mu_{\mathcal{B}}(y)), \quad \forall x \in \mathcal{X}, \forall y \in \mathcal{Y} \quad (8.28)$$

onde a relação $\mathcal{F}$ representa a regra fuzzy (8.27).

Uma base de regras fuzzy representa o mapeamento entre as variáveis de entrada e saída através da união das relações locais definidas pelas regras. A relação fuzzy da base de regras é calculada como $\mathcal{F} = \bigcup_{k=1}^M \mathcal{F}_k$, onde $\mathcal{F}_k$ é a relação fuzzy da regra k . A função de pertinência da base de regras é calculada pela t-conorma dos seus componentes como:

$$\mu_{\mathcal{F}}(x, y) = S(\mu_{\mathcal{F}_1}(x, y), \dots, \mu_{\mathcal{F}_M}(x, y)) \quad (8.29)$$

onde $\mu_{\mathcal{F}_k}(x, y) = T(\mu_{\mathcal{A}_{i(k)}}(x), \mu_{\mathcal{B}_{j(k)}}(y))$ é a relação fuzzy definida pela regra k , escrita como $A_{i(k)} \rightarrow B_{j(k)}$. As funções $i(k)$ e $j(k)$ retornam os índices dos conjuntos fuzzy nas respectivas partições fuzzy que aparecem na regra k .

A Figura 8.15 mostra o exemplo de uma base de regras fuzzy formada por cinco regras $A_{i(k)} \rightarrow B_{j(k)}$, onde $i(k) = j(k) = k, k = 1, \dots, 5$. As regras representam apenas um subconjunto de todas as combinações possíveis das partições fuzzy $\mathbb{A}$ e $\mathbb{B}$. Na Figura 8.13 todas as combinações são consideradas, pois o objetivo é a obtenção de uma partição multidimensional. No caso da Figura 8.15, a relação fuzzy representa o mapeamento entre as variáveis de entrada e saída como uma restrição flexível aos valores admissíveis de saída para cada valor da variável de entrada.

* A implicação fuzzy é um tipo de operador fuzzy generalizado. Não foi incluído no texto porque o foco é a apresentação de sistemas fuzzy que utilizam a t-norma como operador de implicação.

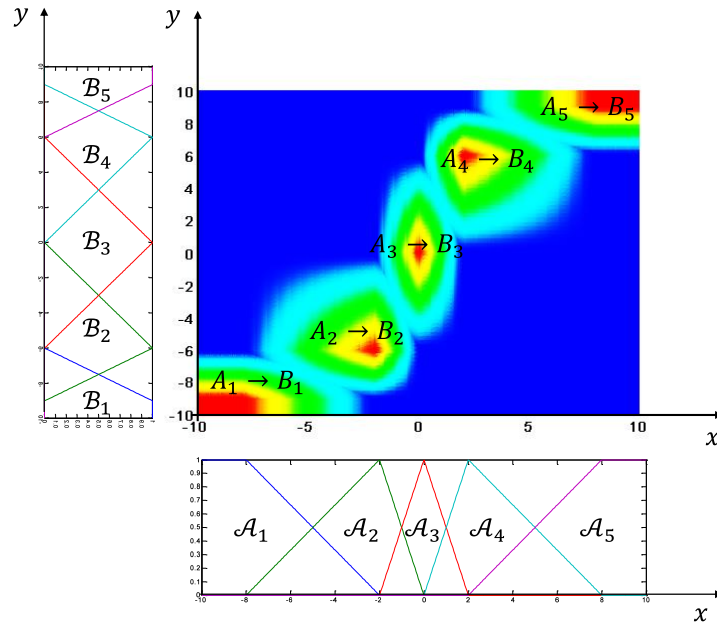


Figura 8.15: Base de regras *fuzzy*.

O raciocínio dedutivo é definido na lógica clássica pela regra de inferência conhecida como *modus ponens*, que permite concluir que B é verdadeiro dado que a premissa (fato) A e a regra $A \rightarrow B$ são verdadeiros. Formalmente, escreve-se $A \rightarrow B, A \vdash B$, ou, na forma da tautologia, $((A \rightarrow B) \wedge A) \rightarrow B$.

No cálculo de regras *fuzzy*, a inferência é realizada pela *modus ponens generalizada*, em que uma entrada A' , similar à premissa da regra A , e uma regra *fuzzy* $A \rightarrow B$ resulta na conclusão B' . A função de pertinência do conjunto *fuzzy* B' é calculada pelo operador **composição-projeção**:

$$\mu_{B'}(y) = \sup_{x \in \mathcal{X}} \left(T(\mu_{A'}(x), \mu_F(x, y)) \right), \quad \forall y \in \mathcal{Y} \quad (8.30)$$

onde $\sup_{x \in \mathcal{X}}$ representa o supremo, isto é, o limite superior do conjunto.

O operador *fuzzy* composição-projeção calcula o conjunto *fuzzy* B' como a imagem do conjunto *fuzzy* A' produzida pela relação $\mathcal{F}$, como $B' = A' \circ \mathcal{F}$. A Figura 8.16 ilustra a operação composição-projeção, mostrando as duas etapas do processamento: a composição, calculada por uma t-norma, é representada pela área mais escura e a projeção, calculada pelo supremo.

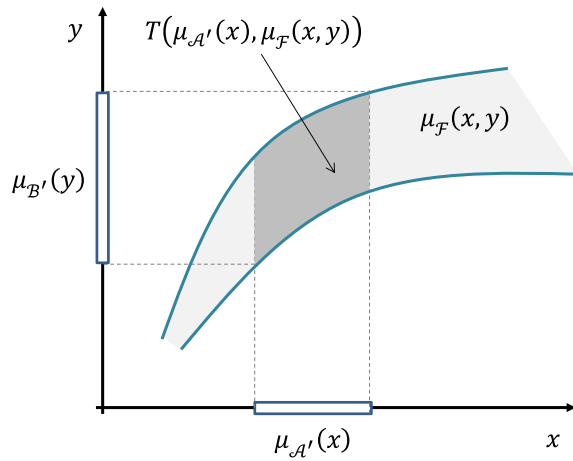


Figura 8.16: Operador composição-projeção.

O cálculo de regras *fuzzy* definido pela equação (8.30) envolve a avaliação de todos os elementos do universo de entrada $\mathcal{X}$ para o cálculo da função de pertinência de cada elemento do universo de saída $\mathcal{Y}$. Esse cálculo é computacionalmente complexo e pode ser simplificado quando a relação definida por cada regra *fuzzy* $A \rightarrow B$ também é calculada por uma t-norma (cf. equação (8.28)).

Substituindo a definição da regra *fuzzy* em (8.28) na equação (8.30) e utilizando as propriedades dos operadores t-norma, a equação (8.30) pode ser reescrita como:

$$\mu_{B'}(y) = T(u(x), \mu_B(y)), \quad \forall y \in \mathcal{Y} \quad (8.31)$$

onde $u(x)$ é o grau de compatibilidade entre a entrada A' e a premissa da regra A , também chamado de **grau de ativação** da regra, calculado como:

$$u(x) = \sup_{x \in \mathcal{X}} \left(T(\mu_{A'}(x), \mu_A(x)) \right) \quad (8.32)$$

Para uma base de regras *fuzzy*, o conjunto *fuzzy* B' é definido pela união das parcelas correspondentes de cada regra *fuzzy* como $B' = \cup_{k=1}^M B'_k$, onde cada parcela é calculada pela equação (8.31). A função de pertinência do conjunto *fuzzy* B' é calculada pela t-conorma das parcelas de cada regra:

$$\mu_{B'}(y) = S \left(T(u_1(x), \mu_{B_{j(1)}}(y)), \dots, T(u_M(x), \mu_{B_{j(M)}}(y)) \right), \quad \forall y \in \mathcal{Y} \quad (8.33)$$

onde $u_k(x) = \sup_{x \in \mathcal{X}} \left(T(\mu_{A'}(x), \mu_{A_{i(k)}}(x)) \right)$ e as funções $i(k)$ e $j(k)$ retornam os índices dos conjuntos *fuzzy* nas respectivas partições *fuzzy* que aparecem na regra k .

A Figura 8.17 apresenta um exemplo do cálculo de regras *fuzzy* para um regulador de velocidades simples. A variável de entrada representa o erro, definido pela diferença entre a velocidade observada e a referência. A variável de saída é a aceleração a ser aplicada ao veículo. O controle pode ser expresso linguisticamente por duas regras:

- Se o erro é “negativo”, então a aceleração é “positiva”;
- Se o erro é “positivo”, então a aceleração é “negativa”.

As regras acima são bastante intuitivas e fazem sentido para o controle da velocidade do veículo, independentemente de seu tamanho e motorização ou da velocidade de referência. O mapeamento entre os conceitos linguísticos e sua realização num veículo real é definido pelas partições *fuzzy* de entrada e saída.

No exemplo, os valores de erro e aceleração são representados pelas partições *fuzzy* $\mathbb{A} = \{\mathcal{A}_1, \mathcal{A}_2\}$ e $\mathbb{B} = \{\mathcal{B}_1, \mathcal{B}_2\}$, onde os termos A_1 e B_1 representam o conceito “negativo” e os termos A_2 e B_2 , o conceito “positivo”, em cada contexto. Simbolicamente, as regras são escritas como:

- $A_1 \rightarrow B_2$;
- $A_2 \rightarrow B_1$.

Quando o valor observado de velocidade $\mathcal{A}'$ é um conjunto *fuzzy* do tipo *singleton*,* o grau de ativação das regras é calculado como:

- $u_1(x(t)) = \sup_{x \in \mathcal{X}} \left(T \left(\mu_{\mathcal{A}'}(x(t)), \mu_{\mathcal{A}_1}(x(t)) \right) \right) = \mu_{\mathcal{A}_1}(x(t));$
- $u_2(x(t)) = \sup_{x \in \mathcal{X}} \left(T \left(\mu_{\mathcal{A}'}(x(t)), \mu_{\mathcal{A}_2}(x(t)) \right) \right) = \mu_{\mathcal{A}_2}(x(t)).$

A função de pertinência do conjunto *fuzzy* $\mathcal{B}'$ é calculada pela equação (8.33) como:

$$\mu_{\mathcal{B}'}(y) = S \left(T \left(u_1(x(t)), \mu_{\mathcal{B}_2}(y) \right), T \left(u_2(x(t)), \mu_{\mathcal{B}_1}(y) \right) \right), \quad \forall y \in \mathcal{Y} \quad (8.34)$$

A Figura 8.17 mostra as etapas do processamento realizado num determinado instante em que $x(t)$ é a entrada e o conjunto *fuzzy* $\mathcal{B}'$ é a saída.

* O conjunto *fuzzy singleton* $\mathcal{A}$ é definido como $\mu_{\mathcal{A}}(x) = \begin{cases} 1, & \text{se } x = x(t) \\ 0, & \text{se } x \neq x(t) \end{cases}$

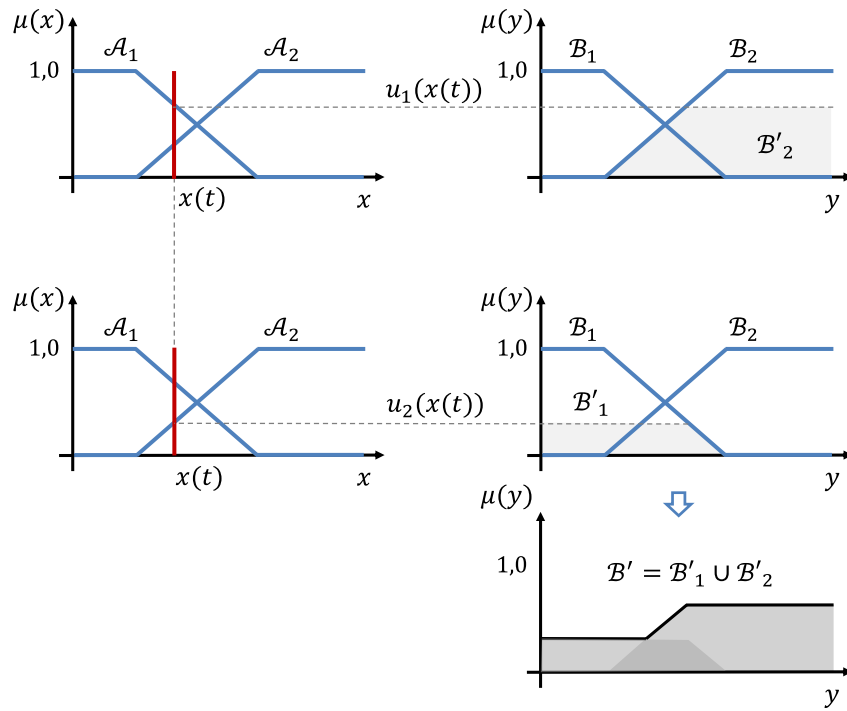


Figura 8.17: Cálculo de regras fuzzy.

O valor final de aceleração a ser empregado no veículo é calculado por um operador chamado de **defuzzificação**, que atribui um valor numérico ao conjunto fuzzy B' . Diversos operadores de defuzzificação são encontrados na literatura [114]. O mais comum é o chamado centro de área (CoA) do conjunto fuzzy, calculado como:

$$\hat{y}(t) = \frac{\sum_{y \in \mathcal{Y}} y \cdot \mu_{B'}(y)}{\sum_{y \in \mathcal{Y}} \mu_{B'}(y)} \quad (8.35)$$

onde a soma é realizada para todo elemento $y \in \mathcal{Y}$.

O cálculo de regras fuzzy definido pela equação (8.33) pode ser estendido para regras com múltiplos antecedentes em que a entrada $\mathbf{x} \in (\mathcal{X}_1 \times \dots \times \mathcal{X}_p)$. Nesse caso, a regra fuzzy k é a expressão $A_{i_1(k)} \wedge \dots \wedge A_{i_p(k)} \rightarrow B_{j(k)}$, onde $i_1(k), \dots, i_p(k)$ identificam os elementos nas partições fuzzy das variáveis $x_1, \dots, x_p$ utilizados na regra. A conjunção fuzzy é calculada pela t-norma dos conjuntos fuzzy correspondentes e o grau de ativação da regra com múltiplos antecedentes é calculado como:

$$u_k(\mathbf{x}) = T(u_k(x_1), \dots, u_k(x_p)) \quad (8.36)$$

onde $u_k(x_i)$ o grau de ativação da entrada x_i , calculado pela equação (8.32).

O resultado da equação (8.36) pode ser aplicado diretamente na equação (8.33) para o cálculo do conjunto *fuzzy* B' .

O cálculo de regras *fuzzy* foi introduzido por Zadeh [587]-[589] e utilizado por Mamdani e Assilian [363] para a síntese linguística de sistemas de controle *fuzzy*. Em geral, um sistema de controle *fuzzy* é um regulador que utiliza duas variáveis de entrada: o erro e sua variação no tempo [335].

Um **sistema *fuzzy*** é um modelo de regras *fuzzy* genérico capaz de ajustar qualquer função contínua [549]. Os modelos variam ligeiramente segundo a escolha das funções de pertinência, dos operadores *fuzzy* e dos métodos de defuzzificação. Quando os operadores *standard* são utilizados na equação (8.33), o modelo é chamado de sistema *fuzzy* de Mamdani.

No sistema *fuzzy* de Takagi e Sugeno (TS) [515], a conclusão da regra é uma função das variáveis de entrada e cada regra é escrita como $A_{i_1(k)} \wedge \dots \wedge A_{i_p(k)} \rightarrow \hat{y}_k = h_k(\mathbf{x})$. A saída correspondente a uma entrada $\mathbf{x}(t)$ é calculada pela soma das saídas de cada regra ponderadas pelo respectivo grau de ativação:

$$\hat{y}(t) = \frac{\sum_{k=1}^M u_k(\mathbf{x}(t)) \hat{y}_k}{\sum_{k=1}^M u_k(\mathbf{x}(t))} \quad (8.37)$$

onde $u_k(\mathbf{x}(t))$ é calculado pela equação (8.36) e M é o número de regras.

Em sua versão mais simples, chamada de ordem zero, a função de saída de cada regra é constante, isto é, $\hat{y}_k = \theta_k$, e a equação (8.37) representa um modelo local linear, como um modelo RBF, em que os parâmetros de saída podem ser ajustados pelo MMQ.

O modelo TS de ordem zero é equivalente a um modelo de Mamdani em que os conjuntos *fuzzy* de saída são *singletons* localizados nos valores dos parâmetros θ_k de cada regra. A Figura 8.18 mostra o exemplo da Figura 8.17 como um modelo TS de ordem zero equivalente, onde os índices dos parâmetros θ_k estão associados a cada regra.

O sistema *fuzzy* TS foi um dos precursores da área de inteligência computacional. Em 1993, Jang mostrou a equivalência entre o sistema *fuzzy* TS de ordem zero e modelos RBF [286] no sistema ANFIS [287], precursor dos sistemas neuro-*fuzzy* [288]. A sinergia entre sistemas *fuzzy* e redes neurais foi muito explorada na literatura, como nos livros de Kosko [320][321] e Pedrycz e Gomide [433].

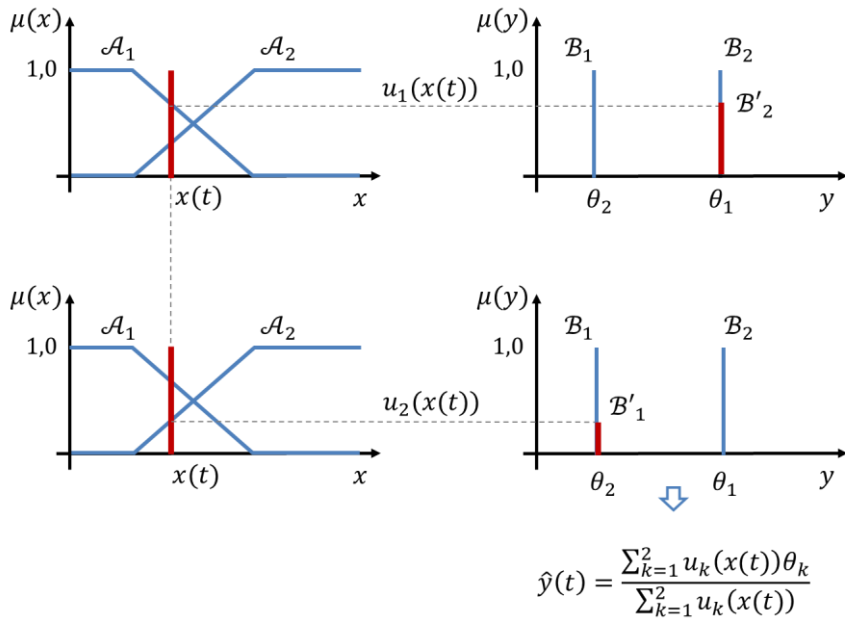


Figura 8.18: Modelo TS equivalente ao modelo de Mamdani da Figura 8.17.

O cálculo de regras *fuzzy* definido pela equação (8.30) é geral e pode ser empregado com qualquer tipo de regra *fuzzy* e qualquer combinação de operadores t-norma e t-conorma. Entretanto, é computacionalmente complexo pois deve ser aplicado a cada elemento dos universos de entrada e saída. A equação (8.33) é uma simplificação que considera que as regras são combinações de conjuntos *fuzzy* de entrada e saída.

Na próxima seção, apresentamos uma formulação particular do cálculo de regras *fuzzy* que é mais adequada para a síntese de modelos de ciência de dados.

8.2.3 Sistemas *fuzzy* simbólicos

Considere o conjunto de dados $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$ em que $\mathbf{x}(t) \in \mathcal{R}^p$ é o vetor de variáveis de entrada e $y(t) \in \mathcal{R}$ é a variável de saída. Em problemas de classificação, a variável de saída $y(t) \in \mathcal{N}$, onde $y(t) = j$ representa a classe $\mathcal{C}_j$.

Um sistema *fuzzy* é um modelo $\hat{y}(t) = \hat{f}(\mathbf{x}(t))$ calculado em três etapas:

- *Fuzzificação*: que calcula o conjunto *fuzzy* de entrada $\mathcal{A}'$ a partir da entrada $\mathbf{x}(t)$;
- *Inferência*: que converte o conjunto *fuzzy* $\mathcal{A}'$ no conjunto *fuzzy* de saída $\mathcal{B}'$;
- *Defuzzificação*: que calcula a saída $\hat{y}(t)$ a partir do conjunto *fuzzy* $\mathcal{B}'$.

Em aplicações de sistemas *fuzzy*, a entrada é representada por valores numéricos, isto é, $\mathbf{x}(t) \in \mathcal{R}^p$, e cada componente da entrada é representado por um conjunto *fuzzy singleton* na etapa de fuzzificação. A etapa de inferência é calculada pela equação (8.33) a partir do grau de ativação das regras (cf. equação (8.36)). A defuzzificação é calculada por um método como o CoA da equação (8.35).

O cálculo de regras geral (cf. equação (8.30)) pode ser aplicado ao conjunto de símbolos das partições *fuzzy* de entrada e saída. A formulação de sistemas *fuzzy* simbólicos é particularmente útil em aplicações de ciência de dados, pois permite uma parametrização independente nas etapas de fuzzificação, inferência e defuzzificação.

O processamento realizado nos sistemas *fuzzy* pode ser entendido pelos níveis ilustrados na Figura 8.19. O nível da observação corresponde aos dados do problema, representados pelas variáveis de entrada e saída. O objetivo do sistema *fuzzy* é modelar a relação $y(t) = f(\mathbf{x}(t))$ entre as variáveis de entrada e saída observada no conjunto de dados. No nível simbólico, o valor das variáveis de entrada é transformado pelo processo de fuzzificação numa representação simbólica, definida pela partição *fuzzy* $\mathbb{A}$. A relação entre as variáveis de entrada e saída é representada por regras $A_i \rightarrow B_j$, onde a premissa $A_i \in \mathcal{L}(\mathbb{A})$ e $B_j \in \mathcal{L}(\mathbb{B})$. O resultado é uma aproximação $\hat{y}(t) = \hat{f}(\mathbf{x}(t))$ calculada pelo processo de defuzzificação.

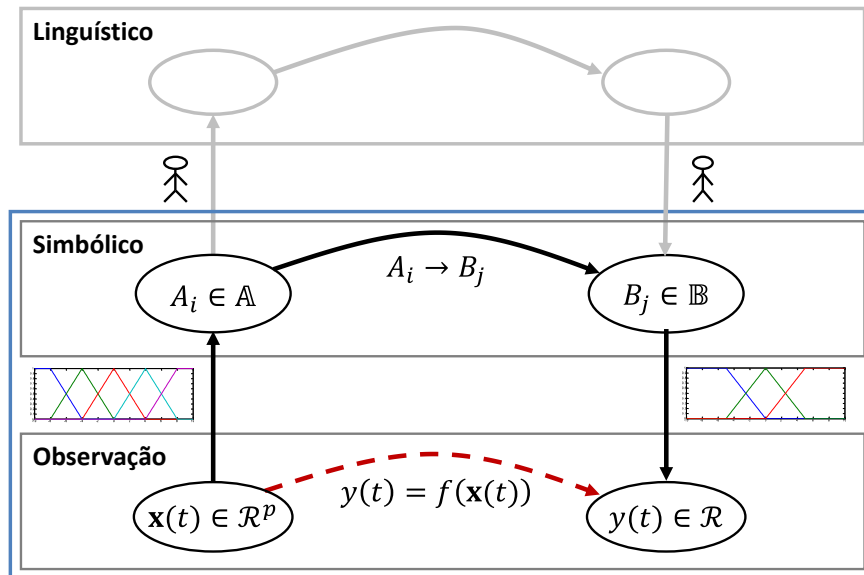


Figura 8.19: Sistemas *fuzzy* simbólicos.

O processamento nos sistemas *fuzzy* simbólicos é realizado pelos seguintes operadores:

- *Fuzzificação*: $\mathbf{u}(t) = F(\mathbf{x}(t), \mathbb{A})$;
- *Inferência*: $\hat{\mathbf{v}}(t) = I(\mathbf{u}(t), \mathcal{F})$;
- *Defuzzificação*: $\hat{\mathbf{y}}(t) = D(\mathbf{v}(t), \mathbb{B})$,

onde $\mathbb{A}$ é a partição *fuzzy* de entrada, $\mathbb{B}$ é a partição *fuzzy* de saída e $\mathcal{F}$, a relação *fuzzy* que define a base de regras. Os operadores em cada etapa são descritos a seguir.

8.2.3.1 Fuzzificação

A fuzzificação é calculada por uma descrição *fuzzy* como:

$$\mathbf{u}(t) = \boldsymbol{\mu}_{\mathbb{A}}(\mathbf{x}(t)) = (\mu_{\mathcal{A}_1}(\mathbf{x}(t)), \dots, \mu_{\mathcal{A}_n}(\mathbf{x}(t))) \quad (8.38)$$

onde cada conjunto *fuzzy* $\mathcal{A}_i$, $i = 1, \dots, n$ é definido no domínio multidimensional da variável de entrada; n é o número de termos da partição de entrada $\mathbb{A}$, isto é, o número de regras e o vetor $\mathbf{u}(t) \in [0,1]^n$ representa o grau de ativação das regras.

Em problemas de pequenas dimensões, a fuzzificação pode ser calculada pela combinação das descrições *fuzzy* de cada variável como na equação (8.26):

$$\mathbf{u}(t) = \mathbf{u}_1(t) \otimes \dots \otimes \mathbf{u}_p(t) \quad (8.39)$$

onde $\mathbf{u}_i(t) = (u_{i1}(t), \dots, u_{in_i}(t)) = \boldsymbol{\mu}_{\mathbb{A}_i}(x_i(t))$ é a descrição *fuzzy* do valor $x_i(t)$ definida pela partição *fuzzy* $\mathbb{A}_i$.

A equação (8.39) resulta numa explosão combinatória do número de regras e, consequentemente, do número de elementos do vetor $\mathbf{u}(t)$, pois utiliza todas as combinações possíveis de conjuntos *fuzzy* nas partições das variáveis.

Algoritmos de indução de regras *fuzzy* permitem selecionar combinações específicas para serem utilizadas em cada regra. O resultado de um algoritmo de indução de regras é uma tabela Ξ , de dimensão $n \times p$, em que o valor do elemento ξ_{ki} representa o índice do elemento da partição *fuzzy* $\mathbb{A}_i$ a ser utilizado na regra k [167]. Dessa forma, o valor $\xi_{ki} = j$ determina que o conjunto *fuzzy* $\mathcal{A}_{ij} \in \mathbb{A}_i$ deve ser utilizado na regra k . O resultado da fuzzificação, para cada regra, é escrito como:

$$u_k(t) = T\left(u_{1\xi_{k1}}(t), \dots, u_{p\xi_{kp}}(t)\right), k = 1, \dots, n \quad (8.40)$$

Na equação (8.40), pelo menos um conjunto *fuzzy* da partição de cada variável deve ser utilizado em cada regra. Modelos mais complexos podem ser obtidos eliminando-se variáveis de algumas regras. O ajuste desse tipo de modelo gera um problema de otimização combinatória que pode ser resolvido com algoritmos genéticos [153].

8.2.3.2 Inferência

A inferência do sistema *fuzzy* simbólico é realizada pela relação *fuzzy* $\mathcal{F}$, que calcula a descrição de saída $\hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$ a partir da descrição de entrada $\mathbf{u}(t)$. A descrição de entrada é um conjunto *fuzzy* definido no conjunto de símbolos $\mathcal{L}(\mathbb{A})$ e a descrição de saída é definida no conjunto de símbolos $\mathcal{L}(\mathbb{B})$.

A relação *fuzzy* $\mathcal{F}$ é definida como $\mathcal{F} = \left\{ \left((A_i, B_j), \mu_{\mathcal{F}}(A_i, B_j) \right), (A_i, B_j) \in \mathcal{L}(\mathbb{A}) \times \mathcal{L}(\mathbb{B}) \right\}$, onde o valor de pertinência $\mu_{\mathcal{F}}(A_i, B_j)$ é o grau de compatibilidade entre os termos A_i e B_j na regra $A_i \rightarrow B_j$.

A relação *fuzzy* $\mathcal{F}$ é ponderada pela matriz Φ , de dimensão $n \times m$, onde n é o número de regras e m é o número de símbolos de saída. Cada elemento da matriz Φ define as relações presentes no modelo e o parâmetro $\varphi_{ij} = \mu_{\mathcal{F}}(A_i, B_j)$ representa o grau de compatibilidade entre os termos A_i e B_j na regra $A_i \rightarrow B_j$. A ponderação de regras permite maior flexibilidade na especificação do modelo *fuzzy* [278]. A inferência é calculada diretamente pelo operador composição-projeção como:

$$\hat{\mathbf{v}}(t) = \mathbf{u}(t) \circ \Phi \quad (8.41)$$

onde $\mathbf{u}(t)$ é o resultado da fuzzificação e Φ representa a relação *fuzzy* $\mathcal{F}$.

O resultado da inferência *fuzzy* é o vetor $\hat{\mathbf{v}}(t)$, em que cada componente $\hat{v}_j(t)$ correspondente ao termo $B_j \in \mathbb{B}$ é calculado como:

$$\hat{v}_j(t) = S \left(T(u_1(t), \varphi_{1j}), \dots, T(u_n(t), \varphi_{nj}) \right), \quad j = 1, \dots, m \quad (8.42)$$

Aplicando os operadores *standard* na equação (8.42) no caso de regras não ponderadas, isto é, $\varphi_{ij} \in \{0,1\}$, o resultado do sistema *fuzzy* simbólico é equivalente ao sistema *fuzzy* de Mamdani. A Figura 8.20 apresenta o cálculo do sistema *fuzzy* simbólico equivalente aos sistemas *fuzzy* de Mamdani da Figura 8.17.

No caso de partições *fuzzy* fortes, a soma dos elementos do vetor $\mathbf{u}(t)$ é igual a 1 (cf. equação (8.6)) e podemos utilizar a t-norma produto e a t-conorma soma limitada na

equação (8.42). O resultado da inferência pode, então, ser calculado pelo produto vetor-matriz:

$$\hat{\mathbf{v}}(t) = \mathbf{u}(t)\Phi \quad (8.43)$$

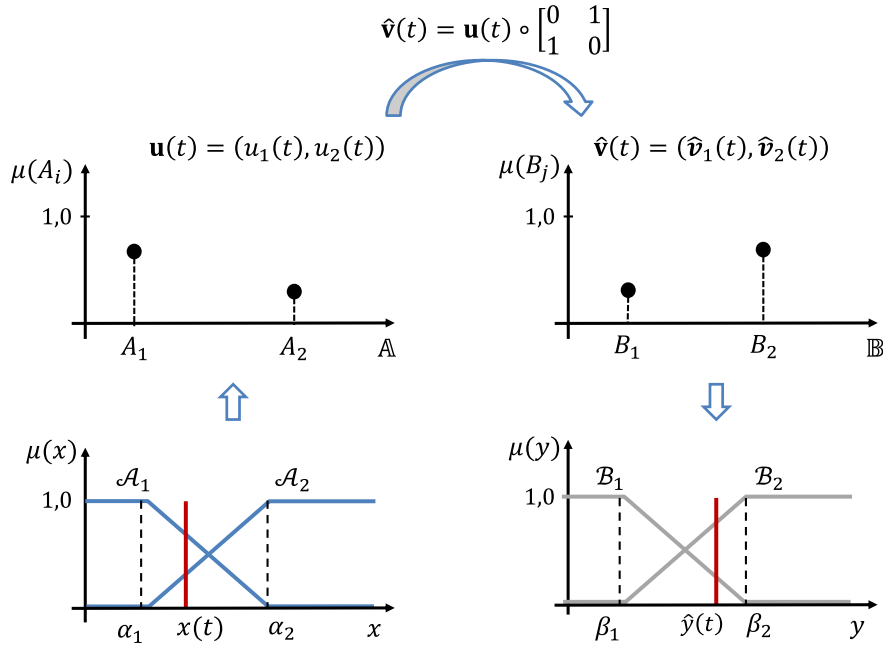


Figura 8.20: Sistema *fuzzy* simbólico equivalente ao modelo de Mamdani da Figura 8.17.

Em problemas de classificação, cada símbolo de saída representa uma classe, isto é, $B_j \equiv C_j$, e os elementos do vetor $\hat{\mathbf{v}}(t)$ representam a pertinência da entrada $\mathbf{x}(t)$ a cada uma das classes $C_j, j = 1, \dots, m$. Em problemas de regressão, o valor numérico da saída é calculado pelo operador de defuzzificação, descrito a seguir.

8.2.3.3 Defuzzificação

O vetor $\hat{\mathbf{v}}(t)$ calculado na inferência é uma aproximação da descrição *fuzzy* do valor de saída $y(t) = f(\mathbf{x}(t))$ obtido pela base de regras Φ . Considerando a partição *fuzzy* da variável de saída definida por uma partição *fuzzy* triangular (cf. equação (8.9)) o valor da aproximação $\hat{y}(t)$ é calculada pelo operador defuzzificação como:

$$\hat{y}(t) = \sum_{j=1}^m v_j(t)\beta_j = \hat{\mathbf{v}}(t)\boldsymbol{\beta}^T \quad (8.44)$$

onde o vetor $\boldsymbol{\beta} = (\beta_1, \dots, \beta_m)$ são os protótipos da partição *fuzzy* triangular de saída.

8.2.3.4 Modelos TS

Sistemas *fuzzy* simbólicos são equivalentes aos sistemas *fuzzy* TS de ordem zero, em que as regras são escritas como $A_k \rightarrow \theta_k, k = 1, \dots, n$. Inserindo a equação (8.43) na equação (8.44) e igualando à saída do sistema *fuzzy* TS (cf. equação (8.37)), temos:

$$\hat{y}(t) = \hat{\mathbf{v}}(t)\boldsymbol{\beta}^T = \mathbf{u}(t)\boldsymbol{\Phi}\boldsymbol{\beta}^T = \mathbf{u}(t)\boldsymbol{\theta}^T \quad (8.45)$$

onde $\boldsymbol{\theta} = (\theta_1, \dots, \theta_n)$ é o vetor de parâmetros do sistema *fuzzy* TS de ordem zero, em que a saída de cada regra $\hat{y}_k = \theta_k$.

A equação (8.45) mostra que um sistema *fuzzy* simbólico com base de regras $\boldsymbol{\Phi}$ é equivalente ao sistema *fuzzy* TS de ordem zero definido pelos parâmetros $\boldsymbol{\theta}$ quando:

$$\boldsymbol{\theta}^T = \boldsymbol{\Phi}\boldsymbol{\beta}^T \quad (8.46)$$

A partição *fuzzy* triangular é muito útil para a operação de defuzzificação, pois é complementar à defuzzificação definida na equação (8.44), de forma que os operadores de fuzzificação e defuzzificação satisfazem a seguinte propriedade [457]:

$$D(F(y, \mathbb{B}), \mathbb{B}) = y, \quad \forall y \in \mathcal{Y} \quad (8.47)$$

Dessa forma, para uma partição triangular $\mathbb{B}$, a fuzzificação e a defuzzificação são complementares e $\boldsymbol{\mu}_{\mathbb{B}}(\theta_i)\boldsymbol{\beta}^T = \theta_i$. Assim, um sistema *fuzzy* simbólico equivalente a um modelo TS de ordem zero é obtido quando cada componente da base de regras é calculado como $\varphi_{ij} = \mu_{\mathbb{B}_j}(\theta_i)$ [168]. A matriz de pesos de regras pode ser escrita como:

$$\boldsymbol{\Phi} = \begin{bmatrix} \boldsymbol{\mu}_{\mathbb{B}}(\theta_1) \\ \vdots \\ \boldsymbol{\mu}_{\mathbb{B}}(\theta_n) \end{bmatrix} \quad (8.48)$$

As equações (8.46) e (8.48) mostram que um modelo TS pode ser obtido de uma base de regras de um sistema *fuzzy* simbólico e, analogamente, uma base de regras pode ser calculada a partir do conjunto de parâmetros de um modelo TS. Essa equivalência só é possível se a partição de saída atender à propriedade definida pela equação (8.47), como no caso da partição *fuzzy* triangular. A extensão dessa propriedade de equivalência a modelos com saídas múltiplas $\mathbf{y}(t) \in \mathcal{R}^m$ é um problema em aberto.

A formulação dos sistemas *fuzzy* simbólicos permite a representação de modelos TS em que a função de saída é um modelo das variáveis de entrada. As regras do modelo TS genérico são escritas como $A_k \rightarrow \theta_k(\mathbf{x}), k = 1, \dots, n$, onde as saídas das regras são

definidas por uma função vetorial das variáveis de entrada $\boldsymbol{\theta}(\mathbf{x}): \mathcal{R}^p \rightarrow \mathcal{R}^n$. O resultado do modelo TS é calculado como:

$$\hat{y}(t) = \mathbf{u}(t)\boldsymbol{\theta}(\mathbf{x}(t))^T \quad (8.49)$$

onde $\boldsymbol{\theta}(\mathbf{x}(t)) = (\theta_1(\mathbf{x}(t)), \dots, \theta_n(\mathbf{x}(t)))$ é o vetor de parâmetros.

No modelo TS de primeira ordem, $\boldsymbol{\theta}(\mathbf{x}(t)) = [\boldsymbol{\theta}_0 + \mathbf{x}(t)\boldsymbol{\theta}_1]$, onde $\boldsymbol{\theta}_0$ é o vetor de parâmetros de ordem zero e $\boldsymbol{\theta}_1$ é uma matriz $p \times n$, com os parâmetros associados às variáveis de entrada nas colunas.

A aplicação de sistemas *fuzzy* simbólicos e suas variantes a um problema simples é ilustrada nos exemplos a seguir.

8.2.3.5 Exemplos

O primeiro exemplo é um classificador para o problema da Figura 8.21, definido por um sistema *fuzzy* simbólico de $n = 4$ regras geradas pela combinação das partições *fuzzy* $\mathbb{A}_1 = \{\mathcal{A}_{11}, \mathcal{A}_{12}\}$ e $\mathbb{A}_2 = \{\mathcal{A}_{21}, \mathcal{A}_{22}\}$. As regras são escritas como:

- Se x_1 é “pequeno” e x_2 é “pequeno”, então (x_1, x_2) é C_2 : $A_{11} \wedge A_{21} \rightarrow C_2$;
- Se x_1 é “pequeno” e x_2 é “grande”, então (x_1, x_2) é C_1 : $A_{11} \wedge A_{22} \rightarrow C_1$;
- Se x_1 é “grande” e x_2 é “pequeno”, então (x_1, x_2) é C_2 : $A_{12} \wedge A_{21} \rightarrow C_2$;
- Se x_1 é “grande” e x_2 é “grande”, então (x_1, x_2) é C_1 : $A_{12} \wedge A_{22} \rightarrow C_1$.

A base de regras do modelo é definida pela matriz $\boldsymbol{\Phi} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}^T$.

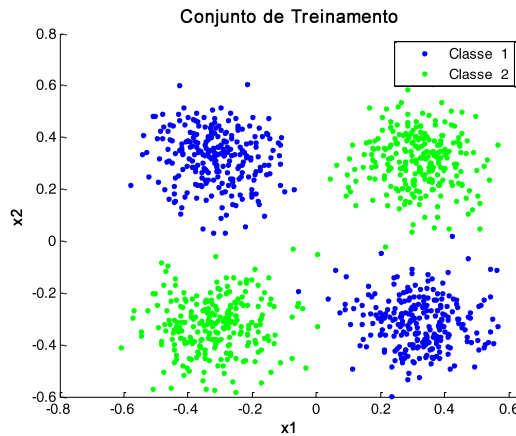


Figura 8.21: Conjunto de dados de duas classes não separáveis linearmente.

O resultado das funções de pertinência de cada classe é mostrado na Figura 8.22, que utiliza a paleta de cores da Figura 8.2. O resultado depende da escolha adequada dos protótipos e funções de pertinências para as partições *fuzzy* A_1 e A_2 . Uma escolha intuitiva é localizar os protótipos das funções de pertinência nas coordenadas das médias de cada um dos quatro grupos do conjunto de dados.

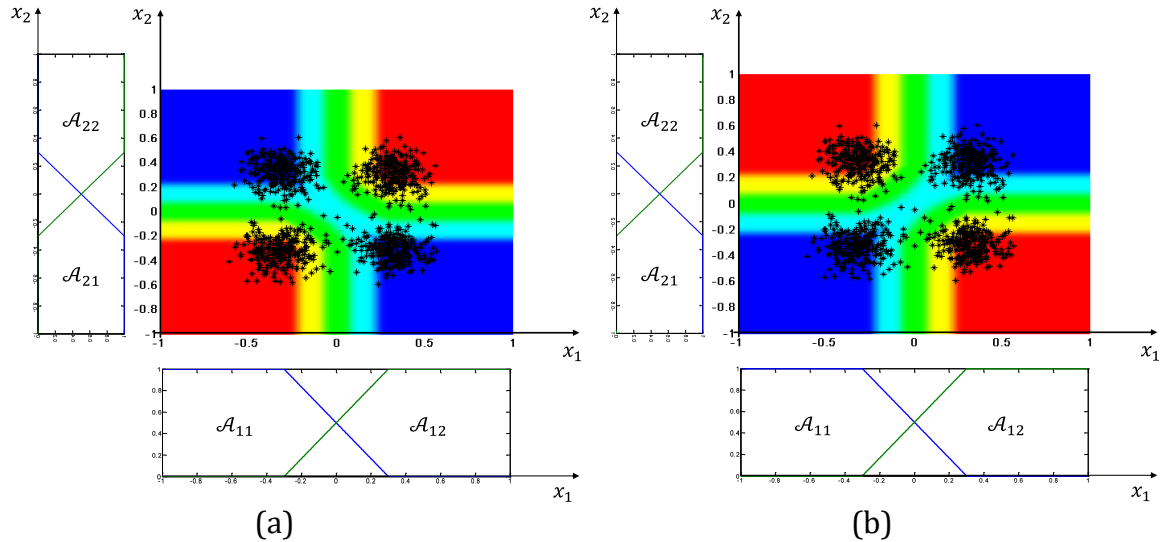


Figura 8.22: Sistema *fuzzy* simbólico: (a) classe C_1 ; (b) classe C_2 .

O segundo exemplo é um sistema *fuzzy* para aproximação de uma função quadrática monovariável. A Tabela 8.2 mostra quatro modelos em que as partições *fuzzy* de entrada e saída têm $n = 3$ e $m = 2$ conjuntos *fuzzy* respectivamente. Os modelos 1 e 4 apresentam regras binárias, enquanto os modelos 2 e 3 apresentam regras ponderadas. Os modelos TS equivalentes, calculados com $\beta = (-1, 1)$, também são mostrados na Tabela 8.2.

Os resultados são mostrados na Figura 8.23, na qual é possível observar que as regras ponderadas dos modelos 2 e 3 podem gerar resultados intermediários entre os extremos definidos pelas regras binárias dos modelos 1 e 4.

Os sistemas *fuzzy* simbólicos geram funções lineares por partes quando a descrição *fuzzy* da variável de entrada é calculada por uma partição *fuzzy* triangular (8.9). Os mesmos modelos produzem resultados diferentes com outras partições *fuzzy* da variável de entrada.

O ajuste dos parâmetros de modelos TS pode ser feito pela solução de um problema de mínimos quadrados simples, como descrito na seção 3.1.2. A formulação pode ser facilmente estendida para modelos TS de ordem superior [436].

Tabela 8.2: Modelos simbólicos de funções quadráticas

Modelo 1	Modelo 2	Modelo 3	Modelo 4
$A_1 \rightarrow B_2$	$A_1 \rightarrow B_1/0.2 \ B_2/0.8$	$A_1 \rightarrow B_1/0.7 \ B_2/0.3$	$A_1 \rightarrow B_1$
$A_2 \rightarrow B_1$	$A_2 \rightarrow B_1/0.9 \ B_2/0.1$	$A_2 \rightarrow B_1/0.4 \ B_2/0.6$	$A_2 \rightarrow B_2$
$A_3 \rightarrow B_2$	$A_3 \rightarrow B_1/0.2 \ B_2/0.8$	$A_3 \rightarrow B_1/0.7 \ B_2/0.3$	$A_3 \rightarrow B_1$
$\Phi_1 = \begin{bmatrix} 0.0 & 1.0 \\ 1.0 & 0.0 \\ 0.0 & 1.0 \end{bmatrix}$	$\Phi_2 = \begin{bmatrix} 0.2 & 0.8 \\ 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$	$\Phi_3 = \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \\ 0.7 & 0.3 \end{bmatrix}$	$\Phi_4 = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix}$
$A_1 \rightarrow \theta_1 = 1.0$	$A_1 \rightarrow \theta_1 = 0.6$	$A_1 \rightarrow \theta_1 = -0.4$	$A_1 \rightarrow \theta_1 = -1.0$
$A_2 \rightarrow \theta_2 = -1.0$	$A_2 \rightarrow \theta_2 = -0.8$	$A_2 \rightarrow \theta_2 = 0.2$	$A_2 \rightarrow \theta_2 = 1.0$
$A_3 \rightarrow \theta_3 = 1.0$	$A_3 \rightarrow \theta_3 = 0.6$	$A_3 \rightarrow \theta_3 = -0.4$	$A_3 \rightarrow \theta_3 = -1.0$

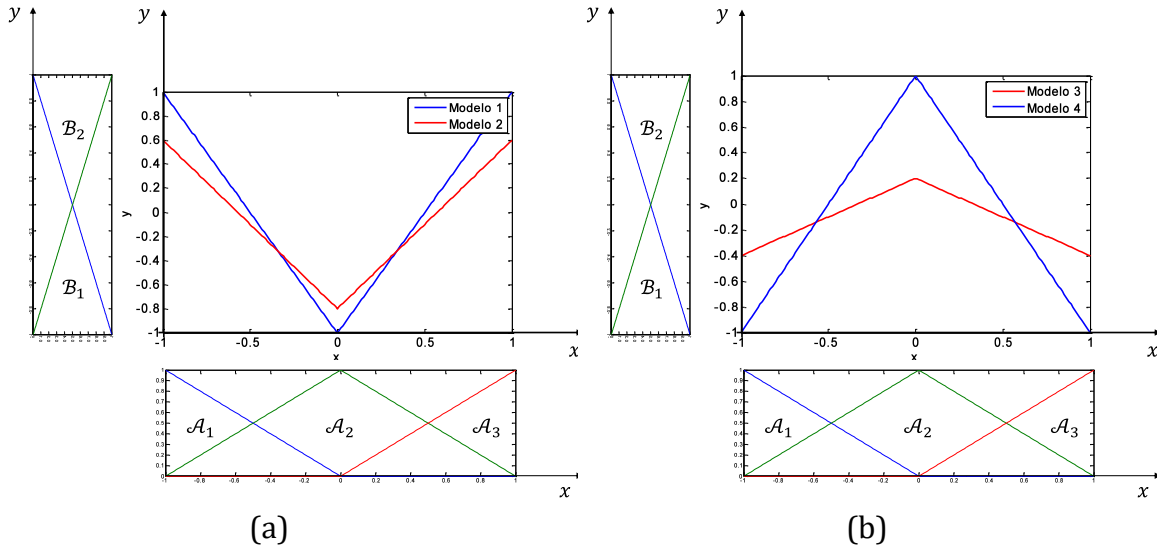


Figura 8.23: Sistema *fuzzy* simbólico da Tabela 8.2: (a) modelos 1 e 2; (b) modelos 3 e 4.

O ajuste de parâmetros do sistema *fuzzy* simbólico também pode ser feito pela solução de um problema QP (cf. equação (7.31)), como um modelo RBF [167][168][170]. O ajuste de parâmetros pela solução de um problema QP é utilizado no ajuste do modelo *fuzzy*-RBF, apresentado na seção 8.3.4.

8.3 Algoritmos para Ciência de Dados

Há diversas aplicações de sistemas *fuzzy* em ciência de dados [271][272]. A principal motivação para utilização de sistemas *fuzzy* é a possibilidade de interpretação das regras *fuzzy* [271][272].

Esta seção apresenta os principais algoritmos para aplicações de ciência de dados. A seção 8.3.1 apresenta o algoritmo *fuzzy c-médias* (FCM), uma adaptação do *k-médias* para o cálculo de partições *fuzzy* sobre um conjunto de dados. O algoritmo *fuzzy pattern matching* [79][142], na seção 8.3.2, é um algoritmo de classificação que resulta num modelo semelhante ao classificador bayesiano simples. Na seção 8.3.3 apresentamos o algoritmo de Wang & Mendel [547], que é um algoritmo clássico para indução de regras *fuzzy*. Finalmente, na seção 8.3.4, apresentamos o modelo *fuzzy*-RBF, que é um modelo linear local que utiliza funções de base radial como funções de pertinência.

8.3.1 Fuzzy c-médias

O algoritmo de agrupamento *fuzzy c-médias* (FCM) foi desenvolvido por Dunn em 1973 [148] e posteriormente aperfeiçoado por Bezdek [47][48].

O algoritmo FCM é uma generalização do *k-médias* para o cálculo de uma partição *fuzzy* de K subconjuntos *fuzzy* do conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$. O algoritmo FCM gera uma partição *fuzzy* forte $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, em que as condições definidas pelas equações (8.4) e (8.6) se verificam.

Assim como o *k-médias*, o algoritmo FCM retorna as coordenadas dos centros de agrupamentos pela minimização da função de avaliação quadrática:

$$J(\mathbf{\Omega}) = \sum_{t=1}^N \sum_{i=1}^K u_i(t)^\gamma \|\mathbf{x}(t) - \mathbf{\omega}_i\|^2 \quad (8.50)$$

onde $u_i(t) = \mu_{\mathcal{C}_i}(\mathbf{x}(t))$ é o valor da função de pertinência de um registro $\mathbf{x}(t)$ ao grupo $\mathcal{C}_i$; $\gamma > 1$ é o parâmetro de forma;* $\mathbf{\omega}_i$ é o vetor das coordenadas do centro do grupo $\mathcal{C}_i$; e a matriz $\mathbf{\Omega} = [\mathbf{\omega}_1^T, \dots, \mathbf{\omega}_K^T]$ armazena as coordenadas dos centros de grupos.

A partição *fuzzy* FCM foi introduzida na equação (8.10) no caso monovariável. O parâmetro de forma γ define o grau de sobreposição entre as funções de pertinência da

* O parâmetro de forma é, em geral, denotado por m na literatura.

partição *fuzzy* resultante. Em sua formulação original, Dunn utilizou $\gamma = 2$ [148] e, posteriormente, Bezdek demonstrou a convergência do algoritmo para qualquer $\gamma > 1$. A Figura 8.7 mostram a partição *fuzzy* FCM com $\gamma = 1.2$.

O algoritmo FCM funciona de forma semelhante ao k-médias, calculando iterativamente os centros de agrupamento como uma média das coordenadas dos registros alocados ao mesmo grupo. No caso do FCM, cada registro pode pertencer a mais de um grupo e a média é ponderada pelos valores de pertinência:

$$\omega_i = \frac{\sum_{t=1}^N u_i(t)^\gamma \mathbf{x}(t)}{\sum_{t=1}^N u_i(t)^\gamma}, \quad i = 1, \dots, K \quad (8.51)$$

Os valores das funções de pertinência são calculados em seguida a partir das coordenadas dos centros de grupo pela extensão da equação (8.10) a múltiplas variáveis:

$$u_i(t) = \frac{\|\mathbf{x}(t) - \omega_i\|^{\frac{2}{\gamma-1}}}{\sum_{j=1}^K \|\mathbf{x}(t) - \omega_j\|^{\frac{2}{\gamma-1}}}, \quad i = 1, \dots, K, t = 1, \dots, N \quad (8.52)$$

Os valores de pertinência são calculados pela equação (8.52) para todos os registros e todos os grupos e são armazenados na matriz de partição $\mathbf{U} \in [0,1]^{N \times K}$, em que a descrição *fuzzy* de cada registro é armazenada nas linhas.

O algoritmo FCM é descrito pelo Algoritmo 8.1. Assim como no k-médias, a inicialização é aleatória, o que gera os mesmos problemas de convergência para mínimos locais. O resultado do algoritmo é a matriz de centros de grupos e a matriz de partição, que possibilita avaliar a pertinência de registros a cada grupo.

O resultado do algoritmo FCM é semelhante ao resultado do k-médias, embora no FCM há o processamento e armazenamento da matriz de partição. Dessa forma, o algoritmo FCM é mais complexo computacionalmente e é indicado quando a interpretação da função de pertinência de grupos é importante para o problema.

O algoritmo FCM tem diversas variantes descritas na literatura [259][420]. O algoritmo FCM-GK, proposto por Gustafson e Kessel [223], utiliza uma distância ponderada para cálculo da função de pertinência na equação (8.52):

$$\|\mathbf{x}(t) - \omega_i\|_{A_i} = (\mathbf{x}(t) - \omega_i) \mathbf{A}_i (\mathbf{x}(t) - \omega_i)^T \quad (8.53)$$

A métrica (8.53) é uma versão *fuzzy* da distância de Mahalanobis (cf. equação (2.9)) através da definição da matriz de covariâncias *fuzzy* calculada como:

$$\mathbf{S}_i = \frac{\sum_{t=1}^N u_i(t)^\gamma (\mathbf{x}(t) - \boldsymbol{\omega}_i)^T (\mathbf{x}(t) - \boldsymbol{\omega}_i)}{\sum_{t=1}^N u_i(t)^\gamma} \quad (8.54)$$

A matriz de ponderação $\mathbf{A}_i$ é calculada como a inversa da matriz de covariância *fuzzy*:

$$\mathbf{A}_i = (\det(\mathbf{S}_i))^{-\frac{1}{p}} \mathbf{S}_i^{-1} \quad (8.55)$$

onde $\det(\mathbf{S}_i)$ é o determinante da matriz $\mathbf{S}_i$, de forma que $\det(\mathbf{A}_i) = 1$, $i = 1, \dots, K$, para que os grupos tenham aproximadamente o mesmo volume.

O algoritmo *fuzzy* c-médias “possibilístico” (PFCM) é uma variante do FCM em que uma penalidade é adicionada à função de avaliação (8.50) para reduzir o valor de pertinência de registros distantes do centro do grupo. O PFCM, proposto por Krishnapuram e Keller [322][323], também pode ser descrito pelo Algoritmo 8.1, com uma expressão diferente para o cálculo da função de pertinência. O PFCM também é utilizado para detecção de *outliers* [54].

Algoritmo 8.1: Algoritmo FCM

Entrada:	Conjunto de dados $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, parâmetro de forma γ e estimativa inicial de centros $\boldsymbol{\Omega} = [\boldsymbol{\omega}_1^T, \dots, \boldsymbol{\omega}_K^T]$
Saída:	Matriz de centros $\boldsymbol{\Omega} = [\boldsymbol{\omega}_1^T, \dots, \boldsymbol{\omega}_K^T]$ e matriz de partição $\mathbf{U}$
01	Início
02	Calcular a matriz de partição inicial $\mathbf{U}$ pela equação (8.52)
03	$\varepsilon \leftarrow \infty$
04	Enquanto $\varepsilon > tol$
05	$\boldsymbol{\Omega}_0 \leftarrow \boldsymbol{\Omega}$
06	Calcular $\boldsymbol{\Omega}$ pela equação (8.51)
07	Calcular $\mathbf{U}$ pela equação (8.52)
08	Calcular critério de parada $\varepsilon = \ \boldsymbol{\Omega}_0 - \boldsymbol{\Omega}\ $
09	Fim Enquanto
10	Retorna $\boldsymbol{\Omega} = [\boldsymbol{\omega}_1^T, \dots, \boldsymbol{\omega}_K^T]$ e $\mathbf{U}$
11	Fim

A Figura 8.24 mostra os resultados do FCM no conjunto de dados *Wine* 2D, utilizando os valores $\gamma = 1.2$ na coluna da esquerda e $\gamma = 1.8$ na coluna da direita. A primeira

linha da Figura 8.24 mostra a matriz de partição e a segunda linha as funções de pertinência. Nesse exemplo, a separação dos registros é a mesma para os dois valores do parâmetro de forma γ , e semelhante ao resultado do k-médias. Entretanto, a matriz de partição e as funções de pertinência geradas pelo algoritmo são diferentes. Dessa forma, a região de incerteza entre os grupos (área verde nas Figura 8.24c e d) é maior no caso de $\gamma = 1.8$. Os registros na região de incerteza podem ter um tratamento diferenciado no contexto do problema.

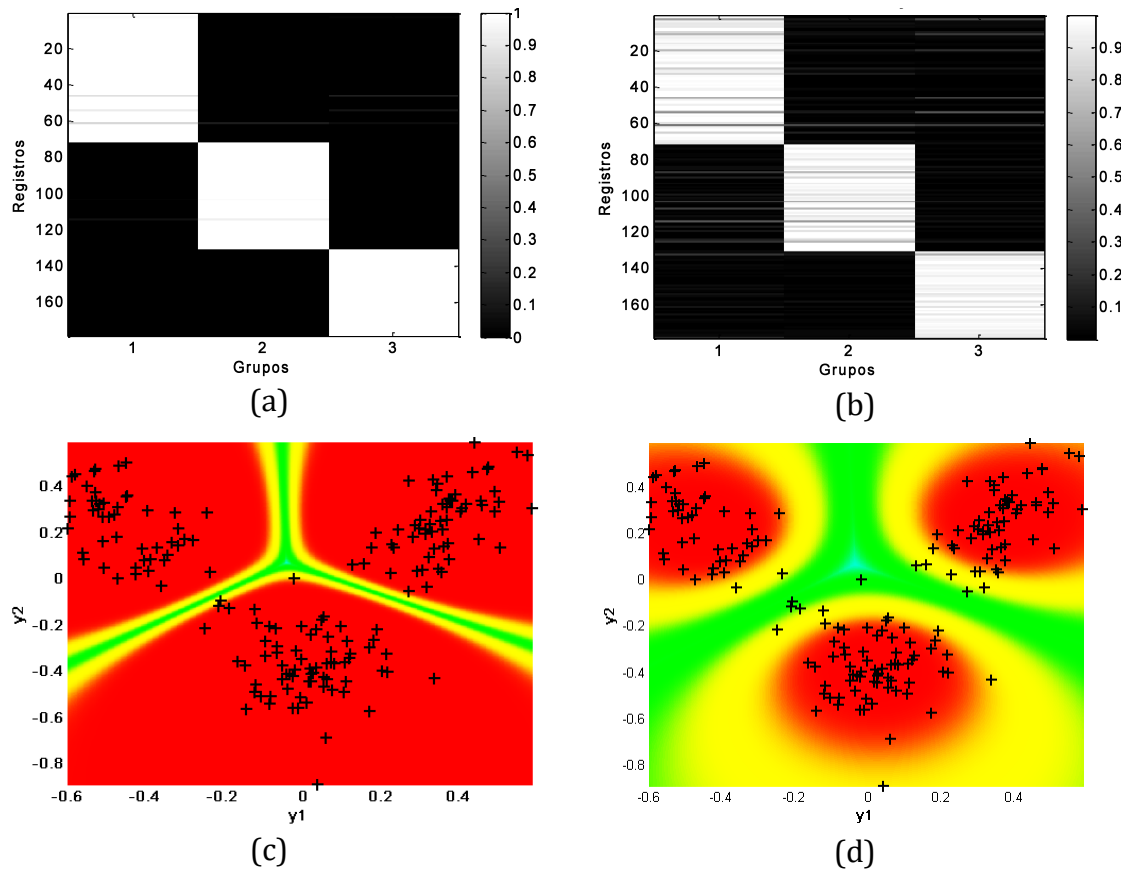


Figura 8.24: Resultados do FCM nos dados *Wine* 2D: (a) (c) $\gamma = 1.2$; (b) (d) $\gamma = 1.8$.

No algoritmo FCM-GK, uma matriz de ponderação $\mathbf{A}_i$ é calculada para cada grupo, o que afeta seu desempenho em problemas com grande número de variáveis. Entretanto, esse algoritmo pode ser uma alternativa em problemas de pequenas dimensões, pois representa melhor os registros de cada grupo. A Figura 8.25 mostra os resultados do FCM-GK com o parâmetro de forma $\gamma = 1.2$ e $\gamma = 1.8$. É possível perceber o formato elipsoide dos grupos formados, principalmente com $\gamma = 1.8$.

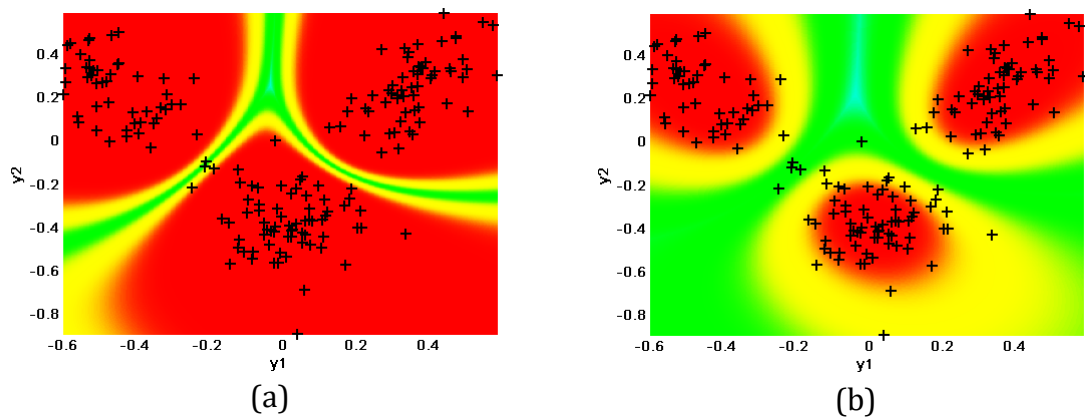


Figura 8.25: Resultados do FCM nos dados *Wine* 2D: (a) $\gamma = 1.2$; (b) $\gamma = 1.8$.

O algoritmo FCM (e suas variantes) é um algoritmo de análise de agrupamentos. A utilização de sistemas *fuzzy* em problemas preditivos é discutida a seguir.

8.3.2 Fuzzy pattern matching

O algoritmo *fuzzy pattern matching* (FPM) foi desenvolvido em 1982 no contexto da teoria das possibilidades para avaliar a compatibilidade entre uma observação e um padrão definido por uma distribuição de possibilidades [79][142]. O algoritmo é simples e eficiente e originou várias extensões [155][214]. Nesta seção, apresentamos uma formulação do algoritmo que utiliza sistemas *fuzzy* simbólicos para classificação.

O algoritmo FPM utiliza a mesma hipótese de independência do classificador bayesiano simples. O algoritmo FPM é mais flexível, pois não precisa de uma hipótese sobre a forma da distribuição de probabilidades da classe no domínio de cada variável. O FPM utiliza um sistema *fuzzy* para a estimativa dessas distribuições, a partir das partições *fuzzy* de cada variável.

Considere o conjunto de dados de treinamento $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$, em que cada registro $(\mathbf{x}(t), y(t))$, $\mathbf{x}(t) \in \mathcal{R}^p$ e $y(t) \in \{1, \dots, m\}$, de forma que $y(t) = j$ associa a classe $\mathcal{C}_j$ ao registro $\mathbf{x}(t)$. O objetivo do classificador *fuzzy* é calcular a partição *fuzzy* $\mathbb{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_m\}$, onde cada conjunto *fuzzy* $\mathcal{C}_j$ está associado à classe $\mathcal{C}_j$.

O algoritmo FPM calcula, para cada variável, um conjunto *fuzzy* que representa uma estimativa parcial das probabilidades *a posteriori* das classes, considerando os registros observados da variável isoladamente. As estimativas parciais são posteriormente agregadas através de um operador de agregação.

Para cada variável $x_i, i = 1, \dots, p$, a partição *fuzzy* $\mathbb{A}_i = \{\mathcal{A}_{i1}, \dots, \mathcal{A}_{in_i}\}$ é utilizada para definir um conjunto de regras $A_{ik} \rightarrow C_j$ ponderadas pelos parâmetros φ_{ikj} . Os parâmetros de cada variável são armazenados na matriz Φ_i , de dimensão $n_i \times m$:

$$\Phi_i = \begin{bmatrix} \varphi_{i11} & \dots & \varphi_{i1m} \\ \vdots & \ddots & \vdots \\ \varphi_{in_i1} & \dots & \varphi_{in_im} \end{bmatrix} \quad (8.56)$$

As estimativas parciais são calculadas através da inferência *fuzzy* (cf. equação (8.43)):

$$\hat{\mathbf{v}}_i(t) = \mathbf{u}_i(t)\Phi_i \quad (8.57)$$

onde $\hat{\mathbf{v}}_i(t) = (\hat{v}_{i1}(t), \dots, \hat{v}_{im}(t))$ e $\hat{v}_{ij}(t) \approx P(C_j|x_i(t))$; $\mathbf{u}_i(t) = \boldsymbol{\mu}_{\mathbb{A}_i}(x_i(t))$ é a descrição *fuzzy* do valor $x_i(t)$; e Φ_i é a base de regras do modelo associado à variável x_i .

O algoritmo FPM deriva seu nome da comparação do valor observado na variável de entrada com os padrões armazenados nas regras. Na formulação de sistemas *fuzzy* simbólicos, a regra $A_{ik} \rightarrow C_j$ é ponderada por um parâmetro que representa a probabilidade *a posteriori* da classe C_j , dada a observação do valor A_{ik} como:

$$\varphi_{ikj} \approx P(C_j|A_{ik}) \quad (8.58)$$

Como no classificador bayesiano simples, a estimativa final $\hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$ é calculada, para cada classe, pela agregação de todas as estimativas parciais através de um operador de agregação:

$$\hat{v}_j(t) = H(\hat{v}_{1j}(t), \dots, \hat{v}_{pj}(t)), \quad j = 1, \dots, m \quad (8.59)$$

onde $\hat{v}_j(t) \approx P(C_j|\mathbf{x}(t))$ e H é um operador de agregação que representa uma semântica para a agregação das conclusões parciais. Em problemas de classificação, o operador de agregação é uma t-norma, que representa a conjunção entre as estimativas parciais de todas as variáveis. Outros operadores podem representar semânticas mais flexíveis em aplicações de apoio à decisão [214][498].

Na equação (8.57), a estimativa parcial é calculada, para cada variável, para todas as classes e, na equação (8.59), uma estimativa final para cada classe é calculada pela agregação das estimativas parciais da classe para todas as variáveis.

A agregação pela t-norma na equação (8.59) implica que o padrão observado confere com o padrão armazenado para todas as variáveis. Considerando a t-norma produto e

os pesos definidos como (8.58), o resultado da agregação é similar ao classificador bayesiano simples:

$$\hat{v}_j(t) = \prod_{i=1}^p \hat{v}_{ij}(t) \approx \prod_{i=1}^p P(C_j | x_i(t)) \quad (8.60)$$

O classificador FPM é apresentado no Algoritmo 8.2, onde o operador T na linha 07 é uma t-norma aplicada aos elementos correspondentes dos vetores. O processamento do algoritmo FPM é ilustrado na Figura 8.26.

Algoritmo 8.2: Classificador *fuzzy pattern matching*

Entrada:	Conjunto de dados de teste $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$, matriz de pesos de regras Φ_i e partições <i>fuzzy</i> $\mathbb{A}_1, \dots, \mathbb{A}_p$
Saída:	Vetor de estimativas $\hat{\mathbf{v}}(t) = (v_1(t), \dots, v_m(t)), t = 1, \dots, N$
01	Início
02	Para $t = 1, \dots, N$
03	Inicializar $\hat{\mathbf{v}}(t) = \mathbf{1}$
04	Para $i = 1, \dots, p$
05	Calcular $\mathbf{u}_i(t) = F(x(t), \mathbb{A}_i)$
06	Calcular $\hat{\mathbf{v}}_i(t) = \mathbf{u}_i(t) \Phi_i$
07	$\hat{\mathbf{v}}(t) \leftarrow T(\hat{\mathbf{v}}(t), \hat{\mathbf{v}}_i(t))$
08	Fim Para
09	Fim Para
10	Retorna $\hat{\mathbf{v}}(t) = (v_1(t), \dots, v_m(t)), t = 1, \dots, N$
11	Fim

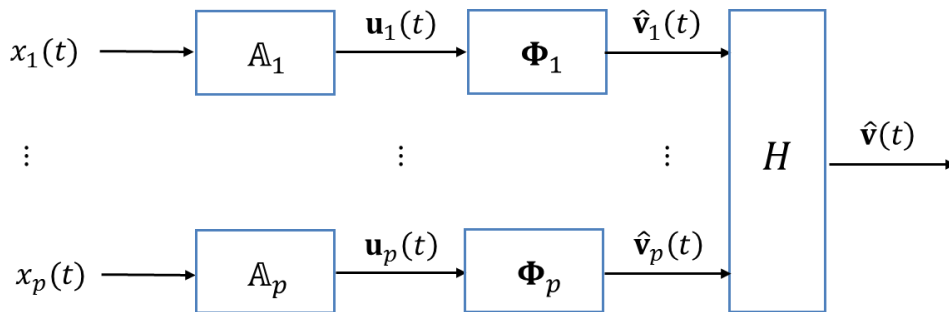


Figura 8.26: Processamento do algoritmo FPM.

O algoritmo FPM pode ser parametrizado pelo o número de conjuntos *fuzzy* da partição de cada variável no vetor $\mathbf{n} = (n_1, \dots, n_p)$. Nesse caso, a partição *fuzzy* de cada variável é definida por protótipos equidistantes. Podemos simplificar ainda mais a parametrização definindo o mesmo número de conjuntos *fuzzy* nas partições *fuzzy* de todas as variáveis $n_i = \eta, i = 1, \dots, p$, onde $\eta \in \mathcal{N}$. Dessa forma, o classificador FPM pode ser definido por apenas um parâmetro. Em geral utiliza-se partições *fuzzy* triangulares.

Os parâmetros da base de regras Φ_i são calculados para refletir a aproximação probabilidade *a posteriori* da classe C_j (cf. equação (8.58)). O ajuste de parâmetros é feito pelo operador *sigma-count* (cf. equação (8.21)):

$$\varphi_{ikj} = \Sigma(C_j | \mathcal{A}_{ik}) = \frac{\sum_{t=1}^N \mu_{\mathcal{A}_{ik}}(x_i(t)) v_j(t)}{\sum_{t=1}^N \mu_{\mathcal{A}_{ik}}(x_i(t))} \quad (8.61)$$

onde $v_j(t) \in \{0,1\}$ é variável indicadora binária da classe C_j :

$$v_j(t) = \begin{cases} 1, & \text{se } \mathbf{x}(t) \in C_j \\ 0, & \text{se } \mathbf{x}(t) \notin C_j \end{cases} \quad j = 1, \dots, m \quad (8.62)$$

Os parâmetros estimados pela equação (8.61) realizam uma aproximação da probabilidade *a posteriori* da classe o que já inclui o efeito da probabilidade *a priori* (cf. equação (5.1)). Em problemas desbalanceados, é possível compensar a influência da probabilidade *a priori* ponderando a equação (8.61) por um vetor $\mathbf{w}_j \in \mathcal{R}^N$, para cada classe C_j , em que o elemento $w_j(t)$ representa o peso do registro $t \in \mathcal{T}$ na referida classe, como no MMQP (cf. seção 5.2.2). Os pesos podem ser definidos pelo complemento da probabilidade *a priori*, como na equação (5.21), ou outro valor que reflita o risco de classificação incorreta. Nesse caso os parâmetros das regras são definidos como:

$$\varphi_{ikj} = \frac{\sum_{t=1}^N \mu_{\mathcal{A}_{ik}}(x_i(t)) v_j(t) w_j(t)}{\sum_{t=1}^N \mu_{\mathcal{A}_{ik}}(x_i(t))} \quad (8.63)$$

O algoritmo FPM tem grande flexibilidade para síntese de classificadores. Além do número de elementos das partições *fuzzy* de cada variável, é possível alterar o resultado do classificador utilizando a t-norma mínimo e realizar o ajuste com ou sem ponderação. O algoritmo FPM pode ser adaptado para problemas de regressão. Entretanto, o desempenho do modelo não é bom, pois a hipótese de independência das variáveis raramente se verifica em problemas de regressão. Outras fórmulas para especificação de pesos em sistemas *fuzzy* foram estudadas por Ishibushi e Yamamoto [280].

A Figura 8.27 mostra os resultados do classificador FPM no problema *Wine* 2D, com $\eta = 5$ elementos na partição *fuzzy* de cada variável e ajuste de parâmetros pela equação (8.63). A Figura 8.27a apresenta os pesos de regras calculados para cada variável e a Figura 8.27b, a superfície de decisão.

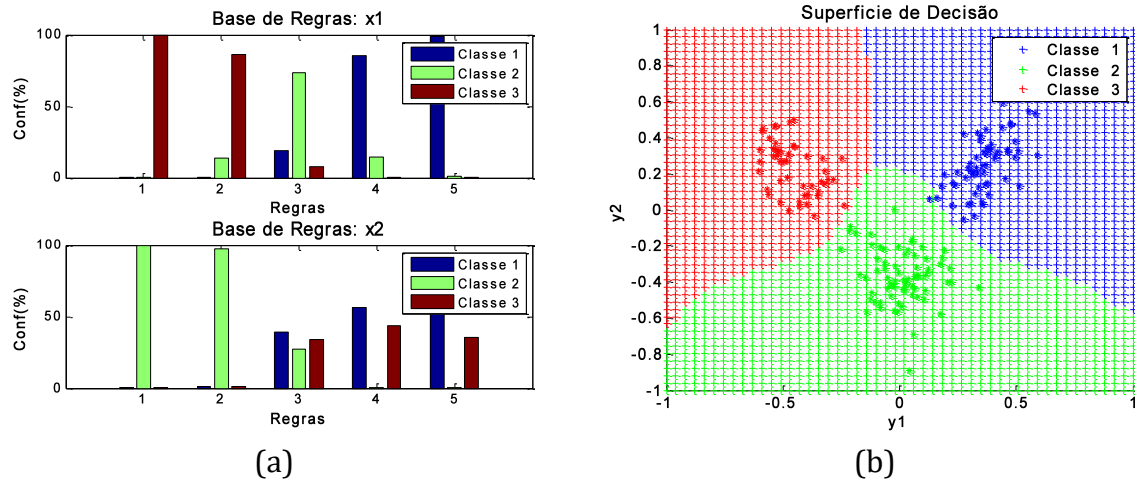


Figura 8.27: Resultado do classificador FPM na base *Wine* 2D:

(a) Pesos de regras para variável x_1 (acima) e x_2 (abaixo); (b) superfície de decisão.

A Figura 8.28 mostra os resultados, em validação cruzada, de 10 ciclos do classificador FPM no conjunto de dados de Diabetes da índias utilizando $\eta = 5$ elementos na partição *fuzzy* de cada variável. A Figura 8.28a mostra o resultado do ajuste de parâmetros pela equação (8.61) e a Figura 8.28b, o resultado obtido pela equação (8.63).

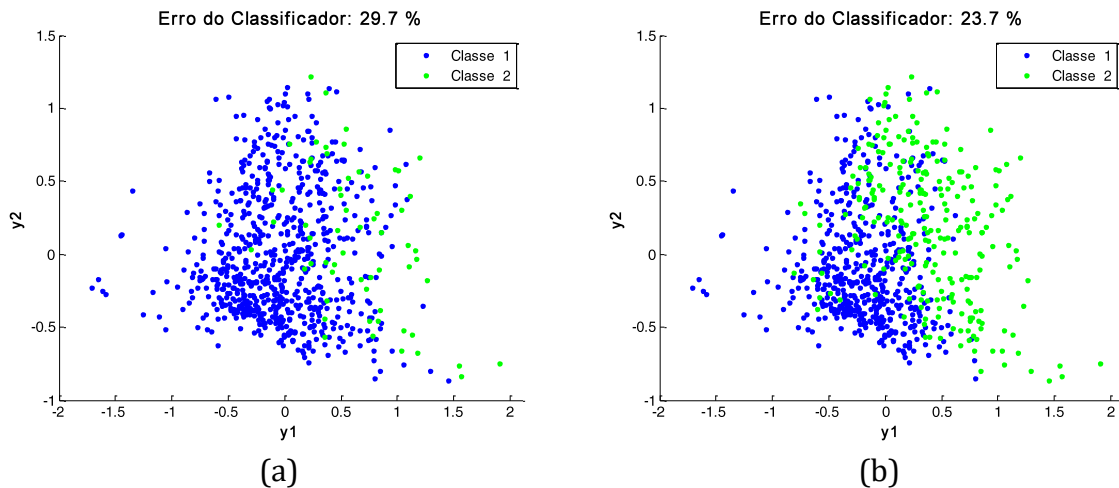


Figura 8.28: Resultado do classificador FPM na base Diabetes:

(a) Sem ponderação do complemento da probabilidade *a priori*; (b) com ponderação.

O classificador FPM tem características semelhantes ao classificador bayesiano simples, com a vantagem de permitir o ajuste de diferentes distribuições para cada variável a partir da escolha do número de elementos da partição *fuzzy*. O ajuste de pesos para o classificador FPM é simples e computacionalmente eficiente. O classificador foi utilizado em aplicações de diagnóstico [155] e previsão de condições de tráfego [160].

O algoritmo FPM também pode ser utilizado em sistemas de apoio a decisão. Essa abordagem foi utilizada em aplicações para localização de aterros sanitários [15][343] e avaliação de imóveis [500], onde um operador OWA foi utilizado para agregação. O algoritmo FPM também foi utilizado para representação de consultas flexíveis em banco de dados [165]. O processamento independente dos atributos pode ser realizado em múltiplos processadores [164], o que torna o algoritmo altamente escalável.

Modelos mais complexos em problemas de classificação e regressão podem ser obtidos com regras *fuzzy* de múltiplos antecedentes geradas pelo algoritmo de indução de regras de Wang & Mendel, apresentado a seguir.

8.3.3 Algoritmo de Wang & Mendel

O número de regras de múltiplos antecedentes cresce exponencialmente em função do número de variáveis caso cada regra seja gerada pela combinação de todas as variáveis de entrada (cf. equação (8.26)). Em aplicações de ciência de dados, os registros se concentram numa região do domínio e muitas das combinações de conjuntos *fuzzy* não são ativadas por nenhum registro do conjunto de treinamento. O algoritmo de indução de regras *fuzzy* proposto em 1992 por Wang e Mendel [547] gera um subconjunto de todas as combinações possíveis capaz de cobrir a região da amostra do conjunto de dados.

O algoritmo WM [547], gera as regras *fuzzy* pela conjunção de conjuntos *fuzzy* definidos em cada variável de entrada (cf. Figura 8.11). O algoritmo de indução das regras verifica, para cada registro, os conjuntos *fuzzy* mais ativados na partição *fuzzy* de cada variável e cria uma nova regra definida pela conjunção desses conjuntos *fuzzy* caso ela ainda não tenha sido criada por registros anteriores. Dessa forma, cada registro dá origem a, no máximo, uma regra e o número de regras geradas pelo algoritmo é, no máximo, o número de registros do conjunto de dados de treinamento. O número de con-

juntos *fuzzy* na partição *fuzzy* de cada variável de entrada, como antes, pode ser generalizado pelo parâmetro η considerando partições regulares. Em geral, utiliza-se as funções de pertinência triangular ou gaussiana.

O algoritmo WM é descrito pelo Algoritmo 8.3. Os conjuntos *fuzzy* de cada regra são armazenados na matriz Ξ (cf. equação (8.40)), que é gerada, para cada nova regra, no passo 08. O número de regras final é o número de linhas da matriz Ξ . O algoritmo WM não utiliza a informação da variável de saída e as regras são geradas considerando apenas a informação das variáveis de entrada.

Algoritmo 8.3: Algoritmo de indução de regras Wang & Mendel

Entrada:	Conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$ e vetor $\mathbf{n} = (n_1, \dots, n_p)$ que define as partições <i>fuzzy</i>
Saída:	Matriz de regras Ξ
01	Início
02	Para $t = 1, \dots, N$
03	Para $i = 1, \dots, p$
04	Calcular $\mathbf{u}_i(t) = F(x_i(t), \mathbb{A}_i)$
05	Atribuir $\xi_{ki} = j$, se $u_{ij}(t) = \max(\mathbf{u}_i(t))$
06	Fim Para
07	Se $\xi_k = (\xi_{k1}, \dots, \xi_{kp}) \notin \Xi$
08	$\Xi \leftarrow \Xi \cup \xi_k$
09	Fim Se
10	Fim Para
11	Retorna matriz de regras Ξ
12	Fim

Em problemas de classificação, as regras *fuzzy* resultantes podem ser avaliadas pelas medidas de suporte e confiança, calculadas pelas estatísticas:

$$Sup(A_i \rightarrow C_j) = \frac{100}{N} \sum_{t=1}^N u_i(t) v_j(t) \quad (8.64)$$

$$Conf(A_i \rightarrow C_j) = \frac{\sum_{t=1}^N u_i(t) v_j(t)}{\sum_{t=1}^N u_i(t)} \quad (8.65)$$

As estatísticas de suporte e confiança são comuns na avaliação de regras de associação [144]. O suporte da regra $A_i \rightarrow C_j$ representa o percentual de registros da classe C_j que

ativam a regra definida por A_i e a confiança da regra $A_i \rightarrow C_j$ é uma estimativa da probabilidade *a posteriori* $P(C_j|A_i)$ (cf. equação (8.61)).

A Figura 8.29 ilustra o funcionamento do algoritmo WM em duas variáveis do conjunto de dados Íris com $\eta = 3$ conjuntos *fuzzy* para cada variável e funções de pertinência triangulares. Observe que, das nove combinações possíveis, somente $n = 5$ regras foram geradas nas regiões cobertas por registros do conjunto de dados de treinamento.

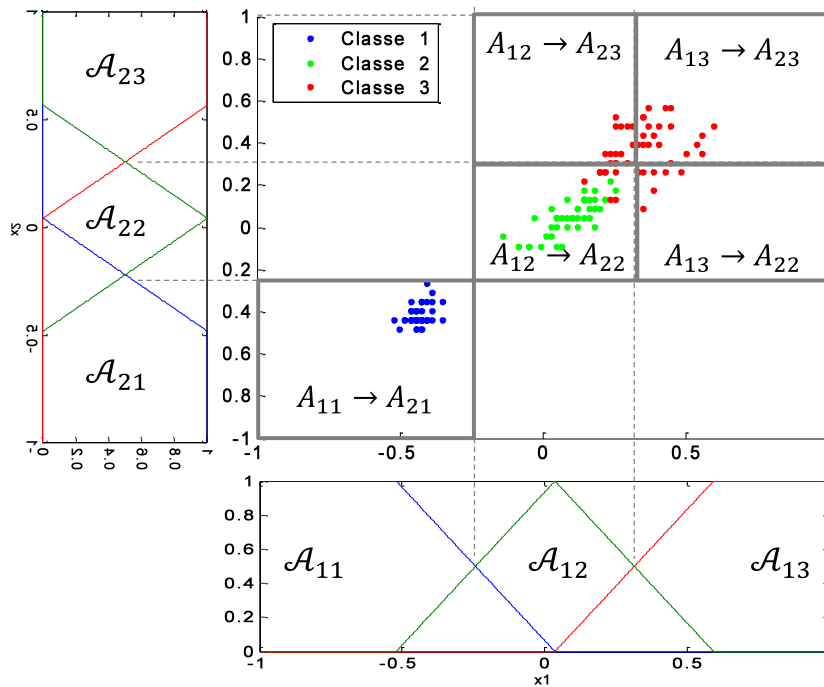
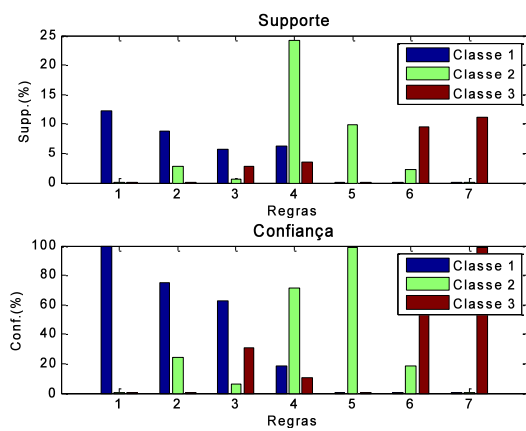


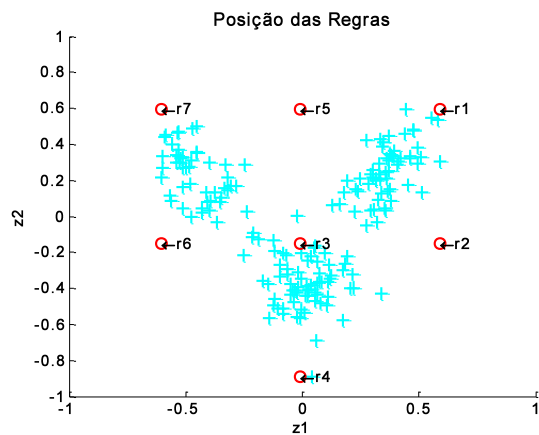
Figura 8.29: Resultado do algoritmo WM nas variáveis x_3 e x_4 da base Íris.

A Figura 8.30 mostra o resultado do algoritmo WM aplicado ao conjunto de dados *Wine* 2D utilizando partições *fuzzy* gaussianas. O algoritmo resultou em $n = 7$ regras, cujos parâmetros foram calculados como solução do problema QP (cf. equação (7.31)). A Figura 8.30a apresenta as métricas de suporte e confiança e a Figura 8.30b mostra a posição das regras no plano das variáveis. Observe que as posições das regras são definidas pelas posições dos protótipos das funções de pertinência em cada variável.

As regras foram ordenadas visando agrupar regras relativas às mesmas classes. A Figura 8.31a-c mostra as funções de pertinência geradas para cada classe, onde a posição das regras é representada por 'o' e os registros são representados por '+'. A Figura 8.31d mostra a superfície de decisão.

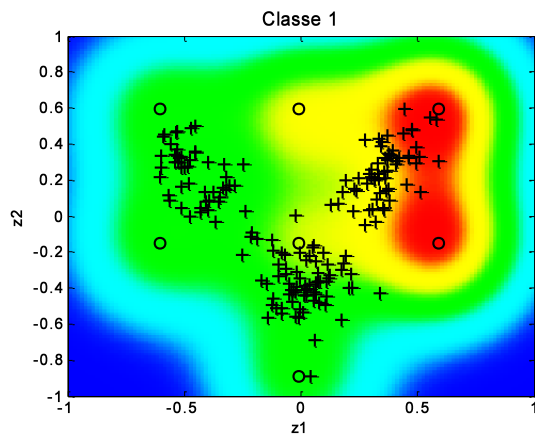


(a)

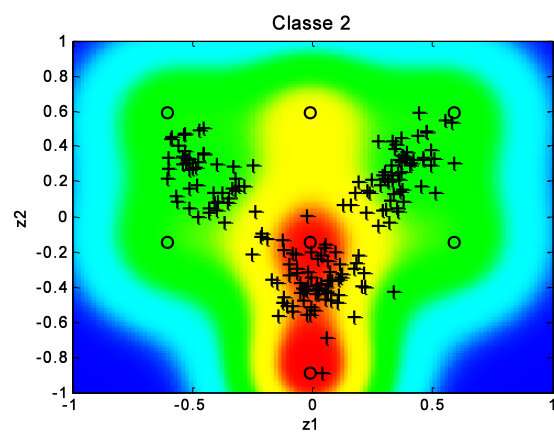


(b)

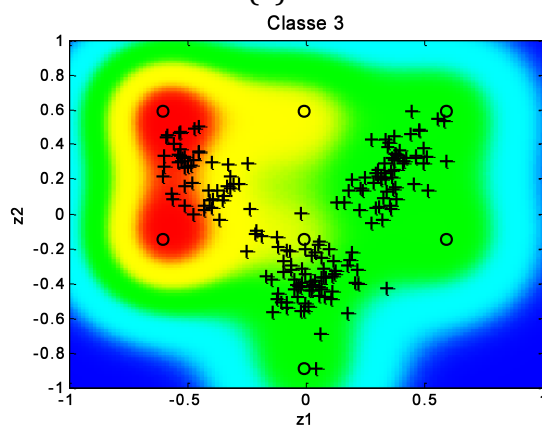
Figura 8.30: Resultados do algoritmo WM na base Wine 2D com $\eta = 3$: (a) suporte e confiança das regras; (b) posição das regras no plano das variáveis.



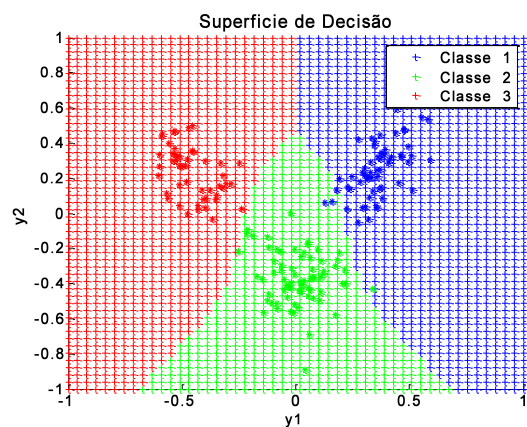
(a)



(b)



(c)



(d)

Figura 8.31: Resultados do algoritmo WM na base Wine 2D com $\eta = 3$: (a) classe C_1 ; (b) classe C_2 ; (c) classe C_3 ; (d) superfície de decisão.

O algoritmo WM é um algoritmo de indução de regras e pode ser empregado também em problemas de regressão. Wang e Mendel utilizam conjuntos *fuzzy* com funções de pertinência gaussiana em problemas de regressão, em que o ajuste de parâmetros é feito pelo algoritmo de mínimos quadrados ortogonais [548]. Os parâmetros são ajustados por um modelo TS de ordem zero que posteriormente pode ser convertido em um sistema *fuzzy* simbólico pela equação (8.48).

O algoritmo WM tem a tendência de gerar muitas regras em problemas de alta dimensionalidade. A aplicação do algoritmo no conjunto de dados *Wine* original, com $p = 13$ variáveis, resulta em $n = 134$ regras, o que é muito considerando que a amostra tem $N = 178$ registros. A posição das regras é definida pelos protótipos das funções de pertinência e podem não corresponder à posição dos agrupamentos de registros, como pode ser observado na Figura 8.31. Por esses motivos, o classificador gerado pelo algoritmo WM não tem um desempenho satisfatório em muitos problemas.

Há diversas variantes do algoritmo WM na literatura com objetivo de reduzir o número de regras geradas [577][580]. O algoritmo WM também tem sido combinado com algoritmos genéticos [167][337][354], o que resulta numa boa relação entre desempenho e interpretação, tendo em vista o grande número de combinações possíveis. Há também algoritmos que realizam o processo de indução de regras de forma incremental, incluindo apenas as regras que contribuem significativamente para a redução do erro do modelo [158][159][407]. Há diversas outras abordagens para a indução de regras *fuzzy* [279] e o assunto permanece um tópico de pesquisa ativo.

O modelo de indução de regras pela combinação de conjuntos *fuzzy* nos domínios das variáveis de entrada tem a vantagem de permitir sua interpretação na forma de regras linguísticas. Entretanto, para problemas de alta dimensionalidade, o número de regras geradas é muito grande, o que inviabiliza a interpretação do modelo e dificulta o ajuste de parâmetros. Nesse caso, a solução é a indução de regras diretamente no domínio multidimensional das variáveis de entrada, como descrito a seguir.

8.3.4 Modelo *fuzzy*-RBF

As regras de um sistema *fuzzy* podem ser definidas diretamente sobre o espaço multidimensional das variáveis de entrada através de uma partição *fuzzy* multidimensional, como no algoritmo FCM (cf. seção 8.3.1).

A partição FCM não apresenta bons resultados como função de base em modelos de aproximação. A versão multidimensional da partição gaussiana (8.8) é bem mais comum em modelos de inteligência computacional. Nesse caso, o valor de ativação de cada regra é calculado como:

$$u_k(t) = \exp\left(-\frac{\|\mathbf{x}(t) - \boldsymbol{\omega}_k\|^2}{\rho_k^2}\right), \quad k = 1, \dots, n \quad (8.66)$$

onde o vetor $\boldsymbol{\omega}_k$ define o centro da regra k e ρ_k é um parâmetro de dispersão.

Em geral, é conveniente utilizar partições *fuzzy* fortes, de tal forma que o valor de ativação (8.66) é padronizado como:

$$\hat{u}_k(t) = \frac{u_k(t)}{\sum_{i=1}^n u_i(t)}, \quad k = 1, \dots, n \quad (8.67)$$

O resultado da fuzzificação é a descrição *fuzzy* $\hat{\mathbf{u}}(t) = (\hat{u}_1(t), \dots, \hat{u}_n(t))$, que pode ser utilizada em modelos de regras *fuzzy* em que a saída é calculada pela equação (8.43):

$$\hat{\mathbf{v}}(t) = \hat{\mathbf{u}}(t)\boldsymbol{\Phi} \quad (8.68)$$

onde $\hat{\mathbf{v}}(t)$ é a descrição *fuzzy* de saída e $\boldsymbol{\Phi}$, a matriz de pesos de regras.

Os modelos *fuzzy* TS também são escritos de forma análoga e a saída do modelo TS de ordem zero é calculada como:

$$\hat{y}(t) = \hat{\mathbf{u}}(t)\boldsymbol{\theta}^T \quad (8.69)$$

onde $\boldsymbol{\theta} = (\theta_1, \dots, \theta_n)$ é o vetor de parâmetros.

A equação (8.69) define um modelo equivalente a um modelo linear local RBF apresentado no capítulo anterior. A equivalência entre modelo RBF e o modelo *fuzzy* TS de ordem zero foi muito discutida na década de 1990 [240][265][286][548][579], quando esse tipo modelo foi chamado de **neuro-fuzzy** [22][288], a partir do sistema ANFIS,* proposto em 1993 [287].

Como todo modelo RBF, o modelo *fuzzy*-RBF com funções de pertinência gaussianas é equivalente ao modelo de máquina de vetor de suporte (SVM) com funções de núcleo

* O sistema ANFIS (*Adaptive Neuro Fuzzy Inference System*) é um modelo TS de primeira ordem com regras definidas pelo usuário através da combinação de conjuntos *fuzzy* das variáveis de entrada.

gaussianas [69][301][478]. Dessa forma, o algoritmo de ajuste de parâmetros do SVM pode ser utilizado para extração de regras de um sistemas *fuzzy* [75][96][99].

O ajuste de parâmetros de saída do modelo *fuzzy* RBF é feito pela solução do problema de mínimos quadrados simples ou com restrições (QP) [170]. Os pesos de regras *fuzzy* do modelo *fuzzy* simbólico correspondente podem ser obtidos pela conversão definida pela equação (8.48).

O número de regras influi na capacidade do modelo *fuzzy*-RBF de ajustar funções complexas. Este efeito pode ser observado no exemplo do conjunto das duas luas, apresentado na Figura 8.32. Nesse caso, a superfície de decisão é não linear e só pode ser estimada adequadamente com um número maior de regras. A primeira linha da Figura 8.32 mostra o resultado com $n = 10$ regras, que não é suficiente para separar as classes. Com $n = 24$ regras as funções de pertinência das classes se ajustam à forma não convexa dos registros, como mostra a segunda linha da Figura 8.32.

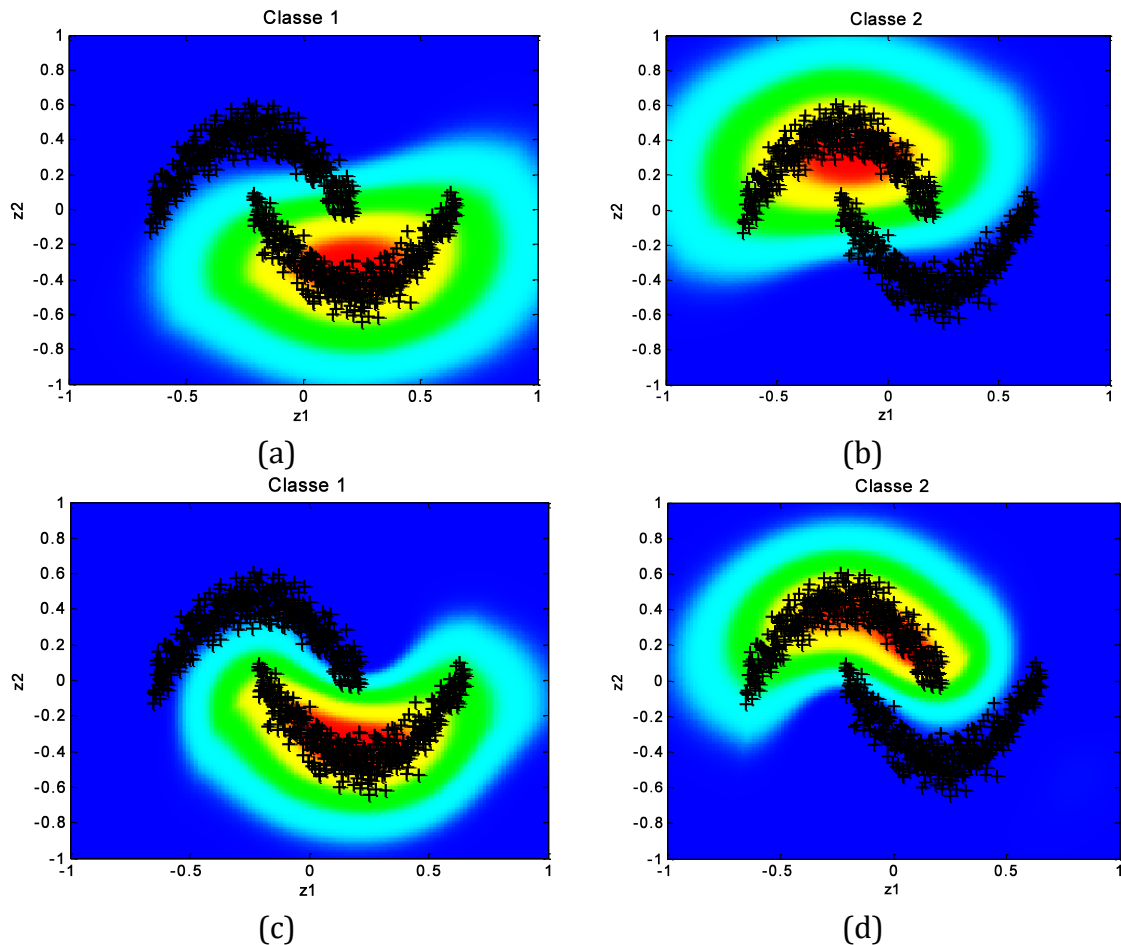


Figura 8.32: Resultados na base duas luas com: (a) e (b) $n = 10$ regras; (c) e (d) $n = 24$ regras.

Os elementos da matriz de parâmetros Φ são mostrados na Figura 8.33, onde as cores correspondem às classes. Podemos observar que há regras que definem os protótipos de cada classe, com parâmetros $\varphi_{k1} = 1$ ou $\varphi_{k2} = 1$, e outras regras que representam regiões próximas à fronteira entre as classes, com $0 < \varphi_{kj} < 1$.

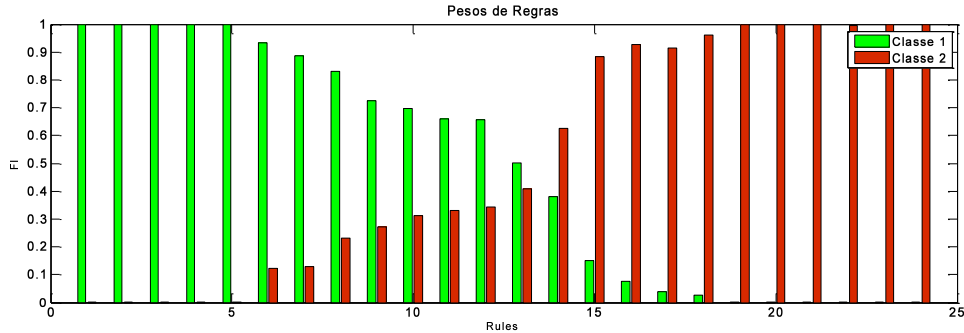


Figura 8.33: Parâmetros do modelo na base duas luas com $n = 24$ regras.

8.4 Aplicações

Há um grande número de aplicações de sistemas *fuzzy* em ciência de dados na literatura [271][272]. A capacidade de interpretação de sistemas *fuzzy* tem sido valorizada em trabalhos recentes [170][204][272][384], embora a própria definição de interpretabilidade ainda seja tema em discussão [13][603].

Esta seção apresenta diferentes aplicações de sistemas *fuzzy* em ciência de dados. Na seção 8.4.1, o desempenho de classificadores *fuzzy* é comparado com outros modelos em testes *benchmark*. A seção 8.4.2 apresenta aplicações para o diagnóstico de falhas de processos complexos. A seção 8.4.3 utiliza um sistema *fuzzy* de apoio à decisão para classificação de imagens. Na seção 8.4.4, apresentamos um sistema *fuzzy* recorrente para modelagem do processo chuva-vazão.

8.4.1 Testes *benchmark*

Os resultados dos classificadores *fuzzy* nos testes *benchmark* são apresentados na Tabela 8.3. Os resultados dos modelos classificadores bayesiano e discriminante linear foram incluídos como referência. O classificador FPM foi executado variando-se o número de conjuntos *fuzzy* da entrada entre 2 e 16, a Tabela 8.3 apresenta os melhores resultados para cada conjunto de dados. Os resultados do modelo *fuzzy*-RBF são os mesmos do modelo RBF apresentado no capítulo anterior, incluídos para comparação.

Tabela 8.3: Resultados dos testes *benchmark*

<i>Benchmark</i>	Bayesiano		M. Lin. 2		FPM (2-16)		<i>Fuzzy</i> -RBF (8-64)	
	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>	<i>ACC</i>	<i>AUC</i>
Íris	98.00	0.9850	95.33	0.9650	95.33	0.9650	95.33	0.9650
Balance	91.68	0.9469	91.36	0.9171	91.68	0.9202	88.32	0.8908
Diabetes	72.92	0.7271	77.21	0.7151	75.65	0.7645	75.91	0.7120
Cancer	95.02	0.9559	97.22	0.9709	97.51	0.9780	97.36	0.9768
Statlog Heart	84.07	0.8433	84.44	0.8375	84.81	0.8442	83.70	0.8342
Wine	99.44	0.9958	99.44	0.9958	98.88	0.9915	94.94	0.9599
Parkinson	83.08	0.8325	86.15	0.8099	74.36	0.8095	90.77	0.8616
Ionosphere	86.89	0.8803	89.46	0.8602	92.59	0.9213	93.45	0.9303
Spambase*	90.11	0.8916	91.37	0.9006	91.60	0.9126	88.40	0.8727
Sonar	72.60	0.7166	80.29	0.8010	76.44	0.7637	81.25	0.8048

Os resultados na Tabela 8.3 mostram que o classificador FPM é melhor que o classificador bayesiano na maioria dos problemas, principalmente nos de maior dimensionalidade. Este resultado deve-se ao fato que o modelo bayesiano é restrito a distribuições normais, enquanto o classificador FPM é adaptativo a diferentes distribuições dos dados. O classificador FPM é um modelo com bom desempenho e baixo custo computacional. O modelo *fuzzy*-RBF é equivalente ao modelo RBF e tem desempenho melhor que o FPM em alguns casos, tendo em vista que ajusta funções de base diretamente no universo multidimensional. Os resultados mostram que os sistemas fuzzy têm resultado competitivo com outros métodos de estatística e inteligência computacional.

8.4.2 Diagnóstico de falhas

O diagnóstico de falhas em sistemas e equipamentos é o conjunto de técnicas de detecção, localização e identificação de falhas [276]. A detecção de falhas é a decisão sobre a presença (ou não) de falhas. O objetivo da localização da falha é isolar o subsistema que

gerou a falha, tendo em vista que seus efeitos podem propagar para outros subsistemas. A identificação da falha é a estimativa de sua gravidade para que medidas corretivas necessárias possam ser realizadas.

Existe uma grande variedade de abordagens para o diagnóstico de falhas envolvendo técnicas de estatística, processamento de sinal e inteligência computacional. A abordagem mais comum é a baseada em modelos [91][200][276], em que o funcionamento normal do processo, representado por um modelo, é comparado ao comportamento observado para a geração de resíduos, que são utilizados para o diagnóstico de falhas. Sistemas *fuzzy* têm aplicação em diagnóstico à base de modelos, tanto para a geração quanto para a análise de resíduos e apoio a decisão [277][431].

A teoria de conjuntos *fuzzy* é adequada para representar os conceitos subjetivos de estado de funcionamento normal e estado de falha. No diagnóstico baseado em modelos, um resíduo é a variável numérica que representa o estado de cada variável do sistema, que pode ser interpretado por conceitos linguísticos como “positivo”, “zero” e “negativo”. Apesar da diferença entre a representação de conjuntos *fuzzy* e distribuições de probabilidades [139][143], o conhecimento ou hipótese de que a série de resíduos segue uma distribuição normal pode auxiliar na determinação dos parâmetros dos conjuntos *fuzzy*. A suavidade da transição entre os valores do resíduo é apresentada por uma paleta de cores, conforme Figura 8.34.

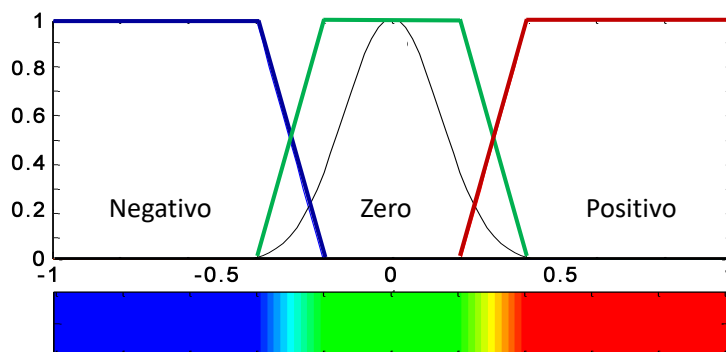


Figura 8.34: Representação de resíduos por conjuntos *fuzzy* e graus de cores.

A aplicação dos sistemas *fuzzy* simbólicos para a análise de resíduos gerados por um modelo causal linear de um processo complexo foi descrita em [156][157]. O estudo foi realizado sobre um processo complexo para reciclagem de resíduos nucleares. O processo real é totalmente acoplado, de forma que uma perturbação em alguma variável rapidamente se propaga por todo o sistema.

O processo foi modelado por um grafo causal cujos vértices são as principais variáveis e as arestas direcionadas representam modelos dinâmicos lineares entre as variáveis de entrada e saída da aresta. O modelo resultante mostrou-se preciso o suficiente para a geração de resíduos indicadores de falhas nas principais variáveis. A relação causal permite a identificação da variável onde se originou a falha, justificando, assim, o raciocínio do sistema e explicando o caminho de propagação das perturbações observadas (cf. Figura 8.35b).

Nas interfaces gráficas do sistema, o processo é representado por uma vista esquemática, mostrada na Figura 8.35a, e uma vista do modelo causal, apresentada na Figura 8.35b. Na vista esquemática, as variáveis estão organizadas segundo suas características funcionais no processo. Na vista causal, o relacionamento das variáveis é modelado por um modelo causal em que cada aresta representa um sistema dinâmico linear.

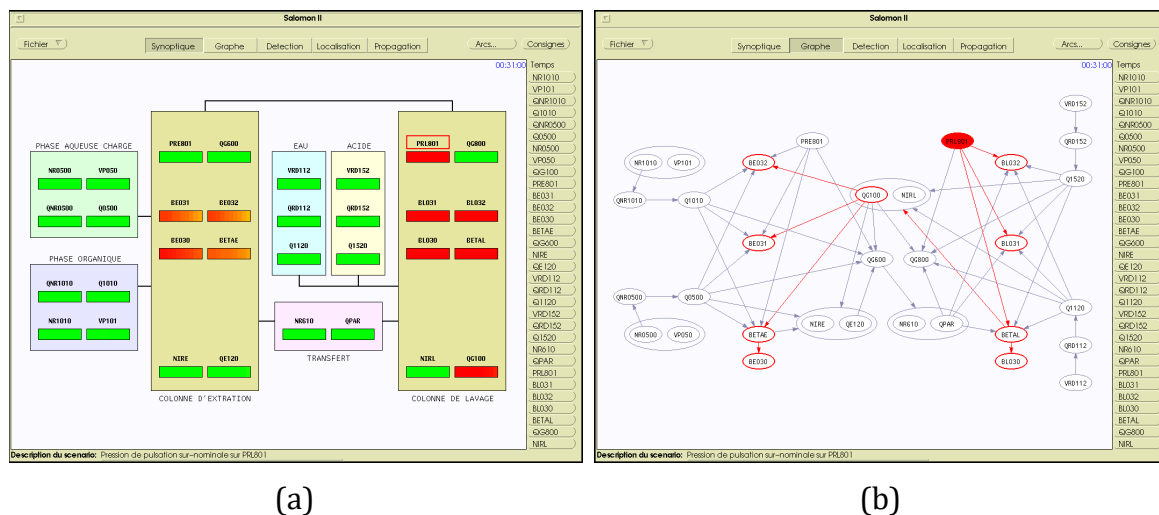
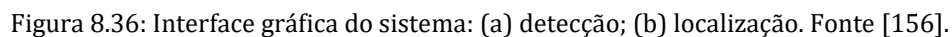


Figura 8.35: Interface gráfica do sistema: (a) esquemática; (b) causal. Fonte: [156].

O comportamento do processo é monitorado na vista esquemática, em que cada variável está associada a um valor qualitativo de seu estado, representado por uma cor. Na situação normal, todas as variáveis aparecem na cor verde. No caso de ocorrência de uma falha, as variáveis afetadas pelos desvios tornam-se gradativamente vermelhas e a variável reconhecida pelo sistema como a origem da falha é assinalada. No exemplo da Figura 8.35, a origem da falha é a variável PRL801. O resultado do diagnóstico é explicado através do grafo causal (Figura 8.35b), onde o caminho de propagação da falha é apresentado, mostrando a variável causa e aquelas que são afetadas pela falha.

Na interface gráfica da Figura 8.36, cada linha representa uma variável e cada coluna, um instante de tempo. A cada novo instante de tempo, uma nova coluna é incluída à direita do gráfico e a coluna mais à esquerda é descartada, de forma que o gráfico aparece movendo-se da direita para a esquerda. Na ausência de falhas, a tela aparece toda verde, sem movimento. Na ocorrência de uma falha, as linhas correspondentes às variáveis com desvios tornam-se gradualmente vermelhas na tela de detecção, como mostra a Figura 8.36a. Na tela de localização, mostrada na Figura 8.36b, apenas a origem das perturbações detectadas é apresentada em vermelho; as variáveis cujos desvios observados podem ser explicados pela propagação causal são apresentadas em azul.



O esquema do reator PWR é mostrado na Figura 8.37. No circuito primário, água líquida é bombeada a alta temperatura e pressão através do reator da perna fria para a perna

INTELIGÊNCIA COMPUTACIONAL

quente. O vapor para as turbinas é gerado no circuito secundário alimentado por água líquida do condensador, que é convertida em vapor no gerador de vapor. A água do circuito secundário não entra em contato com a água do circuito primário.

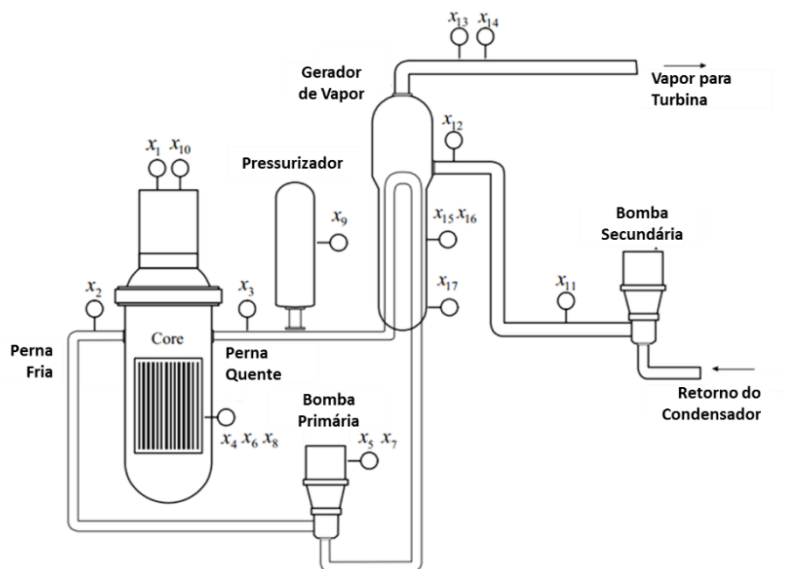


Figura 8.37: Posição das variáveis do reator PWR de Angra 2. Adaptado de [163].

A propagação dos desvios nas variáveis é mostrada na Figura 8.38a. A Figura 8.38b mostra a conclusão do sistema de diagnóstico obtida por um sistema *fuzzy* recorrente [163]. A primeira linha da Figura 8.38b representa o funcionamento normal e as demais linhas, cada uma das 16 falhas modeladas. A Figura 8.38 mostra a simulação de duas falhas cuja assinatura é idêntica até um determinado instante, assinalado na figura, mas que evoluem de forma diferente depois.

Esse exemplo mostra a complexidade do problema de diagnóstico de falhas e como sistemas de diagnóstico podem ser úteis para apoio à decisão de operadores em salas de supervisão de processos em caso de situações críticas. O exemplo também mostra a capacidade de memória do modelo *fuzzy* recorrente para realização de reconhecimento de padrões dinâmicos.

Os problemas descritos nesta seção abordam de processos complexos e de alto risco para os quais há simuladores confiáveis para o treinamento de operadores e a avaliação de cenários de falhas. Em problemas, a abordagem baseada em modelos não é viável por não existir um modelo de simulação confiável ou por seu desenvolvimento ser muito oneroso. Em alguns casos, modelos podem ser obtidos a partir do conhecimento de especialistas [297].

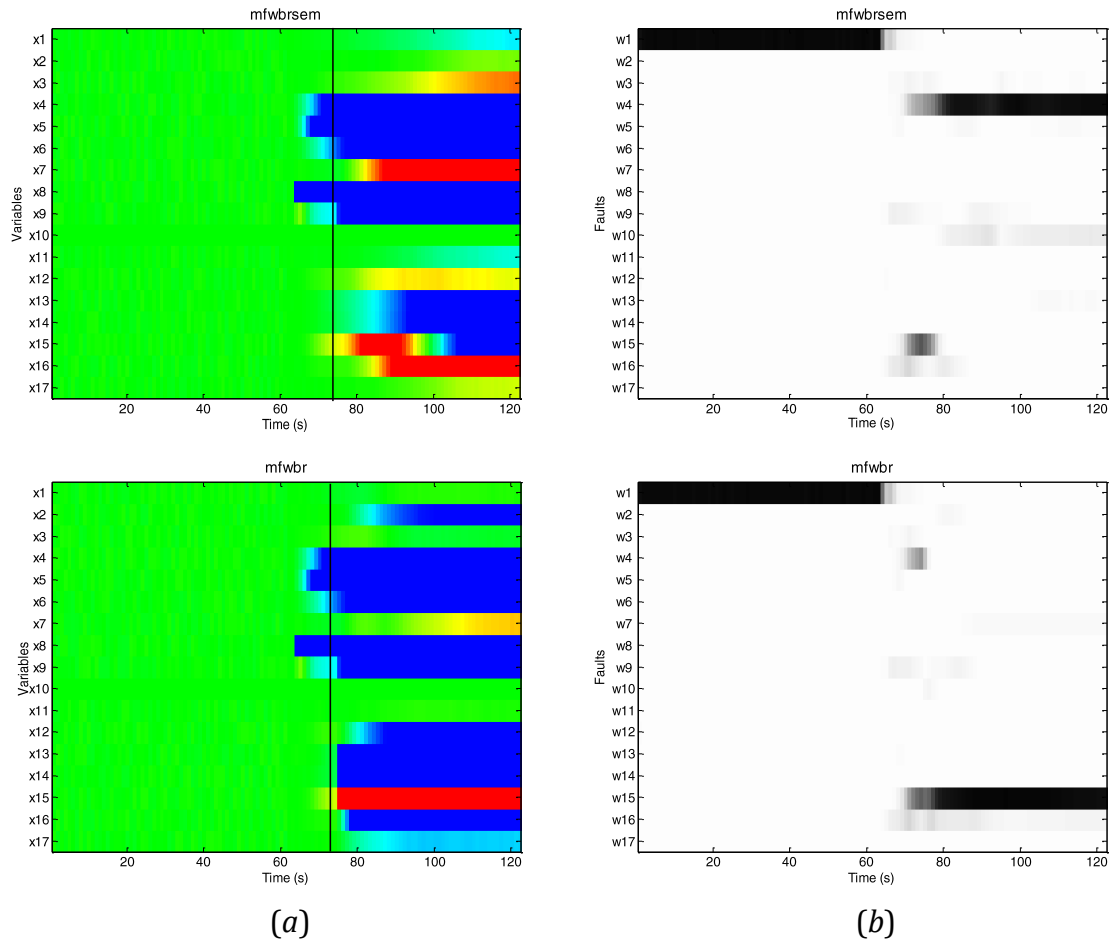


Figura 8.38: Resultados da simulação de falhas no reator nuclear da usina de Angra II: (a) detecção; (b) localização. Fonte: [163].

A próxima seção apresenta uma aplicação em que o conhecimento de especialistas é codificado num sistema *fuzzy* de apoio à decisão.

8.4.3 Classificação espacial

Na seção 6.6.1 foi apresentado a segmentação de imagens SAR das épocas de seca e de cheia da Amazônia ocidental, obtidas pelo satélite JERS-1, com o algoritmo de agrupamento textural de imagens USTC. Essa segmentação foi utilizada para a construção de mapas de sensibilidade ambiental a derramamento de óleo por Beisl [32].

Beisl fez uma avaliação da sensibilidade de cada combinação de mudança de paisagem identificada provocada pela cheia. A avaliação foi feita em função da severidade de um acidente e da probabilidade de ocorrência da mudança. A severidade da mudança de paisagem refere-se ao risco ambiental de um derramamento de óleo. A probabilidade da mudança foi calculada em função da área em *pixels* das imagens correspondentes.

A análise é feita por uma matriz de risco, apresentada na Tabela 8.4. Nessa matriz, a severidade da mudança é descrita por quatro valores linguísticos: “catastrófico”, “crítico”, “marginal” e “desprezível”; e a probabilidade da mudança, em cinco valores: “frequente”, “provável”, “ocasional”, “remoto” e “extremamente remoto”. As combinações de mudanças encontradas nos dados das imagens das duas datas foram posicionadas na matriz de acordo com suas características de severidade e probabilidade.

Tabela 8.4: Matriz de risco para o cálculo do ISAT

		PROBABILIDADE DA MUDANÇA EM UM PERÍODO DO CICLO HIDROLÓGICO				
		EXT.REM.	REMOT.	OCAS.	PROV.	FREQ.
SEVERIDADE AMBIENTAL A DERRAMES DE ÓLEO	CATASTRÓFICO		C_{10}		C_7	C_6, C_{11}
	CRÍTICO	C_{13}	C_5, C_9, C_{14}, C_{15}		C_8, C_{12}	
	MARGINAL			C_4	C_2, C_3	
	DESPREZÍVEL					C_{16}, C_1

O resultado da análise foi chamado de índice de Sensibilidade Ambiental Temporal (SAT), que representa o risco ambiental de uma região ao derrame de óleo em função da mudança de paisagem observada na imagem. O índice de sensibilidade foi representado por seis conceitos, conforme as cores mostradas na Tabela 8.4. O risco mais alto representa situações catastróficas e frequentes e o índice mais baixo, situações de danos desprezíveis que ocorrem com baixa frequência.

Valores numéricos foram atribuídos aos conceitos de risco ambiental correspondentes através do vetor de parâmetros $\theta \in \{0.15, 0.30, 0.40, 0.60, 0.80, 1.00\}$. Dessa forma, cada combinação de mudança de paisagem recebe um valor no intervalo $[0,1]$ correspondente ao índice; valores intermediários são calculados por um sistema *fuzzy*. Os

riscos qualitativos e os valores atribuídos às combinações de mudança de paisagem são apresentados na Tabela 8.5 [32].

A classificação de sensibilidade leva em consideração o acesso à área de vegetação inundada e floresta inundada, que é mais difícil. Dessa forma, a ocorrência de algum acidente nessas regiões teria piores consequências, uma vez que ações de contenção não poderiam ser realizadas para evitar o espalhamento do óleo.

Tabela 8.5: Índice SAT para cada classe de mudança de paisagem identificada

CLASSE	SECA (SET. 1995)	CHEIA (MAIO 1996)	RISCO AMBIENTAL	ISAT
C_1	Água	Água	Baixo	0.30
C_2	Vegetação Inundada	Água	Intermediário	0.40
C_3	Floresta Inundada	Água	Intermediário	0.40
C_4	Floresta Seca	Água	Baixo	0.30
C_5	Água	Vegetação Inundada	Baixo	0.30
C_6	Vegetação Inundada	Vegetação Inundada	Alto	1.00
C_7	Floresta Inundada	Vegetação Inundada	Alto	0.80
C_8	Floresta Seca	Vegetação Inundada	Intermediário	0.60
C_9	Água	Floresta Inundada	Baixo	0.30
C_{10}	Vegetação Inundada	Floresta Inundada	Intermediário	0.40
C_{11}	Floresta Inundada	Floresta Inundada	Alto	1.00
C_{12}	Floresta Seca	Floresta Inundada	Intermediário	0.60
C_{13}	Água	Floresta Seca	Baixo	0.15
C_{14}	Vegetação Inundada	Floresta Seca	Baixo	0.30
C_{15}	Floresta Inundada	Floresta Seca	Baixo	0.30
C_{16}	Floresta Seca	Floresta Seca	Baixo	0.30

Um sistema *fuzzy* TS de ordem zero foi utilizado para ajustar os valores do índice SAT em função dos valores do coeficiente de retroespalhamento, que determina a intensidade de cada *pixel* da imagem. O índice SAT calculado pelo sistema *fuzzy* é um valor contínuo no intervalo [0,1] para cada *pixel* da área de interesse.

Uma amostra de cada imagem JERS-1 SAR nos regimes de seca e cheia foi utilizada para o cálculo da partição *fuzzy* de entrada. A amostra foi realizada proporcionalmente à

extensão de cada classe encontrada no regime de cheia e de seca. Os valores do coeficiente de retroespalhamento das duas imagens foram considerados como variáveis de entrada do sistema *fuzzy* para o cálculo do índice SAT. O centro dos registros da amostra de cada classe de mudança de paisagem foi utilizado como o centro de uma função de pertinência *fuzzy* FCM (cf. equação (8.52)). A Figura 8.39 apresenta a partição *fuzzy* calculada sobre a amostra, em que cada conjunto *fuzzy* representa uma classe de mudança de paisagem.

É possível verificar na Figura 8.39 que algumas classes de mudança são bem mais frequentes que outras. Por exemplo, a classe C_{16} , que representa a região de floresta que se mantém seca nos dois regimes hidrológicos, é a mais frequente.

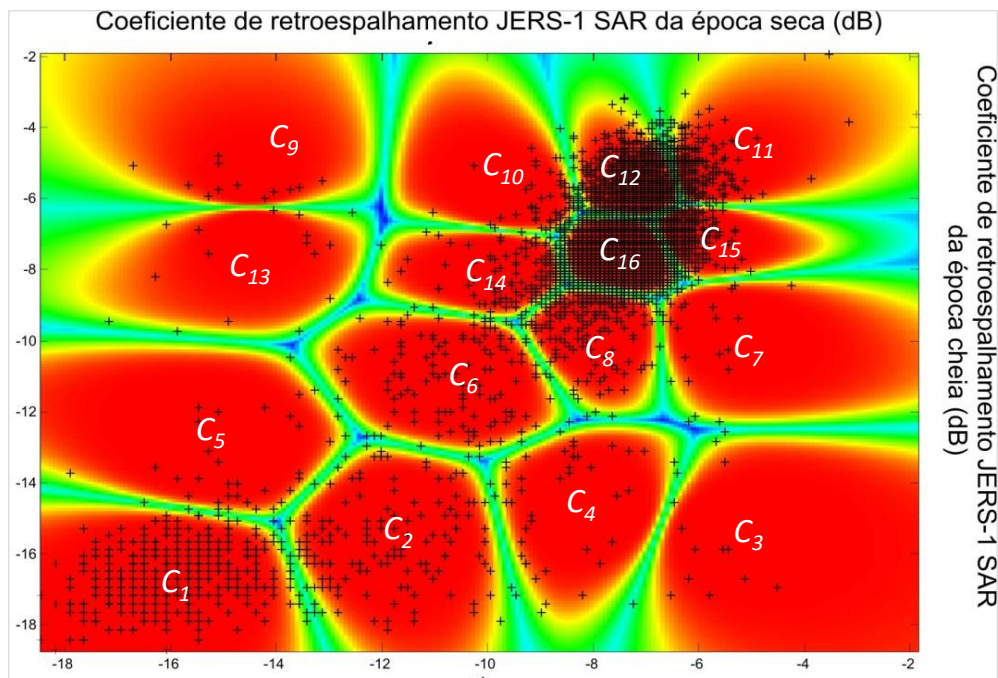


Figura 8.39: Amostra das imagens e partição *fuzzy* correspondente. Fonte: [32].

O índice SAT é calculado por um sistema *fuzzy*-RBF, em que a descrição *fuzzy* de entrada $\hat{\mathbf{u}}(t)$ é calculada pela partição *fuzzy* da Figura 8.39 e o vetor de parâmetros θ é definido na Tabela 8.5.

O resultado é um mapa de sensibilidade mostrado na Figura 8.40 em que os valores do índice SAT variam no intervalo $[0,1]$ e não há transição brusca entre classes adjacentes de mudança de paisagem. O resultado mostra uma grande faixa de floresta inundada com valor SAT entre 0.6 e 0.7 e algumas regiões pontuais com valores maiores.

O trabalho de Beisl mostra que a teoria de conjuntos *fuzzy* permite representar conceitos complexos e dar suporte à manipulação desses conceitos de acordo com os objetivos da análise. Os detalhes de todas as etapas que levaram à construção do índice SAT e do mapa da Figura 8.40 são descritos em [32].

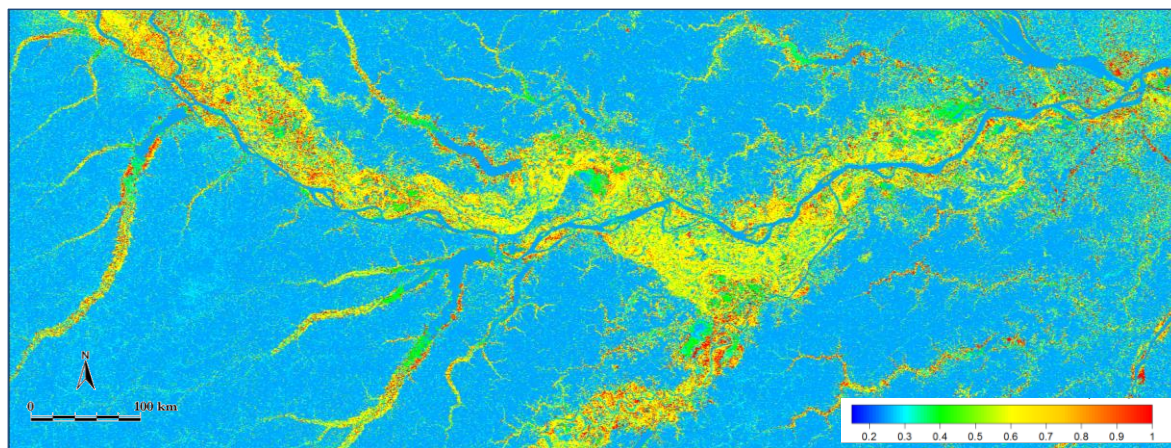


Figura 8.40: Resultados do Índice de Sensibilidade Ambiental Temporal. Fonte: [32].

8.4.4 Modelo chuva-vazão

O problema de modelagem chuva-vazão foi introduzido na seção 4.4.2 como uma aplicação de identificação de modelos dinâmicos lineares. No exemplo, foi mostrado que modelos preditivos não podem ser utilizados para simulação. O ajuste de parâmetros de modelos de simulação é feito pela solução de problemas de otimização não linear, mesmo para modelos lineares.

O modelo descrito nesta seção foi desenvolvido a partir de uma demanda do Operador Nacional do Sistema (ONS) [222] para modelos de previsão de vazões 12 dias à frente utilizando as previsões de precipitação fornecidas pelo modelo regional ETA [103] do Centro de Previsão do Tempo e Estudos Climáticos (CPETEC/Inpe).

O modelo selecionado para a bacia do rio Iguaçu e que passou a ser utilizado para as previsões semanais foi um modelo *fuzzy* recorrente [169][171], cujo ajuste é feito por um algoritmo genético [161]. O modelo *fuzzy* recorrente pode ser traduzido como um autômato finito *fuzzy* para representação da dinâmica do processo [191].

O modelo *fuzzy* recorrente é descrito por equações de estado não lineares como:

$$\begin{cases} \mathbf{z}(t+1) = f(\mathbf{z}(t), \mathbf{x}(t)) \\ \hat{y}(t) = g(\mathbf{z}(t), \mathbf{x}(t)) \end{cases} \quad (8.70)$$

onde $\mathbf{z}(t)$ é o vetor de estados, $\mathbf{x}(t)$ é o vetor de entradas e $\hat{y}(t)$, o valor de saída do modelo. As funções não lineares f e g são, respectivamente, as funções de transição de estados e de saída.

No modelo chuva-vazão da bacia do rio Iguaçu, foi utilizada uma versão simplificada das equações de estado (8.70). Uma única variável de entrada foi definida como a soma de toda a chuva sobre a bacia e os estados do modelo foram definidos pelos valores defasados da previsão de saída, como $\mathbf{z}(t) = (\hat{y}(t-1), \hat{y}(t-2))$. Nesse caso, não há equação de saída e o modelo é escrito diretamente como:

$$\hat{y}(t) = f(\mathbf{z}(t), x(t)) = f(\hat{y}(t-1), \hat{y}(t-2), x(t)) \quad (8.71)$$

No modelo *fuzzy* recorrente, a função de transição f na equação (8.71) é calculada por um sistema *fuzzy* recorrente [161][191]. A característica do sistema *fuzzy* recorrente é que a variável de saída aparece também na premissa das regras e cada regra é escrita na forma de um modelo TS de ordem zero como:

$$\text{Se } \hat{y}(t-1) \text{ é } A_i \text{ E } \hat{y}(t-2) \text{ é } A_j \text{ E } x(t) \text{ é } B_k \rightarrow \hat{y}(t) = \theta_r \quad (8.72)$$

onde A_i e A_j são elementos da partição *fuzzy* da variável de estados $\mathbb{A}$ e B_k é um elemento da partição *fuzzy* da variável de entrada $\mathbb{B}$. O parâmetro θ_r é o parâmetro de saída correspondente à regra r que relaciona os termos A_i , A_j e B_k .

No modelo da bacia do rio Iguaçu, foram utilizados cinco conjuntos *fuzzy* para a representação da chuva na bacia numa partição *fuzzy* triangular em que os protótipos foram ajustados pelos percentis para permitir o tratamento das distribuições altamente assimétricas de chuva (cf. Figura 4.25a). As variáveis de estados (saídas do modelo) são definidas no mesmo domínio, padronizado no intervalo $[0,1]$, e representadas pela mesma partição de dois conjuntos *fuzzy*. A Figura 8.41 mostra as partições *fuzzy* das variáveis de estados e da variável de entrada, onde cada conjunto *fuzzy* é representado pelo termo linguístico correspondente.

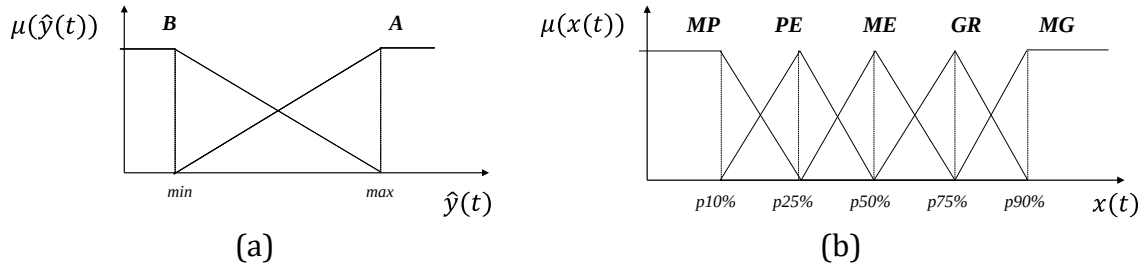


Figura 8.41: Partições *fuzzy* das variáveis: (a) de estados $\mathbb{A}$; (b) de entrada $\mathbb{B}$. Fonte: [169].

A fuzzificação do modelo é calculada pela combinação das descrições *fuzzy* correspondentes (cf. equações (8.38) e (8.39)) como:

$$\mathbf{u}(t) = \mu_{\mathbb{A}}(\hat{y}(t-1)) \otimes \mu_{\mathbb{A}}(\hat{y}(t-2)) \otimes \mu_{\mathbb{B}}(x(t)) \quad (8.73)$$

A combinação de dois termos na partição *fuzzy* das duas variáveis de estado com cinco termos para a representação da variável de entrada, resulta em $n = 20$ regras *fuzzy* recorrentes. A saída do modelo é calculada como um modelo TS de ordem zero (cf. equação (8.45)):

$$\hat{y}(t) = \mathbf{u}(t)\boldsymbol{\theta}^T \quad (8.74)$$

onde $\boldsymbol{\theta}$ é o vetor de parâmetros.

O modelo regional ETA apresenta um erro muito grande a partir do quarto dia de previsão [103], de forma que sua utilização para o ajuste de parâmetros poderia comprometer a qualidade do modelo. A abordagem utilizada para a modelagem, como mostra o esquema da Figura 8.42, foi utilizar a chuva observada para o ajuste do modelo e o resultado da previsão do modelo meteorológico ETA em sua execução. Em execução, o resultado do modelo também pode ser calculado com a chuva observada para avaliar o impacto do erro da previsão meteorológica no desempenho do modelo.

O ajuste dos parâmetros foi realizado por um algoritmo genético [161][169], cuja formulação é praticamente independente da estrutura do modelo. Dessa forma, foi possível testar diversas opções para o número de estados e para o número de elementos nas partições de entrada e saída sem grandes alterações na formulação do problema de ajuste de parâmetros. A solução adotada foi a que obteve os melhores resultados.

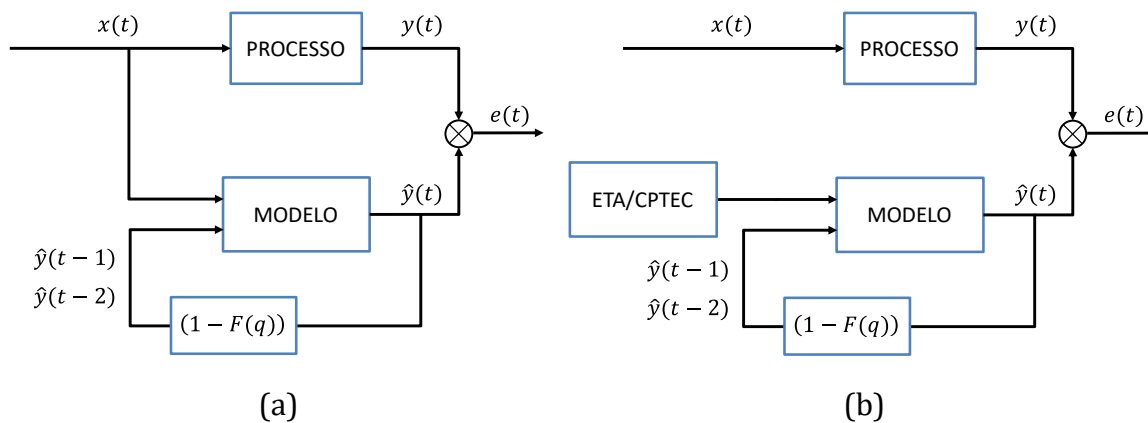


Figura 8.42: Variáveis de entrada do modelo: (a) treinamento; (b) execução.

O algoritmo genético (AG) é um tipo de algoritmo de otimização simples e eficiente, inspirado na evolução natural. No processo de solução do problema de otimização por um AG simples, a cada iteração, um conjunto de candidatos a solução, chamados indivíduos, são gerados pela recombinação dos melhores indivíduos da iteração anterior. Numa aplicação específica, cada indivíduo é codificado para representar as variáveis do problema e poder ser manipulado pelos operadores genéticos de cruzamento e mutação [205]. A avaliação dos indivíduos é feita pela função de avaliação do problema de otimização. O processo do algoritmo é baseado na evolução natural, em que, a cada iteração (chamada de geração), são encontrados indivíduos mais aptos, isto é, que representam melhores candidatos a solução do problema. Há diversas variantes do AG simples e esse é um tema de pesquisa bastante ativo, sendo comumente abordado em livros de inteligência computacional [318][520]. A referência clássica de AGs é o livro de Goldberg [205], mas outras obras também são referências no tema [184][380].

Os resultados de RMS e MAPE do modelo *fuzzy* recorrente para cada dia de previsão são mostrados na Figura 8.43, onde a curva azul representa o modelo executado com a chuva observada e a curva vermelha, o modelo executado com a chuva prevista pelo modelo meteorológico ETA. Os resultados mostram que utilizando os valores observados de chuva, o erro do modelo se estabiliza após o sexto dia de previsão. Quando o modelo utiliza a chuva prevista como variável de entrada, o erro sempre aumenta, tendo em vista que os erros anteriores são reinjetados no modelo devido a sua estrutura recorrente. Entretanto, o desempenho do modelo de previsão de vazões com a informação do modelo meteorológico ainda é melhor que sem a sua utilização [169].

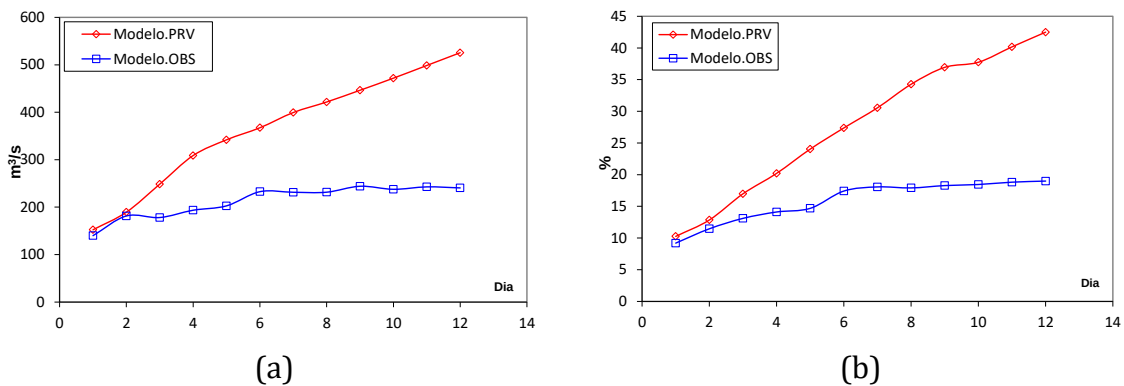


Figura 8.43: Resultados de previsão diária: (a) RMS; (b) MAPE. Fonte: [169].

O hidrograma do 12º dia de previsão é mostrado na Figura 8.44, onde os resultados do modelo com a chuva observada e prevista pelo modelo ETA são comparados aos valores observados de vazão. Esse gráfico mostra o desempenho do modelo que, mesmo executado com a chuva prevista, é capaz de indicar aproximadamente o comportamento da vazão.

O modelo *fuzzy* recorrente foi utilizado nesse projeto para ajustar quatro modelos de vazão natural total e quatro modelos de vazão natural incremental. As previsões de vazões semanais foram calculadas pelas médias das previsões de vazões diárias. Os resultados do modelo com chuva prevista e observada foram avaliados com quatro estatísticas de validação [222].

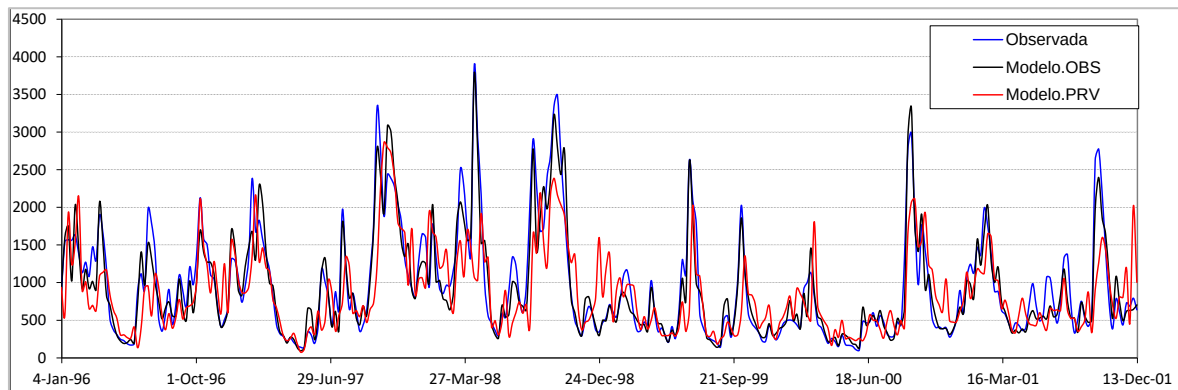


Figura 8.44: Resultados de previsão para o 12º dia de previsão. Fonte: [169].

O desempenho do modelo *fuzzy* recorrente obteve melhor resultado comparado a outros modelos da mesma bacia, na avaliação do ONS [222] e também superou um modelo de rede neural ajustado com o *software* Statistica [169]. O modelo foi implementado na linguagem PL/SQL do banco de dados Oracle e integrado à infraestrutura da

ONS, que o utiliza desde 2006 para a programação semanal da bacia do rio Iguaçu. Em outro estudo, o modelo foi executado com duas entradas separadas, correspondentes a duas sub-bacias, e o resultado final calculado como a média dos dois modelos. Essa estratégia obteve resultados ligeiramente melhores [171].

O modelo *fuzzy* recorrente tem sido estudado com diferentes estruturas de retroalimentação, com o ajuste de parâmetros realizado por uma variante do AG chamada de algoritmo de evolução diferencial [470][471]. Os modelos ajustados em séries *benchmark* obtiveram um bom desempenho tanto em termos de precisão quanto em relação ao custo computacional, mostrando que o algoritmo da evolução diferencial é mais eficiente que o AG.

8.5 Comentários Finais

Este capítulo apresentou uma introdução à teoria de conjuntos *fuzzy* evidenciando suas características para a representação de conceitos linguísticos. Há outras áreas que utilizam conjuntos *fuzzy* em aplicações como apoio a decisão, otimização, análise de incertezas etc. [310][432]. A apresentação neste capítulo foi direcionada para os sistemas *fuzzy* e sua utilização em aplicações de ciência de dados.

Assim como os capítulos anteriores, a seleção do material foi feita visando um balanceamento entre a abrangência e o detalhamento de cada tópico para que a apresentação fosse a mais completa possível e com um nível de detalhe suficiente para o entendimento. A apresentação de conjuntos *fuzzy* neste capítulo enfatiza sua aplicação para a representação de conceitos, na qual o conjunto *fuzzy* $\mathcal{A}$ é diferente do símbolo A . Os sistemas *fuzzy* simbólicos nessa formulação e sua representação vetorial permitem uma apresentação elegante dos algoritmos, podendo ser empregado quando a variável de entrada tem valor numérico ou nominal.

Os principais algoritmos para identificação de sistemas *fuzzy* em aplicações de ciência de dados foram apresentados na seção 8.3, há várias outras abordagens descritas na literatura [279].

Sistemas *fuzzy* não são encontrados com frequência em *softwares* de ciência de dados, principalmente os comerciais, uma vez que requer conhecimento teórico do usuário, o

que dificulta sua utilização. Entretanto, a capacidade de sistemas *fuzzy* de integrar facilmente com conceitos linguísticos torna uma opção em problemas que requerem a interpretação dos resultados e utilização de conhecimento de especialistas.

9 REDES NEURAIS

A capacidade de processamento do cérebro humano e seu desempenho na solução de problemas complexos inspira os cientistas há muito tempo. No século XX, o primeiro modelo matemático do comportamento de um neurônio natural foi proposto por McCulloch e Pitts em 1943 [374]. Em 1949, o psicólogo Hebb apresenta um estudo sobre como a adaptação dos neurônios contribui para a percepção, memória e o aprendizado [245]. O trabalho de Hebb sugere que a conexão entre neurônios é fortalecida pela estimulação repetida e persistente das células, fornecendo uma base biológica para novos paradigmas de aprendizado de redes neurais artificiais.

O modelo McCulloch-Pitts e a teoria do aprendizado de Hebb foram reunidos em 1957 no primeiro modelo de rede neural artificial, chamado de Perceptron. Desenvolvido em *hardware* para reconhecimento de padrões [457], o Perceptron introduz um algoritmo de ajuste de parâmetros baseado na teoria de Hebb. O algoritmo do Perceptron foi aprimorado por Widrow e Hoff em 1961 [561] no modelo ADALINE,* em que o ajuste dos parâmetros é realizado em função do erro de saída através da solução de mínimos quadrados iterativa (LMS).† O algoritmo LMS foi muito utilizado no projeto de filtros lineares e como solução iterativa de sistemas lineares numa época em que os computadores dispunham de recursos limitados de memória e processamento.

Minsky e Papert demonstram que o Perceptron só poderia ajustar funções lineares [389]. O livro, de 1969, teve um impacto negativo no desenvolvimento de redes neurais

* *Adaptive Linear Neuron.*

† *Least Mean Squares.*

durante toda a década de 1970. Em 1986, no auge do desenvolvimento de sistemas especialistas, Rumelhart e coautores mostram a capacidade de redes neurais para representação de conhecimento e o ajuste de parâmetros pela retropropagação (*back-propagation*) do erro [464]. O artigo, publicado na revista *Nature*, se tornou muito conhecido e amplamente citado na literatura.¹⁴⁴ Entretanto, Werbos havia proposto uma técnica similar no contexto de regressão não linear em 1974 [557] e LeCun também havia desenvolvido uma versão do algoritmo em 1985 [330].

Durante a década de 1980, diferentes modelos de redes neurais e novos paradigmas de aprendizado foram desenvolvidos. No modelo de memória associativa de Hopfield [258], a rede neural é um sistema dinâmico que oscila e se estabiliza em atratores, que são associados aos padrões armazenados. Em 1982 também aparecem os mapas autoajustáveis de Kohonen [313][314], que inspiraram o desenvolvimento de uma série de modelos de redes neurais. O aprendizado por reforço também foi utilizado em redes neurais, principalmente em aplicações de robótica [511][512]. Muitos desses modelos foram diretamente inspirados pelas teorias de Hebb [250].

Os problemas de convergência do algoritmo do gradiente e a falta de uma teoria para a escolha da melhor topologia de redes neurais estimularam o desenvolvimento de métodos locais, como os modelos RBF e SVM [66][400]. De 1995 até o início dos anos 2000, houve grande desenvolvimento dos métodos baseados em funções de núcleo, tendo em vista suas características teóricas e a forma convexa do problema de ajuste de parâmetros, como foi discutido no Capítulo 7.

A partir de 2006, a disponibilidade de grandes bases de dados e o aumento da capacidade de processamento dos computadores permitiram o desenvolvimento de modelos de redes neurais cada vez mais complexos, com milhares de unidades de processamento e milhões de parâmetros. Esses modelos são chamados de **redes neurais profundas** ou modelos de **aprendizado profundo** (*deep learning*) [39][212][333].

Modelos de redes neurais profundas têm sido desenvolvidos desde a década de 1980 [464], mas sem muita repercussão na literatura. Em 1998, LeCun e colaboradores [331] apresentaram um sistema de reconhecimento de dígitos manuscritos baseado no modelo de rede neural convolucional chamado LeNet-5. O artigo apresenta uma comparação do desempenho de diversos modelos no conjunto de dados de dígitos manuscritos

MNIST. O conjunto de dados MNIST e os resultados de diversos modelos estão disponíveis na página do autor.¹⁴⁵

Modelos de redes neurais muito complexos são difíceis de treinar. Nos modelos em camadas, o erro retropropagado é diluído a cada camada, de forma que os parâmetros das camadas iniciais da rede quase não são ajustados durante o treinamento. Esse resultado negativo era pouco documentado numa época em que redes mais simples e modelos SVM funcionavam bem na maioria dos problemas e não havia grandes bases de dados anotadas para o ajuste de modelos mais complexos.

Em 2006, Hinton e Salakhutdinov [251] introduzem a ideia de pré-treinamento não supervisionado para ajuste de parâmetros de redes neurais profundas. A partir daí, o desenvolvimento de redes neurais profundas se expande rapidamente com a disponibilidade de grandes bases de dados anotadas [465], algoritmos escaláveis [38] e a implementação de bibliotecas que exploram os recursos de processamento massivamente paralelo de unidades de processamento gráfico (GPU).

A partir de 2012, modelos de redes neurais profundas têm obtido os melhores resultados em diferentes competições de problemas relacionados com habilidades humanas como visão computacional [324], reconhecimento de fala [249] e processamento de textos em linguagem natural [332].

As oportunidades de negócio dessa nova tecnologia na economia digital atraem grandes investimentos para financiar seu desenvolvimento em inúmeras aplicações. A tecnologia vem se difundindo rapidamente devido à disponibilidade bases de dados abertas, repositórios como arXiv¹⁴⁶ e GitHub¹⁴⁷ e à utilização de *software* livre.

O nome “rede neural” é atribuído a diferentes tipos de modelos e a bibliografia sobre o tema é abrangente [240][250][453], com bons livros em português [62][241]. Este capítulo apresenta os principais modelos de redes neurais em camadas. Os conceitos básicos são introduzidos para o modelo linear e, em seguida, utilizados nas topologias mais importantes: o modelo de múltiplas camadas (MLP), os modelos de redes neurais convolucionais (CNN), os modelos de redes neurais recorrentes (RNN) os autocodificadores (AE) e modelos geradores (GAN).

O modelo MLP* é o mais difundido em *softwares* de ciência de dados e linguagens de programação. O modelo MLP com uma camada escondida é robusto em relação à seleção da estrutura e obtém boa precisão na maioria das aplicações, o que permitiu sua utilização em diversas áreas.

O modelo CNN† é uma rede neural em camadas que aproveita as características de relações de vizinhança em que os pesos de conexões representam filtros convolucionais. No modelo CNN os parâmetros são compartilhados em matrizes esparsas, que permite o processamento em alto desempenho. Os modelos CNN foram desenvolvidos para problemas de visão computacional, mas também têm aplicações em outras áreas, como processamento de linguagem natural.

Os modelos RNN‡ são modelos em camadas que têm conexões recorrentes, e cuja topologia resulta em diferentes representações de modelos dinâmicos sendo muito utilizados no processamento de linguagem natural. Esses modelos, quando “desdobrados”, podem ser processados como uma rede neural em camadas. Apesar do número reduzido de parâmetros, o grande número de camadas no treinamento de sequências muito longas dificulta o ajuste de parâmetros.

Os modelos AE§ são modelos de aprendizado não supervisionado cujo objetivo é uma representação interna do conjunto de dados. Redes neurais AE fazem parte de uma categoria ampla de modelos, chamados de aprendizado da representação, cujo objetivo principal é realizar uma representação do espaço de variáveis de entrada. O aprendizado da representação inclui uma nova abordagem de modelos geradores chamados redes geradoras adversárias (GAN),** que são ajustadas para gerar dados sintéticos.

A próxima seção apresenta os modelos lineares de redes neurais artificiais numa introdução dos conceitos básicos. A seção 9.2 introduz o modelo de redes neurais MLP e o algoritmo da retropropagação. A seção 9.3 discute os modelos mais avançados de redes neurais profundas e suas aplicações em visão computacional e processamento de linguagem natural. O capítulo termina com algumas aplicações realizadas em projetos de pesquisa da Coppe/UFRJ na seção 9.4.

* *MultiLayer Perceptron.*

† *Convolutional Neural Network.*

‡ *Recurrent Neural Network.*

§ *AutoEncoder.*

** *Generative Adversarial Network*

9.1 Redes Neurais Lineares

A unidade básica de processamento em diversos tipos de redes neurais é o modelo proposto por McCulloch e Pitts em 1943 [374]. O neurônio artificial de McCulloch-Pitts reproduz simplificada e o comportamento do neurônio natural. A Figura 9.1 mostra uma representação atualizada do modelo McCulloch-Pitts.

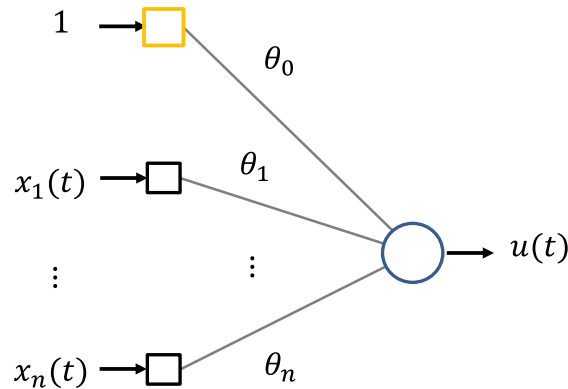


Figura 9.1: O neurônio artificial de McCulloch-Pitts.

No modelo McCulloch-Pitts utilizado em redes neurais, o processamento de cada unidade (neurônio artificial) é calculado como:

$$u(t) = h(z(t)) = h\left(\theta_0 + \sum_{i=1}^n x_i(t)\theta_i\right) \quad (9.1)$$

onde $u(t)$ é a saída, chamada de **valor de ativação**; $z(t)$ é o resultado do processamento linear, chamado de **potencial de ativação**; e h é a **função de ativação**.

As variáveis $x_i(t)$, $i = 1, \dots, n$ representam as entradas do neurônio. A primeira entrada, chamada de unidade *bias*, tem o valor de ativação constante e igual a 1 e o parâmetro θ_0 serve para ajustar o limiar da função de ativação. Os demais **parâmetros** θ_i , $i = 1, \dots, n$ representam a intensidade das conexões. Na literatura, utiliza-se frequentemente a notação do vetor de parâmetros como $\boldsymbol{\theta} = (\theta_0, \dots, \theta_n) = [b, \mathbf{w}]$, onde b é o parâmetro da unidade bias e $\mathbf{w}$ é o vetor de conexões. Essa notação é útil para a formulação do algoritmo da retropropagação e será empregada quando necessário.

No modelo McCulloch-Pitts definido pela equação (9.1), o potencial de ativação $z(t) = \theta_0 + \sum_{i=1}^n x_i(t)\theta_i$ é uma função linear das entradas. O processamento não linear é realizado pela função de ativação. As funções de ativação mais utilizadas são apresentadas na Figura 9.2.

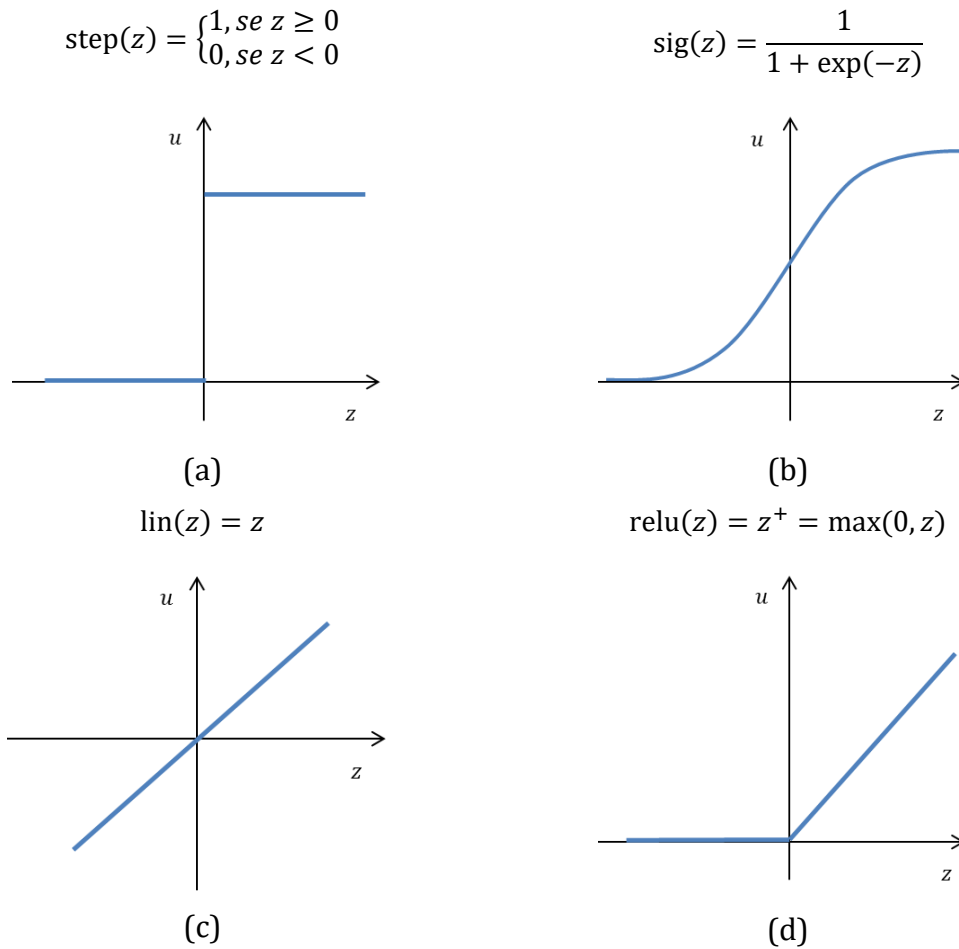


Figura 9.2: Exemplos de função de ativação: (a) função lógica; (b) função logit ou sigmoide; (c) função linear; (d) função linear retificada (ReLU).

O modelo McCulloch-Pitts original empregava a função lógica (Figura 9.2a), cujo processamento basicamente bloqueia ou deixa passar o sinal. Na função lógica, o limiar de ativação é o valor do potencial em que a saída do modelo é positiva deixando passar o sinal. Na equação (9.1), o limiar de ativação é ajustado pelo parâmetro bias θ_0 , de forma que as funções de ativação aparecem centradas na origem na Figura 9.2.

Os modelos de redes neurais no século XX utilizavam a função sigmoide (Figura 9.2b) e sua variante tangente hiperbólica, $\tanh(z) = 2\text{sig}(2z) - 1$. Essas funções de ativação são versões contínuas, com derivadas contínuas, da função lógica, sendo mais adequadas para o ajuste de parâmetros pelo método do gradiente. A função linear (Figura 9.2c) é a mais simples em que o valor de ativação é igual ao potencial de ativação. Recentemente, a função linear retificada (Figura 9.2d), proposta em 2000 [226], tem sido muito utilizada, principalmente em modelos de redes neurais profundas [333].

As redes neurais são formadas pela conexão de unidades McCulloch-Pitts em diferentes topologias que resultam em modelos com diferentes características. Esta seção introduz os conceitos básicos de redes neurais em camada simples, que implementam funções discriminantes lineares. O método do gradiente é introduzido nesta seção para o ajuste de modelos lineares. Os modelos de redes neurais não lineares e o ajuste de parâmetros pelo algoritmo da retropropagação são apresentados na seção 9.2.

9.1.1 Perceptron e ADALINE

O Perceptron foi o primeiro modelo de rede neural artificial com uma regra de ajuste de parâmetros baseada na teoria do aprendizado de Hebb [245]. Na regra de aprendizado do Perceptron, as conexões são ajustadas somente quando o resultado do processamento é errado. O objetivo de Rosenblatt era uma representação do aprendizado no cérebro nas funções de percepção, por isso o nome Perceptron [460].

O Perceptron original é um modelo McCulloch-Pitts com função de ativação lógica desenvolvido em *hardware*. A função do modelo é escrita como:

$$\hat{v}(t) = h(z(t)) = \begin{cases} 1, & \text{se } z(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T \geq 0 \\ 0, & \text{se } z(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T < 0 \end{cases} \quad (9.2)$$

onde $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ e $\boldsymbol{\theta} = (\theta_0, \dots, \theta_p)$. O parâmetro de unidade bias θ_0 realiza o ajuste do limiar de ativação.

O funcionamento do *Perceptron* é ilustrado para o processamento das funções lógicas AND e OR, apresentadas na Tabela 9.1.

Tabela 9.1: Valores das funções AND e OR

AND		OR	
$(x_1(t), x_2(t))$	$v(t)$	$(x_1(t), x_2(t))$	$v(t)$
(0,0)	0	(0,0)	0
(0,1)	0	(0,1)	1
(1,0)	0	(1,0)	1
(1,1)	1	(1,1)	1

O resultado é ilustrado na Figura 9.3, onde a região sombreada tem valor $\hat{v}(t) = 1$. O Perceptron produz uma função discriminante linear com $\boldsymbol{\theta} = (-0.5, 1.0, 1.0)$ no caso da

função OR e $\theta = (-1.5, 1.0, 1.0)$ para a função AND. O potencial de ativação $z(t)$ é o plano que separa as duas classes e define a superfície de decisão pela reta $z(t) = 0$, mostrada nos dois casos da Figura 9.3.

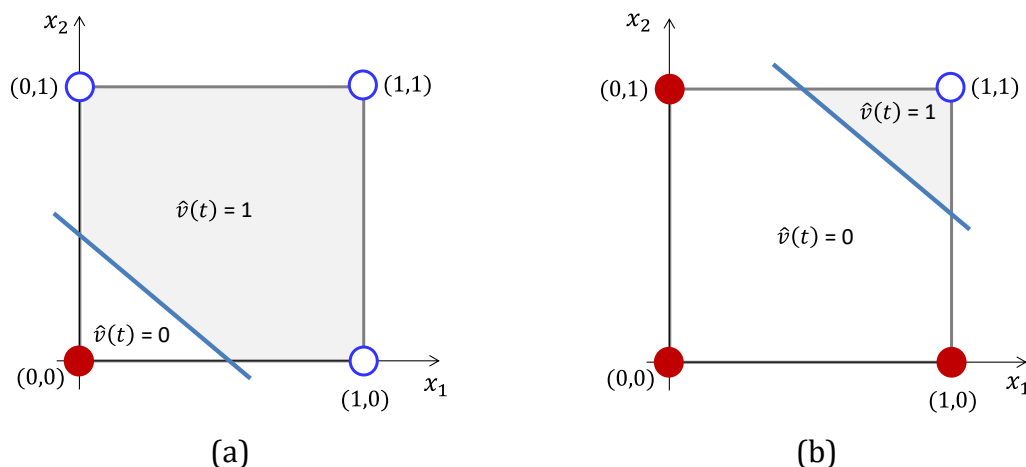


Figura 9.3: O modelo Perceptron: (a) função OR; (b) função AND.

O ajuste de parâmetros do Perceptron é realizado pelo Algoritmo 9.1, chamado de regra do Perceptron. Para cada registro do conjunto de treinamento, o vetor de parâmetros θ utilizado para o cálculo da estimativa do modelo $\hat{v}(t)$ é atualizado para o próximo registro se o resultado do modelo for incorreto. O parâmetro η define a “velocidade” do ajuste e é chamado de **taxa de aprendizagem**.

A regra do Perceptron aplica o conceito de aprendizado hebbiano. Se o padrão é classificado corretamente, nenhum ajuste é realizado. Se houver erro, o peso das conexões é enfraquecido, se o valor correto $v(t) = 0$, ou fortalecido, o valor correto $v(t) = 1$. Em ambos os casos, o ajuste é proporcional à entrada correspondente de acordo com a taxa de aprendizagem. O treinamento do Perceptron é realizado pela repetição da regra de ajuste sobre o conjunto de treinamento até a convergência

Rosenblatt demonstrou a convergência da regra do Perceptron em problemas linearmente separáveis [457]. A demonstração pode ser encontrada em livros clássicos de redes neurais [240]. O Perceptron provocou grande repercussão na época por utilizar uma teoria aprendizado para o ajuste de parâmetros de um neurônio artificial.

Como psicólogo, o interesse de Rosenblatt era a reprodução das funções cognitivas do cérebro. Widrow e Hoff estudaram a capacidade de ajustar funções lógicas do Percep-

tron e criaram o algoritmo LMS [560]. O algoritmo foi utilizado num modelo de reconhecimento de padrões desenvolvido em *hardware* chamado ADALINE. O algoritmo LMS é muito utilizado para a síntese de filtros adaptativos lineares.¹⁴⁸

Algoritmo 9.1: Regra do Perceptron

Entrada:	Conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), v(t)), t = 1 \dots N\}$ e taxa de aprendizagem η
Saída:	Vetor de parâmetros $\boldsymbol{\theta} = (\theta_0, \dots, \theta_p)$
01	Início
02	Inicialização $\boldsymbol{\theta} = \mathbf{0}$; $\varepsilon \leftarrow \infty$
03	Enquanto $\varepsilon > tol$
04	$\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$
05	Para $t = 1, \dots, N$
06	Calcular $\hat{v}(t)$ pela equação (9.2)
07	Se $\hat{v}(t) \neq v(t)$:
08	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \hat{\mathbf{x}}(t)$, se $\hat{v}(t) = 1$ ($v(t) = 0$)
09	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \hat{\mathbf{x}}(t)$, se $\hat{v}(t) = 0$ ($v(t) = 1$)
10	Fim Se
11	Fim Para
12	Calcule $\varepsilon = \ \boldsymbol{\theta}^* - \boldsymbol{\theta}\ $
13	Fim Enquanto
14	Retorna o vetor de parâmetros $\boldsymbol{\theta}$
15	Fim

O algoritmo LMS é um aprimoramento da regra do Perceptron e pode ser entendido de forma intuitiva. Em vez de realizar o ajuste de parâmetros só quando o resultado é incorreto, o ajuste é feito para todos os registros, utilizando o erro de processamento para regular a intensidade do ajuste:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta e(t) \hat{\mathbf{x}}(t) \quad (9.3)$$

onde $e(t) = (v(t) - \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T)$ é o erro do modelo; $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ e $\boldsymbol{\theta} = (\theta_0, \dots, \theta_p)$.

O algoritmo LMS é consistente com a teoria do aprendizado hebbiano, uma vez que o sinal do erro determina se cada conexão será fortalecida ou enfraquecida. Pela regra de ajuste (9.3), se o erro é nulo, não há ajuste; se o erro é negativo, as conexões são

enfraquecidas, e se o erro é positivo, as conexões são fortalecidas. O ajuste de parâmetros proporcional ao erro resulta numa convergência mais suave que a regra do Perceptron, em que o ajuste é sempre proporcional à entrada, independentemente do erro.

O algoritmo LMS permite uma expressão simples e altamente eficiente para o ajuste de parâmetros de modelos lineares. Widrow e Hoff mostraram que a sequência gerada pela regra (9.3) converge para a solução de mínimos quadrados num número finito de iterações. O algoritmo LMS é resumido no Algoritmo 9.2.

Algoritmo 9.2: Algoritmo LMS para classificação binária

Entrada:	Conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), v(t)), t = 1 \dots N\}$ e taxa de aprendizagem η
Saída:	Vetor de parâmetros $\boldsymbol{\theta} = (\theta_0, \dots, \theta_p)$
01	Início
02	Inicialização $\boldsymbol{\theta} = \mathbf{0}$; $\varepsilon \leftarrow \infty$
03	Enquanto $\varepsilon > tol$
04	$\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$
05	Para $t = 1, \dots, N$
06	Calcular o erro $e(t) = (v(t) - \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T)$
07	Realizar o ajuste $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta e(t)\hat{\mathbf{x}}()$
08	Fim Para
09	Calcule $\varepsilon = \ \boldsymbol{\theta}^* - \boldsymbol{\theta}\ $
10	Fim Enquanto
11	Retorna o vetor de parâmetros $\boldsymbol{\theta}$
12	Fim

Os modelos Perceptron e ADALINE foram propostos para o problema de classificação binária, em que a decisão é feita pela função lógica. A extensão desses modelos para m classes é direta pela adição de unidades McCulloch-Pitts. Nesse caso, o modelo é uma rede neural de duas camadas, como mostrado na Figura 9.4. As unidades de entrada não realizam processamento, apenas distribuem os valores das variáveis de entrada (previamente padronizadas) entre as unidades de processamento.

A saída de uma rede neural com uma única camada de processamento, como mostrado na Figura 9.4, é calculada vetorialmente como:

$$\hat{\mathbf{v}}(t) = \mathbf{h}(\mathbf{z}(t)) = \mathbf{h}(\mathbf{b} + \mathbf{x}(t)\mathbf{W}) \quad (9.4)$$

onde $\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$ é o vetor de entradas, $\hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$ é o vetor de saída. O potencial de ativação $\mathbf{z}(t) = \hat{\mathbf{x}}(t)\mathbf{\Theta}$, onde $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ e a matriz de parâmetros é formada pelo vetor de parâmetros da unidade bias $\mathbf{b}$ e a matriz de parâmetros de conexões como $\mathbf{\Theta} = [\mathbf{b}, \mathbf{W}^T]^T$.

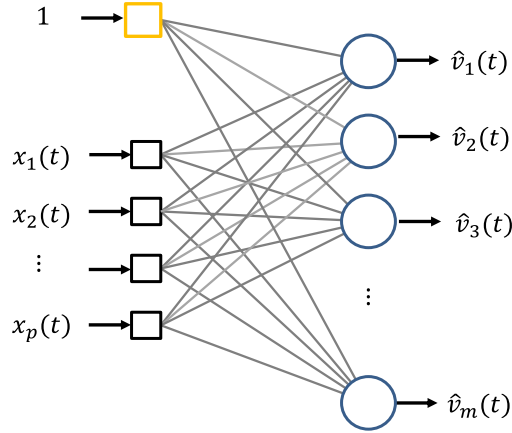


Figura 9.4: Rede neural de duas camadas, uma camada de entrada e uma de processamento.

A próxima seção introduz o método do gradiente para ajuste de parâmetros de modelos lineares de regressão e classificação linear. O algoritmo LMS realiza uma aproximação instantânea do método do gradiente aplicado ao EMQ. Esse algoritmo é generalizado pelo algoritmo do gradiente estocástico, discutido na seção 9.1.3.

9.1.2 Método do gradiente

Na seção 7.1, o ajuste de parâmetros de modelos de aprendizado de máquinas foi apresentado como a solução de um problema de otimização definido como:

$$\begin{aligned} \min_{\boldsymbol{\theta}} l(\boldsymbol{\theta}) \\ \text{sujeito a } \mathbf{r}(\boldsymbol{\theta}) \leq \mathbf{0} \end{aligned} \quad (9.5)$$

onde o vetor $\boldsymbol{\theta} \in \mathcal{R}^n$ são os **parâmetros do modelo**; $l: \mathcal{R}^n \rightarrow \mathcal{R}$ é a **função de avaliação**; e a função vetorial $\mathbf{r}(\boldsymbol{\theta})$ define o conjunto de **restrições** [98].

A solução do problema de otimização sem restrições é um ponto em que o gradiente da função de avaliação se anula. O gradiente da função de avaliação é o vetor cujos elementos são as derivadas da função de avaliação em relação aos parâmetros:

$$\nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta}) = \left[\frac{\partial l(\boldsymbol{\theta})}{\partial \theta_1}, \dots, \frac{\partial l(\boldsymbol{\theta})}{\partial \theta_n} \right] \quad (9.6)$$

onde $\nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta})$ é o gradiente de $l(\boldsymbol{\theta})$ em relação a $\boldsymbol{\theta}$.

O método do gradiente é um método clássico para solução de problemas de otimização [496] e já era estudado por Cauchy em 1847 [76]. O método também é conhecido como descida do gradiente, pois realiza uma busca pelo mínimo da função de custo iterativamente, seguindo na direção de maior declive, que é definida pelo negativo do vetor gradiente. O algoritmo básico é simples: até a convergência, a cada iteração, uma nova estimativa da solução é gerada a partir da estimativa anterior e do gradiente da função de custo no ponto atual.

No caso do problema de ajuste de parâmetros (9.5), a atualização do método do gradiente é escrita como:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta}) \quad (9.7)$$

O negativo do gradiente da função de avaliação é chamado de **direção de busca**, pois define uma direção no espaço de parâmetros. A taxa de aprendizagem η define o quanto se deve avançar na direção de busca para o cálculo da nova estimativa do vetor de parâmetros.

O método do gradiente é geralmente utilizado na solução de problemas de otimização não lineares, nos quais a região viável pode não ser convexa. Nesta seção, o método do gradiente é aplicado ao ajuste de parâmetros de modelos de redes neurais lineares como uma introdução ao método. O ajuste de parâmetros de redes neurais não lineares é realizado pelo algoritmo da retropropagação, apresentado na seção 9.2.3.

9.1.2.1 Regressão linear

O modelo de regressão linear é equivalente a uma rede neural linear de uma unidade com função de ativação linear. A função de custo para o ajuste de parâmetros é o erro quadrático (EQ) no conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$:

$$l(\boldsymbol{\theta}) = \frac{1}{2} \sum_{t=1}^N e(t)^2 = \frac{1}{2} \sum_{t=1}^N (y(t) - \hat{y}(t))^2 \quad (9.8)$$

Onde $e(t) = (y(t) - \hat{y}(t))$ é o **resíduo do modelo**; $\hat{y}(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ e $\boldsymbol{\theta} = (\theta_0, \dots, \theta_p)$.

O fator $\frac{1}{2}$ é incluído para simplificar o cálculo da derivada e não altera a solução do problema, de forma que o mínimo da função (9.8) é também o mínimo do EMQ. A rigor, a função de avaliação (9.8) é também função dos dados do conjunto de treinamento e deveria ser escrita como $l(\boldsymbol{\theta}, \mathbf{X}, \mathbf{Y})$. A notação simplificada $l(\boldsymbol{\theta})$ é utilizada nesta seção porque apenas os parâmetros do modelo variam durante o treinamento.

Cada componente do gradiente $\nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta})$ é calculado pela regra da cadeia como:

$$\frac{\partial l(\boldsymbol{\theta})}{\partial \theta_i} = \frac{\partial l(\boldsymbol{\theta})}{\partial \hat{y}(t)} \frac{\partial \hat{y}(t)}{\partial \theta_i} = -e(t) \hat{x}_i(t), \quad i = 0, \dots, p \quad (9.9)$$

Substituindo o resultado de (9.9) na fórmula de atualização de parâmetros (9.7), o ajuste dos parâmetros do modelo de regressão linear é escrito como:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \left(\sum_{t=1}^N e(t) \hat{\mathbf{x}}(t) \right) \quad (9.10)$$

Cada iteração do algoritmo do gradiente corresponde a uma passagem completa pelo conjunto de treinamento, uma vez que a função de custo (9.8) considera o erro em todo o conjunto. No algoritmo LMS, a atualização de parâmetros é similar, mas considera apenas um registro do conjunto de treinamento (cf. Algoritmo 9.2). Essa abordagem é generalizada no método do gradiente estocástico, apresentado na seção 9.1.3.

A função de ativação linear na saída permite que o modelo produza saídas contínuas e até fora do intervalo de valores da variável de saída no conjunto de treinamento. Em problemas de classificação, a função de ativação sigmoide é utilizada no modelo de regressão logística, descrito a seguir.

9.1.2.2 Regressão logística

No modelo linear de classificação binária, a classe é definida pela variável indicadora:

$$v(t) = \begin{cases} 1, & \text{se } \mathbf{x}(t) \in C \\ 0, & \text{se } \mathbf{x}(t) \notin C \end{cases} \quad (9.11)$$

Um modelo de regressão logística para classificação binária é equivalente a uma rede neural linear de uma unidade com função de ativação sigmoide. A função do modelo é escrita como:

$$\hat{v}(t) = h(z(t)) = \frac{1}{1 + \exp(-z(t))} \quad (9.12)$$

onde $z(t) = \hat{\mathbf{x}}(t)\boldsymbol{\theta}^T$, $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ e $\boldsymbol{\theta} = (\theta_0, \dots, \theta_p)$.

A função de avaliação para ajuste de parâmetros é o negativo do logaritmo da função de verossimilhança (cf. equação (5.28)), calculada no conjunto de dados de treinamento $\mathcal{T} = \{(\mathbf{x}(t), v(t)), t = 1 \dots N\}$ como:

$$l(\boldsymbol{\theta}) = - \sum_{t=1}^N v(t) \log(\hat{v}(t)) + (1 - v(t)) \log(1 - \hat{v}(t)) \quad (9.13)$$

Cada componente do gradiente $\nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta})$ é calculado pela regra da cadeia:

$$\frac{\partial l(\boldsymbol{\theta})}{\partial \theta_i} = \frac{\partial l(\boldsymbol{\theta})}{\partial \hat{v}(t)} \frac{\partial \hat{v}(t)}{\partial z(t)} \frac{\partial z(t)}{\partial \theta_i}, \quad i = 0, \dots, p \quad (9.14)$$

O potencial de ativação é uma função linear dos parâmetros e $\partial z(t)/\partial \theta_i = \hat{x}_i(t)$. O termo $\partial \hat{v}(t)/\partial z(t)$ é a derivada da função de ativação $h'(z(t))$. No caso da função sigmoide, $h'(z(t)) = \hat{v}(t)(1 - \hat{v}(t))$. Dessa forma, cada componente do gradiente $\nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta})$ é calculado como:

$$\frac{\partial l(\boldsymbol{\theta})}{\partial \theta_i} = - \sum_{t=1}^N \left(v(t) \frac{1}{\hat{v}(t)} - (1 - v(t)) \frac{1}{1 - \hat{v}(t)} \right) \hat{v}(t)(1 - \hat{v}(t)) \hat{x}_i(t) \quad (9.15)$$

Realizando as simplificações possíveis na equação (9.15), cada componente do gradiente $\nabla_{\boldsymbol{\theta}} l(\boldsymbol{\theta})$ é calculado como:

$$\frac{\partial l(\boldsymbol{\theta})}{\partial \theta_i} = -e(t) \hat{x}_i(t), \quad (9.16)$$

onde $e(t) = (v(t) - \hat{v}(t))$ é o resíduo do modelo de classificação.

A atualização dos parâmetros da regressão logística pelo método do gradiente tem a mesma formulação da regressão linear (cf. equação (9.10)):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \left(\sum_{t=1}^N e(t) \hat{\mathbf{x}}(t) \right) \quad (9.17)$$

O método do gradiente resulta na mesma formulação para atualização dos parâmetros de modelos de redes neurais lineares em problemas de classificação e de regressão. A

analogia entre os dois tipos de modelo e as respectivas funções de avaliação é discutida na seção 9.2.2. O ajuste de parâmetros do modelo de múltiplas classes é apresentado no caso mais geral de redes não lineares na seção 9.2.3.

A atualização de parâmetros pelo método do gradiente é feita uma vez para todo o conjunto de treinamento e todos os valores do resíduo $e(t)$, $t = 1, \dots, N$, são obtidos com o mesmo conjunto de parâmetros. Essa abordagem é conhecida como treinamento em *batch* na terminologia de redes neurais [241]. A atualização de parâmetros também pode ser realizada a cada registro do conjunto de treinamento, como descrito a seguir.

9.1.3 Gradiente estocástico

No método do **gradiente estocástico** (SGD),* o gradiente é calculado em função de apenas um registro, como no algoritmo LMS (cf. Algoritmo 9.2). Essa abordagem é também chamada de treinamento *online* na terminologia de redes neurais [241]. A equação de ajuste de modelos lineares é escrita como:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta e(t) \hat{\mathbf{x}}(t) \quad (9.18)$$

onde $e(t) = (y(t) - \hat{y}(t))$ para a regressão linear ou $e(t) = (v(t) - \hat{v}(t))$ para a regressão logística.

A equação (9.18) demonstra que o algoritmo LMS é equivalente ao gradiente estocástico no caso de modelos de redes neurais lineares, tanto para regressão linear quanto para regressão logística.

O gradiente estocástico realiza uma busca errática a partir da informação local da função de avaliação. Para cada registro, uma direção de busca diferente é calculada e uma atualização de parâmetros é realizada (cf. Algoritmo 9.2). O método do gradiente define uma direção de busca mais precisa, pois é calculada em cada iteração sobre todo o conjunto de treinamento.

O resultado de aplicação dos dois métodos num exemplo de regressão linear simples é apresentado na Figura 9.5. O mesmo conjunto de dados com $N = 20$ registros foi empregado para ajustar os parâmetros pelos dois métodos, utilizando a mesma inicialização e dois valores diferentes de taxa de aprendizagem η .

* *Stochastic Gradient Descent.*

A Figura 9.5 mostra a trajetória de atualização dos parâmetros dos dois métodos sobre as curvas de nível da função de avaliação no espaço dos parâmetros do modelo. Observa-se que o método do gradiente apresenta uma trajetória mais suave no espaço de parâmetros, que é semelhante nos dois valores de η . No caso do gradiente estocástico, o parâmetro $\eta = 0.2$ resulta numa trajetória mais errática, principalmente próximo ao mínimo, em que o módulo do gradiente é menor. A solução obtida com o gradiente estocástico com $\eta = 0.2$ é ligeiramente diferente da solução obtida pelo método do gradiente. O exemplo com $\eta = 0.02$, a trajetória do vetor de parâmetros produzida pelo gradiente estocástico é menos errática e o resultado se aproxima do resultado do método do gradiente. Em problemas mais complexos, é conveniente reduzir a taxa de aprendizado, o que gera uma convergência mais lenta, mas com resultado mais preciso.

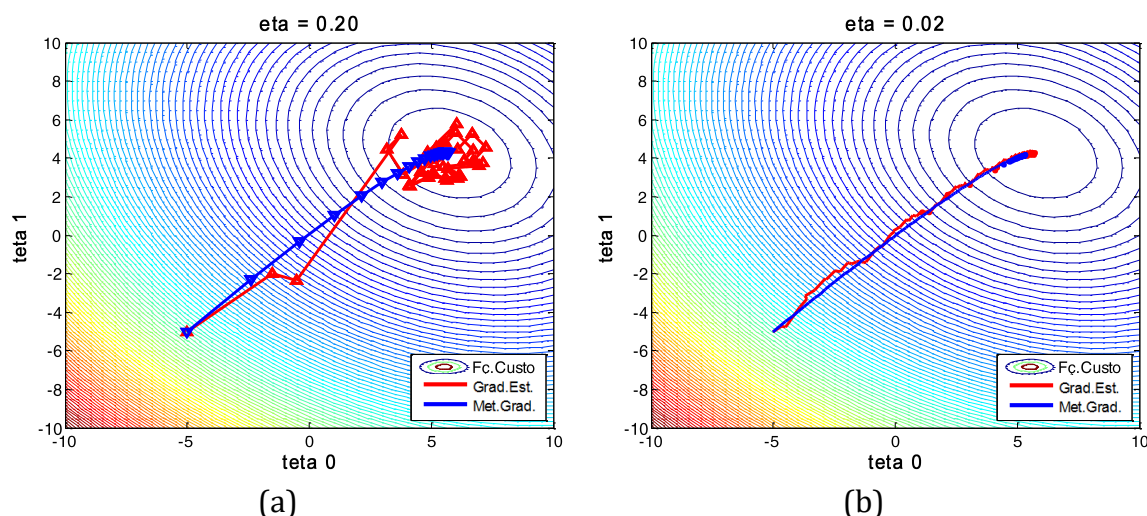


Figura 9.5: Resultados do método do gradiente e gradiente estocástico com diferentes taxas de aprendizagem: (a) $\eta = 0.2$; (b) $\eta = 0.02$.

Com taxa de aprendizagem $\eta = 0.2$, o método do gradiente converge em 24 iterações com $\varepsilon \leq 0.01$ como critério de convergência (cf. Algoritmo 9.2). O gradiente estocástico converge em 60 iterações com mesmo critério de convergência. Entretanto, cada iteração do método do gradiente exige o cálculo da saída do modelo para todos os registros do conjunto de treinamento, o que é chamado de uma **época**. As 60 iterações do gradiente estocástico correspondem a apenas três passagens pelo conjunto de treinamento (épocas). Com $\eta = 0.02$, o método do gradiente converge em 140 épocas e o gradiente estocástico, em 300 iterações, que correspondem a 15 épocas.

Globalmente, o método do gradiente estocástico é mais eficiente, gerando menos avaliações do modelo. O gradiente estocástico também pode ser utilizado para o treinamento de modelos adaptativos em *streaming* de dados.

Na prática, o ajuste de parâmetros de redes neurais é realizado pela estratégia *mini-batch*, em que, a cada iteração, um subconjunto de dados de treinamento é utilizado para cálculo do gradiente e atualização de parâmetros [340]. O método do gradiente com *mini-batch* representa um meio-termo entre a eficiência computacional do SGD e a melhor convergência do método do gradiente.

O método do gradiente é utilizado para o ajuste de parâmetros de modelos de redes neurais de múltiplas camadas. As duas estratégias de treinamento já discutidas podem ser aplicadas a redes de múltiplas camadas, apresentadas a seguir.

9.2 Redes Neurais Não lineares

Modelos não lineares são obtidos com redes neurais de múltiplas camadas, em que uma ou mais camadas intermediárias são incluídas entre a camada de entrada e a de saída, e funções de ativação não lineares. O modelo é processado em camadas, pois cada unidade de uma camada é conectada a todas as unidades da camada seguinte e não há conexões entre unidades da mesma camada. Diferentes modelos são obtidos de acordo com a topologia da rede, ou seja, o número de camadas e o número de unidades em cada camada [62][240][241].

A próxima seção apresenta o processamento do modelo de redes neurais de múltiplas camadas. A seção 9.2.2 apresenta a formulação geral para a função de avaliação pelo princípio de máxima verossimilhança. A seção 9.2.3 apresenta o cálculo dos gradientes pelo algoritmo da retropropagação do erro.

9.2.1 Perceptron de Múltiplas Camadas (MLP)

O modelo de rede neural Perceptron de múltiplas camadas (MLP)* é encontrado na maioria dos *softwares* e bibliotecas de ciência de dados. Na rede MLP, o modelo McCulloch-Pitts é utilizado nas unidades das camadas intermediárias e de saída. O processamento é feito por camadas e a saída de uma camada é a entrada da seguinte. O modelo

* *MultiLayer Perceptron*.

MLP também é chamado de totalmente conectado, pois todas as unidades de uma camada são conectadas com todas as unidades da camada seguinte.

A Figura 9.6 mostra uma rede MLP com uma camada intermediária. A camada de entrada não realiza processamento, como no caso da rede neural linear (cf. Figura 9.4). A camada intermediária é a primeira camada de processamento, cujo resultado é o vetor de ativação $\mathbf{u}(t) = (u_1(t), \dots, u_n(t))$. Em problemas de classificação, a saída do modelo é o vetor com as estimativas das variáveis indicadoras $\hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$, como mostrado na Figura 9.6. Em problemas de regressão, o resultado do modelo é o vetor de estimativas das variáveis de saída $\hat{\mathbf{y}}(t) = (\hat{y}_1(t), \dots, \hat{y}_q(t))$.

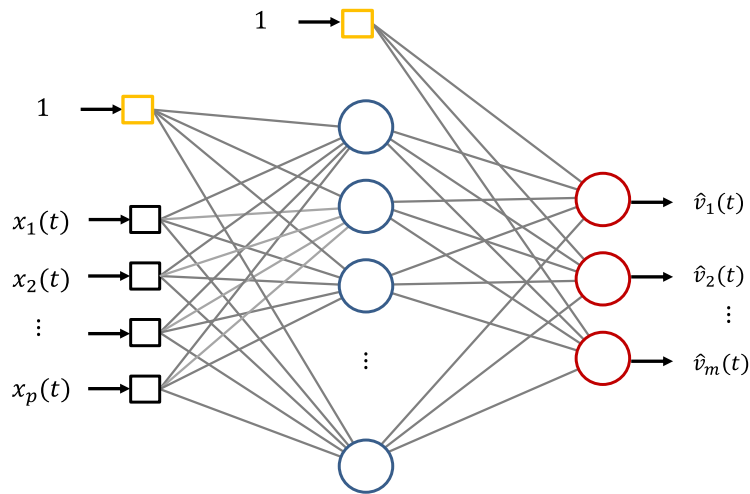


Figura 9.6: Rede neural MLP com uma camada intermediária.

A topologia da rede neural é definida pelo número de camadas e o número de unidades em cada camada [238]. O número de unidades de entrada é definido pelo número de variáveis de entrada e o número de unidades de saída é definido pelo número de classes ou de variáveis de saída. O modelo mais simples tem apenas uma camada intermediária, também chamada de camada escondida, e sua topologia fica definida pelo número de unidades dessa camada. Redes MLP com várias camadas intermediárias são chamadas de **redes neurais profundas** (*deep neural networks*), sendo a “profundidade” da rede o número de camadas de processamento entre a entrada e a saída.

A rede MLP de topologia $p:n:m$ é um modelo de uma camada intermediária com p entradas, n unidades intermediárias e m unidades na camada de saída. O modelo tem apenas duas camadas de processamento, uma vez que a camada de entrada não realiza

processamento. As equações do modelo são escritas pela aplicação sucessiva do modelo de rede neural de uma camada (cf. equação (9.4)):

$$\begin{aligned}\mathbf{u}(t) &= \mathbf{h}^{(1)}(\mathbf{z}^{(1)}(t)) = \mathbf{h}^{(1)}(\mathbf{b}^{(1)} + \mathbf{x}(t)\mathbf{W}^{(1)}) \\ \hat{\mathbf{v}}(t) &= \mathbf{h}^{(2)}(\mathbf{z}^{(2)}(t)) = \mathbf{h}^{(2)}(\mathbf{b}^{(2)} + \mathbf{u}(t)\mathbf{W}^{(2)})\end{aligned}\tag{9.19}$$

onde $\mathbf{x}(t) = (x_1(t), \dots, x_p(t))$ é o vetor de entradas, $\mathbf{u}(t) = (u_1(t), \dots, u_n(t))$ é o vetor de ativação das unidades da camada intermediária e $\hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$ é o vetor de saída. O processamento é similar no caso de modelo de regressão, em que a saída do modelo é o vetor $\hat{\mathbf{y}}(t)$.

Neste capítulo, o índice sobrescrito denota o número da camada, para diferenciar de subscritos, que denotam os elementos de um vetor ou matriz. Dessa forma, nas equações (9.19), as funções $\mathbf{h}^{(1)}$ e $\mathbf{h}^{(2)}$ são as funções vetoriais de ativação da primeira e da segunda camada respectivamente. Na função vetorial de ativação, a mesma função de ativação é aplicada a todos os elementos da entrada, de forma que o resultado é um vetor de mesma dimensão. Os vetores $\mathbf{b}^{(1)}$ e $\mathbf{b}^{(2)}$ representam os parâmetros conectados às unidades bias e $\mathbf{W}^{(1)}$ $\mathbf{W}^{(2)}$ representam as matrizes de conexões, da primeira e da segunda camadas respectivamente. Os parâmetros da camada intermediária e da camada de saída podem ser representados pelas matrizes:

$$\boldsymbol{\Theta}^{(1)} = \begin{bmatrix} \mathbf{b}^{(1)} \\ \mathbf{W}^{(1)} \end{bmatrix}, \boldsymbol{\Theta}^{(2)} = \begin{bmatrix} \mathbf{b}^{(2)} \\ \mathbf{W}^{(2)} \end{bmatrix}\tag{9.20}$$

As redes MLP com funções de ativação lineares nas camadas intermediárias e de saída só podem representar funções lineares, o que pode ser verificado ao se substituir o cálculo de $\mathbf{u}(t)$ na expressão de $\hat{\mathbf{v}}(t)$ nas equações (9.19), considerando funções de ativação lineares. Modelos não lineares são obtidos pela escolha de funções de ativação não lineares. As funções sigmoide (Figura 9.2b) e sua variante, a função tangente hiperbólica, são muito utilizadas em redes MLP com uma camada intermediária e redes recorrentes [240]. A função de ativação linear retificada (Figura 9.2d) foi proposta em 2000 [226], com motivações matemáticas e biológicas. Esse tipo de função de ativação tem obtido resultados melhores e tem sido mais utilizada em modelos de redes neurais profundas. A função linear retificada é mais eficiente computacionalmente e tem derivadas constantes, o que reduz o efeito de diluição do gradiente.

O problema XOR, introduzido no Capítulo 7 (cf. Figura 7.3), pode ser utilizado para ilustrar o processamento de uma rede MLP de topologia 2: 2: 1 com função de ativação lógica nas camadas intermediária e de saída. Nesse caso, os parâmetros do modelo são:

$$\Theta^{(1)} = \begin{bmatrix} \mathbf{b}^{(1)} \\ \mathbf{W}^{(1)} \end{bmatrix} = \begin{bmatrix} -1.5 & -0.5 \\ 1.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}, \Theta^{(2)} = \begin{bmatrix} \mathbf{b}^{(2)} \\ \mathbf{W}^{(2)} \end{bmatrix} = \begin{bmatrix} -0.5 \\ -1.0 \\ 1.0 \end{bmatrix} \quad (9.21).$$

O resultado do processamento em cada camada é apresentado na Tabela 9.2 e ilustrado na Figura 9.7. O potencial de ativação de cada unidade da primeira camada define um plano, cuja interseção com o plano das variáveis, ou seja, $z_1^{(1)} = 0$ e $z_2^{(1)} = 0$, define cada uma das retas mostradas na Figura 9.7a. Os valores de ativação de cada unidade da primeira camada são mostrados Figura 9.7a nas regiões correspondentes.

Tabela 9.2: Valores da rede MLP do modelo XOR

(x_1, x_2)	(u_1, u_2)	$\hat{v}$
(0,0)	(0,0)	0
(0,1)	(0,1)	1
(1,0)	(0,1)	1
(1,1)	(1,1)	0

A primeira camada realiza um mapeamento não linear das variáveis de entrada no espaço formado pelos valores de ativação da primeira camada, onde o problema se torna linearmente separável. As entradas da camada de saída são os valores de ativação da primeira camada, como mostra a Figura 9.7b. O potencial de ativação da segunda camada é o plano $z_1^{(2)}$, cuja interseção com o plano dos valores de ativação da primeira camada ($z_1^{(2)} = 0$) define a reta que separa as classes, mostrada na Figura 9.7b.

O exemplo do problema XOR é muito ilustrativo do *modus operandi* da rede MLP. O número de unidades da camada intermediária determina o número de hiperplanos utilizados pelo modelo. Quanto maior o número de unidades da camada intermediária, maior o número de hiperplanos gerados e, conseqüentemente, maior a capacidade de representação do modelo para ajustar funções complexas.

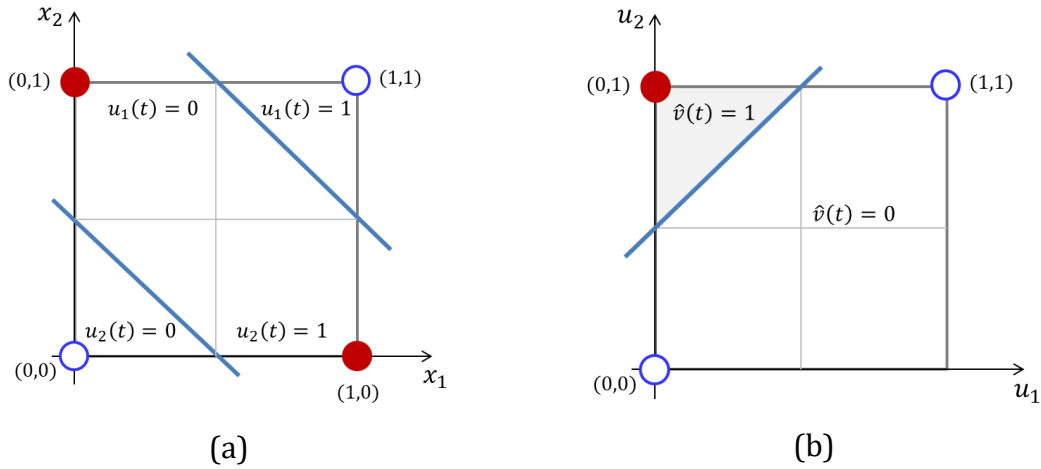


Figura 9.7: Modelo MLP da função XOR: (a) primeira camada; (b) camada de saída.

Uma rede neural MLP com uma camada intermediária com um número finito de unidades pode ajustar qualquer função contínua com a precisão desejada. Esse resultado foi demonstrado em 1987 pela aplicação da solução de Kolmogorov para o 13º problema de Hilbert no contexto de redes neurais [246], que ficou conhecido como o teorema da aproximação universal [119][260]. Em 2007, Bengio e LeCun discutiram as limitações de modelos locais (RBF e SVM) e redes neurais MLP com apenas uma camada intermediária na solução de problemas complexos [38]. Os autores mostram que redes neurais profundas, com várias camadas de processamento, obtêm resultados melhores em problemas complexos.

O modelo de rede neural MLP pode ser estendido para L camadas e o processamento de cada camada para cada registro pode ser escrito como:

$$\mathbf{u}^{(i)} = \mathbf{h}^{(i)}(\mathbf{z}^{(i)}) = \mathbf{h}^{(i)}(\mathbf{b}^{(i)} + \mathbf{u}^{(i-1)}\mathbf{W}^{(i)}), i = 0, \dots, L \quad (9.22)$$

Os índices do registro foram removidos da equação (9.22) para aliviar a notação. O processamento por registro (*online*) será representado em letra minúscula, sendo $\mathbf{u}^{(i-1)}$ a saída da camada $i - 1$, que também é a entrada da camada i ; $\mathbf{b}^{(i)}$ e $\mathbf{W}^{(i)}$ os parâmetros da unidade bias e da matriz de conexões; $\mathbf{h}^{(i)}$ a função vetorial de ativação; e $\mathbf{u}^{(i)}$ a saída da camada i . Considerando a camada i com $n^{(i)}$ unidades e a camada $i - 1$ com $n^{(i-1)}$ unidades, a matriz de conexão $\mathbf{W}^{(i)}$ tem dimensão $n^{(i-1)} \times n^{(i)}$ e o vetor $\mathbf{b}^{(i)}$ tem dimensão $n^{(i)}$, assim como os vetores $\mathbf{z}^{(i)}$ e $\mathbf{u}^{(i)}$. A entrada $\mathbf{x} = \mathbf{u}^{(0)}$ e a saída $\hat{\mathbf{v}} = \mathbf{u}^{(L)}$ no modelo com L camadas de processamento. A saída estimada $\hat{\mathbf{y}}(t) = \mathbf{u}^{(L)}$ no caso de um modelo de regressão.

A equação (9.22) representa o processamento de apenas um registro (*online*). O processamento de múltiplos registros (*batch* ou *mini-batch*) pode ser representado vetorialmente como:

$$\mathbf{U}^{(i)} = \mathbf{H}^{(i)}(\mathbf{Z}^{(i)}) = \mathbf{H}^{(i)}(\mathbf{1}^T \mathbf{b}^{(i)} + \mathbf{U}^{(i-1)} \mathbf{W}^{(i)}), i = 0, \dots, L \quad (9.23)$$

onde $\mathbf{U}^{(i-1)}$ é a matriz de dimensão $N \times n^{(i-1)}$ com os valores de ativação da camada $i - 1$ para todos os registros do conjunto de treinamento (ou subconjunto no caso de processamento *mini-batch*); $\mathbf{Z}^{(i)}$ e $\mathbf{U}^{(i)}$ têm dimensão $N \times n^{(i)}$ e o vetor $\mathbf{1}^T$ é o vetor unitário de dimensão N , que representa as unidades bias. $\mathbf{H}^{(i)}$ é a versão matricial da função de ativação, cujo resultado é a função de ativação aplicada a cada elemento de sua entrada.

A representação vetorial das equações (9.22) e (9.23) encapsulam o processamento da camada em uma única equação. Um modelo de rede neural complexo pode ser obtido pela conexão de diversas camadas, como mostra a Figura 9.8a para o caso do processamento *online* e a Figura 9.8b para o caso do processamento em *batch* (ou *mini-batch*).

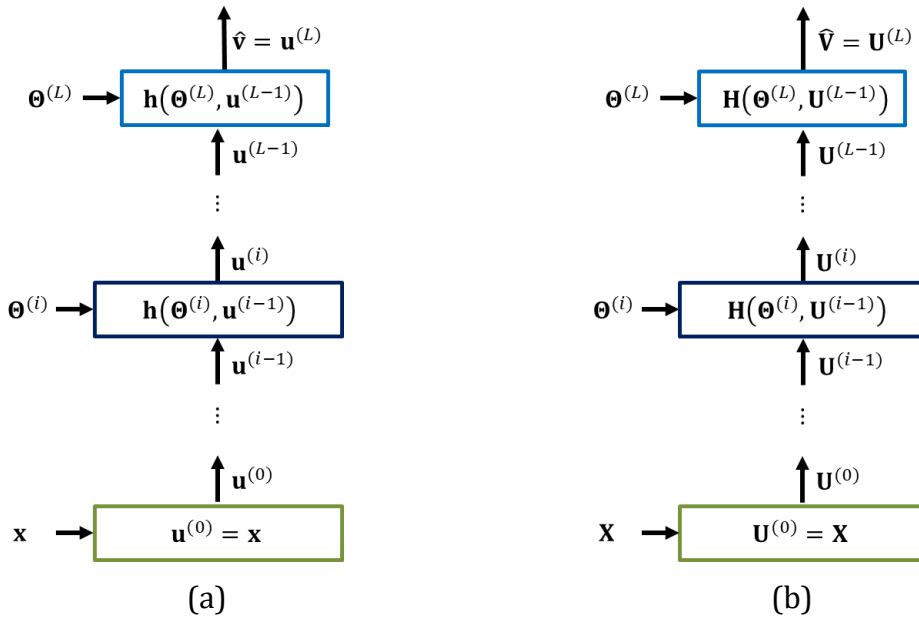


Figura 9.8: Modelo MLP: (a) processamento *online*; (b) *batch* ou *mini-batch*.

O processamento vetorial de todos os registros em *batch* (ou *mini-batch*) pela equação (9.23) utiliza os mesmos parâmetros $\mathbf{b}^{(i)}$ e $\mathbf{W}^{(i)}$ do processamento de um registro (*online*) e pode ser programado dessa forma em linguagens de programação que suportam o processamento vetorial, simplificando a implementação do modelo.

O processamento vetorial pode ser realizado em paralelo em milhares de núcleos das modernas placas de processamento gráfico (GPU). O processamento massivamente paralelo de redes neurais em GPUs tem permitido o desenvolvimento de modelos capazes de ajustar funções altamente complexas em aplicações de visão computacional e processamento de linguagem natural. Esses modelos serão discutidos na seção 9.3.

O número de unidades das camadas de entrada e de saída é determinado pelo número de variáveis de entrada e o número de variáveis de saída (em problemas de regressão) ou de classes (em problemas de classificação). Um modelo de rede neural é definido pelos chamados hiperparâmetros, que determinam a topologia da rede, pelo número de camadas, o número de unidades e a função de ativação de cada camada, além de parâmetros relacionados ao algoritmo de ajuste como a taxa de aprendizagem.

A próxima seção apresenta a formulação das funções de avaliação para o ajuste de parâmetros em problemas de regressão e classificação a partir do princípio de máxima verossimilhança. A seção 9.2.3 apresenta o cálculo do gradiente da função de avaliação pelo algoritmo da retropropagação do erro para o ajuste de parâmetros de modelos de redes neurais de múltiplas camadas.

9.2.2 Avaliação pelo máximo da verossimilhança

Considere um conjunto de dados $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$ gerado por uma distribuição de probabilidades $p(\mathbf{x})$. O modelo paramétrico $\hat{p}(\mathbf{x}, \boldsymbol{\theta})$ que melhor representa o conjunto de dados é o que maximiza a probabilidade (ou verossimilhança) da amostra. O conjunto de parâmetros, chamado de **estimador de verossimilhança**, é calculado pela maximização da probabilidade do modelo nos registros do conjunto de treinamento:

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \prod_{t=1}^N \hat{p}(\mathbf{x}(t), \boldsymbol{\theta}) \quad (9.24)$$

O produto da equação (9.24) é inconveniente para utilização em problemas de otimização, de forma que a função de avaliação mais utilizada é o logaritmo da função de verossimilhança:

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} l(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \sum_{t=1}^N \log(\hat{p}(\mathbf{x}(t), \boldsymbol{\theta})) \quad (9.25)$$

Um exemplo simples é o estimador de verossimilhança de uma amostra de uma variável gerada por uma distribuição normal, ou seja, $p(x) = N(\mu, \sigma^2)$. O logaritmo da função de verossimilhança é calculado como (cf. equação (2.3)):

$$\sum_{t=1}^N \log(\hat{p}(x(t), \boldsymbol{\theta})) = -\frac{1}{2\hat{\sigma}^2} \sum_{t=1}^N (x(t) - \hat{\mu})^2 - \frac{N}{2} \log(\hat{\sigma}^2) - \frac{N}{2} \log(2\pi) \quad (9.26)$$

Calculando as derivadas da equação (9.26) em relação aos parâmetros e igualando a zero, o estimador de verossimilhança $\boldsymbol{\theta}^* = (\hat{\mu}, \hat{\sigma})$ é calculado como:

$$\hat{\mu} = \frac{1}{N} \sum_{t=1}^N x(t) \quad (9.27)$$

$$\hat{\sigma}^2 = \frac{1}{N} \sum_{t=1}^N (x(t) - \hat{\mu})^2 \quad (9.28)$$

Observe que a média estimada corresponde ao valor real, ou seja, $\hat{\mu} = \mu$, mas a variância é subestimada em relação ao valor real, uma vez que $\sigma^2 = \frac{N}{N-1} \hat{\sigma}^2$, que é a chamada correção de Bessel [52].

O estimador de verossimilhança não é enviesado quando $N \rightarrow \infty$ e o logaritmo da função de verossimilhança se aproxima da esperança matemática da probabilidade do modelo com dados gerados pela distribuição de dados real [212].

O estimador de verossimilhança é utilizado em inúmeras aplicações, com diferentes distribuições. Um exemplo interessante é a predição do número de gols de uma partida de futebol, estimado por uma distribuição de Poisson ou binomial negativa [361][135].

Em modelos preditivos, os dados da amostra são gerados por uma distribuição de probabilidades condicional $p(\mathbf{y}|\mathbf{x})$ em problemas de regressão ou $p(\mathbf{v}|\mathbf{x})$ em problemas de classificação. A função de avaliação em problemas de regressão ou classificação pode ser definida a partir da função de verossimilhança, considerando hipóteses sobre a distribuição da amostra, como descrito a seguir.

9.2.2.1 Erro quadrático

No problema de regressão linear introduzido na seção 3.1.2, o conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$ é gerado por uma distribuição desconhecida $p(y|\mathbf{x})$. Em geral, há erros de medida e variáveis não observadas, de forma que o mesmo valor da

variável de entrada pode aparecer na amostra com diferentes valores de saída, como diferentes realizações de uma variável aleatória. No modelo de regressão, os valores observados da variável de saída correspondentes à mesma entrada $\mathbf{x}(t) = \mathbf{x}_0$ são modelados por uma distribuição normal como:

$$\hat{p}(y(t)|\mathbf{x}_0, \boldsymbol{\theta}) = N(\hat{y}(t), \hat{\sigma}^2) \quad (9.29)$$

onde $\hat{y}(t) = \hat{f}(\mathbf{x}(t), \boldsymbol{\theta})$ é saída estimada pelo modelo e $\hat{\sigma}^2$, a variância do resíduo.

No modelo definido pela equação (9.29), a saída estimada é a média dos valores observados para a mesma entrada. O ajuste de parâmetros é realizado pela minimização do negativo* do logaritmo da função de verossimilhança do modelo:

$$-\sum_{t=1}^N \log(\hat{p}(\mathbf{x}(t), \boldsymbol{\theta})) = -\frac{1}{2\hat{\sigma}^2} \sum_{t=1}^N (y(t) - \hat{y}(t))^2 - \frac{N}{2} \log(\hat{\sigma}^2) - \frac{N}{2} \log(2\pi) \quad (9.30)$$

Supondo que a variância do resíduo $\hat{\sigma}^2$ é constante, a função de avaliação para o problema de regressão é o erro quadrático (EQ):

$$l(\boldsymbol{\theta}) = \frac{1}{2} \sum_{t=1}^N (y(t) - \hat{y}(t))^2 \quad (9.31)$$

Os parâmetros que minimizam do EQ da equação (9.31) correspondem ao estimador de máxima verossimilhança quando o resíduo do modelo $e(t) = (y(t) - \hat{y}(t))$ tem distribuição normal com média nula e variância constante. Na prática, mesmo em casos em que a variância do resíduo não é constante, a minimização do EQ obtém bons resultados, desde que a distribuição do resíduo seja simétrica.

Em modelos de redes neurais, a função de ativação da camada de saída deve ser compatível com a função de avaliação e o tipo de problema. Em problemas de regressão, utiliza-se a função de ativação linear e a camada de saída é calculada como:

$$\hat{\mathbf{y}} = \mathbf{u}^{(L)} = \mathbf{b}^{(L)} + \mathbf{u}^{(L-1)} \mathbf{W}^{(L)} \quad (9.32)$$

onde $\hat{\mathbf{y}} = \hat{\mathbf{y}}(t) = (\hat{y}_1(t), \dots, \hat{y}_q(t))$ são as saídas calculadas pelo modelo. Os índices dos registros foram removidos para simplificar a notação.

* O negativo do logaritmo da função de verossimilhança é utilizado pois o problema de ajuste de parâmetros, em geral, é formulado como um problema de minimização (cf. (9.5)).

A função de avaliação para o ajuste de redes neurais de múltiplas camadas em problemas de regressão é o EQ, calculado como:

$$l(\Theta) = \frac{1}{2} \sum_{t=1}^N \|\mathbf{y}(t) - \hat{\mathbf{y}}(t)\|^2 \quad (9.33)$$

onde $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}$ é o conjunto de todos os parâmetros do modelo e $\|\cdot\|^2$ é a norma L^2 ao quadrado.

9.2.2.2 Entropia cruzada

No modelo de regressão logística, introduzido na seção 5.2.3, o conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), v(t)), t = 1 \dots N\}$, onde $v(t)$ é a variável indicadora binária. O modelo é equivalente a uma rede neural linear com função de ativação sigmoide (cf. equação (9.12)). A função de probabilidades do modelo é calculada como uma distribuição de Bernoulli (cf. equação (5.25)):

$$\hat{p}(v(t) = 1 | \mathbf{x}(t), \Theta) = \hat{v}(t)^{v(t)} (1 - \hat{v}(t))^{1-v(t)} \quad (9.34)$$

O negativo do logaritmo da função de verossimilhança é calculado como:

$$l(\Theta) = - \sum_{t=1}^N v(t) \log(\hat{v}(t)) + (1 - v(t)) \log(1 - \hat{v}(t)) \quad (9.35)$$

Um modelo não linear de classificação binária pode ser representado por uma rede MLP com uma única unidade de saída com função de ativação sigmoide. Nesse caso, a função de avaliação (9.35) também é utilizada para o ajuste de parâmetros.

No problema de classificação com múltiplas classes, o conjunto de treinamento é representado por $\mathcal{T} = \{(\mathbf{x}(t), y(t)), t = 1 \dots N\}$, onde a saída $y(t) \in \{1, \dots, m\}$ é o índice das classes $\{C_1, \dots, C_m\}$. As classes são geralmente codificadas pelo vetor de variáveis indicadoras $\mathbf{v}(t)$, em que cada elemento é uma variável binária que indica a presença da classe correspondente como:

$$v_i(t) = \begin{cases} 1, & \text{se } y(t) = i \\ 0, & \text{se } y(t) \neq i \end{cases} \quad i = 1, \dots, m \quad (9.36)$$

A extensão do modelo de regressão logística para múltiplas classes é obtida por uma transformação exponencial normalizada da função logística, de forma que o resultado

possa ser interpretado como uma distribuição de probabilidades [64]. A função de probabilidades do modelo é calculada como uma distribuição de Bernoulli generalizada (ou multinoulli) como [52][212]:

$$\hat{p}(v_i(t) = 1 | \mathbf{x}(t), \boldsymbol{\theta}_i) = \hat{v}_i(t) = \frac{\exp(\hat{\mathbf{x}}(t) \boldsymbol{\theta}_i^T)}{\sum_{i=1}^m \exp(\hat{\mathbf{x}}(t) \boldsymbol{\theta}_i^T)} \quad i = 1, \dots, m \quad (9.37)$$

onde $\hat{\mathbf{x}}(t) = [1, \mathbf{x}(t)]$ e $\boldsymbol{\theta}_i = [b, \mathbf{w}_i]$ são os parâmetros do modelo.

Na equação (9.37), o resultado do modelo $\hat{v}_i(t) = \hat{p}(C_i | \mathbf{x}(t), \boldsymbol{\theta}_i)$ é a probabilidade de observação da classe C_i a partir da entrada e de $\mathbf{x}(t)$ os parâmetros $\boldsymbol{\theta}_i$. A saída do modelo para cada classe é normalizada, de forma que $\sum_{i=1}^m \hat{v}_i(t) = 1$ e o vetor de saída $\hat{\mathbf{v}}(t)$ representa a distribuição de probabilidades das classes calculada pelo modelo.

O modelo definido na equação (9.37) foi introduzido por Bridle em 1990 [64] e foi chamado de função **softmax**, pois define uma função de máximo suavizada. O resultado $\hat{v}_i(t) \gg \hat{v}_j(t)$ implica que $\hat{p}(C_i | \mathbf{x}(t), \boldsymbol{\theta}_i) \approx 1$ e $\hat{p}(C_j | \mathbf{x}(t), \boldsymbol{\theta}_i) \approx 0, \forall C_j \neq C_i$. O modelo de regressão logística para múltiplas classes também é chamado de **regressão softmax**.

Em modelos de redes neurais, a função *softmax* é utilizada como função de ativação da camada de saída em problemas de classificação com múltiplas classes:

$$\hat{\mathbf{v}} = \mathbf{u}^{(L)} = h(\mathbf{z}^{(L)}) = \text{softmax}(\mathbf{b}^{(L)} + \mathbf{u}^{(L-1)} \mathbf{W}^{(L)}) \quad (9.38)$$

onde $\hat{\mathbf{v}} = \hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$ são as saídas calculadas pelo modelo e a função *softmax* é calculada como na equação (9.37) com $\hat{\mathbf{x}} = [1, \mathbf{u}^{(L-1)}]$.

A camada de saída com m tem parâmetros redundantes, uma vez que função *softmax* é normalizada pela soma e o valor de saída de uma classe pode ser calculado em função das outras $m - 1$ classes como $\hat{v}_m(t) = 1 - \sum_{i=1}^{m-1} \hat{v}_i(t)$. Na prática, geralmente o modelo tem m unidades de saída e aplica-se algum tipo de regularização [212].

A função de avaliação utilizada para o ajuste de modelos de redes neurais em problemas de classificação é função de **entropia cruzada**:

$$l(\boldsymbol{\Theta}) = - \sum_{t=1}^N \sum_{i=1}^m v_i(t) \log(\hat{v}_i(t)) = - \sum_{t=1}^N \mathbf{v}(t) \mathbf{log}(\hat{\mathbf{v}}(t))^T \quad (9.39)$$

onde $\hat{\mathbf{v}}(t) = (\hat{v}_1(t), \dots, \hat{v}_m(t))$ é o vetor de saídas estimadas e $\boldsymbol{\Theta} = \{\boldsymbol{\Theta}^{(1)}, \dots, \boldsymbol{\Theta}^{(L)}\}$ é o conjunto de todos os parâmetros do modelo.

O mínimo da função de avaliação definida na equação (9.39) é também o mínimo do negativo do logaritmo da função de verossimilhança da função *softmax* da equação (9.38), uma vez que os valores observados $\mathbf{v}(t)$ não dependem do conjunto de parâmetros Θ . É fácil observar que a função de avaliação (9.39) se reduz à função de avaliação da regressão logística (9.35) para o caso de $m = 2$ classes.

A minimização da entropia cruzada (9.39) corresponde à minimização da dissimilaridade de Kullback-Leibler (KL), que mede a diferença entre duas distribuições de probabilidades. No caso do problema de classificação de múltiplas classes, a dissimilaridade KL é escrita, para cada registro, como:

$$D_{KL}(p||\hat{p}) = - \sum_{i=1}^m v_i(t) \log \left(\frac{\hat{v}_i(t)}{v_i(t)} \right) \quad (9.40)$$

onde $p = \{v_i(t), i = 1, \dots, m\}$ é a distribuição de probabilidades observada e a distribuição de probabilidades do modelo $\hat{p} = \{\hat{v}_i(t), i = 1, \dots, m\}$.

Entropia e entropia cruzada são conceitos da Teoria da Informação [506]. A entropia cruzada $S(p, \hat{p})$ se relaciona com a entropia $S(p)$ e a dissimilaridade KL como:

$$S(p, \hat{p}) = S(p) + D_{KL}(p||\hat{p}) \quad (9.41)$$

onde $S(p, \hat{p}) = - \sum_{i=1}^m v_i(t) \log(\hat{v}_i(t))$ e $S(p) = - \sum_{i=1}^m v_i(t) \log(v_i(t))$.

A função de avaliação da equação (9.39) é a soma da entropia cruzada para todos os registros, uma vez que, para cada registro, $S(p, \hat{p}) = -\mathbf{v}(t)\mathbf{log}(\hat{\mathbf{v}}(t))^T$. Dessa forma, a minimização da entropia cruzada corresponde também à minimização da dissimilaridade KL, uma vez que a entropia da amostra não depende dos parâmetros do modelo.

Há outras funções de avaliação para problemas de classificação,¹⁴⁹ estudadas sobretudo no contexto da teoria de aprendizado estatístico, mas também utilizadas em modelos de redes neurais [459]. A maioria dos modelos atuais utiliza as formulações apresentadas nesta seção para os problemas de regressão e classificação. A próxima seção apresenta o método de cálculo do gradiente da função de avaliação pelo algoritmo da retropropagação.

9.2.3 Algoritmo da retropropagação

O ajuste de parâmetros de redes neurais MLP é feito pelo método do gradiente e, a cada iteração, os parâmetros são ajustados pelo gradiente da função de avaliação:

$$\Delta \Theta = -\eta \nabla_{\Theta} l(\Theta) \quad (9.42)$$

onde $\Delta \Theta$ é o ajuste a ser aplicado em cada iteração e $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}$ é o conjunto de parâmetros de todas as camadas.

O algoritmo da retropropagação é, basicamente, a aplicação da regra da cadeia para o cálculo do gradiente, para cada camada, no sentido contrário ao cálculo da saída, ou seja, da camada de saída para camada de entrada. O erro da camada de saída é retropropagado pelas camadas da rede para o ajuste de parâmetros, como descrito a seguir.

9.2.3.1 Formulação básica

A versão vetorial da regra da cadeia é escrita como:

$$\nabla_{\Theta} l(\Theta) = \nabla_{\mathbf{u}^{(L)}} l(\Theta) \left[\frac{\partial \mathbf{u}^{(L)}}{\partial \Theta} \right] \quad (9.43)$$

onde $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}$ é o conjunto de parâmetros de todas as camadas alinhados como um vetor $\Theta = (\theta_1, \dots, \theta_M)$; $\nabla_{\mathbf{u}^{(L)}} l(\Theta)$ é o gradiente da função de avaliação $l(\Theta)$ em relação à saída $\mathbf{u}^{(L)}$; e $\left[\frac{\partial \mathbf{u}^{(L)}}{\partial \Theta} \right]$ é a matriz de derivadas parciais ou **jacobiano** da saída $\mathbf{u}^{(L)}$ em relação a Θ .

O jacobiano $\left[\frac{\partial \mathbf{u}^{(L)}}{\partial \Theta} \right]$ informa como a variação nos parâmetros afeta a saída do modelo e é calculado como:

$$\left[\frac{\partial \mathbf{u}^{(L)}}{\partial \Theta} \right] = \begin{bmatrix} \frac{\partial u_1^{(L)}}{\partial \theta_1} & \dots & \frac{\partial u_1^{(L)}}{\partial \theta_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial u_{n^{(L)}}^{(L)}}{\partial \theta_1} & \dots & \frac{\partial u_{n^{(L)}}^{(L)}}{\partial \theta_M} \end{bmatrix} \quad (9.44)$$

onde $\mathbf{u}^{(L)} = (u_1^{(L)}, \dots, u_{n^{(L)}}^{(L)})$ é o vetor de saída da rede neural com $n^{(L)}$ unidades na camada de saída L e M é o número total de parâmetros do modelo.

O gradiente da função de avaliação em relação à saída do modelo informa como a função de avaliação varia em função da saída do modelo, ou seja, o **erro do modelo** na camada de saída L , escrito como $\boldsymbol{\varepsilon}^{(L)} = \nabla_{\mathbf{u}^{(L)}} l(\boldsymbol{\Theta})$.

O ajuste de parâmetros da camada de saída é calculado como:

$$\Delta \boldsymbol{\Theta}^{(L)} = -\eta \nabla_{\boldsymbol{\Theta}^{(L)}} l(\boldsymbol{\Theta}) = -\eta \boldsymbol{\varepsilon}^{(L)} \left[\frac{\partial \mathbf{u}^{(L)}}{\partial \boldsymbol{\Theta}^{(L)}} \right] \quad (9.45)$$

A regra da cadeia pode ser aplicada às camadas anteriores, de forma que, para uma camada i , o ajuste de parâmetros é calculado como:

$$\Delta \boldsymbol{\Theta}^{(i)} = -\eta \nabla_{\boldsymbol{\Theta}^{(i)}} l(\boldsymbol{\Theta}) = -\eta \boldsymbol{\varepsilon}^{(i)} \left[\frac{\partial \mathbf{u}^{(i)}}{\partial \boldsymbol{\Theta}^{(i)}} \right] \quad (9.46)$$

onde $\boldsymbol{\varepsilon}^{(i)} = \nabla_{\mathbf{u}^{(i)}} l(\boldsymbol{\Theta})$.

O algoritmo da retropropagação deve seu nome à retropropagação do erro. Na camada de saída, $\boldsymbol{\varepsilon}^{(L-1)}$ é calculado pela regra da cadeia como:

$$\boldsymbol{\varepsilon}^{(L-1)} = \nabla_{\mathbf{u}^{(L-1)}} l(\boldsymbol{\Theta}) = \nabla_{\mathbf{u}^{(L)}} l(\boldsymbol{\Theta}) \left[\frac{\partial \mathbf{u}^{(L)}}{\partial \mathbf{u}^{(L-1)}} \right] = \boldsymbol{\varepsilon}^{(L)} \left[\frac{\partial \mathbf{u}^{(L)}}{\partial \mathbf{u}^{(L-1)}} \right] \quad (9.47)$$

Analogamente, o erro retropropagado para uma camada $i - 1$, $\boldsymbol{\varepsilon}^{(i-1)}$, é calculado como:

$$\boldsymbol{\varepsilon}^{(i-1)} = \nabla_{\mathbf{u}^{(i-1)}} l(\boldsymbol{\Theta}) = \nabla_{\mathbf{u}^{(i)}} l(\boldsymbol{\Theta}) \left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right] = \boldsymbol{\varepsilon}^{(i)} \left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right] \quad (9.48)$$

O algoritmo da retropropagação é encapsulado em cada camada, que recebe o erro $\boldsymbol{\varepsilon}^{(i)}$ da camada seguinte e produz o ajuste de parâmetros $\Delta \boldsymbol{\Theta}^{(i)}$ e o erro retropropagado para a camada anterior $\boldsymbol{\varepsilon}^{(i-1)}$ a partir das matrizes jacobianas $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \boldsymbol{\Theta}^{(i)}} \right]$ e $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right]$.

O algoritmo da retropropagação é apresentado inicialmente para o processamento *on-line* (gradiente estocástico). No processamento em *batch* ou *mini-batch*, basta somar os valores do gradiente para todos os registros.

O processamento *online* de uma camada totalmente conectada foi descrito na equação (9.22) como:

$$\mathbf{u}^{(i)} = \mathbf{h}^{(i)}(\mathbf{z}^{(i)}) = \mathbf{h}^{(i)}(\mathbf{b}^{(i)} + \mathbf{u}^{(i-1)} \mathbf{W}^{(i)}) \quad (9.49)$$

O jacobiano $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \boldsymbol{\Theta}^{(i)}} \right]$ na equação (9.46) é calculado pela regra da cadeia:

$$\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \Theta^{(i)}} \right] = \left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{z}^{(i)}} \right] \left[\frac{\partial \mathbf{z}^{(i)}}{\partial \Theta^{(i)}} \right]^T \quad (9.50)$$

Em cada camada, a função de ativação é aplicada a cada elemento do vetor $\mathbf{z}^{(i)}$, de forma que $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{z}^{(j)}} \right] = 0, \forall i \neq j$. A derivada da função de ativação $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{z}^{(i)}} \right]$ é então uma matriz diagonal e pode ser representada por um vetor de $n^{(i)}$ elementos. Dessa forma, a equação (9.46) pode ser escrita como:

$$\Delta \Theta^{(i)} = -\eta \boldsymbol{\varepsilon}^{(i)} \odot \mathbf{h}'^{(i)} \left[\frac{\partial \mathbf{z}^{(i)}}{\partial \Theta^{(i)}} \right]^T = \boldsymbol{\delta}^{(i)} \left[\frac{\partial \mathbf{z}^{(i)}}{\partial \Theta^{(i)}} \right]^T \quad (9.51)$$

onde $\mathbf{h}'^{(i)} = \left(\frac{\partial u_1^{(i)}}{\partial z_1^{(i)}}, \dots, \frac{\partial u_{n^{(i)}}^{(i)}}{\partial z_{n^{(i)}}^{(i)}} \right)$ é a diagonal da matriz $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{z}^{(i)}} \right]$ e o operador $\odot$ é a multiplicação vetorial elemento a elemento.

O **gradiente local** da camada i é definido na equação (9.51) como:

$$\boldsymbol{\delta}^{(i)} = \boldsymbol{\varepsilon}^{(i)} \odot \mathbf{h}'^{(i)} \quad (9.52)$$

Analogamente, o jacobiano $\left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right] = \left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{z}^{(i)}} \right] \left[\frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right]^T$, de forma que o erro retropropagado também é calculado em função do gradiente local como (cf. equação (9.48)):

$$\boldsymbol{\varepsilon}^{(i-1)} = \boldsymbol{\varepsilon}^{(i)} \left[\frac{\partial \mathbf{u}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right] = \boldsymbol{\varepsilon}^{(i)} \odot \mathbf{h}'^{(i)} \left[\frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right]^T = \boldsymbol{\delta}^{(i)} \left[\frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{u}^{(i-1)}} \right]^T \quad (9.53)$$

Finalmente, aplicando a equação (9.51) na equação (9.49), o ajuste de parâmetros de uma camada de rede neural MLP é calculado como:

$$\Delta \mathbf{W}^{(i)} = -\eta \mathbf{u}^{(i-1)T} \boldsymbol{\delta}^{(i)} \quad (9.54)$$

$$\Delta \mathbf{b}^{(i)} = -\eta \boldsymbol{\delta}^{(i)} \quad (9.55)$$

O erro retropropagado é calculado como:

$$\boldsymbol{\varepsilon}^{(i-1)} = \boldsymbol{\delta}^{(i)} \mathbf{W}^{(i)T} \quad (9.56)$$

A retropropagação do erro em cada camada utiliza os mesmos parâmetros da matriz de conexões do cálculo da saída da rede. Observe que o vetor de parâmetros da unidade bias não participa da retropropagação do erro, uma vez que essa unidade não está conectada com a camada anterior.

O erro do modelo na camada de saída é calculado de acordo com a função de avaliação e a função de ativação na camada de saída. Nos modelos usuais, descritos na seção 9.2.2, o gradiente local é calculado para os problemas de regressão e classificação como:

- No problema de **regressão**, a função de avaliação é o erro quadrático, a função de ativação da camada de saída é linear e o gradiente local é calculado como:

$$\delta^{(L)} = -\mathbf{e}(t) = -(\mathbf{y}(t) - \hat{\mathbf{y}}(t)) \quad (9.57)$$

- No problema de **classificação**, a função de avaliação é entropia cruzada, a função *softmax* é a função de ativação e o gradiente local é calculado como:

$$\delta^{(L)} = -\mathbf{e}(t) = -(\mathbf{v}(t) - \hat{\mathbf{v}}(t)) \quad (9.58)$$

Nas camadas internas, o gradiente local é calculado como na equação (9.52), pelo erro retropropagado e a derivada da função de ativação. As derivadas das funções de ativação usuais são ilustradas na Figura 9.9 e são calculadas como:

- a) Função linear:

$$\text{lin}'(z) = 1 \quad (9.59)$$

- b) Função sigmoide:

$$\text{sig}'(z) = \text{sig}(z)(1 - \text{sig}(z)) \quad (9.60)$$

- c) Função linear retificada:

$$\text{relu}'(z) = \begin{cases} 0, & \text{se } z < 0 \\ 1, & \text{se } z \geq 0 \end{cases} \quad (9.61)$$

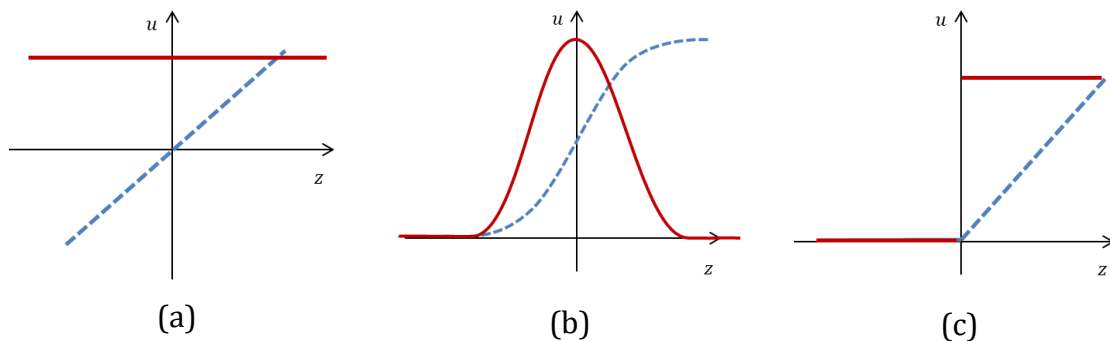


Figura 9.9: Funções de ativação em azul e suas derivadas em vermelho: (a) função linear; (b) função sigmoide; (c) função linear retificada (ReLU).

As equações (9.54) a (9.56) apresentam o ajuste de parâmetros pelo algoritmo da retropropagação para o processamento *online* (gradiente estocástico). No caso do processamento em *batch* ou *mini-batch*, o ajuste de parâmetros é calculado como:

$$\Delta \mathbf{W}^{(i)} = -\eta \sum_{t=1}^N \mathbf{u}^{(i-1)T} \boldsymbol{\delta}^{(i)} = -\eta \mathbf{U}^{(i-1)T} \mathbf{D}^{(i)} \quad (9.62)$$

$$\Delta \mathbf{b}^{(i)} = -\eta \sum_{t=1}^N \boldsymbol{\delta}^{(i)} = -\eta \mathbf{1} \mathbf{D}^{(i)} \quad (9.63)$$

onde $\mathbf{U}^{(i-1)}$ é a matriz de dimensão $N \times n^{(i-1)}$ com os valores de ativação de todos os registros do conjunto (*batch* ou *mini-batch*), a matriz $\mathbf{D}^{(i)}$ de dimensão $N \times n^{(i)}$ armazena os vetores de gradiente local para todos os registros do conjunto e $\mathbf{1} = (1, 1, \dots, 1)$ é o vetor unitário de dimensão N . A matriz $\mathbf{D}^{(i)}$ pode ser calculada vetorialmente como $\mathbf{D}^{(i)} = \mathbf{E}^{(i)} \odot \mathbf{H}'^{(i)}$, sendo $\mathbf{H}'^{(i)}$ a matriz de dimensão $N \times n^{(i)}$ que armazena os vetores de derivadas da função de ativação da camada para cada registro.

O erro retropropagado é calculado vetorialmente para o caso do processamento em *batch* ou *mini-batch* como:

$$\mathbf{E}^{(i-1)} = \mathbf{D}^{(i)} \mathbf{W}^{(i)T} \quad (9.64)$$

O Algoritmo 9.3 apresenta o processo completo de cálculo das saídas e ajuste de parâmetros de uma rede MLP com o processamento vetorial dos registros em *mini-batch*. O critério de parada é o número máximo de iterações (ou épocas) definido pela variável *itmax*. O algoritmo pode ser utilizado para problemas de classificação ou de regressão de acordo com os dados do conjunto de treinamento.

A função *select* na linha 04 realiza, a cada iteração, a extração de *bsize* registros do conjunto de treinamento e retorna as matrizes de entrada e saída. A extração é controlada pelos parâmetros *nbatch* e *bsize*: no processamento *online*, *nbatch* = N e *bsize* = 1; no processamento em *batch*, *nbatch* = 1 e *bsize* = N ; ou outras combinações no processamento *mini-batch*. Além da amostragem sequencial, a função *select* pode ser implementada para realizar a extração aleatória para o treinamento do modelo com amostragem em *bootstrapping*.

A linha 08 representa o cálculo da função de ativação de acordo com o tipo de problema e as características do modelo. A linha 12 representa o cálculo da derivada da função

de ativação, de forma que $\mathbf{H}'^{(L)}(\mathbf{Z}^{(L)}) = \mathbf{1}$ (cf. equações (9.57) e (9.58)). Em cada iteração, os parâmetros $\Theta^{(i)}$, assim como os resultados de potencial $\mathbf{Z}^{(i)}$ e de ativação $\mathbf{U}^{(i)}$ de cada camada, devem ser armazenados para o ajuste de parâmetros. No caso de conjuntos de dados muito grandes, esse armazenamento pode comprometer o desempenho ou mesmo inviabilizar o processamento em *batch*.

Algoritmo 9.3: Treinamento em *batch* (ou *mini-batch*) de uma rede neural MLP

Entrada:	Conjunto de treinamento $\mathcal{T} = \{(\mathbf{x}(t), \mathbf{v}(t)), t = 1 \dots N\}$, e taxa de aprendizagem η e hiperparâmetros do modelo
Saída:	Conjunto de parâmetros $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}$
01	Início
02	Inicialização: $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}; k = 1$
03	Para $k = 1, \dots, itmax$
04	Para $t = 1, \dots, nbatch$
05	$[\mathbf{U}^{(0)}, \mathbf{V}_t] = \text{select}(\mathbf{X}, \mathbf{V}, t, bsize)$
06	Para $i = 1, \dots, L$ # Cálculo das saídas
07	$\mathbf{Z}^{(i)} = \mathbf{1}^T \mathbf{b}^{(i)} + \mathbf{U}^{(i-1)} \mathbf{W}^{(i)}$
08	$\mathbf{U}^{(i)} = \mathbf{H}^{(i)}(\mathbf{Z}^{(i)})$
09	Fim Para
10	$\mathbf{E}^{(L)} = -(\mathbf{V}_t - \mathbf{U}^{(L)})$
11	Para $i = L, \dots, 1$ # Ajuste de parâmetros
12	$\mathbf{D}^{(i)} = \mathbf{E}^{(i)} \odot \mathbf{H}'^{(i)}(\mathbf{Z}^{(i)})$
13	$\mathbf{W}^{(i)} \leftarrow \mathbf{W}^{(i)} - \eta \mathbf{U}^{(i-1)T} \mathbf{D}^{(i)}$
14	$\mathbf{b}^{(i)} \leftarrow \mathbf{b}^{(i)} - \eta \mathbf{1} \mathbf{D}^{(i)}$
15	$\mathbf{E}^{(i-1)} = \mathbf{D}^{(i)} \mathbf{W}^{(i)T}$
16	Fim Para
17	Fim Para
18	Fim Para
19	Retorna o vetor de parâmetros $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}$
20	Fim

A Figura 9.10 resume o fluxo do processamento da saída e do ajuste de parâmetros pelo algoritmo da retropropagação em uma rede MLP.

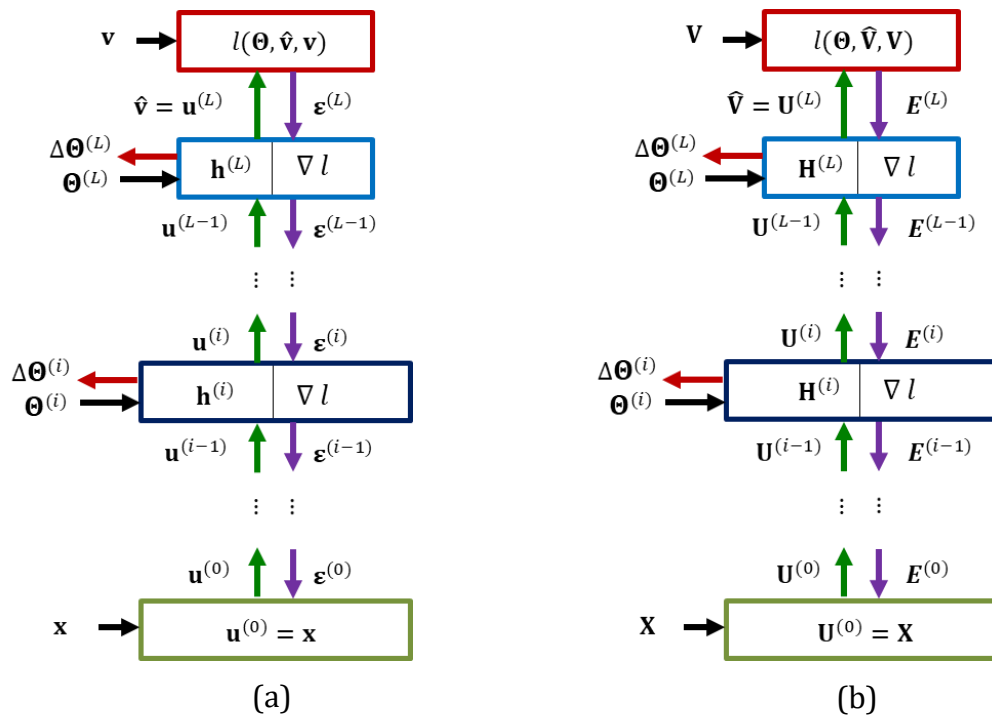


Figura 9.10: Modelo MLP: (a) processamento *online*; (b) processamento em *batch* ou *mini-batch*.

O algoritmo da retropropagação apresenta bons resultados em problemas simples de regressão e classificação. Em geral, os parâmetros são inicializados com valores aleatórios pequenos [240][464]. O resultado do modelo pode ser diferente de acordo com a inicialização, principalmente se o número de registros do conjunto de treinamento for pequeno. Uma solução é executar o ajuste diversas vezes com diferentes inicializações e utilizar uma combinação de resultados, o que pode ter alto custo computacional em modelos complexos e conjuntos de dados grandes.

O algoritmo de retropropagação é um algoritmo clássico que aparece em diversas referências de redes neurais [240], aprendizado de máquinas e reconhecimento de padrões [52][238][521], com boas referências de autores brasileiros [62][173]. O algoritmo tem sido estudado há décadas e livros inteiros foram dedicados ao assunto. A coletânea editada por Chauvin e Rumelhart em 1995 [85] reúne aplicações e variantes do algoritmo em diferentes topologias de redes neurais MLP e recorrentes.

A retropropagação do erro é um cálculo recorrente (cf. equação (9.52)). Em cada camada, o gradiente local é multiplicado pela matriz de conexões (cf. equação (9.56)) e o erro retropropagado é utilizado para o cálculo do gradiente local da camada anterior, e assim sucessivamente. Esse processo produz a diluição do gradiente e outros proble-

mas de convergência. Há diversas variantes e extensões do algoritmo da retropropagação para melhorar o desempenho do ajuste de parâmetros de redes neurais [85][398]; as principais são discutidas a seguir.

9.2.3.2 Principais variantes

No famoso artigo de 1986 [464], os autores sugerem uma modificação no cálculo do ajuste de parâmetros para aceleração da convergência. A ideia é adicionar uma parcela do ajuste calculado na iteração anterior ao gradiente calculado na iteração corrente. O termo adicional gera uma inércia (*momentum*), que suaviza a trajetória de ajuste no espaço dos parâmetros do modelo, melhorando a convergência. A atualização dos parâmetros do modelo $\Theta = \{\Theta^{(1)}, \dots, \Theta^{(L)}\}$ é calculada como:

$$\Delta\Theta(k) = -\eta\nabla_{\Theta}l(\Theta) + \alpha\Delta\Theta(k-1) \quad (9.65)$$

onde k é a iteração corrente, $\alpha \in [0,1]$ é a taxa de momento, $\Delta\Theta(k)$ e $\Delta\Theta(k-1)$ são, respectivamente, os ajustes realizados na iteração corrente e na anterior.

Considerando uma região do espaço de parâmetros em que a função de avaliação tem uma curvatura constante como $\nabla_{\Theta}l(\Theta) \approx \mathbf{d}$, a regra de atualização do momento pode ser escrita como [521]:

$$\Delta\Theta(k) \approx -\eta(1 + \alpha + \alpha^2 + \alpha^3 + \dots)\mathbf{d} = -\frac{\eta}{1-\alpha}\mathbf{d} \quad (9.66)$$

A equação (9.66) mostra que, nesse caso, a atualização do momento contribui para aumentar a taxa de aprendizado do método do gradiente, uma vez que $\alpha < 1$.

Diferentes abordagens de taxas de aprendizagem adaptativas são utilizadas para acelerar a convergência do método do gradiente [282][463]. O algoritmo RPROP* [455] é uma heurística para ajustar o valor da taxa de aprendizagem em função do sinal dos gradientes calculados nas iterações anterior e corrente. Se os gradientes têm o mesmo sinal, a taxa de aprendizagem é incrementada ou decrementada, caso contrário:

$$\eta \leftarrow \begin{cases} (1 + \xi) \eta, & \text{se } \mathbf{d}(k-1)\mathbf{d}(k)^T > 0 \\ (1 - \xi) \eta, & \text{se } \mathbf{d}(k-1)\mathbf{d}(k)^T \leq 0 \end{cases} \quad (9.67)$$

onde $\mathbf{d}(k-1)$ e $\mathbf{d}(k)$ representam o gradiente da função de avaliação $\nabla_{\Theta}l(\Theta)$, calculados na iteração anterior e na corrente, respectivamente, e $\xi \approx 0.1$ é o parâmetro de

* *Resilient Backpropagation.*

atualização da taxa de aprendizagem, que varia num intervalo predeterminado, por exemplo: $\eta \in [0.01; 10]$.

Algumas variantes do algoritmo da retropropagação utilizam uma taxa de aprendizagem para cada componente do gradiente. Uma extensão conhecida por obter bons resultados é o algoritmo AdaGrad [145]. Nesse caso, o ajuste de parâmetros numa iteração k do algoritmo é realizado como:

$$\Delta \Theta(k) = -\hat{\eta}(k) \odot \mathbf{d}(k) \quad (9.68)$$

onde $\mathbf{d}(k) = \nabla_{\Theta} l(\Theta)$ é o gradiente da função de avaliação (alinhados como um vetor) calculado na iteração k e $\odot$ é o operador de multiplicação vetorial elemento a elemento.

O vetor de taxas de aprendizagem adaptativas $\hat{\eta}$ é calculado pela soma cumulativa do gradiente ao quadrado nas iterações anteriores:

$$\mathbf{g}(k) = \mathbf{g}(k-1) + \mathbf{d}(k) \odot \mathbf{d}(k) \quad (9.69)$$

$$\hat{\eta}(k) = -\frac{\eta}{\sqrt{\epsilon + \mathbf{g}(k)}} \quad (9.70)$$

onde $\mathbf{g}(0) = \mathbf{0}$; $\epsilon \approx 10^{-6}$ é uma constante para evitar a divisão por zero na equação (9.70) e η é o valor escalar da taxa de aprendizagem.

Uma variante do AdaGrad é o RMSprop, em que o vetor de taxa de aprendizagem adaptativa é calculado como na equação (9.70), mas a soma do gradiente ao quadrado é filtrada por um parâmetro ρ :

$$\mathbf{g}(k) = \rho \mathbf{g}(k-1) + (1 - \rho) \mathbf{d}(k) \odot \mathbf{d}(k) \quad (9.71)$$

onde os valores sugeridos dos parâmetros são $\rho = 0.9$ e $\eta = 0.001$ [463].

Em problemas pequenos, resultados efetivos de aceleração de convergência podem ser obtidos com métodos de segunda ordem, em que uma aproximação da derivada segunda da função de avaliação é utilizada no ajuste de parâmetros [212][487]. Os algoritmos de segunda ordem mais utilizados para ajuste de parâmetros em redes neurais são o algoritmo Levenberg-Marquardt [341][369] e o método do gradiente conjugado [248][418]. Esses algoritmos podem se tornar instáveis no caso de problemas de classificação com a entropia cruzada como função de ativação e não são viáveis no caso de modelos muito complexos.

Modelos de redes neurais MLP em problemas complexos e/ou de grandes dimensões utilizam frequentemente técnicas de regularização.

9.2.3.3 Regularização

O ajuste de parâmetros de redes neurais pode ser feito com uma versão regularizada da função de avaliação, introduzindo-se uma penalidade sobre o vetor de parâmetros:

$$l_R(\Theta) = l(\Theta) + \Omega(\Theta) \quad (9.72)$$

onde $l(\Theta)$ é a versão não regularizada da função de avaliação e $\Omega(\Theta)$, uma função de penalidade.

Há diferentes abordagens para a função de penalidade. A regularização mais comum é aplicar uma penalidade sobre a norma L^2 dos parâmetros, como na regularização de Tikhonov (cf. seção 3.2.2). Essa técnica é conhecida como *weight decay* na literatura de redes neurais [238][240]. As funções de avaliação usuais em cada tipo de problema são escritas com regularização L^2 como:

- No problema de **regressão**: erro quadrático regularizado:

$$l_R(\Theta) = \frac{1}{2} \sum_{t=1}^N \|\mathbf{y}(t) - \hat{\mathbf{y}}(t)\|^2 + \mu \|\Theta\|^2 \quad (9.73)$$

- No problema de **classificação**: entropia cruzada regularizada:

$$l_R(\Theta) = - \sum_{t=1}^N \mathbf{v}(t) \log(\hat{\mathbf{v}}(t))^T + \mu \|\Theta\|^2 \quad (9.74)$$

onde μ é a constante de regularização.

Os parâmetros das unidades bias não são regularizados, uma vez que seu objetivo é apenas posicionar o centro da função de ativação. O ajuste de parâmetros da matriz de conexões de cada camada é calculado como:

- No processamento *online*:

$$\Delta \mathbf{W}^{(i)} = -\eta \mathbf{u}^{(i-1)T} \boldsymbol{\delta}^{(i)} + \mu \mathbf{W}^{(i)}, \quad i = 1, \dots, L \quad (9.75)$$

- No processamento em *batch* (ou *mini-batch*):

$$\Delta \mathbf{W}^{(i)} = -\eta \mathbf{U}^{(i-1)T} \mathbf{D}^{(i)} + \mu \mathbf{W}^{(i)}, \quad i = 1, \dots, L \quad (9.76)$$

Redes neurais profundas, com muitas camadas intermediárias, apresentam problemas de convergência quando ajustadas pelo método do gradiente. O efeito do erro na camada de saída é menor cada vez que o gradiente local é retropropagado [37][251]. Até meados dos anos 2000, esse fato era pouco reportado na literatura. Uma estratégia para solução do problema é o pré-treinamento não supervisionado, proposto por Hinton e Salakhutdinov em 2006 [251], que será discutido na seção 9.3.3. Novos algoritmos e estratégias de parametrização têm permitido o ajuste de parâmetros de modelos complexos de redes neurais profundas. Uma técnica simples e que permite o treinamento de redes neurais profundas é conhecida como *dropout* [503].

A ideia do algoritmo *dropout* é retirar aleatoriamente algumas unidades do modelo em cada iteração do algoritmo de treinamento (cf. Figura 9.11). Ao término do treinamento, o modelo completo é utilizado para predição.

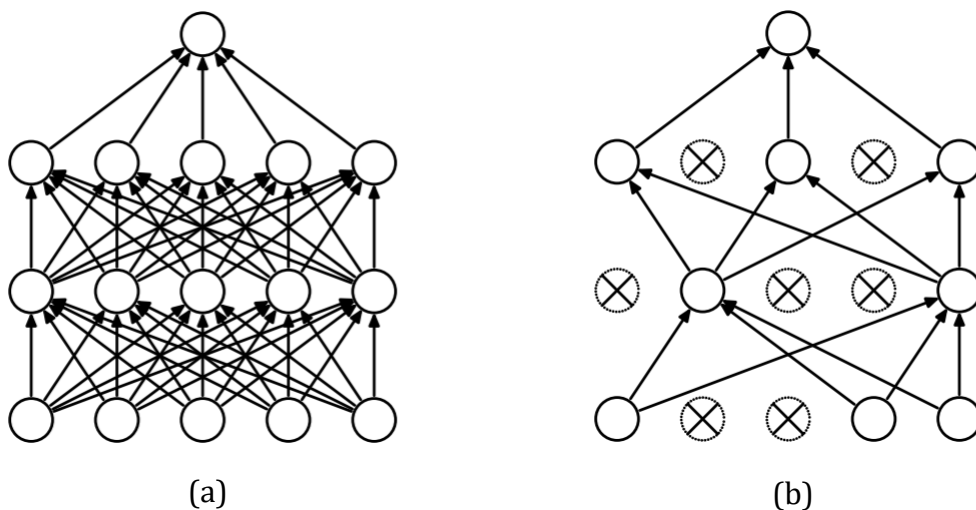


Figura 9.11: Exemplo de *dropout*: (a) rede MLP completa; (b) rede MLP com unidades retiradas para o treinamento. Fonte: reproduzido de [503].

Considere o processamento *online* de uma camada genérica de uma rede neural MLP (cf. equação (9.22)) escrito como (cf. [503]):

$$\mathbf{z}^{(i+1)} = \mathbf{b}^{(i+1)} + \mathbf{u}^{(i)} \mathbf{W}^{(i+1)} \quad (9.77)$$

$$\mathbf{u}^{(i+1)} = \mathbf{h}^{(i+1)}(\mathbf{z}^{(i+1)}) \quad (9.78)$$

Em cada iteração do algoritmo de treinamento, um vetor $\mathbf{p}^{(i)} \in \{0,1\}^{n^{(i)}}$ é obtido, para cada camada $i = 1, \dots, L$, pela realização de uma distribuição de Bernoulli com probabilidade π , que é o único parâmetro do *dropout*. O artigo original sugere $\pi = 0.5$ [503],

que corresponde ao valor máximo de regularização, mas outros valores podem ser utilizados [26]. As unidades são retiradas pela multiplicação do vetor $\mathbf{p}^{(i)}$ pela entrada da camada. O processamento *online* é calculado como:

$$\hat{\mathbf{z}}^{(i+1)} = \mathbf{b}^{(i+1)} + (\mathbf{p}^{(i)} \odot \mathbf{u}^{(i)}) \mathbf{W}^{(i+1)} \quad (9.79)$$

$$\mathbf{u}^{(i+1)} = \mathbf{h}^{(i+1)}(\hat{\mathbf{z}}^{(i+1)}) \quad (9.80)$$

As unidades “apagadas” não são consideradas no cálculo do potencial de saída, sem alterar as dimensões da matriz de conexões. No caso do processamento em *batch* (ou *mini-batch*), o mesmo vetor binário é aplicado a todos os registros do subconjunto de treinamento (cf. equação (9.23)):

$$\hat{\mathbf{Z}}^{(i+1)} = \mathbf{1}^T \mathbf{b}^{(i+1)} + (\mathbf{1}^T \mathbf{p}^{(i)} \odot \mathbf{U}^{(i)}) \mathbf{W}^{(i+1)} \quad (9.81)$$

$$\mathbf{U}^{(i+1)} = \mathbf{H}^{(i+1)}(\hat{\mathbf{Z}}^{(i+1)}) \quad (9.82)$$

O ajuste de parâmetros é feito, em cada iteração, com o vetor de entradas removidas $\hat{\mathbf{u}}^{(i)} = \mathbf{p}^{(i)} \odot \mathbf{u}^{(i)}$. No caso do processamento *online*:

$$\Delta \mathbf{W}^{(i+1)} = -\eta \hat{\mathbf{u}}^{(i)T} \boldsymbol{\delta}^{(i+1)} \quad (9.83)$$

ou no caso do processamento em *batch* (ou *mini-batch*):

$$\Delta \mathbf{W}^{(i+1)} = -\eta \hat{\mathbf{U}}^{(i)T} \boldsymbol{\delta}^{(i+1)} \quad (9.84)$$

onde $\hat{\mathbf{U}}^{(i)} = \mathbf{1}^T \mathbf{p}^{(i)} \odot \mathbf{U}^{(i)}$.

A cada iteração, um novo vetor binário aleatório $\mathbf{p}^{(i)}$ é gerado para cada camada. O processamento da saída e o ajuste de parâmetros da matriz de conexões é feito com as unidades apagadas. O processo se repete até a convergência.

Após o treinamento, o modelo completo é utilizado para predição, ou seja utilizando a equação (9.22) no caso de processamento *online* ou a equação (9.23) no processamento em *batch* (ou *mini-batch*). O modelo final é uma combinação (ou *ensemble*) dos diversos modelos gerados durante o treinamento [212].

O *dropout* é um método simples e eficiente que pode ser utilizado em qualquer modelo de rede neural profunda e que ainda pode ser combinado com outras técnicas de regularização. O *dropout* tem bons resultados para o ajuste de modelos complexos treinados com grandes conjuntos de dados. Em problemas pequenos, não é tão eficaz.

A interrupção do treinamento também é utilizada como regularização. Nesse caso, o conjunto de treinamento é particionado em dois subconjuntos: um para o cálculo do ajuste de parâmetros e outro para ser utilizado como uma avaliação “isenta” do desempenho do modelo, chamado de conjunto de validação. O processo iterativo é interrompido quando alguma estatística calculada no conjunto de validação começa a aumentar após iterações sucessivas. A interrupção do treinamento pela avaliação no conjunto de validação visa maximizar a capacidade de generalização do modelo.

Os métodos de regularização visam controlar a capacidade do modelo, que é uma função das variáveis de entrada e de outros hiperparâmetros do modelo. O problema de seleção de variáveis, ou seleção de características, é bem estudado e há diversas referências sobre o tema [224][238][350]. No caso de redes neurais, a escolha da topologia é uma etapa importante do ajuste de modelos e diferentes abordagens são descritas na literatura [188][267][274][486].

Em problemas simples, a topologia de uma rede neural MLP pode ser selecionada pela abordagem de seleção incremental [253][312]. Essa técnica é uma heurística gulosa que, a cada iteração, avalia todas as variáveis disponíveis e seleciona uma para ser incluída no modelo de acordo com a maior redução da função de avaliação. Na técnica conhecida como *wrapper* [312], diversas combinações de modelos são testadas numa busca exaustiva (*grid search*) ou aleatória. Em modelos mais complexos de redes neurais, esse tipo de abordagem iterativa não é viável e apenas algumas combinações dos hiperparâmetros são avaliadas.

Atualmente, diversos modelos pré-treinados estão disponíveis em repositórios como GitHub¹⁵⁰ e podem ser adaptados para problemas específicos, pela técnica chamada de transferência de aprendizado [426][556].

9.2.3.4 Exemplo

A Figura 9.12 e a Figura 9.13 mostram exemplos de dois modelos de redes neurais no problema de duas espirais obtidos com o aplicativo *TensorFlow Playground*.¹⁵¹ A ferramenta realiza o treinamento de modelos de redes neurais para amostras de dados extraídas de quatro problemas de classificação e um problema de regressão em duas dimensões. A interface permite a escolha de diferentes configurações de topologia, função de ativação, regularização para cada tipo de problema. As simulações apresentadas

foram executadas com 60% de dados de treinamento, ruído de 20% e subconjunto de treinamento (*batch size*) de 10 registros.

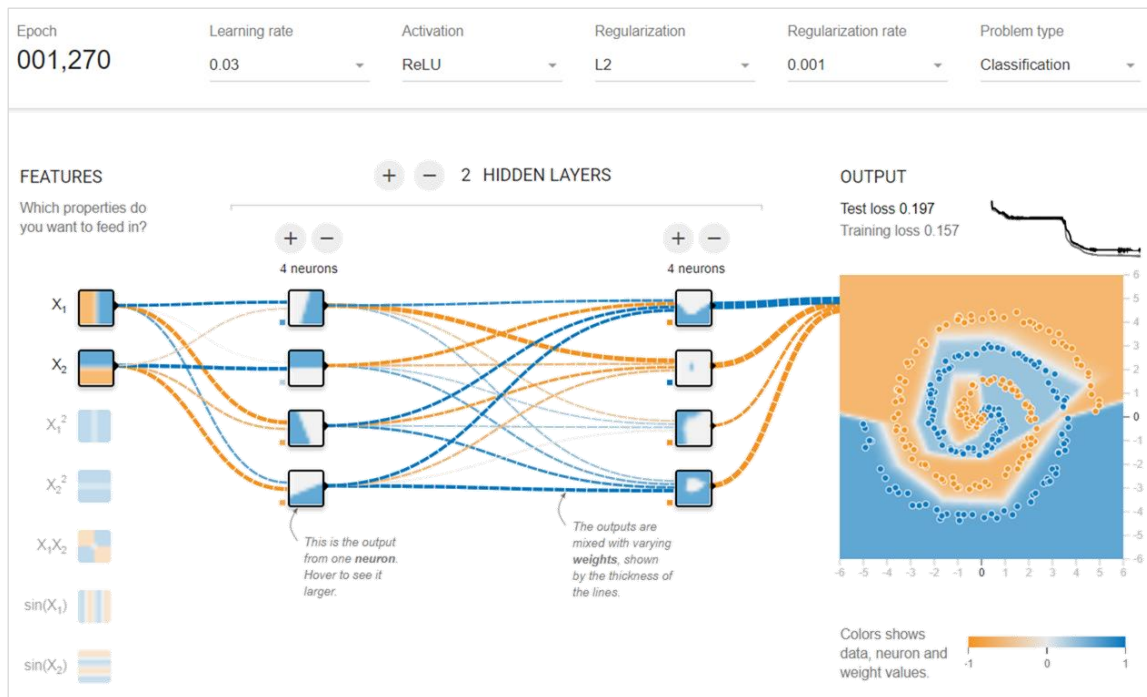


Figura 9.12: Modelo 2:4:4:1. Fonte: *TensorFlow Playground*.

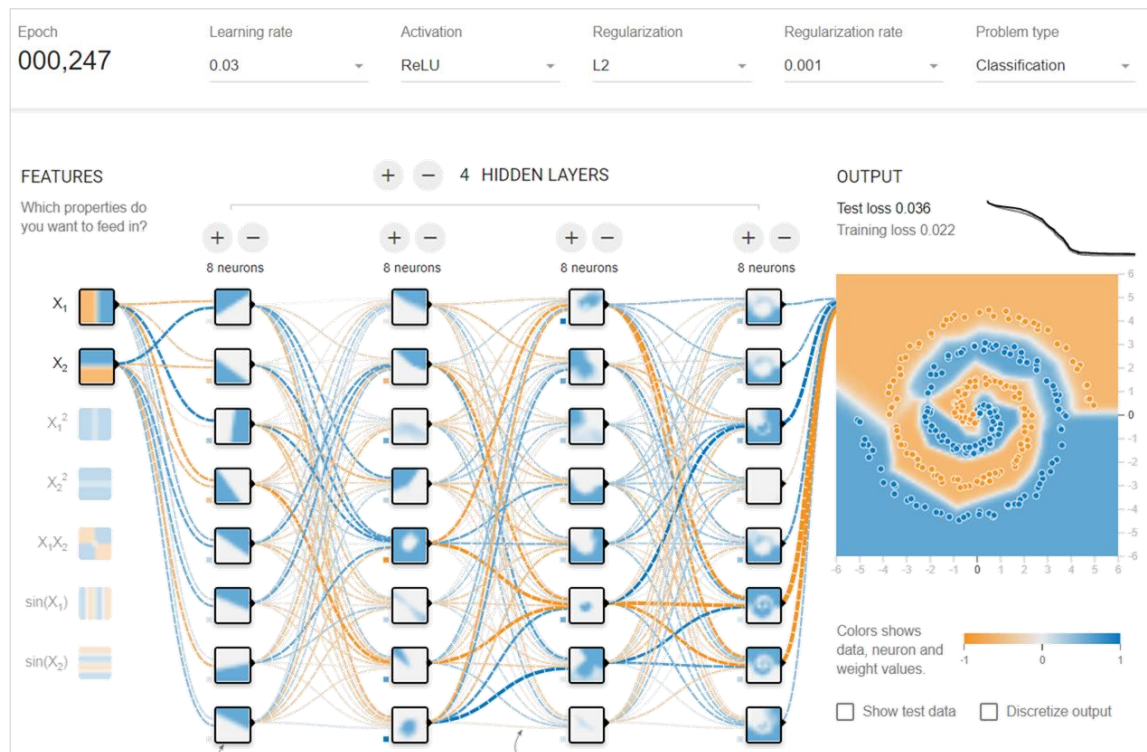


Figura 9.13: Modelo 2:8:8:8:1. Fonte: *TensorFlow Playground*.

Na Figura 9.12, o modelo com $L = 2$ camadas intermediárias e quatro unidades em cada camada não realiza o particionamento completo das classes, mesmo depois de mais de 1.250 iterações. Na Figura 9.13, um modelo de $L = 4$ camadas intermediárias e oito unidades em cada camada converge para uma boa solução em menos de 250 iterações.

O *TensorFlow Playground* também mostra, para cada unidade de processamento da rede neural, a região do espaço de entrada que mais ativa aquela unidade, sendo que a unidade de saída é apresentada em destaque à direita. Na primeira camada, as regiões ativadas em todas as unidades são definidas por hiperplanos. Nas camadas seguintes, as unidades apresentam regiões ativadas cada vez mais complexas, até que, na penúltima camada, as regiões ativadas aproximam o formato definido pela distribuição dos dados. Essa visão, principalmente na Figura 9.13, mostra como cada camada interna de uma rede neural realiza a representação de características cada vez mais complexas e com maior nível de abstração do espaço de entrada.

Redes neurais totalmente conectadas geram um número muito grande de parâmetros, o que inviabiliza o ajuste de modelos com muitas camadas em problemas de alta dimensionalidade. Outras topologias que permitem o compartilhamento de parâmetros têm sido utilizadas com sucesso em problemas como visão computacional e processamento de linguagem natural, como descrito a seguir.

9.3 Redes Neurais Profundas

O desenvolvimento de grandes bases de dados anotadas, bem como o processamento eficiente massivamente paralelo em GPUs, têm permitido o ajuste de modelos de redes neurais profundas em problemas complexos de aprendizado de máquina [212][333]. LeCun, Bengio e Hinton, autores do artigo [333], receberam o prêmio Turing¹⁵² em 2019 por suas contribuições ao desenvolvimento de redes neurais profundas.

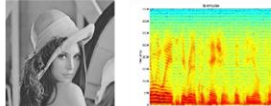
Uma rede neural profunda processa dados de alta dimensionalidade em problemas complexos como visão computacional e processamento de linguagem natural, cujos dados são representados por tensores. Um **tensor** generaliza a noção de escalares, vetores e matrizes: um tensor de ordem 0 é um escalar; um tensor de ordem 1 é um vetor; um tensor de ordem 2 é uma matriz e assim sucessivamente. De forma geral, um tensor de ordem k é uma lista de tensores de ordem $k - 1$, de mesma dimensão:

$$\mathbb{X}^k = [\mathbb{X}_1^{k-1}, \dots, \mathbb{X}_n^{k-1}] \quad (9.85)$$

onde $\mathbb{X}^k$ representa um tensor de ordem k .

A Tabela 9.3 mostra alguns exemplos de dados de diversas dimensões e sua representação tensorial como vetores, matrizes e tensores. Um sinal de áudio pode ser armazenado num vetor. Uma matriz, ou tensor de ordem 2, é uma lista de vetores e pode armazenar uma imagem em escalas de cinza ou um espectrograma.* Uma imagem colorida tem três canais de cores (RGB), sendo armazenada num tensor de ordem 3, como uma lista de imagens de cada canal. Um conjunto de imagens em escala de cinza também é um tensor de ordem 3, mas a terceira dimensão representa os registros, ou seja, objetos diferentes. Na imagem multicanal, os canais representam múltiplas visões do mesmo objeto. Há imagens de objetos tridimensionais, como cubos sísmicos e imagens de tomografia computadorizada, em que uma única imagem é armazenada como tensor de ordem 3. Um conjunto de imagens coloridas, como o ImageNet,¹⁵³ é uma lista de tensores de ordem 3, sendo armazenado por um tensor de ordem 4.

Tabela 9.3: Exemplos de tensores.

Dimensão	Exemplo	Representação	Notação
1D	Áudio 	Vetor ou Tensor de ordem 1	$\mathbf{x} = [x_1, \dots, x_m]$
2D	Imagem, espectrograma 	Matriz ou Tensor de ordem 2	$\mathbf{X} = [\mathbf{x}_1^T, \dots, \mathbf{x}_n^T]$ $\mathbf{X} = \begin{bmatrix} x_{11} & \dots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \dots & x_{mn} \end{bmatrix}$
3D	Imagem multi-canal 	Tensor de ordem 3	$\mathbb{X}^3 = [\mathbf{X}_1, \dots, \mathbf{X}_n]$ $\mathbb{X}^3 = [\mathbb{X}_1^2, \dots, \mathbb{X}_n^2]$
kD	Conjunto de imagens (4D) 	Tensor de ordem k	$\mathbb{X}^k = [\mathbb{X}_1^{k-1}, \dots, \mathbb{X}_n^{k-1}]$

* Espectrogramas são gráficos que representam a densidade espectral de energia do sinal no plano tempo x frequência.

O desenvolvimento de redes neurais profundas tem sido impulsionado pela oferta de códigos abertos e plataformas de desenvolvimento com bibliotecas de operações tensoriais e outros recursos. As plataformas mais importantes para desenvolvimento de modelos de redes neurais profundas são:

- Theano,¹⁵⁴ primeiro projeto de código aberto para redes neurais profundas, foi desenvolvido em Python pelo *Montreal Institute for Learning Algorithms* (MILA) da Universidade de Montreal. Foi descontinuada em 2017.
- TensorFlow,¹⁵⁵ desenvolvido e mantido pelo Google com interface nas linguagens Python e C. É uma das plataformas mais completas e mais utilizadas [1]. Em 2017, foi lançada a biblioteca TensorFlow Lite, que permite converter e executar modelos de redes neurais profundas em dispositivos móveis e sistemas embarcados.
- PyTorch,¹⁵⁶ desenvolvido e mantido pelo Facebook com interface na linguagem Python e baseado na plataforma Torch, desenvolvida na linguagem Lua.
- Cognitive ToolKit (CNTK),¹⁵⁷ desenvolvido e mantido pela Microsoft. Tem interface para as linguagens Python e C++.
- MXNet,¹⁵⁸ desenvolvida pela Fundação Apache, tem interface em múltiplas linguagens (C++, Python, R, entre outras) e é mantida por empresas como Amazon, Intel, Baidu e instituições de pesquisa como Carnegie Mellon, MIT, entre outras.
- Caffe,¹⁵⁹ desenvolvida pela Universidade de Berkeley. Tem interface em Python e recursos para processamento em GPU e CPU. Em 2017, foi estendida pelo Facebook na plataforma Caffe2.¹⁶⁰
- Keras,¹⁶¹ é uma plataforma de alto nível escrita em Python e que executa rotinas de TensorFlow, CNTK ou Theano. Foi desenvolvida para permitir a experimentação rápida.

Todas as plataformas acima são de código livre e apresentam características particulares e detalhes de implementação [568]. As principais plataformas como TensorFlow, PyTorch e CNTK oferecem também recursos de cálculo automático do gradiente com base no grafo computacional do modelo. Há também pacotes comerciais para o desenvolvimento de rede neurais profundas, como o Matlab¹⁶² e SAS.¹⁶³

Em 2017, Facebook e Microsoft iniciaram o desenvolvimento de um ecossistema para interoperabilidade de modelos de diferentes plataformas, chamado de *Open Neural Network Exchange* (ONNX).¹⁶⁴ A iniciativa foi seguida por outros *players* como IBM,

Huawei, Intel, entre outros. Atualmente, o ONNX suporta PyTorch, CNTK, Caffé2, MXNet e converte modelos de Keras e TensorFlow.

As plataformas de aprendizado profundo têm bibliotecas que realizam as operações tensoriais necessárias ao processamento de redes neurais profundas de forma eficiente, otimizando os recursos disponíveis em *hardware*. Operações tensoriais podem ser realizadas em *hardware* de alto desempenho como os Tensor Cores de placas NVIDIA¹⁶⁵ e as unidades de processamento tensorial (TPU)* na Google Cloud.¹⁶⁶

Modelos de redes neurais profundas para cada tipo de problema são formados pela conexão de diferentes camadas de processamento. As camadas totalmente conectadas geram um número muito grande de parâmetros. Esta seção apresenta outras topologias de camadas mais adequadas a problemas complexos. Na seção 9.3.1, apresentamos as redes neurais convolucionais e aplicações de visão computacional. Na seção 9.3.2, apresentamos os modelos de redes neurais recorrentes aplicações em problemas de processamento de linguagem natural. Na seção 9.3.3 apresentamos modelos cujo objetivo é extrair uma representação do conjunto de dados, como modelos autocodificadores e modelos geradores.

9.3.1 Redes neurais convolucionais

Numa rede neural totalmente MLP para classificação de imagens, a imagem de entrada é transformada num vetor para ser utilizado como entrada do modelo. Entretanto, a utilização de uma imagem como vetor aumenta muito a dimensionalidade do modelo e destrói estruturas intrínsecas presentes na imagem, que poderiam ser mais bem exploradas pelo modelo. Por exemplo, uma imagem pequena, de tamanho 100×100 pixels, pode ser transformada num vetor com 10 mil elementos. Nesse caso, uma rede neural MLP de classificação binária com apenas 100 unidades na camada intermediária seria ajustada por 1.000.100 parâmetros!

Alguns tipos de dados possuem relações de similaridade, como acontece tipicamente com sons, imagens e textos. Exemplos de relações de similaridade entre vizinhos foram vistos na função de autocorrelação em séries temporais (seção 4.1) e na função de semi variograma (equação (6.43)) para extração de características de imagens.

* *Tensor Processing Units.*

As redes neurais convolucionais (CNN)* são modelos que utilizam filtros para extrair características da entrada através do **operador de convolução**. Filtros de convolução são definidos por um pequeno número de parâmetros que, quando utilizados em modelos de redes neurais, resultam em matrizes de conexão esparsas e no compartilhamento de parâmetros.

9.3.1.1 Operador de convolução

O operador de convolução é semelhante à função de correlação cruzada definida na seção 4.1 (cf. equação (4.3)). O operador de convolução discreto unidimensional entre duas funções $f(t)$ e $g(t)$, denotado $(f * g)(t)$, é um operador linear definido como:

$$(f * g)(t) = \sum_{\tau=-\infty}^{\tau=+\infty} f(\tau)g(t + \tau) \quad (9.86)$$

Em redes convolucionais, o operador de convolução[†] discreto 1D, aplicado entre um vetor $\mathbf{x} = (x_1, \dots, x_n)$ e um **filtro** $\mathbf{w} = (w_1, \dots, w_k)$, resulta no vetor de características $\mathbf{u} = (u_1, \dots, u_{n-k})$, cujos elementos são calculados como:

$$u_t = (\mathbf{x} * \mathbf{w})_t = \sum_{i=1}^k x_{t+i} w_i, \quad t = 1, \dots, n - k \quad (9.87)$$

O resultado da convolução 1D é equivalente ao modelo AR. Considere o exemplo de filtragem de uma série temporal pelo filtro de média móvel com $k = 10$ parâmetros, mostrado na Figura 9.14(a). A Figura 9.14(b) mostra o sinal original, em azul, e o sinal filtrado em vermelho, produzido pela equação (9.87).

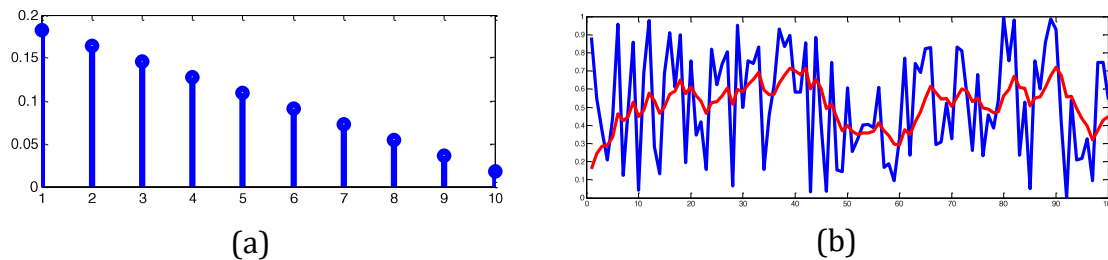


Figura 9.14: Exemplo de convolução 1D: (a) filtro de convolução; (b) sinal de entrada, em azul, e resultado da convolução, em vermelho.

* *Convolutional Neural Network*.

[†] Na literatura de processamento de sinal, o operador de convolução (9.87) é chamado de correlação cruzada em que o deslocamento é realizado sobre a entrada.

Diferentes filtros produzem diferentes características. Um filtro de detecção de borda é mostrado na Figura 9.15. A Figura 9.15(a) apresenta os parâmetros do filtro; na Figura 9.15(b), o sinal de entrada aparece em azul e o resultado do filtro em vermelho.

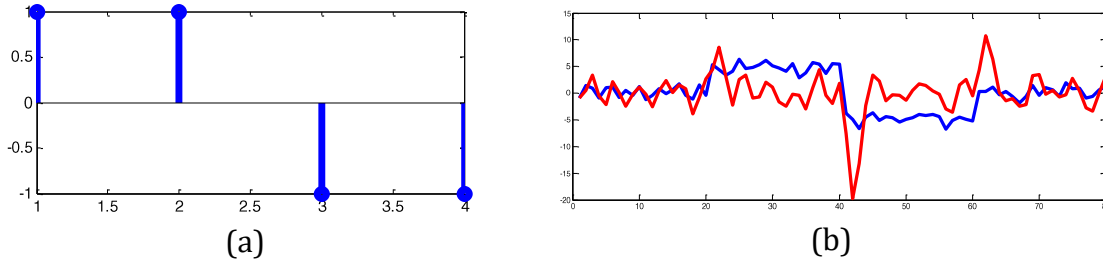


Figura 9.15: Exemplo de filtro de detecção de bordas: (a) parâmetros do filtro; (b) sinal de entrada em azul, resultado do filtro em vermelho.

O operador de convolução pode ser estendido a entradas multidimensionais. No caso de uma imagem 2D em escala de cinza, a entrada $\mathbf{x}$ e o filtro $\mathbf{w}$ são matrizes e a saída $\mathbf{u}$ é calculada como:

$$u_{t_1 t_2} = (\mathbf{x} * \mathbf{w})_{t_1 t_2} = \sum_{i_1=1}^{k_1} \sum_{i_2=1}^{k_2} x_{(t_1+i_1)(t_2+i_2)} w_{i_1 i_2} \quad (9.88)$$

No caso de uma entrada $\mathbb{X}$ e de um filtro $\mathbb{W}$ como tensores de c dimensões, a saída é calculada como:

$$\mathbb{U}_{t_1 t_2 \dots t_c} = (\mathbb{X} * \mathbb{W})_{t_1 t_2 \dots t_c} = \sum_{i_1=1}^{k_1} \sum_{i_2=1}^{k_2} \dots \sum_{i_c=1}^{k_c} x_{(t_1+i_1)(t_2+i_2) \dots (t_c+i_c)} w_{i_1 i_2 \dots i_c} \quad (9.89)$$

A aplicação mais comum de redes neurais convolucionais é o processamento de imagens 2D multicanais, em que uma camada convolucional é processada por um conjunto de filtros aplicados à entrada (cf. Figura 9.16). O TensorFlow oferece a opção de operador de convolução para objetos 3D, que pode realizar a convolução entre uma entrada e um filtro 5D, considerando as dimensões adicionais correspondentes ao número de canais e ao número de registros (no caso de processamento em *batch*).

No processamento de imagens 2D multicanais encontrado nas plataformas de aprendizado profundo, o operador de convolução generalizado é definido pelo seguinte conjunto de parâmetros [147]:

- n_1, n_2 : número de linhas e colunas da entrada;
- m_1, m_2 : número de linhas e colunas da saída;

- k_1, k_2 : número de linhas e colunas do filtro;
- s_1, s_2 : deslocamento em cada direção ou *stride*;
- p_1, p_2 : preenchimento de borda em cada direção ou *padding*.

Numa camada convolucional i , cada filtro tem o mesmo número de canais da entrada, ou da camada anterior, $c^{(i-1)}$. Cada filtro extrai uma característica da entrada e o número de filtros é igual ao número de canais da saída $c^{(i)}$ (cf. Figura 9.16).

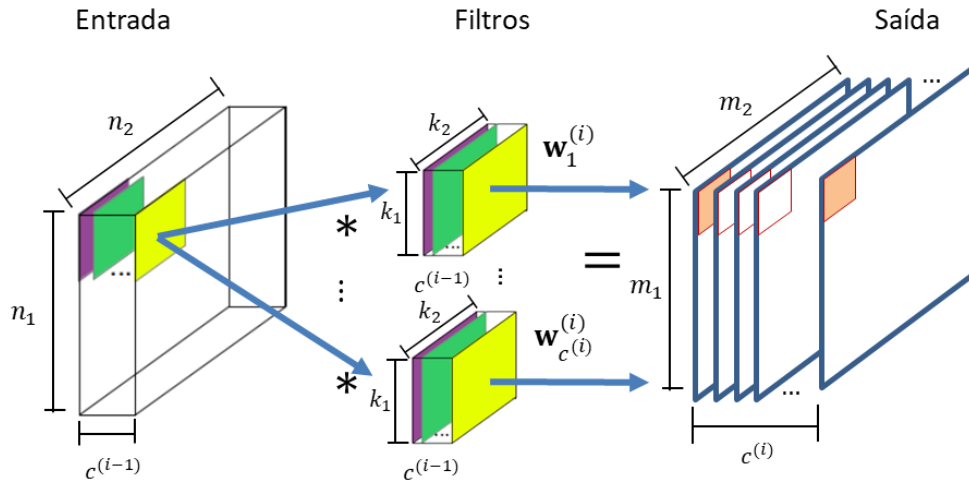


Figura 9.16: Exemplo de $c^{(i)}$ filtros de convolução aplicados a uma entrada de $c^{(i-1)}$ canais, produzindo uma saída de $c^{(i)}$ canais.

A dimensão da saída depende da dimensão da entrada e da escolha dos parâmetros de deslocamento (*stride*) e preenchimento de borda (*padding*). No operador de convolução (cf. equação (9.88)), o incremento é unitário, ou seja, $s_1 = s_2 = 1$, mas as plataformas de aprendizado profundo permitem que o deslocamento seja maior. O preenchimento de borda adiciona um determinado número de pixels, em geral de valor zero (*zero padding*), para reduzir o efeito de borda do filtro (cf. Figura 9.18).

No caso de entradas e filtro quadrados ($n_1 = n_2 = n$ e $k_1 = k_2 = k$), com mesmos deslocamentos e preenchimentos nas duas direções ($s_1 = s_2 = s$ e $p_1 = p_2 = p$), a saída também é quadrada e as dimensões de saída $m_1 = m_2 = m$ são calculadas como:

$$m = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1 \quad (9.90)$$

onde $\lfloor x \rfloor$ é o maior valor inteiro menor que x .

Alguns casos particulares comuns para a escolha do parâmetro de *padding* são:

- *No padding (valid convolution)*: não utiliza preenchimento, ou seja, $p = 0$, e a saída é calculada como $m = \left\lfloor \frac{n-k}{s} \right\rfloor + 1$ (cf. Figura 9.17).
- *Half padding (same convolution)*: o preenchimento é calculado para que a dimensão de saída seja igual à dimensão de entrada, ou seja, $m = n$. No caso em que $s = 1$, $p = \left\lfloor \frac{k}{2} \right\rfloor$ (cf. Figura 9.18).
- *Full padding*: o preenchimento é calculado para que a saída tenha o maior valor possível. Nesse caso, $p = k - 1$ e a saída $m = \left\lfloor \frac{n+k-1}{s} \right\rfloor + 1$ (cf. Figura 9.19).

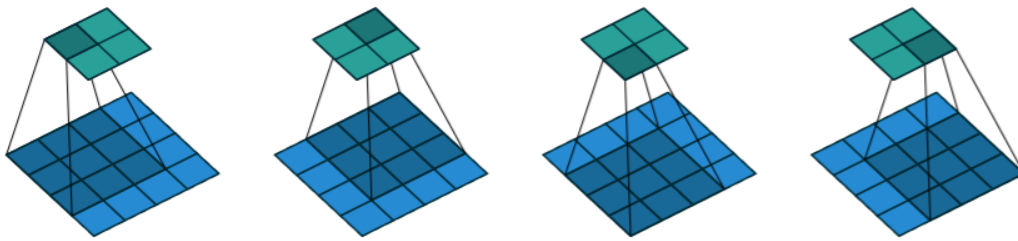


Figura 9.17: Exemplo de convolução sem preenchimento (*no padding* ou *valid convolution*) com $n = 4$, $k = 3$, $s = 1$, $p = 0$. Fonte: reproduzido de [147].

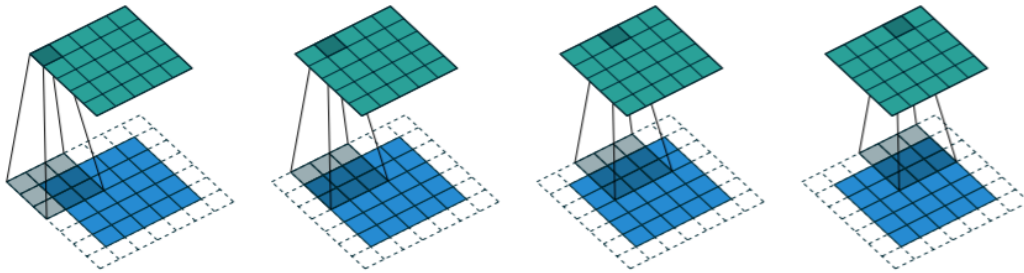


Figura 9.18: Exemplo de convolução com meio preenchimento (*half padding* ou *same convolution*) com $n = 5$, $k = 3$, $s = 1$, $p = 1$. Fonte: reproduzido de [147].

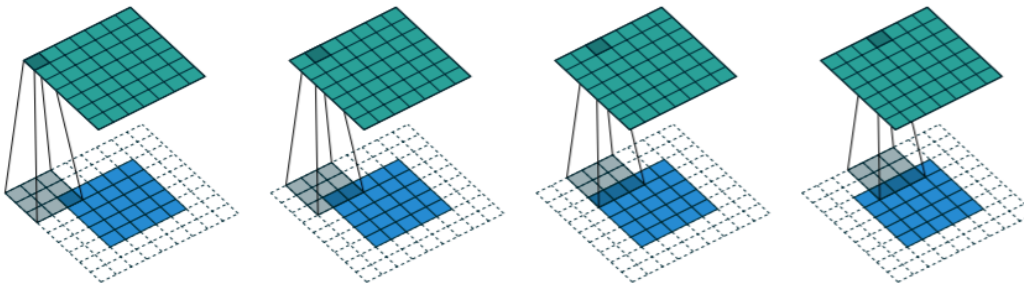


Figura 9.19: Exemplo de convolução com preenchimento completo (*full padding*) com $n = 5$, $k = 3$, $s = 1$, $p = 2$. Fonte: reproduzido de [147].

Grande parte dos operadores convolucionais disponíveis nas plataformas de aprendizado profundo são baseadas no Theano. Há um tutorial¹⁶⁷ sobre os principais operadores disponíveis na plataforma, descritas em [147], com animações intuitivas dos diferentes operadores e parâmetros. Recentemente, novos operadores foram propostos, como a convolução dilatada [583] e a convolução separável [262].

9.3.1.2 Camada convolucional

Uma CNN é uma rede neural em camadas, com uma ou mais camadas convolucionais, que realizam a operação de convolução da entrada com filtros, cujos parâmetros são ajustados pelo algoritmo da retropropagação. Um conjunto de camadas convolucionais permite a extração de características da entrada em diferentes níveis de abstração, uma vez que o filtro de uma camada i extrai características da camada $i - 1$.

O operador convolucional é linear. Independentemente das suas dimensões, o tensor de entrada pode ser convertido num vetor e a operação de convolução pode ser calculada vetorialmente como:

$$\mathbf{u} = \mathbf{x}\mathbf{W} \tag{9.91}$$

A matriz $\mathbf{W}$ na equação (9.91) é uma matriz esparsa chamada de matriz de Toeplitz, que contém apenas os parâmetros do filtro que se repetem ao longo da diagonal. Esse **compartilhamento de parâmetros** é uma característica importante das redes neurais convolucionais.

A Figura 9.20a mostra um exemplo da matriz de conexões no caso de uma entrada de dimensão $n = 100$ com um filtro de $k = 10$ parâmetros (cf. Figura 9.14). A matriz é muito esparsa, a área branca da figura representa as posições da matriz com valor zero e os graus de cinza, o valor dos parâmetros, que se repetem na diagonal. Para efeito de comparação, a Figura 9.20(b) mostra uma matriz de conexão entre duas camadas totalmente conectadas com 100 unidades cada, resultando em 10 mil parâmetros.

A apresentação do operador de convolução como uma multiplicação matricial é elegante matematicamente para descrever a rede convolucional como uma topologia em camadas. Entretanto, a implementação do operador convolucional como uma multiplicação de um vetor por uma matriz esparsa é ineficiente computacionalmente e, na prática, as bibliotecas utilizam algoritmos mais eficientes que exploram recursos de processamento massivamente paralelo disponíveis no *hardware*.

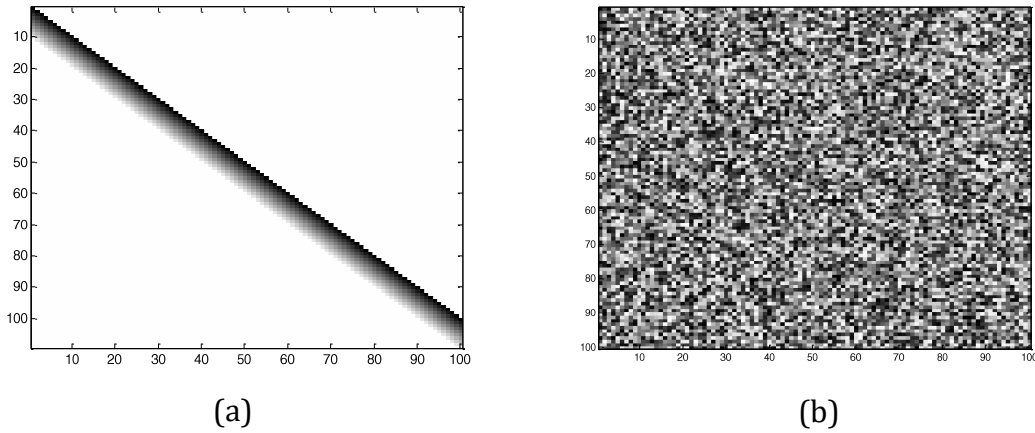


Figura 9.20: Exemplo de matriz de conexões: (a) redes neurais convolucionais, com parâmetros repetidos na diagonal (CNN); (b) redes neurais totalmente conectadas (MLP).

Uma camada convolucional realiza o processamento de um conjunto de características em paralelo a partir de uma mesma entrada, em que cada característica é extraída por um filtro diferente, como mostra a Figura 9.21(a). Cada filtro é representado por uma matriz esparsa, e o resultado da convolução com a entrada é calculado como uma rede neural (cf. equação (9.91)). A camada convolucional pode ser representada como uma caixa, como a Figura 9.21(b), onde $\mathbf{u}^{(i-1)}$ ($\mathbb{u}^{(i-1)}$) é o vetor (tensor) de características de entrada e $\mathbf{u}^{(i)}$ ($\mathbb{u}^{(i)}$) é o vetor (tensor) de características calculado na camada.

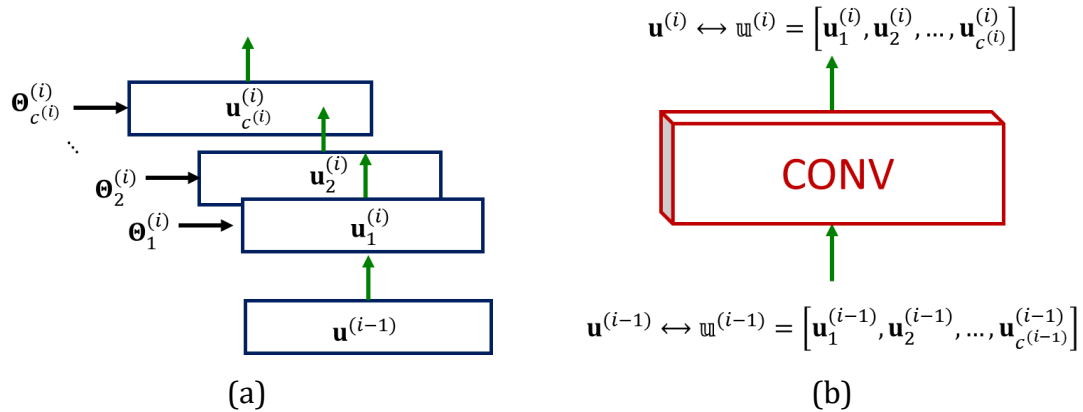


Figura 9.21: Representação da camada convolucional: (a) filtros em paralelo; (b) representação esquemática.

O processamento *online* da saída de cada filtro (canal de saída) de uma camada convolucional é calculado como (cf. equação (9.22)):

$$\mathbf{u}_j^{(i)} = \mathbf{h}_j^{(i)}(\mathbf{z}_j^{(i)}) = \mathbf{h}_j^{(i)}(\mathbf{b}_j^{(i)} + \mathbf{u}^{(i-1)}\mathbf{w}_j^{(i)}), \quad j = 1, \dots, c^{(i)} \quad (9.92)$$

onde $\mathbf{W}_j^{(i)}$ é a matriz correspondente ao filtro $\mathbf{w}_j^{(i)}$; $\mathbf{u}^{(i-1)}$ é a representação vetorial do tensor de entrada $\mathbb{U}^{(i-1)} = [\mathbf{u}_1^{(i-1)}, \mathbf{u}_2^{(i-1)}, \dots, \mathbf{u}_{c^{(i-1)}}^{(i-1)}]$; $\mathbf{u}^{(i)}$ é a representação vetorial do tensor de saída $\mathbb{U}^{(i)} = [\mathbf{u}_1^{(i)}, \mathbf{u}_2^{(i)}, \dots, \mathbf{u}_{c^{(i)}}^{(i)}]$, em que $\mathbf{u}_j^{(i)}$ é a saída do filtro $\mathbf{w}_j^{(i)}$, e $\mathbf{h}_j^{(i)}$ é a função de ativação do filtro $\mathbf{w}_j^{(i)}$.

No caso do processamento de imagens 2D multicanais (cf. Figura 9.16), para cada camada, o vetor de entrada $\mathbf{u}^{(i-1)}$ tem dimensão $n^{(i-1)} = n_1^{(i-1)} \times n_2^{(i-1)} \times c^{(i-1)}$ e a dimensão do vetor de saída da camada convolucional é $\mathbf{u}^{(i)}$ e $n^{(i)} = n_1^{(i)} \times n_2^{(i)} \times c^{(i)}$.

Analogamente, no processamento em *batch* (ou *mini-batch*) de N registros, cada filtro (canal de saída) de uma camada convolucional é calculado como (cf. equação (9.23)):

$$\mathbf{U}_j^{(i)} = \mathbf{H}_j^{(i)}(\mathbf{Z}_j^{(i)}) = \mathbf{H}_j^{(i)}(\mathbf{1}^T \mathbf{b}_j^{(i)} + \mathbf{U}^{(i-1)} \mathbf{W}_j^{(i)}), \quad j = 1, \dots, c^{(i)} \quad (9.93)$$

onde $\mathbf{U}^{(i-1)}$ é a matriz de entrada de dim $N \times n^{(i-1)}$ (tensor $\mathbb{U}^{(i-1)}$), $\mathbf{H}_j^{(i)}$ é a versão matricial da função de ativação e $\mathbf{U}^{(i)}$ é a matriz de saída de dim $N \times n^{(i)}$ (tensor $\mathbb{U}^{(i)}$).

A representação da entrada e da saída de uma camada convolucional como um vetor é coerente com o conceito de grafo computacional e de encapsulamento de cada camada no modelo. Essa representação permite que uma camada convolucional seja conectada a uma camada de outro tipo, como uma camada totalmente conectada, como é comum em modelos de redes neurais convolucionais.

Uma CNN tem várias camadas convolucionais e cada camada representa características mais abstratas da entrada. Essa hierarquia de níveis de abstração é observada em diversos problemas de percepção. No caso de processamento de imagens, a partir do *pixel*, são formadas arestas, texturas, partes do objeto até o objeto completo. No caso de processamento de linguagem natural, a partir de caracteres, formam-se palavras, frases até documentos. No problema de reconhecimento de fala, a partir do sinal sonoro, formam-se fonemas, palavras e frases.

O compartilhamento de parâmetros em cada filtro de uma camada convolucional permite que a característica representada pelo filtro apareça em qualquer ponto da entrada. Por exemplo, se um filtro é ativado pela presença de um número na imagem, é

importante que possa detectar o número em qualquer lugar da imagem. Essa propriedade é chamada de **equivariância à translação*** [212].

Em problemas de visão computacional, é importante também que o modelo seja robusto a pequenas rotações do objeto. No exemplo do filtro ativado por um número, o filtro deve ser ativado se o número aparecer ligeiramente rotacionado. Essa propriedade é chamada de **invariância à rotação**† [212] e é obtida por uma camada adicional chamada de *pooling*.

A camada de **pooling** funciona de forma similar à camada convolucional, através de uma função de *pooling*, que é calculada como um filtro convolucional. Para facilitar a apresentação, considere o exemplo de uma entrada 1D $\mathbf{x} = (x_1, \dots, x_n)$ e um filtro de k elementos. As funções de *pooling* mais utilizadas são:

- *Max-pooling*: a saída é o valor máximo da entrada:

$$u_t = \max_{i=1, \dots, k} (x_{t+i}) \quad (9.94)$$

- *Average-pooling*: a saída é média da entrada:

$$u_t = \frac{1}{k} \sum_{i=1}^k x_{t+i} \quad (9.95)$$

- *L²-pooling*: a saída é norma L^2 da entrada ao quadrado:

$$u_t = \|(x_{t+1}, \dots, x_{t+k})\|^2 \quad (9.96)$$

A função de *pooling* pode ser calculada como uma camada convolucional, em que os parâmetros do filtro são calculados por uma função de *pooling*:

$$\widehat{\mathbf{w}}_j^{(i)} = \text{pool}(\mathbf{u}_j^{(i-1)}) \quad (9.97)$$

A camada de *pooling* não utiliza função de ativação e não precisa de unidade bias. Dessa forma, pode ser calculada, para cada filtro, como:

$$\mathbf{u}_j^{(i)} = \mathbf{u}_j^{(i-1)} \widehat{\mathbf{w}}_j^{(i)} \quad (9.98)$$

A maioria das plataformas utiliza os mesmos parâmetros do operador de convolução como *stride* e *padding* para a camada de *pooling*. A função de *max-pooling* é a mais utilizada, sem padding e com parâmetros $s = k = 2$, de forma que a saída é a metade da

* Um operador de translação f é equivariante à translação se $(f(\mathbf{x}) * \mathbf{w}) = f(\mathbf{x} * \mathbf{w})$.

† Um operador de rotação f é invariante à rotação se $(f(\mathbf{x}) * \mathbf{w}) = (\mathbf{x} * \mathbf{w})$.

entrada, ou seja, $m = n/2$. Em geral, como não tem parâmetros ajustáveis, a camada de *pooling* não incrementa o índice das camadas, sendo considerada em conjunto com a camada convolucional.

Algumas topologias de CNN utilizam algum tipo de padronização nas camadas convolucionais para melhorar o ajuste de parâmetros. A normalização por resposta local (LRN)* [537] realiza uma padronização de forma independente para cada unidade em relação aos canais da entrada. A normalização por *batch* (BN)[†] [275] aplica uma padronização *z-score* (cf. equação (2.16)) em relação ao *batch* de treinamento, antes da função de ativação. Após o ajuste de parâmetros, o *z-score* é calculado em relação a todo o conjunto de dados de treinamento e fixo no modelo. A normalização LRN é específica de redes convolucionais, mas a normalização por *batch* (BN) também pode ser utilizada em redes totalmente conectadas.

9.3.1.3 Ajuste de parâmetros

Em cada camada convolucional, diferentes características são extraídas pelos filtros, cujos parâmetros são ajustados pelo algoritmo da retropropagação. A rede convolucional é, assim, um modelo flexível e computacionalmente eficiente que pode ser utilizado em diferentes aplicações.

O ajuste de parâmetros de uma camada convolucional é calculado, para cada filtro, de forma semelhante a uma camada totalmente conectada (cf. equações (9.54) e (9.55)):

$$\Delta \mathbf{W}_j^{(i)} = -\eta \mathbf{u}^{(i-1)T} \boldsymbol{\delta}_j^{(i)} \quad (9.99)$$

$$\Delta \mathbf{b}_j^{(i)} = -\eta \boldsymbol{\delta}_j^{(i)} \quad (9.100)$$

onde $\boldsymbol{\delta}_j^{(i)} = \boldsymbol{\varepsilon}_j^{(i)} \odot \mathbf{h}'_j^{(i)}$ é o gradiente local da saída $\mathbf{u}_j^{(i)}$; $\mathbf{h}'_j^{(i)}$ é a derivada da função de ativação correspondente; e $\boldsymbol{\varepsilon}_j^{(i)}$ é a componente do erro retropropagado $\boldsymbol{\varepsilon}^{(i)}$, que é a representação vetorial do tensor de erro retropropagado $\mathfrak{e}^{(i)} = [\boldsymbol{\varepsilon}_1^{(i)}, \boldsymbol{\varepsilon}_2^{(i)}, \dots, \boldsymbol{\varepsilon}_{c^{(i)}}^{(i)}]$.

Numa camada convolucional, a mesma entrada está conectada a vários filtros. Dessa forma, o erro retropropagado é calculado pela soma da retropropagação realizada por cada filtro (cf. equação (9.56)):

* *Local Response Normalization.*

† *Batch Normalization.*

$$\boldsymbol{\varepsilon}^{(i-1)} = \sum_{j=1}^{c^{(i)}} \boldsymbol{\delta}_j^{(i)} \mathbf{w}_j^{(i)T} \quad (9.101)$$

onde $\boldsymbol{\delta}_j^{(i)}$ é o gradiente local da saída $\mathbf{u}_j^{(i)}$ e $\boldsymbol{\varepsilon}^{(i-1)}$, de dimensão $n^{(i-1)}$, é a representação vetorial do tensor de erro retropropagado $\boldsymbol{\varepsilon}^{(i-1)} = [\boldsymbol{\varepsilon}_1^{(i-1)}, \boldsymbol{\varepsilon}_2^{(i-1)}, \dots, \boldsymbol{\varepsilon}_{c^{(i-1)}}^{(i-1)}]$.

A Figura 9.22 ilustra a retropropagação do erro numa camada convolucional. A componente de erro retropropagado correspondente a cada filtro é utilizada para o cálculo do gradiente local e do ajuste de parâmetros (cf. equações (9.99) e (9.100)). O erro retropropagado para a camada anterior é calculado pela soma das componentes correspondentes de cada filtro pela equação (9.101).

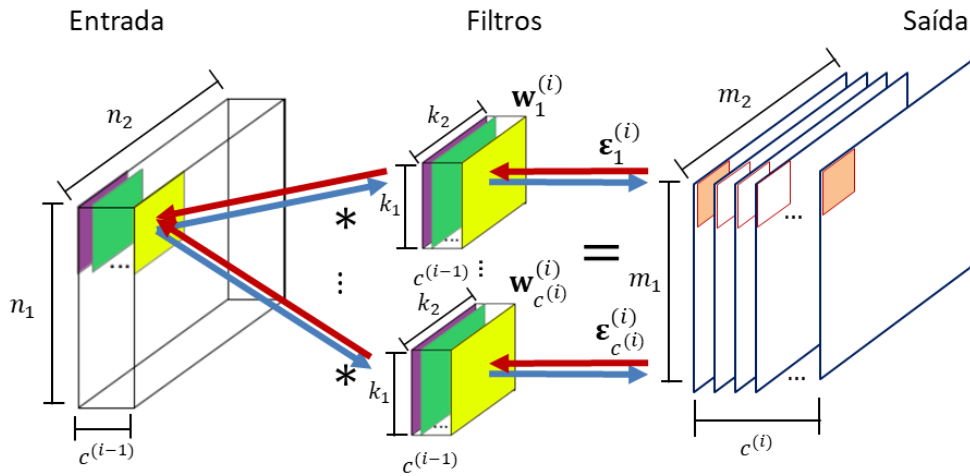


Figura 9.22: Retropropagação do erro numa camada convolucional.

O ajuste de parâmetros calculado pelas equações (9.99) e (9.100) resulta em valores diferentes para os parâmetros do filtro (cf. Figura 9.20(a)). Para manter o compartilhamento de parâmetros, os parâmetros de cada filtro são reajustados pela média dos resultados calculados pelo algoritmo da retropropagação:

$$\Delta \mathbf{w}_j^{(i)} = \frac{1}{K} \sum_{k=1}^K \Delta \mathbf{w}_{jk}^{(i)}, \quad j = 1, \dots, c^{(i)} \quad (9.102)$$

onde K é o número de repetições dos parâmetros em cada filtro na matriz de conexões (cf. Figura 9.20).

As plataformas de *deep learning* têm bibliotecas que realizam de forma eficiente o processamento e ajuste de parâmetros de modelos usuais de redes neurais. Além disso, também possuem recursos de diferenciação automática, como Autograd¹⁶⁸ no PyTorch

e GradientTape¹⁶⁹ no TensorFlow que calculam automaticamente a derivada de uma função programada pelo usuário. No Autograd, a plataforma registra a sequência de comandos para o cálculo do valor de saída a partir da entrada e calcula automaticamente o valor da derivada. O GradientTape gera a sequência de comandos para o cálculo da derivada, de forma que derivadas de ordem superior podem ser calculadas a partir do código gerado pelo sistema. O cálculo automático de derivadas é um assunto muito explorado na literatura com diversas abordagens desenvolvidas [367].

Redes convolucionais têm aplicações em diversas áreas, mas têm sido mais utilizadas em aplicações visão computacional. Um resumo dos principais modelos de classificação de imagens é apresentado a seguir.

9.3.1.4 Aplicações em classificação de imagens

Os modelos de redes neurais profundas começaram a apresentar bons resultados em problemas complexos a partir da construção de grandes bases de dados anotadas. Atualmente, há vários conjuntos de dados anotados em diversas aplicações disponíveis para o desenvolvimento de modelos e algoritmos.¹⁷⁰

Um dos conjuntos de dados mais utilizados em classificação de imagens é o conjunto de dígitos manuscritos MNIST. O conjunto de dados tem 60 mil registros de treinamento e 10 mil registros de testes e está disponível na maioria das plataformas. O MNIST é utilizado como exemplo em plataformas como o TensorFlow¹⁷¹ e CNTK,¹⁷² além de tutoriais na internet.¹⁷³

O MNIST foi utilizado para desenvolver uma das primeiras redes neurais convolucionais na década de 1990 por LeCun e seu grupo [332]. O modelo, chamado de **LeNet-5**, é a rede convolucional de cinco camadas mostrada na Figura 9.23. A partir da imagem de entrada de tamanho 32×32 em escala de cinza ($c^{(0)} = 1$), a primeira camada contém $c^{(1)} = 6$ filtros convolucionais de tamanho 5×5 , que resulta em 6 canais de dimensão 28×28 . A seguir, uma camada de *pooling* que reduz a dimensionalidade para $6 \times 14 \times 14$, de forma que a saída da primeira camada tem $n^{(1)} = 1.176$ unidades. A segunda camada repete o processamento da primeira, com uma camada convolucional seguida de uma camada de *pooling*, produzindo uma saída de dimensão $16 \times 5 \times 5$ com $n^{(2)} = 400$ unidades. As camadas seguintes são camadas totalmente conectadas (MLP) com, respectivamente, $n^{(3)} = 120$ e $n^{(4)} = 84$ unidades. Todas as camadas utilizam função de ativação tangente hiperbólica. A camada de saída tem $n^{(5)} = 10$ unidades,

que correspondem ao número de classes (dígitos), com função de ativação *softmax*. O modelo obtém bons resultados no conjunto MNIST, sendo robusto a perturbações como ruídos, rotações e deformações.¹⁷⁴

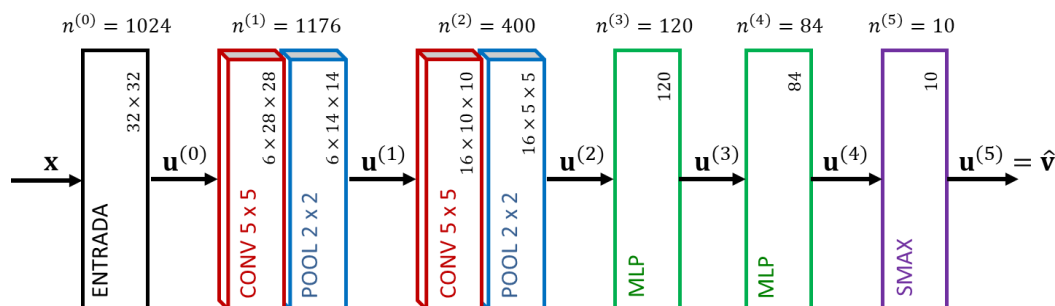


Figura 9.23: Topologia do modelo LeNet-5. Fonte: adaptado de [332]

O ajuste de parâmetros de modelos de redes neurais profundas pelo método do gradiente requer uma grande quantidade de dados anotados. Em 2008, um conjunto de 80 milhões de pequenas imagens¹⁷⁵ coloridas ($3 \times 32 \times 32$), foi coletado por grupos de pesquisa do MIT e NYU [527] a partir do banco de dados léxico Wordnet¹⁷⁶ [385] e buscas na internet. Um subconjunto dessas imagens foi anotado no Instituto Canadense de Pesquisa Avançada (CIFAR)* e disponibilizado em dois conjuntos de dados,¹⁷⁷ CIFAR-10 e CIFAR-100. Os dois conjuntos contêm as mesmas 60 mil imagens coloridas, sendo 50 mil para treinamento e 10 mil para teste. O CIFAR-10 tem 10 classes (6 mil registros para cada classe) e o CIFAR-100, 100 classes (600 registros para cada classe) agrupadas em 20 categorias (superclasses).

Em 2010, pesquisadores de Stanford e Princeton organizaram o desafio ImageNet de reconhecimento visual de grande escala (ILSVRC)[†] [465]. O problema de classificação de imagens do primeiro desafio se tornou um *benchmark* muito utilizado para comparação de algoritmos de classificação de imagens. As imagens foram coletadas na internet a partir da Wordnet e anotadas utilizando o *Amazon Mechanical Turk*,¹⁷⁸ que distribui na internet pequenos serviços que possam ser realizados e remunerados *online*.

A competição de 2010 foi um problema de classificação de imagens com um conjunto de treinamento de mais de 1,2 milhão de imagens coloridas ($3 \times 224 \times 224$) anotadas em mil categorias em dois níveis de classificação, como no CIFAR-100. Há, por exemplo,

* Canadian Institute for Advanced Research.

† ImageNet Large Scale Visual Recognition Challenge.

120 raças de cachorro. O conjunto de validação contém 50 mil imagens e o conjunto de testes, 100 mil imagens [465]. Os modelos são avaliados de duas formas: o Top-1, em que apenas o valor de maior probabilidade da saída é avaliado, e o Top-5, em que os cinco primeiros valores de saída são comparados com o valor anotado.

A partir de 2011, além do problema de classificação de imagens, foi proposto também o problema de localização, em que o algoritmo deve estimar as coordenadas de uma caixa que contém o objeto que define a classe da imagem. Essas coordenadas também são fornecidas no conjunto de treinamento. A partir de 2013, foi proposto o problema de detecção de múltiplos objetos na imagem, semelhante ao problema de localização, mas com diversos objetos na mesma imagem [465].

Em 2012, o vencedor nas duas categorias popularizou os modelos CNN em problemas de visão computacional. O modelo passou a ser chamado de **AlexNet** em função do autor principal Alex Krizhevsky, mas foi submetido com o nome de SuperVision. O resultado do modelo no conjunto de testes obteve 16,4% de erro no Top-5, bem melhor que o segundo colocado, com 25,8% erro, que representava o estado da arte da área de visão computacional na época [324][465].

O AlexNet tem uma arquitetura semelhante à LeNet-5, porém com mais camadas. O modelo introduziu alguns das principais técnicas para o treinamento de redes neurais profundas discutidas na seção 9.2.3, como a normalização LRN e Dropout. O AlexNet também utilizou a técnicas de aumento do número de registros do conjunto de treinamento através de transformações nas imagens como corte e espelhamento [324].

A Figura 9.24 mostra a topologia do modelo AlexNet e as dimensões das camadas. A primeira camada é uma camada convolucional com 96 filtros de dimensão 11×11 , com stride $s = 4$, seguida por uma camada de *pooling*, como no modelo LeNet-5. A segunda camada é um bloco de normalização LNR, 256 filtros de convolução 5×5 e *pooling*. No bloco seguinte, com três camadas com 384 filtros de convolução 3×3 são posicionadas entre a camada de normalização LNR e a camada de *pooling*. No final, duas camadas totalmente conectadas (MLP) de 4.096 unidades e a camada de saída *softmax* com mil unidades. O modelo AlexNet tem mais de 60 milhões de parâmetros.

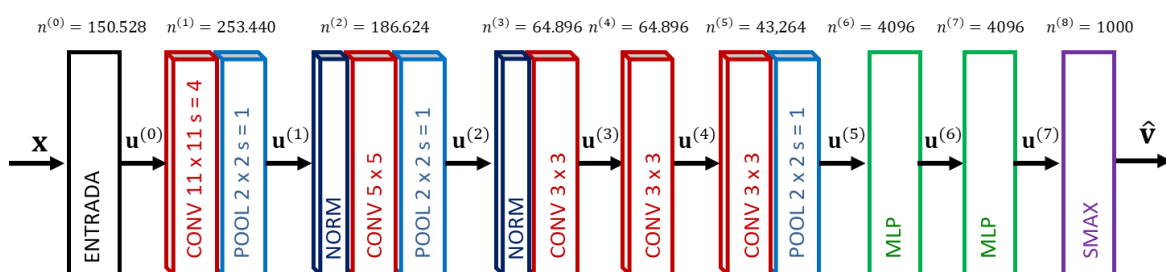


Figura 9.24: Topologia do modelo AlexNet. Fonte: adaptado de [324].

O artigo que descreve o modelo AlexNet foi publicado em 2012 [324]. Na época, não havia bibliotecas para processamento de redes neurais profundas em GPU como a cuDNN,¹⁷⁹ de 2015. O código do modelo original, com processamento em duas placas GPU, em linguagem CUDA 3.2, está disponível no *site* do autor.¹⁸⁰

Alguns resultados do modelo AlexNet são mostrados na Figura 9.25a, onde o resultado anotado aparece abaixo de cada figura e as respostas sugeridas no Top-5 são mostradas no quadro subsequente. A barra azul representa a probabilidade *softmax* e a barra é assinalada em vermelho quando uma das cinco saídas sugeridas do modelo é correta. A linha superior da Figura 9.25 apresenta exemplos de acertos e a linha inferior, exemplos de erros, em que o resultado do modelo é bastante plausível.

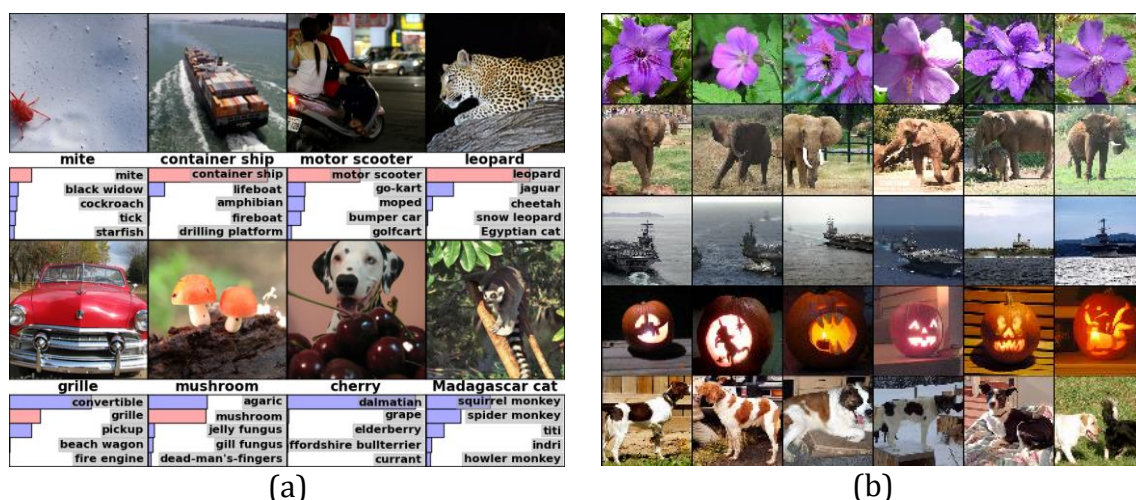


Figura 9.25: Resultados do modelo AlexNet (a) classificação de imagens;

(b) recuperação de imagens. Fonte: *site* do autor.¹⁸¹

O modelo AlexNet também pode ser utilizado para recuperação de imagens por similaridade, utilizando a última camada do modelo, com $c^{(7)} = 4.096$ unidades, como vetor de características. A primeira coluna da Figura 9.25b apresenta as imagens consultadas e as demais colunas, as imagens mais similares encontradas.

Um número maior de camadas convolucionais melhora o desempenho do modelo, mas também aumenta o número de parâmetros. O modelo **VGG** é uma rede convolucional com 19 camadas, formada por blocos de camadas de convolução em sequência com filtros pequenos e uma camada de *pooling*. As camadas de convolução em sequência aumentam o **campo receptivo**, que é a área de entrada capturada pelo filtro, o que permite a extração de características complexas com filtros pequenos [492]. Os autores avaliaram modelos com 11, 13, 16 e 19 camadas. O modelo de 19 camadas tem 144M de parâmetros e ficou em segundo lugar no problema de classificação de imagens do ILSVRC em 2014, com 7,5% de erro no Top-5. O modelo VGG-19 está disponível no GitHub¹⁸² para a plataforma Caffe.

O vencedor do ILSVRC de 2014 foi o modelo chamado **GoogLeNet** [513], com erro de 6,67% no Top-5 do problema de classificação de imagens. A inovação do GoogLeNet foi o módulo **Inception**, mostrado na Figura 9.26, um bloco que distribui a entrada por diferentes filtros convolucionais. Os filtros são de tamanhos diferentes, mas produzem características de mesma dimensão, relacionadas com porções diferentes da entrada, que são concatenadas nas saídas na dimensão dos canais.

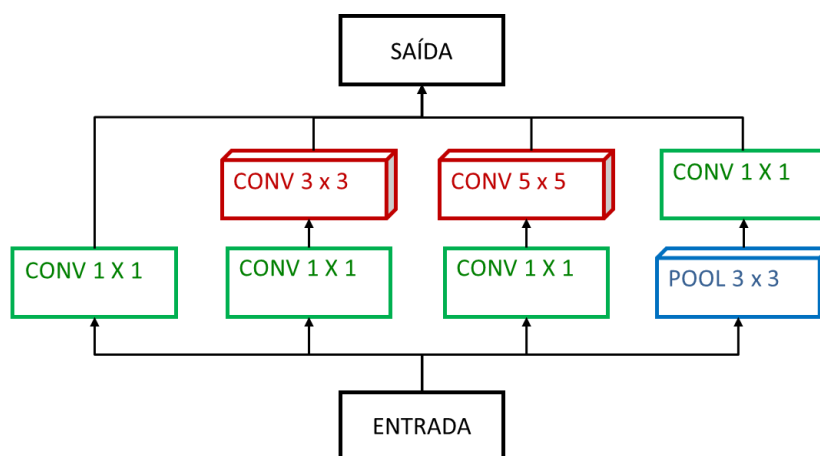


Figura 9.26: Módulo Inception. Fonte: adaptado de [513].

O módulo Inception inclui a convolução 1×1 , que reduz drasticamente o número de parâmetros na rede (4M, comparado a 60M da AlexNet). A convolução 1×1 funciona, para cada unidade da entrada, como uma rede totalmente conectada aplicada aos canais da unidade correspondente. Dessa forma, a convolução 1×1 não altera o tamanho da entrada, apenas o número de canais.

Considere uma entrada $\mathbf{u}^{(i-1)}$ de dimensão $n^{(i-1)} = n_1^{(i-1)} \times n_2^{(i-1)} \times c^{(i-1)}$, transformada na matriz $\hat{\mathbf{U}}^{(i-1)}$ de dimensão $N^{(i-1)} \times c^{(i-1)}$, com $N^{(i-1)} = n_1^{(i-1)} \times n_2^{(i-1)}$. A saída da convolução 1×1 é como uma camada MLP em modo *batch* (cf. equação (9.23)):

$$\hat{\mathbf{U}}^{(i)} = \mathbf{H}^{(i)}(\mathbf{Z}^{(i)}) = \mathbf{H}^{(i)}(\mathbf{1}^T \mathbf{b}^{(i)} + \hat{\mathbf{U}}^{(i-1)} \mathbf{W}^{(i)}) \quad (9.103)$$

onde a matriz de conexões $\mathbf{W}^{(i)}$ é uma matriz cheia de dimensão $c^{(i-1)} \times c^{(i)}$; o vetor de parâmetros da unidade bias $\mathbf{b}^{(i)}$ tem dimensão $c^{(i)}$; e a saída $\hat{\mathbf{U}}^{(i)}$ tem dimensão $N^{(i)} \times c^{(i)}$, onde $N^{(i)} = N^{(i-1)} = n_1^{(i-1)} \times n_2^{(i-1)}$.

O modelo GoogLeNet é mostrado na Figura 9.27. O bloco de entrada é formado por uma camada de convolução de 64 filtros 7×7 , *stride* 2, seguida por uma camada de *max pooling*. Depois, uma camada de convolução com 192 filtros 3×3 , *stride* 2, entre duas camadas de normalização LNR. Seguem três blocos com três módulos Inception como da Figura 9.26 cada. No final, uma camada de *pooling* pela média, uma camada totalmente conectada de 1,024 unidades e a camada de saída *softmax* com mil unidades. Na saída do primeiro e do segundo bloco Inception, é adicionada uma sequência de saída formada por *pooling* pela média, duas camadas totalmente conectadas e uma camada *softmax*. Em 2015, outras topologias do tipo Inception foram propostas [514]. O modelo Inception v4 implementado na plataforma Keras está disponível no GitHub.¹⁸³

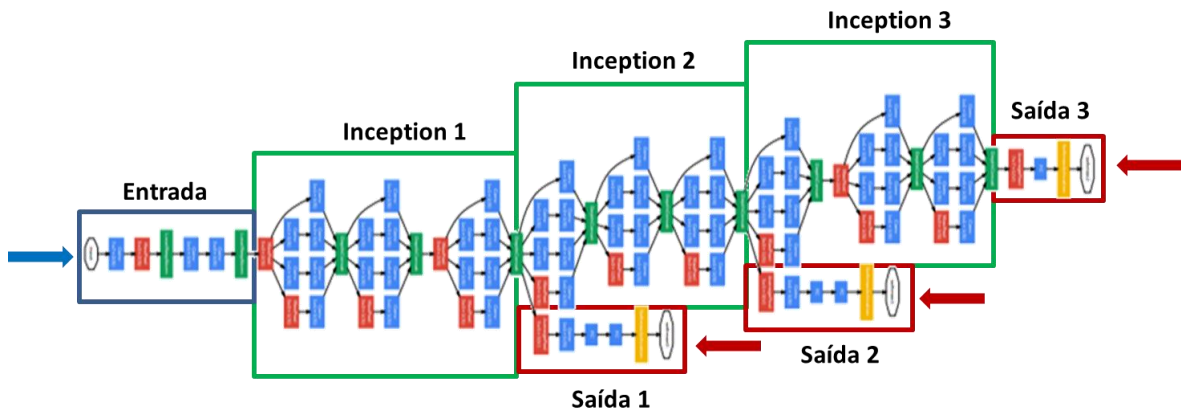


Figura 9.27: Modelo GoogLeNet. Fonte: adaptado de [513].

Há um limite para o ganho de desempenho com o aumento do número de camadas convolucionais devido ao efeito de diluição do gradiente no algoritmo da retropropagação. O modelo GoogLeNet reduz esse efeito com camadas de saída adicionais conectadas diretamente às camadas inferiores. O modelo ResNet [242], proposto em 2015

por pesquisadores da Microsoft, utiliza uma ligação direta em cada bloco de camadas da rede, com um módulo de resíduo.

A ideia do módulo de resíduo é ilustrada na Figura 9.28. Na Figura 9.28a um exemplo de camadas convolucionais usuais, encontradas em redes como AlexNet e VGG. A partir da entrada $\mathbf{x}$, cada camada de convolução é seguida por uma função de ativação ReLU para o processamento da saída $\mathbf{y}$. No módulo de resíduo, mostrado na Figura 9.28b, uma conexão direta da entrada é adicionada à saída da segunda camada de filtros, antes da função de ativação:

$$\mathbf{y} = \mathbf{h}(\mathbf{f}(\mathbf{x})) = \mathbf{h}(\mathbf{g}(\mathbf{x}) + \mathbf{x}) \quad (9.104)$$

onde as unidades bias e os índices foram retirados para simplificar a notação [242].

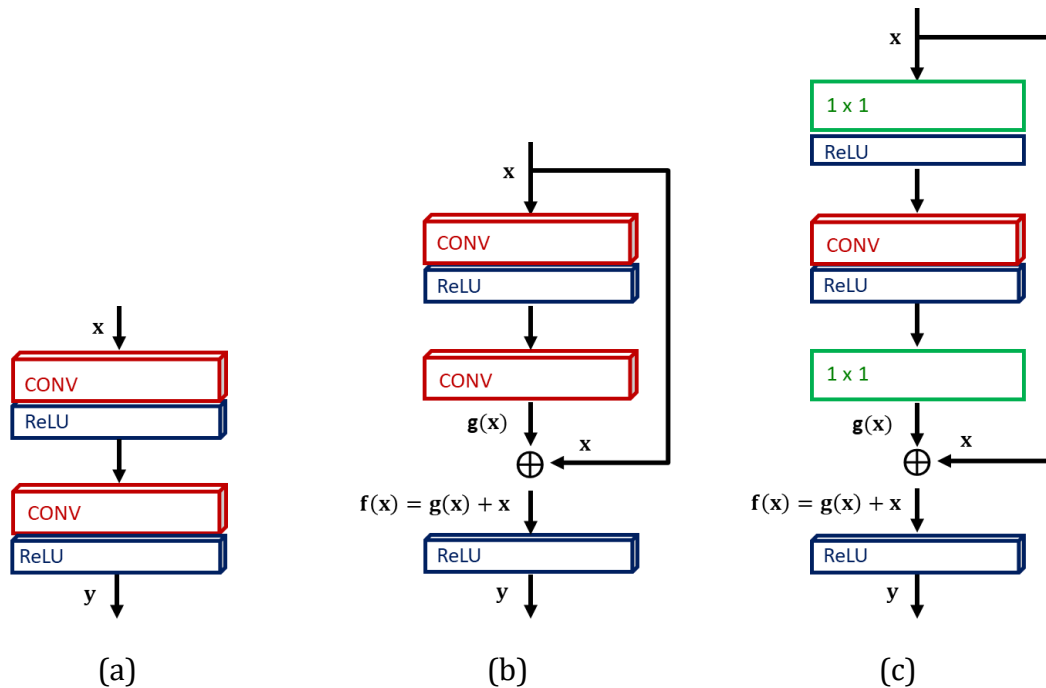


Figura 9.28: Módulos da rede ResNet: (a) rede convolucional comum (AlexNet, VGG); (b) módulo de resíduo; (c) módulo de resíduo com gargalo. Fonte: adaptado de [242].

A função a ser ajustada pelos filtros na camada é o resíduo do resultado da camada anterior, ou seja: $\mathbf{g}(\mathbf{x}) = \mathbf{f}(\mathbf{x}) - \mathbf{x}$. A rede ResNet é uma rede convolucional com vários módulos de resíduo em sequência. Nas redes maiores, é utilizado o módulo de resíduo com gargalo, mostrado na Figura 9.28c, em que camadas de convolução 1×1 são incluídas antes e depois da camada convolucional para reduzir o número de canais e, em seguida, aumentar novamente. Esse tipo de módulo reduz o número de parâmetros de

redes muito profundas. O módulo de resíduo com gargalo foi testado em redes de 50, 101 e 152 camadas [242]. Posteriormente, os autores testaram outras variantes, incluindo a padronização por *batch* e função de ativação linear na saída do módulo de resíduo, chegando a mil camadas [243]. Outras variantes do módulo resíduo, como o SEnet, têm sido estudadas [265].

O modelo ResNet foi o vencedor do ILSVRC de 2015 e da competição do conjunto de dados COCO¹⁸⁴ organizado pela Microsoft [344]. No problema de classificação de imagens do ILSVRC, um *ensemble* de modelos ResNet com 152 camadas obteve 3,5% de erro no Top-5. Esse resultado é melhor que a média do resultado obtido por seres humanos e, por isso, é chamado de desempenho super-humano. O modelo pré-treinado desenvolvido pelos autores está disponível na plataforma Caffe¹⁸⁵ e versões em Torch¹⁸⁶ e TensorFlow¹⁸⁷ também podem ser encontradas no GitHub.

A comparação de modelos em competições como ILSVRC é feita apenas em função da sua precisão e diversas aplicações de visão computacional têm recursos limitados como, por exemplo, sistemas embarcados. Recentemente, os principais modelos foram analisados em termos de precisão, uso de memória, quantidade de parâmetros e operações, tempo de inferência e consumo de energia [73]. Um dos resultados é reproduzido na Figura 9.29, que apresenta uma comparação dos modelos discutidos anteriormente e sintetiza a história recente de modelos de redes neurais profundas.

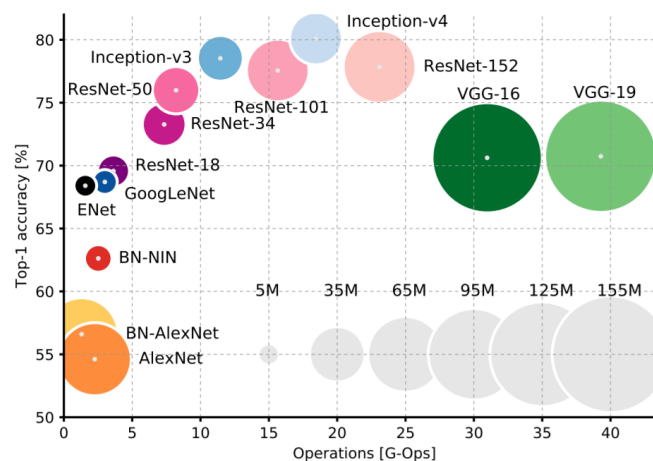


Figura 9.29: Comparação dos modelos de CNN no conjunto ImageNet. Fonte: reproduzido de [73].

A Figura 9.29 mostra a acurácia Top-1 de cada modelo no problema de classificação de imagens do ILSVRC em função do número de operações para o processamento de um

registro na rede. A área do círculo representa o número de parâmetros. Observe a diferença entre o modelo VGG-19 com 144M de parâmetros (em verde) e o GoogLeNet com 4M de parâmetros (em azul), que têm desempenho similar. Os modelos mais recentes buscam equilibrar o desempenho e o número de parâmetros.

Um ponto interessante na Figura 9.29 é que os modelos que realizam mais operações não obtêm necessariamente melhor desempenho, de forma que novas topologias podem melhorar ainda mais o desempenho dos modelos.

O modelo DenseNet foi proposto em 2017 por pesquisadores do Facebook como uma CNN em que uma camada convolucional se conecta a todas as camadas seguintes no mesmo bloco de camadas, como mostra a Figura 9.30, reproduzida do artigo [268]. O modelo DenseNet obtém melhor precisão com menos parâmetros em comparação com os modelos ResNet.

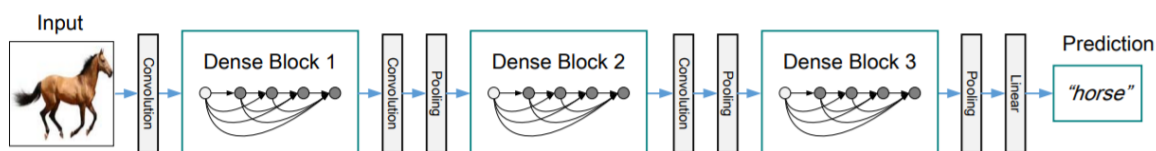


Figura 9.30: Modelo DenseNet. Fonte: reproduzido de [268].

O desenvolvimento de redes neurais profundas se difundiu rapidamente com a disponibilidade de artigos, códigos e modelos pré-treinados.¹⁸⁸ Um modelo treinado num conjunto de dados genérico como ImageNet extrai características de imagens que podem ser adaptadas para outras aplicações fixando os parâmetros das camadas convolucionais e realizando treinamento das últimas camadas. Modelos pré-treinados permitem o desenvolvimento de aplicações de visão computacional em outros domínios com métodos chamados de transferência de aprendizado [556].

Nesta seção, foram discutidos os principais modelos de CNN para problema de classificação de imagens [448]. Há outros problemas de visão computacional que utilizam modelos de CNN, como localização do objeto principal, detecção de múltiplos objetos na imagem e segmentação supervisionada, bem como outros conjuntos de dados anotados como o PASCAL,¹⁸⁹ o COCO,¹⁹⁰ entre outros. Em geral, os modelos de problemas mais complexos utilizam adaptações dos modelos de classificação de imagens. A área de vi-

são computacional tem avançado muito e rapidamente com o desenvolvimento de redes neurais profundas, tanto em termos de novos modelos quanto de novos conjuntos de dados, como o DensePose¹⁹¹, disponibilizado pelo Facebook em 2018.

As redes convolucionais têm outras aplicações como redes convolucionais em grafos [569] e processamento de linguagem natural [221][598]. Modelos de sequências são aplicações de redes neurais recorrentes, como descrito a seguir.

9.3.2 Redes neurais recorrentes

Modelos lineares de séries temporais e sistemas dinâmicos foram apresentados no Capítulo 4, onde diferentes estruturas foram discutidas. A modelagem de sistemas dinâmicos não lineares é complexa e há diferentes abordagens na literatura [51].

Modelos dinâmicos não lineares podem ser obtidos como uma extensão de estruturas semelhantes de modelos lineares. O modelo NARMAX,* por exemplo, é uma estrutura ARMAX não linear em que uma rede neural MLP realiza o processamento entrada-saída. A Figura 9.31 mostra um modelo NARMAX representado por um diagrama de blocos (Figura 9.31a) e como uma rede neural de duas camadas (Figura 9.31b). O triângulo vermelho nas duas figuras representa o operador de defasagem (*shift*).

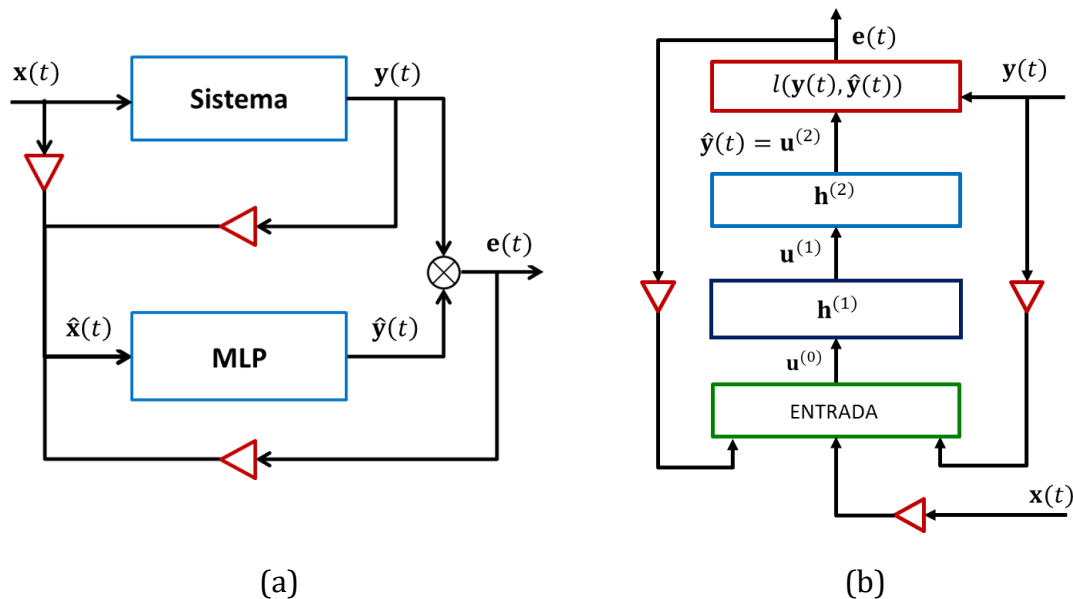


Figura 9.31: Modelos NARMAX: (a) diagrama de blocos; (b) rede neural MLP,

* Non-linear AutoRegressive Moving Average with eXogeneous input.

Modelos autorregressivos são mapeamentos estáticos que não representam internamente a dinâmica do sistema. Os modelos de erro de saída (OE) e de espaço de estados representam melhor a dinâmica do sistema e são utilizados para simulação. No modelo de espaços de estados, o vetor de estados representa a informação do passado necessária para previsão do futuro do sistema [293]. O modelo de espaços de estados linear foi apresentado na seção 4.3.5 e um modelo não linear ajustado por um sistema *fuzzy* recorrente foi discutido na seção 8.4.4 para o problema de previsão de vazões.

Redes neurais recorrentes (RNN)* são modelos não lineares que utilizam um vetor de estados para armazenar uma memória interna. Há diferentes topologias de redes neurais recorrentes para modelagem de sistemas dinâmicos não lineares em geral, mas as RNN têm se destacado para previsão de sequências não numéricas, com aplicações no processamento de linguagem natural.

9.3.2.1 Modelos de espaço de estado não linear

A característica dos modelos Perceptron e ADALINE é o processamento em camadas, em que não há conexões entre as unidades da mesma camada. Em 1982, Hopfield apresentou um modelo de rede neural recorrente como uma camada totalmente conectada [258], capaz de armazenar registros de memória, como os atratores de uma função de energia. Em 1986, Jordan apresentou uma rede neural que é uma versão não linear do modelo de erro de saída (OE), em que os valores de saídas calculadas nos instantes anteriores são utilizados como entrada [291]. O modelo de rede neural defasada no tempo (TDNN)[†] foi proposto em 1990 para solução de um problema simples de reconhecimento de fala [328]. O modelo TDNN é utilizado também em modelagem de sistemas dinâmicos não lineares, como um modelo NARX.

O modelo de Jordan foi a primeira rede neural recorrente, mas não utilizava explicitamente as variáveis de estado [291]. A formulação de redes neurais como modelos não lineares de espaço de estados foi feita por Elman em 1990 [151]. A Figura 9.32 mostra o modelo de Jordan na Figura 9.32a e o modelo de Elman na Figura 9.32b. No modelo de Jordan, a saída calculada pelo modelo é reinjetada na entrada. O modelo de Elman utiliza o vetor de ativação da primeira camada como um vetor de estados, que representa a memória do sistema, e que é reinjetado na entrada.

* *Recurrent Neural Network.*

† *Time Delay Neural Network.*

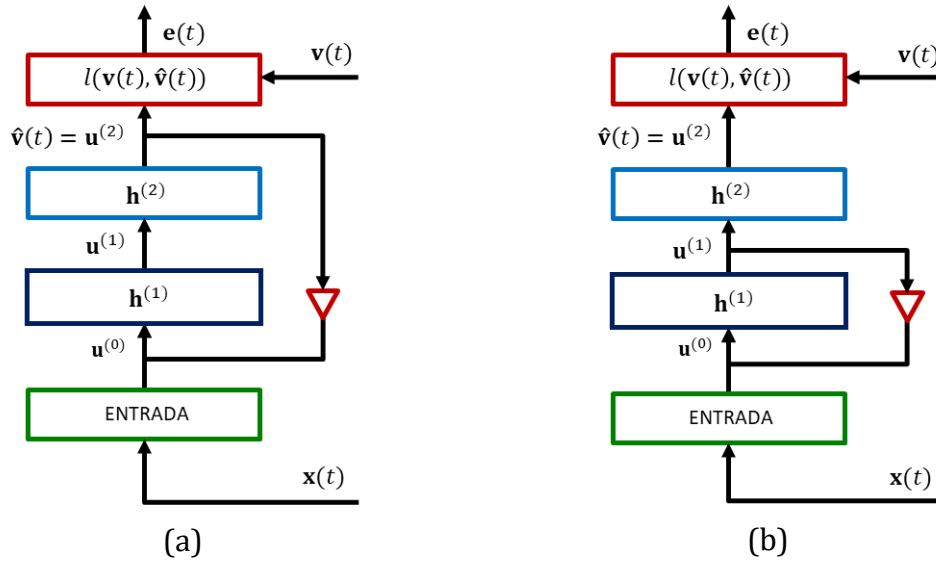


Figura 9.32: Modelos de redes neurais recorrentes: (a) modelo erro de saída não linear de Jordan; (b) modelo de espaço de estados não linear de Elman.

O modelo de Elman é uma rede neural de duas camadas (cf. equação (9.19)), em que a saída da primeira camada é reinjetada na entrada no instante seguinte, como um modelo de espaço de estados não linear:

$$\mathbf{u}^{(1)}(t) = \mathbf{h}^{(1)}(\mathbf{z}^{(1)}(t)) = \mathbf{h}^{(1)}(\mathbf{b}^{(1)} + \mathbf{x}(t)\mathbf{W}_x^{(1)} + \mathbf{u}^{(1)}(t-1)\mathbf{W}_u^{(1)}) \quad (9.105)$$

$$\hat{\mathbf{v}}(t) = \mathbf{u}^{(2)}(t) = \mathbf{h}^{(2)}(\mathbf{z}^{(2)}(t)) = \mathbf{h}^{(2)}(\mathbf{b}^{(2)} + \mathbf{u}^{(1)}(t)\mathbf{W}^{(2)}) \quad (9.106)$$

A primeira camada recebe o vetor de entradas (ou uma codificação da entrada) e o vetor de ativação calculado na iteração anterior, de modo que há uma matriz de conexão para cada variável e uma unidade bias. A camada recorrente na equação (9.105) pode ser escrita como $\mathbf{u}^{(1)}(t) = \mathbf{h}^{(1)}(\mathbf{b}^{(1)} + \mathbf{u}^{(0)}(t)\mathbf{W}^{(1)})$, onde $\mathbf{u}^{(0)}(t) = [\mathbf{x}(t), \mathbf{u}^{(1)}(t-1)]$ como mostra a Figura 9.32b. No modelo de Jordan, $\mathbf{u}^{(0)}(t) = [\mathbf{x}(t), \mathbf{u}^{(2)}(t-1)]$ como mostra a Figura 9.32a. A segunda camada é calculada de forma similar a uma rede MLP.

Redes neurais recorrentes podem ter diferentes topologias [430], bem como múltiplas camadas, como redes neurais profundas. O modelo de Elman é a RNN padrão (*vanilla*), e será utilizado para apresentação do algoritmo de ajuste de parâmetros. Nesse caso, a equação (9.105) pode ser rescrita de uma forma simplificada, eliminando os índices das camadas e deixando os índices do tempo, como:

$$\mathbf{u}(t) = \mathbf{h}(\mathbf{b} + \mathbf{x}(t)\mathbf{W}_x + \mathbf{u}(t-1)\mathbf{W}_u) \quad (9.107)$$

$$\hat{\mathbf{v}}(t) = \mathbf{g}(\mathbf{c} + \mathbf{u}(t)\mathbf{W}) \quad (9.108)$$

onde $\mathbf{u}(t) = \mathbf{u}^{(1)}(t)$; $\mathbf{c} = \mathbf{b}^{(2)}$ e $\mathbf{W} = \mathbf{W}^{(2)}$. A função de ativação sigmoide ou tangente hiperbólica é utilizada com frequência na camada recorrente ($\mathbf{h} = \mathbf{h}^{(1)}$). A função de ativação na camada de saída ($\mathbf{g} = \mathbf{h}^{(2)}$) é a função *softmax*, em problemas de classificação ou a função linear em problemas de regressão.

As redes neurais recorrentes podem ser representadas por camadas “desdobradas” no tempo [464]. A Figura 9.33 mostra um modelo de Elman como uma rede neural de camadas horizontais e verticais, em que as camadas verticais representam o processamento de entrada-saída do modelo em cada instante e as camadas horizontais, os instantes de tempo ou itens na sequência. Na rede desdobrada, a camada recorrente é conectada tanto com a camada de saída do modelo no mesmo item da sequência quanto com a camada recorrente do próximo item. Os índices das camadas horizontais representam o tempo ou itens da sequência. O modelo de RNN padrão utiliza os mesmos parâmetros no mesmo nível vertical, como ilustrado na Figura 9.33.

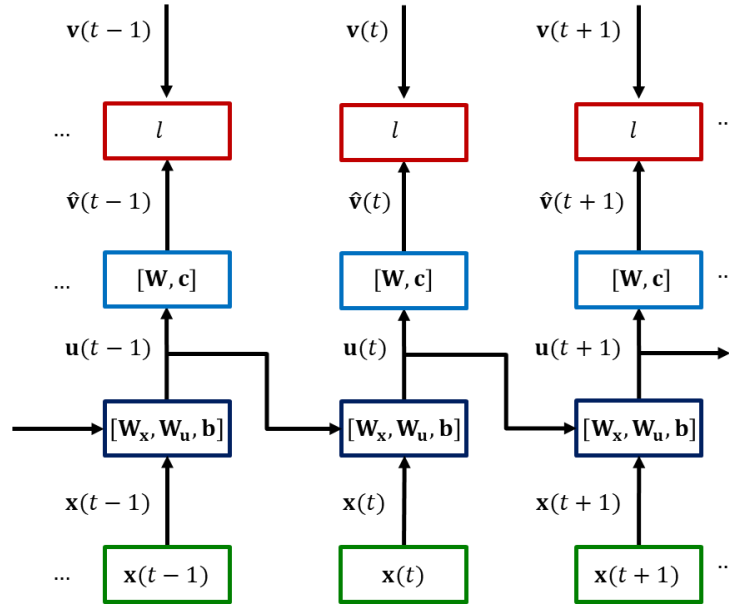


Figura 9.33: Rede neural recorrente desdobrada no tempo.

Redes neurais recorrentes desdobradas no tempo são redes em camadas com parâmetros compartilhados, que permitem a construção de diversas topologias para a solução de diferentes problemas. As camadas de entrada podem ser (uma ou mais) camadas

convolucionais para o processamento de diferentes tipos de dados de entrada. Na verdade, a formulação de redes neurais profundas como camadas encapsuladas permite a combinação de camadas em grafos computacionais complexos.

A Figura 9.34 mostra diferentes configurações da RNN padrão desdobradas no tempo que são adequadas a aplicações distintas. A Figura 9.34a apresenta o exemplo em que todos os itens têm entrada e saída, como num sistema dinâmico ou para uma sequência de imagens. A Figura 9.34b mostra um modelo que tem somente uma entrada e várias saídas, como no caso em que uma imagem é apresentada como entrada e uma sequência de palavras é apresentada na saída como a descrição da imagem. O exemplo da Figura 9.34c acontece em aplicações de tradução, em que o modelo armazena uma sequência de palavras de entrada antes de iniciar o processamento da saída. A Figura 9.34d é um exemplo de topologia utilizada em análise de sentimentos, em que a saída do modelo é um conceito em função de uma sequência de palavras de entrada.

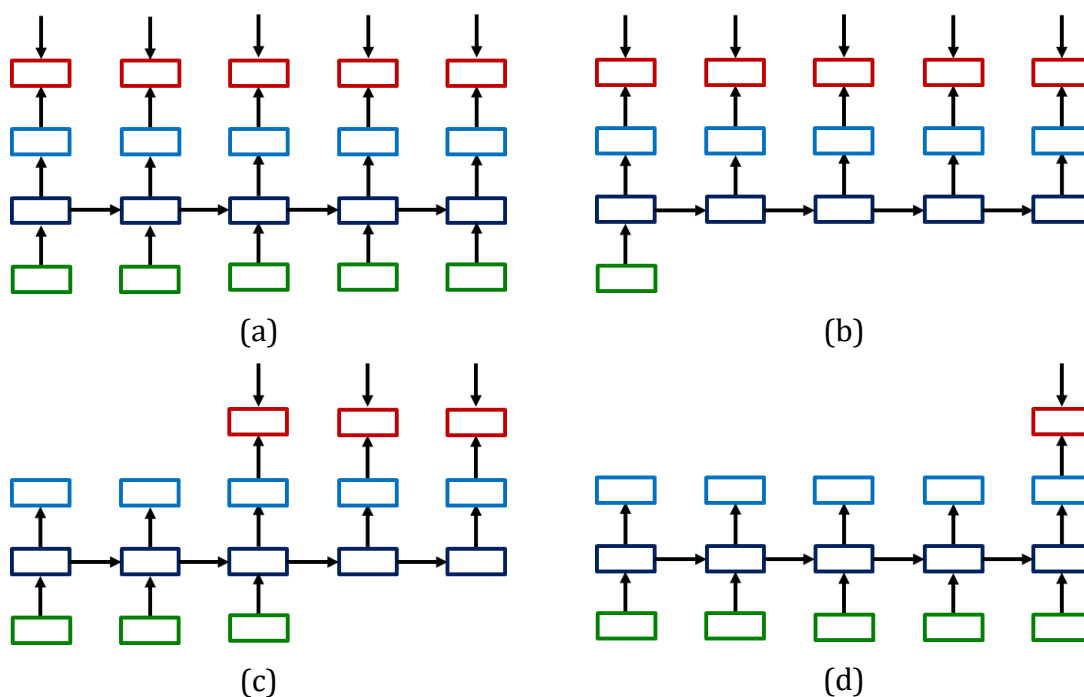


Figura 9.34: Topologias de rede neural recorrente desdobradas no tempo:

- (a) múltiplas entradas e múltiplas saídas; (b) uma entrada e múltiplas saídas;
(c) múltiplas entradas e múltiplas saídas defasadas; (d) múltiplas entradas e saída simples.

O ajuste de parâmetros das diferentes topologias de RNN é realizado pela aplicação do algoritmo da retropropagação através do tempo, como descrito a seguir.

9.3.2.2 Retropropagação através do tempo

A camada recorrente de uma RNN padrão representa a dinâmica temporal do sistema como $\mathbf{u}(t) = \mathbf{h}(\mathbf{u}(t-1), \mathbf{x}(t), \Theta)$. O vetor de estados $\mathbf{u}(t-1)$ armazena toda a informação necessária para a previsão de $\mathbf{u}(t)$, que é utilizado na camada de saída. A dependência temporal exige que $\mathbf{u}(t)$ só possa ser calculado após $\mathbf{u}(t-1)$, de forma que o processamento é sequencial no tempo. Na direção da saída, a cada instante, a rede realiza o processamento usual de uma rede neural MLP, com a entrada adicional $\mathbf{u}(t-1)$ na camada recorrente. Em aplicações de processamento de linguagem natural, uma sequência inteira é processada, o que permite conexões bidirecionais, mas o processamento de um item só pode ser realizado após o anterior (ou posterior).

O algoritmo de retropropagação é aplicado de forma usual na direção reversa do processamento da saída. Tendo em vista que os registros do conjunto de treinamento são conhecidos, o algoritmo da retropropagação pode ser utilizado na direção reversa do tempo no algoritmo da retropropagação através do tempo (BPTT)* [563]. Há outras abordagens para ajuste de RNN, como o aprendizado recorrente em tempo real (RTRL)†, em que a recorrência é aplicada no ajuste de parâmetros e a atualização de parâmetros numa iteração é calculada em função da atualização anterior. Nos modelos de reservatório [355], somente os parâmetros da camada de saída são ajustados, e os demais parâmetros gerados aleatoriamente por uma distribuição conhecida. Essa abordagem também é utilizada por diferentes tipos de modelos [517].

Williams e Zisler [562] apresentam uma revisão e análise de complexidade de algoritmos de ajuste de parâmetros de redes recorrentes. Os resultados mostram que o algoritmo BPTT é mais eficiente que o RTRL para o processamento de sequências.

O algoritmo BPTT é o mais utilizado em modelos de sequência. O algoritmo é uma aplicação direta da retropropagação do erro na RNN desdobrada no tempo, com duas camadas verticais e N camadas horizontais. A Figura 9.35 mostra os fluxos de vetores de ativação e de erro propagado. O erro retropropagado da camada de saída, em cada instante, é semelhante ao de uma rede MLP. A camada recorrente recebe as contribuições do erro retropropagado da saída $\epsilon_v(t)$ e o erro retropropagado no tempo $\epsilon_u(t)$. O algoritmo BPTT é utilizado quando todas as N entradas são conhecidas, de forma que a

* *Backpropagation Through Time.*

† *Real Time Recurrent Learning.*

camada recorrente pode receber a contribuição do erro retropropagado dos estados seguintes, exceto no último item da sequência (registro $t = N$).

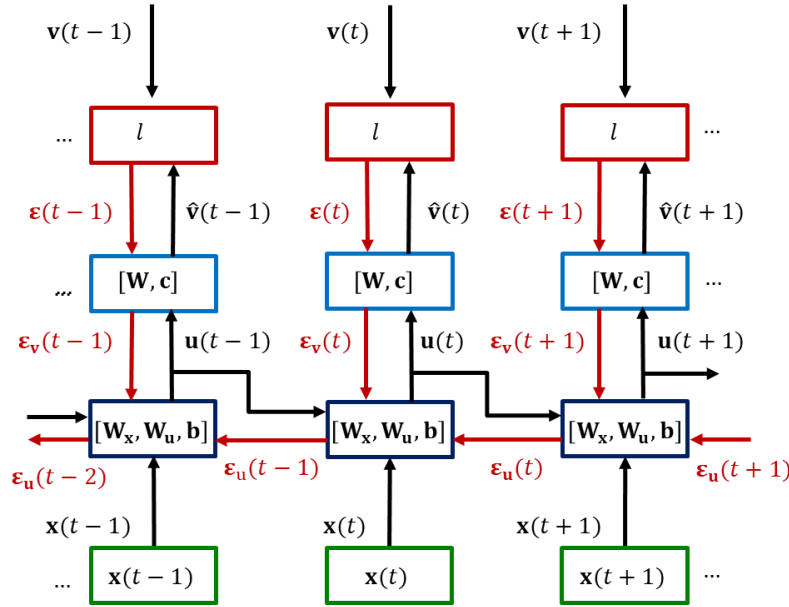


Figura 9.35: Retropropagação do erro horizontal e vertical numa rede neural recorrente desdobrada no tempo.

Na camada de saída, o ajuste é feito como na rede neural MLP (cf. equações (9.54) e (9.55)). Considerando a notação utilizada nas equações (9.107) e (9.108):

$$\Delta \mathbf{W} = -\eta \mathbf{u}(t)^T \delta_2(t) \quad (9.109)$$

$$\Delta \mathbf{c} = -\eta \delta_2(t) \quad (9.110)$$

onde $\delta_2(t) = \boldsymbol{\varepsilon}(t) \odot \mathbf{g}'$ é o gradiente local da camada de saída, que é calculado como $\delta_2(t) = -(\mathbf{v}(t) - \hat{\mathbf{v}}(t))$ (cf. equações (9.57) e (9.58)).

O erro retropropagado da camada de saída para a camada recorrente é calculado como:

$$\boldsymbol{\varepsilon}_v(t) = \delta_2(t) \mathbf{W}^T \quad (9.111)$$

Na camada recorrente, o gradiente local é calculado de acordo com a topologia da rede utilizada, como mostrado na Figura 9.34. Na rede completa, ou seja, que possui registros de entrada e saída em todos os itens (cf. Figura 9.34a), o gradiente local da camada recorrente é calculado pela soma dos erros retropropagados de saída e recorrente:

$$\delta_1(t) = \mathbf{h}' \odot (\boldsymbol{\varepsilon}_v(t) + \boldsymbol{\varepsilon}_u(t)). \quad (9.112)$$

Nos itens da sequência que não têm valores observados de saída (cf. Figura 9.34c e d), $\epsilon_v(t) = 0$ e o gradiente local é calculado como $\delta_1(t) = \mathbf{h}' \odot \epsilon_u(t)$. A saída do último item ($t = N$) deve ser observada e só há a contribuição do erro de saída para o gradiente local, calculado como $\delta_1(N) = \mathbf{h}' \odot \epsilon_v(N)$.

A partir do último registro, o erro é retropropagado no tempo (sequência) para a camada recorrente do instante (item) anterior como:

$$\epsilon_u(t-1) = \delta_1(t) \mathbf{W}_u^T \quad (9.113)$$

A atualização dos parâmetros é realizada como no algoritmo da retropropagação em *batch* (cf. equações (9.62) e (9.63)):

$$\Delta \mathbf{W}_x = -\eta \sum_{t=1}^N \mathbf{x}(t)^T \delta_1(t) \quad (9.114)$$

$$\Delta \mathbf{W}_u = -\eta \sum_{t=1}^N \mathbf{u}(t-1)^T \delta_1(t) \quad (9.115)$$

$$\Delta \mathbf{b} = -\eta \sum_{t=1}^N \delta_1(t) \quad (9.116)$$

Observe que a recorrência no cálculo do gradiente local (cf. equações (9.112) e (9.113)) impede o processamento vetorial do *batch*, deve ser sequencial para cada registro.

O algoritmo BPTT também pode ser utilizado reajustando os parâmetros a cada *batch* de N registros. Dessa forma, a rede recorrente pode ajustar um modelo com parâmetros adaptativos, variantes no tempo. Há outras variantes para o ajuste de RNN [562], mas o BPTT é o algoritmo que tem sido mais utilizado em redes neurais profundas, principalmente em aplicações de processamento de linguagem natural.

A técnica chamada de **reforço do professor** (*teacher forcing*) é utilizada no ajuste de RNN em que a camada recorrente recebe uma conexão da camada de saída (como no modelo de Jordan). Durante o treinamento, o modelo é ajustado com os valores observados da saída. Na execução, o valor correspondente ao máximo da função *softmax* é reinjetado na entrada [562]. Essa técnica não funciona com modelos de saída numérica (cf. Figura 4.29), mas obtém bons resultados em modelos de sequências devido à amostragem realizada pela saída correspondente ao valor máximo da função *softmax*.

O compartilhamento de parâmetros na rede neural recorrente faz com que a retropropagação do erro seja realizada sempre com a mesma matriz de parâmetros de conexões

(cf. equação (9.113)). Isso faz com que o ajuste de parâmetros de redes recorrentes tenha problemas de estabilidade [40][429], o que pode provocar aumento ou redução exponencial do gradiente local.

A análise de estabilidade do gradiente local calculado pelo algoritmo BPTT é feita por analogia a um sistema dinâmico linear, modelado por equações de estado. Considere o caso em que a função de ativação é linear, sem o erro de saída ($\epsilon_v(t) = 0$). Nesse caso, $\delta_1(t) = \epsilon_u(t)$ e, após K iterações da equação (9.113), o erro retropropagado é calculado como:

$$\delta_1(t) = \prod_{\tau=1}^K \delta_1(t + \tau) \mathbf{W}_u^T = \delta_1(N) (\mathbf{W}_u^T)^K \quad (9.117)$$

onde $(\mathbf{W}_u^T)^K$ é a matriz $\mathbf{W}_u$ transposta e elevada à K -ésima potência.

O comportamento do gradiente local nesse caso é explicado pela decomposição espectral $(\mathbf{W}_u^T)^K = \mathbf{Q}^T \mathbf{\Lambda}^K \mathbf{Q}$, onde $\mathbf{Q}$ é a matriz dos autovetores e $\mathbf{\Lambda}$, a matriz diagonal de autovalores. A componente correspondente do gradiente local tem o seguinte comportamento para cada $\lambda_i \in \mathbf{\Lambda}$:

- Se $|\lambda_i| > 1$, a componente $\delta_{1i}(t) \rightarrow \infty$: o gradiente local diverge;
- Se $|\lambda_i| < 1$, a componente $\delta_{1i}(t) \rightarrow 0$: o gradiente local se anula.

O caso em que o gradiente local diverge, pode ser resolvido com a técnica de corte do gradiente [429]. Nesse caso, se $\|\delta_1(t)\| > \xi$, então o gradiente local é ajustado como:

$$\hat{\delta}_1(t) = \frac{\delta_1(t)}{\|\delta_1(t)\|} \quad (9.118)$$

A diluição do gradiente local é um problema complexo e intrínseco ao método do gradiente em redes neurais profundas. Em redes neurais recorrentes, esse problema impede a capacidade de modelar dependências em sequências longas [40], o que pode ser contornado por unidades com portas, como descrito a seguir.

9.3.2.3 Modelos com portas: LSTM e GRU

Em 1997, Hochreiter e Schmidhuber [252] introduziram o modelo chamado de memória longa de curto prazo (LSTM),* posteriormente aprimorado por Gers e colaboradores [198][199].

A camada recorrente de um modelo LSTM, chamada de célula de memória, contém um vetor de estados interno, chamado *cell state*, e um conjunto de portas (*gates*) que controlam o fluxo de informações no interior da célula.

As portas do modelo LSTM são calculadas vetorialmente como camadas de redes neurais recorrentes:

$$[\mathbf{p}_f(t) \ \mathbf{p}_i(t) \ \mathbf{p}_g(t) \ \mathbf{p}_o(t)] = \text{sig} \left([\mathbf{x}(t) \ \mathbf{u}(t-1) \ 1] \begin{bmatrix} \mathbf{W}_{xf} & \mathbf{W}_{xi} & \mathbf{W}_{xg} & \mathbf{W}_{xo} \\ \mathbf{W}_{uf} & \mathbf{W}_{ui} & \mathbf{W}_{ug} & \mathbf{W}_{uo} \\ \mathbf{b}_f & \mathbf{b}_i & \mathbf{b}_g & \mathbf{b}_o \end{bmatrix} \right) \quad (9.119)$$

onde **sig** é a versão vetorial da função de ativação sigmoide.

As portas são utilizadas para a atualização dos estados, por operações de multiplicação vetorial elemento a elemento:

- Vetor de estados interno (*cell state*):

$$\mathbf{s}(t) = \mathbf{p}_f(t) \odot \mathbf{s}(t-1) + \mathbf{p}_i(t) \odot \mathbf{p}_g(t) \quad (9.120)$$

- Vetor de estados recorrente:

$$\mathbf{u}(t) = \mathbf{p}_o(t) \odot \tanh(\mathbf{s}(t)) \quad (9.121)$$

onde **tanh** é a versão vetorial da função de ativação tangente hiperbólica, que é a função sigmoide escalada no intervalo $[-1,1]$: $\tanh(x) = 2\text{sig}(x) - 1$.

Os valores de ativação das portas controlam a propagação do fluxo: valores de ativação baixos “fecham” as portas e valores altos “abrem”. A porta $\mathbf{p}_f(t)$, introduzida por Gers e colaboradores [198], é chamada de “esquecimento” porque controla a porção da ativação do vetor de estados interno no item anterior a ser utilizada na atualização do item corrente. As portas $\mathbf{p}_g(t)$ e $\mathbf{p}_i(t)$ controlam a entrada, evitando ruído e informações irrelevantes. A porta de saída $\mathbf{p}_o(t)$ controla a porção do estado interno utilizada

* *Long Short-Term Memory*.

na atualização do estado recorrente. Graves [216] apresenta a formulação do algoritmo de ajuste de parâmetros da camada LSTM pelo algoritmo BPTT.

A Figura 9.36 mostra o fluxo de processamento interno de uma camada LSTM. As setas vermelhas mostram o fluxo retropropagado do erro nos estados internos, que atravessa a célula diretamente sem a multiplicação pela matriz de pesos, evitando assim o efeito de diluição ou explosão do gradiente local.

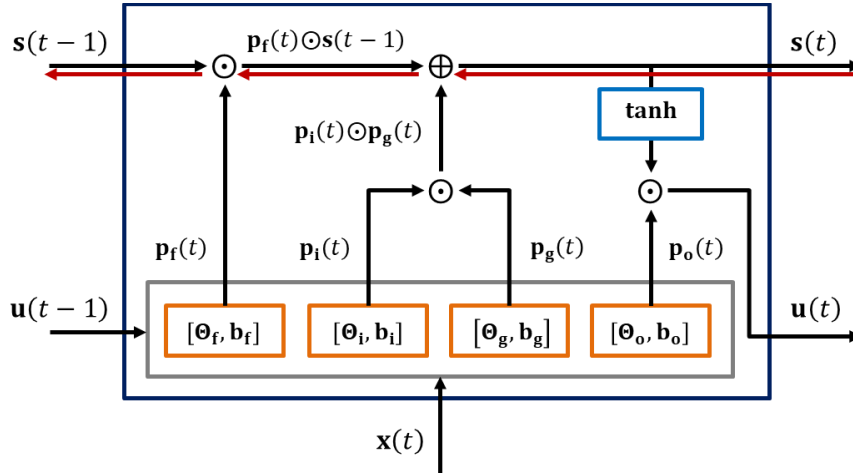


Figura 9.36: Visão esquemática da camada LSTM.

Há diversas variantes do modelo LSTM descritas na literatura [199][216]. O modelo apresentado nesta seção segue a formulação de Gers e colaboradores [198], que também é apresentado no livro [212], com as devidas adaptações de notação.

Uma variante mais simples do modelo LSTM é chamada de unidade recorrente com portas (GRU)* e foi introduzida por Cho e colaboradores no contexto de tradução automática [101][102]. O modelo GRU contém apenas duas portas, calculadas como:

$$[\mathbf{p}_a(t) \quad \mathbf{p}_r(t)] = \text{sig} \left(\begin{bmatrix} \mathbf{x}(t) & \mathbf{u}(t-1) & \mathbf{1} \end{bmatrix} \begin{bmatrix} \mathbf{W}_{xa} & \mathbf{W}_{xr} \\ \mathbf{W}_{ua} & \mathbf{W}_{ur} \\ \mathbf{b}_a & \mathbf{b}_r \end{bmatrix} \right) \quad (9.122)$$

onde $\mathbf{p}_a(t)$ é a porta de atualização e $\mathbf{p}_r(t)$, a porta de *reset*.

A porta $\mathbf{p}_g(t)$ é calculada como no modelo LSTM [199], mas com função de ativação tangente hiperbólica e a porta $\mathbf{p}_r(t)$ controlando a porção do estado anterior que é utilizada na recorrência:

* *Gated Recurrent Unit*.

$$\mathbf{p}_g(t) = \tanh \left([\mathbf{x}(t) \ \mathbf{p}_r(t) \odot \mathbf{u}(t-1) \ \mathbf{1}] \begin{bmatrix} \mathbf{W}_{xg} \\ \mathbf{W}_{ug} \\ \mathbf{b}_g \end{bmatrix} \right) \quad (9.123)$$

A atualização dos estados é feita como:

$$\mathbf{u}(t) = \mathbf{p}_a(t) \odot \mathbf{u}(t-1) + (\mathbf{1} - \mathbf{p}_a(t)) \odot \mathbf{p}_g(t) \quad (9.124)$$

Diversas variantes dos modelos LSTM e GRU foram testadas em aplicações de processamento de sequências [105][220][292]. Os resultados mostram que os modelos com portas são melhores que o modelo de RNN padrão. Entretanto, nenhuma variante é superior em todas as aplicações. Greff e colaboradores [220] mostram que o modelo LSTM [199] funciona bem na maioria das aplicações e a porta de esquecimento influencia bastante no desempenho.

Modelos LSTM e GRU têm sido utilizados com sucesso em diferentes aplicações de processamento de sequências, como reconhecimento de fala [218][219], tradução automática [510], legendagem (*captioning*) de imagens [543][574]. Uma aplicação de previsão de séries temporais do mercado financeiro a partir de textos de notícias [536] é apresentada na seção 9.4.4. Alguns dos principais resultados recentes de RNN em aplicações de processamento de linguagem natural são discutidos a seguir.

9.3.2.4 Vetorização de palavras

O processamento de linguagem natural (NLP)* é o conjunto de técnicas computacionais para a análise automática e representação da linguagem humana. A abordagem usual de NLP consiste em utilizar modelos de inteligência computacional para tarefas específicas como: reconhecimento de fala, tradução automática, recuperação da informação, sumarização de textos, modelagem de tópicos, análise de sentimentos, perguntas e respostas, entre outros [72]. Em alguns casos, diferentes tarefas são empregadas na mesma aplicação, como no caso do Google Tradutor,¹⁹² que pode ser consultado por voz ou realizar a tradução de texto em fotos na versão para dispositivos móveis.

O processamento de linguagem natural é um problema complexo, em função do grande número de palavras. O dicionário *Aurélio online*¹⁹³ contém a definição de mais de 200 mil palavras ($\mathcal{O}(10^5)$) e um conjunto de textos utilizado para ajuste de modelos, chamado *corpus*, pode conter de 20 milhões a 200 milhões de palavras¹⁹⁴ ($\mathcal{O}(10^8)$).

* *Natural Language Processing*.

A representação das palavras (e documentos) é, portanto, crucial para o desenvolvimento dos modelos. A representação por variáveis indicadoras (codificação *one-hot*), utilizada para representação de variáveis nominais em modelos de inteligência computacional, não é muito útil no NLP. Além da grande dimensão, uma base ortogonal não representa a similaridade entre palavras e o contexto em que a palavra aparece. Um dicionário de sinônimos poderia resolver o problema de palavras diferentes com significados semelhantes, como “cão” e “cachorro”. Entretanto, “cão” e “gato” não são sinônimos, mas aparecem em contextos semelhantes, de forma que a representação deveria capturar alguma similaridade entre essas palavras.

A linguística computacional utiliza a hipótese de semântica distributiva, de que palavras que aparecem nos mesmos contextos estão relacionadas às mesmas palavras em sua vizinhança [203]. A representação vetorial pela matriz de co-ocorrências dos termos na dimensão do vocabulário completo é computacionalmente inviável. A análise de semântica latente (LSA)*, introduzida em 1990, é utilizada para redução de dimensionalidade da representação vetorial de palavras e documentos a partir da decomposição de valores singulares [122].

A projeção realizada pela LSA é capaz de capturar a similaridade de palavras (e documentos) com uma dimensão reduzida de fatores, sendo muito utilizada em recuperação de informação e mineração de textos. Entretanto, essa representação não captura a relação de cada palavra com as palavras próximas, ou seja, seu contexto na frase, que pode ser muito importante em aplicações de NLP.

Modelos de redes neurais em aplicações de NLP são baseados num modelo estatístico de linguagem. Considere o conjunto de palavras $\mathcal{L} = \{L_1, \dots, L_M\}$ como um dicionário de M palavras e um *corpus* contendo textos no mesmo idioma como uma sequência de N palavras $\mathcal{V} = \{v(1), \dots, v(N)\}$, onde $v(t) \in \mathcal{L}$. Um modelo estatístico de linguagem é representado pela probabilidade condicional da ocorrência de uma palavra a partir da observação de todas as anteriores:

$$p(v(t)) = \prod_{k=1}^{k=t-1} p(v(k) | \{v(1), \dots, v(k-1)\}) \quad (9.125)$$

* *Latent Semantic Analysis*.

O modelo descrito pela equação (9.135) é intratável, de forma que é aproximado utilizando apenas as n palavras anteriores:

$$p(v(t)) \approx \prod_{k=t-n+1}^{k=t-1} p(v(k)|\{v(1), \dots, v(k-1)\}) \quad (9.126)$$

Modelos estatísticos de linguagem têm sido muito utilizados em NLP com sequências de palavras (ou letras) de tamanho fixo chamadas de n -gramas [94][299]. Os modelos de n -gramas são ajustados por contagens; quando uma combinação de palavras não aparece no *corpus* a contagem é zero e o modelo não produz uma saída adequada.

Em 2003, Bengio e colaboradores [36] apresentaram um modelo estatístico de linguagem na forma de uma rede neural MLP de três camadas. O objetivo do modelo era ajustar simultaneamente uma representação distribuída para as palavras e uma aproximação da distribuição de probabilidades condicional do modelo. Essa abordagem permite tanto a representação de palavras similares quanto a de palavras diferentes em contextos similares. Por exemplo, as palavras “cão” e “gato” são diferentes, mas são mapeadas em vetores similares. O modelo utilizado por Bengio e colaboradores [36] era muito complexo e o processamento para grandes *corpora* não era muito eficiente.

A representação distribuída ou **vetorização de palavras** (*word embedding*) se tornou muito conhecida através do modelo **word2vec**, desenvolvido em 2013 por Mikolov e colaboradores [382][383] e patentado pelo Google.¹⁹⁵ O word2vec é uma aproximação de um modelo estatístico de linguagem por uma rede neural MLP de duas camadas de processamento, com função de ativação linear na camada intermediária e *softmax* na saída, sem unidades bias. O modelo realiza a previsão de uma palavra em função da sua vizinhança, mas a previsão é utilizada apenas para o ajuste dos parâmetros, uma vez que o objetivo é a representação vetorial das palavras e não a previsão em si. O modelo word2vec é calculado com duas topologias:

- **CBOW:** Nessa topologia, as palavras da vizinhança (contexto) são apresentadas como entrada e a palavra central é a saída. O modelo é calculado como:

$$\begin{aligned} \mathbf{u}(t) &= \sum_{\substack{k=-m \\ k \neq 0}}^m \mathbf{x}(t+k) \mathbf{W}_x \\ \hat{\mathbf{v}}(t) &= \text{softmax}(\mathbf{u}(t) \mathbf{W}_u^T) \end{aligned} \quad (9.127)$$

- **Skip-gram:** nesse caso, a entrada é a palavra central e o modelo é ajustado com as palavras da vizinhança:

$$\begin{aligned} \mathbf{u}(t) &= \mathbf{x}(t)\mathbf{W}_x \\ \hat{\mathbf{v}}(t+k) &= \text{softmax}(\mathbf{u}(t)\mathbf{W}_u^T), \quad -m \leq k \leq m, k \neq 0 \end{aligned} \quad (9.128)$$

onde $\mathbf{x}(t)$ é a representação da entrada por variáveis indicadoras (*one-hot*); $\mathbf{W}_x$ é a matriz de conexões (dim: $M \times n$) da camada intermediária e $\mathbf{W}_u$ (dim: $n \times M$), a matriz de conexões da saída; M é o número de palavras do dicionário; m é o número de vizinhos para cada lado e n , a dimensão da representação vetorial. Em geral, $50 \leq n \leq 300$, mas pode ser maior dependendo da aplicação.

A saída $\hat{\mathbf{v}}(t)$ do modelo word2vec é a distribuição de probabilidades das palavras do dicionário calculada pelo modelo. A probabilidade de ocorrência de uma palavra específica $v(t) = L_i \in \mathcal{L}$ é aproximada pela função *softmax* como:

$$p(v(t)) \approx \hat{v}_i(t) = \frac{\exp(\mathbf{u}(t)\mathbf{w}_i^T)}{\sum_{i=1}^M \exp(\mathbf{u}(t)\mathbf{w}_i^T)} \quad (9.129)$$

onde $\mathbf{w}_i$ é a linha i da matriz de conexões de saída $\mathbf{W}_u$.

O objetivo do word2vec é a vetorização das palavras do dicionário, que são armazenadas nas linhas da matriz $\mathbf{W}_x$. A Figura 9.37 mostra o processamento das duas topologias do modelo word2vec. A Figura 9.37a mostra a topologia CBOW, cuja entrada é o contexto, definido pelo conjunto das palavras da vizinhança $\{v(t-m), \dots, v(t+m)\}$, sem a palavra central $v(t)$. Os vetores $\{\mathbf{x}(t-m), \dots, \mathbf{x}(t+m)\}$ representam as entradas correspondentes por variáveis indicadoras (*one-hot*). O vetor $\mathbf{u}(t)$ é a soma dos vetores de representação das palavras de entrada, ou seja, a soma das linhas correspondentes da matriz $\mathbf{W}_x$. A Figura 9.37b apresenta a topologia *skip-gram*, em que a entrada é a palavra central $v(t)$, o vetor $\mathbf{x}(t)$ é a sua representação por variáveis indicadoras (*one-hot*) e $\mathbf{u}(t)$ é a representação da entrada, ou seja, a linha correspondente da matriz $\mathbf{W}_x$.

Ambas as topologias têm apenas uma saída. Na topologia CBOW, o modelo é ajustado apenas com o valor central (cf. Figura 9.37a), enquanto que na *skip-gram* o modelo é ajustado com as palavras da vizinhança (cf. Figura 9.37b). A comparação do resultado do modelo com diferentes valores de saída, na topologia *skip-gram*, gera uma informação mais rica para a retropropagação do erro.

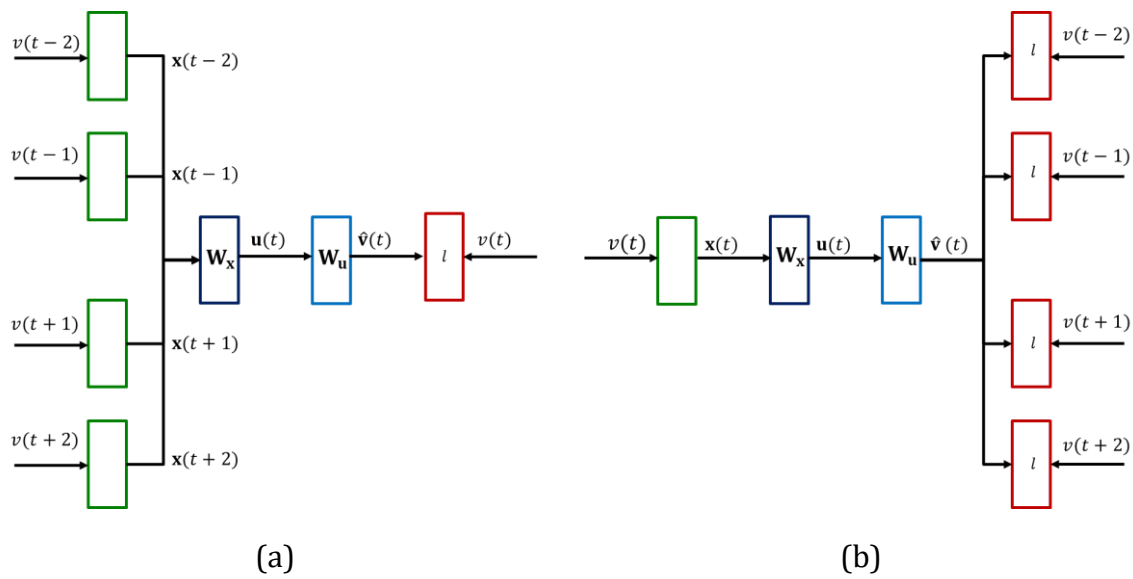


Figura 9.37: Modelo word2vec: (a) topologia CBOW; (b) topologia *Skip-gram*.

Fonte: adaptado de [382].

Os parâmetros $\Theta = [W_x, W_u]$ do modelo word2vec são ajustados pelo algoritmo da retropropagação, utilizando o negativo do logaritmo da verossimilhança como função de avaliação. No modelo original [383], o processamento é realizado utilizando o *softmax* hierárquico [403], que reduz o custo computacional em função da alta dimensionalidade do vetor de saída. Atualmente, o modelo é ajustado com a técnica chamada de estimativa de contraste de ruído [394].

Em geral, o resultado da vetorização (matriz W_x) é calculado *off-line* ou se utiliza alguma representação pré-treinada. Em aplicações de um domínio específico, uma adaptação da vetorização pode ser ajustada por uma camada de entrada da rede neural. O código do modelo original word2vec e os vetores pré-treinados estão disponíveis no repositório de código aberto do Google,¹⁹⁶ no TensorFlow¹⁹⁷ além de diversos projetos no GitHub.¹⁹⁸

Os resultados dos modelos word2vec não só representam adequadamente as palavras similares, também são capazes de aproximar operações com palavras. Por exemplo, considere as palavras $\{L_1, L_2, L_3\}$ e suas representações vetoriais $\{w_1, w_2, w_3\}$, ou seja, as linhas correspondentes da matriz W_x . A operação $L_1 - L_2 + L_3 = L_4$ pode ser aproximada por $w_1 - w_2 + w_3 = w_4$, sendo a palavra L_4 a que tem representação mais próxima do resultado w_4 .

A Tabela 9.4, adaptada de [382], apresenta o resultado em alguns exemplos de operação com palavras. Na primeira coluna, as palavras que realizam a operação $L_1 - L_2$. Nas duas outras colunas, exemplos de palavras L_3 e da palavra L_4 , correspondente ao resultado da operação realizada entre os vetores $\mathbf{w}_4$. O modelo é capaz de inferir o nome das capitais de países e estados, a profissão de pessoas ilustres e até o sistema operacional de empresas de tecnologia. Há erros, como o nome de Vladimir Putin ou tapas como comida típica da França, mas, ainda assim, os resultados obtidos estão bem relacionados ao contexto.

Tabela 9.4: Resultados de operações com vetores. Fonte: adaptado de [382]

Relação: $L_1 - L_2$	Ex 1 $L_3: L_4$	Ex 2 $L_3: L_4$
France - Paris	Italy: Rome	Japan: Tokyo
big - bigger	small: larger	cold: colder
Miami - Florida	Baltimore: Maryland	Dallas: Texas
Einstein - scientist	Messi: midfielder	Mozart: violinist
Sarkozy - France	Berlusconi: Italy	Merkel: Germany
copper - Cu	zinc: Zn	gold: Au
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev
Microsoft - Windows	Google: Android	Apple: iPhone
Microsoft - Ballmer	Apple: Jobs	IBM: McNealy
Japan - sushi	Germany: bratwurst	France: tapas

Algumas variantes do word2vec foram propostas recentemente. O GloVe* foi desenvolvido em 2014 pelo grupo de NLP da Universidade de Stanford. O modelo combina a matriz de contagem de co-ocorrências de palavras e a capacidade preditiva da representação vetorial do word2vec [435]. O código-fonte e o modelo pré-treinado estão disponíveis no *site* do grupo.¹⁹⁹ Em 2016, pesquisadores do Facebook, entre os quais Tomas Mikolov (ex-Google), apresentaram o fastText [56]. O modelo realiza a vetorização de partes de palavras, ou n -gramas de caracteres, e a vetorização de uma palavra é a soma da vetorização dos seus n -gramas. O modelo é mais eficiente, pois o número de n -gramas é menor que o número de palavras, além de permitir a comparação entre partes das palavras e realizar a vetorização de palavras novas, ou seja, não contidas no

* *Global Vectors for Word Representation*.

corpus utilizado para o ajuste de parâmetros. O código-fonte e o modelo pré-treinado para inglês e outros 157 idiomas estão disponíveis no *site*.²⁰⁰

Para o português, as vetorizações pré-treinadas com word2vec, GloVe e fastText podem ser encontradas no *site* do Núcleo Interinstitucional de Linguística Computacional (NILC)²⁰¹ da USP em São Carlos. A ideia de vetorização tem sido estendida para outros tipos de dados além de linguagem natural, com resultados interessantes na representação de grafos [213][231]. O problema de vetorização se insere no contexto de aprendizado da representação, que será discutido na seção 9.3.3.

O modelo word2vec não é recorrente, mas a vetorização de palavras tem sido o pré-processamento padrão em aplicações de NLP, que usualmente utilizam redes neurais recorrentes. Young e colaboradores [581] apresentam uma revisão dos principais métodos e aplicações de RNN em NLP. As principais aplicações descritas no artigo são:

- **POS* tagging:** é o problema de identificação da classe gramatical de uma palavra. Os modelos utilizam uma topologia em que há uma saída observada para cada entrada, como na Figura 9.34a. O artigo compara o resultado de diferentes abordagens no *corpus* do Wall Street Journal (WSJ-PTB), que contém 1,17 milhão de *tokens* e é utilizado para desenvolvimento e avaliação de sistemas de *POS tagging*. A acurácia dos modelos é superior a 97%.
- **Reconhecimento de entidade nomeada (NER)[†]:** é o problema de identificação de menções a entidades, que podem ser nomes de pessoas, locais, empresas etc., em textos. Os modelos de NER são capazes de atribuir uma entidade a uma ou mais palavras, como na Figura 9.34d. O artigo apresenta os resultados obtidos pelos modelos no *corpus* CoNLL²⁰² 2003 pela medida F (cf. seção 5.3.1).
- **Análise de sentimentos:** é o problema em que o modelo atribui um conceito ordenado (de negativo a positivo) a uma sentença [427]. Os modelos de análise de sentimentos atribuem um resultado (nesse caso, cinco conceitos) a um conjunto de palavras de entrada como na Figura 9.34d. O artigo compara modelos aplicados no conjunto de dados chamado de Stanford Sentiment Treebank (STT),²⁰³ que é uma organização em árvore de um *corpus* com 11.855 sentenças extraídas de revisões de filmes [497]. O modelo de referência é obtido pelo

* *Part Of Speech*.

† *Named-Entity Recognition*.

grupo de Stanford, que utiliza uma topologia em árvore, chamada rede neural recursiva [212][497]. O melhor resultado utiliza um modelo LSTM em árvore.

- **Pergunta-resposta:** há diferentes definições para esse problema. O artigo compara alguns modelos no conjunto de dados SQuAD [447], que contém mais de 100 mil perguntas sobre cerca de 500 artigos da Wikipédia respondidas por voluntários utilizando a plataforma Daemo.²⁰⁴ O modelo é treinado com um texto de referência, recebe uma pergunta como entrada e deve ser capaz de apresentar a resposta (cf. Figura 9.34c). A versão SQuAD 2.0 contém 50 mil perguntas sem respostas similares às perguntas na base original. Nesse caso, o modelo deve não só responder à pergunta, mas também identificar que uma pergunta não tem resposta e se abster de responder. O *site* do SQuAD 2.0²⁰⁵ apresenta os resultados dos modelos. Alguns dos melhores resultados utilizam um modelo de representação de linguagem chamado BERT [123], publicado em 2018, que tem sido muito utilizado em aplicações de NLP.

Os modelos de NLP são combinados com modelos de visão computacional em aplicações de legendagem automática de imagens. Nesse caso, a última camada de um modelo pré-treinado de classificação de imagens é utilizada como entrada de um modelo de RNN para gerar a legenda [543][571].

Há também problemas clássicos em NLP, como reconhecimento de fala [19][100][424] e a tradução automática [196][442] que, por sua relevância e por utilizar topologias interessantes de redes neurais profundas, são apresentadas a seguir.

9.3.2.5 Reconhecimento de fala

No problema de **reconhecimento de fala**, o modelo recebe como entrada um áudio com o registro de uma sentença em linguagem natural e deve produzir como saída a transcrição da sentença. Esse problema é estudado há muito tempo e diversas abordagens foram desenvolvidas [19]. O conjunto de dados mais utilizado é o TIMIT,²⁰⁶ elaborado em 1990 por MIT, SRI e Texas Instruments, com apoio da Agência de Projetos Avançados de Defesa dos EUA (DARPA). O TIMIT contém 6.300 sentenças, 10 sentenças proferidas por cada um dos 630 voluntários de oito principais regiões com sotaques característicos dos Estados Unidos. Os dados do TIMIT incluem os áudios e as transcrições dos fonemas e das palavras das sentenças.

Modelos de redes neurais têm grande potencial em aplicações de reconhecimento de fala. Uma revisão de 1989 foi publicada no primeiro artigo da revista *Neural Computation* por Lippmann [346]. Até 2012, o estado da arte para reconhecimento de fala era a utilização de um espectrograma para codificação do áudio, um modelo acústico de extração de fonemas por um misturas de gaussianas (GMM)* e um modelo linguístico por cadeias de Markov com estados ocultos (HMM).†

Em 2012, Hinton e colaboradores [249] utilizaram redes neurais profundas (DNN) no lugar de GMM para o modelo acústico. O artigo apresenta uma visão conjunta de pesquisadores da Universidade de Toronto, Microsoft, Google e IBM. Os modelos DNN-HMM obtêm resultados expressivamente melhores que os modelos GMM-HMM em diversos conjuntos de dados. Em 2014, um modelo híbrido utilizando redes neurais convolucionais (CNN) combinado com HMM foi desenvolvido por pesquisadores da Microsoft [2]. O modelo CNN-HMM apresentou resultados ligeiramente melhores aos do modelo DNN-HMM.

Graves e colaboradores [218] apresentaram um modelo de rede neural profunda para o reconhecimento de fonemas em 2013. A abordagem, chamada de ponta a ponta (*end-to-end*), dispensa o treinamento de módulos separados e tem sido utilizada na solução de outras aplicações com redes neurais profundas. A entrada do modelo é um trecho de áudio codificado pela transformada Fourier (FFT) num vetor de 123 elementos. O modelo utiliza uma rede neural recorrente bidirecional, mostrada na Figura 9.38. O modelo bidirecional tem duas camadas recorrentes, uma que realiza o processamento na direção da sequência (*forward*) e outra que realiza o processamento na direção contrária (*backward*). Os resultados das duas camadas são utilizados para o cálculo da saída com função de ativação linear [218]:

$$\mathbf{u}^+(t) = \mathbf{h}(\mathbf{b}^+ + \mathbf{x}(t)\mathbf{W}_x^+ + \mathbf{u}^+(t-1)\mathbf{W}_u^+) \quad (9.130)$$

$$\mathbf{u}^-(t) = \mathbf{h}(\mathbf{b}^- + \mathbf{x}(t)\mathbf{W}_x^- + \mathbf{u}^-(t+1)\mathbf{W}_u^-) \quad (9.131)$$

$$\hat{\mathbf{v}}(t) = \mathbf{c} + \mathbf{u}^+(t)\mathbf{W}^+ + \mathbf{u}^-(t)\mathbf{W}^- \quad (9.132)$$

onde o sobrescrito + indica o processamento na direção da sequência (*forward*) e o sobrescrito -, o processamento na direção contrária (*backward*).

* *Gaussian Mixture Models.*

† *Hidden Markov Models.*

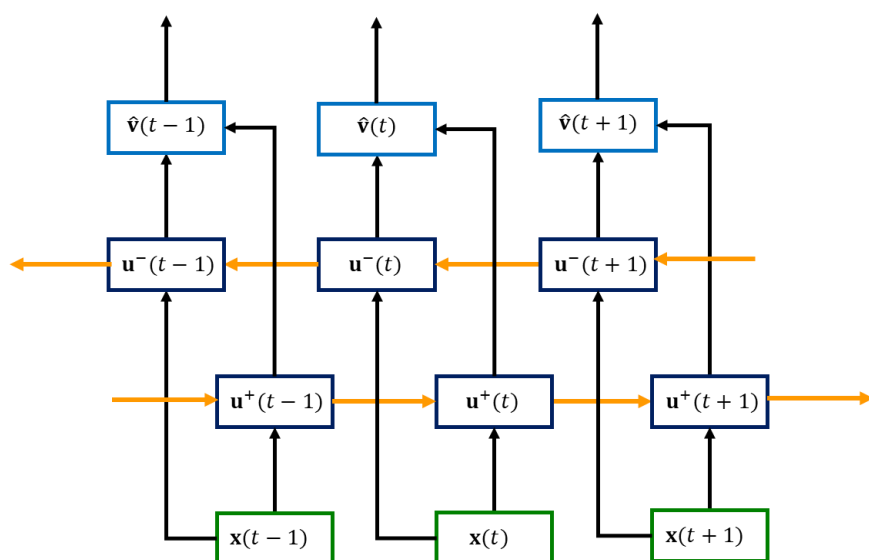


Figura 9.38: Rede neural recorrente bidirecional. Fonte: adaptado de [218].

A função de avaliação para ajuste de parâmetros da RNN padrão é calculada de forma independente para cada item da sequência (cf. Figura 9.34a), não levando em consideração a ordem do resultado na avaliação do erro. O modelo de Graves e colaboradores [218] utiliza a função de avaliação de classificação temporal (CTC) [217]. A ideia da classificação temporal é considerar a saída do modelo como uma distribuição de probabilidades das sequências possíveis de saída, condicionadas à sequência de entrada.

O modelo de Graves e colaboradores [218] avalia modelos com três a cinco camadas LSTM bidirecionais de diferentes tamanhos. O artigo apresenta os resultados de diferentes topologias para o reconhecimento de fonemas (39 classes) da base TIMIT utilizando a avaliação pela classificação temporal (CTC) e uma alternativa baseada em *beam search* [215]. Os melhores resultados foram obtidos com três camadas LSTM bidirecionais com 250 unidades cada.

Em 2014, Hannun e colaboradores [236] apresentaram o modelo chamado DeepSpeech, desenvolvido no laboratório de pesquisas da Baidu. O modelo realiza o processo de reconhecimento de fala de ponta a ponta e produz uma sequência de palavras a partir de uma entrada de áudio codificada como um espectrograma. A topologia do modelo DeepSpeech é mostrada na Figura 9.39. As três primeiras camadas são totalmente conectadas (MLP), as duas camadas seguintes são recorrentes e bidirecionais (cf. Figura

9.38) e a camada de saída, com função de ativação *softmax* para classificação de palavras utilizando a função de avaliação do modelo CTC [217]. O código do modelo para TensorFlow está disponível no GitHub.²⁰⁷

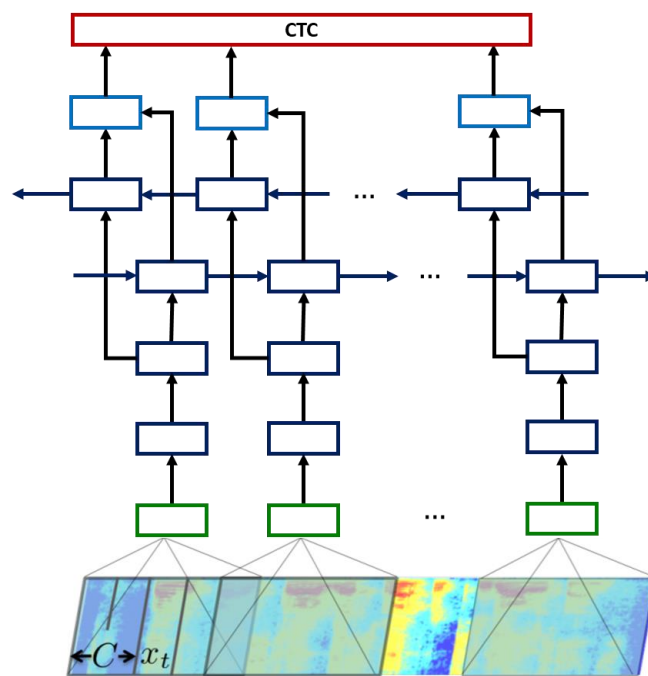


Figura 9.39: Topologia do modelo DeepSpeech. Fonte: adaptado de [236].

Em 2015, os pesquisadores da Baidu apresentaram uma nova versão do modelo, chamada de DeepSpeech 2 [16]. O mesmo modelo, com pequenas variações, foi utilizado para reconhecimento de fala nos idiomas inglês e mandarim. Esta versão é bem mais complexa que a anterior, com três camadas convolucionais e sete camadas recorrentes GRU bidirecionais, seguidas por uma camada totalmente conectada e a camada de saída com função de ativação *softmax*. O modelo também utiliza a função de avaliação CTC [217] e diversos recursos de processamento de alto desempenho que resultam numa aceleração (*speed up*) de sete vezes.

Em 2016, o laboratório de pesquisa em inteligência artificial do Facebook (FAIR)* disponibilizou o wav2letter,²⁰⁸ um modelo de reconhecimento de fala que combina redes neurais convolucionais e busca em grafos [111]. O modelo é uma CNN de 12 camadas, em que a primeira camada é uma convolução 1D diretamente sobre o sinal de áudio bruto. Em 2018, o FAIR disponibilizou uma nova versão do modelo em linguagem C++,

* Facebook Artificial Intelligence Research.

chamada de wav2letter++,²⁰⁹ como “o mais rápido sistema de reconhecimento de fala de código aberto” [443]. Ainda em 2018, pesquisadores do FAIR em Paris desenvolveram um modelo de reconhecimento de fala com três módulos de redes convolucionais: um módulo de entrada, um modelo acústico e um modelo de linguagem. O modelo completo recebe o sinal de áudio bruto como entrada e produz, como saída intermediária, letras que são subsequentemente organizadas em palavras [595].

Modelos de reconhecimento de fala têm sido desenvolvidos continuamente, diversas soluções surgem a cada ano e vários projetos estão disponíveis no GitHub²¹⁰. As aplicações de reconhecimento de fala fomentam um mercado para empresas especializadas em soluções comerciais, como a Nuance.²¹¹ As soluções mais avançadas das empresas de tecnologia são oferecidas comercialmente como, por exemplo, o Google Speech API,²¹² o Bing Speech API²¹³ da Microsoft e a plataforma Watson²¹⁴ da IBM. As soluções comerciais incluem também outros recursos, como a sintetização de fala, processamento na nuvem e integração dos modelos com outras soluções da plataforma nas aplicações do cliente.

9.3.2.6 Tradução automática

No problema de **tradução automática**, a entrada do modelo é uma sentença (sequência de palavras) em um idioma e a saída é uma sentença em outro idioma com significado similar ao da entrada. O problema é complexo, pois dois idiomas podem ter estruturas gramaticais diferentes e a tradução de uma sentença não é única, além de palavras homônimas, ambiguidade e subjetividade inerentes à linguagem natural. O modelo também deve ser capaz de trabalhar com sentenças de tamanhos diferentes, o que dificulta a utilização de alguns métodos estatísticos. Por outro lado, há uma enorme demanda por sistemas de tradução automática e as empresas de tecnologia empenham grande esforço no desenvolvimento de novas tecnologias.

Modelos de RNN para tradução automática seguem uma topologia chamada de codificador-decodificador [101][102] ou sequência a sequência [510]. Esse tipo de modelo utiliza parâmetros diferentes para codificação em um idioma e decodificação em outro.

A Figura 9.40 mostra um modelo simples em que uma sentença em português é traduzida para o inglês, sendo $t = 1, \dots, m_I$ os índices da sequência de entrada e $k = 1, \dots, m_O$ os índices de saída. O modelo é uma RNN com uma camada de vetorização, uma camada

recorrente e uma camada de saída. O último registro da entrada tem uma representação especial < FIM > que indica o término da sequência de entrada, sinalizando que o modelo deve começar a gerar a sequência de saída. No módulo de decodificação, a palavra associada ao maior valor da função *softmax* é reinjetada como entrada para o cálculo de próximo item da sequência, sendo treinada com a saída observada (*teacher forcing*). O último item de saída gera o código < FIM >, que finaliza o processo.

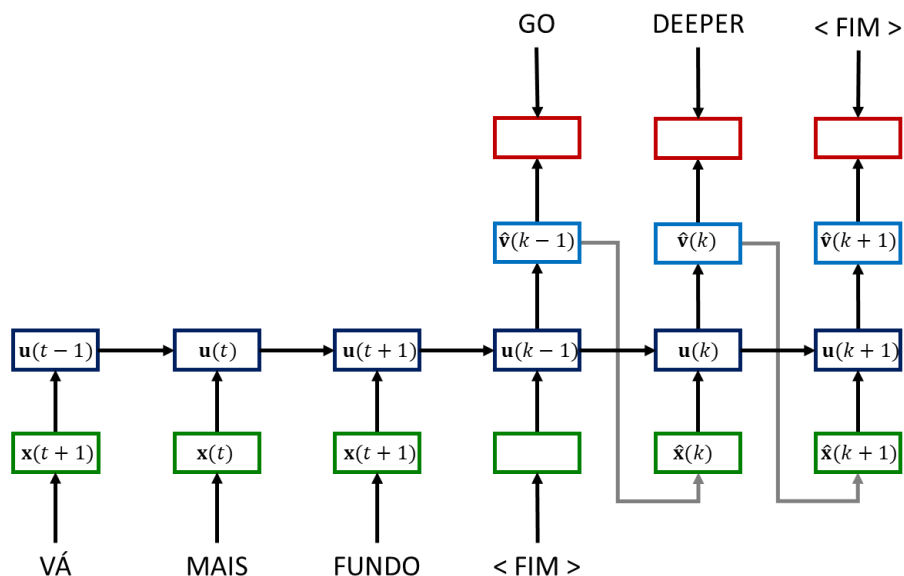


Figura 9.40: Modelo codificador-decodificador. Fonte: adaptado de [510].

A Figura 9.40 é uma simplificação didática. Para obter bons resultados, os modelos utilizados são muito mais complexos. O modelo de Sutskever e coautores [510], desenvolvido no Google em 2014, utiliza quatro camadas LSTM para codificação e decodificação, cada uma com mil unidades e vetorização também de dimensão mil, resultando num total de 384 milhões de parâmetros. O modelo foi treinado com um vocabulário de 160 mil palavras de entrada e 80 mil de saída. É executado numa máquina de oito GPUs, em que cada camada LSTM é executada numa GPU diferente e outras quatro GPUs se dedicam apenas ao cálculo da função *softmax* de saída [510]. O modelo obtém um resultado de 37% de acerto na estatística BLEU [428] na tradução de inglês-francês do conjunto WMT'14,²¹⁵ superando o modelo de referência.

O modelo de Cho e colaboradores [102], também de 2014, utiliza uma conexão entre o vetor de estados do último termo do codificador, correspondente ao < FIM >, e todas as entradas do módulo decodificador. A ideia é que o vetor de estados do último item da sequência de entrada resume toda a informação anterior. Esse conceito foi estendido

por Bahdanau e coautores [25], que utiliza um RNN bidirecional com uma camada de atenção conectada à saída das camadas recorrente dos módulos codificador (na direção *forward*) e decodificador. A saída da camada de atenção é utilizada no cálculo da saída do módulo decodificador. A Figura 9.41 ilustra o modelo, onde as conexões bidirecionais do codificador não são mostradas, para simplificar o desenho.

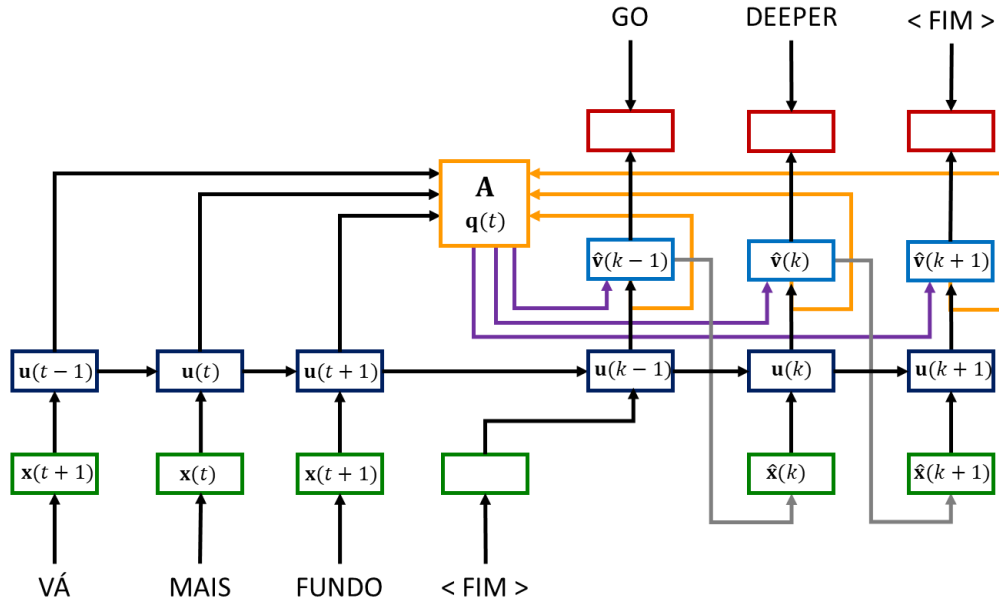


Figura 9.41: Modelo codificador-decodificador com modelo de atenção.

A camada de atenção conecta os módulos de codificação e decodificação e permite que o modelo focalize no mapeamento de partes do texto que está sendo traduzido.

A saída do módulo de decodificação é calculada como $\hat{\mathbf{v}}(k) = \mathbf{f}(\hat{\mathbf{v}}(k-1), \mathbf{u}(k), \mathbf{q}(k))$, em que $\mathbf{u}(k)$ é a saída da camada recorrente do decodificador e $\mathbf{q}(k)$ é a saída da camada de atenção, calculada como [25]:

$$\mathbf{q}(k) = \sum_{t=1}^{m_I} \alpha_{kt} \mathbf{u}(t) \quad (9.133)$$

onde $\mathbf{u}(t)$ é a saída da camada recorrente do codificador; $t = 1, \dots, m_I$ são os índices da sequência de entrada; e $k = 1, \dots, m_O$, os índices de saída.

Os parâmetros de alinhamento α_{kt} resultam numa matriz $\mathbf{A}$ de dimensão $m_O \times m_I$, na qual os elementos são calculados pela função *softmax* [25]:

$$\alpha_{kt} = \text{softmax}(g(\mathbf{u}(k), \mathbf{u}(t))) \quad (9.134)$$

onde $g(\mathbf{u}(k), \mathbf{u}(t))$ é uma função de alinhamento que avalia quanto as entradas em torno da posição t estão associadas com a saída na posição k .

A Figura 9.41 é uma simplificação do modelo de Bahdanau e colaboradores [25], que utiliza uma rede neural recorrente bidirecional no módulo codificador com mil unidades do tipo GRU. O módulo decodificador é também utiliza mil unidades GRU, mas não é bidirecional. A camada de atenção é uma rede MLP de duas camadas de processamento com mil unidades na camada intermediária. A função de alinhamento é ajustada por uma rede neural MLP em conjunto com o modelo codificador-decodificador.

Os resultados de tradução inglês-francês no conjunto WNT'14 mostram que o modelo é capaz de alinhar corretamente a posição das palavras nos dois idiomas e traduzir textos mais longos, em comparação com o modelo codificador-decodificador de Cho e colaboradores [102]. A Figura 9.42, reproduzida do artigo [25], mostra dois exemplos de tradução de inglês (nas colunas) para francês (nas linhas). Cada *pixel* na Figura 9.42 mostra o valor do parâmetro de alinhamento α_{tk} calculado na camada de atenção pela equação (9.134).

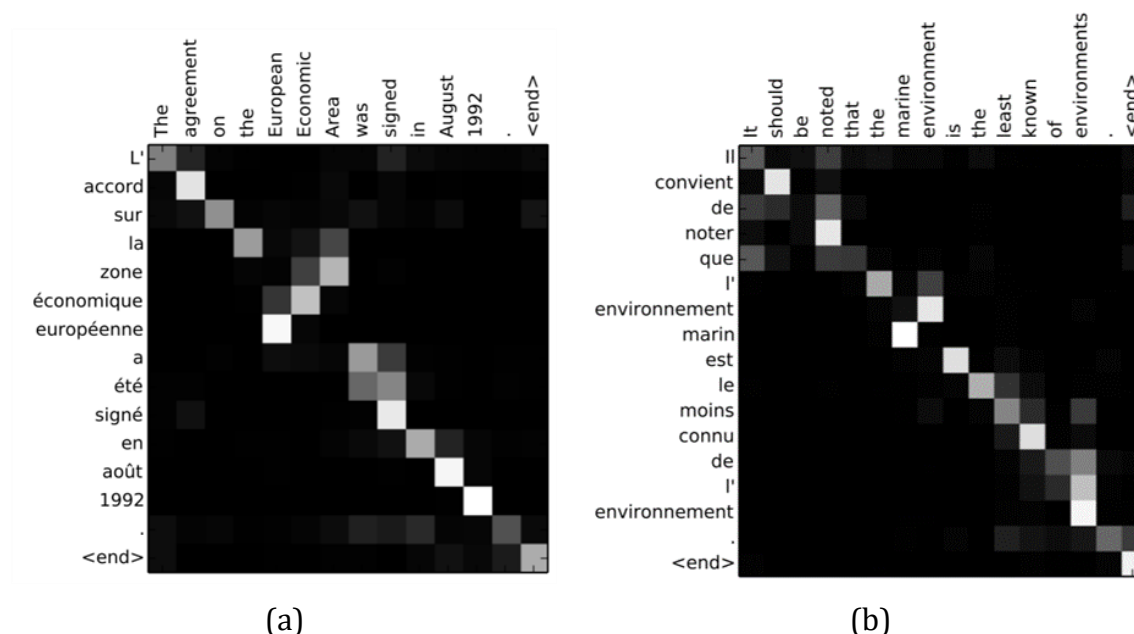


Figura 9.42: Resultados dos parâmetros de alinhamento. Fonte: reproduzido de [25].

Observe que, na Figura 9.42a, o alinhamento é capaz de inverter corretamente a ordem das palavras, de forma que “*European Economic Area*” é traduzido como “*zone économique européenne*”. O modelo de atenção de Bahdanau, Cho e Bengio [25] tem sido

muito utilizado em outras aplicações de NLP, como reconhecimento de fala [81] e legendagem automática [571].

Em 2016, o Google apresentou a nova versão do sistema de tradução automática utilizado pela empresa em seus aplicativos, chamado de GNMT [566], que incorpora os resultados alcançados até então no desenvolvimento de redes neurais profundas num modelo de grandes proporções, como mostra a Figura 9.43, reproduzida do artigo [566]. O modelo é uma versão estendida da topologia sequência a sequência [510] com oito camadas LSTM nos módulos de codificação e decodificação e um modelo de atenção. A primeira camada do módulo de codificação é bidirecional, o modelo utiliza conexões residuais nos dois módulos e o modelo de atenção é similar ao de Bahdanau e colaboradores [25]. Cada camada é executada em uma GPU diferente, de forma que a camada superior só inicia seu processamento quando a camada inferior termina. O artigo descreve ainda diversos detalhes de implementação do modelo para melhorar o desempenho computacional [566]. Além de GPUs, o processamento utiliza também CPU e um *hardware* proprietário do Google chamado unidade de processamento tensorial (TPU), atualmente disponível na nuvem da empresa.²¹⁶

O sistema GNMT é multi-idioma e pode inferir a tradução entre dois idiomas sem que tenha sido treinado especificamente nessa combinação. Por exemplo, sendo treinado para português-inglês e inglês-espanhol, o modelo é capaz de realizar traduções português-espanhol com precisão razoável. O sistema está em uso, e sendo aperfeiçoado, desde 2017 nos aplicativos do Google.

Em 2017, pesquisadores do Facebook desenvolveram o modelo Fairseq de tradução automática baseado em redes convolucionais [197], cujo código (em PyTorch) e modelos pré-treinados estão disponíveis no GitHub.²¹⁷ As redes convolucionais realizam processamento em paralelo, o que permite desempenho computacional melhor que os modelos de redes neurais recorrentes, que são sequenciais.

Há diversos recursos para sistemas de tradução automática [410][442]. Os modelos codificador-decodificador discutidos nesta seção estão disponíveis num tutorial TensorFlow,²¹⁸ que apresenta uma implementação passo a passo no com modelos pré-treinados. O grupo de NLP de Stanford também disponibiliza uma página com diversos recursos, como códigos e modelos pré-treinados.²¹⁹

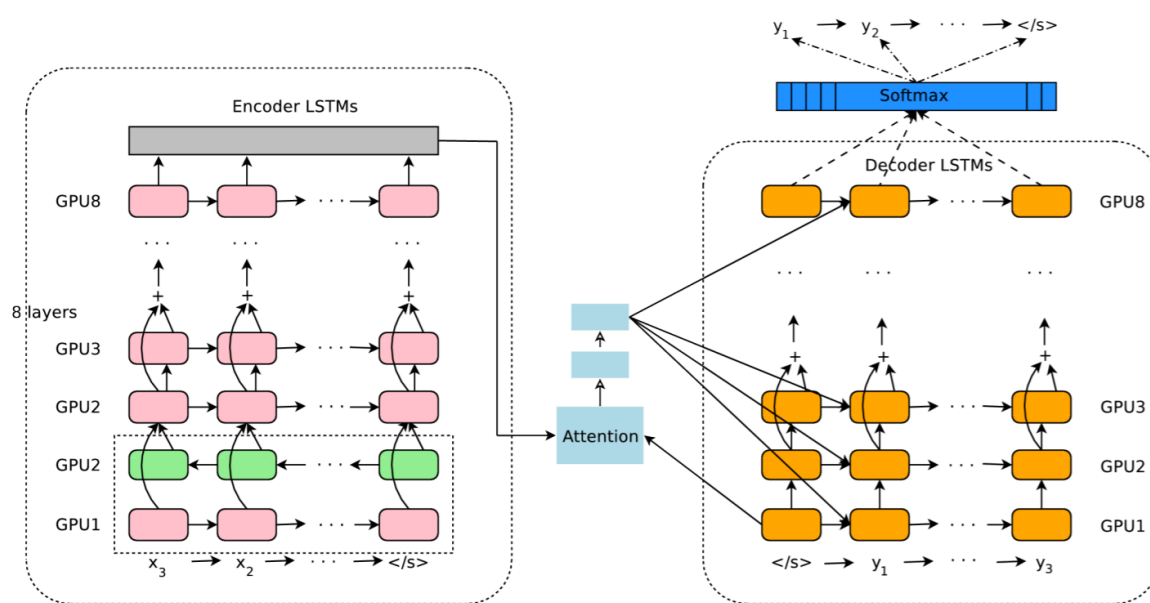


Figura 9.43: Modelo de tradução automática do Google. Fonte: reproduzido de [566].

Há também soluções comerciais de tradução automática, como na plataforma de nuvem Azure,²²⁰ da Microsoft, e na plataforma de serviços cognitivos Watson,²²¹ da IBM.

O desenvolvimento de modelos de redes neurais profundas tem avançado continuamente com integração de resultados, desenvolvimento de novos conjuntos de dados e demanda crescente em diversos setores de atividade e ciências. As aplicações apresentadas nesta seção resumem as principais técnicas e demonstram o potencial de redes neurais recorrentes para o processamento de linguagem natural.

Os modelos de redes neurais profundas foram apresentados em problemas de aprendizado supervisionado. Entretanto, a oferta de dados não anotados é muito maior que a de dados anotados e há problemas em que a solução exige algoritmos de aprendizado não supervisionado. Nesse contexto, há modelos de redes neurais que são utilizados para a extração de características da entrada, como no modelo word2vec. Essa abordagem é discutida na próxima seção.

9.3.3 Aprendizado da representação

Grande parte do tempo empregado na solução de um problema em ciência de dados é dedicado à coleta e ao pré-processamento de dados e à busca pelos atributos (ou características) que representam o problema.

A Figura 9.44 apresenta uma comparação entre diferentes tipos de modelos. As caixas em cinza são ajustadas a partir de dados e as caixas em branco são definidas por especialistas ou pelas condições do problema. Na abordagem à base de conhecimento, os atributos são definidos por especialistas e utilizados para o cálculo da saída. Modelos físicos são exemplos desse tipo de abordagem. Técnicas clássicas de reconhecimento de padrões calculam a saída em duas etapas: extração de características por especialistas e análise de características [146]. A extração de características por especialistas é o cálculo de atributos relevantes, que são utilizados como variáveis de entrada para o modelo, que mapeia as características na saída. Modelos lineares de classificação e regressão se enquadram nessa categoria.

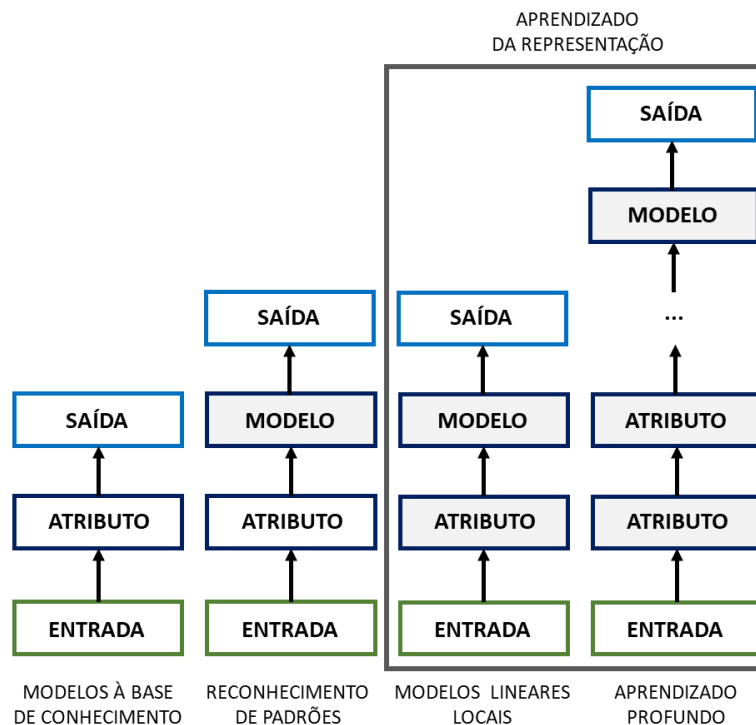


Figura 9.44: Aprendizado da representação. Fonte: adaptado de [212].

Na abordagem conhecida como **aprendizado da representação** [35], os atributos e o modelo são ajustados a partir de uma base de dados. Nos modelos lineares locais, o

vetor de características é calculado diretamente a partir de dados. O modelo RBF realiza o mapeamento explícito do espaço de características e o modelo SVM reproduz o espaço de características a partir da matriz de núcleo. Em redes neurais profundas, as camadas são representações da entrada e cada camada faz uma representação com maior nível de abstração (cf. Figura 9.13).

Em alguns casos, o ajuste de parâmetros é feito de forma supervisionada, mas o objetivo do modelo é uma representação da entrada. O word2vec é um exemplo desse tipo de modelo em que uma representação da entrada é obtida pelos parâmetros da rede neural ajustada para a previsão de uma palavra em função das vizinhas. Essa abordagem, chamada de auto supervisionada, tem sido aplicada em diversas áreas [315].

Em 2006, Hinton e Salakhutdinov [251] apresentaram uma técnica de ajuste de parâmetros de redes neurais de múltiplas camadas de forma não supervisionada, uma camada de cada vez. O modelo é obtido pelo empilhamento de modelos de máquina de Boltzmann restrita (RBM)*, que é funcionalmente similar a uma rede neural MLP. O artigo [251] é um marco no desenvolvimento de redes neurais profundas por apresentar uma técnica geral não supervisionada para inicialização de parâmetros para o algoritmo da retropropagação.

Em 2014, Goodfellow e colaboradores [211] apresentaram o modelo de redes adversárias (GAN),[†] inspirado na teoria dos jogos, em que dois modelos adversários são ajustados simultaneamente por uma função de avaliação minimax. O modelo GAN é um modelo capaz de gerar amostras de distribuições de probabilidade complexas.

O aprendizado da representação tem se desenvolvido rapidamente. Esta seção apresenta os trabalhos pioneiros dos modelos autocodificadores e redes adversárias.

9.3.3.1 Autocodificadores

Um **autocodificador** é um modelo que reproduz a entrada a partir de uma representação interna das variáveis. O autocodificador realiza o cálculo $\hat{\mathbf{x}} = f(\mathbf{x}) \approx \mathbf{x}$ em duas etapas: uma função de codificação ou **codificador**, que calcula uma representação interna da entrada, e uma função de decodificação ou **decodificador**, que calcula uma aproximação da entrada a partir da representação interna.

* *Restricted Boltzmann Machine.*

† *Generative Adversarial Networks.*

A análise de componentes principais (ACP), apresentada na seção 2.4.1, é um autocodificador linear. No caso da ACP, a matriz de componentes principais realiza a codificação das variáveis de entrada numa representação interna de menor dimensão, que pode ser decodificada através da matriz transposta, uma vez a transformação é linear e ortogonal.

Reescrevendo a ACP, cada registro da variável de entrada (padronizada) é mapeado na sua representação interna pela transformação linear (cf. equação (2.17)):

$$\mathbf{u}(t) = \mathbf{x}(t)\mathbf{\Theta} \quad (9.135)$$

onde $\mathbf{u}(t) \in \mathcal{R}^n$ é a representação interna, que tem dimensão menor que a entrada, isto é, $n < p$; a matriz $\mathbf{\Theta}$ de dimensão $p \times n$ é a matriz de transformação linear ortogonal cujas colunas representam os componentes principais.

A função de decodificação é calculada como (cf. Figura 2.21):

$$\hat{\mathbf{x}}(t) = \mathbf{u}(t)\mathbf{\Theta}^T \quad (9.136)$$

Um autocodificador não linear pode ser ajustado por uma rede neural MLP de duas camadas (cf. Figura 9.45): a primeira camada realiza uma representação interna da entrada e a segunda camada calcula uma aproximação da entrada:

$$\begin{aligned} \mathbf{u}(t) &= \mathbf{h}^{(1)}(\mathbf{b}^{(1)} + \mathbf{x}(t)\mathbf{W}^{(1)}) \\ \hat{\mathbf{x}}(t) &= \mathbf{h}^{(2)}(\mathbf{b}^{(2)} + \mathbf{u}(t)\mathbf{W}^{(2)}) \end{aligned} \quad (9.137)$$

onde $\mathbf{u}(t)$ é a representação interna do autocodificador; $\mathbf{W}^{(1)}$ e $\mathbf{W}^{(2)}$ ($\mathbf{b}^{(1)}$ e $\mathbf{b}^{(2)}$) são as matrizes de conexões (parâmetros das unidades bias), respectivamente, das funções de codificação e decodificação; e $\mathbf{h}^{(1)}$ e $\mathbf{h}^{(2)}$ são as funções de ativação vetorial da primeira e da segunda camadas.

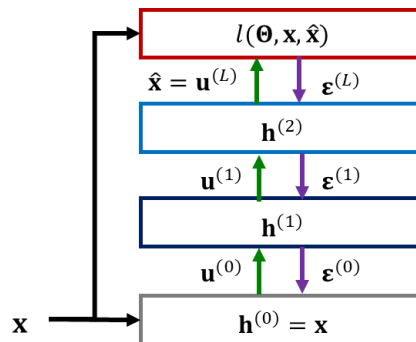


Figura 9.45: Autocodificador MLP.

Tipicamente, $\mathbf{h}^{(2)}$ é a função de ativação linear e os parâmetros são ajustados de tal forma que $\mathbf{W}^{(2)} = \mathbf{W}^{(1)T}$, pela minimização do erro quadrático (EQ):

$$l(\Theta) = \frac{1}{2} \sum_{t=1}^N \|\mathbf{x}(t) - \hat{\mathbf{x}}(t)\|^2 \quad (9.138)$$

onde Θ é o conjunto de parâmetros do modelo.

A camada intermediária da MLP realiza uma representação distribuída das variáveis de entrada, que pode ser utilizada como inicialização de modelos preditivos ou como entrada para algoritmos de análise de agrupamentos.

Uma aplicação direta de autocodificadores é a extração de ruído, em modelos chamados de *denoising autoencoder* (DAE) [542]. Nesse caso, o autocodificador é calculado como $\hat{\mathbf{x}} = f(\tilde{\mathbf{x}})$, onde $\tilde{\mathbf{x}}$ é uma perturbação da entrada obtida pela adição de um ruído aleatório ou *dropout* [251]. A avaliação do modelo é realizada pelo EQ, como na equação (9.138).

Na topologia usual, chamada de representação **compacta**, a dimensão da representação é menor que a dimensão da entrada, de forma que o autocodificador realiza uma compactação das variáveis de entrada. Na representação **esparsa**, a dimensão da representação é maior que a da entrada e uma restrição de regularização deve ser adicionada para impedir que a representação contenha uma cópia da entrada.

Autocodificadores esparsos podem ser obtidos pela regularização L^2 da função de avaliação. No autocodificador comprimido (CAE) [456], o termo de regularização é calculado em função da matriz jacobiana:

$$l(\Theta) = \frac{1}{2} \sum_{t=1}^N \|\mathbf{x}(t) - \hat{\mathbf{x}}(t)\|^2 + \mu \left\| \frac{\partial \mathbf{u}^{(1)}}{\partial \mathbf{x}} \right\|^2 \quad (9.139)$$

onde $\left[\frac{\partial \mathbf{u}^{(1)}}{\partial \mathbf{x}} \right] = \left[\frac{\partial \mathbf{u}^{(1)}}{\partial \mathbf{z}^{(1)}} \right] \left[\frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{x}} \right]^T$ é a matriz jacobiana e $\left\| \frac{\partial \mathbf{u}^{(1)}}{\partial \mathbf{x}} \right\|^2 = \|\mathbf{h}'^{(1)} \mathbf{W}^{(1)}\|^2$.

O ajuste de parâmetros de autocodificadores pode ser realizado de forma incremental [37][251]. No modelo resultante, cada camada representa uma característica em maior nível de abstração, calculada a partir da camada anterior, como descrito a seguir.

9.3.3.2 Autocodificadores empilhados

Redes neurais MLP com várias camadas obtêm resultados piores que modelos de uma ou duas camadas intermediárias [37], devido ao problema de diluição do gradiente no algoritmo da retropropagação. O problema acontece em razão de o gradiente local se tornar menor a cada retropropagação, de forma que os parâmetros das camadas mais internas do modelo quase não são alterados. Uma estratégia para o ajuste de redes neurais profundas é realizar o treinamento supervisionado, adicionando camadas de forma incremental [339]. Entretanto, esse tipo de treinamento não obtém uma representação da entrada em níveis hierárquicos de abstração.

A estratégia de pré-treinamento não supervisionado foi proposta em 2006 por Hinton e Salakhutdinov [251]. Essa técnica permite realizar o treinamento de topologias profundas de modo que cada camada é ajustada como um autocodificador da camada anterior e realiza uma abstração desta. Os resultados mostram que o pré-treinamento não supervisionado é uma técnica efetiva para a inicialização dos parâmetros do modelo de redes neurais profundas [37]. As camadas intermediárias podem ser utilizadas em outros tipos de tarefa, como análise de agrupamentos [35].

Autocodificadores empilhados são modelos de redes neurais profundas que utilizam o pré-treinamento não supervisionado para o ajuste de parâmetros [251]. A Figura 9.46 mostra a topologia completa em que um conjunto de autocodificadores gera os parâmetros iniciais dos modelo codificador e decodificador. O ajuste de parâmetros é realizado em etapas. A primeira camada é ajustada como um autocodificador da entrada. A representação interna do primeiro autocodificador é utilizada como entrada para o ajuste da segunda camada, e assim sucessivamente. A matriz de conexões de cada autocodificador é usada para inicializar a camada correspondente do codificador. Cada camada do decodificador é inicializada com a matriz de conexões transposta da camada correspondente do codificador, considerando que essa restrição foi utilizada no ajuste do autocodificador. O modelo final é obtido com a adição de uma camada de saída que produz o erro para o ajuste fino dos parâmetros pelo algoritmo da retropropagação.

Em modelos totalmente não supervisionados, a própria entrada pode ser utilizada para produzir o erro, como mostra a linha tracejada na Figura 9.46 [251]. Nesse caso, a última camada do codificador é a representação mais abstrata da entrada e pode ser utilizada, por exemplo, em aplicações de análise de agrupamentos. Usualmente, utiliza-se

a representação compacta nos autocodificadores, de forma que a última camada tem dimensão bem menor que a entrada, o que facilita o agrupamento.

Há dois fluxos de retropropagação de erro na Figura 9.46: no ajuste dos modelos auto-codificadores, que são sempre de duas camadas ($\epsilon^{(1)}$ e $\epsilon^{(2)}$), e na retropropagação do erro da camada de saída ($\epsilon'^{(l)}$), que realiza o ajuste fino dos parâmetros dos modelos codificador e decodificador.

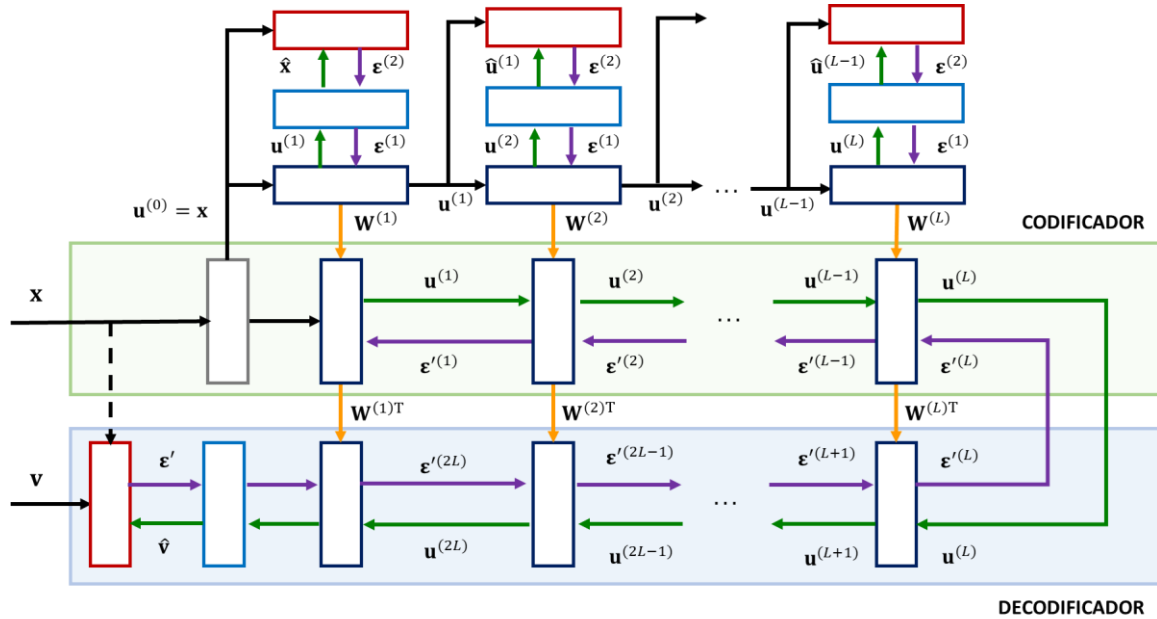


Figura 9.46: Pré-treinamento não supervisionado.

As redes neurais convolucionais também são utilizadas em autocodificadores convolucionais [371]. O codificador utiliza uma topologia CNN e o decodificador realiza a operação de deconvolução. A convolução é uma operação linear e a deconvolução pode ser calculada com a matriz de conexões transposta. No decodificador, a função de ativação é calculada após a deconvolução, como uma camada convolucional usual. Em modelos que utilizam a camada de *pooling*, uma camada de aumento de dimensionalidade, chamada de *upsampling*, é adicionada de forma correspondente.

Os modelos SegNet [24] e U-Net [458] são utilizados para segmentação supervisionada de imagens, com que a saída formada por uma imagem da dimensão da entrada e com número de canais igual ao número de classes. A topologia é similar a um autocodificador, mas SegNet e U-Net são modelos de classificação. Para cada pixel, a probabilidade de cada classe é calculada pela função *softmax*. A Figura 9.47 mostra o modelo SegNet, em que o módulo codificador é uma CNN VGG 16 e as ligações entre o codificador e

decodificador representam os índices utilizados nas camadas de *max-pooling*. O código e modelos pré-treinados SegNet²²² e U-Net.²²³ estão disponíveis e têm sido bastante utilizados em diversas aplicações.

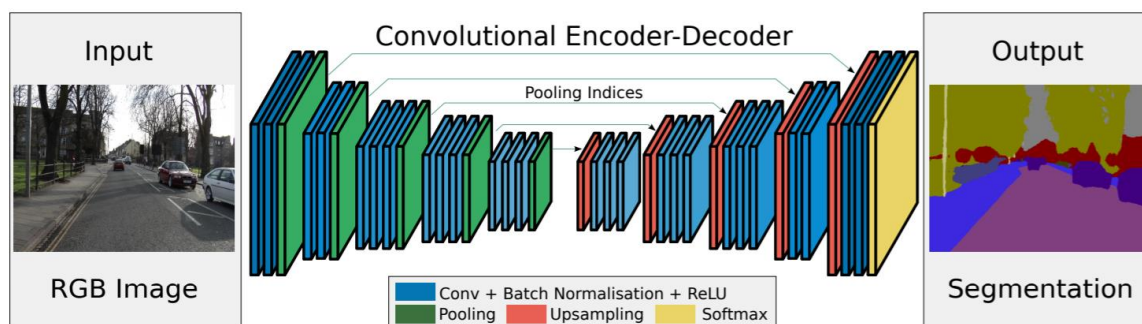


Figura 9.47: Modelo SegNet. Fonte: reproduzido de [24].

O aprendizado da representação pode ser utilizado para o ajuste de um modelo da função de distribuição de probabilidades de uma amostra, como em modelos geradores, apresentados a seguir.

9.3.3.3 Modelos geradores

Modelos geradores são modelos que podem gerar novas amostras de dados de acordo com uma representação interna da função de distribuição de probabilidades. Métodos clássicos de estatística utilizam uma formulação explícita da função de distribuição de probabilidades cujos parâmetros são ajustados pelo máximo da verossimilhança.

Em espaços de alta dimensionalidade, o problema é muito mais complexo. Gerar uma imagem de um rosto implica ajustar uma distribuição de probabilidades num espaço da dimensão do número de *pixels* da imagem. A representação explícita da distribuição, neste caso, é computacionalmente intratável e modelos de redes neurais profundas podem ser utilizados para realizar uma aproximação da representação explícita ou implícita da distribuição.

Um modelo de aproximação da representação explícita da distribuição é o autocodificador variacional (VAE) [307][308][452]. Os VAE são autocodificadores, no sentido de que o modelo é ajustado para reproduzir a entrada. Entretanto, no modelo VAE [308], a entrada para o módulo decodificador é o vetor de **variáveis latentes** $\mathbf{z} \sim N(\boldsymbol{\mu}, \boldsymbol{\sigma}\mathbf{I})$, que é uma amostra de uma distribuição normal de matriz de covariâncias diagonal, cujos parâmetros de média e variância são aproximados pelo codificador. Dessa forma,

o codificador calcula os parâmetros da distribuição que será utilizada para gerar a entrada do decodificador e assim gerar amostras aleatórias da distribuição do modelo.

A Figura 9.48 mostra a topologia do modelo VAE. A camada oval representa uma camada de amostragem de uma distribuição normal cuja saída é o vetor de variáveis latentes. O ajuste de parâmetros é realizado por uma adaptação do algoritmo da retropropagação para lidar com a amostragem da variável latente [307][452]. Após o ajuste de parâmetros, apenas o decodificador é utilizado a partir da variável latente gerada aleatoriamente por uma distribuição normal, cujos parâmetros de média e variância representam a codificação da entrada.

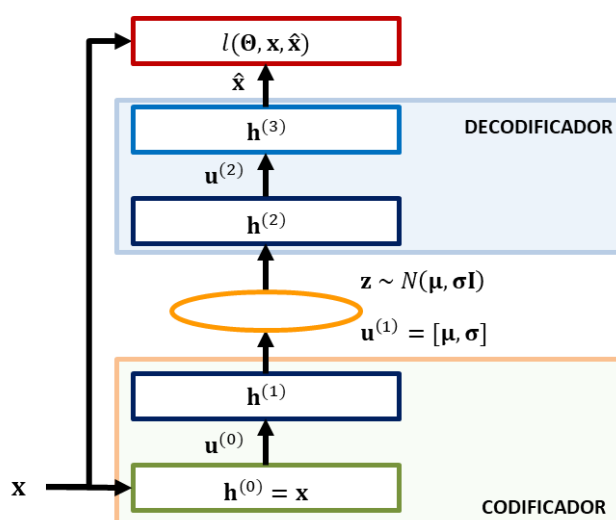


Figura 9.48: Modelo VAE.

A Figura 9.49 [308] mostra dois exemplos de modelos VAE com um espaço de variáveis latentes de duas dimensões ajustados com um conjunto de dados de imagens de rostos (Figura 9.49a) e com a base de dados MNIST (Figura 9.49b). Para cada valor da variável latente $\mathbf{z} = (z_1, z_2)$ numa grade regular, uma imagem é gerada pelo módulo decodificador do modelo. Observe que pequenas variações nas componentes da variável latente produzem pequenas variações na saída. No modelo de faces da Figura 9.49a, a componente horizontal parece estar relacionada com a rotação do rosto e a componente vertical, com a expressão. No modelo da base MNIST, a combinação de variáveis latentes produz um algarismo de saída e podemos observar uma transição suave dos algarismos de acordo com a variação das variáveis latentes.

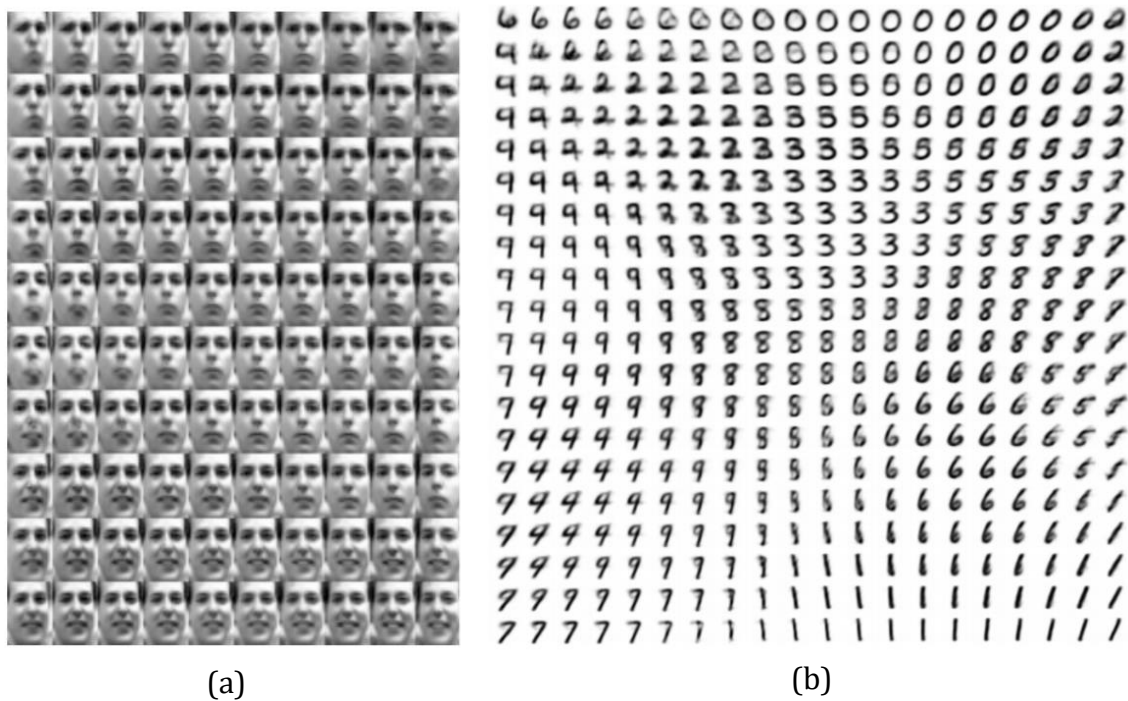


Figura 9.49: Modelo VAE treinado com variáveis latentes 2D: (a) conjunto de dados de faces de Brendan Frey; (b) conjunto de dados MNIST. Fonte: reproduzido de [308].

As redes neurais adversárias (GAN) realizam uma representação implícita da distribuição de probabilidades [210][211][212]. A topologia GAN utiliza duas redes neurais adversárias: o modelo **gerador** $G(\mathbf{z}, \Theta_G)$ e o modelo **discriminante** $D(\mathbf{x}, \Theta_D)$. Os modelos são adversários porque o primeiro visa gerar amostras realistas e o segundo deve identificar corretamente se uma amostra é observada ou foi gerada pelo modelo.

A Figura 9.50 mostra o processamento do modelo GAN. A base de dados é formada por um conjunto de amostras de uma variável aleatória de distribuição desconhecida, mas observada $\mathbf{x} \sim p(\mathbf{x})$. A partir de uma variável latente $\mathbf{z}$ amostrada de uma distribuição conhecida ($\mathbf{z} \sim \pi(\mathbf{z})$), o objetivo do modelo gerador é realizar uma representação implícita da distribuição observada e gerar uma aproximação $\hat{\mathbf{x}} \sim \hat{p}(\mathbf{x})$ dos registros da base de dados. O modelo discriminante é ajustado para reconhecer as amostras observadas da base de dados e as amostras “falsas” produzidas pelo modelo gerador. No equilíbrio, o modelo discriminante não é capaz de diferenciar as amostras e o modelo gerador produz realizações plausíveis da base de dados.

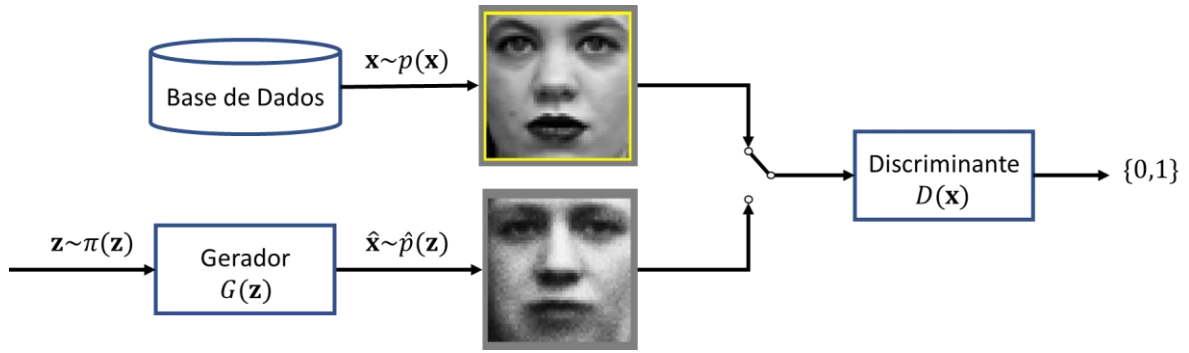


Figura 9.50: Modelos de redes neurais adversárias. Fonte: adaptado de [210].

O ajuste de parâmetros do modelo GAN é baseado na teoria dos jogos e busca o equilíbrio entre dois agentes, como num jogo de soma zero. Goodfellow e colaboradores [211] apresentam o ajuste de modelos GAN pelo Algoritmo 9.4.

No Algoritmo 9.4, a cada época, são processadas $nstep$ iterações de ajuste de parâmetros do modelo discriminante, mantendo fixos os parâmetros do modelo gerador. Os parâmetros são ajustados pela **maximização** da função de avaliação do discriminante:

$$l_D(\Theta_D) = \frac{1}{2} \sum_{t=1}^N \log(D(\mathbf{x}(t), \Theta_D)) + (1 - \log(D(\hat{\mathbf{x}}(t), \Theta_D))) \quad (9.140)$$

onde $\hat{\mathbf{x}}(t) = G(\mathbf{z}(t), \Theta_G)$ é o resultado do gerador e Θ_G é fixo.

O conjunto de dados de cada classe é formado por duas amostras com 50% dos registros cada: uma gerada a partir do banco de dados e outra gerada pelo modelo. A variável de saída é $v(t) = 0$ quando o registro vem do gerador e $v(t) = 1$ quando o registro vem da base de dados. Dessa forma, o modelo discriminante acerta quando $D(\hat{\mathbf{x}}(t), \Theta_D) \approx 0$ e $D(\mathbf{x}(t), \Theta_D) \approx 1$, que corresponde ao máximo da função de avaliação $l_D(\Theta_D)$ na equação (9.140).

A estratégia de maximização da função de avaliação da equação (9.140) é ótima para o modelo discriminante, mas há diferentes versões para a função de avaliação do modelo gerador [212]. No jogo de soma zero, $l_G(\Theta_G) = -l_D(\Theta_D)$ e os parâmetros do modelo gerador são ajustados pela **minimização** da seguinte função de avaliação:

$$l_G(\Theta_G) = \frac{1}{2} \sum_{t=1}^N \log(1 - D(\hat{\mathbf{x}}(t), \Theta_D)) \quad (9.141)$$

onde $\hat{\mathbf{x}}(t) = G(\mathbf{z}(t), \Theta_G)$ é o resultado do gerador e Θ_D é fixo.

Algoritmo 9.4: Algoritmo de treinamento de redes adversárias (GAN). Fonte: traduzido de [211]

Entrada:	Conjunto de treinamento $\mathcal{T} = \{\mathbf{x}(t), t = 1 \dots N\}$
Saída:	Conjunto de parâmetros $\Theta = \{\Theta_D, \Theta_G\}$
01	Início
02	Para $k = 1, \dots, itmax$
03	Para $i = 1, \dots, nstep$
04	Realiza amostragem $\mathcal{Z} = \{\mathbf{z}(t), t = 1 \dots M\}$
05	Realiza amostragem $\mathcal{X} = \{\mathbf{x}(t), t = 1 \dots M\}$
06	Atualiza os parâmetros de $D(\mathbf{x}, \Theta_D)$
07	Fim Para
08	Realiza amostragem $\mathcal{Z} = \{\mathbf{z}(t), t = 1 \dots M\}$
09	Atualiza os parâmetros de $G(\mathbf{z}, \Theta_G)$
10	Fim Para
11	Retorna o conjunto de parâmetros $\Theta = \{\Theta_D, \Theta_G\}$
12	Fim

A função de avaliação do gerador $l_G(\Theta_G)$ é mínima quando o erro do discriminante é máximo, ou seja, quando $D(\hat{\mathbf{x}}(t), \Theta_D) \approx 1$. O modelo é chamado de redes adversárias porque o discriminante é uma rede neural que visa aproximar $D(\hat{\mathbf{x}}(t), \Theta_D) \approx 0$, pelo ajuste de Θ_D , e o gerador visa aproximar $D(\hat{\mathbf{x}}(t), \Theta_D) \approx 1$ pelo ajuste dos parâmetros Θ_G do modelo gerador $\hat{\mathbf{x}}(t) = G(\mathbf{z}(t), \Theta_G)$.

Topologicamente, o resultado da otimização da função de custo total é o chamado **ponto de sela**, em que a função de avaliação do discriminante $l_D(\Theta_D)$ é máxima e a função de avaliação do gerador $l_G(\Theta_G)$ é mínima.

O funcionamento do modelo GAN unidimensional é mostrado na Figura 9.51, reproduzida de [211]. O modelo gerador é apresentado em verde, o modelo discriminante em azul e a distribuição dos dados observados em preto. A Figura 9.51a mostra o modelo próximo da convergência, em que G é um modelo similar e D é um classificador parcialmente preciso. A Figura 9.51b mostra o ajuste do modelo discriminante após a convergência no laço interno do algoritmo. A atualização do modelo gerador aproxima a distribuição dos dados, como mostra a Figura 9.51c. Na convergência, mostrada na Figura 9.51d, a função discriminante é incapaz de distinguir as duas distribuições.

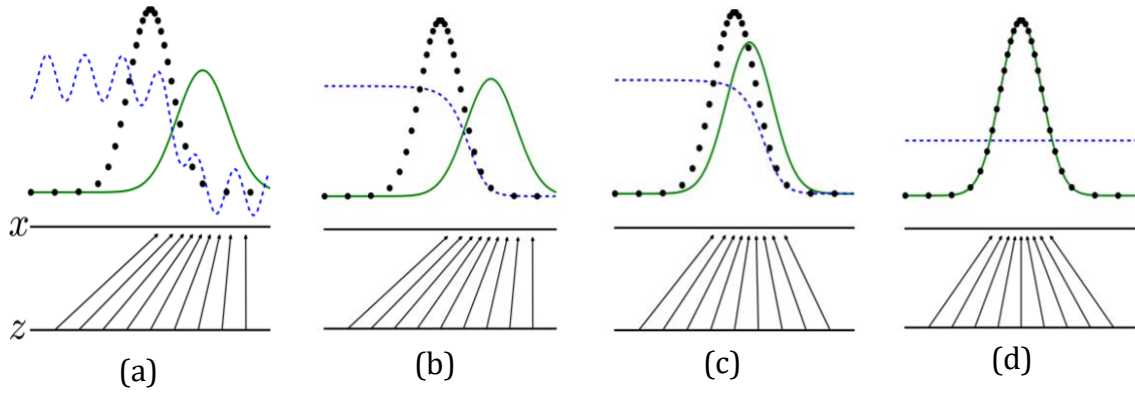


Figura 9.51: Convergência do modelo GAN unidimensional. Fonte: reproduzido de [211].

A equação (9.141) apresenta a solução do jogo de soma zero, também chamado de minimax. Para o ajuste de redes neurais, no entanto, a função de avaliação definida pela equação (9.141) não é adequada no início do treinamento, pois $l_G(\Theta_G) \approx 0$ quando $D(\hat{\mathbf{x}}(t), \Theta_D) \approx 0$ e o gradiente se anula. Dessa forma, Goodfellow e colaboradores [211] propõem uma solução que gera o mesmo resultado que a anterior, mas é melhor para o cálculo do gradiente, pois evita o gradiente nulo no início do treinamento. Nesse caso, a função de avaliação do gerador é calculada como:

$$l_G(\Theta_G) = -\frac{1}{2} \sum_{t=1}^N \log(D(\hat{\mathbf{x}}(t), \Theta_D)) \quad (9.142)$$

onde $\hat{\mathbf{x}}(t) = G(\mathbf{z}(t), \Theta_G)$ é o resultado do gerador e Θ_D é fixo.

Os resultados do modelo GAN ajustados pelo Algoritmo 9.4 são mostrados em dois exemplos na Figura 9.52, reproduzida de [211]. A Figura 9.52a apresenta os resultados na base MNIST e a Figura 9.52b mostra os resultados no conjunto de dados de reconhecimento facial *Faces Dataset* [509]. A coluna marcada em amarelo (a direita) representa registros das bases de dados e as demais figuras foram geradas pelo modelo gerador, a partir de amostras da variável latente com distribuição uniforme.

O modelo GAN se tornou muito popular a partir dos trabalhos originais de Goodfellow e colaboradores [210][211][212] e diversas variantes²²⁴ têm sido propostas desde então. Os modelos recentes têm obtido resultados impressionantes. Em 2017, Karras e colaboradores [296], pesquisadores da NVIDIA, desenvolveram o modelo GAN chamado de crescimento progressivo. O resultado do modelo ajustado com uma base de

imagens de celebridades CelebA²²⁵ é mostrado na Figura 9.53, que exibe as imagens geradas artificialmente pelo modelo gerador.

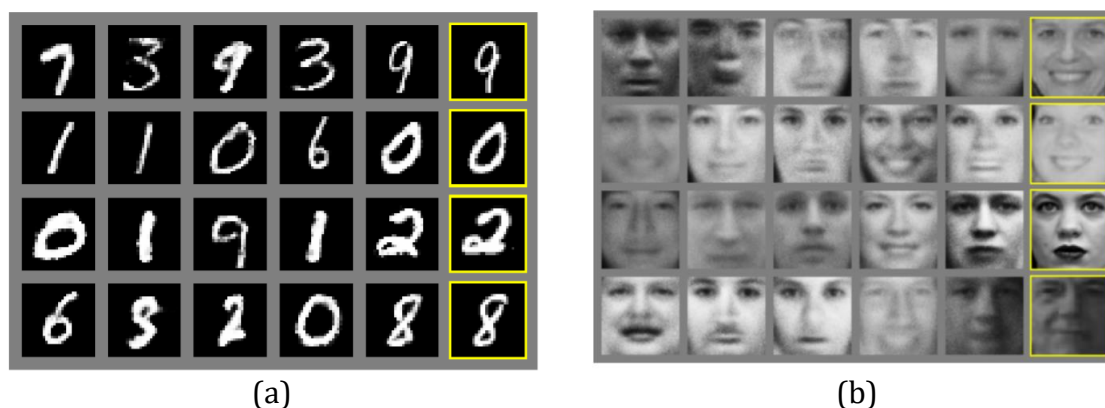


Figura 9.52: Resultados do modelo GAN: (a) base de dados MNIST; (b) base de dados de reconhecimento facial Toronto Faces Dataset. Fonte: reproduzido de [211].



Figura 9.53: Modelo GAN progressivo treinado com base de dados de celebridades CelebA. Fonte: reproduzido de [296].

Comparando a Figura 9.53 com a Figura 9.52a, percebe-se a evolução dos modelos GAN em apenas três anos. Redes adversárias representam um novo paradigma de ajuste de modelos de redes neurais e têm sido utilizadas em diferentes aplicações, como geração de imagens [550], tradução de imagens [281], super-resolução [334], geração de imagens a partir de texto [597], entre diversas outras que surgem muito rapidamente. Entretanto, o desempenho do modelo GAN tem gerado preocupações em relação ao uso de redes neurais profundas em aplicações maliciosas [67].

9.4 Aplicações

Modelos de redes neurais têm sido largamente utilizados em inúmeras aplicações em diferentes setores de atividades [283][347][518]. As redes neurais MLP são, em geral, robustos à escolha da estrutura do modelo, de forma não se exige do usuário um conhecimento muito profundo sobre o modelo. Além disso, redes neurais MLP são encontrados em diversos *softwares* livres e comerciais, o que facilita sua adoção.

Neste capítulo, apresentamos algumas aplicações de redes neurais em projetos de pesquisa da Coppe/UFRJ. Na seção 9.4.1, apresentamos um modelo de previsão de inadimplência desenvolvido em parceria com uma empresa de telecomunicações. Na seção 9.4.2, discutimos uma aplicação de modelos de redes neurais para previsão de emissões de combustíveis, desenvolvido em parceria com a Petrobras. Na seção 9.4.3, o resultado de previsão de demanda diária de energia por um conjunto de modelos lineares é comparado com um modelo de redes neurais utilizado pelo ONS. Na seção 9.4.4, uma aplicação de redes neurais profundas para previsão do mercado financeiro.

9.4.1 Previsão de inadimplência

A inadimplência é um problema grave em serviços com grande base de usuários. Em setores regulados, como telecomunicações, a empresa fica ainda sujeita aos limites da regulação. A previsão de inadimplentes é, assim, uma ferramenta importante para a empresa direcionar ações de *marketing* e definir estratégias de prevenção.

A inadimplência é diferente de fraude. O cliente inadimplente está insolvente ou em atraso, ou seja, não pagou pelo serviço por razões diversas, mas, em princípio, não teve a intenção de lesar a companhia. No caso de fraude, o cliente tem a intenção de obter ganhos ilícitos com a utilização do serviço. A fraude é crime previsto na legislação, mas inadimplência é risco do negócio. A diferença entre as duas situações é tênue, mas o valor de fraudes é geralmente maior e há um número maior de inadimplentes que fraudadores. Do ponto de vista de ciência de dados, fraude é tratada como problema de detecção de *outliers* e a previsão de inadimplentes é um problema de classificação.

Um modelo de previsão de inadimplentes numa empresa de telecomunicações foi realizado em 2005 por Pinheiro [438][439]. Na época, a empresa contava com 10,2 milhões de linhas de telefonia fixa. O projeto foi focado em clientes residenciais e o conjunto de dados para construção do modelo foi obtido por uma seleção aleatória de 5%

da base de dados de bilhetagem correspondentes a três meses de operação. De um total de 152 variáveis disponíveis, 47 foram selecionadas por especialistas para o ajuste dos modelos [439].

Inicialmente, foi feita uma análise de agrupamentos para a caracterização do perfil do cliente inadimplente. Foram identificados cinco grupos de clientes de acordo com seu comportamento em relação à inadimplência. A descrição dos grupos é apresentada na Tabela 9.5. A análise dos grupos mostrou que 32,5% dos clientes (grupos C_1 e C_2) eram responsáveis por 54,7% do faturamento e 50% do total de inadimplência [439].

Tabela 9.5: Segmentação de clientes inadimplentes. Fonte: [439]

	População (%)	Faturamento (%)	Inadimplência (%)
C_1	13.7	34.8	26.0
C_2	18.8	19.9	24.0
C_3	28.8	16.2	19.0
C_4	16.9	12.6	16.3
C_5	21.8	16.6	14.7

Para o modelo de previsão, os clientes foram classificados como inadimplentes caso o atraso no pagamento da conta fosse superior a 29 dias; caso contrário, sua classificação era normal. Essa divisão da base resultou num conjunto de dados desbalanceado, com 72,2% dos clientes considerados normais (OK) e 27,8% identificados como inadimplentes (NO). O percentual de clientes inadimplentes no estudo, isto é, com débitos em atraso por mais de 29 dias, mostra a dimensão do problema considerando o número de clientes da empresa.

Em operação, o modelo gera uma previsão dos clientes que pagariam (ou não pagariam) a conta em 29 dias a partir do registro de débito, verificado no vencimento da conta. Caso o cliente fosse considerado normal, o atraso seria apenas um problema de esquecimento ou falta de recursos momentânea e nenhuma ação seria necessária. No caso de clientes identificados como inadimplentes potenciais, a empresa poderia planejar ações para mitigar as perdas.

Os modelos foram desenvolvidos na plataforma SAS utilizando os algoritmos disponíveis para síntese de redes neurais MLP. Além da modelagem direta com único modelo,

foi considerada uma solução de *ensemble* formada pela combinação de 10 modelos, cada um ajustado numa base previamente segmentada por um algoritmo de agrupamento, como mostra a Figura 9.54. A classificação, nesse caso, é feita em duas etapas: primeiro, o registro é alocado ao segmento cujo centroide é mais próximo; em seguida, o modelo de rede neural ajustado com os dados daquele segmento é aplicado. O resultado do *ensemble* é obtido por votação simples [604].

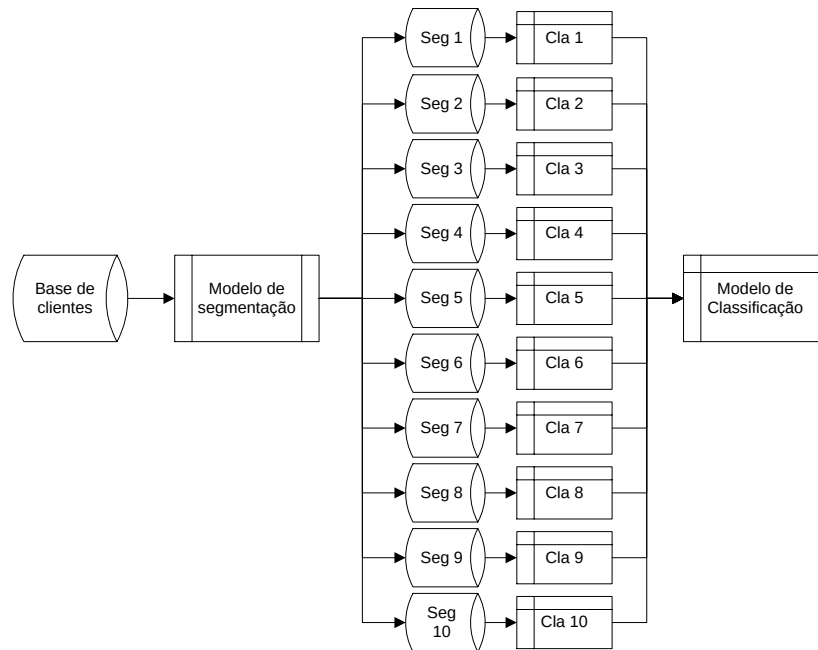


Figura 9.54: Combinação de modelos por um modelo de segmentação. Fonte: reproduzido de [438].

Os resultados dos modelos em função das estatísticas de precisão (*PRE*) e recuperação (*REC*) (cf. equações (5.31) e (5.32)) são apresentados na Tabela 9.6. Observe que o modelo combinado obteve melhor taxa de precisão nas duas classes, mas uma taxa de recuperação muito melhor no caso de clientes inadimplentes (NO).

Tabela 9.6: Resultados dos modelos. Fonte: [438]

Classes	Modelo simples		Modelo combinado	
	<i>PRE</i> (%)	<i>REC</i> (%)	<i>PRE</i> (%)	<i>REC</i> (%)
OK (C_1)	81.6	84.0	91.8	84.5
NO (C_2)	83.5	81.0	85.7	92.5

O procedimento operacional da empresa prevê o envio de uma notificação 15 dias após ser verificada uma conta em débito. Considerando apenas o custo de postagem da notificação, a economia gerada por não enviar notificação para os clientes que vão pagar a conta antes de 29 dias de atraso representa alguns milhões por mês.

Em 2005, quando esse estudo foi realizado [438], os tomadores de decisão de grandes empresas ainda não estavam totalmente convencidos da importância da utilização de modelos de ciência de dados ante os altos investimentos em *hardware*, *software* e treinamento para o desenvolvimento dos modelos. Atualmente, a aplicação de modelos de ciência de dados é estratégica, podendo impactar diretamente na sobrevivência da empresa em mercados cada vez mais competitivos.

9.4.2 Previsão de emissões de combustível

A emissão de poluentes por veículos é uma das grandes preocupações das indústrias de automóveis e empresas de petróleo. Motores menos poluentes e combustíveis mais eficientes têm sido desenvolvidos nos últimos anos em função das restrições impostas por regulamentações cada vez mais restritivas. No entanto, o impacto de melhorias no cenário real é muito difícil de ser avaliado devido a uma série de razões, mas, principalmente, por conta da dificuldade de modelos de emissões confiáveis.

As companhias de petróleo e montadoras fazem ensaios de emissões para o desenvolvimento de novos combustíveis e novas tecnologias de motores. Mesmo em condições controladas, os ensaios de emissões de poluentes são bastante imprecisos e sua execução consome tempo e recursos. Modelos de ciência de dados para previsão de emissões podem ajudar no desenvolvimento de novos combustíveis, reduzindo os ensaios.

A Petrobras realiza ensaios em laboratório sobre as influências de composições de gasolina nas emissões de poluentes e autonomies de motores em veículos representativos da frota nacional [539][540]. Essa base de dados, que contém dados de mais de 10 anos de ensaios, foi utilizada para o desenvolvimento de modelos de redes neurais para previsão de emissões.

As variáveis utilizadas nos modelos são apresentadas na Tabela 9.7. Das 17 variáveis de entrada, oito são para a descrição dos veículos e nove para a descrição dos combustíveis. A descrição dos veículos é feita por variáveis numéricas, que representam as ca-

racterísticas mecânicas (volume de deslocamento, potência e peso), o ano de fabricação, e variáveis nominais, que representam as tecnologias empregadas para ignição, alimentação e transmissão, bem como o tipo de combustível utilizado (álcool ou gasolina). A descrição dos combustíveis é feita pela massa específica, pelas concentrações de álcool (AEAC), enxofre, aromáticos, olefínicos e pelas temperaturas em que 10%, 50%, 90% e 100% (PFE) do volume é evaporado em condições controladas. Os modelos foram ajustados para realizar a previsão de seis variáveis de saída: quatro variáveis de emissões de poluentes e duas de autonomia.

Tabela 9.7: Variáveis utilizadas pelo modelo. Fonte: adaptado de [539]

Variáveis de Entrada		Variáveis de Saída	
Veículo	Combustível	Emissões	Autonomia
Ano do modelo	Massa esp. (kg/m ³)	THC	Urbana (km/l) (FE)
Volume (l)	AEAC %	NOx	Estrada (km/l)
Potência máxima (cv)	Enxofre (ppm)	CO ₂	
Peso (kg)	Aromáticos (%)	CO	
Ignição	Olefínicos (%)		
Alimentação	T10%(°C)		
Transmissão	T50%(°C)		
Combustível	T90%(°C)		
	PFE(°C)		

O conjunto de dados de ensaios contém muitos valores ausentes, além de uma série de inconsistências. Após o processo de limpeza e consistência dos dados, o conjunto de treinamento contava um total de 2.583 registros. Os registros com valores ausentes foram eliminados e o conjunto de treinamento efetivamente utilizado para o ajuste dos modelos continha $N = 1.040$ registros completos.

Os dados representam os ensaios de diversas composições de combustíveis testadas nos principais veículos da frota nacional. Para o mesmo combustível, foram realizados diversos ensaios com diferentes veículos e, analogamente, para cada veículo foram realizados ensaios com diversos combustíveis.

As emissões dos ensaios dos diversos veículos (combustíveis) para o mesmo combustível (veículo) são apresentadas na Figura 9.55, onde os registros foram ordenados pelo

total de emissões (padronizadas), e plotados em função da primeira componente canônica de cada grupo de variáveis (cf. seção 2.4.4). A Figura 9.55a mostra as emissões por combustível, enquanto a Figura 9.55b apresenta as emissões por veículo. Os gráficos de barra mostram os números do total de ensaios de cada combustível (Figura 9.55a) e de cada veículo (Figura 9.55b).

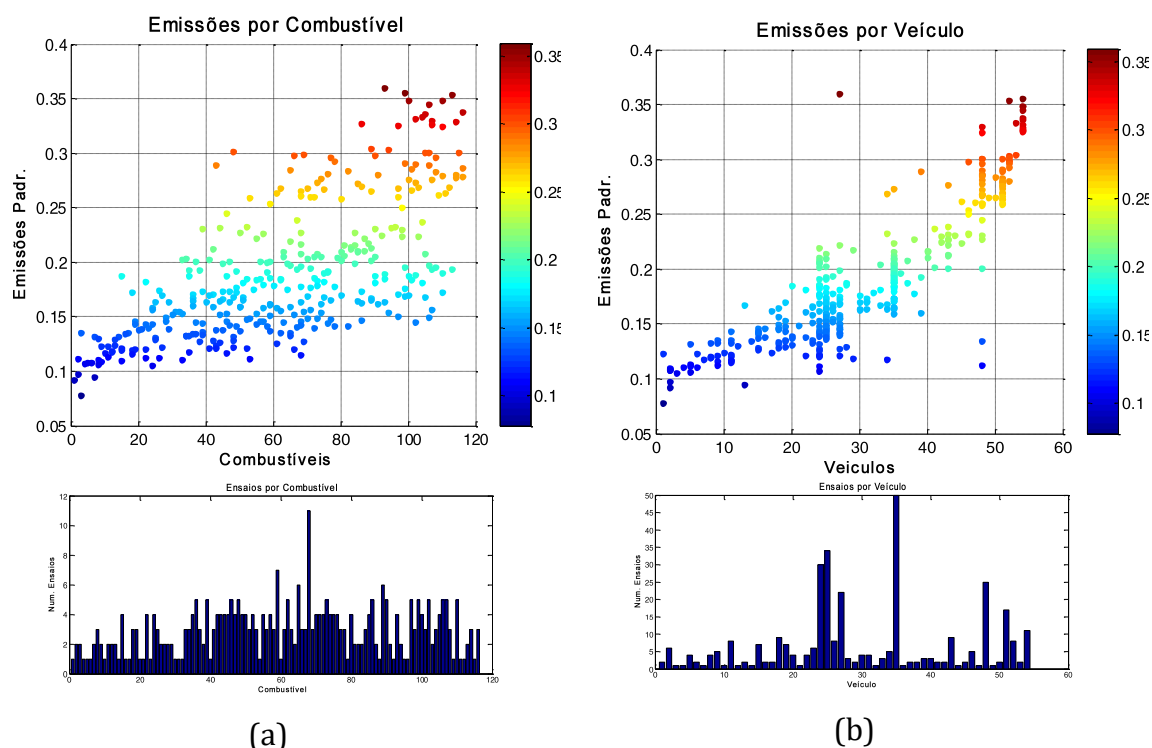


Figura 9.55: Visualização de dados: (a) por combustíveis; (b) por veículos. Fonte: adaptado de [539].

Podemos verificar na Figura 9.55 que, para o mesmo combustível, as emissões podem variar bastante de acordo com o veículo utilizado no teste. Entretanto, para o mesmo veículo, a dispersão em função dos diferentes combustíveis é menor. Esse resultado sugere que as tecnologias veiculares são mais robustas às variações de combustível que o contrário. Essa conclusão é preliminar e pode estar enviesada em função dos ensaios realizados, uma vez que algumas combinações veículo/combustível não foram analisadas nos ensaios registrados.

Diversas topologias de redes neurais MLP foram geradas para cada variável de saída e os melhores modelos foram escolhidos por validação cruzadas de $k = 5$ ciclos. Os resultados são mostrados na Tabela 9.8 onde A.U. representa autonomia urbana e A.E. autonomia de estrada. O resultado do erro RMS de cada modelo em validação cruzada

é comparado ao desvio padrão dos ensaios para cada variável de saída. Podemos observar que os erros dos modelos são comparáveis à imprecisão dos ensaios e, no caso de CO₂, o RMS é menor que o desvio padrão dos ensaios.

Tabela 9.8: Resultados. Fonte: adaptado de [539]

	THC	NO _x	CO ₂	CO	A.U.	A.E.
RMS	0.0306	0.0420	7.6108	0.1924	0.7148	0.3713
Desvio do ensaio	0.0270	0.0380	12.6507	0.1707	0.3335	0.2732

Os resultados desta seção mostram que redes neurais MLP podem obter resultados de boa precisão em problemas complexos. Na próxima seção, será apresentado um exemplo em que modelos MLP se tornam excessivamente complexos e bons resultados podem ser obtidos com modelos lineares organizados para serem ajustados em bases de dados mais homogêneas.

9.4.3 Previsão de demanda de energia elétrica

O Operador Nacional do Sistema (ONS) é responsável pela operação do Sistema Interligado Nacional (SIN). Além da operação semanal (cf. seções 4.4.2 e 8.4.4), o ONS realiza o planejamento diário de operação, para o qual é necessária a previsão da demanda de carga por hora no dia seguinte. Os modelos de previsão de demanda são também utilizados pelos agentes de geração e de comercialização, em suas áreas de atuação.

Esse estudo de caso foi realizado para a região Sudeste (SE) do Brasil a partir de três anos de dados de demanda total de energia por hora, consolidados na base de dados do ONS, entre 2009 e 2012. Os dados representam a demanda total de energia elétrica, incluindo instalações residenciais e não residenciais.

A série de demanda de carga diária é mostrada na Figura 9.56, onde a linha preta representa os períodos do horário normal e horário de verão. A Figura 9.57 apresenta o valor da demanda em escala de cores, onde os períodos de horário de verão são representados pela linha preta. No gráfico da Figura 9.57, as linhas representam cada 30 minutos do dia e as colunas representam os dias.

A série na Figura 9.56 apresenta os efeitos da sazonalidade anual, com menor consumo de energia no inverno e uma tendência de alta no período observado. As oscilações de

maior frequência são os ciclos semanais, que se reflete também na mobilidade urbana (cf. Figura 2.29). É possível identificar claramente eventos de baixo consumo nas semanas que antecedem as viradas de ano e alguns feriados prolongados.

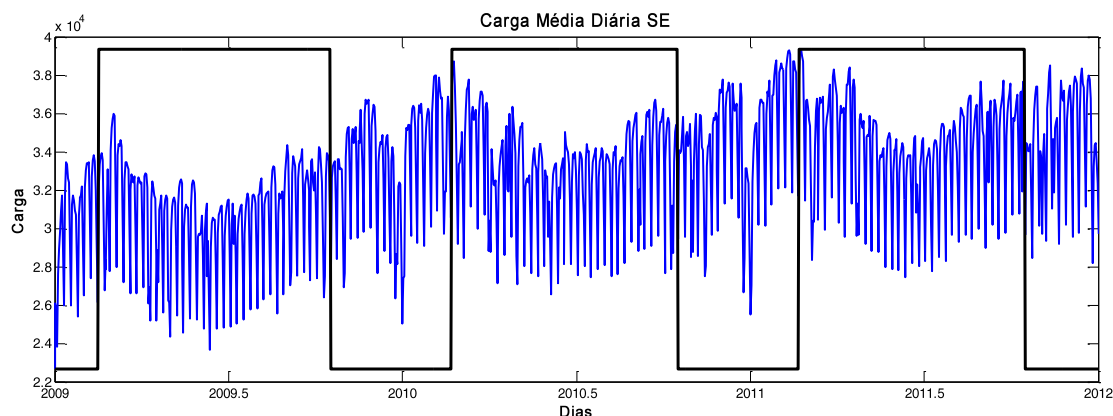


Figura 9.56: Série de demanda diária de carga no SE.

A visualização da Figura 9.57 permite verificar o efeito do horário de verão, que atrasa o pico de demanda e resulta em economia de energia. A Figura 9.57 mostra a tendência de elevação da demanda de energia no período observado, tanto no horário comercial quanto no noturno. A Figura 9.57 também permite verificar a ocorrência de madrugadas quentes no verão, que provocam elevação pontual da demanda de energia.

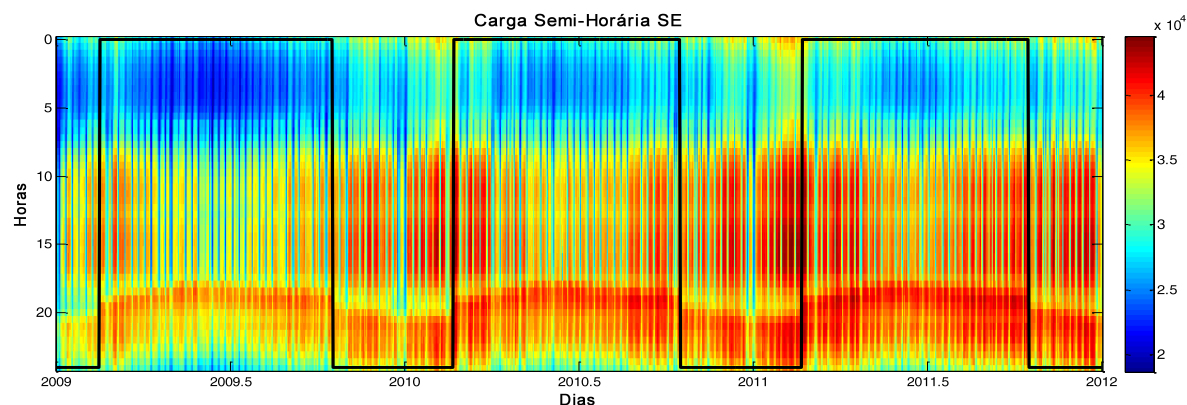


Figura 9.57: Série de demanda de energia a cada 30 minutos no SE.

Os modelos de previsão de carga utilizam frequentemente a temperatura como variável de entrada, mas devem levar em conta também os efeitos de ciclos semanais e a sazonalidade anual. Esta seção apresenta duas alternativas de modelagem para a previsão de demanda de energia para o dia seguinte na região SE. A primeira é um modelo

de redes neurais utilizado pelo setor elétrico no Brasil e no mundo. A segunda abordagem utiliza múltiplos modelos lineares, um para cada hora e classe de dia, que obtém um desempenho comparável à anterior, sendo bem mais simples e eficiente.

9.4.3.1 Modelo ANNSTLF

O modelo *Artificial Neural Network Short-Term Load Forecaster* (ANNSTLF) [305][306] foi desenvolvido pelo Electric Power Research Institute (EPRI)²²⁶ e implementado num *software* que se tornou praticamente um padrão da indústria, sendo utilizado no Brasil pelo ONS e diversos agentes do setor elétrico.

O modelo ANNSTLF [306] é composto de dois módulos de redes neurais (cf. Figura 9.58), cada um com 79 unidades de entrada e 24 unidades de saída. O primeiro módulo, chamado de *Base Load Forecaster* (BLF), é o modelo de base que representa a carga em cada uma das 24 horas do dia de previsão. O segundo módulo, chamado *Change Load Forecaster* (CLF), é um modelo que representa a diferença entre a carga do dia de previsão e a carga do dia anterior. Os resultados dos dois módulos são combinados num modelo ajustado pelo método de mínimos quadrados recursivos (RLS).

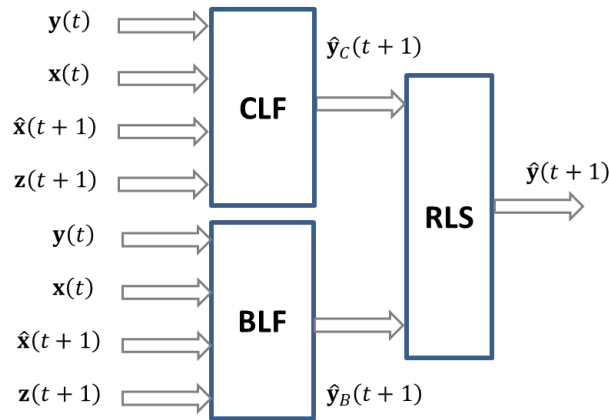


Figura 9.58: Estrutura do modelo ANNSTLF. Fonte: adaptado de: [306].

As entradas dos modelos BLF e CLF são as seguintes:

- $\mathbf{y}(t) = (y_1(t), \dots, y_{24}(t))$: carga média horária para cada hora no dia t ;
- $\mathbf{x}(t) = (x_1(t), \dots, x_{24}(t))$: variáveis de condições meteorológicas (ou temperatura efetiva) para cada hora observadas no dia t ;
- $\hat{\mathbf{x}}(t + 1) = (\hat{x}_1(t + 1), \dots, \hat{x}_{24}(t + 1))$: variáveis de previsão de condições meteorológicas para cada hora para dia de previsão $t + 1$;

- $\mathbf{z}(t + 1) = (z_1(t + 1), \dots, z_7(t + 1))$: tipo de dia, variáveis indicadoras que identificam o dia da semana.

As saídas de cada um dos módulos são as seguintes:

- BLF: $\hat{\mathbf{y}}_B(t + 1) = (\hat{y}_{B1}(t + 1), \dots, \hat{y}_{B24}(t + 1))$, carga média horária prevista para cada hora no dia de previsão $t + 1$;
- CLF: $\hat{\mathbf{y}}_C(t + 1) = (\hat{y}_{C1}(t + 1), \dots, \hat{y}_{C24}(t + 1))$, variação de carga média horária prevista para cada hora no dia de previsão $t + 1$ em relação ao dia anterior.

O valor de carga final previsto é uma combinação linear entre os resultados dos modelos BLF e CLF. A previsão de cada hora é calculada como:

$$\hat{y}_i(t + 1) = \alpha_{Bi}\hat{y}_{Bi}(t + 1) + \alpha_{Ci}\hat{y}_{Ci}(t + 1), \quad i = 1, \dots, 24 \quad (9.143)$$

onde os conjuntos de parâmetros $\boldsymbol{\alpha}_B = (\alpha_{B1}, \dots, \alpha_{B24})$ e $\boldsymbol{\alpha}_C = (\alpha_{C1}, \dots, \alpha_{C24})$ são continuamente ajustados através do método dos mínimos quadrados recursivos (RLS).

Os modelos de redes neurais BLF e CLF são ajustados pelo algoritmo Levenberg-Marquardt [369], com um conjunto de dados de treinamento de três anos. Para reduzir os efeitos do comportamento não estacionário, os modelos são reajustados continuamente ao longo do tempo. O modelo ANNSTLF inclui também um procedimento especial para os feriados e picos de carga, com um algoritmo dos vizinhos mais próximos que leva em consideração as variáveis meteorológicas (temperatura efetiva).

O modelo ANNSTLF é muito preciso e robusto. Entretanto, é muito complexo, o que demanda alto custo computacional para o ajuste de parâmetros. Além disso, o *software* que implementa o ANNSTLF também é fechado e difícil de ser integrado ao *workflow* de captura e análise de dados do ONS e outros agentes.

9.4.3.2 Multi modelos lineares

A correlação entre carga e temperatura é forte, mas o perfil de demanda diária de energia também varia com o dia da semana, além de ter comportamento diferente no horário de verão. A abordagem de multi modelos lineares é baseada na segmentação dos dados de acordo com as categorias de dia da semana em horário de verão e normal.

A Tabela 9.9 mostra as características de cada tipo de dia em termos do tipo de horário (verão ou normal) e do dia da semana. Os registros de terça a sexta-feira foram agrupados numa mesma classe, sendo que a segunda-feira foi tratada à parte devido ao período da madrugada. Os feriados em dias de semana foram tratados como domingos.

Tabela 9.9: Classificação de tipos de dia

Classe	Horário	Dia	Classe	Horário	Dia
C_1	HV	Domingo	C_5	HN	Domingo
C_2	HV	Segunda	C_6	HN	Segunda
C_3	HV	Terça-sexta	C_7	HN	Terça-sexta
C_4	HV	Sábado	C_8	HN	Sábado

A Figura 9.59a mostra a distribuição do número de registros em cada classe, onde se observa a predominância de registros da classe C_7 , que representa dia de semana (de terça a sexta) em horário normal. A Figura 9.59b mostra do gráfico de projeção da carga em função da temperatura, onde podemos observar que a correlação entre valores de carga e temperatura em cada classe é maior que em relação a todos os registros.

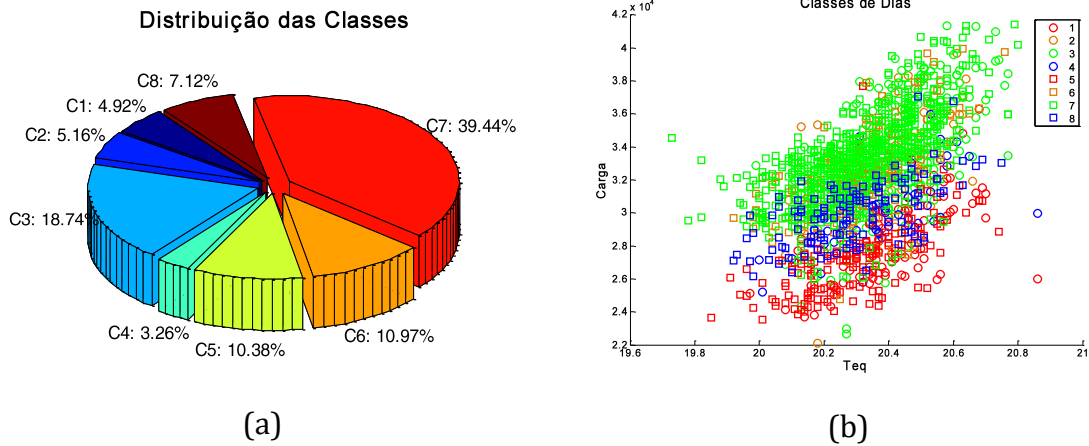


Figura 9.59: Classes de dia: (a) distribuição; (b) correlação com a temperatura.

Para capturar as características de sazonalidade, foi criada uma variável que representa o dia do ano como o seno e o cosseno do ângulo correspondente ao número de dias desde o 1º de janeiro do mesmo ano como:

$$\begin{aligned} w_x(t) &= \sin(d) \\ w_y(t) &= \cos(d) \end{aligned} \quad (9.144)$$

onde d é o número de dias desde 1º de janeiro do ano correspondente até o dia t .

A abordagem de multi modelos lineares utiliza as seguintes variáveis:

- $\mathbf{y}(t) = (y_1(t), \dots, y_{24}(t))$: carga média horária observada para o dia t ;
- $\hat{\mathbf{y}}(t) = (\hat{y}_1(t), \dots, \hat{y}_{24}(t))$: carga média horária estimada para o dia t ;
- $\mathbf{x}(t) = (x_1(t), \dots, x_{24}(t))$: temperatura horária observada para o dia t ;
- $\hat{\mathbf{x}}(t) = (\hat{x}_1(t), \dots, \hat{x}_{24}(t))$: temperatura horária estimada para o dia t ;
- $\mathbf{z}(t) = (z_1(t), \dots, z_8(t))$: classe do dia;
- $\mathbf{w}(t) = (w_x(t), w_y(t))$: dia do ano, representada pelo seno e cosseno.

A Figura 9.60 mostra a estrutura da abordagem multi modelos em cada hora. Um modelo linear é ajustado para cada tipo de dia e para cada hora utilizando a série histórica da hora nos dias determinados pelo tipo de dia. Um total de $24 \times 8 = 192$ modelos lineares foram ajustados, sendo que apenas 24 modelos, correspondentes à previsão de cada hora, são utilizados na previsão de cada dia de acordo com o tipo de dia.

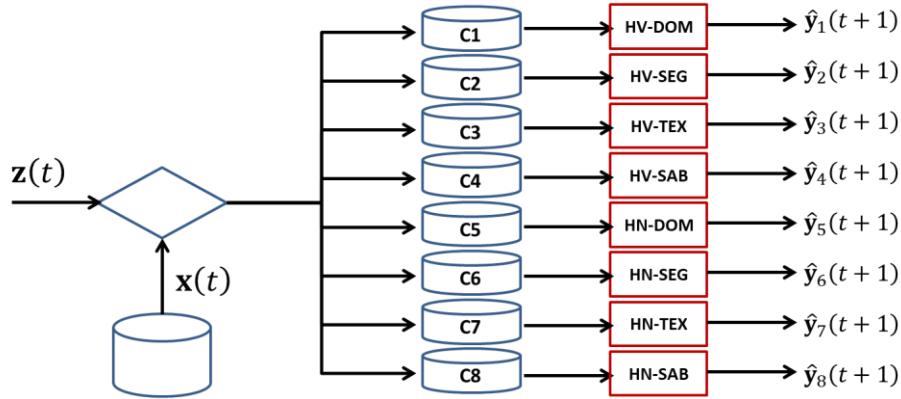


Figura 9.60: Abordagem multi modelos para cada hora.

Diversos modelos lineares foram testados. Os melhores resultados para a previsão de carga no dia seguinte para a hora i , $\hat{y}_i(t+1)$ foram obtidos por um modelo linear que utiliza como entrada um vetor de cinco variáveis:

- $z_k(t+1)$: tipo de dia no dia de previsão como variável indicadora;
- $w_x(t+1)$ e $w_y(t+1)$: variáveis de sazonalidade no dia de previsão;
- $\hat{x}_i(t+1)$: a previsão de temperatura para a hora i no dia de previsão;
- $\hat{y}_{mi}(t+1)$: previsão do modelo de média móvel de carga para a hora i .

A variável $\hat{y}_{mi}(t+1)$ é um modelo de previsão da série de média móvel de carga da hora i , cujo objetivo é ajustar a tendência da série. Os resultados apresentados a seguir foram obtidos com uma janela móvel de 15 registros, utilizando a última média móvel observada como uma previsão de média móvel horária, isto é, $\hat{y}_{mi}(t+1) = y_{mi}(t)$.

Os multi modelos lineares foram reajustados diariamente utilizando 450 registros anteriores, o que corresponde a aproximadamente 15 meses de histórico, dependendo do número de registros ausentes. Os modelos foram ajustados com os valores observados de temperatura $x_i(t + 1)$, sendo que os valores previstos de temperatura $\hat{x}_i(t + 1)$ devem ser utilizados na previsão. Os resultados são apresentados a seguir.

9.4.3.3 Resultados

A Figura 9.61 mostra os resultados do modelo testado no período 2010-2012. A figura superior mostra a série temporal de demanda da região SE observada (em preto), a série de demanda prevista com os valores de temperatura observada (em azul) e a série de demanda prevista com valores de temperatura prevista (em vermelho). A figura inferior mostra a série de erros MAPE, onde se observa que os maiores erros ocorrem em datas específicas, correspondentes a feriados, que não são tratados pelos modelos.

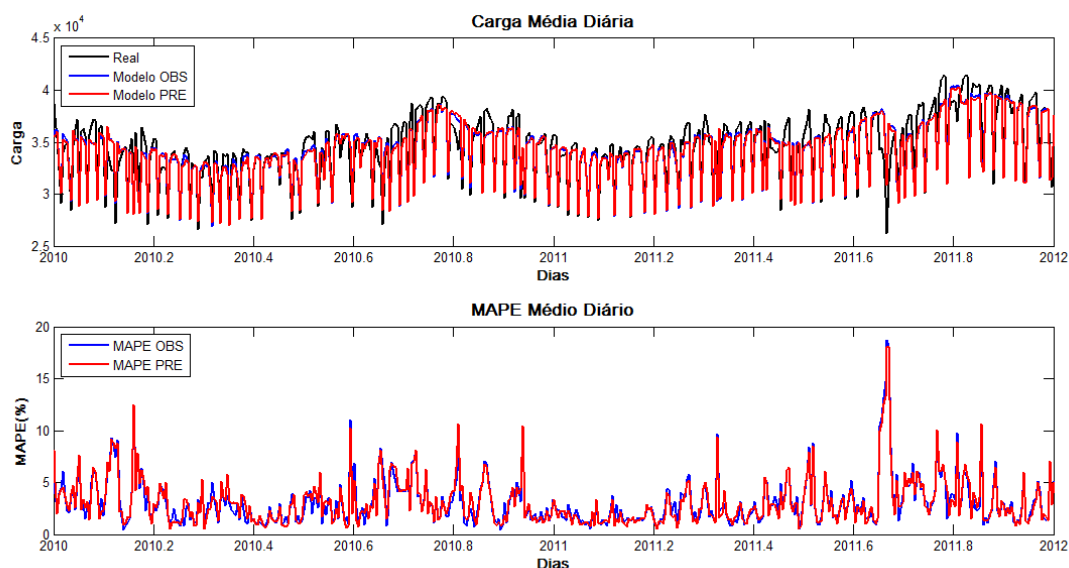


Figura 9.61: Resultados do modelo multilinear.

A comparação entre os resultados do modelo linear e do modelo ANNSTLF é mostrada na Tabela 9.10. A avaliação foi feita em relação a todas as horas para a média diária e para a hora de ponta nos dias de semana, fins de semana e todos os dias. Observa-se que os resultados do modelo linear são muito semelhantes aos resultados do ANNSTLF, sendo que o modelo de redes neurais tem sido desenvolvido há anos e tem várias heurísticas para ajuste de feriados, horários de ponta etc.

Tabela 9.10: Resultados do modelo multilinear

		MAPE (%)		
		Segunda-Sexta	Sábado-Domingo	Segunda-Domingo
Total de horas	ANNSTLF	2,4	2,8	2,5
	LINEAR	2,6	2,6	2,6
Média diária	ANNSTLF	0,9	0,1	0,6
	LINEAR	2,1	1,0	1,2
Hora de ponta	ANNSTLF	0,6	1,7	1,1
	LINEAR	3,9	2,5	3,9

Os resultados apresentados nesta seção mostram que a utilização de múltiplos modelos lineares, ajustados em dados homogêneos para cada hora e tipo de dia, pode produzir bons resultados de previsão. Além disso, modelos lineares podem ser facilmente implementados em qualquer plataforma, podendo ser integrados ao *workflow* de análise e previsão, que foi a demanda do ONS nesse projeto.

9.4.4 Previsão de mercado financeiro

O mercado financeiro é um sistema dinâmico complexo, não estacionário e influenciado por inúmeros fatores, como economia local e global, cenário político, entre diversos outros. A previsão das séries temporais do mercado financeiro é um assunto bem explorado na literatura [78][484]. Recentemente, modelos de processamento de linguagem natural com redes neurais profundas têm sido aplicados para a previsão de séries temporais financeiras, a partir de notícias jornalísticas [134][409][480].

Pela hipótese do mercado eficiente, o preço dos ativos reflete toda a informação disponível até o momento e o movimento futuro do mercado é, portanto, aleatório e impossível de se prever. A hipótese do mercado eficiente é uma aproximação e tem sido discutida há anos por economistas [362], mas deixa claro que o movimento do mercado está fortemente ligado às informações financeiras.

Um modelo de NLP com redes neurais profundas para a previsão do mercado financeiro foi proposto por Vargas e colaboradores [535][536]. O modelo utiliza um *corpus* de 106.494 artigos em inglês coletados da seção de finanças da Reuters entre outubro

de 2006 e novembro de 2013 [133]. Os artigos são armazenados com marcação de título, conteúdo e data de publicação. A data de publicação é empregada para alinhamento de notícias com alguma série financeira. Ding e colaboradores [133][134] mostraram que os títulos dos artigos são mais úteis para previsão do que os conteúdos. Os modelos foram ajustados para previsão da série S&P 500 [535] e a cotação da empresa Chevron (CVX) [536] no período coberto pelo *corpus* de notícias.

Os títulos das notícias são agrupados por dia e filtrados para conter apenas artigos mais relevantes para as séries estudadas [133]. A saída observada é uma variável binária que representa a variação da série financeira no fechamento do dia seguinte. Se houver uma variação positiva, ou seja, ganho, no dia seguinte, $v(t + 1) = 1$. Caso contrário, se houver variação negativa, $v(t + 1) = -1$.

Ding e colaboradores [133][134] separaram o conjunto de dados em subconjuntos de treinamento, validação e testes, que foram também utilizados por Vargas e colaboradores [536]. O particionamento dos dados é mostrado na Tabela 9.11.

Tabela 9.11: Resumo do *corpus* de notícias. Fonte: adaptado de [536]

	Treino	Validação	Teste
Período	02/10/2006 18/06/2012	19/06/2012 21/02/2013	22/02/2013 21/11/2013
No. de instâncias	1419	168	187
No. de documentos	13149	1976	2046
Média de artigos/dia	10	12	11

O modelo de Vargas e colaboradores [535][536] é apresentado na Figura 9.62. O modelo utiliza dois tipos de entradas: $\mathbf{x}_A(t)$ referente às notícias e $\mathbf{x}_B(t)$ referente a indicadores financeiros. A entrada de notícias $\mathbf{x}_A(t)$ é uma sequência de todas as notícias do dia que é vetorizada numa camada word2vec. A vetorização do título é calculada como a média da vetorização das palavras, calculadas com a codificação padrão do word2vec.²²⁷ A saída da primeira camada da sub-rede de notícias $\mathbf{u}_A^{(0)}(t)$ é a vetorização de todos os títulos do dia. Vargas e colaboradores [535][536] utilizam uma vetorização de dimensão $n = 300$ e $p = 10$ artigos/dia. A ativação $\mathbf{u}_A^{(0)}(t)$ é, portanto, uma

matriz de dimensão $p \times n$. A segunda camada da sub-rede de notícias realiza a convolução entre a matriz $\mathbf{u}_A^{(0)}(t)$ e um conjunto de filtros $k \times n$ em que k corresponde ao número de artigos “lidos” pelo filtro e n é a dimensão de vetorização. Foram utilizados 192 filtros $k \times n$, 64 com $k = 3$, 64 com $k = 4$ e 64 com $k = 5$ [535][536]. Após o *max-pooling*, o vetor de ativação $\mathbf{u}_A^{(1)}(t)$ tem dimensão $((p - 2)/2) + 1 \times 192$, onde p é o número de notícias/dia. A terceira camada é uma camada LSTM com 128 unidades que produz como saída o vetor $\mathbf{u}_A^{(2)}(t)$ [535][536]. A entrada dos indicadores financeiros utiliza uma janela temporal de cinco dias, que alimenta diretamente uma camada LSTM com 128 unidades para calcular o vetor de ativação $\mathbf{u}_B^{(1)}(t)$. Os resultados das duas camadas LSTM correspondentes às entradas são concatenados e conectados à camada de saída com função de ativação sigmoide (regressão logística), pois a saída é binária.

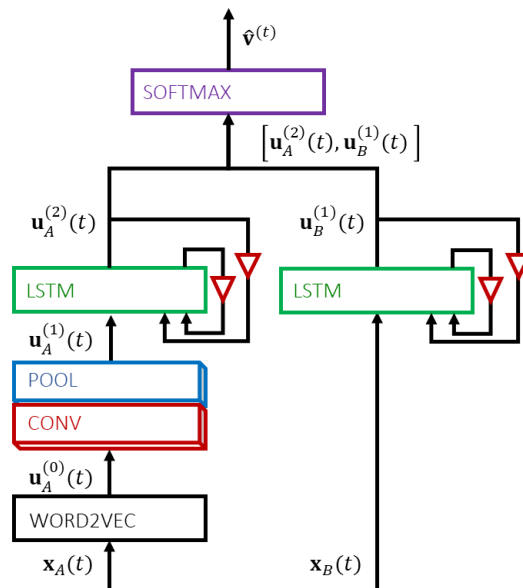


Figura 9.62: Topologia para a previsão da variação de séries financeiras a partir de notícias e indicadores financeiros. Fonte: adaptado de [536].

Seis indicadores financeiros foram utilizados no artigo [535], retirados do livro de Kirkpatrick e Dahlquist [309]. No artigo [536] foi testado um conjunto maior de indicadores com resultados similares, pois os indicadores são muito correlacionados.

Os resultados do modelo (cf. Figura 9.62), chamado SI-RCNN, são apresentados na Tabela 9.12, para a série S&P 500 [535] e Chevron (CVX) [536]. Para a série S&P 500 também são apresentados os resultados obtidos por Ding e colaboradores [133][134].

Tabela 9.12: Resultados dos modelos. Fonte: adaptado de [535][536].

	S&P 500 [535]		CVX [536]	
	Treino ACC (%)	Teste ACC (%)	Treino ACC (%)	Teste ACC (%)
SI-RCNN	63,09%	62,03%	84.08	56.84
E-NN [133]	58,94%	58,83%		
WB-CNN [134]	61,73%	60,57%		

O modelo apenas com indicadores, sem notícias, obteve um resultado pior [536], demonstrando a importância da informação textual. O artigo realiza uma simulação de investimentos e mesmo com uma acurácia de 56,84% no conjunto de testes, o modelo é capaz de realizar um retorno de 14% nas bolsas NYSE e NASDAQ no período.

O método foi aplicado em uma base de notícias em português [50]. O *corpus* em português, coletado por Figueiredo [181], contém 90.552 artigos em português da seção de economia da Reuters, no período de 28/09/2007 a 17/01/2016. Há 2.050 dias no período observado, mas apenas 2.030 dias com notícias registradas, ou seja, há 20 dias sem registros. Bichara [50] utilizou também um *corpus* filtrado para avaliação dos algoritmos, apenas com notícias relevantes para o setor financeiro. O *corpus* filtrado (*banknews*) tem 24.349 notícias, nos mesmos 2.030 dias do *corpus* completo. A série financeira das ações do Banco do Brasil (BBAS3) foi utilizada para previsão.

Bichara [50] utilizou $p = 57$ notícias/dia no *corpus* completo e $p = 15$ notícias/dia para o treinamento com *banknews*. Foram avaliadas diferentes variações de modelos e estratégias de treinamento, considerando o *corpus* utilizado para o treinamento (completo e *banknews*), com ou sem a retirada das *stopwords*, dois modelos de vetorização de palavras (word2vec e GloVe) em português, do repositório NILC²²⁸ da USP [237]. No total, 120 combinações foram avaliadas.

Os resultados do *corpus* filtrado (*banknews*) são apresentados na Tabela 9.13. Os resultados dos melhores modelos entre as 120 combinações foram bem parecidos com o modelo SI-RCNN [535][536] com notícias em inglês.

Os resultados apresentados por Bichara [50] confirmam que a utilização de notícias combinadas com indicadores aumenta o desempenho do modelo, em comparação com

a utilização apenas de indicadores, nos diversos cenários avaliados. Os modelos treinados com o *corpus* filtrado (*banknews*) obtiveram resultados melhores que os modelos treinados com a base completa. A vetorização GloVe e word2vec, bem como a retirada ou não de *stopwords*, tiveram desempenho equivalente.

Tabela 9.13: Resultados dos modelos. Fonte: adaptado de [50]

Modelo	Validação ACC (%)	Teste ACC (%)
SI-RCNN [536]	60,45	56,84
Máxima	61,46	56,59
Mediana	55,12	53,66
Mínima	51,22	49,76

Bichara [50] também avaliou um modelo de $m = 3$ classes, em que foi incluída uma classe intermediária entre as classes de variação positiva ou negativa. Nesse caso, a variação do preço de fechamento só é considerada da classe POSITIVA se for maior que 0,85% e NEGATIVA se for menor que -0,85%. A classe foi anotada como NEUTRA nos dias em que a variação observada esteve no intervalo $[-0,85\% \ 0,85\%]$. O limiar para a definição das classes foi definido para que o conjunto de dados ficasse balanceado.

No modelo de três classes, Bichara [50] avaliou o ajuste de parâmetros com dados numa janela deslizante, que é mais apropriado para modelos séries temporais não estacionárias e para o funcionamento contínuo do modelo na prática. Janelas de diferentes tamanhos foram testadas e os resultados melhoraram com janelas maiores, tendo em vista o grande número de parâmetros do modelo.

Bichara [50] também considerou o contexto de apoio a decisão, em que o resultado do modelo pode ser utilizado em diferentes estratégias de investimento, de acordo com a incerteza da previsão, medida pela entropia do resultado da função *softmax*. O resultado de classificação só é considerado como decisão (compra ou vender), se a entropia do resultado for menor que um limiar predefinido. Modelos mais conservadores são obtidos com limiar de entropia menor e, portanto, maior certeza do resultado. Modelos mais conservadores tendem a ter melhor desempenho de classificação, mas podem resultar em baixo retorno devido ao número limitado de operações. Os melhores resultados foram obtidos com treinamento em janela deslizante de 750 dias, *ensemble* de 10

modelos com limiar de entropia 0,8. Esse modelo obteve um AUC de 56% e retorno de 38,7% na simulação de investimento.

Os resultados nesta seção demonstram o grande potencial de aplicações de modelos de redes neurais profundas na previsão de séries temporais complexas para o apoio à tomada de decisão no mercado de ações.

9.5 Comentários Finais

Este capítulo apresentou os principais modelos de redes neurais, desde o Perceptron até o modelo tradutor do Google. O assunto é vasto e tem se expandido rapidamente, de forma que apenas os modelos mais importantes foram abordados, com referências ao repositório de códigos e modelos pré-treinados.

Modelos de inteligência computacional para ciência de dados são cada vez mais importantes em diferentes setores como resultado das transformações tecnológicas e da maior capacidade de produção e processamento de dados. A utilização de grandes bases de dados para o ajuste de modelos alinhados com os objetivos estratégicos tem se tornado um desafio para as organizações. A complexidade das aplicações exige o desenvolvimento de soluções criativas dos problemas, para os quais a inteligência computacional é uma caixa de ferramentas bastante útil.

Durante o tempo em que este livro foi escrito, houve um desenvolvimento muito rápido de novos modelos e resultados de uma área de pesquisa muito ativa e que se desenvolve muito rapidamente. Os assuntos abordados no livro formam uma base para que o leitor possa aprender novos modelos, considerando que os princípios fundamentais estão bem compreendidos.

Este livro é um texto introdutório para os futuros cientistas de dados, mostrando os fundamentos matemáticos no desenvolvimento e na análise dos modelos de inteligência computacional em aplicações de ciência de dados.

Referências bibliográficas

- [1] ABADI, M. et. al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. arXiv:1603.04467, 2016.
- [2] ABDEL-HAMID, O.; MOHAMED, A-R.; JIANG, H.; DENG, L.; PENN, G.; YU, D. Convolutional Neural Networks for Speech Recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, v. 22, n. 10, p. 1533-1545, 2014.
- [3] AGGARWAL, C. C.; WANG, H. (Eds.). *Managing and Mining Graph Data*. Springer, 2010.
- [4] AGGARWAL, C. C. (Ed.). *Social Network Data Analytics*. Springer, 2011.
- [5] AGGARWAL, C. C. *Data Mining: The Textbook*. Springer; 2015.
- [6] AHUJA, R. K.; MAGNANTI, T. L.; ORLIN, J. B. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [7] AKAIKE, H. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, v. 19, n. 6, p. 716-723, 1974.
- [8] ALBALATE, A.; MINKER, W. *Semi-Supervised and Unsupervised Machine Learning*. Wiley, 2011.
- [9] ALCALÁ-FDEZ, J.; SÁNCHEZ, L.; GARCÍA, S.; DEL JESUS, M. J.; VENTURA, S.; GARRELL, J. M.; OTERO, J.; ROMERO, C.; BACARDIT, J.; RIVAS, V. M.; FERNÁNDEZ, J. C.; HERRERA, F. KEEL: A Software Tool to Assess Evolutionary Algorithms to Data Mining Problems. *Soft Computing*, v. 13, n. 3, p. 307-318, 2009.

- [10] ALCALÁ-FDEZ, J.; FERNANDEZ, A.; LUENGO, J.; DERRAC, J.; GARCÍA, S.; SÁNCHEZ, L.; HERRERA, F. KEEL Data-Mining Software Tool: Data Set Repository, Integration of Algorithms and Experimental Analysis Framework. *Journal of Multiple-Valued Logic and Soft Computing*, v. 17, n. 2-3, p. 255-287, 2011.
- [11] ALDROUBI, A.; GRÖCHENIG, K. Nonuniform sampling and reconstruction in shift-invariant spaces. *SIAM Review*, v. 43, n. 4, p. 585-620, 2001.
- [12] ALOISE, D.; DESHPANDE, A.; HANSEN, P.; POPAT, P. NP-hardness of Euclidean sum-of-squares clustering. *Machine Learning*, v. 75, p. 245-249, 2009.
- [13] ALONSO, J. M.; MAGDALENA, L.; GONZÁLEZ-RODRÍGUEZ, G. Looking for a good fuzzy system interpretability index: An experimental approach. *International Journal of Approximate Reasoning*, v. 51, n. 1, p. 115-134, 2009.
- [14] ALPAYDIN, E. *Introduction to Machine Learning*. 2. ed. The MIT Press, 2009.
- [15] ALVES, M. C. M.; LIMA, B. S. L. P.; EVSUKOFF, A. G.; VIEIRA, I. N. Developing a fuzzy decision support system to determine the location of a landfill site. *Waste Management & Research*, v. 27, p. 641-651, 2009.
- [16] AMODEI D. et al. Deep Speech 2: End-to-End Speech Recognition in English and Mandarin. arXiv:1512.02595. 2015.
- [17] ANASTASSIOU, G. A.; STEFFENS, K.-G. *The History of Approximation Theory: From Euler to Bernstein*. Birkhäuser, 2006.
- [18] ANDRADE, W. P. *Concretos: Massa, Estrutura, Concreto Projetado, Concreto Compactado – Ensaio e Propriedades*. São Paulo: Pini, 1997.
- [19] ANUSUYA, M. A.; KATTI, S. K. Speech Recognition by Machine, A Review. arXiv:1001.2267, 2010.
- [20] ARONSZAJN, N. Theory of reproducing kernels. *Trans. of American Mathematics Society*, v. 68, n. 3, p. 337-404, 1950.
- [21] ARTHUR, D.; VASSILVITSKII, S. K-means++: the advantages of careful seeding. In: EIGHTEENTH ANNUAL ACM-SIAM SYMPOSIUM ON DISCRETE ALGORITHMS. *Proceedings*. 2007. p. 1.027-1.035.

- [22] BABUKA, R.; VERBRUGGEN, H. Neuro-fuzzy methods for nonlinear system identification. *Annual Reviews in Control*, v. 27, n. 1, p. 73-85, 2003.
- [23] BACH, F. R.; JORDAN, M. I. Learning spectral clustering. In: 2003 NEURAL INFORMATION PROCESSING SYSTEMS (NIPS 2003). *Proceedings*. 2003. p. 305-312.
- [24] BADRINARAYANAN, V.; KENDALL, A.; CIPOLLA, R. SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation arXiv:1511.00561. 2015.
- [25] BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural Machine Translation by Jointly Learning to Align and Translate. arXiv:1409.0473, 2014.
- [26] BALDI, P.; SADOWSKI, P. J. Understanding Dropout. In: 2013 NEURAL INFORMATION PROCESSING SYSTEMS (NIPS 2013). *Proceedings*. 2013. p 2814-2822.
- [27] BALL, G.; HALL, D. *ISODATA, a novel method of data analysis and pattern classification*. Stanford: Stanford Research Institute, 1965. (Technical report NTIS AD 699616).
- [28] BANDYOPADHYAY, S.; SAHA, S.; PEDRYCZ, W. Use of a fuzzy granulation-degranulation criterion for assessing cluster validity. *Fuzzy Sets and Systems*, v. 170, n. 1, p. 22-42, 2011.
- [29] BARABÁSI, A.-L.; ALBERT, R. Emergence of scaling in random networks. *Science*, v. 286 (5439), p. 509-512, 1999.
- [30] BARABÁSI, A.-L. The origin of bursts and heavy tails in human dynamics. *Nature*, v. 435, p. 207-211, 2005.
- [31] BARRETO, A. M. S.; BARBOSA, H. J. C.; EBECKEN, N. F. F. GOLS – Genetic orthogonal least squares algorithm for training RBF networks. *Neurocomputing*, v. 69, n. 16-18, p. 2.041-2.064, 2006.
- [32] BEISL, C. H. *Utilização de Mosaicos JERS-1 SAR e de Lógica Fuzzy para Elaboração de Mapas de Sensibilidade Ambiental Temporal a Derrames de Óleo na Amazônia Central*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2009.

- [33] BELSLEY, D. A.; KUH, E.; WELSCH, R. E. *Regression Diagnostics: Identifying Influential Data and Sources of Collinearity*. Wiley, 1980. p. 244-261.
- [34] BENGIO, Y. Learning deep architectures for AI. *Foundations and Trends in Machine Learning*, v. 2, n. 1, p. 1-127, 2009.
- [35] BENGIO, Y.; COURVILLE, A.; VINCENT, P. Representation learning: a review and new perspectives. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, v. 35, n. 8, p. 1.798-1.828, 2013.
- [36] BENGIO, Y.; DUCHARME, R.; VINCENT, P.; JAUVIN, C. A neural probabilistic language model. *Journal of machine learning research*, v. 3, n. 2, p. 1137-1155, 2003.
- [37] BENGIO, Y.; LAMBLIN, P.; POPOVICI, D.; LAROCHELLE, H. Greedy layer-wise training of deep networks. In: SCHÖLKOPF, B.; PLATT, J.; HOFMANN, T. (Eds.). *Advances in Neural Information Processing Systems 19* (NIPS'06). MIT Press, 2007. p. 153-160.
- [38] BENGIO, Y.; LECUN, Y. Scaling learning algorithms towards AI. In: BOTTOU, L.; CHAPELLE, O.; DECOSTE, D.; WESTON, J. (Eds.). *Large Scale Kernel Machines*. MIT Press, 2007.
- [39] BENGIO, Y.; GOODFELLOW, I.; COURVILLE, A. *Deep Learning*. MIT Press, 2015. Disponível em: <http://www.iro.umontreal.ca/~bengioy/dlbook>.
- [40] BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, v. 5, n. 2, p. 157-166, 1994.
- [41] BENSALID, A. M.; HALL, L. O.; BEZDEK, J. C.; CLARKE, L. P.; SILBINGER, M. L.; ARRINGTON, J. A.; MURTAGH, R. F. Validity-guided (re)clustering with applications to image segmentation. *IEEE Transactions on Fuzzy Systems*, v. 4, n. 2, p. 112-123, 1996.
- [42] BERRY, M. W.; DRMAC, Z.; JESSUP, E. R. Matrices, vector spaces, and information retrieval. *SIAM Review*, v. 41, n. 2, p. 335-362, 1999.
- [43] BERRY, M. W.; KOGAN, J. (Eds.). *Text Mining: Applications and Theory*. John Wiley & Sons, 2010.

- [44] BERTHOLD, M. R.; HAND, D. J. (Eds.). *Intelligent Data Analysis*. 2. ed. Springer, 2007.
- [45] BERTHOLD, M. R.; BORGELT, C.; HÖPPNER, F.; KLAWONN, F. *Guide to Intelligent Data Analysis: How to Intelligently Make Sense of Real Data*. Springer, 2010.
- [46] BEUTHEL, C.; BOUSSOFFARA, P.; ELZER, P.; ZINSER, K.; TISSEN, A. Advantages of mass-data-displays in process s&c. In: 6TH IFAC/IFIP/IFORS/IEA SYMPOSIUM ON ANALYSIS, DESIGN, AND EVALUATION OF MAN-MACHINE SYSTEMS IFAC. *Proceedings*. 1995. p. 439-444.
- [47] BEZDEK, J. C. *Pattern Recognition with Fuzzy Objective Function Algorithms*. Nova York: Plenum, 1981.
- [48] BEZDEK, J. C. A convergence theorem for the fuzzy ISODATA clustering algorithms. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, v. 2, n. 1, p. 1-8, 1980.
- [49] BEZDEK, J. C.; HATHAWAY, R. J. VAT: a tool for visual assessment of (cluster) tendency. In: INTERNATIONAL JOINT CONFERENCE ON NEURAL NETWORKS – IJCNN 2002. *Proceedings*. 2002. p. 12-17.
- [50] BICHARA, G. L. G. *Redes Neurais Profundas para Auxílio à Tomada de Decisão no Mercado de Ações*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2019.
- [51] BILLINGS, S. A. *Nonlinear System Identification: NARMAX Methods in the Time, Frequency, and Spatio-Temporal Domains*. Wiley, 2013.
- [52] BISHOP, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [53] BLONDEL, V. D.; DECUYPER, A.; KRINGS, G. (2015). A survey of results on mobile phone datasets analysis. *EPJ Data Science*, v. 4, n. 10, 2018. doi:10.1140/epjds/s13688-015-0046-0
- [54] BODINI JR., A. C. *Utilização de Técnicas de Data Mining na Detecção de Outliers em Auxílio à Auditoria Operacional com um Estudo de Caso com Dados do Sistema de Informações Hospitalares*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2009.
- [55] BONDY, A.; MURTY, U. S. R. *Graph Theory*. Springer, 2008.

- [56] BOJANOWSKI, P.; GRAVE, E.; JOULIN, A.; MIKOLOV, T. Enriching Word Vectors with Subword Information. arXiv:1607.04606, 2016.
- [57] DE BOOR, C. A Practical Guide to Splines. *Applied Mathematical Sciences*, Springer, v. 27, 2001.
- [58] BOUTSINAS, B.; GNARDELLIS, T. On distributing the clustering process. *Pattern Recognition Letters*, v. 23, n. 8, p. 999-1.008, 2002.
- [59] BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C. *Time Series Analysis: Forecasting and Control*. 4. ed. Wiley , 2008.
- [60] BRADLEY, A. P. The use of the area under the roc curve in the evaluation of machine learning algorithms. *Pattern Recognition*, v. 30, n. 7, p. 1.145-1.159, 1997.
- [61] BRAGA, L. P. V. *Introdução à Mineração de Dados*. 2. ed. Rio de Janeiro: E-papers, 2005.
- [62] BRAGA, A. P.; CARVALHO, A. P. L. F. de; LUDERMIR, T. B. *Redes Neurais Artificiais, Teoria e Aplicações*. 2. ed. LTC, 2007.
- [63] BREIMAN, L.; SPECTOR, P. Submodel selection and evaluation in regression. The x-random case. *International Statistical Review*, v. 60, n. 3, p. 291-319, 1992.
- [64] BRIDLE, J. S. Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters. *Advances in Neural Information Processing Systems*. Morgan Kaufmann Publishers, p. 211-217, 1990.
- [65] BROCKWELL, P. J.; DAVIS, R. A. *Time Series: Theory and Methods*. 2. ed. Springer, 2009.
- [66] BROOMHEAD, D.; LOWE, D. Multivariable functional interpolation and adaptive networks. *Complex Systems*, v. 2, p. 321-355, 1988.
- [67] BRUNDAGE, M. et al. The Malicious Use of Artificial Intelligence: Forecasting, Prevention and Mitigation. arXiv:1802.07228, 2018.
- [68] BUHMANN, M. D. *Radial Basis Functions Theory and Implementations*. Cambridge University Press, 2003.

- [69] BURGESS, C. J. C. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, v. 2, p. 121-167, 1998.
- [70] CALABRESE, F.; FERRARI, L.; BLONDEL, V. D. Urban sensing using mobile phone network data: a survey of research. *ACM Computing Surveys*, v. 47, n. 2, 2015.
- [71] CALINSKI, T.; HARABASZ, J. A dendrite method for cluster analysis. *Communications in Statistics – Simulation and Computation*, v. 3, n. 1, p. 1-27, 1974.
- [72] CAMBRIA, E.; WHITE, B. Jumping NLP Curves: A Review of Natural Language Processing Research. *IEEE Computational Intelligence Magazine*, v. 9, n. 2, p. 48-57, 2014.
- [73] CANZIANI, A.; PASZKE, A.; CULURCIELLO, E. An Analysis of Deep Neural Network Models for Practical Applications. arXiv:1605.07678, 2016.
- [74] CARVALHO, L. A. V. de. *Data mining – A Mineração de Dados no Marketing, Medicina, Economia, Engenharia e Administração*. Ciência Moderna, 2005.
- [75] CASTRO, J. L.; FLORES-HIDALGO, L. D.; MANTAS, C. J.; PUCHE, J. M. Extraction of fuzzy rules from support vector machines. *Fuzzy Sets and Systems*, v. 158, p. 2.057-2.077, 2007.
- [76] CAUCHY, A. Méthode générale pour la résolution des systèmes d'équations simultanées. *Compte Rendu de l'Academie des Sciences*, v. 24, n. 1, p. 536-538, 1847.
- [77] CAUSEY, R. L. *Logic, Sets and Recursion*. 2. ed. Jones & Bartlett Publishers, 2005.
- [78] CAVALCANTE, R. C.; BRASILEIRO, R. C.; SOUZAM, V. L.; NOBREGA, V. L.; OLIVEIRA, A. L. Computational intelligence and financial markets: A survey and future directions. *Expert Systems with Applications*, v. 55, p. 194-211. 2016.
- [79] CAYROL, M.; FARRENY, H.; PRADE, H. Fuzzy pattern matching. *Kybernetes*, v. 11, n. 2, p. 103-116, 1982.
- [80] CELEBI, M. E.; KINGRAVI, H. A.; VELA, P. A. A comparative study of efficient initialization methods for the k-means clustering algorithm. *Expert Systems with Applications*, v. 40, p. 200-210, 2013.

- [81] CHAN, W.; JAITLEY, N.; LE, Q. V.; VINYALS, O. Listen, Attend and Spell. arXiv:1508.01211, 2015.
- [82] CHAPELLE, O.; SCHÖLKOPF, B.; ZIEN, A. (Eds.). *Semi-Supervised Learning*. The MIT Press, 2006.
- [83] CHARIKAR, M.; GUHA, S.; TARDOS, E.; SHMOYS, D. B. A constant-factor approximation algorithm for the k-median problem. In: 31ST ANNUAL ACM SYMPOSIUM ON THEORY OF COMPUTING. *Proceedings*. 1999. p. 1-10.
- [84] CHARNES, A.; FROME, E. L.; YU, P. L. The equivalence of generalized least squares and maximum likelihood estimates in the exponential family. *Journal of the American Statistical Association*, v. 71, n. 353, p. 169-171, 1976.
- [85] CHAUVIN, Y.; RUMELHART, D. E. (Eds.). *Backpropagation: Theory, Architectures, and Applications*. Psychology Press, 1995.
- [86] CHAVES, J. C. *Avaliação de Segmentações Urbanas Através de Inteligência Computacional*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2011.
- [87] CHAVES, J. C. *Metodologia para Estimativa de Matrizes Origem-Destino a partir de Dados de Telefonia Celular*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2018.
- [88] CHAVES, J. C.; CARVALHO, G. S. D.; SILVA, M. A. H. B.; EVSUKOFF, A. G. Padrões de Mobilidade Humana na Região Metropolitana do Rio de Janeiro. Em: REIA, J.; FRANCISCO, P. A. P.; BARROS, M.; MAGRANI, E. (Eds.). *Horizonte Presente: Tecnologia e Sociedade em Debate*. Editora Letramento, p. 86-105, 2018.
- [89] CHAWLA, N. V.; BOWYER, K. W.; HALL, L. O.; KEGELMEYER, W. P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, v. 16, p. 321-357, 2002.
- [90] CHAWLA, N. V.; JAPKOWICZ, N.; KOLCZ, A. Editorial: SIGKDD Explorations, Special Issue on Class Imbalances. *SIGKDD Explorations*, v. 6, n. 1, 2004.
- [91] CHEN, J.; PATTON, R. J. *Robust Model-Based Fault Diagnosis for dynamic Systems*. Kluwer Academic Publishers, 1999.

- [92] CHEN, S.; BILLINGS, S. A.; LUO, W. Orthogonal least squares methods and their application to nonlinear system identification. *International Journal of Control*, v. 50, n. 5, p. 1.873-1.896, 1989.
- [93] CHEN, S.; COWAN, C. F. N.; GRANT, P. M. Orthogonal least squares learning algorithm for radial basis function networks. *IEEE Trans. on Neural Networks*, v. 2, n. 2, p. 302-309, 1991.
- [94] CHEN, S. F.; GOODMAN, J. T. An empirical study of smoothing techniques for language modeling. *Computer, Speech and Language*, v. 13, n. 4, p. 359-393, 1999.
- [95] CHEN, S.; WU, Y.; LUK, B. L. Combined genetic algorithm optimization and regularized orthogonal least squares learning for radial basis function networks. *IEEE Neural Networks*, v. 10, n. 5, p. 1.239-1.243, 1999.
- [96] CHEN, Y.; WANG, J. Z. Support vector learning for fuzzy rule-based classification systems. *IEEE Trans. on Fuzzy Systems*, v. 11, n. 6, p. 716-728, 2003.
- [97] CHENEY, E. W. *Introduction to Approximation Theory*. McGraw-Hill, 1966.
- [98] CHERKASSKY, V.; MULIER, F. M. *Learning from Data: Concepts, Theory, and Methods*. 2. ed. Wiley-IEEE Press, 2007.
- [99] CHIANG, J.; HAO, P. Support vector learning mechanism for fuzzy rule-based modeling: a new approach. *IEEE Trans. on Fuzzy Systems*, v. 12, n. 1, p. 1-12, 2004.
- [100] CHIU, C.-C.; et al. State-of-the-art Speech Recognition with Sequence-to-Sequence Models. arXiv:1712.01769, 2017.
- [101] CHO, K.; VAN MERRIENBOER, B.; GULCEHRE, C.; BOUGARES, F.; SCHWENK, H.; BENGIO, Y. Learning phrase representations using rnn encoderdecoder for statistical machine translation. arXiv:1406.1078, 2014.
- [102] CHO, K.; VAN MERRIËNBOER, B.; BAHDANAU, D.; BENGIO, Y. On the properties of neural machine translation: Encoder-decoder approaches. arXiv:1409.1259, 2014.

- [103] CHOU, S. C.; TANAJURA, C. A. S.; XUE, Y.K.; NOBRE, C. A. Validation of the coupled Eta/SSiB model over South America. *Journal of Geophysical Research*, v. 107, n. 20, p. 537-555, 2002.
- [104] CHUNG, F. R. K. *Spectral Graph Theory*: American Mathematical Society CBMS 92, 1997.
- [105] CHUNG, J.; GULCEHRE, C.; CHO, K.; BENGIO, Y. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. arXiv:1412.3555, 2014.
- [106] CICHOCKI, A.; ZDUNEK, R.; PHAN, A. H.; AMARI, S. *Nonnegative Matrix and Tensor Factorizations: Applications to Exploratory Multi-way Data Analysis and Blind Source Separation*. Wiley, 2009.
- [107] CISTER, A. M. *Mineração de Dados para a Análise de Atrito em Telefonía*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2005.
- [108] CLAUSET, A.; SHALIZI, C. R.; NEWMAN, M. E. J. Power-law distributions in empirical data. *SIAM Review*, v. 51, n. 4, p. 661-703, 2009.
- [109] CLOSE, C. M.; FREDERICK, D. K.; NEWELL, J. C. *Modeling and Analysis of Dynamic Systems*. 3. ed. Wiley, 2001.
- [110] COHEN, J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, v. 20, n. 1, p. 37-46, 1960.
- [111] COLLOBERT, R.; PUHRSCHE, C.; SYNNAEVE, G. Wav2Letter: an End-to-End ConvNet-based Speech Recognition System. arXiv:1609.03193, 2016.
- [112] COLEMAN, T. F.; LI, Y. A reflective Newton method for minimizing a quadratic function subject to bounds on some of the variables. *SIAM Journal on Optimization*, v. 6, n. 4, p. 1.040-1.058, 1996.
- [113] COOK, D. J.; HOLDER, L. B. (Eds.). *Mining Graph Data*. John Wiley & Sons, 2006.
- [114] CORDON, O.; HERRERA, F.; PEREGRIN, A. Applicability of the fuzzy operators in the design of fuzzy logic controllers. *Fuzzy Sets and Systems*, v. 86, p. 15-41, 1997.
- [115] CORMACK, G. V. Email Spam Filtering: A Systematic Review. *Foundations and Trends in Information Retrieval*, v. 1, n. 4, p. 335-455, 2008.
- [116] CORTES, C.; VAPNIK, V. Support-vector networks. *Machine Learning*, v. 20, p. 273-297, 1995.

- [117] COTTINEAU C.; VANHOOF, M. Mobile Phone Indicators and Their Relation to the Socioeconomic Organisation of Cities. *Int. Journal of Geo-Information*, v. 8, n. 1, p. 1-19, 2019. <https://doi.org/10.3390/ijgi8010019>
- [118] CRISTIANINI, N.; SHAWE-TAYLOR, J. *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. Cambridge University Press, 2000.
- [119] CYBENKO, G. Approximations by superpositions of sigmoidal functions, *Mathematics of Control. Signals, and Systems*, v. 2, n. 4, p. 303-314, 1989.
- [120] DARWICHE, A. *Modeling and Reasoning with Bayesian Networks*. Cambridge University Press, 2009.
- [121] DARWICHE, A. Bayesian networks. *Communications of the ACM*, v. 53, n. 12, 2010.
- [122] DEERWESTER, S.; DUMAIS, S. T.; FURNAS, G. W.; LANDAUER, T. K.; HARSHMAN, R. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*. v. 41, n. 6, p. 391-407, 1990.
- [123] DEVLIN, J. CHANG, M.-W.; LEE, K.; TOUTANOVA, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805, 2018.
- [124] DEMPSTER, A. P.; LAIRD, N. M.; RUBIN, D. B. Maximum Likelihood from Incomplete Data via the EM Algorithm. *Journal of the Royal Statistical Society*, v. 39, n. 1, p 1-38, 1977.
- [125] DEMŠAR, J. Statistical comparisons of classifiers over multiple datasets. *Journal of Machine Learning Research*, v. 7, p. 1-30, 2006.
- [126] DHILLON, I. S.; MODHA, D. M. Concept decompositions for large sparse text data using clustering. *Machine Learning*, v. 42, n. 1, p. 143-175, 2001.
- [127] DHILLON, I. S.; GUAN, Y.; KULIS, B. Kernel k-means, spectral clustering and normalized cuts. In: 10TH ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING (KDD'04). *Proceedings*. 2004. p. 551-556.

- [128] DHILLON, I. S.; GUAN, Y.; KULIS, B. Weighted graph cuts without eigenvectors: a multilevel approach. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 29, n. 11, p. 1.944-1.957, 2007.
- [129] DIESTEL, R. *Graph Theory*. 3. ed. Springer, 2008.
- [130] DING, C.; DING, C. H. Q.; HE, X.; ZHA, H.; GU, M.; SIMON, H. A min-max cut algorithm for graph partitioning and data clustering. In: IEEE INTERNATIONAL CONFERENCE ON DATA MINING (ICDM 2001). *Proceedings*. 2001. p. 107-114.
- [131] DING, C.; HE, X. K-means clustering via principal component analysis. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML 2004). *Proceedings*. 2004. p. 225-232.
- [132] DING, C.; LI, T. Adaptive dimension reduction using discriminant analysis and k-means clustering. In: 24TH INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML 2007). *Proceedings*. 2007. p. 521-528.
- [133] DING, X.; ZHANG, Y.; LIU, T.; DUAN, J. Using Structured Events to Predict Stock Price Movement: An Empirical Investigation. In: CONFERENCE ON EMPIRICAL METHODS ON NATURAL LANGUAGE PROCESSING (EMNLP 2014). *Proceedings*. 2014. p. 1415-1425.
- [134] DING, X.; ZHANG, Y.; LIU, T.; DUAN, J. Deep Learning for Event-Driven Stock Prediction. In: INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE (IJCAI 2015). *Proceedings*. 2015. p. 2327-2333.
- [135] DINIZ, M. A.; IZBICKI, R.; LOPES, D.; SALASAR, L. E. Comparing probabilistic predictive models applied to football. *Journal of the Operational Research Society*, doi: 10.1080/01605682.2018.1457485, p. 1-13, 2018.
- [136] DOBSON, A. J.; BARNETT, A. *An Introduction to Generalized Linear Models*. 3. ed. Chapman & Hall, 2008.
- [137] DOMINGOS, P. A few useful things to know about machine learning. *Communications of the ACM*, v. 55, n. 10, p. 78-87, 2012.
- [138] DOUGHERTY, C. *Introduction to Econometrics*. 4. ed. Oxford University Press, 2011.

- [139] DUBOIS, D.; PRADE, H. Fuzzy sets and probability: misunderstandings, bridges and gaps. In: SECOND IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS 1993. *Proceedings*. São Francisco, 1993. p. 1.059-1.068.
- [140] DUBOIS, D.; PRADE, H. *Possibility Theory*. Plenun Press, 1988.
- [141] DUBOIS, D.; PRADE, H. The three semantics of fuzzy sets. *Fuzzy Sets and Systems*, v. 90, n. 2, p. 141-150, 1997.
- [142] DUBOIS, D.; PRADE, H.; TESTEMALE, C. Weighted fuzzy pattern matching. *Fuzzy Sets and Systems*, v. 28, n. 3, p. 313-331, 1988.
- [143] DUBOIS, D. Possibility theory and statistical reasoning. *Computational Statistics & Data Analysis*, v. 51, n. 1, p. 47-69, 2006.
- [144] DUBOIS, D.; HÜLLERMEIER, E.; PRADE, H. A systematic approach to the assessment of fuzzy association rules. *Data Mining and Knowledge Discovery*, v. 13, n. 2, p. 167-192, 2006.
- [145] DUCHI J.; HAZAN, E.; SINGER, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, v. 12, p. 2121-2159, 2011.
- [146] DUDA, R. O.; HART, P. E.; STORK, D. G. *Pattern Classification*. 2. ed. Wiley-Interscience, 2000.
- [147] DUMOULIN, V.; VISIN, F. A guide to convolution arithmetic for deep learning. arXiv:1603.07285, 2016.
- [148] DUNN, J. C. A Fuzzy relative of the ISODATA process and its use in detecting compact well-separated clusters. *Journal of Cybernetics*, v. 3, n. 3, p. 32-57, 1973.
- [149] ELDÉN, L. *Matrix Methods in Data Mining and Pattern Recognition*. Society for Industrial and Applied Mathematics, 2007.
- [150] ELDER, J. F.; SENI, G. *Ensemble Methods in Data Mining: Improving Accuracy Through Combining Predictions*. Morgan & Claypool Publishers, 2010.
- [151] ELMAN, J. L. Finding Structure in Time. *Cognitive Science*, v. 14, n. 2, p. 179-211, 1990.
- [152] ENGELBRECHT, A. P. *Computational Intelligence: An Introduction*. 2. ed. John Wiley & Sons, 2007.

- [153] ESPEJO, P.; VENTURA, S.; HERRERA, F. A Survey on the Application of Genetic Programming to Classification. *IEEE Trans. on Systems, Man, and Cybernetics – Part C: Applications and Reviews*, v. 40, n. 2, p. 121-144, 2010.
- [154] EVERITT, B. S.; HAND, D. J. Finite mixture distributions. Chapman & Hall. 1981.
- [155] EVSUKOFF, A. G.; GENTIL, S.; BRANCO, A. C. S. A knowledge acquisition method for fuzzy expert systems in diagnosis problems. In: SIXTH IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. Barcelona, 1997. p. 1.411-1.416.
- [156] EVSUKOFF, A. G. *Le Raisonnement Approche pour la Surveillance de Procédés*. Thèse. l'Institut National Polytechnique de Grenoble, INPG, 1998.
- [157] EVSUKOFF, A. G.; GENTIL, S.; MONTMAIN, J. Fuzzy reasoning in co-operative supervision systems. *Control Engineering Practice*, v. 8, n. 4, p. 389-407, 2000.
- [158] EVSUKOFF, A. G.; BRANCO, A. C. S.; GALICHET, S. Incremental Fuzzy Modeling. In: 10TH IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. Melbourne, 2001. p. 1.594-1.597.
- [159] EVSUKOFF, A. G.; BRANCO, A. C. S.; GALICHET, S. Structure identification and parameter optimization for nonlinear fuzzy modeling. *Fuzzy Sets and Systems*, v. 132, n. 2, p. 173-188, 2002.
- [160] EVSUKOFF, A. G.; EBECKEN, N. F. F. Mining fuzzy rules for a traffic information system, Proc. of the Seventh International Conference on Knowledge-Based Intelligent Information & Engineering Systems. *Lecture Notes in Artificial Intelligence (LNAI)*, v. 2.773, p. 237-243, 2003.
- [161] EVSUKOFF, A. G.; EBECKEN, N. F. F. Identification of recurrent fuzzy systems with genetic algorithms. In: 2004 IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. v. 3. Budapeste, 2004. p. 1.703-1.708.
- [162] EVSUKOFF, A. G.; GONÇALVES, F. T. T.; BEDREGAL, R. P.; EBECKEN, N. F. F. Fuzzy Classification of Surface Geochemistry Data Applied to the Determination of HC Anomalies. In: IEEE JOINT 2ND INTERNATIONAL CONFERENCE ON SOFT COMPUTING AND INTELLIGENT SYSTEMS (SCIS & ISIS'2004). *Proceedings*. 2004.

- [163] EVSUKOFF, A. G.; GENTIL, S. Recurrent neuro-fuzzy system for fault detection and isolation in nuclear reactors. *Advanced Engineering Informatics*, v. 19, n. 1, p. 55-66, 2005.
- [164] EVSUKOFF, A. G.; COSTA, M. C. A.; EBECKEN, N. F. F. Parallel Implementation of a Fuzzy Rule Based Classifier. *Lecture Notes in Computer Science*. v. 3.402, p. 184-193, 2005.
- [165] EVSUKOFF, A. G.; BRANCO, A. C. S.; EBECKEN, N. F. F. Writing Weighted Fuzzy Queries from Fuzzy Classifier Rules. In: 11TH IPMU INTERNATIONAL CONFERENCE. *Proceedings*. 2006.
- [166] EVSUKOFF, A. G.; FAIRBAIRN, E. M. R.; FARIA, E. F. de; SILVOSO, M. M.; TOLEDO FILHO, R. D. Modeling adiabatic temperature rise during concrete hydration: a data mining approach. *Computers and Structures*, v. 84, p. 2.351-2.362, 2006.
- [167] EVSUKOFF, A. G. Learning fuzzy rule-based classifier with rule weights optimization and structure selection by a genetic algorithm. In: 2007 IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. Londres, 2007.
- [168] EVSUKOFF, A. G.; GALICHET, S.; LIMA, B. S. L. P. de; EBECKEN, N. F. F. Design of interpretable fuzzy rule-based classifiers using spectral analysis with structure and parameters optimization. *Fuzzy Sets and Systems*, v. 160, n. 7, p. 857-881, 2009.
- [169] EVSUKOFF, A. G.; LIMA, B. S. L. P. de; EBECKEN, N. F. F. Long-term runoff modeling using rainfall forecasts with application to the Iguaçu river basin. *Water Resources Management*, v. 25, p. 963-985, 2010.
- [170] EVSUKOFF, A. G.; BRANCO, A. C. S.; GALICHET, S. Intelligent data analysis and model interpretation with spectral analysis fuzzy symbolic modeling. *International Journal of Approximate Reasoning*, v. 52, n. 6, p. 728-750, 2011.
- [171] EVSUKOFF, A. G.; CATALDI, M.; LIMA, B. S. L. P. de. A multi-model approach for long-term runoff modeling using rainfall forecasts. *Expert Systems with Applications*, v. 39, n. 5, p. 4.938-4.946, 2012.
- [172] EYCKHOFF, P. *System Identification: Parameters and State Estimation*. John Wiley & Sons, 1974.

- [173] FACELI, K.; LORENA A. C.; GAMA J.; CARVALHO, A. C. P. L. F. de *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*, LTC, 2011.
- [174] FARIA, E. F. de. *Predição da Exotermia da Reação de Hidratação do Concreto Através de Modelo Termo-Químico e Modelo de Dados*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2004.
- [175] FAUSETT, L. *Fundamentals of Neural Networks: Architectures, Algorithms and Applications*. Prentice-Hall, 1994.
- [176] FAWCETT, T. An introduction to ROC analysis. *Pattern Recognition Letters*, v. 27, n. 8, p. 861-874, 2006.
- [177] FAYYAD, U. M.; PIATETSKY-SHAPIO, G.; SMYTH, P.; UTHURUSAMY, R. (Eds.). *Advances in Knowledge Discovery and Data Mining*. MIT Press, 1996.
- [178] FELDMAN, R.; SANGER, J. *The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data*. Cambridge University Press, 2006.
- [179] FIEDLER, M. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, v. 23, n. 98, p. 298-305, 1973.
- [180] FIEDLER, M. A property of eigenvectors of nonnegative symmetric matrices and its application to graph theory. *Czechoslovak Mathematical Journal*, v. 25, n. 4, p. 619-633, 1975.
- [181] FIGUEIREDO, M. *Método para Representação de Conceitos por Meio de Técnicas de Análise de Textos em Sequência Temporal*. Tese (doutorado), COPPE/UFRJ, Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2017.
- [182] FILIPPONE, M.; CAMASTRA, F.; MASULLI, F.; ROVETTA, S. A survey of kernel and spectral methods for clustering. *Pattern Recognition*, v. 41, n. 1, p. 176-190, 2008.
- [183] FISHER, R. A. The Use of Multiple Measurements in Taxonomic Problems. *Annals of Eugenics*, v. 7, p. 179-188, 1936.
- [184] FREITAS, A. *Data Mining and Knowledge Discovery with Evolutionary Algorithms*. Springer, 2002.
- [185] FREUND, Y. Boosting a weak learning algorithm by majority. *Information and Computation*, v. 121, n. 2, p. 256-285, 1995.

- [186] FREUND, Y.; SCHAPIRE, R. A decision-theoretic generalization of online learning and an application to boosting. *Journal on Computational Learning Theory*, v. 55, p. 119-139, 1997.
- [187] FRIEDMAN, J. H. An overview of predictive learning and function approximation. In: CHERKASSKY, V.; FRIEDMAN, J. H.; WECHSLER, H. (Eds.). *From Statistics to Neural Networks*. Springer, 1994. (NATO ASI Series F, 136).
- [188] FUJITA, O. Statistical estimation of the number of hidden units for feedforward neural networks. *Neural Networks*, v. 11, n. 5, p. 851-859, 1998.
- [189] FUKUNAGA, K. *Introduction to Statistical Pattern Recognition*. 2. ed. Academic Press, 1990.
- [190] GALICHET, S.; FOULLOY, L. Fuzzy controllers: synthesis and equivalences. *IEEE Trans. on Fuzzy Systems*, v. 3, n. 2, p. 140-148, 1995.
- [191] GAMA, C. A.; EVSUKOFF, A. G.; WEBER, P.; EBECKEN, N. F. F. Parameter Identification of Recurrent Fuzzy Systems with Fuzzy Finite-State Automata Representation. *IEEE Transactions of Fuzzy Systems*, v. 16, n. 1, p. 213-224, 2008.
- [192] GAMA, J. *Knowledge Discovery from Data Stream*. Chapman and Hall; CRC, 2010.
- [193] GAMA, J.; CARVALHO, A. C. P. L. F. de; FACELI, K.; LORENA, A. C.; OLIVEIRA, M. *Extração de Conhecimento de Dados Data Mining*. Sílabo, 2012.
- [194] GAN, G.; MA, C.; WU, J. *Data Clustering: Theory, Algorithms and Applications*. SIAM, 2007.
- [195] GARCIA, S.; FERNANDEZ, A.; LUENGO, J.; HERRERA, F. Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power. *Information Sciences*, v. 180, n. 10, p. 2.044-2.064, 2010.
- [196] GARG, A.; AGARWAL, M.; Machine Translation: A Literature Review. arXiv:1901.01122, 2019.
- [197] GEHRING, J.; AULI, M.; GRANGIER, D.; YARATS, D.; DAUPHIN Y. N. Convolutional Sequence to Sequence Learning. arXiv:1705.03122, 2017.
- [198] GERS, F. A.; SCHMIDHUBER, J.; CUMMINS, F. Learning to Forget: Continual Prediction with LSTM. *Neural Computation*, v. 12, n. 10, p. 2451-2471, 2000.

- [199] GERS, F.; SCHRAUDOLPH, N.; SCHMIDHUBER, J. Learning precise timing with LSTM recurrent networks. *Journal of Machine Learning Research*, v. 3, p. 115–143, 2002.
- [200] GERTLER, J. *Fault Detection and Diagnosis in Engineering Systems*. Nova York: Marcel Decker Inc., 1998.
- [201] GERTLER, J. Fault detection and isolation using parity relations. *Control Eng. Practice*, v. 5, n. 5, p. 653-661, 1997.
- [202] GILL, P. E.; MURRAY, W.; WRIGHT, M. H. *Practical Optimization*. Academic Press, 1981.
- [203] GLENBERG, A. M.; ROBERTSON, D. A. Symbol grounding and meaning: A comparison of high-dimensional and embodied theories of meaning. *Journal of memory and language*, v. 43, n. 3, p. 379–401, 2000.
- [204] GUILLAUME, S. Designing fuzzy inference systems from data: an interpretability-oriented review. *IEEE Trans. on Fuzzy Systems*, v. 9, n. 3, p. 423-443, 2001.
- [205] GOLDBERG, D. E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley Professional, 1989.
- [206] GOLUB, G. H.; VAN LOAN, C. F. *Matrix Computations*. 3. ed. The Johns Hopkins University Press, 1996.
- [207] GOLUB, G. H.; ZHA, H. *The canonical correlations and their numerical computation*. Dept. of Computer Science, Pennsylvania State University, 1992. (Report CS-92-21).
- [208] GONZALEZ, J.; ROJAS, I.; ORTEGA, J.; POMARES, H.; FERNANDEZ, J.; DIAZ, A. F. Multiobjective evolutionary optimization of the size, shape, and position parameters of radial basis function networks for function approximation. *IEEE Transactions on Neural Networks*, v. 14, n. 6, p. 1.478-1.495, 2003.
- [209] GOODFELLOW, I. J. On distinguishability criteria for estimating generative models. arXiv:1412.6515. 2014.
- [210] GOODFELLOW, I. NIPS 2016 Tutorial: Generative Adversarial Networks. arXiv:1701.00160. 2017.

- [211] GOODFELLOW, I.; POUGET-ABADIE, J.; MIRZA, M.; XU, B.; WARDE-FARLEY, D.; OZAIR, S.; COURVILLE, A.; BENGIO, Y. Generative adversarial networks. arXiv:1406.2661. 2014.
- [212] GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016.
- [213] GOYAL, P.; Ferrara, E. Graph embedding techniques, applications, and performance: A survey. *Knowledge-Based Systems*, v. 151, p. 78-94, 2018.
- [214] GRABISCH, M.; SUGENO, M. Multi-attribute classification using fuzzy integral. In: IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. 1992. p. 47-54.
- [215] GRAVES, A. Sequence Transduction with Recurrent Neural Networks arXiv:1211.3711, 2012.
- [216] GRAVES, A. Generating Sequences with Recurrent Neural Networks. arXiv:1308.0850, 2013.
- [217] GRAVES, A.; FERNANDEZ, S.; GOMEZ, F.; SCHMIDHUBER, J. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In: 23RD INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML 2006). *Proceedings*. 2006. p. 369– 376.
- [218] GRAVES, A.; MOHAMED, A.; HINTON, G. Speech recognition with deep recurrent neural networks. arXiv:1303.5778. In: INTERNATIONAL CONFERENCE ON ACOUSTICS, SPEECH, AND SIGNAL PROCESSING (ICASSP'2013). *Proceedings*. 2013. p. 6645–6649.
- [219] GRAVES, A.; JAITLEY, N. Towards end-to-end speech recognition with recurrent neural networks. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML'2014). *Proceedings*. 2014.
- [220] GREFF, K.; SRIVASTAVA, R. K.; KOUTNÍK, J.; STEUNEBRINK, B. R.; SCHMIDHUBER, R. LSTM: A Search Space Odyssey *IEEE Trans. on Neural Networks and Learning Systems*, v. 28, n. 10, p. 2222-2232, 2017.
- [221] GU, J. et al. Recent advances in convolutional neural networks. *Pattern Recognition*, v.77, p. 354-377, 2018.

- [222] GUILHON, L. G. F.; ROCHA, V. F.; MOREIRA, J. C. Comparação de métodos de previsão de vazões naturais afluentes aos aproveitamentos hidrelétricos. *Revista Brasileira de Recursos Hídricos*, v. 12, p. 1-7, 2007.
- [223] GUSTAFSON, D. E.; KESSEL, W. C. Fuzzy clustering with a fuzzy covariance matrix. In: IEEE CONF. ON DECISION AND CONTROL (IEEE CDC 1978). *Proceedings*. 1978. p. 761-766.
- [224] GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. *Journal of Machine Learning Research*, v. 3, p. 1.157-1.182, 2003.
- [225] HAGAN, M. T.; DEMUTH, H. B.; BEALE, M. H. *Neural Network Design*. University of Colorado, 2002.
- [226] HAHNLOSER, R.; SARPESHKAR, R.; MAHOWALD, M. A.; DOUGLAS, R. J.; SEUNG, S. Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit. *Nature* v. 405, n. 6789, p. 947-951, 2000.
- [227] HALKIDI, M.; BATISTAKIS, Y.; VAZIRGIANNIS, M. Clustering validity methods: part I. *ACM SIGMOD Record*, v. 31, n. 2, 2002.
- [228] HALKIDI, M.; BATISTAKIS, Y.; VAZIRGIANNIS, M. Clustering validity checking methods: part II. *ACM SIGMOD Record*, v. 31, n. 3, 2002.
- [229] HAMERLY, G.; ELKAN, C. Alternatives to the k-means algorithm that find better clusterings. In: 11TH INTERNATIONAL CONFERENCE ON INFORMATION AND KNOWLEDGE MANAGEMENT. *Proceedings*. 2002. p. 600-607.
- [230] HAMILTON, J. D. *Time Series Analysis*. Princeton University Press, 1994.
- [231] HAMILTON, W. L.; YING, R.; LESKOVEC, J. Representation Learning on Graphs: Methods and Applications, arXiv:1709.05584, 2017.
- [232] HAN, J.; KAMBER, M. *Data Mining: Concepts and Techniques*. 1. ed. Morgan Kaufmann Publishers, 2000.
- [233] HAN, J.; KAMBER, M. *Data Mining: Concepts and Techniques*. 2. ed. Morgan Kaufmann Publishers, 2006.
- [234] HAN, J.; KAMBER, M. *Data Mining: Concepts and Techniques*. 3. ed. Morgan Kaufmann Publishers, 2011.

- [235] HAND, D. J.; TILL, R. J. A simple generalization of the area under the roc curve for multiple class classification problems. *Machine Learning*, v. 45, p. 171-186, 2001.
- [236] HANNUN, A.; CASE, C.; CASPER, J.; CATANZARO, B.; DIAMOS, G.; ELSSEN, E.; PRENGER, R.; SATHEESH, S.; SENGUPTA, S.; COATES, A.; NG, A. Y. Deep Speech: Scaling up end-to-end speech recognition. arXiv:1412.5567, 2014.
- [237] HARTMANN, N.; FONSECA, E.; SHULBY, C.; TREVISIO, M.; RODRIGUES, J.; ALUISIO, S. Portuguese Word Embeddings: Evaluating on Word Analogies and Natural Language Tasks. arXiv:1708.06025, 2017
- [238] HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. Springer, 2003.
- [239] HATHAWAY, R. J.; BEZDEK, J. C. Visual cluster validity for prototype generator clustering models. *Pattern Recognition Letters*, v. 24, n. 9-10, p. 1.563-1.569, 2003.
- [240] HAYKIN, S. *Neural Networks and Learning Machines*. 3. ed. Prentice Hall, 2008.
- [241] HAYKIN, S. *Redes Neurais, Princípios e Práticas*. Bookman, 2001.
- [242] HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep Residual Learning for Image Recognition. arXiv:1512.03385, 2015.
- [243] HE, K.; ZHANG, X.; REN, S. SUN, J. Identity Mappings in Deep Residual Networks. arXiv:1603.05027, 2016.
- [244] HEATH, M. T. *Scientific Computing*. 2. ed. McGraw-Hill, 2002.
- [245] HEBB, D. O. *The Organization of Behavior*. John Wiley & Sons, 1949.
- [246] HECHT-NIELSEN, R. Kolmogorov's mapping neural network existence theorem. In: IEEE FIRST ANNUAL INTERNATIONAL CONFERENCE ON NEURAL NETWORKS. *Proceedings*. v. 3. 1987. p. 11-14.
- [247] HECKERMAN, D. Bayesian Networks for Data Mining. *Data Mining and Knowledge Discovery*, v. 1, n. 1, 1997.
- [248] HESTENES, M. R.; STIEFEL, E. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, v. 49, n. 6, p. 409-436, 1952.

- [249] HINTON, G.; DENG, L.; YU, D.; DAHL, G.; MOHAMED, A.-R.; JAITLEY, N.; VANHOUCHE, V.; NGUYEN, P.; SAINATH, T.; KINGSBURY, B. Deep Neural Networks for Acoustic Modeling in Speech Recognition. *IEEE Signal Processing Magazine*, v. 29, n. 6, p. 82-97, 2012.
- [250] HINTON, G. E. Connectionist learning procedures. *Artificial Intelligence*, v. 40, n. 1-3, p. 185-234, 1989.
- [251] HINTON, G. E.; SALAKHUTDINOV, R. Reducing the dimensionality of data with neural networks. *Science*, v. 313, n. 5.786, p. 504-507, 2006.
- [252] HOCHREITER, S.; SCHMIDHUBER, J. Long Short-Term Memory. *Neural Computation*, v. 9, n. 8, p. 1735-1780, 1997.
- [253] HOCKING, R. R. The Analysis and Selection of Variables in Linear Regression. *Biometrics*, v. 32, n. 1, p. 1-49, 1976.
- [254] HODGE, V.; AUSTIN, J. A survey of outlier detection methodologies. *Artificial Intelligence Review*, v. 22, n. 2, p. 85-126, 2004.
- [255] HOERL, A. E.; KENNARD, R. W. Ridge regression: applications to nonorthogonal problems. *Technometrics*, v. 12, n. 1, p. 69-82, 1970.
- [256] HOERL, A. E.; KENNARD, R. W.; BALDWIN, K. F. Ridge regression: some simulations. *Communication in Statistics*, v. 4, p. 105-123, 1975.
- [257] HOFMANN, T.; SCHÖLKOPF, B.; SMOLA, A. J. Kernel Methods in Machine Learning. *The Annals of Statistics*, v. 36, n. 3, p. 1.171-1.220, 2008.
- [258] HOPFIELD, J. J. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences of the USA*, v. 79, n. 8, p. 2.554-2.558, 1982.
- [259] HÖPPNER, F.; KLAWONN, F.; KRUSE, R.; RUNKLER, T. *Fuzzy Cluster Analysis: Methods for Classification, Data Analysis and Image Recognition*. John Wiley & Sons, 1999.
- [260] HORNIK, K. Approximation Capabilities of Multilayer Feedforward Networks. *Neural Networks*, v. 4, n. 2, p. 251-257, 1991.
- [261] HOTELLING, H. Relations between two sets of variates. *Biometrika*, v. 28, p. 312-377, 1936.

- [262] HOWARD, A. G.; ZHU, M.; CHEN, B.; KALENICHENKO, D.; WANG, W.; WEYAND, T.; ANDREETTO, M.; ADAM, H. MobiLeNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861, 2017.
- [263] HRUSCHKA JR., E. R. *Imputação bayesiana no Contexto da Mineração de Dados*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2003.
- [264] HSU, W.; LEE, M. L.; WANG, J. *Temporal and Spatio-temporal Data Mining*. IGI Publishing, 2007.
- [265] HU, J.; SHEN, L.; ALBANIE, S.; SUN, G.; WU, E. Squeeze-and-Excitation Networks. arXiv:1709.01507, 2017
- [266] HUANG, D. S. Radial basis probabilistic neural networks: model and application. *International Journal of Pattern Recognition and Artificial Intelligence*, v. 13, n. 7, p. 1.083-1.101, 1999.
- [267] HUANG, G.-B.; BABRI, H. A. Upper bounds on the number of hidden neurons in feedforward networks with arbitrary bounded nonlinear activation functions. *IEEE Transactions on Neural Networks*, v. 9, n. 1, p. 224-229, 1998.
- [268] HUANG, G.; LIU, Z.; VAN DER MAATEN, L.; WEINBERGER, K. Q. Densely Connected Convolutional Networks. arXiv:1608.06993, 2016.
- [269] HUANG, T.-M.; KECMAN, V.; KOPRIVA, I. *Kernel Based Algorithms for Mining Huge Data Sets: Supervised, Semi-supervised, and Unsupervised Learning*. Springer Berlin Heidelberg, 2009.
- [270] HUBERT, M.; ROUSSEUW, P. J.; VAN AELST, S. Multivariate outlier detection and robustness – review article. *Handbook of Statistics*, v. 24, p. 263-302, 2005.
- [271] HÜLLERMEIER, E. Fuzzy methods in machine learning and data mining: status and prospects. *Fuzzy Sets and Systems*, v. 156, p. 387-406, 2005.
- [272] HÜLLERMEIER, E. Fuzzy sets in machine learning and data mining. *Applied Soft Computing*, v. 11, n. 2, p. 1.493-1.505, 2011.
- [273] VAN HULSE, J.; KHOSHGOFTAAR, T. Knowledge discovery from imbalanced and noisy data. *Data & Knowledge Engineering*, v. 68, n. 12, p. 1.513-1.542, 2009.

- [274] HUNTER, D.; YU, H.; PUKISH III, M. S.; KOLBUSZ, J.; WILAMOWSKI, B. M. Selection of proper neural network sizes and architectures: a comparative study. *IEEE Transactions on Industrial Informatics*, v. 8, n. 2, p. 228-240, 2012.
- [275] IOFFE, S.; SZEGEDY, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, arXiv:1502.03167, 2015.
- [276] ISERMAN, R. Supervision, fault-detection and fault-dianosis methods – an introduction. *Control Eng. Practice*, v. 5, n. 5, p. 639-652, 1997.
- [277] ISERMAN, R. On fuzzy logic applications for automatic control, supervision, and fault diagnosis. *IEEE Trans. on Systems Man and Cybernetics, Part A: Systems and Humans*, v. 28, n. 2, p. 221-234, 1998.
- [278] ISHIBUCHI, H.; NAKASHIMA, T. Effect of rule weights in fuzzy rule-based classification systems. *IEEE Trans. on Fuzzy Systems*, v. 9, n. 4, p. 506-515, 2001.
- [279] ISHIBUCHI, H.; NAKASHIMA, T.; NII, M. *Classification and Modeling with Linguistic Information Granules: Advanced Approaches to Linguistic Data Mining*. Springer, 2004.
- [280] ISHIBUCHI, H.; YAMAMOTO, T. Rule weight specification in fuzzy rule-based classification systems. *IEEE Trans. on Fuzzy Systems*, v. 13, n. 4, p. 428-435, 2005.
- [281] ISOLA, P.; ZHU, J.-Y.; ZHOU, T. EFROS, A. A. Image-to-Image Translation with Conditional Adversarial Networks. arXiv:1611.07004. 2016.
- [282] JACOBS, R. A. Increased rates of convergence through learning rate adaptation. *Neural Networks*, v. 1, n. 4, p. 295-308, 1988.
- [283] JAIN, L. C.; VEMURI, V. R. (Eds.). *Industrial Applications of Neural Networks*. CRC Press, 1998.
- [284] JAIN, A. K.; MURTY, M. N.; FLYNN, P. J. Data clustering: a review. *ACM Computing Surveys*, v. 31, n. 3, p. 264-323, 1999.
- [285] JAIN, A. K. Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, v. 31, n. 8, p. 651-666, 2010.
- [286] JANG, J.-S. R. Functional equivalence between radial basis function networks and fuzzy inference systems. *IEEE Trans. on Neural Networks*, v. 4, n. 1, p. 156-159, 1993.

- [287] JANG, J.-S. R. ANFIS: adaptive-network-based fuzzy inference system. *IEEE Trans. on Systems, Man and Cybernetics*, v. 23, n. 3, p. 665-685, 1993.
- [288] JANG, J.-S. R.; SUN, C.-T.; MIZUTANI, E. *Neuro-Fuzzy and Soft Computing: A Computational Approach to Learning and Machine Intelligence*. Prentice Hall, 1997.
- [289] JIANG, L.; WANG, D.; CAI, Z.; YAN, X. Survey of Improving Naive Bayes for Classification. In: ALHAJJ, R.; GAO, H.; LI, J.; LI, X.; ZAIANE, O. R. (Eds.). *Advanced Data Mining and Applications*. 2007. p. 134-145.
- [290] JONES, N. C.; PEVZNER, P. A. *An Introduction to Bioinformatics Algorithms*. The MIT Press, 2004.
- [291] JORDAN, M. I. *Serial order: A parallel distributed processing approach*. TR 8604, University of California, Institute for Cognitive Science, 1986.
- [292] JOZEFOWICZ, R., ZAREMBA, W.; SUTSKEVER, I. An empirical exploration of recurrent network architectures. In: 32ND INTERNATIONAL CONFERENCE ON INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML'2015). *Proceedings*. 2015. P. 2342-2350.
- [293] KALMAN, R. E. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, v. 82, n. 1, p. 35-45, 1960.
- [294] KAO, A.; POTEET, S. R. (Eds.). *Natural Language Processing and Text Mining*. Springer, 2007.
- [295] KARGUPTA, H.; HAN, J.; YU, P. S.; MOTWANI, R.; KUMAR, V. (Eds.). *Next Generation of Data Mining*. Chapman & Hall; CRC, 2008.
- [296] KARRAS, T.; AILA, T.; LAINE, S.; LEHTINEN, J. Progressive Growing of GANs for Improved Quality, Stability, and Variation. arXiv:1710.10196. 2017.
- [297] KASZKUREWICZ, E.; BHAYA, A.; EBECKEN, N. F. F. A fault detection and diagnosis module for oil plants in offshore platforms. *Expert Systems with Applications*, v. 12, n. 2, p. 189-194, 1997.
- [298] KATAYAMA, T. *Subspace Methods for System Identification*. Springer, 2010.

- [299] KATZ, S. M. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, v. 35, n. 3, p. 400–401, 1987.
- [300] KAUFMAN, L.; ROUSSEEUW, P. J. *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons, 1990.
- [301] KECMAN, V. *Learning and Soft Computing*. MIT Press, 2001.
- [302] KHAN, S. S.; AHMAD, A. Cluster center initialization algorithm for K-means clustering. *Pattern Recognition Letters*, v. 25, n. 11, p. 1.293-1.302, 2004.
- [303] KHAN, S. S.; MADDEN, M. G. A survey of recent trends in one class classification. In: 20TH IRISH CONFERENCE ON ARTIFICIAL INTELLIGENCE AND COGNITIVE SCIENCE. *Proceedings*. 2009. p. 188-197.
- [304] KHEIR, N. (Ed.). *Systems Modeling and Computer Simulation*. 2. ed. CRC Press, 1995.
- [305] KHOTANZAD, A.; AFKHAMI-ROHANI, R.; ABAYE, A.; DAVIS, M.; MARATUKULAM, D. *IEEE Trans. on Neural Networks*, v. 8, n. 4, p. 835-846, 1997.
- [306] KHOTANZAD, A.; AFKHAMI-ROHANI, R.; MARATUKULAM, D. ANNSTLF – Artificial Neural Network Short-Term Load Forecaster generation three. *IEEE Transactions on Power Systems*, v. 13, n. 4, p. 1.413-1.422, 1998.
- [307] KINGMA, D. P. Fast gradient-based inference with continuous latent variable models in auxiliary form. arxiv:1306.0733. 2013.
- [308] KINGMA, D. P.; WELLING, M. Auto-Encoding Variational Bayes. arXiv:1312.6114. 2013.
- [309] KIRKPATRICK, C. D.; DAHLQUIST, J. R. *Technical Analysis: The Complete Resource for Financial Market Technicians*. FT Press, New Jersey. 2011.
- [310] KLIR, G. J.; YUAN, B. *Fuzzy Sets and Fuzzy Logic: Theory and Applications*. Prentice Hall, 1995.
- [311] KLIR, G. J.; YUAN, B. (Eds.). *Fuzzy sets, fuzzy logic, and fuzzy systems: selected papers by Lotfi A. Zadeh*. World Scientific Publishing, 1996.
- [312] KOHAVI, R.; JOHN, G. Wrappers for feature selection. *Artificial Intelligence*, v. 97, n. 1-2, p. 273-324, 1997.

- [313] KOHONEN, T. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, v. 43, p. 59-69, 1982.
- [314] KOHONEN, T. *Self-Organization and Associative Memory*. Berlim: Springer, 1989.
- [315] KOLESNIKOV, A.; ZHAI, X.; BEYER, L. Revisiting Self-Supervised Visual Representation Learning. arXiv:1901.09005. 2019.
- [316] KOLLER, D.; FRIEDMAN, N. *Probabilistic Graphical Models – Principles and Techniques*. MIT Press, 2009.
- [317] KOLMOGOROV, A. N. On the representation of continuous functions of several variables by superposition of continuous function of one variable and addition. *Proceedings of the USSR Academy of Sciences (DAN SSSR)*, v. 114, p. 369-373, 1957.
- [318] KONAR, A. *Computational Intelligence: Principles, Techniques and Applications*. Springer, 2005.
- [319] KORENIUS, T.; LAURIKKALA, J.; JUHOLA, M. On principal component analysis, cosine and Euclidean measures in information retrieval. *Information Sciences*, v. 177, n. 22, p. 4.893-4.905, 2007.
- [320] KOSKO, B. *Neural Networks and Fuzzy Systems: A Dynamical Systems Approach to Machine Intelligence*. Prentice Hall, 1992.
- [321] KOSKO, B. *Fuzzy Engineering*. Prentice Hall, 1996.
- [322] KRISHNAPURAM, R.; KELLER, J. A possibilistic approach to clustering. *IEEE Trans. Fuzzy Systems*, v. 1, n. 2, p. 98-110, 1993.
- [323] KRISHNAPURAM, R.; KELLER, J. The possibilistic c-Means algorithm: Insights and recommendations. *IEEE Trans. Fuzzy Systems*, v. 4, n. 3, p. 385-393, 1996.
- [324] KRIZHEVSKY A.; SUTSKEVER, I.; HINTON, G. E. ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, v. 25, p. 1097-1105, 2012.
- [325] KUNCHEVA, L. I.; SKURICHINA, M.; DUIN, R. P. W. An experimental study on diversity for bagging and boosting with linear classifiers. *Information Fusion*, v. 3, n. 4, p. 245-258, 2002.

- [326] KULKARNI, S.; HARMAN, G. *An Elementary Introduction to Statistical Learning Theory*. John Wiley & Sons, 2011.
- [327] LANDIS, J. R.; KOCH, G. G. The measurement of observer agreement for categorical data. *Biometrics*, v. 33, n. 1, p. 159-174, 1977.
- [328] LANG, K. J.; WAIBEL, A. H.; HINTON, G. E. A time-delay neural network architecture for isolated word recognition. *Neural Networks*, v. 3, n. 1, p. 23-43, 1990.
- [329] LAURSEN, G. H. N.; THORLUND, J. *Business Analytics for Managers: Taking Business Intelligence Beyond Reporting*. SAS Institute; John Wiley & Sons, 2010.
- [330] LECUN, Y. Une procédure d'apprentissage pour réseau a seuil asymmetrique, *Proceedings of Cognitiva*, v. 85, p. 599-604, 1985.
- [331] LECUN, Y.; BOSER, B.; DENKER, J. S.; HENDERSON, D.; HOWARD, R. E.; HUBBARD, W.; JACKEL, L. D. Handwritten digit recognition with a back-propagation network. In: *Advances in Neural Information Processing Systems (NIPS 1990)*. Proceedings. 1990. p. 396-404.
- [332] LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2.278-2.324, 1998.
- [333] LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, p. 436-444, 2015.
- [334] LEDIG, C. et al. Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network. arXiv:1609.04802. 2016
- [335] LEE, C. C. Fuzzy logic in control systems: fuzzy logic controller I. *IEEE Trans. on Systems, Man and Cybernetics*, v. 20, n. 2, p. 404-418, 1990.
- [336] LEE, C. C. Fuzzy logic in control systems: fuzzy logic controller II. *IEEE Trans. on Systems, Man and Cybernetics*, v. 20, n. 2, p. 419-435, 1990.
- [337] LEE, C. W.; SHIN, Y. C. Construction of fuzzy systems using least-squares method and genetic algorithm. *Fuzzy Sets and Systems*, v. 137, n. 3, p. 297-323, 2003.
- [338] LEE, Q. V.; RANZATO, M. A.; MONGA, R.; DEVIN, M.; CHEN, K.; CORRADO, G.; DEAN, J.; NG, A. Building high-level features using large scale unsupervised

- learning. In: INTERNATIONAL CONFERENCE IN MACHINE LEARNING (ICML 2012). *Proceedings*. 2012. p. 81-88. arXiv: 1112.6209.
- [339] LENGELLÉ, R.; DENOEU, T. Training MLPs layer by layer using an objective function for internal representations. *Neural Networks*, v. 9, p. 83-97, 1996.
- [340] LESKOVEC, J.; RAJARAMAN, A.; ULLMAN, J. D. *Mining of Massive Datasets*. Cambridge University Press, 2014.
- [341] LEVENBERG, K. A method for the solution of certain nonlinear problems in least squares. *The Quarterly of Applied Mathematics*, v. 2, p. 164-168, 1944.
- [342] LI, W.; NG, W.-K.; LIU, Y.; ONG, K.-L. Enhancing the effectiveness of clustering with spectra analysis. *IEEE Trans. on Knowledge and Data Engineering*, v. 19, n. 7, p. 887-902, 2007.
- [343] LIMA, F. R. N. *Aplicação de Lógica Fuzzy para Localização de Aterros Sanitários*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2005.
- [344] LIN T.-Y. et al. Microsoft COCO: Common Objects in Context. arXiv:1405.0312, 2015.
- [345] LINDEN, R. *Algoritmos Genéticos – Uma Importante Ferramenta da Inteligência Computacional*. 2. ed. Brasport, 2008.
- [346] LIPPMANN, R. P. Review of Neural Networks for Speech Recognition. *Neural Computation*, v. 1, n. 1, 1-38, 1989.
- [347] LISBOA, P. J. G.; EDISBURY, B.; VELLIDO, A. (Eds.). *Business Applications of Neural Networks: The State-Of-The-Art of Real-World Applications*. World Scientific Pub Co, 2000.
- [348] LIU, B. *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*. Springer, 2007.
- [349] LIU, G. R.; GU, Y. T. *An Introduction to Meshfree Methods and Their Programming*. Springer, 2005.
- [350] LIU, H.; MOTODA, H. (Eds.). *Computational Methods of Feature Selection*. Chapman and Hall; CRC, 2007.

- [351] LIU, T.; LIU, S.; CHEN, Z.; MA, W.-Y. An evaluation on feature selection for text clustering. In: 20TH INTERNATIONAL CONFERENCE ON MACHINE LEARNING ICML2003. *Proceedings*. 2003. p. 488-495.
- [352] LJUNG, L. *System Identification: Theory for the User*. 2. ed. Prentice Hall, 1999.
- [353] LLOYD, S. P. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, v. 28, n. 2, p. 129-137, 1982.
- [354] LUGHOFFER, E. D. FLEXFIS: A robust incremental learning approach for evolving Takagi-Sugeno fuzzy models. *IEEE Trans. on Fuzzy Systems*, v. 16, n. 6, p. 1.393-1.410, 2008.
- [355] LUKOŠEVIČIUS, M.; JAEGER, H. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, v. 3, n.3, p. 127–149, 2009.
- [356] VON LUXBURG, U. A tutorial on spectral clustering. *Statistics and Computing*, v. 17, n. 4, p. 395-416, 2007.
- [357] VAN DER MAATEN, L. J. P.; POSTMA, E. O.; VAN DEN HERIK, H. J. Dimensionality Reduction: A Comparative Review. *Tilburg University Technical Report*, TiCC-TR 2009-005, 2009.
- [358] MACCUISH, J. D.; MACCUISH, N. E. *Clustering in Bioinformatics and Drug Discovery*. 3. ed. CRC Press, 2010.
- [359] MACQUEEN, J. B. Some Methods for classification and Analysis of Multivariate Observations. In: 5TH BERKELEY SYMPOSIUM ON MATHEMATICAL STATISTICS AND PROBABILITY. *Proceedings*. University of California Press, 1967. p. 281-297.
- [360] MAHAJAN, M.; NIMBHORKAR, P.; VARADARAJAN, K. The Planar k-Means Problem is NP-Hard. *Lecture Notes in Computer Science*, v. 5.431, p. 274-285, 2009.
- [361] MAHER M. J., Modelling Association Football scores. *Statistica Neerlandica*, v. 36, n. 3, p. 109-118, 1982.
- [362] MALKIEL, B. G. The Efficient Market Hypothesis and Its Critics. *The Journal of Economic Perspectives*. v. 17, n. 1, p. 59-82, 2003.

- [363] MAMDANI, E. H.; ASSILIAN, S. An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, v. 7, n. 1, p. 1-13, 1975.
- [364] MANNING, C. D.; RAGHAVAN, P.; SCHUTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [365] MARCHETTE, D. J.; SOLKA, J. L. Using data images for outlier detection. *Computational Statistics & Data Analysis*, v. 43, n. 4, p. 541-552, 2003.
- [366] MARDIA, K. V.; KENT, J. T.; BIBBY, J. M. *Multivariate Analysis*. Academic Press, 1980.
- [367] MARGOSSIAN, C. C. A Review of automatic differentiation and its efficient implementation. arXiv:1811.05031, 2018.
- [368] MARKOV, Z.; LAROSE, D. T. *Data Mining the Web: Uncovering Patterns in Web Content, Structure, and Usage*. John Wiley & Sons, 2007.
- [369] MARQUARDT, D. An algorithm for least-squares estimation of nonlinear parameters. *SIAM Journal on Applied Mathematics*, v. 11, n. 2, p. 431-441, 1963.
- [370] MARSLAND, S. *Machine Learning: An Algorithmic Perspective*. Chapman and Hall; CRC, 2009.
- [371] MASCI, J.; MEIER, U.; CIRE, D.; SCHMIDHUBER, J. Stacked convolutional autoencoders for hierarchical feature extraction. In: 21TH INTERNATIONAL CONFERENCE ON ARTIFICIAL NEURAL NETWORKS (ICANN 2011). *Proceedings*. 2011. p. 52-59.
- [372] MATSUNAGA, L. A. *Uma Metodologia de Categorização Automática de Textos Para a Distribuição dos Projetos de Lei às Comissões Permanentes da Câmara Legislativa do Distrito Federal*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2007.
- [373] MAURIS, G.; FOULLOY, L. A fuzzy symbolic approach to formalize sensory measurements: an application to a comfort sensor. *IEEE Trans. on Instrumentation and Measurement*, v. 51, n. 4, p. 712-715, 2002.
- [374] MCCULLOCH, W.; PITTS, W. A logical calculus of ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, v. 5, p. 115-133, 1943.

- [375] MCDONALD, G. C.; SCHWING, R. C. Instabilities of regression estimates relating air pollution to mortality. *Technometrics*, v. 15, p. 463-482, 1973.
- [376] MEMON, N.; XU, J. J.; HICKS, D. L.; CHEN, H. (Eds.). *Data Mining for Social Network Data*. Springer, 2010.
- [377] MERCER, J. Functions of positive and negative type and their connection with the theory of integral equations. *Philosophical Transactions of the Royal Society*, v. 209, p. 415-446, 1909.
- [378] MEYERSON, A.; O'CALLAGHAN, L.; PLOTKIN, S. A k-median algorithm with running time independent of data size. *Machine Learning*, v. 56, p. 61-87, 2004.
- [379] MICCHELLI, C. A. Interpolation of scattered data: distance matrices and conditionally positive definite functions. *Constructive Approximation*, v. 2, p. 11-22, 1986.
- [380] MICHALEWICZ, Z. *Genetic Algorithms + Data Structures = Evolution Programs*. 3. ed. Springer, 1996.
- [381] VAN MIEGHEM, P. *Graph Spectra for Complex Networks*. Cambridge University Press, 2011.
- [382] MIKOLOV, T.; CHEN, K.; CORRADO, G.; DEAN, J. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781, 2013,
- [383] MIKOLOV, T.; SUTSKEVER, I; CHEN, K.; CORRADO, G.; DEAN, J. Distributed Representations of Words and Phrases and their Compositionality, arXiv:1310.4546
- [384] MIKUT, R.; JÄKEL, J.; GRÖLL, L. Interpretability issues in data-based learning of fuzzy systems. *Fuzzy Sets and Systems*, v. 150, p. 179-197, 2005.
- [385] MILLER, G. A. WordNet: A Lexical Database for English. *Communications of the ACM*, v. 38, n. 11, p. 39-41, 1995.
- [386] MILLER, H. J.; HAN, J. (Eds.). *Geographic Data Mining and Knowledge Discovery*. Chapman & Hall; CRC, 2001.
- [387] MILLER, H. J.; HAN, J. (Eds.). *Geographic Data Mining and Knowledge Discovery*. 2. ed. Chapman & Hall; CRC, 2009.

- [388] MILLIGAN, G.; COOPER, M. An examination of procedures for determining the number of clusters in a data set. *Psychometrika*, v. 50, n. 2, p. 159-179, 1985.
- [389] MINSKY, M.; PAPERT, S. *Perceptrons: An Introduction to Computational Geometry*. 2. ed. The MIT Press, 1972.
- [390] MIRANDA, F. P.; MACDONALD, J. A.; CARR, J. R. Application of the semivariogram textural classifier (STC) for vegetation discrimination using SIR-B data of Borneo. *International Journal of Remote Sensing*, v. 13, n. 12, p. 2.349-2.354, 1992.
- [391] MIRANDA, F. P.; FONSECA, L. E. N.; CARR, J. R. Semivariogram textural classification of JERS-1 (Fuyo-1) SAR data obtained over a flooded area of the Amazon rainforest. *International Journal of Remote Sensing*, v. 19, n. 13, p. 549-556, 1998.
- [392] MITCHELL, T. *Machine Learning*. McGraw Hill, 1997.
- [393] MITSA, T. *Temporal Data Mining*. Chapman & Hall; CRC, 2010.
- [394] MNH, A.; KAVUKCUOGLU, K. Learning word embeddings efficiently with noise-contrastive estimation. 27th CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS (NIPS 2013), 2013.
- [395] MODENESI, M. V.; COSTA, M. C. A.; EVSUKOFF, A. G.; EBECKEN, N. F. F. Parallel fuzzy c-means cluster analysis. *Proceedings of the High Performance Computing for Computational Science – VECPAR 2006*, LNCS, v. 4.395, p. 52-65, 2007.
- [396] MODENESI, M. V.; EVSUKOFF, A. G.; COSTA, M. C. A. A load balancing knapsack algorithm for parallel fuzzy c-means cluster Analysis. *Proceedings of the High Performance Computing for Computational Science – VECPAR 2008*, LNCS, v. 5.336, p. 269-279, 2008.
- [397] MODENESI, M. V. *Análise de Agrupamentos FCM Utilizando Processamento Paralelo*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2008.
- [398] MONTAVON, G.; ORR, G.; MÜLLER, K.-R. (Eds.). *Neural Networks: Tricks of the Trade. Lecture Notes in Computer Science*, Springer, v. 7.700, 2012.

- [399] MONTGOMERY, D. C.; PECK, E. A.; VINING, G. G. *Introduction to Linear Regression Analysis*. 4. ed. Wiley-Interscience, 2006.
- [400] MONTGOMERY, D. C. *Estatística Aplicada e Probabilidade para Engenheiros*. 4. ed. LTC, 2009.
- [401] MOODY, J.; DARKEN, C. J. Fast learning in networks of locally-tuned processing units. *Journal of Neural Computation*, v. 1, n. 2, p. 281-294, 1989.
- [402] DE MOOR, B.; VAN OVERSCHEE, P. *Subspace Identification for Linear Systems: Theory, Implementation Applications*. Springer, 1996.
- [403] MORIN, F.; BENGIO, Y. Hierarchical Probabilistic Neural Network Language Model. 10th INTERNATIONAL WORKSHOP ON ARTIFICIAL INTELLIGENCE AND STATISTICS. *Proceedings*. 2005, p. 246-252.
- [404] MORETTIN, P. A.; TOLOI, C. M. *Análise de Séries Temporais*. Edgard Blucher, 2006.
- [405] MURPHY, K. P. *Machine Learning, A Probabilistic Perspective*. MIT Press, 2012.
- [406] MYERS, R. H.; MONTGOMERY, D. C.; VINING, G. G. *Generalized Linear Models, with Applications in Engineering and the Sciences*. 2. ed. John Wiley & Sons, 2010.
- [407] NAKOULA, Y.; GALICHET, S.; FOULLOY, L. Simultaneous learning of rules and linguistic terms. In: 5TH INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. Nova Orleans, 1996. p. 1.743-1.749.
- [408] NASCIMENTO, M. C. V.; CARVALHO, A. C. P. L. F. de. Spectral methods for graph clustering – a survey. *European Journal of Operational Research*, v. 211, n. 2, p. 221-231, 2011.
- [409] NASSIRTOUSSI, A. K.; AGHABOZORGI, S.; WAH T. Y.; NGO, D. C. L. Text mining for market prediction: A systematic review. *Expert Systems with Applications*, v. 41, n. 16, p. 7653-7670, 2014.
- [410] NEUBIG, G. Neural Machine Translation and Sequence-to-sequence Models: A Tutorial. arXiv:1703.01619, 2017.
- [411] NEWMAN, M. E. J. Mixing patterns in networks. *Physical Review E*, v. 67, n. 2, 026126, 2003.

- [412] NEWMAN, M. E. J. Detecting community structure in networks. *The European Physical Journal B*, v. 38, n. 2, p. 321-330, 2004.
- [413] NEWMAN, M. E. J.; GIRVAN, M. Finding and evaluating community structure in networks. *Physical Review E*, v. 69, n. 2, 026113, 2004.
- [414] NEWMAN, M. E. J. *Networks: An Introduction*. Oxford University Press, 2010.
- [415] NG, A. Y.; JORDAN, M. I.; WEISS, Y. On spectral clustering: analysis and an algorithm. In: DIETTERICH, T. G.; BECKER, S.; GHAHRAMANI, Z. (Eds.). *Advances in Neural Information Processing Systems*. v. 14. MIT Press, 2002.
- [416] NG, R.; HAN, J. Efficient and effective clustering method for spatial data mining. In: INTERNATIONAL CONFERENCE ON VERY LARGE DATABASES. *Proceedings*. Santiago, Chile, 12-15 set. 1994. p. 144-155.
- [417] NIXON, M. *Feature Extraction and Image Processing for Computer Vision*. 3. ed. Academic Press, 2012.
- [418] NOTAY, Y. Flexible conjugate gradients. *SIAM Journal on Scientific Computing*, v. 22, n. 4, p. 1.444-1.460, 2000.
- [419] ODEN, T.; MOSER, R.; GHATTAS, O. *Computer Predictions with Quantified Uncertainty*. Texas: The Institute for Computational Engineering and Sciences, 2010. (ICES Report 10-39).
- [420] OLIVEIRA, J. V. de; PEDRYCZ, W. (Eds.). *Advances in Fuzzy Clustering and its Applications*. John Wiley & Sons, 2007.
- [421] OSTROVSKY, R.; RABANI, Y.; SCHULMAN, L. J.; SWAMY, C. The effectiveness of Lloyd-type methods for the k-means problem. In: 47TH ANNUAL IEEE SYMPOSIUM ON FOUNDATIONS OF COMPUTER SCIENCE (FOCS'06). *Proceedings*. 2006. p. 165-174.
- [422] OWCZARCZUK, M. Churn models for prepaid customers in the cellular telecommunication industry using large data marts. *Expert Systems with Applications*, v. 37, n. 6, p. 4.710-4.712, 2010.
- [423] PAKHIRA, M. K.; BANDYOPADHYAY, S.; MAULIK, U. Validity index for crisp and fuzzy clusters. *Pattern Recognition*, v. 37, n. 3, p. 487-501, 2004.

- [424] PALASKAR, S.; SANABRIA, R.; METZE F. End-to-End Multimodal Speech Recognition. arXiv:1804.09713, 2018.
- [425] PALIWAL, M.; KUMAR, U. A. Neural networks and statistical techniques: A review of applications. *Expert Systems with Applications*, v. 36, n. 1, p. 2-17, 2009.
- [426] PAN, S. J.; YANG, Q. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, v. 22, n. 10, p1345-1359, 2010.
- [427] PANG, B.; LEE, L. Opinion Mining and Sentiment Analysis. *Foundations and Trends in Information Retrieval*, v. 2 n. 1-2, p. 1-135, 2008.
- [428] PAPINENI, K.; ROUKOS, S.; WARD, T.; ZHU, W.-J. Bleu: a Method for Automatic Evaluation of Machine Translation. In: 40TH ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS (ACL 2002). *Proceedings*. 2002, pp. 311-318.
- [429] PASCANU, R.; MIKOLOV, T.; BENGIO, Y. On the difficulty of training Recurrent Neural Networks. arXiv:1211.5063, 2012.
- [430] PASCANU, R.; GULCEHRE, C.; CHO, K.; BENGIO, Y. How to Construct Deep Recurrent Neural Networks. arXiv:1312.6026, 2013.
- [431] PATON, R. J.; UPPAL, F. J.; LOPEZ-TORIBIO, C. J. Soft computing approaches to fault diagnosis for dynamic systems: a survey. *Control and Intelligent Systems*, v. 7, n. 2-3, p. 198-211, 2000.
- [432] PEDRYCZ, W.; GOMIDE, F. *An Introduction to Fuzzy Sets: Analysis and Design*. MIT Press, 1998.
- [433] PEDRYCZ, W.; GOMIDE, F. *Fuzzy Systems Engineering: Toward Human-Centric Computing*. IEEE Press; John Wiley & Sons, 2007.
- [434] PELLEGRINO, D.; MOORE, A. X-means: extending k-means with efficient estimation of the number of clusters. In: SEVENTEENTH INTERNATIONAL CONFERENCE ON MACHINE LEARNING. *Proceedings*. São Francisco, 2000. p. 727-734.
- [435] PENNINGTON, J.; SOCHER, R.; MANNING, C. D. GloVe: Global Vectors for Word Representation. In: EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING (EMNLP 2014). *Proceedings*. 2014

- [436] PEREIRA, G. C.; EVSUKOFF, A. G.; COUTINHO, R.; EBECKEN, N. F. F. Coastal environmental management by fuzzy system modeling. In: 2004 INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. Budapeste, 2004. p. 215-220.
- [437] PINHEIRO, M. S. *Uma Abordagem Usando Sintagmas Nominais como Descritores no Processo de Mineração de Opiniões*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2009.
- [438] PINHEIRO, C. A. R. *Redes Neurais para Prevenção de Inadimplência em Operadoras de Telefonia*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2005.
- [439] PINHEIRO, C. A.; EVSUKOFF, A. G.; EBECKEN, N. F. F. Revenue recovering with insolvency prevention on a Brazilian telecom operator. *SIGKDD Explorations*, v. 8, n. 1, p. 65-70, 2006.
- [440] POGGIO, T.; GIROSI, F. Networks for approximation and learning. *Proceedings of the IEEE*, v. 78, n. 9, 1990.
- [441] POLLOCK, D. S. G. *Handbook of Time Series Analysis, Signal Processing, and Dynamics*. Academic Press, 1999.
- [442] POPESCU-BELIS, A. Context in Neural Machine Translation: A Review of Models and Evaluations. arXiv:1901.09115, 2019.
- [443] PRATAP, V.; HANNUN, A.; XU, Q.; CAI, J.; KAHN, J.; SYNNAEVE, G.; LIPTCHINSKY, V.; COLLOBERT, R. wav2letter++: The Fastest Open-source Speech Recognition System. arXiv:1812.07625. 2018.
- [444] PRENTER, P. M. *Splines and Variational Methods*. Dover Publications, 2008.
- [445] QUINLAN, J. R. Induction of Decision Trees. *Machine Learning*, v. 1, p. 81-106, 1986.
- [446] QUINLAN, J. R. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1992.
- [447] RAJPURKAR, P.; ZHANG, J.; LOPYREV, K.; LIANG P. SQuAD: 100,000+ Questions for Machine Comprehension of Text. arXiv:1606.05250, 2016.

- [448] RAWAT, W.; WANG, Z. Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review. *Neural Computation*, v. 29, n. 9, p.2352-2449, 2017.
- [449] RAYKOV, T.; MARCOULIDES, G. A. *An Introduction to Applied Multivariate Analysis*. Routledge Academic, 2008.
- [450] REZAEI, B. A cluster validity index for fuzzy clustering. *Fuzzy Sets and Systems*, v. 161, n. 23, p. 3.014-3.025, 2010.
- [451] REZENDE, S. O. (Ed.). *Sistemas Inteligentes: Fundamentos e Aplicações*. Manole, 2002.
- [452] REZENDE, D. J.; MOHAMED, S.; WIERSTRA, D. Stochastic backpropagation and approximate inference in deep generative models. arXiv:1401.4082.
- [453] RIPLEY, B. D. *Pattern Recognition and Neural Networks*. Cambridge University Press, 2008.
- [454] RICE, J. R. *The Approximation of Functions*. Addison-Wesley, 1964.
- [455] RIEDMILLER M.; BRAUN H. A direct adaptive method for faster backpropagation learning: the RPROP algorithm. In IEEE INTERNATIONAL CONFERENCE ON NEURAL NETWORKS. *Proceedings*. São Francisco, 1993, p. 586-591.
- [456] RIFAI, S.; VINCENT, P.; MULLER, X.; GLOROT, X.; BENGIO, Y. Contractive Auto-Encoders: Explicit Invariance During Feature Extraction. In: 28TH INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML 2011). *Proceedings*. 2011. p. 833-840
- [457] RONDEAU, L.; RUELAS, R.; LEVRAT, L.; LAMOTTE, M. A defuzzification method respecting the fuzzification. *Fuzzy Sets Systems*, v. 86, p. 311-320, 1997.
- [458] RONNEBERGER, O.; FISCHER, P.; BROX, T. U-Net: Convolutional Networks for Biomedical Image Segmentation. arXiv:1505.04597. 2015.
- [459] ROSASCO, L.; De VITO, E. D.; CAPONNETTO, A.; PIANA, M.; VERRI, A. Are Loss Functions All the Same? *Neural Computation*, v. 16 n. 5, p. 1063–1076, 2004.
- [460] ROSENBLATT, F. *The Perceptron, A Perceiving and Recognizing Automaton*. Cornell Aeronautical Laboratory (CAL), 1957. (Project Report N. 85-460-1).
- [461] ROSS, T. J. *Fuzzy Logic with Engineering Applications*. 3. ed. Wiley, 2010.

- [462] ROUSSEEUW, P. J. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, v. 20, n. 1, p. 53-65, 1987.
- [463] RUDER, S. An overview of gradient descent optimization algorithms. arXiv:1609.04747, 2016.
- [464] RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, p. 533-536, 1986.
- [465] RUSSAKOVSKY, O; DENG, J; SU, H. KRAUSE, J; SATHEESH, S; MA, S; HUANG, Z; KARPATY, A; KHOSLA, A. BERNSTEIN, M; BERG, A. C.; FEI-FEI, L. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, v. 115, n. 3, p. 211-252, 2015.
- [466] RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 3. ed. Prentice Hall, 2009.
- [467] SALTON, G.; WONG, A.; YANG, C. S. A vector space model for automatic indexing. *Communications of ACM*, v. 18, n. 11, p. 613-620, 1975.
- [468] SANTOS, C. K. dos. *Análise de Agrupamento em Coleções de Documentos: Uma Abordagem Baseada na Detecção Espectral de Comunidades em Redes Complexas*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2009.
- [469] SANTOS, C. K. dos; EVSUKOFF, A. G.; EBECKEN, N. F. F. Potential collaboration discovery using document clustering and community structure detection. In: ACM INTERNATIONAL WORKSHOP ON COMPLEX NETWORKS MEET INFORMATION AND KNOWLEDGE MANAGEMENT (CNIKM '09). *Proceedings*. 2009. p. 39-46.
- [470] SANTOS, C. K. dos. *Identificação de Modelos Baseados em Sistemas Fuzzy Recorrentes com Duplo Feedback Usando Evolução Diferencial*. Tese (doutorado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2013.
- [471] SANTOS, C. K. dos; ESPINDOLA, R.; EVSUKOFF, A. G. Development of recurrent fuzzy systems with external feedback. *Revista IEEE América Latina*, v. 13, 2015.
- [472] SCHABENBERGER, O.; GOTWAY, C. A. *Statistical Methods for Spatial Data Analysis*. Chapman & Hall; CRC, 2004.

- [473] SCHAEFFER, S. E. Graph clustering. *Computer Science Review*, v. 1, n. 1, p. 27-64, 2007.
- [474] SHAMAN, P.; STINE, R. A. The bias of autoregressive coefficient estimators. *Journal of American Statistic Society*, v. 83, p. 842-848, 1988.
- [475] SCHAPIRE, R. E.; FREUND, Y. *Boosting: Foundations and Algorithms*. The MIT Press, 2012.
- [476] SCHÖLKOPF, B.; SMOLA, A.; MÜLLER, K.-R. Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, v. 10, n. 5, p. 1.299-1.319, 1998.
- [477] SCHÖLKOPF, B.; BURGESS, C. J. C.; SMOLA, A. J. (Eds.). *Advances in Kernel Methods: Support Vector Learning*. MIT Press, 1998.
- [478] SCHÖLKOPF, B.; SMOLA, A. J. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2001.
- [479] SCHÖLKOPF, B.; TSUDA, K.; VERT, J.-P. (Eds.). *Kernel Methods in Computational Biology*. MIT Press, 2004.
- [480] SCHUMAKER, R. P.; CHEN, H. A quantitative stock prediction system based on financial news, *Information Processing and Management*, v. 45, n. 5, p. 571-583, 2009.
- [481] SCHWARZ, G. E. Estimating the dimension of a model. *Annals of Statistics*, v. 6, n. 2, p. 461-464, 1979.
- [482] SCOTT, D. W. *Multivariate Density Estimation*. Theory, Practice and Visualization. Nova York: Wiley, 1992.
- [483] SHANKER, S. G. *Wittgenstein's Remarks on the Foundations of AI*. Routledge, 1998.
- [484] SHARMA, A.; BHURIYA, D.; SINGH, U. Survey of stock market prediction using machine learning approach. In: INTERNATIONAL CONFERENCE OF ELECTRONICS, COMMUNICATION AND AEROSPACE TECHNOLOGY (ICECA 2017). *Proceedings*. 2017.
- [485] SHAWE-TAYLOR, J.; CRISTIANINI, N. *Kernel Methods for Pattern Analysis*. Cambridge University Press, 2004.

- [486] SHEELA, K. G.; DEEPA, S. N. Review on Methods to Fix Number of Hidden Neurons in Neural Networks. *Mathematical Problems in Engineering*, Article ID 425740, 2013.
- [487] SHEPHERD, A. J. *Second-Order Methods for Neural Networks*. Springer, 1997.
- [488] SHI, J.; MALIK, J. Normalized cuts and image segmentation. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION. *Proceedings*. 1997. p. 731-737.
- [489] SHI, J.; MALIK, J. Normalized cuts and image segmentation. *Transactions on Pattern Analysis and Machine Intelligence*, v. 22, n. 8, p. 888-905, 2000.
- [490] SILVERMAN, B. W. *Density Estimation*. Londres: Chapman and Hall, 1986.
- [491] SIMOFF, S.; BÖHLEN, M. H.; MAZEIKA, A. (Eds.). *Visual Data Mining: Theory, Techniques and Tools for Visual Analytics*. Springer, 2008.
- [492] SIMONYAN, K.; ZISSERMAN, A. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556, 2014.
- [493] SKILLICORN, D. *Understanding Complex Datasets: Data Mining with Matrix Decompositions*. Chapman and Hall; CRC, 2007.
- [494] SMOLA, A. J.; SCHÖLKOPF, B. A tutorial on support vector regression. *Journal of Statistics and Computing*, v. 14, n. 3, p. 199-222, 2004.
- [495] SNEATH, P. H. A. The application of computers to taxonomy. *Journal of General Microbiology*, v. 17, n. 1, p. 201-226, 1957.
- [496] SNYMAN, J. A. *Practical Mathematical Optimization: An Introduction to Basic Optimization Theory and Classical and New Gradient-Based Algorithms*. Springer Publishing, 2005.
- [497] SOCHER, R.; PERELYGIN, A.; WU, J. Y.; CHUANG, J. Y.; MANNING, C. D.; NG, A. Y.; POTTS C. Recursive deep models for semantic compositionality over a sentiment treebank. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING (EMNLP 2013). *Proceedings*. 2013.
- [498] SONG, Q.; SHEPPERD, M. Missing data imputation techniques. *International Journal of Business Intelligence and Data Mining*, v. 2, n. 3 p. 261-291, 2007.

- [499] SOKAL, R. R.; MICHENER, C. D. A Statistical Method for Evaluating Systematic Relationships. *The University of Kansas Scientific Bulletin*, n. 38, p. 1.409-1.438, 1958.
- [500] SOUZA, M. P. de *Utilização de Lógica Fuzzy no Apoio a Decisão Multicritério*. Dissertação (mestrado). Rio de Janeiro: Coppe/UFRJ, Programa de Engenharia Civil, 2006.
- [501] SPRENT, P.; SMEETON, N. C. *Applied Nonparametric Statistical Methods*. 4. ed. Chapman and Hall; CRC, 2007.
- [502] SRIVASTAVA, A.; SAHAMI, M. (Eds.). *Text Mining: Classification, Clustering, and Applications*. Chapman & Hall; CRC, 2009.
- [503] SRIVASTAVA, N.; HINTON, G.; KRIZHEVSKY, A.; SUTSKEVER, I.; SALAKHUTDINOV, R. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, v. 15, p. 1929-1958, 2014.
- [504] STEINWART, I.; CHRISTMANN, A. *Support Vector Machines*. Nova York: Springer, 2008.
- [505] STEFFENS, K.-G. *The History of Approximation Theory from Euler to Bernstein*. Springer, 2006.
- [506] STONE, J. V. *Information Theory: A Tutorial Introduction*. Sebtel Press, 2015.
- [507] STORK, D. G.; YOM-TOV, E. *Computer Manual in MATLAB to Accompany Pattern Classification*. 2. ed. Wiley-Interscience, 2004.
- [508] SU, T.; DY, J. A Deterministic Method for Initializing K-means Clustering. In: 16TH IEEE INTERNATIONAL CONFERENCE ON TOOLS WITH ARTIFICIAL INTELLIGENCE (ICTAI 2004). *Proceeding*. 2004. p. 784-786.
- [509] SUSSKIND, J.; ANDERSON, A.; Hinton, G. E. The Toronto face dataset. Technical Report UTML TR 2010-001, U. Toronto. 2010.
- [510] SUTSKEVER, I.; VINYALS, O.; LE, Q. V. Sequence to sequence learning with neural networks. arXiv:1409.3215, 2014
- [511] SUTTON, R. S. Learning to predict by the method of temporal differences. *Machine Learning*, v. 3, p. 9-44, 1988.

- [512] SUTTON, R. S.; BARTO, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [513] SZEGEDY, C. et al. Going Deeper with Convolutions. arXiv:1409.4842, 2014.
- [514] SZEGEDY, C.; VANHOUCKE, V.; IOFFE, S.; SHLENS, J.; WOJNA, Z. Rethinking the Inception Architecture for Computer Vision. arXiv:1512.00567, 2015.
- [515] TAKAGI, T.; SUGENO, M. Fuzzy identification of systems and its application to modeling and control. *IEEE Trans. Systems Man Cybernet*, v. 15, n. 1, p. 116-132, 1985.
- [516] TAN, P.-N.; STEINBACH, M.; KUMAR, V. *Introduction Data Mining*. Morgan Kaufmann Publishers, 2006.
- [517] TANAKA, G. et al. Recent Advances in Physical Reservoir Computing: A Review. arXiv:1808.04962, 2018.
- [518] TAYLOR, J. G. (Ed.). *Neural Networks and Their Applications*. Wiley, 1996.
- [519] TAYLOR, J. S.; CRISTIANINI, N. *Kernel Methods for Pattern Analysis*. Cambridge University Press, 2004.
- [520] TETTAMANZI, A.; TOMASSINI, M. *Soft Computing: Integrating Evolutionary, Neural, and Fuzzy Systems*. Springer, 2001.
- [521] THEODORIDIS, S.; KOUTROUMBAS, K. *Pattern Recognition*. 3. ed. Academic Press, 2006.
- [522] THEODORIDIS, S.; PIKRAKIS, A.; KOUTROUMBAS, K.; CAVOURAS, D. *Introduction to Pattern Recognition: A Matlab Approach*. Academic Press, 2010.
- [523] TIBSHIRANI, R. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society. Series B (Methodological)*, v. 58, n. 1, p.267-288, 1996.
- [524] TIKHONOV, N. On solving ill-posed problem and method of regularization. *Doklady Akademii Nauk USSR*, v. 153, p. 501-504, 1963.
- [525] TITTERINGTON, D. M.; SEDRANSK, J. Imputation of missing values using density estimation. *Statistics & Probability Letters*, v. 8, n. 5, p. 411-418, 1989.
- [526] TOOLE, J. L.; COLAK, S.; STURT, B.; ALEXANDER, L. P.; EVSUKOFF, A.; GONZÁLEZ, M. C. The path most traveled: Travel demand estimation using big data

resources. *Transportation Research Part C: Emerging Technologies*, v. 58, p. 162-177, 2015.

- [527] TORRALBA, A.; FERGUS, R.; FREEMAN, W. T. 80 Million Tiny Images: A Large Data Set for Nonparametric Object and Scene Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 30, n. 11, p. 1958-1970, 2008.
- [528] TRAN, T. N.; WEHRENS, R.; BUYDENS, L. M. C. Clustering multispectral images: a tutorial. *Chemometrics and Intelligent Laboratory Systems*, v. 77, p. 3-17, 2005.
- [529] TURING, A. M. Computing machinery and intelligence. *Mind*, v. 59, p. 433-460, 1950.
- [530] TSIPTISIS, K.; CHORIANOPOULOS, A. *Data Mining Techniques in CRM: Inside Customer Segmentation*. Wiley, 2009.
- [531] VAPNIK, V.; CHERVONENKIS, A. A note on one class of perceptrons. *Automation and Remote Control*, v. 25, 1964.
- [532] VAPNIK, V. N. *Statistical Learning Theory*. Wiley-Interscience, 1998.
- [533] VAPNIK, V. *The Nature of Statistical Learning Theory*. 2. ed. Springer, 1999.
- [534] VAPNIK, V. *Estimation of Dependences Based on Empirical Data*. 2. ed. Springer, 2006.
- [535] VARGAS M. R.; LIMA B. S. L. P.; EVSUKOFF, A. G. Deep learning for stock market prediction from financial news articles. In: IEEE INTERNATIONAL CONFERENCE ON COMPUTATIONAL INTELLIGENCE AND VIRTUAL ENVIRONMENTS FOR MEASUREMENT SYSTEMS AND APPLICATIONS (CIVEMSA 2017). *Proceedings*. 2017.
- [536] VARGAS, M. R.; DOS ANJOS, C. E. M.; BICHARA, G. L. G.; EVSUKOFF, A. G. Deep Learning for Stock Market Prediction Using Technical Indicators and Financial News Articles. In: INTERNATIONAL JOINT CONFERENCE ON NEURAL NETWORKS (IJCNN 2018). *Proceedings*. 2018.
- [537] VEDALDI, A.; LENC, K. MatConvNet - Convolutional Neural Networks for MATLAB, arXiv:1412.4564, 2014.

- [538] VENDRAMIN, L.; CAMPELLO, R. J. G. B.; HRUSCHKA, E. R. Relative clustering validity criteria: A comparative overview. *Statistical Analysis and Data Mining*, v. 3, n. 4, p. 209-235, 2010.
- [539] VICENTINI, P. C.; ESPINDOLA, R.; EVSUKOFF, A.; EBECKEN, N. Modeling the Effect of the Brazilian Gasoline Composition on the Vehicular Emissions – A Neural Network Approach. *SAE Technical Paper 2012-36-0500*. 2012. doi:10.4271/2012-36-0500.
- [540] VICENTINI, P. C. Influência do teor de enxofre da gasolina nas emissões de escapamento de veículos leves do ciclo Otto. In: XI SIMPÓSIO INTERNACIONAL DE ENGENHARIA AUTOMOTIVA. *Anais*. São Paulo, 2011.
- [541] VIEIRA, V. F.; XAVIER, C. R.; EBECKEN, N. F. F.; EVSUKOFF, A. Performance Evaluation of Modularity Based Community Detection Algorithms in Large Scale Networks. *Mathematical Problems in Engineering*, v. 2014, p. 1-15, 2014.
- [542] VINCENT, P.; LAROCHELLE, H.; BENGIO, Y.; MANZAGOL, P.-A. Extracting and Composing Robust Features with Denoising Autoencoders. In: 25TH INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML 2008). *Proceedings*. 2008. p. 1096-1103.
- [543] VINYALS, O.; TOSHEV, A.; BENGIO, S.; ERHAN, D. Show and tell: a neural image caption generator. arXiv 1411.4555. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2015), 2015, pp. 3156-3164
- [544] WAGNER, D.; WAGNER, S. *Comparing Clusterings – An Overview*. Universität Karlsruhe, Faculty of Informatics, 2007. (Technical Report 2006-04).
- [545] WALKER, G. On Periodicity in series of related terms. *Proceedings of the Royal Society of London. Series A*, v. 131, n. 818, p. 518-532, 1931.
- [546] WAND, M. P.; JONES, M. C. *Kernel Smoothing*. Chapman and Hall; CRC, 1995.
- [547] WANG, L. X.; MENDEL, J. M. Generating fuzzy rules by learning from examples. *IEEE Transactions on Systems, Man and Cybernetics*, v. 22, n. 6, p. 1.414-1.427, 1992.
- [548] WANG, L. X.; MENDEL, J. M. Fuzzy basis functions, universal approximation, and orthogonal least-squares learning. *IEEE Trans Neural Networks*, v. 3, n. 5, p. 807-814, 1992.

- [549] WANG, L. X. Fuzzy systems are universal approximators. In: IEEE INTERNATIONAL CONFERENCE ON FUZZY SYSTEMS. *Proceedings*. San Diego, 1992. p. 1.163-1.170.
- [550] WANG, T.-C.; LIU, M.-Y.; ZHU, J.-Y.; TAO, A.; KAUTZ, J.; CATANZARO B. High-Resolution Image Synthesis and Semantic Manipulation with Conditional GANs. arXiv:1711.11585. 2017.
- [551] WARD, J. H. Hierarchical Grouping to optimize an objective function. *Journal of American Statistical Association*, v. 58, n. 301, p. 236-244, 1963.
- [552] WASITO, I.; MIRKIN, B. Nearest neighbours in least-squares data imputation algorithms with different missing patterns. *Computational Statistics & Data Analysis*, v. 50, p. 926-949, 2006.
- [553] WEBB, A. *Statistical Pattern Recognition*. 2. ed. Wiley, 2002.
- [554] WEISS, Y. Segmentation using eigenvectors: a unifying view. In: 7TH IEEE INTERNATIONAL CONFERENCE ON COMPUTER VISION (ICCV'99). *Proceedings*. 1999. p. 975-982.
- [555] WEISS, S. M.; INDURKHYA, N.; ZHANG, T.; DAMERAU, F. *Text Mining: Predictive Methods for Analyzing Unstructured Information*. Springer, 2005.
- [556] WEISS K.; KHOSHGOFTAAR, T. M.; WANG D. A survey of transfer learning. *Journal of Big Data*, v. 3, n. 9, 2016.
- [557] WERBOS, P. J. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis. Harvard University, 1974.
- [558] WERBOS, P. J. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, v. 78, n. 10, p. 1.550-1.560, 1990.
- [559] WERBOS, P. J. *The Roots of Backpropagation: From Ordered Derivatives to Neural Networks and Political Forecasting*. Wiley-Interscience, 1994.
- [560] WIDROW, B. Thinking About Thinking: The Discovery of the LMS Algorithm. *IEEE Signal Processing Magazine*, v. 101, 2005.
- [561] WIDROW, B.; HOFF JR., M. E. Adaptive switching circuits. In: IRE WESCON Conf. Rec. *Proceedings*. part 4. 1960. p. 96-104.

- [562] WILLIAMS, R. J.; ZIPSER D. A Learning Algorithm for Continually Running Fully Recurrent Neural Networks. *Neural Computation*. v. 1 n. 2, p.270-280, 1989.
- [563] WILLIAMS, R. J.; ZIPSER D. Gradient-based learning algorithms for recurrent networks and their computational complexity. *Backpropagation*, Psychology Press, p. 433-486, 1995
- [564] WITTEN, I. H.; FRANK, E. *Data Mining: Practical Machine Learning Tools and Techniques*. 2. ed. Morgan Kaufmann, 2005.
- [565] WOLPERT, D. The lack of a priori distinctions between learning algorithms. *Neural Computation*, v. 8, p. 1.341-1.390, 1997.
- [566] WU et al. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. arXiv:1609.08144, 2016.
- [567] WU, K.-L.; YANG, M.-S. A cluster validity index for fuzzy clustering. *Pattern Recognition Letters*, v. 26, n. 9, p. 1.275-1.291, 2005.
- [568] WU, Y.; LIU, L.; PU, C.; CAO, W.; SAHIN, S.; WEI, W.; ZHANG, Q. A Comparative Measurement Study of Deep Learning as a Service Framework. arXiv:1810.12210, 2018.
- [569] WU, Z.; PAN, S.; CHEN, F.; LONG, G.; ZHANG, C.; YU, P. S. A Comprehensive Survey on Graph Neural Networks. arXiv:1901.00596, 2019.
- [570] XIE, X. L.; BENI, G. A validity measure for fuzzy clustering. *IEEE Trans. On Pattern Analysis Machine Intelligence*, v. 13, n. 8, p. 841-847, 1991.
- [571] XU, K.; BA, J.; KIROS, R.; CHO, K.; COURVILLE, A.; SALAKHUTDINOV, R.; ZEMEL, R.; BENGIO Y. Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. arXiv:1502.03044, 2015.
- [572] XU, R.; WUNSCH II, D. C. Survey of Clustering Algorithms. *IEEE Trans. on Neural Networks*, v. 16, n. 3, p. 645-678, 2005.
- [573] XU, R.; WUNSCH II, D. C. *Clustering*. IEEE Press Series on Computational Intelligence, 2008.
- [574] XU, K.; Ba, J. L.; KIROS, R.; CHO, K.; COURVILLE, A.; SALAKHUTDINOV, R.; ZEMEL, R. S.; BENGIO, Y. Show, attend and tell: Neural image caption generation with visual attention. arXiv:1502.03044, 2015,

- [575] YAGER, R. R. On ordered weighted averaging aggregation operators in multi-criteria decision making. *IEEE Trans. on Systems, Man and Cybernetics*, v. 18, n. 1, p. 183-190, 1988.
- [576] YAGER, R. R. Families of OWA operators. *Fuzzy Sets and Systems*, v. 59, p. 125-143, 1993.
- [577] YAM, Y.; BARANYI, P.; YANG, C.-T. Reduction of fuzzy rule base via singular value decomposition. *IEEE Trans. on Fuzzy Systems*, v. 7, n. 2, p. 120-132, 1999.
- [578] YEE, T. W.; WILD, C. J. Vector generalized additive models. *Journal of the Royal Statistical Society*, v. 58, n. 3, p. 481-493, 1996.
- [579] YEE, P. V.; HAYKIN, S. *Regularized Radial Basis Function Networks: Theory and Applications*. Wiley-Interscience, 2001.
- [580] YEN, J.; WANG, L. Simplifying fuzzy rule-based models using orthogonal transformation methods. *IEEE Trans. on Systems, Man, And Cybernetics—Part B: Cybernetics*, v. 29, n. 1, p. 13-24, 1999.
- [581] YOUNG, T.; HAZARIKA, D.; PORIA, S.; CAMBRIA, E. Recent Trends in Deep Learning Based Natural Language Processing. arXiv:1708.02709. *IEEE Computational Intelligence Magazine*, v. 13, n. 3, p. 55 - 75, 2018.
- [582] YU, P. S.; HAN, J.; FALOUTSOS, C. (Eds.). *Link Mining: Models, Algorithms, and Applications*. Springer, 2010.
- [583] YU, F.; KOLTUN, V. Multi-Scale Context Aggregation by Dilated Convolutions. arXiv:1511.07122, 2015
- [584] YULE, G. U. On a Method of investigating periodicities in disturbed series, with special reference to wolfer's sunspot numbers. *Philosophical Transactions of the Royal Society of London. Series A*, v. 226, p. 267-298, 1927.
- [585] ZADEH, L. A. Fuzzy sets. *Information and Control*, v. 8, n. 3, p. 338-353, 1965.
- [586] ZADEH, L. A. Outline of a new approach to the analysis of complex systems and decision processes. *IEEE Trans. Systems, Man and Cybernetics*, v. 3, n. 1, p. 28-44, 1973.
- [587] ZADEH, L. A. The concept of a linguistic variable and its application to approximate reasoning – I. *Information Sciences*, v. 8, n. 3, p. 199-249, 1975.

- [588] ZADEH, L. A. The concept of a linguistic variable and its application to approximate reasoning – II. *Information Sciences*, v. 8, n. 4, p. 301-357, 1975.
- [589] ZADEH, L. A. The concept of a linguistic variable and its application to approximate reasoning – III. *Information Sciences*, v. 9, n. 1, p. 43-80, 1975.
- [590] ZADEH, L. A. Fuzzy sets as a basis for a theory of possibility. *Fuzzy Sets and Systems*, v. 1, n. 1, p. 3-28, 1978.
- [591] ZADEH, L. A. Discussion: probability theory and fuzzy logic are complementary rather than competitive. *Technometrics*, v. 37, n. 3, p. 271-276, 1995.
- [592] ZALIK, K. R. Cluster validity index for estimation of fuzzy clusters of different sizes and densities. *Pattern Recognition*, v. 43, n. 10, p. 3.374-3.390, 2010.
- [593] ZAKI, M. J.; MEIRA JR., W. *Data Mining and Analysis*. Fundamental Concepts and Algorithms. Cambridge University Press, 2014.
- [594] ZANNI, L.; SERAFINI, T.; ZANGHIRATI, G. Parallel Software for Training Large Scale Support Vector Machines on Multiprocessor Systems. *Journal of Machine Learning Research*, v. 7, p. 1.467-1.492, 2006.
- [595] ZEGHIDOUR, N.; XU, Q.; LIPTCHINSKY, V.; USUNIER, N.; SYNNAEVE, G.; COLLOBERT, R. Fully Convolutional Speech Recognition. arXiv:1812.06864, 2018.
- [596] ZHA, H.; DING, C.; GU, M.; HE, X.; SIMON, H. Spectral relaxation for k-means clustering. In: 2001 NEURAL INFORMATION PROCESSING SYSTEMS (NIPS 2001). *Proceedings*. 2001. p. 1.057-1.064.
- [597] ZHANG, H.; XU, T.; LI, H.; ZHANG, S.; WANG, X.; HUANG, X.; METAXAS, D. StackGAN: Text to Photo-realistic Image Synthesis with Stacked Generative Adversarial Networks. arXiv:1612.03242. 2016.
- [598] ZHANG, Q.; ZHANG, M.; CHEN, T.; SUN, Z.; MA, Y.; YU, B. Recent Advances in Convolutional Neural Network Acceleration. arXiv:1807.08596, 2018.
- [599] ZHANG, Y.; WANG, W.; ZHANG, X.; LI, Y. A cluster validity index for fuzzy clustering. *Information Sciences*, v. 178, n. 4, p. 1.205-1.218, 2008.

- [600] ZHAO, Y.; KARYPIS, G. Empirical and theoretical comparisons of selected criterion functions for document clustering. *Machine Learning*, v. 55, p. 311-331, 2004.
- [601] ZHAO, W.; MA, H.; HE, Qing. Parallel k-means clustering based on MapReduce. *Proceedings of the 1st International Conference on Cloud Computing CloudCom'09*, LNCS, v. 5.931, p. 674-679, 2009.
- [602] ZHONG, S. Efficient online spherical k-means clustering. In: 2005 IEEE INTERNATIONAL JOINT CONFERENCE ON NEURAL NETWORKS (IJCNN' 05). *Proceedings*. 2005. p. 3.180-3.185.
- [603] ZHOU, S.-M.; GAN, J. Q. Low-level interpretability and high-level interpretability: a unified view of data-driven interpretable fuzzy system modeling. *Fuzzy Sets and Systems*, v. 159, p. 3.001-3.131, 2008.
- [604] ZHOU, Z.-H. *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall, 2012.
- [605] ZHU, X.; GOLDBERG, A. B. *Introduction to Semi-Supervised Learning*. Morgan and Claypool Publishers, 2009.

Referências na internet

- ¹ <http://plato.stanford.edu/entries/turing-test/>
- ² <https://cis.ieee.org/about/what-is-ci>
- ³ https://en.wikipedia.org/wiki/List_of_countries_by_number_of_Internet_users
- ⁴ https://en.wikipedia.org/wiki/Dot-com_bubble
- ⁵ <http://www.cs.waikato.ac.nz/ml/weka/>
- ⁶ <http://www.kdnuggets.com/software/index.html>
- ⁷ <http://www.dicionariodoaurelio.com/dicionario.php?P=Modelo>
- ⁸ <http://www.netflixprize.com/>
- ⁹ <https://www.kaggle.com/>
- ¹⁰ <http://www-stat.stanford.edu/~tibs/ElemStatLearn/>
- ¹¹ <http://www.crc.ricoh.com/~stork/DHS.html>
- ¹² <http://cgi.di.uoa.gr/~stpatrec/welcome3d.html>
- ¹³ <http://research.microsoft.com/en-us/um/people/cmbishop/prml/>
- ¹⁴ <http://www.control.isy.liu.se/books/sysid/>
- ¹⁵ <http://www.mathworks.com/products/sysid/>
- ¹⁶ <http://soma.mcmaster.ca/index.php>
- ¹⁷ <http://www.learning-with-kernels.org/>
- ¹⁸ <http://www.support-vector.net/nello.html>
- ¹⁹ <http://www.learning-from-data.com/>
- ²⁰ <http://www.support-vector.ws/>
- ²¹ <http://www-users.cs.umn.edu/~kumar/dmbook/index.php>
- ²² <http://www.cs.waikato.ac.nz/~ml/weka/book.html>
- ²³ <http://www.cs.cmu.edu/afs/cs.cmu.edu/user/mitchell/ftp/mlbook.html>
- ²⁴ <https://www.deeplearningbook.org/>
- ²⁵ <http://www.cs.uic.edu/~liub/WebMiningBook.html>
- ²⁶ <http://online.stanford.edu/>
- ²⁷ <http://ocw.mit.edu/index.htm>
- ²⁸ <https://www.coursera.org/>
- ²⁹ <http://www.udacity.com/>
- ³⁰ <https://www.coursera.org/course/ml>
- ³¹ <http://videolectures.net/>
- ³² <http://www.slideshare.net/>

33 <http://www.scribd.com/>
34 <http://www.kdnuggets.com/>
35 <http://archive.ics.uci.edu/ml/>
36 <http://kdd.ics.uci.edu/>
37 <https://www.kaggle.com/>
38 <http://sci2s.ugr.es/keel/>
39 <http://www.periodicos.capes.gov.br/portugues/index.jsp>
40 <http://www.sciencedirect.com/>
41 <http://ieeexplore.ieee.org/>
42 <http://www.springer.com/>
43 <http://portal.acm.org/>
44 <http://www.kdnuggets.com/publications/index.html>
45 <http://www.kdd.org/conferences.php>
46 <http://www.cs.uvm.edu/~icdm/>
47 <https://nips.cc/>
48 <https://icml.cc/>
49 <http://www.wcci2010.org/>
50 <http://ieee-cis.org/>
51 <http://abricom.org.br/>
52 <http://www.kdnuggets.com/websites/blogs.html>
53 <http://www.kdnuggets.com/websites/twitter-analytics-data-mining.html>
54 <http://www.kdnuggets.com/websites/social-networks.html>
55 <https://arxiv.org/>
56 <https://github.com/>
57 <http://www.sas.com/technologies/analytics/datamining/miner/index.html#section=1>
58 <http://www.spss.com/software/modeling/>
59 <http://www.statsoft.com/products/statistica-data-miner/>
60 <http://azure.microsoft.com/en-us/services/machine-learning/>
61 <http://aws.amazon.com/pt/machine-learning/details/>
62 <https://cloud.google.com/prediction/>
63 <http://www.megaputer.com/polyanalyst.php>
64 <http://www.megaputer.com/textanalyst.php>
65 <http://www.cs.waikato.ac.nz/ml/weka/index.html>
66 <http://www.pentaho.com/>
67 <https://rapidminer.com/>
68 <http://www.knime.org/>
69 <https://orange.biolab.si/>
70 <http://pandas.pydata.org/>
71 <http://scikit-learn.org/stable/>
72 <https://towardsdatascience.com/top-10-python-libraries-for-data-science-cd82294ec266>
73 [http://en.wikipedia.org/wiki/R_\(programming_language\)](http://en.wikipedia.org/wiki/R_(programming_language))
74 <http://www.revolutionanalytics.com/products/>
75 <http://www.mathworks.com/products/>
76 <http://www.mathworks.com/products/statistics/>
77 <http://www.mathworks.com/products/neuralnet/>
78 <http://www.mathworks.com/products/global-optimization/>
79 <http://www.mathworks.com/products/optimization/>
80 <http://www.mathworks.com/products/sysid/>
81 <http://www.mathworks.com/products/parallel-computing/>
82 <http://www.mathworks.com/products/netbuilder/>
83 <http://www.mathworks.com/products/javabuilder/>

84 <http://www.mathworks.com/products/matlabxl/>
 85 <http://www.mathworks.com/matlabcentral/>
 86 <http://www.mathworks.com/support/books/>
 87 <http://epicgraphic.com/data-cake/>
 88 http://en.wikipedia.org/wiki/Normal_distribution
 89 https://en.wikipedia.org/wiki/List_of_probability_distributions
 90 <http://archive.ics.uci.edu/ml/datasets/Ionosphere>
 91 <http://lib.stat.cmu.edu/datasets/pollution>
 92 <http://archive.ics.uci.edu/ml/datasets/Water+Treatment+Plant>
 93 <http://www.netlib.org/eispack/>
 94 <http://www.caam.rice.edu/software/ARPACK/>
 95 <https://archive.ics.uci.edu/ml/datasets/Parkinsons>
 96 <http://archive.ics.uci.edu/ml/datasets/Wine>
 97 <https://dataverse.harvard.edu/dataverse/MRRJ>
 98 <http://data.rio/>
 99 <http://www.qgis.org/en/site/>
 100 http://www.furnas.com.br/hotsites/sistemapfurnas/usina_hidr_funil.asp
 101 <http://www.cse.illinois.edu/heath/scicomp/pubdata/index.html>
 102 <http://www.kdnuggets.com/software/visualization.html>
 103 http://homepage.tudelft.nl/19j49/Matlab_Toolbox_for_Dimensionality_Reduction.html
 104 <http://archive.ics.uci.edu/ml/datasets/Concrete+Slump+Test>
 105 <http://lib.stat.cmu.edu/datasets/pollution>
 106 <http://www.mathworks.com/products/sysid/>
 107 <http://www.stat.wisc.edu/~reinsel/bjr-data/>
 108 http://en.wikipedia.org/wiki/Binomial_expansion
 109 http://en.wikipedia.org/wiki/Pascal's_triangle
 110 <http://www.mathworks.com/products/sysid/>
 111 <http://robjhyndman.com/TSDL/>
 112 <http://en.wikipedia.org/wiki/Sunspots>
 113 <http://www.brasil.gov.br/sobre/economia/energia/setor-eletrico/sistema-interligado-nacional>
 114 <http://previsaonumerica.cptec.inpe.br/>
 115 <http://plato.stanford.edu/entries/bayes-theorem/>
 116 https://en.wikipedia.org/wiki/Abductive_reasoning
 117 https://scikit-learn.org/stable/modules/naive_bayes.html
 118 <http://archive.ics.uci.edu/ml/datasets/Pima+Indians+Diabetes>
 119 <http://r-statistics.co/Logistic-Regression-With-R.html>
 120 https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
 121 https://en.wikipedia.org/wiki/Bernoulli_distribution
 122 <http://archive.ics.uci.edu/ml/>
 123 <http://archive.ics.uci.edu/ml/datasets/Iris>
 124 <http://archive.ics.uci.edu/ml/datasets/Balance+Scale>
 125 <http://archive.ics.uci.edu/ml/datasets/Pima+Indians+Diabetes>
 126 <http://archive.ics.uci.edu/ml/datasets/Breast+Cancer>
 127 <http://archive.ics.uci.edu/ml/datasets/Statlog+%28Heart%29>
 128 <http://archive.ics.uci.edu/ml/datasets/Wine>
 129 <http://archive.ics.uci.edu/ml/datasets/Parkinsons>
 130 <http://archive.ics.uci.edu/ml/datasets/Ionosphere>
 131 <http://archive.ics.uci.edu/ml/datasets/Spambase>
 132 [http://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+\(Sonar,+Mines+vs.+Rocks\)](http://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks))
 133 <http://www.keel.es/>
 134 http://en.wikipedia.org/wiki/Churn_rate

135 <https://www.coursera.org/course/pgm>
136 <http://www.mathworks.com/help/toolbox/stats/kmeans.html>
137 <http://www.cs.utexas.edu/users/dml/datamining/spkmeans.html>
138 http://en.wikipedia.org/wiki/Voronoi_diagram
139 <http://nlp.stanford.edu/IR-book/html/htmledition/variant-tf-idf-functions-1.html>
140 <http://www.coc.ufrj.br/pt/>
141 <http://www.cytoscape.org/>
142 <http://yann.lecun.com/exdb/mnist/>
143 http://en.wikipedia.org/wiki/Hilbert_space
144 <https://www.nature.com/articles/323533a0/metrics>
145 <http://yann.lecun.com/exdb/mnist/>
146 <https://arxiv.org/>
147 <https://github.com/>
148 <http://www.ni.com/example/31220/en/>
149 https://en.wikipedia.org/wiki/Loss_functions_for_classification
150 <https://github.com/>
151 <https://playground.tensorflow.org/>
152 <https://amturing.acm.org/>
153 <http://www.image-net.org/>
154 <http://deeplearning.net/software/theano/>
155 <https://www.tensorflow.org>
156 <https://pytorch.org>
157 <https://www.microsoft.com/en-us/cognitive-toolkit/>
158 <https://mxnet.apache.org/>
159 <http://caffe.berkeleyvision.org/>
160 <https://caffe2.ai/>
161 <https://keras.io/>
162 <https://mathworks.com>
163 https://support.sas.com/documentation/prod-p/vdmml/index_deep_learn_guide.html
164 <https://onnx.ai/get-started.html>
165 <https://www.nvidia.com/en-us/data-center/tensorcore/>
166 <https://cloud.google.com/tpu/docs/>
167 http://deeplearning.net/software/theano/tutorial/conv_arithmetic.html
168 <https://github.com/HIPS/autograd/blob/master/docs/tutorial.md>
169 https://www.tensorflow.org/tutorials/eager/automatic_differentiation
170 https://en.wikipedia.org/wiki/List_of_datasets_for_machine_learning_research
171 <https://www.tensorflow.org/tutorials/>
172 https://cntk.ai/pythondocs/CNTK_103D_MNIST_ConvolutionalNeuralNetwork.html
173 <https://www.kaggle.com/c/mnist-tutorial-machine-learning-challenge>
174 <http://yann.lecun.com/exdb/mnist/>
175 <http://groups.csail.mit.edu/vision/TinyImages/>
176 <https://wordnet.princeton.edu/>
177 <http://www.cs.toronto.edu/~kriz/cifar.html>
178 <https://www.mturk.com/>
179 <https://developer.nvidia.com/cudnn>
180 <https://www.cs.toronto.edu/~kriz/index.html>
181 <https://www.cs.toronto.edu/~kriz/index.html>
182 <https://gist.github.com/ksimonyan/3785162f95cd2d5fee77>
183 <https://github.com/kentsommer/keras-inceptionV4/>
184 <http://cocodataset.org/>
185 <https://github.com/KaimingHe/deep-residual-networks>

186 <https://github.com/facebook/fb.resnet.torch>
 187 <https://github.com/tensorflow/models/tree/master/official/resnet>
 188 <https://paperswithcode.com/>
 189 <http://host.robots.ox.ac.uk/pascal/VOC/>
 190 <http://cocodataset.org/#home>
 191 <https://research.fb.com/facebook-open-sources-densepose/>
 192 <https://translate.google.com/>
 193 <https://dicionariodoaurelio.com/>
 194 <http://comet.ffi.ch.usp.br/corporaportugues>
 195 <https://patents.google.com/patent/US9037464B1/en>
 196 <https://code.google.com/archive/p/word2vec/>
 197 <https://www.tensorflow.org/tutorials/representation/word2vec>
 198 <https://github.com/search?q=word2vec>
 199 <https://nlp.stanford.edu/projects/glove/>
 200 <https://fasttext.cc/>
 201 <http://nilc.icmc.usp.br/embeddings>
 202 <https://www.clips.uantwerpen.be/conll2003/>
 203 <https://nlp.stanford.edu/sentiment/>
 204 <https://www.daemo.org/home>
 205 <https://rajpurkar.github.io/SQuAD-explorer/>
 206 <https://nvlpubs.nist.gov/nistpubs/Legacy/IR/nistir4930.pdf>
 207 <https://github.com/mozilla/DeepSpeech>
 208 <https://research.fb.com/downloads/wav2letter/>
 209 <https://github.com/facebookresearch/wav2letter>
 210 <https://github.com/topics/speech-recognition>
 211 <https://www.nuance.com/mobile/speech-recognition-solutions.html>
 212 <https://cloud.google.com/speech-to-text/>
 213 <https://azure.microsoft.com/pt-br/services/cognitive-services/speech-services/>
 214 <https://www.ibm.com/watson/services/speech-to-text/>
 215 <http://www.statmt.org/wmt14/translation-task.html>
 216 <https://ai.google/research/pubs/pub45610>
 217 <https://github.com/facebookresearch/fairseq>
 218 <https://github.com/tensorflow/nmt>
 219 <https://nlp.stanford.edu/projects/nmt/>
 220 <https://azure.microsoft.com/pt-br/services/cognitive-services/translator-text-api/>
 221 <https://www.ibm.com/watson/services/language-translator/>
 222 <http://mi.eng.cam.ac.uk/projects/segnet/>
 223 <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>
 224 <https://github.com/wuyx/the-gan-zoo>
 225 <http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>
 226 <https://www.epri.com/>
 227 <https://code.google.com/archive/p/word2vec/>
 228 <http://nilc.icmc.usp.br/embeddings>